

Android Framework 开发揭秘

前言

本书通过对Android系统源代码的分析，主要介绍Android框架的初始化过程及主要组件的工作原理。作者直接分析和整理了Android框架的主要源代码，并详细讲解了理解框架工作原理所需的各种基础知识和构成实际Android平台骨干的服务框架。其中的主要内容包括：通过启动程序了解Android框架的概要，移植Android以及如何开发适合各种机器的应用程序，分析Android框架所需的基础知识，JNI (Java Native Interface)与Binder基础知识，Zygote. Service Manager. Service Server等Android核心组件，Android服务框架的结构与理解，通过经典Binder、Handler、AMS等面试题解析加深你对Android Framework框架层的理解，为你的大厂面试保驾护航。

本书不仅可以供具备一定开发经验的Android开发人员参考阅读，也可作为Android开发初学者的Android框架入门教材使用。

前言.....	1
第一章 系统启动流程分析.....	4
第一节 Android启动概览.....	4
1.1 Linux 内核.....	5
1.2 init进程分析.....	8
第二节 init.rc解析.....	15
2.1 Action.....	17
2.2 Service.....	17
2.3 Command.....	17
2.4 Options.....	19
2.5 service解析流程.....	19
第三节 Zygote.....	23
3.1 Zygote概叙.....	24
3.2 Zygote触发过程.....	25
3.3 Zygote启动过程.....	27
3.4 System Server 启动流程.....	51
3.5 什么情况下Zygote进程会重启呢？.....	54
3.6 fork函数.....	54
3.7 子进程与父进程的区别.....	55
3.8 写时拷贝 (copy-on-write).....	55
3.9 孤儿进程、僵尸进程.....	55
3.10 多线程进程的Fork调用.....	55
第四节 面试题.....	56
4.1 你了解 Android 系统启动流程吗？.....	56
4.2 system_server 为什么要在 Zygote 中启动，而不是由 init 直接启动呢？.....	56
4.3 为什么要专门使用 Zygote 进程去孵化应用进程，而不是让 system_server 去孵化呢？.....	56
4.4 能说说具体是怎么导致死锁的吗？.....	56
4.5 Zygote 为什么不采用 Binder 机制进行 IPC 通信？.....	56

第二章 Binder解析.....	57
第一节 宏观认识Binder.....	57
1.1 binder框架.....	57
1.2 binder涉及到的类.....	58
1.3 binder驱动.....	58
第二节binder的jni方法注册.....	58
2.1 zygote启动.....	58
2.2 register_androids_Binder.....	60
第三节binder驱动.....	62
3.1 binder_init.....	62
3.2 binder_open.....	63
3.3 binder_mmap.....	63
3.4 binder_ioctl.....	65
第四节 数据结构.....	65
4.1 file_operations.....	65
4.2 binder_proc.....	66
4.3 binder_node.....	66
第五节 启动service_manager.....	67
5.1启动servicemanager进程.....	67
5.2 main.....	68
第六节获取service_manager.....	71
6.1 defaultServiceManager.....	71
第七节 addService流程.....	74
7.1 SytemServer.....	75
7.2 getIServiceManager.....	75
7.3 SMP.addService--1.....	77
7.3 SMP.addService--2.....	78
第八节 Binder面试题全解析.....	86
8.1 Binder有什么优势.....	86
8.2 Binder是如何做到一次拷贝的.....	86
8.3 MMAP的内存映射原理了解吗.....	87
8.4 Binder机制是如何跨进程的.....	88
8.5 说说四大组件的通信机制.....	88
8.6 为什么Intent不能传递大数据.....	89
第三章 Handler解析.....	90
第一节 源码分析.....	90
1.1 Handler机制的相关类.....	90
1.2 创建Looper.....	90
1.3 Looper.loop().....	92
1.4 创建Handler.....	93
1.5 创建Message.....	93
1.6 Message和Handler的绑定.....	94
1.7 Handler发送消息.....	94
1.8 Handler处理消息.....	94
1.9小结.....	94
第二节 难点问题.....	95
2.1线程同步问题.....	95
2.2消息机制之同步屏障.....	97
2.3小结.....	101
第三节Handler常问面试题.....	101
3.1 HandlerThread是什么？为什么它会存在？.....	101
3.2简述下 Handler 机制的总体原理？.....	103
3.3 Looper 存在哪？如何可以保证线程独有？.....	103
3.4如何理解 ThreadLocal 的作用？.....	103

3.5主线程 Main Looper 和一般 Looper 的异同?	103
3.6 Handler 或者说 Looper 如何切换线程?	104
3.7 Looper 的 loop() 死循环为什么不卡死?	104
3.8 Looper 的等待是如何能够准确唤醒的?	104
3.9 Message 如何获取? 为什么这么设计?	104
3.10 MessageQueue 如何管理 Message?	104
3.11理解 Message 和 MessageQueue 的异同?	105
3.12 Message 的执行时刻如何管理?	105
3.13 Handler、Message 和 Runnable 的关系如何理解?	105
3.14 IdleHandler 空闲 Message 了解过吗? 有什么用?	105
3.15 异步 Message 或同步屏障了解过吗? 怎么用? 什么原理?	105
3.16 Looper 和 MessageQueue、Message 及 Handler 的关系?	106
3.17 Native 侧的 NativeMessageQueue 和 Looper 的作用是?	106
3.18 Native 侧如何使用 Looper?	106
3.19 Handler 为什么可能导致内存泄露? 如何避免?	107
3.20 Handler 在系统当中的应用	107
3.21 Android 为什么不允许并发访问 UI?	107
第四章 AMS 解析	107
第一节 引言	107
第二节 Android架构	108
第三节 通信方式	110
第四节 系统启动系列	112
第五节 AMS	113
第六节 AMS 面试题解析	121
6.1 ActivityManagerService是什么? 什么时候初始化的? 有什么作用?	121
6.2 ActivityThread是什么?ApplicationThread是什么?他们的区别	121
6.3 Instrumentation是什么? 和ActivityThread是什么关系?	121
6.4 ActivityManagerService和zygote进程通信是如何实现的。	121
6.5 ActivityRecord、TaskRecord、ActivityStack, ActivityStackSupervisor, ProcessRecord	122
6.6 ActivityManager、ActivityManagerService、ActivityManagerNative、ActivityManagerProxy的关系	126
第五章 WMS解析	128
第一节Activity与Window相关概念	128
1.1 Window	128
1.2WindowManager	129
1.3WindowManagerService	129
1.4Token	129
1.5Window的类型	129
第二节 Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析	132
第三节Android窗口管理服务WindowManagerService对窗口的组织方式分析	154
第四节 Android窗口管理服务WindowManagerService对输入法窗口 (Input Method Window) 的管理分析	176
第五节 Android窗口管理服务WindowManagerService对壁纸窗口 (Wallpaper Window) 的管理分析	191
第六节Android窗口管理服务WindowManagerService计算窗口Z轴位置的过程分析	217
第七节Android窗口管理服务WindowManagerService显示Activity组件的启动窗口 (Starting Window) 的过程分析	233
第八节Android窗口管理服务WindowManagerService切换Activity窗口 (App Transition) 的过程分析	256
第九节 Android窗口管理服务WindowManagerService显示窗口动画的原理分析	277
第六章PKMS Android10.0 源码解读	295
第一节 前言 PKMS 是什么东西?	295
第二节 PKMS 概述信息	295

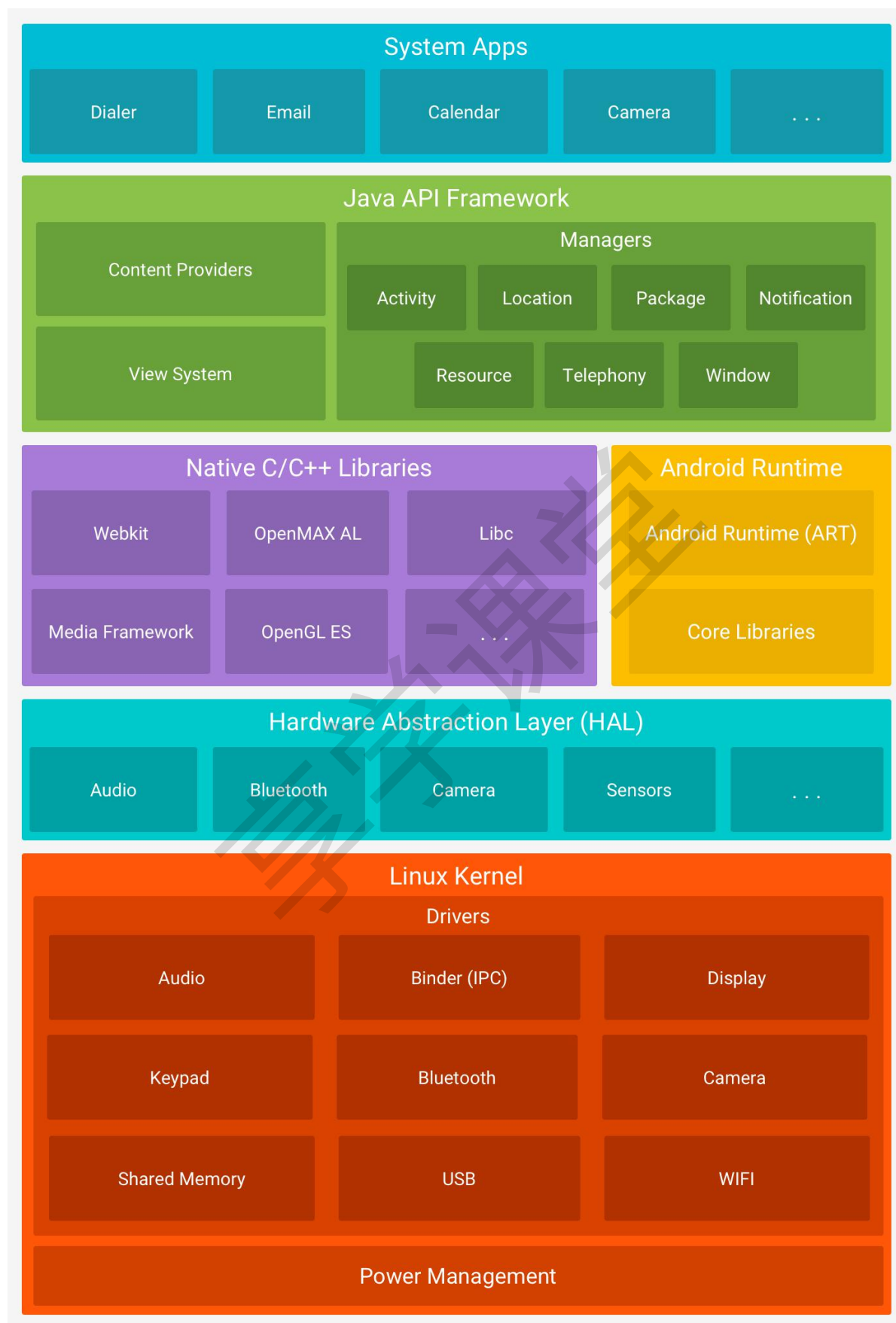
第三节	PKMS角色位置.....	297
第四节	PKMS 启动过程分析.....	298
第五节	APK的扫描.....	323
第六节	APK的安装.....	335
第七节	PMS之权限扫描.....	358
第八节	PackageManagerService大综合笔记.....	365

第一章 系统启动流程分析

第一节 Android启动概览

安卓课堂

众所周知，Android是谷歌开发的一款基于Linux的开源操作系统，下图所示为 Android 平台的主要组件



1.1 Linux 内核

Android 平台的基础是 Linux 内核。例如，Android Runtime (ART) 依靠 Linux 内核来执行底层功

能，例如线程和低层内存管理。

使用 Linux 内核可让 Android 利用主要安全功能，并且允许设备制造商为著名的内核开发硬件驱动程序。

1. 硬件抽象层 (HAL)

硬件抽象层 (HAL) 提供标准界面，向更高级别的 Java API 框架显示设备硬件功能。HAL 包含多个库模块，其中每个模块都为特定类型的硬件组件实现一个界面，例如相机或蓝牙模块。当框架 API 要求访问设备硬件时，Android 系统将为该硬件组件加载库模块。

2. Android Runtime

对于运行 Android 5.0 (API 级别 21) 或更高版本的设备，每个应用都在其自己的进程中运行，并且有其自己的 Android Runtime (ART) 实例。ART 编写为通过执行 DEX 文件在低内存设备上运行多个虚拟机，DEX 文件是一种专为 Android 设计的字节码格式，经过优化，使用的内存很少。编译工具链 (例如 Jack) 将 Java 源代码编译为 DEX 字节码，使其可在 Android 平台上运行。

ART 的部分主要功能包括：

- 预先 (AOT) 和即时 (JIT) 编译优化的垃圾回收 (GC)
 - 在 Android 9 (API 级别 28) 及更高版本的系统中，支持将应用软件包中的 Dalvik Executable 格式 (DEX) 文件转换为更紧凑的机器代码。
 - 更好的调试支持，包括专用采样分析器、详细的诊断异常和崩溃报告，并且能够设置观察点以监控特定字段
- 在 Android 版本 5.0 (API 级别 21) 之前，Dalvik 是 Android Runtime。如果您的应用在 ART 上运行效果很好，那么它应该也可在 Dalvik 上运行，但反过来不一定。

Android 还包含一套核心运行时库，可提供 Java API 框架所使用的 Java 编程语言中的大部分功能，包括一些 Java 8 语言功能。

3. 原生 C/C++ 库

许多核心 Android 系统组件和服务 (例如 ART 和 HAL) 构建自原生代码，需要以 C 和 C++ 编写的原生库。Android 平台提供 Java 框架 API 以向应用显示其中部分原生库的功能。例如，您可以通过 Android 框架的 Java OpenGL API 访问 OpenGL ES，以支持在应用中绘制和操作 2D 和 3D 图形。

如果开发的是需要 C 或 C++ 代码的应用，可以使用 Android NDK 直接从原生代码访问某些原生平台库。

4. Java API 框架

您可通过以 Java 语言编写的 API 使用 Android OS 的整个功能集。这些 API 形成创建 Android 应用所需的构建块，它们可简化核心模块化系统组件和服务的重复使用，包括以下组件和服务：

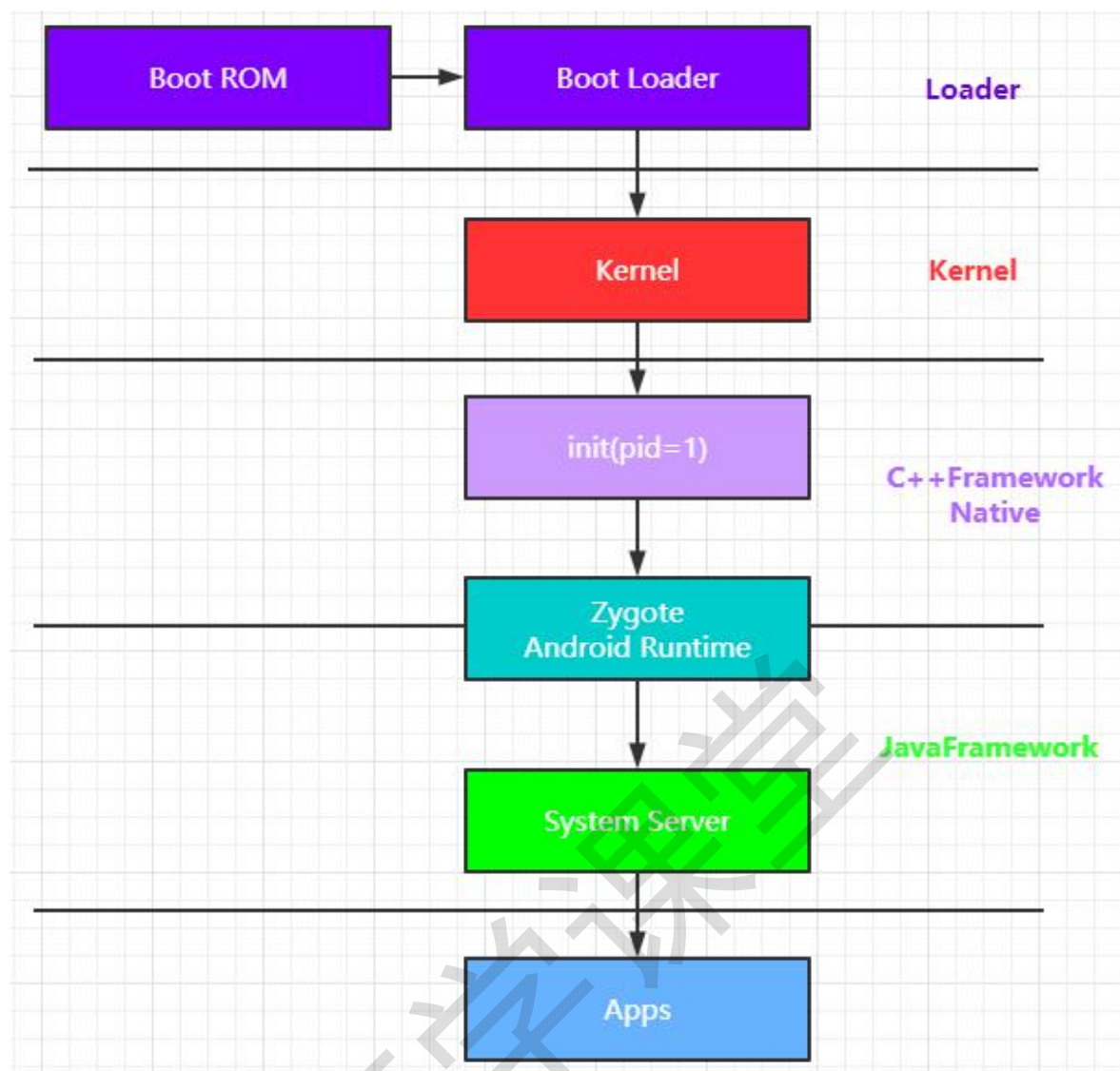
- 丰富、可扩展的视图系统，可用以构建应用的 UI，包括列表、网格、文本框、按钮甚至可嵌入的网络浏览器
- 资源管理器，用于访问非代码资源，例如本地化的字符串、图形和布局文件通知管理器，可让所有应用在状态栏中显示自定义提醒
- Activity 管理器，用于管理应用的生命周期，提供常见的导航返回栈
- 内容提供程序，可让应用访问其他应用 (例如“联系人”应用) 中的数据或者共享其自己的数据 开发者可以完全访问 Android 系统应用使用的框架 API。

5. 系统应用

Android 随附一套用于电子邮件、短信、日历、互联网浏览和联系人等的核心应用。平台随附的应用与用户可以选择安装的应用一样，没有特殊状态。因此第三方应用可成为用户的默认网络浏览器、短信 Messenger 甚至默认键盘 (有一些例外，例如系统的“设置”应用)。

系统应用可作用户的应用，以及提供开发者可从其自己的应用访问的主要功能。例如，如果您的应用要发短信，您无需自己构建该功能，可以改为调用已安装的短信应用向您指定的接收者发送消息。

接下来，我们来看下Android系统启动的大概流程，如下图所示：



- ◆ 第一步：启动电源以及系统启动

当电源按下，引导芯片代码开始从预定义的地方（固化在ROM）开始执行。加载引导程序到RAM，然后执行

- ◆ 第二步：引导程序

引导程序是在Android操作系统开始运行前的一个小程序。引导程序是运行的第一个程序，因此它是针对特定的主板与芯片的。设备制造商要么使用很受欢迎的引导程序比如redboot、uboot、qi bootloader或者开发自己的引导程序，它不是Android操作系统的一部分。引导程序是OEM厂商或者运营商加锁和限制的地方。

引导程序分两个阶段执行。

第一个阶段，检测外部的RAM以及加载对第二阶段有用的程序；

第二阶段，引导程序设置网络、内存等等。这些对于运行内核是必要的，为了达到特殊的目标，引导程序可以根据配置参数或者输入数据设置内核。

Android引导程序可以在\bootable\bootloader\legacy\usbloader找到。传统的加载器包含两个文件，需要在这里说明：

init.s初始化堆栈，清零BBS段，调用main.c的_main()函数；

main.c初始化硬件（闹钟、主板、键盘、控制台），创建linux标签

- ◆ 第三步：内核

Android内核与桌面linux内核启动的方式差不多。内核启动时，设置缓存、被保护存储器、计划列表，加载驱动。当内核完成系统设置，它首先在系统文件中寻找”init”文件，然后启动root进程或者系统的第一个进程

- 第四步：init进程

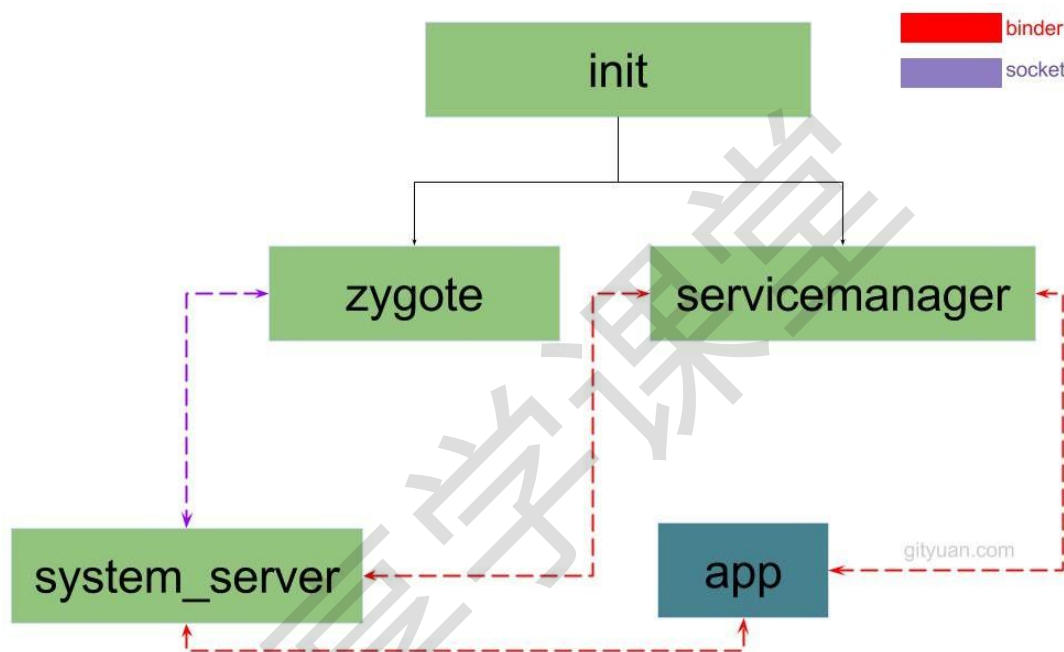
init进程是Linux系统中用户空间的第一个进程，进程号固定为1。Kernel启动后，在用户空间启动init进程，并调用init中的main()方法执行init进程的职责。

- 第五步：启动Lancher App

1.2 init进程分析

其中init进程是Android系统中及其重要的第一个进程，接下来我们来看下init进程注意做了些什么

1. 创建和挂载启动所需要的文件目录
2. 初始化和启动属性服务
3. 解析init.rc配置文件并启动Zygote进程



```
1 // \system\core\init\init.cpp main() L545
2 /*
3  * 1.C++中主函数有两个参数，第一个参数argc表示参数个数，第二个参数是参数列表，也就是具体的参数
4  * 2.init的main函数有两个其它入口，一是参数中有ueventd，进入ueventd_main，二是参数中有watchdogd，进入watchdogd_main
5  */
6 int main(int argc, char** argv)
7 {
8     /*
9     * 1.strcmp是String的一个函数，比较字符串，相等返回0
10    * 2.C++中0也可以表示false
11    * 3.basename是C库中的一个函数，得到特定的路径中的最后一个 '/' 后面的内容，
12    * 比如/sdcard/miui_recovery/backup，得到的结果是backup
13    */
14    if (!strcmp(basename(argv[0]), "ueventd")) { //当argv[0]的内容为ueventd
        //1表示true，也就执行ueventd_main,ueventd主要是负责设备节点的创建、权限设定等一些列工作
    }
```

```

15         return ueventd_main(argc, argv);
16     }
17
18     if (!strcmp(basename(argv[0]), "watchdogd")) { //watchdogd俗称看门狗，用于系统出问题
时重启系统
19         return watchdogd_main(argc, argv); 20     }
21
22     if (argc > 1 && !strcmp(argv[1], "subcontext")) {
23         InitKernelLogging(argv);
24         const BuiltinFunctionMap function_map;
25         return SubcontextMain(argc, argv, &function_map); 26     }
27
28     if (REBOOT_BOOTLOADER_ON_PANIC) {
29         InstallRebootSignalHandlers(); //初始化重启系统的处理信号，内部通过
sigaction 注册信号，当监听到该信号时重启系统
30     }
31
32     bool is_first_stage = (getenv("INIT_SECOND_STAGE") == nullptr); //查看是否有环境变量
INIT_SECOND_STAGE
33     /*
34      * 1. init的main方法会执行两次，由is_first_stage控制，first_stage就是第一阶段要做的事
35      */
36     if (is_first_stage) {
37         boot_clock::time_point start_time = boot_clock::now(); 38
39         // Clear the umask.
40         umask(0); //清空文件权限41
41
42         clearenv();
43         setenv("PATH", _PATH_DEFPATH, 1);
44         // Get the basic filesystem setup we need put together in the initramdisk
45         // on / and then we'll let the rc file figure out therest.
46         //mount是用来挂载文件系统的，mount属于Linux系统调用
47         mount("tmpfs", "/dev", "tmpfs", MS_NOSUID, "mode=0755");
48         mkdir("/dev/pts", 0755); //创建目录，第一个参数是目录路径，第二个是读写权限
49         mkdir("/dev/socket", 0755);
50         mount("devpts", "/dev/pts", "devpts", 0, NULL);
51         #define MAKE_STR(x) STRING(x)
52         mount("proc", "/proc", "proc", 0, "hidepid=2,gid="
MAKE_STR(AID_READPROC));
53         // Don't expose the raw commandline to unprivileged processes.
54         chmod("/proc/cmdline", 0440); //用于修改文件/目录的读写权限
55         gid_t groups[] = { AID_READPROC };
56         setgroups(arraysize(groups), groups); // 用来将list 数组中所标明的组加入到目前进
程的组设置中
57         mount("sysfs", "/sys", "sysfs", 0, NULL);
58         mount("selinuxfs", "/sys/fs/selinux", "selinuxfs", 0, NULL);
59         //mknod用于创建Linux中的设备文件
60         mknod("/dev/kmsg", S_IFCHR | 0600, makedev(1, 11)); 61
62         if constexpr (WORLD_WRITABLE_KMSG) {
63             mknod("/dev/kmsg_debug", S_IFCHR | 0622, makedev(1, 11)); 64     }
65

```

```

66     mknod("/dev/random", S_IFCHR | 0666, makedev(1, 8));
67     mknod("/dev/urandom", S_IFCHR | 0666, makedev(1, 9)); 68
69     // Mount staging areas for devices managed by vold
70     // See storage config details at
    http://source.android.com/devices/storage/
71     mount("tmpfs", "/mnt", "tmpfs", MS_NOEXEC | MS_NOSUID | MS_NODEV, 72
        "mode=0755,uid=0,gid=1000");
73     // /mnt/vendor is used to mount vendor-specific partitions that can not be
74     // part of the vendor partition, e.g. because they are mounted read-write.
75     mkdir("/mnt/vendor", 0755); 76
77     // Now that tmpfs is mounted on /dev and we have /dev/kmsg, we can actually
78     // talk to the outside world...
79     InitKernelLogging(argv); //将标准输入输出重定向到"/sys/fs/selinux/null" 80
81     LOG(INFO) << "init first stage started!"; 82
83     if (!DoFirstStageMount()) {
84         LOG(FATAL) << "Failed to mount required partitions early ..."; 85     }
86
87     //Avb即Android Verified boot, 功能包括Secure Boot, verifying boot 和 dm-verity,
88     //原理都是对二进制文件进行签名, 在系统启动时进行认证, 确保系统运行的是合法的二进
    制镜像文件。
89     //其中认证的范围涵盖: bootloader, boot.img, system.img
90     SetInitAvbVersionInRecovery(); //在刷机模式下初始化avb的版本, 不是刷机模式 直接跳过
91
92     // Enable seccomp if global boot option was passed (otherwise it is enabled in
    zygote).
93     global_seccomp(); 94
95     // Set up SELinux, loading the SELinux policy.
96     SelinuxSetupKernelLogging();
97     SelinuxInitialize(); //加载SELinux policy, 也就是安全策略, 98
99     // We're in the kernel domain, so re-exec init to transition to the init domain
    now
100    // that the SELinux policy has been loaded. 101
101    /*
102    * 1. 这句英文大概意思是, 我们执行第一遍时是在kernel domain, 所以要重新执行
    init文件, 切换到init domain,
103    * 这样SELinux policy才已经加载进来了
104    * 2. 后面的security_failure函数会调用panic重启系统105
105    */
106    if (selinux_android_restorecon("/init", 0) == -1) {
107        PLOG(FATAL) << "restorecon failed of /init failed";
108    }
109
110    setenv("INIT_SECOND_STAGE", "true", 1); 111
112    static constexpr uint32_t kNanosecondsPerMillisecond = 1e6;

```

```

113         uint64_t start_ms = start_time.time_since_epoch().count() /
kNanosecondsPerMillisecond;

114         setenv("INIT_STARTED_AT", std::to_string(start_ms).c_str(), 1); 115
116         char* path = argv[0];
117         char* args[] = { path, nullptr };
118         execv(path, args); //重新执行main方法，进入第二阶段119
120         // execv() only returns if an error happened, in which casewe
121         // panic and never fall through this conditional.
122         PLOG(FATAL) << "execv(\"" << path << "\") failed";
123     }
124
125     // At this point we're in the second stage of init.
126     InitKernelLogging(argv);
127     LOG(INFO) << "init second stage started!"; 128
129     // Set up a session keyring that all processes will have access to. It
130     // will hold things like FBE encryption keys. No process should override
131     // its session keyring.
132     keyctl_get_keyring_ID(KEY_SPEC_SESSION_KEYRING, 1); 133
134     // Indicate that booting is in progress to background fw loaders, etc.
135     close(open("/dev/.booting", O_WRONLY | O_CREAT | O_CLOEXEC, 0000)); 136
137     property_init(); //初始化属性系统，并从指定文件读取属性
138
139     // If arguments are passed both on the command line and inDT,
140     // properties set in DT always have priority over the command-line ones.
141     //接下来的一系列操作都是从各个文件读取一些属性，然后通过property_set设置系统属性
142
143     // If arguments are passed both on the command line and inDT,
144     // properties set in DT always have priority over the command-line ones.
145     /*
146     * 1. 这句英文的大概意思是，如果参数同时从命令行和DT传过来，DT的优先级总是大于命令行的
147     * 2. DT即device-tree，中文意思是设备树，这里面记录自己的硬件配置和系统运行参数，参考
http://www.wowotech.net/linux\_kernel/why-dt.html
148     */
149     process_kernel_dt(); //处理DT属性
150     process_kernel_cmdline(); //处理命令行属性151
152     // Propagate the kernel variables to internal variables
153     // used by init as well as the current required properties.
154     export_kernel_boot_props(); //处理其他的一些属性155
156     // Make the time that init started available for bootstat tolog.
157     property_set("ro.boottime.init", getenv("INIT_STARTED_AT"));
158     property_set("ro.boottime.init.selinux", getenv("INIT_SELINUX_TOOK")); 159
160     // Set libavb version for Framework-only OTA match in Treble build.
161     const char* avb_version = getenv("INIT_AVB_VERSION");
162     if (avb_version) property_set("ro.boot.avb_version", avb_version); 163
164     // Clean up our environment.

```



```

165     unsetenv("INIT_SECOND_STAGE");//清空这些环境变量，因为之前都已经存入到系统属性 中去了
166     unsetenv("INIT_STARTED_AT");
167     unsetenv("INIT_SELINUX_TOOK");
168     unsetenv("INIT_AVB_VERSION"); 169
170     // Now set up SELinux for second stage.
171     SelinuxSetupKernelLogging();
172     SelabelInitialize();
173     SelinuxRestoreContext(); 174
175     epoll_fd = epoll_create1(EPOOL_CLOEXEC);//创建epoll实例，并返回epoll的文件
描述符
176     if (epoll_fd == -1) {
177         PLOG(FATAL) << "epoll_create1 failed";
178     }
179
180     sigchld_handler_init();//主要是创建handler处理子进程终止信号，创建一个匿名socket并
注册到epoll进行监听
181
182     if (!IsRebootCapable()) {
183         // If init does not have the CAP_SYS_BOOT capability, it is running in a
容器.
184         // In that case, receiving SIGTERM will cause the system to shut down.
185         InstallSigtermHandler();
186     }
187
188     property_load_boot_defaults();//从文件中加载一些属性，读取usb配置
189     export_oem_lock_status();//设置ro.boot.flash.locked 属性
190     start_property_service();//开启一个socket监听系统属性的设置
191     set_usb_controller();//设置sys.usb.controller 属性
192
193     const BuiltinFunctionMap function_map;//方法映射           "class_start" ->
"do_class_start"
194     /*
195     *1.C++中::表示静态方法调用，相当于java中static的方法196
196     */
197     Action::set_function_map(&function_map);//将function_map存放到Action中作 为成员属性
198
199     subcontexts = InitializeSubcontexts(); 200
201     ActionManager& am = ActionManager::GetInstance();
202     ServiceList& sm = ServiceList::GetInstance(); 203
204     LoadBootScripts(am, sm);//解析xxx.rc
205
206     // Turning this on and letting the INFO logging be discarded adds 0.2s to
207     // Nexus 9 boot time, so it's disabled by default.
208     if (false) DumpState();//打印一些当前Parser的信息，默认是不执行的209
210     am.QueueEventTrigger("early-init");//QueueEventTrigger用于触发Action,这里 触发 early-
init事件
211
212     // Queue an action that waits for coldboot done so we know ueventd has set up all of
/dev...

```



```

213     am.QueueBuiltinAction(wait_for_coldboot_done_action,
    "wait_for_coldboot_done");//QueueBuiltinAction用于添加Action，第一个参数是Action要执行的Command,第二个是Trigger
214     // ... so that we can start queuing up actions that require stuff from
    /dev.
215     am.QueueBuiltinAction(MixHwrngIntoLinuxRngAction, "MixHwrngIntoLinuxRng");
216     am.QueueBuiltinAction(SetMmapRndBitsAction, "SetMmapRndBits");
217     am.QueueBuiltinAction(SetKptrRestrictAction, "SetKptrRestrict");
218     am.QueueBuiltinAction(keychord_init_action, "keychord_init");
219     am.QueueBuiltinAction(console_init_action, "console_init");
220
221     // Trigger all the boot actions to get us started.
222     am.QueueEventTrigger("init");
223
224     // Repeat mix_hwrng_into_linux_rng in case /dev/hw_random or
    /dev/random
225     // wasn't ready immediately after wait_for_coldboot_done
226     am.QueueBuiltinAction(MixHwrngIntoLinuxRngAction,
    "MixHwrngIntoLinuxRng");
227
228     // Don't mount filesystems or start core system services in charger mode.
    std::string bootmode = GetProperty("ro.bootmode", ""); if
229     (bootmode == "charger") {
230         am.QueueEventTrigger("charger");
231     } else {
232         am.QueueEventTrigger("late-init");
233     }
234
235     // Run all property triggers based on current state of the properties.
236     am.QueueBuiltinAction(queue_property_triggers_action,
    "queue_property_triggers");
237
238     while (true) {
239         // By default, sleep until something happens.
240         int epoll_timeout_ms = -1; //epoll超时时间，相当于阻塞时间
241
242         if (do_shutdown && !shutting_down)
243             { do_shutdown = false;
244             if (HandlePowerctlMessage(shutdown_command)) {
245                 shutting_down = true;
246             }
247         }
248
249         if (!(waiting_for_prop || Service::is_exec_service_running())) {
250             am.ExecuteOneCommand();//执行一个command
251         }
252     }
253     /*

```

```
254         * 1.waiting_for_prop和IsWaitingForExec都是判断一个Timer是否为空，相当
      于一个标志位
255         * 2.waiting_for_prop负责属性设置，IsWaitingForExe负责service运行
256         * 3. 当有属性设置或Service开始运行时，这两个值就不为空，直到执行完毕才置为空
257         * 4. 其实这两个判断条件主要作用就是保证属性设置和service启动的完整性，也可以
      说是为了同步
258         */
259         if (!(waiting_for_prop || Service::is_exec_service_running())) {
260             if (!shutting_down) {
```

```

261         auto next_process_restart_time = RestartProcesses(); //重启服
务
262
263         // If there's a process that needs restarting, wake up in
time for that.
264         if (next_process_restart_time)
265             { epoll_timeout_ms =
std::chrono::ceil<std::chrono::milliseconds>(
266                 *next_process_restart_time -
boot_clock::now())
267                 .count();
268                 if (epoll_timeout_ms < 0) epoll_timeout_ms = 0;
269             }
270     }
271
272     // If there's more work to do, wake up again immediately.
273     if (am.HasMoreCommands()) epoll_timeout_ms = 0; //当还有命令要执行时，将
epoll_timeout_ms设置为0
274 }
275
276     epoll_event ev;
277     /*
278     * 1. epoll_wait与epoll_create1、epoll_ctl是一起使用的
279     * 2. epoll_create1用于创建epoll的文件描述符，epoll_ctl、epoll_wait都把它创建的fd
作为第一个参数传入
280     * 3. epoll_ctl用于操作epoll，EPOLL_CTL_ADD：注册新的fd到epfd中，
EPOLL_CTL_MOD：修改已经注册的fd的监听事件，EPOLL_CTL_DEL：从epfd中删除一个fd；
281     * 4. epoll_wait用于等待事件的产生，epoll_ctl调用EPOLL_CTL_ADD时会传入需要监听什么
类型的事件，
282     * 比如EPOLLIN表示监听fd可读，当该fd有可读的数据时，调用epoll_wait经过
epoll_timeout_ms时间就会把该事件的信息返回给&ev 283
283     */
284     int nr = TEMP_FAILURE_RETRY(epoll_wait(epoll_fd, &ev, 1,
epoll_timeout_ms));
285     if (nr == -1) {
286         PLOG(ERROR) << "epoll_wait failed";
287     } else if (nr == 1) {
288         ((void (*)(void)) ev.data.ptr)(); //当有event返回时，取出
ev.data.ptr（之前epoll_ctl注册时的回调函数），直接执行
289         //在signal_handler_init和start_property_service有注册两个fd的监 听，一个用
于监听SIGCHLD（子进程结束信号），一个用于监听属性设置
290     }
291 }
292
293     return 0;
294 }

```

第二节 init.rc解析

init.rc是一个非常重要的配置文件，它是由Android初始化语言（Android Init Language）编写的脚本

本，它主要包含五种类型语句：Action（Action中包含了一系列的Command）、Commands（init语言中的命令）、Services（由init进程启动的服务）、Options（对服务进行配置的选项）和Import（引入其他配置文件）。init.rc的配置代码如下所示。

```
1 # \system\core\rootdir\init.rc  
2 on init # L41
```

博学课道

```

3      sysctl ktz 0
4
5      # Mix device-specific information into the entropy pool
6      copy /proc/cmdline /dev/urandom
7      copy /default.prop /dev/urandom
8      ...
9
10 on <trigger> [&& <trigger>]* //设置触发器
11     <command>
12     <command> //动作触发之后要执行的命令
13
14 service <name> <pathname> [ <argument> ]* //<service的名字><执行程序路径><传递参数>
15     <option> //Options是Services的参数配置. 它们影响Service如何运行及运行时机
16     group <groupname> [ <groupname>\* ] //在启动Service前将group改为第一个groupname,第一个groupname是必须有的,
17     //默认值为root(或许默认值是无), 第二个groupname可以不设置,用于追加组(通过setgroups)
18     priority <priority> //设置进程优先级. 在-20~19之间, 默认值是0,能通过setpriority实现
19     socket <name> <type> <perm> [ <user> [ <group> [ <seclabel> ] ] ] //创建一个unix域的socket,名字叫/dev/socket/name, 并将fd返回给Service. type 只能是"dgram", "stream" or "seqpacket".
20     ...

```

2.1 Action

Action: 通过触发器trigger, 即以on开头的语句来决定执行相应的service的时机, 具体有如下时机:

- on early-init; 在初始化早期阶段触发;
- on init; 在初始化阶段触发;
- on late-init; 在初始化晚期阶段触发;
- on boot/charger; 当系统启动/充电时触发, 还包含其他情况, 此处不一一列举;
- on property:=; 当属性值满足条件时触发

2.2 Service

服务Service, 以 service开头, 由init进程启动, 一般运行在init的一个子进程, 所以启动service前需要判断对应的可执行文件是否存在。init生成的子进程, 定义在rc文件, 其中每一个service在启动时会通过fork方式生成子进程。

例如: `service servicemanager /system/bin/servicemanager` 代表的是服务名为servicemanager, 服务执行的路径为/system/bin/servicemanager。

2.3 Command

下面列举常用的命令

- class_start <service_class_name>: 启动属于同一个class的所有服务;
- start <service_name>: 启动指定的服务, 若已启动则跳过;
- stop <service_name>: 停止正在运行的服务
- setprop : 设置属性值
- mkdir : 创建指定目录

- ◆ `symlink <sym_link>`: 创建连接到的<sym_link>符号链接;
- ◆ `write` : 向文件path中写入字符串;
- ◆ `exec`: `fork`并执行, 会阻塞init进程直到程序完毕;
- ◆ `exprot` : 设定环境变量;

高学课道

- ◆ loglevel : 设置log级别

2.4 Options

Options是Service的可选项，与服务配合使用

- ◆ disabled: 不随class自动启动，只有根据service名才启动;
- ◆ oneshot: service退出后不再重启;
- ◆ user/group: 设置执行服务的用户/用户组，默认都是root; class: 设置所属的类名，当所属类启动/退出时，服务也启动/停止，默认为default; onrestart:当服务重启时执行相应命令;
- ◆ socket: 创建名为/dev/socket/<name>的socket

critical: 在规定时间内该service不断重启，则系统会重启并进入恢复模式

default: 意味着disabled=false, oneshot=false, critical=false。

```
1 service zygote /system/bin/app_process64 -Xzygote /system/bin --zygote --
  start-system-server
2     class main
3     priority -20
4     user root
5     group root readproc reserved_disk
6     socket zygote stream 660 root system
7     onrestart write /sys/android_power/request_state wake
8     onrestart write /sys/power/state on
9     onrestart restart audioserver
10    onrestart restart cameraserver
11    onrestart restart media
12    onrestart restart netd
13    onrestart restart wificond
14    writepid /dev/cpuset/foreground/tasks
```

2.5 service解析流程

```
1 // \system\core\init\init.cpp LoadBootScripts() L110
2 static void LoadBootScripts(ActionManager& action_manager, ServiceList&
  service_list) {
3     Parser parser = CreateParser(action_manager, service_list); //创建解析器
4
5     std::string bootscript = GetProperty("ro.boot.init_rc", "");
6     if (bootscript.empty()) {
7         parser.ParseConfig("/init.rc");
8         if (!parser.ParseConfig("/system/etc/init"))
9             { late_import_paths.emplace_back("/system/etc/init");
10         }
11         if (!parser.ParseConfig("/product/etc/init"))
12             { late_import_paths.emplace_back("/product/etc/init")
13             ;
14         }
15         if (!parser.ParseConfig("/odm/etc/init"))
16             { late_import_paths.emplace_back("/odm/etc/init");
17         }
18         if (!parser.ParseConfig("/vendor/etc/init"))
19             { late_import_paths.emplace_back("/vendor/etc/init");
20         }
21     } else {
22         parser.ParseConfig(bootscript); //开始解析
```

```

1 // \system\core\init\init.cpp CreateParser() L100
2 Parser CreateParser(ActionManager& action_manager, ServiceList&
  service_list) {
3     Parser parser;
4
5     parser.AddSectionParser("service", std::make_unique<ServiceParser>
  (&service_list, subcontexts)); //service解析
6     parser.AddSectionParser("on", std::make_unique<ActionParser>
  (&action_manager, subcontexts));
7     parser.AddSectionParser("import", std::make_unique<ImportParser>
  (&parser));
8
9     return parser;
10 }

```

```

1 // \system\core\init\parser.cpp ParseData() L 42
2 void Parser::ParseData(const std::string& filename, const std::string& data,
  size_t* parse_errors) {
3     ...
4
5     for (;;) {
6         switch (next_token(&state))
7         { case T_EOF:
8             end_section();
9             return;
10        case T_NEWLINE:
11            ...
12            if (section_parsers_.count(args[0]))
13            { end_section();
14              section_parser = section_parsers_[args[0]].get();
15              section_start_line = state.line;
16              if (auto result =
17                section_parser->ParseSection(std::move(args),
18                filename, state.line); // L95
19                !result)
20                { (*parse_errors)++;
21                  LOG(ERROR) << filename << ": " << state.line << ": "
22                  << result.error();
23                  section_parser = nullptr;
24              }
25            } else if (section_parser) {
26                if (auto result = section_parser-
27                >ParseLineSection(std::move(args), state.line); // L102
28                !result)
29                { (*parse_errors)++;
30                  LOG(ERROR) << filename << ": " << state.line << ": "
31                  << result.error();
32              }
33            }
34            args.clear();
35            break;
36        case T_TEXT:
37            args.emplace_back(state.text);
38            break;

```



```

35     }
36 }
37 }

```

```

1 // \system\core\init\service.cpp ParseSection() L1180
2 Result<Success> ServiceParser::ParseSection(std::vector<std::string>&&args,
3                                             const std::string& filename, int
4 line) {
5     if (args.size() < 3) {
6         return Error() << "services must have a name and a program";
7     }
8     const std::string& name = args[1];
9     if (!IsValidName(name)) {
10         return Error() << "invalid service name '" << name << "'";
11     }
12
13     Subcontext* restart_action_subcontext = nullptr;
14     if (subcontexts_) {
15         for (auto& subcontext : *subcontexts_) {
16             if (Startswith(filename, subcontext.path_prefix()))
17                 { restart_action_subcontext = &subcontext;
18                 break;
19             }
20         }
21     }
22
23     std::vector<std::string> str_args(args.begin() + 2, args.end());
24     service_ = std::make_unique<Service>(name, restart_action_subcontext,
25 str_args); //构建出一个service对象
26     return Success();
27 }

```

```

1 // \system\core\init\service.cpp ParseLineSection() L1206
2 Result<Success> ServiceParser::ParseLineSection(std::vector<std::string>&&
3 args, int line) {
4     return service_ ? service_>ParseLine(std::move(args)) : Success(); 4 }

```

```

1 // \system\core\init\service.cpp EndSection() L1210
2 Result<Success> ServiceParser::EndSection() {
3     if (service_) {
4         Service* old_service = service_list->FindService(service_>name());
5         if (old_service) {
6             if (!service_>is_override()) {
7                 return Error() << "ignored duplicate definition of service
8 '" << service_>name()
9                                     << "'";
10             }
11             service_list->RemoveService(*old_service);
12             old_service = nullptr;
13         }
14
15         service_list->AddService(std::move(service_));

```

```

16     }
17
18     return Success();
19 }

```

```

1 // \system\core\init\service.cpp AddService() L1082
2 void ServiceList::AddService(std::unique_ptr<Service> service) {
3     services_.emplace_back(std::move(service)); 4
4 }

```

上面解析完成后，接下来就是启动Service，这里我们以启动Zygote来分析

```

1 # \system\core\rootdir\init.rc L680
2 on nonencrypted
3     class_start main //class_start是一个命令，通过do_class_start函数处理
4     class_start late_start

```

```

1 // \system\core\init\builtins.cpp do_class_start() L101
2 static Result<Success> do_class_start(const BuiltinArguments& args) {
3     // Starting a class does not start services which are explicitly
    disabled.
4     // They must be started individually.
5     for (const auto& service : ServiceList::GetInstance()) {
6         if (service->classnames().count(args[1])) {
7             if (auto result = service->StartIfNotDisabled(); !result) {
8                 LOG(ERROR) << "Could not start service '" << service->name()
9                     << "' as part of class '" << args[1] << "': " <<
    result.error();
10            }
11        }
12    }
13    return Success();
14 }

```

```

1 // \system\core\init\service.cpp StartIfNotDisabled() L977
2 Result<Success> Service::StartIfNotDisabled() {
3     if (!(flags_ & SVC_DISABLED)) {
4         return Start();
5     } else {
6         flags_ |= SVC_DISABLED_START; 7 }
8     return Success();
9 }

```

```

1 // \system\core\init\service.cpp Start() L785
2 Result<Success> Service::Start() {
3     //如果service已经运行，则不启动
4     if (flags_ & SVC_RUNNING) {
5         if ((flags_ & SVC_ONESHOT) && disabled) {
6             flags_ |= SVC_RESTART; 7
            }
8         // It is not an error to try to start a service that is already
    running.
9         return Success();

```

```

10     }
11
12     ...
13     //判断需要启动的service的对应的执行文件是否存在，不存在则不启动service struct
14     stat sb;
15     if (stat(args_[0].c_str(), &sb) == -1) { flags_
16         |= SVC_DISABLED;
17         return ErrnoError() << "Cannot find '" << args_[0] << "'";
18     }
19
20     std::string scon;
21     if (!seclabel_.empty()) { scon =
22         seclabel_;
23     } else {
24         auto result = ComputeContextFromExecutable(args_[0]); if
25         (!result) {
26             return result.error();
27         }
28         scon = *result;
29     }
30
31     LOG(INFO) << "starting service '" << name_ << "'...";
32     //如果子进程没有启动，则调用fork函数创建子进程
33     pid_t pid = -1;
34     if (namespace_flags_) {
35         pid = clone(nullptr, nullptr, namespace_flags_ | SIGCHLD, nullptr);
36     } else {
37         pid = fork();
38     }
39
40     if (pid == 0) { //当期代码逻辑在子进程中运行
41         umask(077);
42
43
44         //调用execv函数，启动service子进程
45         if (!ExpandArgsAndExecv(args_)) {
46             PLOG(ERROR) << "cannot execve('" << args_[0] << "')";
47         }
48
49         _exit(127);
50     }
51     return Success();
52 }

```

第三节 Zygote

3.1 Zygote概叙

Zygote中文翻译为“受精卵”，正如其名，它主要用于孵化子进程。在Android系统中有以下两种程序：

java应用程序，主要基于ART虚拟机，所有的应用程序apk都属于这类native程序，也就是利用C或C++语言开发的程序，如bootanimation。所有的Java应用程序进程及系统服务SystemService进程都由Zygote进程通过Linux的fork()函数孵化出来的，这也就是为什么把它称为Zygote的原因，因为他就像一个受精卵，孵化出无数子进程，而native程序则由Init程序创建启动。Zygote进程最初的名字不是“zygote”而是“app_process”，这个名字是在Android.mk文件中定义的

Zygote是Android中的第一个art虚拟机，他通过socket的方式与其他进程进行通信。这里的“其他进程”其实主要是系统进程——SystemService

Zygote是一个C/S模型，Zygote进程作为服务端，它主要负责创建Java虚拟机，加载系统资源，启动SystemServer进程，以及在后续运行过程中启动普通的应用程序，其他进程作为客户端向它发出“孵化”请求，而Zygote接收到这个请求后就“孵化”出一个新的进程。比如，当点击Launcher里的应用程序图标去启动一个新的应用程序进程时，这个请求会到达框架层的核心服务ActivityManagerService中，当AMS收到这个请求后，它通过调用Process类发出一个“孵化”子进程的Socket请求，而Zygote监听到这个请求后就立刻fork一个新的进程出来

3.2 Zygote触发过程

1. init.zygoteXX.rc

```
1 | import /init.${ro.zygote}.rc
```

`${ro.zygote}` 会被替换成 `ro.zygote` 的属性值，这个是由不同的硬件厂商自己定制的，有四个值，

- `zygote32`: `zygote` 进程对应的执行程序是 `app_process` (纯 32bit 模式)
- `zygote64`: `zygote` 进程对应的执行程序是 `app_process64` (纯 64bit 模式)
- `zygote32_64`: 启动两个 `zygote` 进程 (名为 `zygote` 和 `zygote_secondary`)，对应的执行程序分别是 `app_process32` (主模式)
- `zygote64_32`: 启动两个 `zygote` 进程 (名为 `zygote` 和 `zygote_secondary`)，对应的执行程序分别是 `app_process64` (主模式)、`app_process32`

2. start zygote

位置: `system\core\rootdir\init.rc` 560

```
1 | # It is recommended to put unnecessary data/ initialization from post-fs-  
  data  
2 | # to start-zygote in device's init.rc to unblock zygote start.  
3 | on zygote-start && property:ro.crypto.state=unencrypted  
4 |     # A/B update verifier that marks a successful boot.  
5 |     exec_start update_verifier_nonencrypted  
6 |     start netd  
7 |     start zygote  
8 |     start zygote_secondary  
9 |  
10 | on zygote-start && property:ro.crypto.state=unsupported  
11 |     # A/B update verifier that marks a successful boot.  
12 |     exec_start update_verifier_nonencrypted  
13 |     start netd  
14 |     start zygote  
15 |     start zygote_secondary  
16 |  
17 | on zygote-start && property:ro.crypto.state=encrypted &&  
  property:ro.crypto.type=file  
18 |     # A/B update verifier that marks a successful boot.  
19 |     exec_start update_verifier_nonencrypted  
20 |     start netd  
21 |     start zygote  
22 |     start zygote_secondary
```

`zygote-start` 是在 `on late-init` 中触发的

```
1 | # Mount filesystems and start core system services.
```

```

2 on late-init
3     trigger early-fs 4
5     # Mount fstab in init. {$device}.rc by mount_all command. Optional parameter
6     # '--early' can be specified to skip entries with 'latemount'.
7     # /system and /vendor must be mounted by the end of the fs stage,
8     # while /data is optional.
9     trigger fs
10    trigger post-fs 11
12    # Mount fstab in init. {$device}.rc by mount_all with '--late' parameter
13    # to only mount entries with 'latemount'. This is needed if '--early' is
14    # specified in the previous mount_all command on the fs stage.
15    # With /system mounted and properties form /system + /factory available,
16    # some services can be started.
17    trigger late-fs 18
19    # Now we can mount /data. File encryption requires keymaster to decrypt
20    # /data, which in turn can only be loaded when system properties are present.
21    trigger post-fs-data 22
23    # Now we can start zygote for devices with file based encryption
24    trigger zygote-start #
25    zygote-start 是在 on late-init 中触发的
26
27    # Load persist properties and override properties (if enabled) from
    /data.
28    trigger load_persist_props_action 29
30    # Remove a file to wake up anything waiting for firmware.
31    trigger firmware_mounts_complete 32
33    trigger early-boot
34    trigger boot

```

```

1 if (bootmode == "charger")
2     { am.QueueEventTrigger("charger");
3 } else {
4     am.QueueEventTrigger("late-init");
5 }

```

3. app_processXX

位置\frameworks\base\cmds\app_process\

```

1 app_process_src_files := \
2     app_main.cpp \
3
4 LOCAL_SRC_FILES := $(app_process_src_files)
5 ...
6 LOCAL_MODULE := app_process
7 LOCAL_MULTILIB := both
8 LOCAL_MODULE_STEM_32 := app_process32
9 LOCAL_MODULE_STEM_64 := app_process64

```

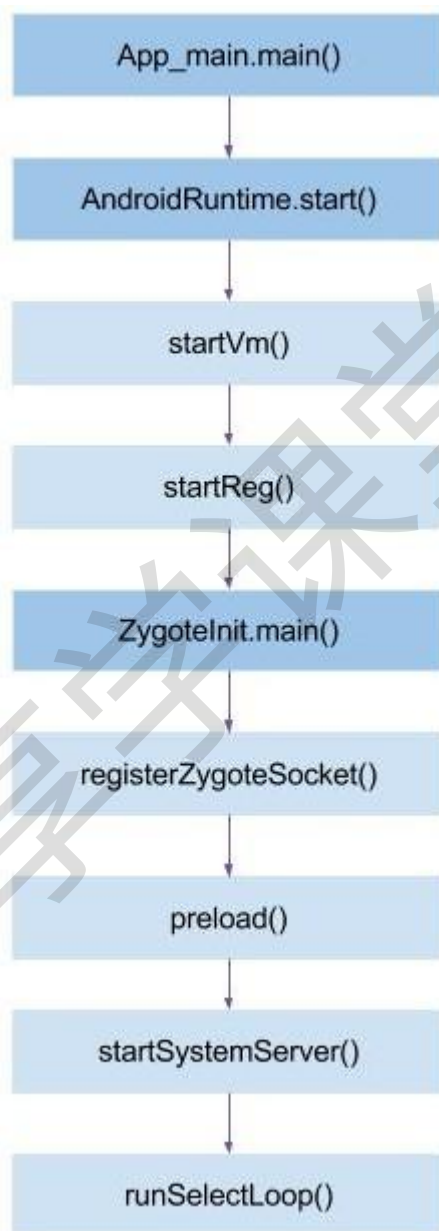
3.3 Zygote启动过程

位置\frameworks\base\cmds\app_process\app_main.cpp

在app_main.cpp的main函数中，主要做的事情就是参数解析。这个函数有两种启动模式：

1. 一种是zygote模式，也就是初始化zygote进程，传递的参数有--start-system-server --socket-name=zygote，前者表示启动SystemService，后者指定socket的名称
2. 一种是application模式，也就是启动普通应用程序，传递的参数有class名字以及class带的参数

两者最终都是调用AppRuntime对象的start函数，加载ZygoteInit或RuntimeInit两个Java类，并将之前整理的参数传入进去



app_process

```
1 // \frameworks\base\cmds\app_process\app_main.cpp main() L280
2 if (strcmp(arg, "--zygote") == 0) {
3     zygote = true;
4     niceName = ZYGOTE_NICE_NAME;
5 } else if (strcmp(arg, "--start-system-server") == 0) {
6     startSystemService = true;
7 } else if (strcmp(arg, "--application") == 0) {
```

```

8         application = true;
9     }
10    // L349
11    if (zygote) {
12        //这些Java的应用都是通过 AppRuntime.start (className)开始的
13        //其实AppRuntime是AndroidRuntime的子类，它主要实现了几个回调函数，而start()方法
        是实现AndroidRuntime这个方法类里
14        runtime.start("com.android.internal.os.ZygoteInit", args, zygote);
15    } else if (className)
16        { runtime.start("com.android.internal.os.RuntimeInit", args,
17          zygote);

```

app_process 里面定义了三种应用程序类型:

1. Zygote: com.android.internal.os.ZygoteInit
2. System Server, 不单独启动，而是由Zygote启动
3. 其他指定类名的Java 程序

```

1  // \frameworks\base\core\jni\androidRuntime.cpp
2  class AppRuntime : public AndroidRuntime
3  {
4  public:
5      AppRuntime(char* argBlockStart, const size_t argBlockLength)
6          : AndroidRuntime(argBlockStart, argBlockLength)
7          , mClass(NULL)
8          {
9      }
10
11     void setClassNameAndArgs(const String8& className, int argc, char *
const *argv) {
12         mClassName = className;
13         for (int i = 0; i < argc; ++i)
14             { mArgs.add(String8(argv[i]));
15         }
16     }
17
18     virtual void onVmCreated(JNIEnv* env)
19     {
20         if (mClassName.isEmpty()) {
21             return; // Zygote. Nothing to do here.
22         }
23
24         /*
25          * This is a little awkward because the JNI FindClass call uses the
26          * class loader associated with the native method we're executing
27          in.
28          * If called in onStarted (from RuntimeInit.finishInit because we're
29          calling
30          * from a boot class' native method, and so wouldn't look for the
31          class
32          * we're trying to look up in CLASSPATH. Unfortunately it needs to,
33          * because the "am" classes are not boot classes.
34          *
35          * The easiest fix is to call FindClass here, early on before we
36          start

```



```

34         * executing boot class Java code and thereby deny ourselves access to
35         * non-boot classes.
36     */
37     char* slashClassName = toSlashClassName(mClassName.string());
38     mClass = env->FindClass(slashClassName);
39     if (mClass == NULL) {
40         ALOGE("ERROR: could not find class '%s'\n",
mClassName.string()); 41 }
42     free(slashClassName); 43
44     mClass=reinterpret_cast<jclass>(env->NewGlobalRef(mClass)); 45     }
46
47     virtual void onStarted() 48 {
49         sp<ProcessState> proc = ProcessState::self();
50         ALOGV("App process: starting thread pool.\n");
51         proc->startThreadPool(); 52
53         AndroidRuntime* ar = AndroidRuntime::getRuntime();
54         ar->callMain(mClassName, mClass, mArgs); 55
56         IPCThreadState::self()->stopProcess();
57         hardware::IPCThreadState::self()->stopProcess(); 58     }
59
60     virtual void onZygoteInit() 61
61     {
62         sp<ProcessState> proc = ProcessState::self();
63         ALOGV("App process: starting thread pool.\n");
64         proc->startThreadPool(); 65     }
66
67     virtual void onExit(int code) 68
68     {
69         if (mClassName.isEmpty()) {
70             // if zygote
71             IPCThreadState::self()->stopProcess();
72             hardware::IPCThreadState::self()->stopProcess(); 73         }
74
75         AndroidRuntime::onExit(code); 76
76     }
77
78
79     String8 mClassName;
80     Vector<String8> mArgs;
81     jclass mClass; 82
82 };
83 }

```

什么是Runtime ?

h <https://stackoverflow.com/questions/3900549/what-is-runtime>

As per Wikipedia: [runtime library/run-time system](#).

In computer programming, a runtime library is a special program library used by a compiler, to implement functions built into a programming language, during the runtime (execution) of a computer program. This often includes functions for input and output, or for memory management.

A run-time system (also called runtime system or just runtime) is software designed to support the execution of computer programs written in some computer language. The run-time system contains implementations of basic low-level commands and may also implement higher-level commands and may support type checking, debugging, and even code generation and optimization. Some services of the run-time system are accessible to the programmer through an application programming interface, but other services (such as task scheduling and resource management) may be inaccessible.

归纳起来的意思就是，Runtime 是支撑程序运行的基础库，它是与语言绑定在一起的。比如：

- ◆ C Runtime: 就是C standard lib, 也就是我们常说的libc。(有意思的是，Wiki会自动将“C runtime” 重定向到 “C Standard Library”).
- ◆ Java Runtime: 同样，Wiki将其重定向到” Java Virtual Machine”，这里当然包括Java 的支撑类库(. jar).
- ◆ AndroidRuntime: 显而易见，就是为Android应用运行所需的运行时环境。这个环境包括以下东东：
 - Dalvik VM: Android的Java VM，解释运行Dex格式Java程序。每个进程运行一个虚拟机（什么叫运行虚拟机？说白了，就是一些C代码，不停的去解释Dex格式的二进制码(Bytecode)，把它们转成机器码(Machine code)，然后执行，当然，现在大多数的Java 虚拟机都支持JIT，也就是说，bytecode可能在运行前就已经被转换成机器码，从而大大提高了性能。过去一个普遍的认识是Java 程序比C, C++等静态编译的语言慢，但随着JIT的介入和发展，这个已经完全是过去时了，JIT的动态性运行允许虚拟机根据运行时环境，优化机器码的生成，在某些情况下，Java甚至可以比C/C++跑得更快，同时又兼具平台无关的特性，这也是为什么Java如今如此流行的原因之一吧）。
 - Android的Java 类库，大部分来自于 Apache Hamony，开源的Java API 实现，如 java.lang, java.util, java.net. 但去除了AWT, Swing 等部件。
 - JNI: C和Java互调的接口。
 - Libc: Android也有很多C代码，自然少不了libc，注意的是，Android的libc叫 bionic C

```
1 // \frameworks\base\core\jni\androidRuntime.cpp start() L1091
2 void AndroidRuntime::start(const char* className, const Vector<String8>&
options, bool zygote)
3 {
4     ...
5     JNIEnv* env;
6     //JNI_CreateJavaVM L1015
7     if (startVm(&mJavaVM, &env, zygote) != 0)
8     { return;
9     }
10
11     onVmCreated(env);
12
13     /*
14      * Register android functions.
```

```

15     */
16     if (startReg(env) < 0) {
17         ALOGE("Unable to register all android natives\n");
18         return;
19     }
20     ...
21 }

```

◆ Java虚拟机的启动大致做了以下一些事情：

1. 从property读取一系列启动参数。
2. 创建和初始化结构体全局对象（每个进程）gDVM，及对应与JavaVM和JNIEnv的内部结构体JavaVMExt，JNIEnvExt。
3. 初始化java虚拟机，并创建虚拟机线程
4. 注册系统的JNI，Java程序通过这些JNI接口来访问底层的资源。

```

1     loadJniLibrary("javacore");
2     loadJniLibrary("nativehelper");

```

5. 为Zygote的启动做最后的准备，包括设置SID/UID，以及mount 文件系统
6. 返回JavaVM 给Native代码，这样它就可以向上访问Java的接口

```

1 // \frameworks\base\core\jni\androidRuntime.cpp startVm() L596
2 int AndroidRuntime::startVm(JavaVM** pJavaVM, JNIEnv** pEnv, bool zygote)
3 {
4     ...
5     // L1015
6     /*
7      * Initialize the VM.
8      *
9      * The JavaVM* is essentially per-process, and the JNIEnv* is per-
10     thread.
11     * If this call succeeds, the VM is ready, and we can start issuing
12     * JNI calls.
13     */
14     //startVM的前半部分是在处理虚拟机的启动参数，处理完配置参数后，会调用libart.so提供
    的一个接口：JNI_CreateJavaVM函数
15     if (JNI_CreateJavaVM(pJavaVM, pEnv, &initArgs) < 0) {
16         ALOGE("JNI_CreateJavaVM failed\n");
17         return -1;
18     }
19     ...

```

```

1 // \art\runtime\java_vm_ext.cc JNI_CreateJavaVM() L1139
2 extern "C" jint JNI_CreateJavaVM(JavaVM** p_vm, JNIEnv** p_env, void*
    vm_args) {
3     ScopedTrace trace( FUNCTION );
4     const JavaVMInitArgs* args = static_cast<JavaVMInitArgs*>(vm_args);
5     if (JavaVMExt::IsBadJniVersion(args->version)) {
6         LOG(ERROR) << "Bad JNI version passed to CreateJavaVM: " << args-
7         >version;
8         return JNI_EVERSION;
9     }
10    RuntimeOptions options;
11    for (int i = 0; i < args->nOptions; ++i) {
12        JavaVMOption* option = &args->options[i];

```

```

12     options.push_back(std::make_pair(std::string(option->optionString), option-
    >extraInfo));
13 }
14 bool ignore_unrecognized = args->ignoreUnrecognized;
15 //通过Runtime的create方法创建单例的Runtime对象
16 if (!Runtime::Create(options, ignore_unrecognized)) {
17     return JNI_ERR; 18
19 }
20 // Initialize native loader. This step makes sure we have
21 // everything set up before we start using JNI.
22 android::InitializeNativeLoader(); 23
24 Runtime* runtime = Runtime::Current();
25 bool started = runtime->Start();
26 if (!started) {
27     delete Thread::Current()->GetJniEnv();
28     delete runtime->GetJavaVM();
29     LOG(WARNING) << "CreateJavaVM failed";
30     return JNI_ERR; 31
31 }
32
33 *p_env = Thread::Current()->GetJniEnv();
34 *p_vm = runtime->GetJavaVM();
35 return JNI_OK; 36
36 }

```

首先通过Runtime的create方法创建单例的Runtime对象，runtime负责提供art虚拟机的运行时环境，然后调用其init方法来初始化虚拟机

```

1 // \art\runtime\runtime.cc Init() L1109
2 bool Runtime::Init(RuntimeArgumentMap&& runtime_options_in) { 3 ...
4 // L1255 创建堆管理对象。
5 heap_ = new gc::Heap(runtime_options.GetOrDefault(Opt::MemoryInitialSize),
6                     runtime_options.GetOrDefault(Opt::HeapGrowthLimit),
7                     runtime_options.GetOrDefault(Opt::HeapMinFree),
8                     runtime_options.GetOrDefault(Opt::HeapMaxFree), 9
9                     runtime_options.GetOrDefault(Opt::HeapTargetUtilization),
10                    foreground_heap_growth_multiplier,
11                    runtime_options.GetOrDefault(Opt::MemoryMaximumSize), 12
12                    runtime_options.GetOrDefault(Opt::NonMovingSpaceCapacity),
13                    runtime_options.GetOrDefault(Opt::Image), 14
14                    runtime_options.GetOrDefault(Opt::ImageInstructionSet),
15                    // Override the collector type to CC if the read barrier
16                    config.
17                    kUseReadBarrier ? gc::kCollectorTypeCC :
18                    xgc_option.collector_type_,
19                    kUseReadBarrier ?
20                    BackgroundGcOption(gc::kCollectorTypeCCBackground) 18 :
21                    runtime_options.GetOrDefault(Opt::BackgroundGc),
22                    runtime_options.GetOrDefault(Opt::LargeObjectSpace),

```

```

20 runtime_options.GetOrDefault(Opt::LargeObjectThreshold),
21         runtime_options.GetOrDefault(Opt::ParallelGCThreads),
22         runtime_options.GetOrDefault(Opt::ConcGCThreads),
23         runtime_options.Exists(Opt::LowMemoryMode), 24
runtime_options.GetOrDefault(Opt::LongPauseLogThreshold),
25
runtime_options.GetOrDefault(Opt::LongGCLogThreshold),
26         runtime_options.Exists(Opt::IgnoreMaxFootprint),
27         runtime_options.GetOrDefault(Opt::UseTLAB),
28         xgc_option.verify_pre_gc_heap_,
29         xgc_option.verify_pre_sweeping_heap_,
30         xgc_option.verify_post_gc_heap_,
31         xgc_option.verify_pre_gc_rosalloc_,
32         xgc_option.verify_pre_sweeping_rosalloc_,
33         xgc_option.verify_post_gc_rosalloc_,
34         xgc_option.gcstress_,
35         xgc_option.measure_,
36
runtime_options.GetOrDefault(Opt::EnableHSpaceCompactForOOM),
37
runtime_options.GetOrDefault(Opt::HSpaceCompactForOOMMinIntervalsMs)); 38 ...
39 // L1408创建java虚拟机对象
40 std::string error_msg;
41 java_vm_ = JavaVMExt::Create(this, runtime_options, &error_msg);
42 if (java_vm_.get() == nullptr) {
43     LOG(ERROR) << "Could not initialize JavaVMExt: " << error_msg;
44     return false; 45
45 }
46
47 // Add the JNIEnv handler.
48 // TODO Refactor this stuff.
49 java_vm_>AddEnvironmentHook(JNIEnvExt::GetEnvHandler); 50
51 Thread::Startup(); 52
53 // ClassLinker needs an attached thread, but we can't fully attach a thread
without creating
54 // objects. We can't supply a thread group yet; it will be fixed later. Since we are
the main
55 // thread, we do not get a java peer.
56 // L1424 连接主线程
57 Thread* self = Thread::Attach("main", false, nullptr, false);
58 CHECK_EQ(self->GetThreadId(), ThreadList::kMainThreadId);
59 CHECK(self != nullptr); 60
...
61 // L1437 创建类连接器
62 if (UNLIKELY(IsAotCompiler())) {
63     class_linker_ = new AotClassLinker(intern_table_);
64 } else {
65     class_linker_ = new ClassLinker(intern_table_); 66 }

```

```
67     if (GetHeap()->HasBootImageSpace()) {  
68         //初始化类连接器  
69         bool result = class_linker_->InitFromBootImage(&error_msg);  
70         if (!result) {
```

深度学习

```

71     LOG(ERROR) << "Could not initialize from image: " << error_msg;
72     return false;
73 }
74 ...
75 }
76 }
77 }

```

1. new gc::heap(), 创建Heap对象, 这是虚拟机管理对内存的起点。
 2. new JavaVmExt(), 创建Java虚拟机实例。
 3. Thread::attach(), attach主线程
 4. 创建ClassLinker
 5. 初始化ClassLinker, 成功attach到runtime环境后, 创建ClassLinker实例负责管理java class
- 到这里, 虚拟机的创建和初始化就完成了

```

1  // \art\runtime\thread.cc Attach() L775
2  template <typename PeerAction>
3  Thread* Thread::Attach(const char* thread_name, bool as_daemon, PeerAction peer_action) {
4      Runtime* runtime = Runtime::Current();
5      if (runtime == nullptr) {
6          LOG(ERROR) << "Thread attaching to non-existent runtime: " <<
7              ((thread_name != nullptr) ? thread_name : "(Unnamed)");
8          return nullptr; 9
9      }
10     Thread* self;
11     {
12         MutexLock mu(nullptr, *Locks::runtime_shutdown_lock_);
13         if (runtime->IsShuttingDownLocked()) {
14             LOG(WARNING) << "Thread attaching while runtime is shutting down: " <<
15                 ((thread_name != nullptr) ? thread_name : "(Unnamed)");
16             return nullptr;
17         } else {
18             Runtime::Current()->StartThreadBirth();
19             self = new Thread(as_daemon);
20             bool init_success = self->Init(runtime->GetThreadList(), runtime-
21 >GetJavaVM());
22             Runtime::Current()->EndThreadBirth();
23             if (!init_success) {
24                 delete self;
25                 return nullptr; 25
26             }
27         }
28
29         self->InitStringEntryPoints(); 30
31         CHECK_NE(self->GetState(), kRunnable);
32         self->SetState(kNative); 33
34         // Run the action that is acting on the peer.
35         if (!peer_action(self)) {
36             runtime->GetThreadList()->Unregister(self);
37             // Unregister deletes self, no need to do this here.

```

```
38     return nullptr; 39  
    }
```

博学文库


```

40
41     if (VLOG_IS_ON(threads)) {
42         if (thread_name != nullptr) {
43             VLOG(threads) << "Attaching thread " << thread_name;
44         } else {
45             VLOG(threads) << "Attaching unnamed thread.";
46         }
47         ScopedObjectAccess soa(self);
48         self->Dump(LOG_STREAM(INFO));
49     }
50
51     {
52         ScopedObjectAccess soa(self);
53         runtime->GetRuntimeCallbacks()->ThreadStart(self);
54     }
55
56     return self;
57 }

```

除了系统的JNI接口（“javacore”，“nativehelper”），android framework 还有大量的Native实现，

Android将所有这些接口一次性的通过start_reg()来完成

```

1  // \frameworks\base\core\jni\androidRuntime.cpp startReg() L1511
2  int AndroidRuntime::startReg(JNIEnv* env)
3  {
4      ATRACE_NAME("RegisterAndroidNatives");
5      /*
6       * This hook causes all future threads created in this process to be
7       * attached to the JVM. (This needs to go away in favor of JNI
8       * Attach calls.)
9       */
10     androidSetCreateThreadFunc((android_create_thread_fn)
javaCreateThreadEtc);
11
12     ALOGV("--- registering native functions ---\n");
13
14     /*
15      * Every "register" function calls one or more things that return
16      * a local reference (e.g. FindClass). Because we haven't really
17      * started the VM yet, they're all getting stored in the base frame
18      * and never released. Use Push/Pop to manage the storage.
19      */
20     env->PushLocalFrame(200);
21
22     if (register_jni_procs(gRegJNI, NELEM(gRegJNI), env) < 0)
23     { env->PopLocalFrame(NULL);
24       return -1;
25     }
26     env->PopLocalFrame(NULL);
27
28     //createJavaThread("fubar", quickTest, (void*) "hello");
29
30     return 0;
31 }

```

```

1 // \system\core\libutils\Threads.cpp run() L662
2 status_t Thread::run(const char* name, int32_t priority, size_t stack) 3 {
4     LOG_ALWAYS_FATAL_IF(name == nullptr, "thread name not provided to
Thread::run");
5
6     Mutex::Autolock _l(mLock); 7
8     if (mRunning) {
9         // thread already started
10    return INVALID_OPERATION; 11
11    }
12
13    // reset status and exitPending to their default value, so we can
14    // try again after an error happened (either below, or in readyToRun())
15    mStatus = NO_ERROR;
16    mExitPending = false;
17    mThread = thread_id_t(-1); 18
19    // hold a strong reference on ourself
20    mHoldSelf = this; 21
22    mRunning = true; 23
24    bool res;
25    // L685 Android native层有两种Thread的创建方式
26    if (mCanCallJava) {
27        res = createThreadEtc(_threadLoop,
28                             this, name, priority, stack, &mThread);
29    } else {
30        res = androidCreateRawThreadEtc(_threadLoop,
31                                       this, name, priority, stack, &mThread); 32 }
33
34    if (res == false) {
35        mStatus = UNKNOWN_ERROR; // something happened!
36        mRunning = false;
37        mThread = thread_id_t(-1);
38        mHoldSelf.clear(); // "this" may have gone away after this. 39
40        return UNKNOWN_ERROR; 41
41    }
42
43    // Do not refer to mStatus here: The thread is already running (may, in fact
44    // already have exited with a valid mStatus result). The NO_ERROR indication
45    // here merely indicates successfully starting the thread and does not
46    // imply successful termination/execution.
47    return NO_ERROR; 48
49    // Exiting scope of mLock is a memory barrier and allows new thread to run
50 }

```

它们的区别在是是否能够调用Java端函数，普通的thread就是对pthread_create的简单封装

```

1 // \system\core\libutils\Threads.cpp run() L117
2 int androidCreateRawThreadEtc(android_thread_func_t entryFunction,
3                               void *userData,
4                               const char* threadName android_unused,
5                               int32_t threadPriority,
6                               size_t threadStackSize,
7                               android_thread_id_t *threadId)
8 {
9     ...
10    errno = 0;
11    pthread_t thread;
12    int result = pthread_create(&thread, &attr,
13                               (android_pthread_entry)entryFunction, userData); 14...
15    return 1;
16 }

```

```

1 // \frameworks\base\core\jni\androidRuntime.cpp javaCreateThreadEtc() L1271
2 int AndroidRuntime::javaCreateThreadEtc(
3     android_thread_func_t entryFunction,
4     void* userData,
5     const char* threadName,
6     int32_t threadPriority,
7     size_t threadStackSize,
8     android_thread_id_t* threadId)
9 {
10     void** args = (void**) malloc(3 * sizeof(void*)); // javaThreadShell
must free
11     int result;
12
13     LOG_ALWAYS_FATAL_IF(threadName == nullptr, "threadName not provided to
javaCreateThreadEtc");
14
15     args[0] = (void*) entryFunction;
16     args[1] = userData;
17     args[2] = (void*) strdup(threadName); // javaThreadShell must free
18
19     result = androidCreateRawThreadEtc(AndroidRuntime::javaThreadShell,
args,
20     threadName, threadPriority, threadStackSize, threadId);
21     return result;
22 }

```

```

1 // \frameworks\base\core\jni\androidRuntime.cpp javaThreadShell() L1242
2 int AndroidRuntime::javaThreadShell(void* args) {
3     void* start = ((void**)args)[0];
4     void* userData = ((void**)args)[1];
5     char* name = (char*) ((void**)args)[2]; // we own this storage
6     free(args);
7     JNIEnv* env;
8     int result;
9
10    /* hook us into the VM */
11    if (javaAttachThread(name, &env) != JNI_OK)
12        return -1;
13 }

```

```

14  /* start the thread running */
15  result = (*android_thread_func_t)start)(userData);
16
17  /* unhook us */
18  javaDetachThread();
19  free(name);
20
21  return result;
22  }

```

```

1  // \frameworks\base\core\jni\androidRuntime.cpp javaThreadShell() L1200
2  static int javaAttachThread(const char* threadName, JNIEnv** pEnv)
3  {
4      JavaVMAttachArgs args;
5      JavaVM* vm;
6      jint result;
7
8      vm = AndroidRuntime::getJavaVM();
9      assert(vm != NULL);
10
11     args.version = JNI_VERSION_1_4;
12     args.name = (char*) threadName;
13     args.group = NULL;
14
15     result = vm->AttachCurrentThread(pEnv, (void*) &args);
16     if (result != JNI_OK)
17         ALOGI("NOTE: attach of thread '%s' failed\n", threadName);
18
19     return result;
20 }

```

```

1  // \frameworks\base\core\java\com\android\internal\os\RuntimeInit.java main() L325
2  public static final void main(String[] argv) {
3      enableDdms();
4      if (argv.length == 2 && argv[1].equals("application")) {
5          if (DEBUG) Slog.d(TAG, "RuntimeInit: Starting application"); 6
7  //将System.out 和 System.err 输出重定向到Android 的Log系统（定义在
  android.util.Log)
8      redirectLogStreams();
9      } else {
10         if (DEBUG) Slog.d(TAG, "RuntimeInit: Starting tool"); 11     }
12
13  //commonInit(): 初始化了一下系统属性，其中最重要的一点就是设置了一个未捕捉异常的handler，
  当代码有任何未知异常，就会执行它，调试过Android代码的同学经常看到的” *** FATAL EXCEPTION IN
  SYSTEM PROCESS” 打印就出自这里
14     commonInit();
15
16     /*
17      * Now that we're running in interpreted code, call back into native code
18      * to run the system.
19      */
20     nativeFinishInit();

```

```

21
22         if (DEBUG) Slog.d(TAG, "Leaving RuntimeInit!");
23     }

```

```

1 // \frameworks\base\core\jni\androidRuntime.cpp nativeFinishInit() L225
2 /*
3  * Code written in the Java Programming Language calls here from main().
4  */
5 static void com_android_internal_os_RuntimeInit_nativeFinishInit(JNIEnv* env,
6     jobject clazz)
7 {
8     gCurRuntime->onStarted();
9 }

```

```

1 // \frameworks\base\cmds\app_process\app_main.cpp onStarted() L78
2 virtual void onStarted()
3 {
4     sp<ProcessState> proc = ProcessState::self();
5     ALOGV("App process: starting thread pool.\n");
6     proc->startThreadPool();
7
8     AndroidRuntime* ar = AndroidRuntime::getRuntime();
9     ar->callMain(mClassName, mClass, mArgs);
10
11     IPThreadState::self()->stopProcess();
12     hardware::IPThreadState::self()->stopProcess();
13 }

```

ZygotInit

```

1 // \frameworks\base\core\java\com\android\internal\os\ZygotInit.java main() L750
2 public static void main(String argv[]) {
3     ZygoteServer zygoteServer = new ZygoteServer(); //新建Zygote服务器端4
4
5     ...
6
7     final Runnable caller;
8     try {
9         ...
10        boolean startSystemServer = false;
11        String socketName = "zygote"; Dalvik VM进程系统
12        String abiList = null;
13        boolean enableLazyPreload = false;
14        for (int i = 1; i < argv.length; i++) {
15            //还记得app_main.cpp中传的start-system-server参数吗，在这里总有用 到了
16            if ("start-system-server".equals(argv[i])) {
17                startSystemServer = true;
18            } else if ("--enable-lazy-preload".equals(argv[i])) {
19                enableLazyPreload = true;
20            } else if (argv[i].startsWith(ABI_LIST_ARG)) {
21                abiList = argv[i].substring(ABI_LIST_ARG.length());
22            } else if (argv[i].startsWith(SOCKET_NAME_ARG)) {
23                socketName =
24                argv[i].substring(SOCKET_NAME_ARG.length());

```

```

24         } else {
25             throw new RuntimeException("Unknown command line argument:
" + argv[i]);
26         }
27     }
28
29     if (abiList == null) {
30         throw new RuntimeException("No ABI list supplied.");
31     }
32
33     zygoteServer.registerServerSocketFromEnv(socketName); //注册Socket
34     // In some configurations, we avoid preloading resources and
35     classes eagerly.
36     // In such cases, we will preload things prior to our first
37     fork.
38     // 在有些情况下我们需要在第一个fork之前进行预加载资源
39     if (!enableLazyPreload) {
40
41         preload(bootTimingsTraceLog);
42
43     } else {
44         Zygote.resetNicePriority();
45     }
46
47     // Do an initial gc to clean up after startup
48     bootTimingsTraceLog.traceBegin("PostZygoteInitGC");
49     //主动进行一次资源GC
50     gcAndFinalize();
51     bootTimingsTraceLog.traceEnd(); // PostZygoteInitGC
52
53     bootTimingsTraceLog.traceEnd(); // ZygoteInit
54     // Disable tracing so that forked processes do not inherit stale tracing tags
55     from
56     // Zygote.
57     Trace.setTracingEnabled(false, 0);
58
59     Zygote.nativeSecurityInit();
60
61     // Zygote process unmounts root storagespaces.
62     Zygote.nativeUnmountStorageOnInit();
63
64     ZygoteHooks.stopZygoteNoThreadCreation();
65
66     if (startSystemServer) {
67         Runnable r = forkSystemServer(abiList, socketName,
68         zygoteServer);
69
70         // {@code r == null} in the parent (zygote) process, and
71         {@code r != null} in the

```

```
69         // child (system_server) process. if
70         (r != null) {
71             r.run(); return;
72         }
73     }
74
75     Log.i(TAG, "Accepting command socket connections");
```

博学课业网

```

76         // The select loop returns early in the child process after a
fork and
77         // loops forever in the zygote.
78         caller = zygoteServer.runSelectLoop(abiList);
79     } catch (Throwable ex) {
80         Log.e(TAG, "System zygote died with exception", ex);
81         throw ex;
82     } finally {
83         zygoteServer.closeServerSocket();
84     }
85
86     // We're in the child process and have exited the select loop.
Proceed to execute the
87     // command.
88     if (caller != null)
89         { caller.run();
90     }
91 }

```

preload() 的作用就是提前将需要的资源加载到VM中，比如class、resource等

```

1  // \frameworks\base\core\java\com\android\internal\os\ZygoteInit.java preload() L123
2  static void preload(TimingsTraceLog bootTimingsTraceLog) {
3      Log.d(TAG, "begin preload");
4      bootTimingsTraceLog.traceBegin("BeginIcuCachePinning");
5      beginIcuCachePinning();
6      bootTimingsTraceLog.traceEnd(); // BeginIcuCachePinning
7      bootTimingsTraceLog.traceBegin("PreloadClasses");
8      //加载指定的类到内存并且初始化，使用的Class.forName(class, true, null);方式
9      preloadClasses();
10     bootTimingsTraceLog.traceEnd(); // PreloadClasses
11     bootTimingsTraceLog.traceBegin("PreloadResources");
12     //加载Android通用的资源，比如drawable、color...
13     preloadResources();
14     bootTimingsTraceLog.traceEnd(); // PreloadResources
15     Trace.traceBegin(Trace.TRACE_TAG_DALVIK, "PreloadAppProcessHALs");
16     nativePreloadAppProcessHALs();
17     Trace.traceEnd(Trace.TRACE_TAG_DALVIK);
18     Trace.traceBegin(Trace.TRACE_TAG_DALVIK, "PreloadOpenGL");
19     //加载OpenGL...
20     preloadOpenGL();
21     Trace.traceEnd(Trace.TRACE_TAG_DALVIK);
22     //加载共用的Library
23     preloadSharedLibraries();
24     //加载Text资源，字体等
25     preloadTextResources();
26     // Ask the WebViewFactory to do any initialization that must run in the zygote
process,
27     // for memory sharing purposes.
28     // 为了内存共享，WebViewFactory进行任何初始化都要在Zygote进程中
29     WebViewFactory.prepareWebViewInZygote();
30     endIcuCachePinning();
31     warmUpJcaProviders();
32     Log.d(TAG, "end preload"); 33

```


博学文库

preloadClasses 将framework.jar里的preloaded-classes 定义的所有class load到内存里，

preloaded-classes 编译Android后可以在framework/base下找到。而preloadResources 将系统的Resource(不是在用户apk里定义的resource) load到内存。资源preload到Zygoted的进程地址空间，所有fork的子进程将共享这份空间而无需重新load，这大大减少了应用程序的启动时间，但反过来增加了系统的启动时间。通过对preload 类和资源数目进行调整可以加快系统启动。Preload也是Android启动最耗时的部分之一

```

1 // \frameworks\base\core\java\com\android\internal\os\ZygoteInit.java
  gcAndFinalize() L439
2 static void gcAndFinalize() {
3     final VMRuntime runtime = VMRuntime.getRuntime();
4
5     /* runFinalizationSync() lets finalizers be called in Zygote,
6      * which doesn't have a HeapWorker thread.
7      */
8     System.gc();
9     runtime.runFinalizationSync();
10    System.gc();
11 }

```

gc()调用只是通知VM进行垃圾回收，是否回收，什么时候回收全由VM内部算法决定。GC的回收有一个复杂的状态机控制，通过多次调用，可以使得尽可能多的资源得到回收。gc()必须在fork之前完成（接下来的StartSystemServer就会有fork操作），这样将来被复制出来的子进程才能有尽可能少的垃圾内存没有释放

```

1 // \frameworks\base\core\java\com\android\internal\os\ZygoteInit.java gcAndFinalize()
  L657
2 private static Runnable forkSystemServer(String abilist, String socketName,
3     ZygoteServer zygoteServer) {
4     long capabilities = posixCapabilitiesAsBits(
5         OsConstants.CAP_IPC_LOCK,
6         OsConstants.CAP_KILL,
7         OsConstants.CAP_NET_ADMIN,
8         OsConstants.CAP_NET_BIND_SERVICE,
9         OsConstants.CAP_NET_BROADCAST,
10        OsConstants.CAP_NET_RAW,
11        OsConstants.CAP_SYS_MODULE,
12        OsConstants.CAP_SYS_NICE,
13        OsConstants.CAP_SYS_PTRACE,
14        OsConstants.CAP_SYS_TIME,
15        OsConstants.CAP_SYS_TTY_CONFIG,
16        OsConstants.CAP_WAKE_ALARM,
17        OsConstants.CAP_BLOCK_SUSPEND 18    );
18
19     /* Containers run without some capabilities, so drop any caps that are not
20      * available. */
21     StructCapUserHeader header = new StructCapUserHeader(
22         OsConstants._LINUX_CAPABILITY_VERSION_3, 0);
23     StructCapUserData[] data;
24     try {
25         data = Os.capget(header);
26     } catch (ErrnoException ex) {
27         throw new RuntimeException("Failed to capget()", ex);
28     }
29 }

```

```

27     }
28     capabilities &= ((long) data[0].effective) | (((long)
data[1].effective) << 32);
29
30     /* Hardcoded command line to start the system server */ 31
32     //启动SystemServer的命令行，部分参数写死
33     String args[] = {
34         "--setuid=1000",
35         "--setgid=1000",
36         "--
setgroups=1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1018,1021,1023,1
024,1032,1065,3001,3002,3003,3006,3007,3009,3010",
37         "--capabilities=" + capabilities + "," + capabilities,
38         "--nice-name=system_server",
39         "--runtime-args",
40         "--target-sdk-version=" + VMRuntime.SDK_VERSION_CUR_DEVELOPMENT,
41         "com.android.server.SystemServer", 42     };
43     ZygoteConnection.Arguments parsedArgs =null; 44
45     int pid;
46
47     try {
48         parsedArgs = new ZygoteConnection.Arguments(args);
49         ZygoteConnection.applyDebuggerSystemProperty(parsedArgs);
50         ZygoteConnection.applyInvokeWithSystemProperty(parsedArgs); //会设 置InvokeWith
参数，这个参数在接下来的初始化逻辑中会有调用
51
52         boolean profileSystemServer = SystemProperties.getBoolean(
53             "dalvik.vm.profilesystemserver", false);
54         if (profileSystemServer) {
55             parsedArgs.runtimeFlags |= Zygote.PROFILE_SYSTEM_SERVER; 56         }
57
58         /* Request to fork the system server process */
59         /* 创建 system server 进程 */
60         pid = Zygote.forkSystemServer(
61             parsedArgs.uid, parsedArgs.gid,
62             parsedArgs.gids,
63             parsedArgs.runtimeFlags,
64             null,
65             parsedArgs.permittedCapabilities,
66             parsedArgs.effectiveCapabilities);
67     } catch (IllegalArgumentException ex) {
68         throw new RuntimeException(ex); 69     }
69
70
71     /* For child process */
72     if (pid == 0) { //如果是第一次创建的话pid==0
73         if (hasSecondZygote(abiList)) {
74             waitForSecondaryZygote(socketName); 75         }
76
77         zygoteServer.closeServerSocket();
78         return handleSystemServerProcess(parsedArgs); 79 }

```

博学课业网

```
81         return null;
82     }
```

ZygoteInit.forkSystemServer() 方法fork 出一个新的进程，这个进程就是SystemServer进程。fork出来的子进程在handleSystemServerProcess 里开始初始化工作，主要工作分为：

1. prepareSystemServerProfile () 方法中将SYSTEMSERVERCLASSPATH中的AppInfo加载到VM中。
2. 判断fork args中是否有invokWith参数，如果有则进行WrapperInit.execApplication（不进行深入讲解了）。如果没有则调用

```
1  // \frameworks\base\core\java\com\android\internal\os\ZygoteInit.java
   handleSystemServerProcess() L453
2  private static Runnable handleSystemServerProcess(ZygoteConnection.Arguments
   parsedArgs) {
3      ...
4      if (profileSystemServer && (Build.IS_USERDEBUG || Build.IS_ENG))
   {
5          try {
6              //将SYSTEMSERVERCLASSPATH中的AppInfo加载到VM中
7              prepareSystemServerProfile(systemServerClasspath);
8          } catch (Exception e) {
9              Log.wtf(TAG, "Failed to set up system server profile",
   e);
10         }
11     }
12 }
13
14 if (parsedArgs.invokewith != null) {
15     ...
16     //判断fork args中是否有invokwith参数，如果有则进行
   wrapperInit.execApplication
17     wrapperInit.execApplication(parsedArgs.invokewith,
18         parsedArgs.niceName, parsedArgs.targetSdkVersion,
19         VMRuntime.getCurrentInstructionSet(), null, args);
20
21     throw new IllegalStateException("Unexpected return from
   wrapperInit.execApplication");
22 } else {
23     ...
24     /*
25     * Pass the remaining arguments to SystemServer.
26     */
27     //调用zygoteInit
28     return ZygoteInit.zygoteInit(parsedArgs.targetSdkVersion,
   parsedArgs.remainingArgs, cl);
29 }
30
31
32 /* should never reach here */
33 }
```

```

1 // \frameworks\base\core\java\com\android\internal\os\RuntimeInit.java
  applicationInit() L345
2     protected static Runnable applicationInit(int targetSdkVersion, String[]
  argv,
3         ClassLoader classLoader) {
4         ...
5
6         // Remaining arguments are passed to the start class's static main
7         //findStaticMain来运行args的startClass的main方法
8         return findStaticMain(args.startClass, args.startArgs, classLoader);
9     }

```

```

1 // \frameworks\base\core\java\com\android\internal\os\RuntimeInit.java
  findStaticMain() L287
2     protected static Runnable findStaticMain(String className, String[] argv,
3         ClassLoader classLoader) {
4         Class<?> cl;
5
6         try {
7             cl = Class.forName(className, true, classLoader);
8         } catch (ClassNotFoundException ex)
9             { throw new RuntimeException(
10                "Missing class when invoking static main " + className,
11                ex);
12         }
13
14         Method m;
15         try {
16             m = cl.getMethod("main", new Class[] { String[].class });
17         } catch (NoSuchMethodException ex)
18             { throw new RuntimeException(
19                "Missing static main on " + className, ex);
20         } catch (SecurityException ex)
21             { throw new
22                RuntimeException(
23                "Problem getting static main on " + className, ex);
24         }
25
26     }

```

很明显这是一个耗时操作所以使用线程来完成:

```

1 // \frameworks\base\core\java\com\android\internal\os\RuntimeInit.java
  MethodAndArgsCaller L479
2     static class MethodAndArgsCaller implements Runnable {
3         /** method to call */
4         private final Method mMethod;
5
6         /** argument array */
7         private final String[] mArgs;
8
9         public MethodAndArgsCaller(Method method, String[] args)
10             { mMethod = method;
11               mArgs = args;
12         }
13

```

```

14         public void run()
15         { try {
16             mMethod.invoke(null, new Object[] { mArgs });
17         } catch (IllegalAccessException ex)
18             { throw new
19               RuntimeException(ex);
20         } catch (InvocationTargetException ex)
21             { Throwable cause = ex.getCause();
22               if (cause instanceof RuntimeException)
23                 { throw (RuntimeException) cause;
24               } else if (cause instanceof Error)
25                 { throw (Error) cause;
26               }
27             throw new RuntimeException(ex);
28         }
29     }

```

3.4 System Server 启动流程

System Server 是Zygote fork 的第一个Java 进程， 这个进程非常重要，因为他们有很多的系统线程， 提供所有核心的系统服务

看到大名鼎鼎的WindowManager， ActivityManager了吗？对了，它们都是运行在system_server的进程里。还有很多“Binder-x”的线程，它们是各个Service为了响应应用程序远程调用请求而创建的。除此之外，还有很多内部的线程，比如 ” UI thread”， “InputReader”， “InputDispatch” 等等，我，现在在我们只关心System Server是如何创建起来的。

SystemServer的main() 函数。

```

1 public static void main(String[] args) {
2     new SystemServer().run();
3 }

```

记下来我分成4部分详细分析SystemServer run方法的初始化流程：

初始化必要的SystemServer环境参数，比如系统时间、默认时区、语言、load一些Library等等，初始化Looper，我们在主线程中使用到的looper就是在SystemServer中进行初始化的

初始化Context，只有初始化一个Context才能进行启动Service等操作，这里看一下源码：

```

1 private void createSystemContext() {
2     ActivityThread activityThread = ActivityThread.systemMain();
3     mSystemContext = activityThread.getSystemContext();
4     mSystemContext.setTheme(DEFAULT_SYSTEM_THEME);
5     final Context systemUiContext = activityThread.getSystemUiContext();
6     systemUiContext.setTheme(DEFAULT_SYSTEM_THEME);
7 }

```

看到没有ActivityThread就是这个时候生成的

继续看ActivityThread中如何生成Context：

```

1 public ContextImpl getSystemContext() {
2     synchronized (this) {
3         if (mSystemContext == null) {
4             mSystemContext = ContextImpl.createSystemContext(this);
5         }
6         return mSystemContext;
7     }
8 }

```

ContextImpl是Context类的具体实现，里面封装完成了生成几种常用的createContext的方法：

```

1 static ContextImpl createSystemContext(ActivityThread mainThread)
2 { LoadedApk packageInfo = new LoadedApk(mainThread);
3     //省略代码
4     return context;
5 }
6
7 static ContextImpl createSystemUiContext(ContextImpl systemContext)
8 { final LoadedApk packageInfo = systemContext.mPackageInfo;
9     //省略代码
10    return context;
11 }
12
13 static ContextImpl createAppContext(ActivityThread mainThread, LoadedApk
14 packageInfo) {
15     if (packageInfo == null) throw new
16     IllegalArgumentException("packageInfo");
17     //省略代码
18     return context;
19 }
20
21 static ContextImpl createActivityContext(ActivityThread mainThread,
22 LoadedApk packageInfo, ActivityInfo activityInfo, IBinder
23 activityToken, int displayId,
24 Configuration overrideConfiguration) {
25     //省略代码
26     return context;
27 }

```

初始化SystemServiceManager,用来管理启动service, SystemServiceManager中封装了启动Service的startService方法启动系统必要的Service, 启动service的流程又分成三步走：

```

1 // Start services.
2 try {
3     traceBeginAndSlog("StartServices");
4     startBootstrapServices();
5     startCoreServices();
6     startOtherServices();
7     SystemServerInitThreadPool.shutdown();
8 } catch (Throwable ex)
9 {
10 } finally {
11     traceEnd();
12 }

```


启动BootstrapServices,就是系统必须需要的服务,这些服务直接耦合性很高,所以干脆就放在一个方法里面一起启动,比如PowerManagerService、RecoverySystemService、DisplayManagerService、ActivityManagerService等等启动以基本的核心Service,很简单,只有三个BatteryService、UsageStatsService、WebViewUpdateService启动其它需要用到的Service,比如NetworkScoreService、AlarmManagerService

5. 善后工作是不是到此之后,Zygote的工作变得很轻松了,可以宜养天年了?可惜现代社会,哪个父母把孩子养大就可以撒手不管了?尤其是像System Server 这样肩负社会重任的大儿子,出问题了父母还是要帮一把的。这里,Zygote会默默的在后台凝视这自己的大儿子,一旦发现System Server 挂掉了,将其回收,然后将自己杀掉,重新开始新的一生, 可怜天下父母心啊。这段实现在代码 : dalvik/vm/native/dalvik_system_zygote.cpp 中,

```
1 static void Dalvik_dalvik_system_Zygote_forkSystemServer(  
2     const u4* args, JValue* pResult){ 3     ...  
4     pid_t pid;  
5     pid = forkAndSpecializeCommon(args, true); 6     ...  
7     if (pid > 0) {  
8         int status;  
9         gDvm.systemServerPid = pid;  
10        /* WNOHANG 会让waitpid 立即返回, 这里只是为了预防上面的赋值语句没有完成之前  
SystemServer就crash 了*/  
11        if (waitpid(pid, &status, WNOHANG) == pid) {  
12            ALOGE("System server process %d has died. Restarting Zygote!",  
pid);  
13            kill(getpid(), SIGKILL); 14  
            }  
15        }  
16        RETURN_INT(pid); 17  
    }  
18  
19    /* 真正的处理在这里 */  
20    static void sigchldHandler(int s){ 21  
    ...  
22        pid_t pid;  
23        int status;  
24        ...  
25        while ((pid = waitpid(-1, &status, WNOHANG)) > 0) { 26  
            ...  
27            if (pid == gDvm.systemServerPid) { 28  
                ...  
29                kill(getpid(), SIGKILL); 30  
            }  
31        }  
32        ...  
33    }  
34  
35    static void Dalvik_dalvik_system_Zygote_fork(const u4* args, JValue* pResult){  
36        pid_t pid;  
37        ...  
38        setSignalHandler(); //signalHandler 在这里注册  
39        ...  
40        pid = fork();  
41        ...  
42        RETURN_INT(pid);
```

在Unix-like系统，父进程必须用 `waitpid` 等待子进程的退出，否则子进程将变成”Zombie”（僵尸）进

程，不仅系统资源泄漏，而且系统将崩溃（没有system server，所有Android应用程序都无法运行）。但是`waitpid()` 是一个阻塞函数（WNOHANG参数除外），所以通常做法是在signal 处理函数里进行无阻塞的处理，因为每个子进程退出的时候，系统会发出 SIGCHID 信号。Zygote会把自己杀掉，那父亲死了，所有的应用程序不就成为孤儿了？不会，因为父进程被杀掉后系统会自动给所有的子进程发生SIGHUP信号，该信号的默认处理就是将杀掉自己退出当前进程。但是一些后台进程（Daemon）可以通过设置SIG_IGN参数来忽略这个信号，从而得以在后台继续运行。

总结

1. init 根据init.rc 运行 app_process，并携带 ‘- zygote’ 和 ‘- startSystemServer’ 参数。
2. `AndroidRuntime.cpp::start()` 里将启动JavaVM，并且注册所有framework相关的系统JNI接口。
3. 第一次进入Java世界，运行`ZygoteInit.java::main()` 函数初始化Zygote. Zygote 并创建Socket的server 端。
4. 然后fork一个新的进程并在新进程里初始化SystemServer. Fork之前，Zygote是preload常用的Java类库，以及系统的resources，同时GC() 清理内存空间，为子进程省去重复的工作。
5. SystemServer 里将所有的系统Service初始化，包括ActivityManager 和 WindowManager，他们是应用程序运行起来的前提。
6. 依次同时，Zygote监听服务端Socket，等待新的应用启动请求。
7. ActivityManager ready 之后寻找系统的“Startup” Application，将请求发给Zygote。
8. Zygote收到请求后，fork出一个新的进程。
9. Zygote监听并处理SystemServer 的 SIGCHID 信号，一旦System Server崩溃，立即将自己杀死。init会重启Zygote。

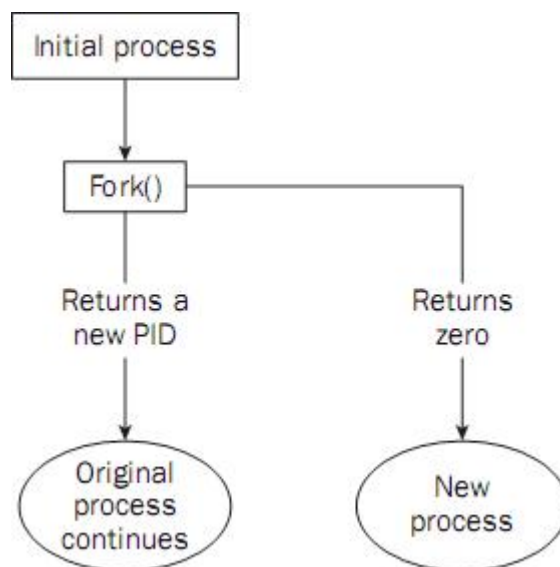
3.5 什么情况下Zygote进程会重启呢？

- servicemanager 进程被杀；
- (onresart)surfaceflinger 进程被杀；
- (onresart)Zygote 进程自己被杀；
- (oneshot=false)system_server进程被杀；(waitpid)

3.6 fork函数

```
1 | pid_t fork(void)
```

1. 参数：不需要参数
2. 需要的头文件<sys/types.h> 和 <unistd.h>
3. 返回值分两种情况：
 - 返回0表示成功创建子进程，并且接下来进入子进程执行流程
 - 返回PID (>0)，成功创建子进程，并且继续执行父进程流程代码返回非正数 (<0)，创建子进程失败，失败原因主要有：
 - 进程数超过系统所能创建的上限，errno会被设置为EAGAIN系统内存不足，errno会被设置为ENOMEM



使用 `fork()` 函数得到的子进程是父进程的一个复制品，它从父进程处继承了整个进程的地址空间：包括进程上下文（进程执行活动全过程的静态描述）、进程堆栈、打开的文件描述符、信号控制设定、进程优先级、进程组号等。子进程所独有的只有它的进程号，计时器等（只有少量信息）。因此，使用 `fork()` 函数的代价是很大的

3.7 子进程与父进程的区别

1. 除了文件锁以外, 其他的锁都会被继承
2. 各自的进程ID和父进程ID不同
3. 子进程的未决告警被清除;
4. 子进程的未决信号集设置为空集。

3.8 写时拷贝 (copy-on-write)

Linux 的 `fork()` 使用是通过写时拷贝 (copy-on-write) 实现。写时拷贝是一种可以推迟甚至避免拷贝数据的技术。内核此时并不复制整个进程的地址空间，而是让父子进程共享同一个地址空间。只用在需要写入的时候才会复制地址空间，从而使各个进程拥有各自的地址空间。也就是说，资源的复制是在需要写入的时候才会进行，在此之前，只有以只读方式共享

3.9 孤儿进程、僵尸进程

`fork` 系统调用之后，父子进程将交替执行，执行顺序不定。如果父进程先退出，子进程还没退出那么子进程的父进程将变为 `init` 进程（托孤给了 `init` 进程）。（注：任何一个进程都必须有父进程）如果子进程先退出，父进程还没退出，那么子进程必须等到父进程捕获到了子进程的退出状态才真正结束，否则这个时候子进程就成为僵进程（**僵尸进程**：只保留一些退出信息供父进程查询）

3.10 多线程进程的Fork调用

在 POSIX 标准中，`fork` 的行为是这样的：复制整个用户空间的数据（通常使用 `copy-on-write` 的策略，所以可以实现的速度很快）以及所有系统对象，然后仅复制当前线程到子进程。这里：所有父进程中别的线程，到了子进程中都是突然蒸发掉的

假设这么一个环境，在 `fork` 之前，有一个子线程 `lock` 了某个锁，获得了对锁的所有权。`fork` 以后，在子进程中，所有的额外线程都人间蒸发了。而锁却被正常复制了，在子进程看来，这个锁没有主人，所以没有任何人可以对它解锁。当子进程想 `lock` 这个锁时，不再有任何手段可以解开了。程序发生死锁

第四节 面试题

4.1 你了解 Android 系统启动流程吗？

A: 当按电源键触发开机，首先会从 ROM 中预定义的地方加载引导程序 BootLoader 到 RAM 中，并执行 BootLoader 程序启动 Linux Kernel，然后启动用户级别的第一个进程：init 进程。init 进程会解析 init.rc 脚本做一些初始化工作，包括挂载文件系统、创建工作目录以及启动系统服务进程等，其中系统服务进程包括 Zygote、service manager、media 等。在 Zygote 中会进一步去启动 system_server 进程，然后在 system_server 进程中会启动 AMS、WMS、PMS 等服务，等这些服务启动之后，AMS 中就会打开 Launcher 应用的 home Activity，最终就看到了手机的“桌面”。

4.2 system_server 为什么要在 Zygote 中启动，而不是由 init 直接启动呢？

A: Zygote 作为一个孵化器，可以提前加载一些资源，这样 fork() 时基于 Copy-On-Write 机制创建的其他进程就能直接使用这些资源，而不用重新加载。比如 system_server 就可以直接使用 Zygote 中的 JNI 函数、共享库、常用的类、以及主题资源。

4.3 为什么要专门使用 Zygote 进程去孵化应用进程，而不是让 system_server 去孵化呢？

A: 首先 system_server 相比 Zygote 多运行了 AMS、WMS 等服务，这些对一个应用程序来说是不需要的。另外进程的 fork() 对多线程不友好，仅会将发起调用的线程拷贝到子进程，这可能会导致死锁，而 system_server 中肯定是有许多线程的。

4.4 能说说具体是怎么导致死锁的吗？

在 POSIX 标准中，fork 的行为是这样的：复制整个用户空间的数据（通常使用 copy-on-write 的策略，所以可以实现的速度很快）以及所有系统对象，然后仅复制当前线程到子进程。这里：所有父进程中别的线程，到了子进程中都是突然蒸发掉的

对于锁来说，从 OS 看，每个锁有一个所有者，即最后一次 lock 它的线程。假设这么一个环境，在 fork 之前，有一个子线程 lock 了某个锁，获得了对锁的所有权。fork 以后，在子进程中，所有的额外线程都人间蒸发了。而锁却被正常复制了，在子进程看来，这个锁没有主人，所以没有任何人可以对它解锁。当子进程想 lock 这个锁时，不再有任何手段可以解开了。程序发生死锁

4.5 Zygote 为什么不采用 Binder 机制进行 IPC 通信？

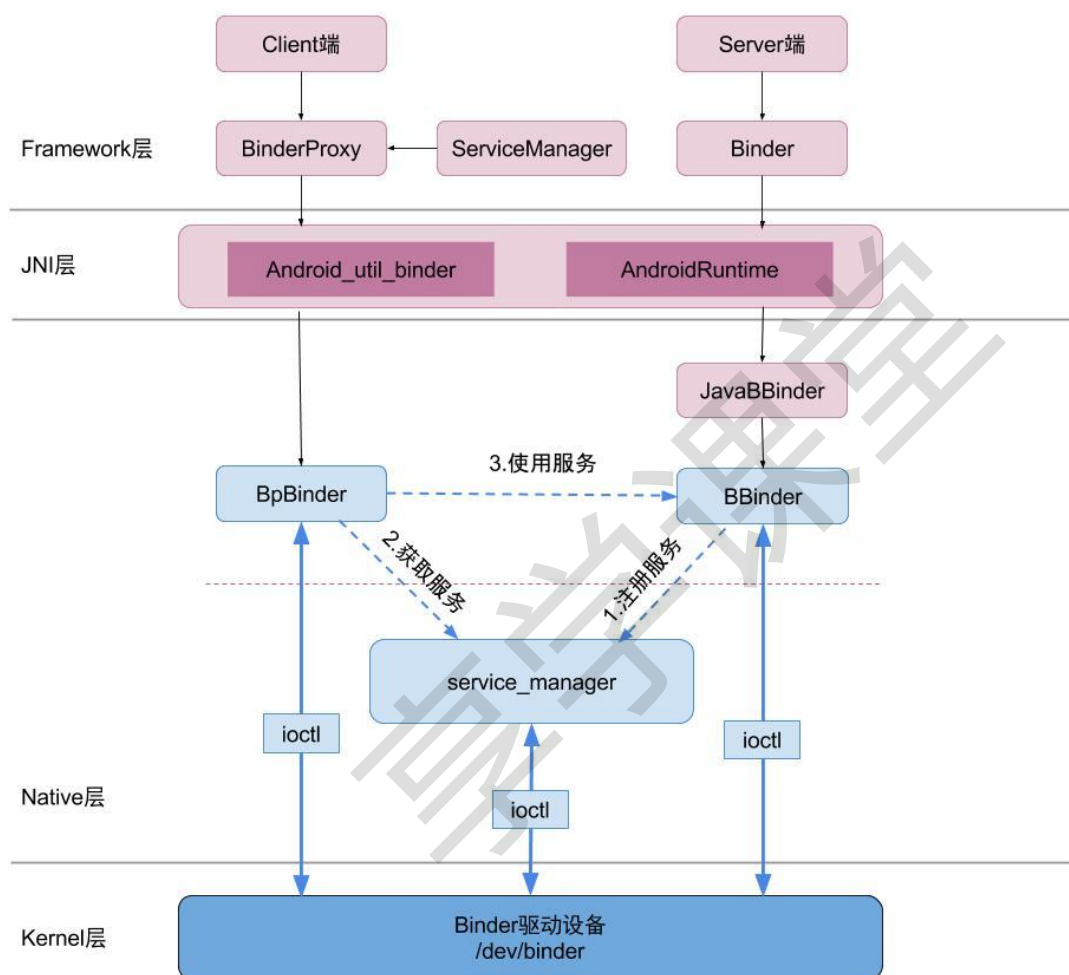
A: Binder 机制中存在 Binder 线程池，是多线程的，如果 Zygote 采用 Binder 的话就存在上面说的 fork() 与多线程的问题了。其实严格来说，Binder 机制不一定要多线程，所谓的 Binder 线程只不过是循环读取 Binder 驱动的消息而已，只注册一个 Binder 线程也是可以工作的，比如 service manager 就是这样的。实际上 Zygote 尽管没有采用 Binder 机制，它也不是单线程的，但它在 fork() 前主动停止了其他线程，fork() 后重新启动了。

附：本内容由享学课堂移动开发技术团队独家提供
<https://ke.qq.com/course/341933?tuin=9f978cfe&taid=9303882710988717>，未经享学课堂授权不得转载、摘编或利用其它方式使用上述作品。已经享学课堂授权使用作品的，应在授权范围内使用，并注明“来源：享学课堂”。违反上述声明者，将追究其相关法律责任。

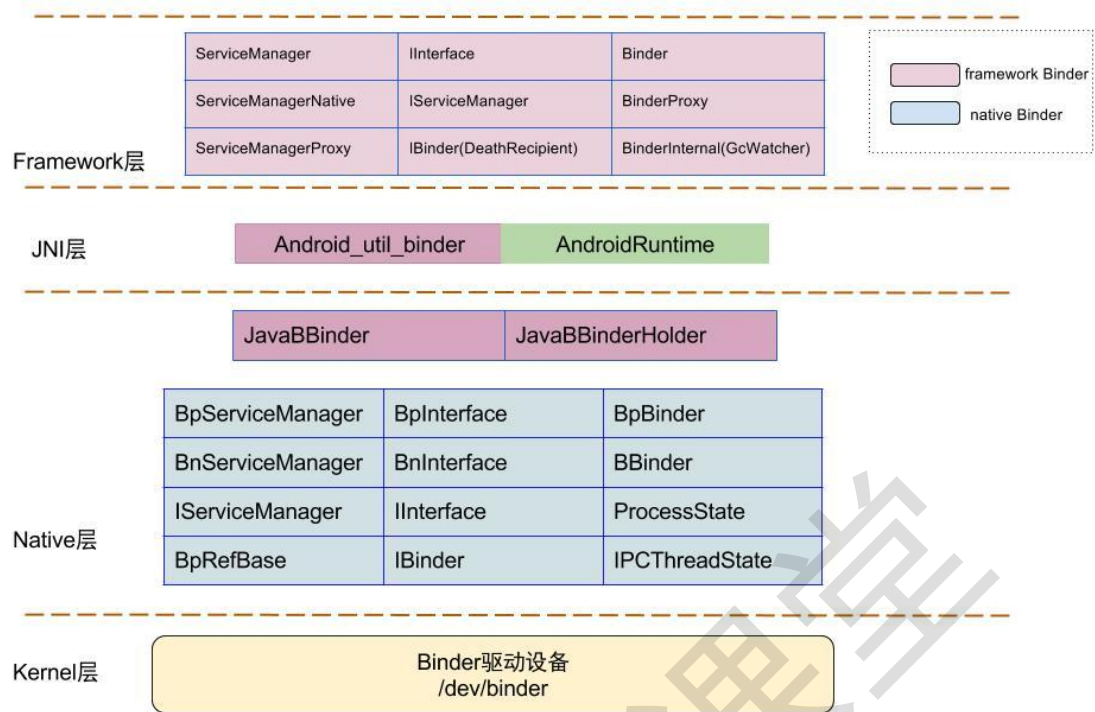
第二章 Binder解析

第一节 宏观认识Binder

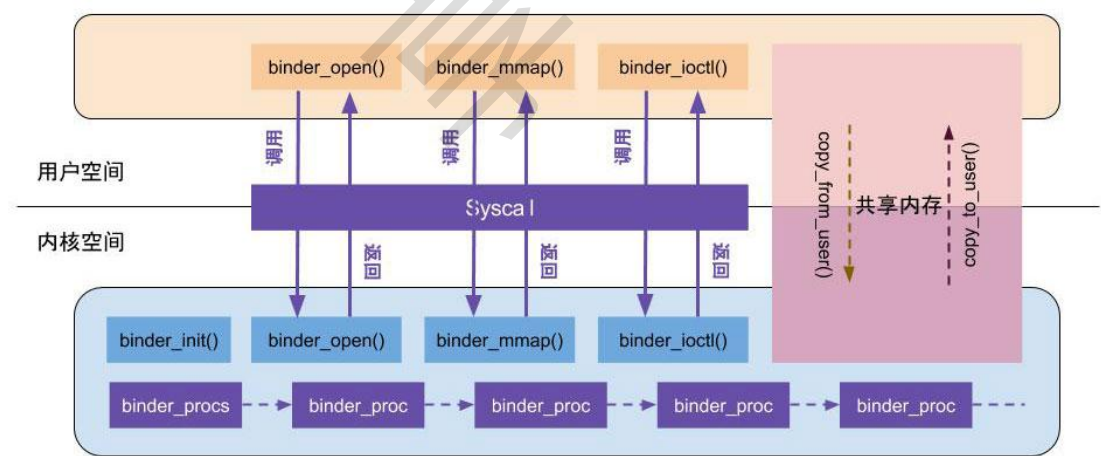
1.1 binder框架



1.2 binder涉及到的类



1.3 binder驱动



第二节binder的jni方法注册

2.1 zygote启动

2.1.1 启动zygote进程

zygote是由init进程通过解析init.zygote.rc文件而创建的，zygote所对应的可执行程序`appprocess`，所对应的源文件是`appmain.cpp`，进程名为zygote。
// system/core/rootdir/init.zygote32.rc

```

service zygote /system/bin/app_process -Xzygote /system/bin --zygote --start-system-server
    class main
    socket zygote stream 660 root system
    onrestart write /sys/android_power/request_state wake
    onrestart write /sys/power/state on
    onrestart restart media
    onrestart restart netd
    writepid /dev/cpuset/foreground/tasks

```

2.1.2 执行app_main.cpp中的main方法

启动zygote的入口函数是app_main.cpp中的main方法。
frameworks/base/cmds/app_process/app_main.cpp

```

// 186
int main(int argc, char* const argv[])

// 248 将zygote标志位置为true。
if (strcmp(arg, "--zygote") == 0) {
    zygote = true;
}

// 306 运行AndroidRuntime.cpp的start方法
if (zygote) {
    runtime.start("com.android.internal.os.ZygoteInit", args, zygote);
}

```

2.1.3 AndroidRuntime::start

调用startReg方法来完成jni方法的注册。
frameworks/base/core/jni/AndroidRuntime.cpp

```

// 1007
void AndroidRuntime::start(const char* className, const Vector<String8>& options, bool zygote)

// 1051
if (startReg(env) < 0) {
frameworks/base/core/jni/AndroidRuntime.cpp

1440 int AndroidRuntime::startReg(JNIEnv* env)

// 1459 注册jni方法
if (register_jni_procs(gRegJNI, NELEM(gRegJNI), env) < 0) {
frameworks/base/core/jni/AndroidRuntime.cpp

// 1283
static int register_jni_procs(const RegJNIRec array[], size_t count, JNIEnv* env)
{
    // 循环注册jni方法
    for (size_t i = 0; i < count; i++) {
        if (array[i].mProc(env) < 0) {
            return -1;
        }
    }
    return 0;
}
frameworks/base/core/jni/AndroidRuntime.cpp

// 1296
static const RegJNIRec gRegJNI[] = {

    // 1312
    REG_JNI(register_android_os_Binder),

}

```

2.2 registerandroidos_Binder

frameworks/base/core/jni/android_util_Binder.cpp

```
// 1282
int register_android_os_Binder(JNIEnv* env)
{
    if (int_register_android_os_Binder(env) < 0)
        return -1;
    if (int_register_android_os_BinderInternal(env) < 0)
        return -1;
    if (int_register_android_os_BinderProxy(env) < 0)
        return -1;
}
```

2.2.1 intregisterandroidosBinder

frameworks/base/core/jni/android_util_Binder.cpp

```
// 843
static const JNINativeMethod gBinderMethods[] = {
    /* name, signature, funcPtr */
    { "getCallingPid", "()I", (void*)android_os_Binder_getCallingPid },
    { "getCallingUid", "()I", (void*)android_os_Binder_getCallingUid },
    { "clearCallingIdentity", "()J", (void*)android_os_Binder_clearCallingIdentity },
    { "restoreCallingIdentity", "(J)V", (void*)android_os_Binder_restoreCallingIdentity },
    { "setThreadStrictModePolicy", "(I)V", (void*)android_os_Binder_setThreadStrictModePolicy },
    { "getThreadStrictModePolicy", "()I", (void*)android_os_Binder_getThreadStrictModePolicy },
    { "flushPendingCommands", "()V", (void*)android_os_Binder_flushPendingCommands },
    { "init", "()V", (void*)android_os_Binder_init },
    { "destroy", "()V", (void*)android_os_Binder_destroy },
    { "blockUntilThreadAvailable", "()V", (void*)android_os_Binder_blockUntilThreadAvailable }
};

// 857
const char* const kBinderPathName = "android/os/Binder";

// 859
static int int_register_android_os_Binder(JNIEnv* env)
{
    // 查找文件 kBinderPathName = "android/os/Binder", 返回对应Class对象
    jclass clazz = FindClassOrDie(env, kBinderPathName);

    // 通过gBinderOffsets结构体, 保存Java层Binder类的信息, 为JNI层访问Java层提供通道
    gBinderOffsets.mClass = MakeGlobalRefOrDie(env, clazz);
    gBinderOffsets.mExecTransact = GetMethodIDOrDie(env, clazz, "execTransact", "(IJJI)Z");
    gBinderOffsets.mObject = GetFieldIDOrDie(env, clazz, "mObject", "J");

    // 通过RegisterMethodsOrDie, 将为gBinderMethods数组完成映射关系, 从而为Java层访问JNI层提供通道
    return RegisterMethodsOrDie(
        env, kBinderPathName,
        gBinderMethods, NELEM(gBinderMethods));
}
```

2.2.2 intregisterandroidosBinderInternal

frameworks/base/core/jni/android_util_Binder.cpp

```
// 925
static const JNINativeMethod gBinderInternalMethods[] = {
    /* name, signature, funcPtr */
    { "getContextObject", "()Landroid/os/IBinder;", (void*)android_os_BinderInternal_getContextObject },
    { "joinThreadPool", "()V", (void*)android_os_BinderInternal_joinThreadPool },
    { "disableBackgroundScheduling", "(Z)V", (void*)android_os_BinderInternal_disableBackgroundScheduling },
    { "handleGc", "()V", (void*)android_os_BinderInternal_handleGc }
};
```



```
// 933
const char* const kBinderInternalPathName = "com/android/internal/os/BinderInternal";

// 935
static int int_register_android_os_BinderInternal(JNIEnv* env)
{
    // 查找文件kBinderInternalPathName = "com/android/internal/os/BinderInternal", 返回Class对象
    jclass clazz = FindClassOrDie(env, kBinderInternalPathName);

    // 通过gBinderInternalOffsets, 保存Java层BinderInternal类的信息, 为JNI层访问java层提供通道
    gBinderInternalOffsets.mClass = MakeGlobalRefOrDie(env, clazz);
    gBinderInternalOffsets.mForceGc = GetStaticMethodIDOrDie(env, clazz, "forceBinderGc", "()V");

    // 通过RegisterMethodsOrDie(), 将为gBinderInternalMethods数组完成映射关系, 从而为Java层访问JNI层提供通道
    return RegisterMethodsOrDie(
        env, kBinderInternalPathName,
        gBinderInternalMethods, NELEM(gBinderInternalMethods));
}
```

2.2.3 intregisterandroidosBinderProxy

frameworks/base/core/jni/android_util_Binder.cpp

```
// 1241
static const JNINativeMethod gBinderProxyMethods[] = {
    /* name, signature, funcPtr */
    {"pingBinder",      "()Z", (void*)android_os_BinderProxy_pingBinder},
    {"isBinderAlive",   "()Z", (void*)android_os_BinderProxy_isBinderAlive},
    {"getInterfaceDescriptor", "()Ljava/lang/String;", (void*)android_os_BinderProxy_getInterfaceDescriptor},
    {"transactNative",   "(ILandroid/os/Parcel;Landroid/os/Parcel;I)Z", (void*)android_os_BinderProxy_transact},
    {"linkToDeath",     "(Landroid/os/IBinder$DeathRecipient;)V", (void*)android_os_BinderProxy_linkToDeath},
    {"unlinkToDeath",   "(Landroid/os/IBinder$DeathRecipient;)Z", (void*)android_os_BinderProxy_unlinkToDeath},
    {"destroy",         "()V", (void*)android_os_BinderProxy_destroy},
};

// 1252
const char* const kBinderProxyPathName = "android/os/BinderProxy";

// 1254
static int int_register_android_os_BinderProxy(JNIEnv* env)
{
    // 查找文件 kBinderProxyPathName = "android/os/BinderProxy", 返回对应Class对象
    jclass clazz = FindClassOrDie(env, "java/lang/Error");
    gErrorOffsets.mClass = MakeGlobalRefOrDie(env, clazz);

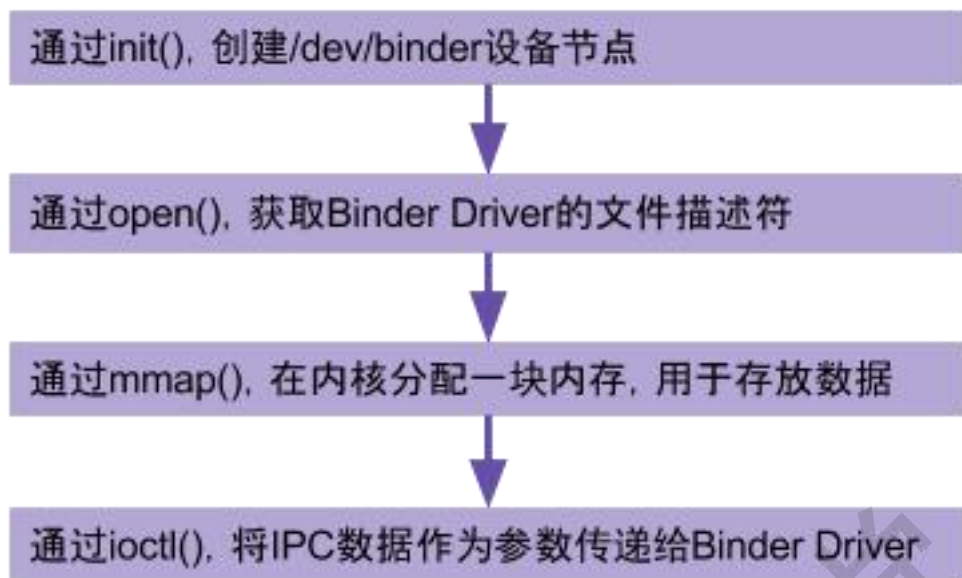
    // 通过gBinderProxyOffsets, 保存Java层BinderProxy类的信息, 为JNI层访问Java提供通道
    clazz = FindClassOrDie(env, kBinderProxyPathName);
    gBinderProxyOffsets.mClass = MakeGlobalRefOrDie(env, clazz);
    gBinderProxyOffsets.mConstructor = GetMethodIDOrDie(env, clazz, "<init>", "()V");
    gBinderProxyOffsets.mSendDeathNotice = GetStaticMethodIDOrDie(env, clazz, "sendDeathNotice",
        "(Landroid/os/IBinder$DeathRecipient;)V");

    gBinderProxyOffsets.mObject = GetFieldIDOrDie(env, clazz, "mObject", "J");
    gBinderProxyOffsets.mSelf = GetFieldIDOrDie(env, clazz, "mSelf",
        "Ljava/lang/ref/WeakReference;");
    gBinderProxyOffsets.mOrgue = GetFieldIDOrDie(env, clazz, "mOrgue", "J");

    clazz = FindClassOrDie(env, "java/lang/Class");
    gClassOffsets.mGetName = GetMethodIDOrDie(env, clazz, "getName", "()Ljava/lang/String;");

    // 通过RegisterMethodsOrDie(), 将为gBinderProxyMethods数组完成映射关系, 从而为Java层访问JNI层提供通道
    return RegisterMethodsOrDie(
        env, kBinderProxyPathName,
        gBinderProxyMethods, NELEM(gBinderProxyMethods));
}
```

第三节binder驱动



3.1 binder_init

kernel/drivers/staging/android/binder.c

```
// 4290 设备驱动入口函数
device_initcall(binder_init);
```

```
// 4213
static int __init binder_init(void)
```

```
// 4220 创建名为binder的单线程的工作队列
binder_deferred_workqueue = create_singlethread_workqueue("binder");
```

```
// 4269
ret = init_binder_device(device_name);
kernel/drivers/staging/android/binder.c
```

```
// 4186
static int __init init_binder_device(const char *name)
{
```

```
    int ret;
    struct binder_device *binder_device;
```

```
// 4191 为binder设备分配内存
binder_device = kzalloc(sizeof(*binder_device), GFP_KERNEL);
```

```
// 4195 初始化设备
binder_device->miscdev.fops = &binder_fops; // 设备的文件操作结构, 这是file_operations结构
binder_device->miscdev.minor = MISC_DYNAMIC_MINOR; // 次设备号 动态分配
binder_device->miscdev.name = name; // 设备名, "binder"
```

```
binder_device->context.binder_context_mgr_uid = INVALID_UID;
binder_device->context.name = name;
```

```
// 4202 misc驱动注册
ret = misc_register(&binder_device->miscdev);
```

```
// 4208 将hlist节点添加到binder_devices为表头的设备链表
```

```

hlist_add_head(&binder_device->hlist, &binder_devices);

return ret;
}

```

3.2 binder_open

kernel/drivers/staging/android/binder.c

```

// 3454
static int binder_open(struct inode *nodp, struct file *filp)

// 3462 为binder_proc结构体在kernel分配内存空间
proc = kzalloc(sizeof(*proc), GFP_KERNEL);

// 3465 将当前线程的task保存到binder进程的tsk
get_task_struct(current);
proc->tsk = current;
INIT_LIST_HEAD(&proc->todo); // 初始化todo列表
init_waitqueue_head(&proc->wait); // 初始化wait队列
proc->default_priority = task_nice(current); // 将当前进程的nice值转换为进程优先级

// 3474 同步锁，因为binder支持多线程访问
binder_lock(__func__);

binder_stats_created(BINDER_STAT_PROC); // binder_proc对象创建数加1
hlist_add_head(&proc->proc_node, &binder_procs); // 将proc_node节点添加到binder_procs的队列头部
proc->pid = current->group_leader->pid; // 进程pid
INIT_LIST_HEAD(&proc->delivered_death); // 初始化已分发的死亡通知列表
filp->private_data = proc; // 将这个binder_proc与filp关联起来，这样下次通过filp就能找到这个proc了

binder_unlock(__func__); // 释放同步锁

```

3.3 binder_mmap

kernel/drivers/staging/android/binder.c

```

// 3355
static int binder_mmap(struct file *filp, struct vm_area_struct *vma)

// 3366 保证映射内存大小不超过4M
if ((vma->vm_end - vma->vm_start) > SZ_4M)
    vma->vm_end = vma->vm_start + SZ_4M;

// 3382 同步锁，保证一次只有一个进程分配内存，保证多进程间的并发访问
mutex_lock(&binder_mmap_lock);
// 是否已经做过映射，执行过则进入if，goto跳转，释放同步锁后结束binder_mmap方法
if (proc->buffer) {
    goto err_already_mapped;
}
// 采用 VM_IOREMAP方式，分配一个连续的内核虚拟内存，与进程虚拟内存大小一致
area = get_vm_area(vma->vm_end - vma->vm_start, VM_IOREMAP);
// 内存分配不成功直接报错
if (area == NULL) {
    ret = -ENOMEM;
    failure_string = "get_vm_area";
    goto err_get_vm_area_failed;
}
// 将proc中的buffer指针指向这块内核的虚拟内存
proc->buffer = area->addr;
// 计算出用户空间和内核空间的地址偏移量。地址偏移量 = 用户虚拟内存地址 - 内核虚拟内存地址
proc->user_buffer_offset = vma->vm_start - (uintptr_t)proc->buffer;

```

```

mutex_unlock(&binder_mmap_lock); // 释放锁

// 3407 分配物理页的指针数组，数组大小为vma的等效page个数
proc->pages = kzalloc(sizeof(proc->pages[0]) * ((vma->vm_end - vma->vm_start) / PAGE_SIZE), GFP_KERNEL);

// 3418 分配物理页面，同时映射到内核空间和进程空间，先分配1个物理页。
if (binder_update_page_range(proc, 1, proc->buffer, proc->buffer + PAGE_SIZE, vma)) {
kernel/drivers/staging/android/binder.c

// 576
static int binder_update_page_range(struct binder_proc *proc, int allocate,
                                void *start, void *end,
                                struct vm_area_struct *vma)

// 609 allocate为1，代表分配内存过程。如果为0则代表释放内存过程
if (allocate == 0)
    goto free_range;

// 624 分配一个page的物理内存
*page = alloc_page(GFP_KERNEL | __GFP_HIGHMEM | __GFP_ZERO);

// 630 物理空间映射到虚拟内核空间
ret = map_kernel_range_noflush((unsigned long)page_addr,
                                PAGE_SIZE, PAGE_KERNEL, page);

// 641 物理空间映射到虚拟进程空间
ret = vm_insert_page(vma, user_page_addr, page[0]);
kernel/drivers/staging/android/binder.c

// 3355
static int binder_mmap(struct file *filp, struct vm_area_struct *vma)

// 3425
list_add(&buffer->entry, &proc->buffers); // 将buffer连入buffers链表中
buffer->free = 1; // 此内存可用
binder_insert_free_buffer(proc, buffer); // 将buffer插入 proc->free_buffers 链表中
proc->free_async_space = proc->buffer_size / 2; // 异步的可用空闲空间大小
barrier();
proc->files = get_files_struct(current);
proc->vma = vma;
proc->vma_vm_mm = vma->vm_mm;

```

3.3.1 binderinsertfree_buffer

kernel/drivers/staging/android/binder.c

```

// 494
static void binder_insert_free_buffer(struct binder_proc *proc,
                                    struct binder_buffer *new_buffer)

// 511
while (*p) {
    parent = *p;
    buffer = rb_entry(parent, struct binder_buffer, rb_node);

    // 计算得出空闲内存的大小
    buffer_size = binder_buffer_size(proc, buffer);

    if (new_buffer_size < buffer_size)
        p = &parent->rb_left;
    else
        p = &parent->rb_right;
}
rb_link_node(&new_buffer->rb_node, parent, p);
// 将 buffer插入 proc->free_buffers 链表中
rb_insert_color(&new_buffer->rb_node, &proc->free_buffers);

```

3.4 binder_ioctl

kernel/drivers/staging/android/binder.c

```
// 3241
static long binder_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)

// 3254 进入休眠状态，直到中断唤醒
ret = wait_event_interruptible(binder_user_error_wait, binder_stop_on_user_error < 2);

// 3259 根据当前进程的pid，从binder_proc中查找binder_thread，
//      如果当前线程已经加入到proc的线程队列则直接返回，
//      如果不存在则创建binder_thread，并将当前线程添加到当前的proc
thread = binder_get_thread(proc);

// 3265 进行binder的读写操作
switch (cmd) {
    case BINDER_WRITE_READ:
        ret = binder_ioctl_write_read(filp, cmd, arg, thread);
        if (ret)
```

3.4.1 binder_ioctl_write_read

kernel/drivers/staging/android/binder.c

```
// 3136
static int binder_ioctl_write_read(struct file *filp,
                                   unsigned int cmd, unsigned long arg,
                                   struct binder_thread *thread)

// 3150 把用户空间数据ubuf拷贝到bwr
if (copy_from_user(&bwr, ubuf, sizeof(bwr))) {

// 3160
if (bwr.write_size > 0) { // 当写缓存中有数据，则执行binder写操作
    ret = binder_thread_write(proc, thread,
                              bwr.write_buffer,
                              bwr.write_size,
                              &bwr.write_consumed);
}
if (bwr.read_size > 0) { // 当读缓存中有数据，则执行binder读操作
    ret = binder_thread_read(proc, thread, bwr.read_buffer,
                              bwr.read_size,
                              &bwr.read_consumed,
                              filp->f_flags & O_NONBLOCK);
    // 进程todo队列不为空，则唤醒该队列中的线程
    if (!list_empty(&proc->todo))
        wake_up_interruptible(&proc->wait);
}

// 3192 把内核空间数据bwr拷贝到ubuf
if (copy_to_user(ubuf, &bwr, sizeof(bwr))) {
```

第四节 数据结构

4.1 file_operations

```
static const struct file_operations binder_fops = {
    .owner = THIS_MODULE,
    .poll = binder_poll,
    .unlocked_ioctl = binder_ioctl,
    .compat_ioctl = binder_ioctl,
    .mmap = binder_mmap,
```

```
.open = binder_open,
.flush = binder_flush,
.release = binder_release,
};
```

4.2 binder_proc

每个进程调用open()打开binder驱动都会创建该结构体，用于管理IPC所需的各种信息。

```
struct binder_proc {
    struct hlist_node proc_node; // 进程节点
    struct rb_root threads; // binder_thread红黑树的根节点
    struct rb_root nodes; // binder_node红黑树的根节点
    struct rb_root refs_by_desc; // binder_ref红黑树的根节点(以 handle为 key)
    struct rb_root refs_by_node; // binder_ref红黑树的根节点(以 ptr为 key)
    int pid; // 相应进程 id
    struct vm_area_struct *vma; // 指向进程虚拟地址空间的指针
    struct mm_struct *vma_vm_mm; // 相应进程的内存结构体
    struct task_struct *tsk; // 相应进程的 task结构体
    struct files_struct *files; // 相应进程的文件结构体
    struct hlist_node deferred_work_node;
    int deferred_work;
    void *buffer; // 内核空间的起始地址
    ptrdiff_t user_buffer_offset; // 内核空间与用户空间的地址偏移量

    struct list_head buffers; // 所有的 buffer
    struct rb_root free_buffers; // 空闲的 buffer
    struct rb_root allocated_buffers; // 已分配的 buffer
    size_t free_async_space; // 异步的可用空闲空间大小

    struct page **pages; // 指向物理内存页指针的指针
    size_t buffer_size; // 映射的内核空间大小
    uint32_t buffer_free; // 可用内存总大小
    struct list_head todo; // 进程将要做的事
    wait_queue_head_t wait; // 等待队列
    struct binder_stats stats; // binder统计信息
    struct list_head delivered_death; // 已分发的死亡通知
    int max_threads; // 最大线程数
    int requested_threads; // 请求的线程数
    int requested_threads_started; // 已启动的请求线程数
    int ready_threads; // 准备就绪的线程个数
    long default_priority; // 默认优先级
    struct dentry *debugfs_entry;
    struct binder_context *context;
};
```

4.3 binder_node

```
struct binder_node {
    int debug_id; // 节点创建时分配，具有全局唯一性，用于调试使用
    struct binder_work work;
    union {
        struct rb_node rb_node; // binder节点正常使用，union
        struct hlist_node dead_node; // binder节点已销毁，union
    };
    struct binder_proc *proc; // binder所在的进程，见后面小节
    struct hlist_head refs; // 所有指向该节点的 binder引用队列
    int internal_strong_refs;
    int local_weak_refs;
    int local_strong_refs;
    binder_uintptr_t ptr; // 指向用户空间 binder_node的指针，对应于 flat_binder_object.binder
    binder_uintptr_t cookie; // 指向用户空间 binder_node的指针，附件数据，对应于 flat_binder_object.cookie
    unsigned has_strong_ref:1; // 占位 1bit
```

```

unsigned pending_strong_ref:1; // 占位 1bit
unsigned has_weak_ref:1; // 占位 1bit
unsigned pending_weak_ref:1; // 占位 1bit
unsigned has_async_transaction:1; // 占位 1bit
unsigned accept_fds:1; // 占位 1bit
unsigned min_priority:8; // 占位 8bit, 最小优先级
struct list_head async_todo; // 异步todo队列
};

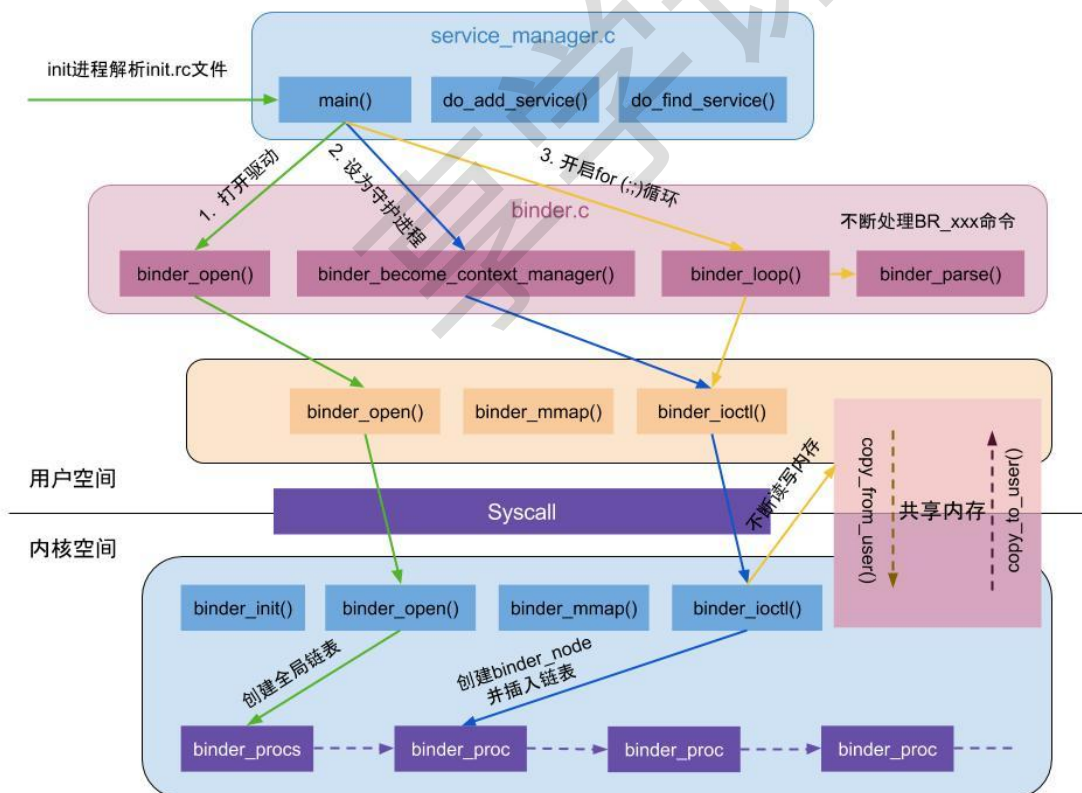
binder_buffer
struct binder_buffer {
    struct list_head entry; // buffer实体的地址
    struct rb_node rb_node; // buffer实体的地址
    /* by address */
    unsigned free:1; // 标记是否是空闲buffer, 占位1bit
    unsigned allow_user_free:1; // 是否允许用户释放, 占位1bit
    unsigned async_transaction:1; // 占位1bit
    unsigned debug_id:29; // 占位29bit

    struct binder_transaction *transaction; // 该缓存区的需要处理的事务

    struct binder_node *target_node; // 该缓存区所需处理的Binder实体
    size_t data_size; // 数据大小
    size_t offsets_size; // 数据偏移量
    size_t extra_buffers_size;
    uint8_t data[0]; // 数据地址
};

```

第五节 启动service_manager



5.1 启动servicemanager进程

ServiceManager是由init进程通过解析init.rc文件而创建的，其所对应的可执行程序servicemanager，所对应的源文件是service_manager.c，进程名为servicemanager。

```
system/core/rootdir/init.rc
```

```
// 602
service servicemanager /system/bin/servicemanager
    class core
    user system
    group system
    critical
    onrestart restart healthd
    onrestart restart zygote
    onrestart restart media
    onrestart restart surfaceflinger
    onrestart restart drm
```

5.2 main

启动ServiceManager的入口函数是service_manager.c中的main()方法。
frameworks/native/cmds/servicemanager/service_manager.c

```
// 354
int main(int argc, char **argv)

// 358 打开 binder驱动, 申请 128k字节大小的内存空间---见后面小节
bs = binder_open(128*1024);

// 364 设为守护进程, 成为 binder大管理者---见后面小节
if (binder_become_context_manager(bs)) {

// 391 进入无限循环, 处理client端发来的请求---见后面小节
binder_loop(bs, svcmgr_handler);
```

5.2.1 binder_open

frameworks/native/cmds/servicemanager/binder.c

```
// 96
struct binder_state *binder_open(size_t mapsize)

// 98 这个结构体记录了 service_manager 中有关于 binder 的所有信息
struct binder_state *bs;

// 107 打开 binder驱动, 得到文件描述符
bs->fd = open("/dev/binder", 0_RDWR);

// 123
bs->mapsize = mapsize; // service_manager自己设置的, 大小为 128kb
/*通过系统调用, mmap内存映射, mmap必须是 page的整数倍(即 4kb的整数倍)*/
bs->mapped = mmap(NULL, mapsize, PROT_READ, MAP_PRIVATE, bs->fd, 0);
```

5.2.2 binderbecomecontext_manager

frameworks/native/cmds/servicemanager/binder.c

```
// 146
int binder_become_context_manager(struct binder_state *bs)
{
    return ioctl(bs->fd, BINDER_SET_CONTEXT_MGR, 0);
}
```

2-2-1. binder_ioctl

kernel/drivers/staging/android/binder.c

```
// 3241
static long binder_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)

// 3277
```



```
case BINDER_SET_CONTEXT_MGR:
    ret = binder_ioctl_set_ctx_mgr(filp);
```

2-2-2. binder_ioctl_set_ctx_mgr

kernel/drivers/staging/android/binder.c

```
// 3200
static int binder_ioctl_set_ctx_mgr(struct file *filp)

// 3208 保证只创建一次 mgr_node对象, 不为 null就直接返回
if (context->binder_context_mgr_node) {

// 3216
/* uid是否有效, 当前是无效的 */
if (uid_valid(context->binder_context_mgr_uid)) {

} else {
    /* 设置当前线程 euid作为 service_manager的 uid */
    context->binder_context_mgr_uid = curr_euid;
}

// 创建 service_manager实体
context->binder_context_mgr_node = binder_new_node(proc, 0, 0);

// 3233 将 binder_context_mgr_node的强弱引用各加 1
context->binder_context_mgr_node->local_weak_refs++;
context->binder_context_mgr_node->local_strong_refs++;
context->binder_context_mgr_node->has_strong_ref = 1;
context->binder_context_mgr_node->has_weak_ref = 1;
```

2-2-2-1. binder_newnode

kernel/drivers/staging/android/binder.c

```
// 923
static struct binder_node *binder_new_node(struct binder_proc *proc,
                                           binder_uintptr_t ptr,
                                           binder_uintptr_t cookie)

// 931 首次进来为空
while (*p) {

// 943 给新创建的binder_node 分配内核空间
node = kzalloc(sizeof(*node), GFP_KERNEL);

// 947 将新创建的 node对象添加到 proc红黑树
rb_link_node(&node->rb_node, parent, p);
rb_insert_color(&node->rb_node, &proc->nodes);

// 950 初始化 binder_node
node->proc = proc;
node->ptr = ptr;
node->cookie = cookie;
node->work.type = BINDER_WORK_NODE; // 设置 binder_work的 type
INIT_LIST_HEAD(&node->work.entry);
INIT_LIST_HEAD(&node->async_todo);
```

5.2.3 binder_loop

frameworks/native/cmds/servicemanager/binder.c

```
// 372
void binder_loop(struct binder_state *bs, binder_handler func)

// 378
bwr.write_size = 0; // 初始化为 0
bwr.write_consumed = 0;
bwr.write_buffer = 0;

readbuf[0] = BC_ENTER_LOOPER; // 读写要处理的命令
```

```
binder_write(bs, readbuf, sizeof(uint32_t)); // 设置线程的 looper状态为循环状态
```

```
for (;;) {
    bwr.read_size = sizeof(readbuf); // 不为 0, 进入 binder_thread_read
    bwr.read_consumed = 0;
    bwr.read_buffer = (uintptr_t) readbuf;

    /* 不断地 binder读数据, 没有数据会进入休眠状态 */
    res = ioctl(bs->fd, BINDER_WRITE_READ, &bwr);
}
```

2-3-1. binder_write

frameworks/native/cmds/servicemanager/binder.c

```
// 151
int binder_write(struct binder_state *bs, void *data, size_t len)

// 156
bwr.write_size = len; // 大于 0, 进入 binder_thread_write
bwr.write_consumed = 0;
bwr.write_buffer = (uintptr_t) data; // 此处 data为 BC_ENTER_LOOPER
bwr.read_size = 0; // read 不会进去
bwr.read_consumed = 0;
bwr.read_buffer = 0;
/* 设置线程的 looper状态为循环状态 */
res = ioctl(bs->fd, BINDER_WRITE_READ, &bwr);
```

2-3-2. binderthreadwrite

kernel/drivers/staging/android/binder.c

```
// 2250
static int binder_thread_write(struct binder_proc *proc,
                               struct binder_thread *thread,
                               binder_uintptr_t binder_buffer, size_t size,
                               binder_size_t *consumed)

// 2262 获取命令, 即 BC_ENTER_LOOPER
if (get_user(cmd, (uint32_t __user *)ptr))

// 2472
case BC_ENTER_LOOPER:

// 2481 设置该线程的 looper状态
thread->looper |= BINDER_LOOPER_STATE_ENTERED;
```

2-3-3. binderthreadread

kernel/drivers/staging/android/binder.c

```
// 2652
static int binder_thread_read(struct binder_proc *proc,
                              struct binder_thread *thread,
                              binder_uintptr_t binder_buffer, size_t size,
                              binder_size_t *consumed, int non_block)

// 2664 设置命令为 BR_NOOP
if (*consumed == 0) {
    if (put_user(BR_NOOP, (uint32_t __user *)ptr))

// 2671 wait_for_proc_work 为 true
wait_for_proc_work = thread->transaction_stack == NULL &&
    list_empty(&thread->todo);

// 2694 准备就绪的线程个数加 1
if (wait_for_proc_work)
    proc->ready_threads++;

// 2702
if (wait_for_proc_work) {
```

```

if (non_block) { // 非阻塞操作，service_manager是阻塞的，所以 if不命中
} else // 进入 else，开始等待
    ret = wait_event_freezable_exclusive(proc->wait, binder_has_proc_work(proc, thread));
}

```

第六节获取service_manager

获取Service Manager是通过defaultServiceManager()方法来完成。

6.1 defaultServiceManager

frameworks/native/libs/binder/IServiceManager.cpp

```

// 33
sp<IServiceManager> defaultServiceManager()
{
    /* 单例模式，如果不为空直接返回 */
    if (gDefaultServiceManager != NULL) return gDefaultServiceManager;

    {
        AutoMutex _l(gDefaultServiceManagerLock);
        /* 创建或者获取 SM时，SM可能未准备就绪，这时通过 sleep 1秒后，循环尝试获取直到成功 */
        while (gDefaultServiceManager == NULL) {
            /* 分为三块分析 */
            gDefaultServiceManager = interface_cast<IServiceManager>(
                ProcessState::self()->getContextObject(NULL));
            if (gDefaultServiceManager == NULL)
                sleep(1);
        }
    }

    return gDefaultServiceManager;
}

```

6.1.1 ProcessState::self

frameworks/native/libs/binder/ProcessState.cpp

```

// 70
sp<ProcessState> ProcessState::self()
{
    /* 单例模式 */
    if (gProcess != NULL) {
        return gProcess;
    }
    gProcess = new ProcessState; // 实例化 ProcessState
    return gProcess;
}

```

1-1-1. ProcessState::ProcessState

frameworks/native/libs/binder/ProcessState.cpp

```

// 339
ProcessState::ProcessState()
    : mDriverFD(open_driver())

// 358 采用内存映射函数 mmap，给 binder分配一块大小为 (1M-8K)的虚拟地址空间，用来接收事务
mVMStart = mmap(0, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE | MAP_NORESERVE, mDriverFD, 0);

```

1-1-1-1. open_driver

frameworks/native/libs/binder/ProcessState.cpp

```

// 311
static int open_driver()

// 313 打开 /dev/binder设备，建立与内核的 Binder驱动力的交互通道

```

```
int fd = open("/dev/binder", 0_RDWR);

// 328 通过 ioctl设置 binder驱动, 能支持的最大线程数
size_t maxThreads = DEFAULT_MAX_BINDER_THREADS;
result = ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads);
```

6.1.2 ProcessState::getContextObject

frameworks/native/libs/binder/ProcessState.cpp

```
// 85
sp<IBinder> ProcessState::getContextObject(const sp<IBinder>& /*caller*/)
{
    // 参数为0, 获取service_manager服务
    return getStrongProxyForHandle(0);
}
```

1-2-1. ProcessState::getStrongProxyForHandle

frameworks/native/libs/binder/ProcessState.cpp

```
// 179
sp<IBinder> ProcessState::getStrongProxyForHandle(int32_t handle)

// 查找 handle对应的资源项
handle_entry* e = lookupHandleLocked(handle);

// 192 当handle值所对应的IBinder不存在或弱引用无效时
if (b == NULL || !e->refs->attemptIncWeak(this)) {

// 214 通过ping操作测试binder是否准备就绪
status_t status = IPCThreadState::self()->transact(
    0, IBinder::PING_TRANSACTION, data, NULL, 0);

// 220 创建BpBinder对象
b = new BpBinder(handle);
```

1-2-2. BpBinder::BpBinder

frameworks/native/libs/binder/BpBinder.cpp

```
// 89
BpBinder::BpBinder(int32_t handle)
    : mHandle(handle)
{
    /* 支持强弱引用计数, OBJECT_LIFETIME_WEAK表示目标对象的生命周期受弱指针控制 */
    extendObjectLifetime(OBJECT_LIFETIME_WEAK);
    /* handle所对应的 binder弱引用 + 1 */
    IPCThreadState::self()->incWeakHandle(handle);
}
```

6.1.3 interface_cast

frameworks/native/include/binder/IInterface.h

```
// 41
template<typename INTERFACE>
inline sp<INTERFACE> interface_cast(const sp<IBinder>& obj)
{
    // 等价于: IServiceManager::asInterface
    return INTERFACE::asInterface(obj);
}
```

1-3-1. IServiceManager::asInterface

对于asInterface()函数, 通过搜索代码, 你会发现根本找不到这个方法是在哪里定义这个函数的, 其实是通过模板函数来定义的。

frameworks/native/include/binder/IInterface.h

```
// 74
#define DECLARE_META_INTERFACE(INTERFACE) \
    static const android::String16 descriptor;
```

```

static android::sp<I##INTERFACE> asInterface(
    const android::sp<android::IBinder>& obj);
virtual const android::String16& getInterfaceDescriptor() const;
I##INTERFACE();
virtual ~I##INTERFACE();

// 83
#define IMPLEMENT_META_INTERFACE(INTERFACE, NAME)
const android::String16 I##INTERFACE::descriptor(NAME);
const android::String16&
    I##INTERFACE::getInterfaceDescriptor() const {
    return I##INTERFACE::descriptor;
}
android::sp<I##INTERFACE> I##INTERFACE::asInterface(
    const android::sp<android::IBinder>& obj)
{
    android::sp<I##INTERFACE> intr;
    if (obj != NULL) {
        intr = static_cast<I##INTERFACE*>(
            obj->queryLocalInterface(
                I##INTERFACE::descriptor).get());
        if (intr == NULL) {
            intr = new Bp##INTERFACE(obj);
        }
    }
    return intr;
}
I##INTERFACE::I##INTERFACE() {}
I##INTERFACE::~~I##INTERFACE() {}

```

1-3-2. DECLAREMETAINTERFACE

frameworks/native/include/binder/IServiceManager.h

```

// 33
DECLARE_META_INTERFACE(ServiceManager)

展开即可得:
static const android::String16 descriptor;

static android::sp< IServiceManager > asInterface(const android::sp<android::IBinder>& obj)

virtual const android::String16& getInterfaceDescriptor() const;

IServiceManager ();
virtual ~IServiceManager();

```

该过程主要是声明asInterface(), getInterfaceDescriptor()方法。

1-3-3. IMPLEMENTMETAINTERFACE

frameworks/native/libs/binder/IServiceManager.cpp

```

// 185
IMPLEMENT_META_INTERFACE(ServiceManager, "android.os.IServiceManager")

展开即可得:
const android::String16 IServiceManager::descriptor( "android.os.IServiceManager" );

const android::String16& IServiceManager::getInterfaceDescriptor() const
{
    return IServiceManager::descriptor;
}

android::sp<IServiceManager> IServiceManager::asInterface(const android::sp<android::IBinder>& obj)
{
    android::sp<IServiceManager> intr;
    if(obj != NULL) {
        intr = static_cast<IServiceManager *>(
            obj->queryLocalInterface(IServiceManager::descriptor).get());
        if (intr == NULL) {

```

```

        // 等价于 new BpServiceManager(BpBinder)
        intr = new BpServiceManager(obj);
    }
}
return intr;
}

```

```

IServiceManager::IServiceManager () { }
IServiceManager::~IServiceManager() { }

```

6.1.4 BpServiceManager

frameworks/native/libs/binder/IServiceManager.cpp

```

// 126
class BpServiceManager : public BpInterface<IServiceManager>

```

```

// 129
BpServiceManager(const sp<IBinder>& impl)
    : BpInterface<IServiceManager>(impl)

```

1-4-1. BpInterface::BpInterface

frameworks/native/include/binder/IInterface.h

```

// 134
template<typename INTERFACE>
inline BpInterface<INTERFACE>::BpInterface(const sp<IBinder>& remote)
    : BpRefBase(remote)

```

1-4-2. BpRefBase

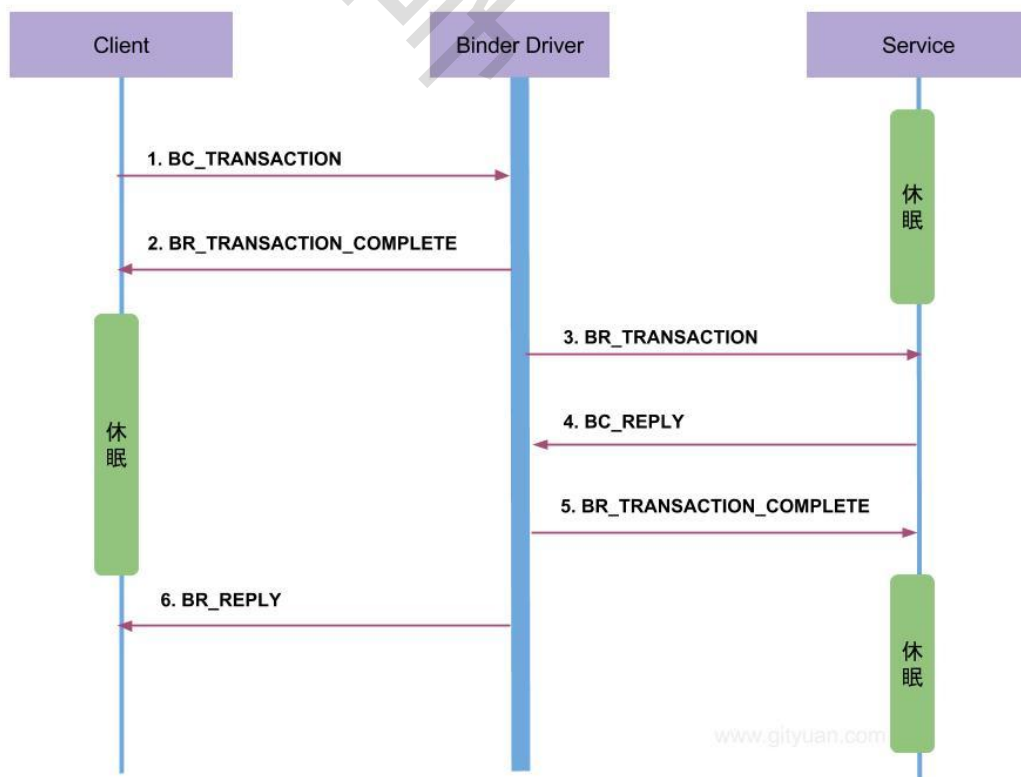
frameworks/native/libs/binder/Binder.cpp

```

// 241 mRemote指向 new BpBinder(0)，从而 BpServiceManager能够利用 Binder进行通过通信
BpRefBase::BpRefBase(const sp<IBinder>& o)
    : mRemote(o.get()), mRefs(NULL), mState(0)

```

第七节 addService流程



7.1 SystemServer

frameworks/base/services/java/com/android/server/SystemServer.java

```
// 167
public static void main(String[] args) {
    new SystemServer().run();
}
```

7.1.1 run

frameworks/base/services/java/com/android/server/SystemServer.java

```
// 176
private void run() {

// 263 创建 SystemServiceManager
mSystemServiceManager = new SystemServiceManager(mSystemContext);

// 268
startBootstrapServices();
```

7.1.2 startBootstrapServices

frameworks/base/services/java/com/android/server/SystemServer.java

```
// 322
private void startBootstrapServices() {

// 329 获取 AMS 的对象
mActivityManagerService = mSystemServiceManager.startService(
    ActivityManagerService.Lifecycle.class).getService();

// 378
mActivityManagerService.setSystemProcess();
```

7.1.3 setSystemProcess

frameworks/base/services/core/java/com/android/server/am/ActivityManagerService.java

```
// 2172
public void setSystemProcess() {

// 2174 添加 AMS("activity")到 service_manager中
ServiceManager.addService(Context.ACTIVITY_SERVICE, this, true);
```

7.1.4 ServiceManager.addService

frameworks/base/core/java/android/os/ServiceManager.java

```
// 87
public static void addService(String name, IBinder service, boolean allowIsolated) {

// 89 ---见后面小节（分别分析 getServiceManager和 addService）
getServiceManager().addService(name, service, allowIsolated);
```

7.2 getServiceManager

frameworks/base/core/java/android/os/ServiceManager.java

```
// 33
private static IServiceManager getServiceManager() {
    /* 采用单例形式返回 ServiceManagerProxy对象 */
    if (sServiceManager != null) {
        return sServiceManager;
    }
}
```

```
// 相当于 new ServiceManagerProxy(new BinderProxy); ---见后面小节（分别分析 asInterface和 getContextObject）
sServiceManager = ServiceManagerNative.asInterface(BinderInternal.getContextObject());
return sServiceManager;
}
```

7.2.1 BinderInternal.getContextObject

frameworks/base/core/java/com/android/internal/os/BinderInternal.java

```
// 88
public static final native IBinder getContextObject();
```

2-1-1. android.os.BinderInternal.getContextObject

frameworks/base/core/jni/android_util_Binder.cpp

```
// 899
static jobject android_os_BinderInternal_getContextObject(JNIEnv* env, jobject clazz)
{
    /* 打开 binder驱动（ProcessState是单例的），创建 BpBinder(handle) 对象，并返回 */
    sp<IBinder> b = ProcessState::self()->getContextObject(NULL);
    return javaObjectForIBinder(env, b);
}
```

2-1-2. javaObjectForIBinder

frameworks/base/core/jni/android_util_Binder.cpp

```
// 547
jobject javaObjectForIBinder(JNIEnv* env, const sp<IBinder>& val)

// 563 从 BpBinder中查找 BinderProxy对象，第一次为 null
jobject object = (jobject)val->findObject(&gBinderProxyOffsets);

// 576 创建 BinderProxy对象
object = env->NewObject(gBinderProxyOffsets.mClass, gBinderProxyOffsets.mConstructor);

// 580 BinderProxy.mObject成员变量记录 BpBinder对象
env->SetLongField(object, gBinderProxyOffsets.mObject, (jlong)val.get());

// 587 将 BinderProxy对象信息添加到 BpBinder的成员变量 mObjects中
val->attachObject(&gBinderProxyOffsets, refObject,
    jnienv_to_javavm(env), proxy_cleanup);

// 593 BinderProxy.mOrgue成员变量记录死亡通知对象
env->SetLongField(object, gBinderProxyOffsets.mOrgue, reinterpret_cast<jlong>(drl.get()));
```

7.2.2 ServiceManagerNative.asInterface

frameworks/base/core/java/android/os/ServiceManagerNative.java

```
// 33
static public IServiceManager asInterface(IBinder obj)

// 38 因为 obj为 BinderProxy，默认返回 null
IServiceManager in =
    (IServiceManager)obj.queryLocalInterface(descriptor);

// 44
return new ServiceManagerProxy(obj);
```

7.2.3 ServiceManagerProxy

frameworks/base/core/java/android/os/ServiceManagerNative.java\$ServiceManagerProxy.java

```
// 109
class ServiceManagerProxy implements IServiceManager {
    // mRemote为 BinderProxy对象
    public ServiceManagerProxy(IBinder remote) {
```



```

        mRemote = remote;
    }

```

7.3 SMP.addService--1

frameworks/base/core/java/android/os/ServiceManagerNative.java\$ServiceManagerProxy.java

```

// 142
public void addService(String name, IBinder service, boolean allowIsolated)
    throws RemoteException {
    Parcel data = Parcel.obtain();
    Parcel reply = Parcel.obtain();

    // 148 此处 service == AMS
    data.writeStrongBinder(service);

```

7.3.1 Parcel.writeStrongBinder

frameworks/base/core/java/android/os/Parcel.java

```

// 583
public final void writeStrongBinder(IBinder val) {
    nativeWriteStrongBinder(mNativePtr, val);
}

```

7.3.2 android.os.Parcel.writeStrongBinder

frameworks/base/core/jni/android_os_Parcel.cpp

```

// 298
static void android_os_Parcel_writeStrongBinder(JNIEnv* env, jclass clazz, jlong nativePtr, jobject object)

// 300 将java层 Parcel转换为 native层 Parcel
Parcel* parcel = reinterpret_cast<Parcel*>(nativePtr);

// 302 ---见后面小节（分别分析 ibinderForJavaObject和 writeStrongBinder）
const status_t err = parcel->writeStrongBinder(ibinderForJavaObject(env, object));

```

3-2-1. ibinderForJavaObject

frameworks/base/core/jni/android_util_Binder.cpp

```

// 603
sp<IBinder> ibinderForJavaObject(JNIEnv* env, jobject obj)

// 607
if (env->IsInstanceOf(obj, gBinderOffsets.mClass)) { // 是否是 Java层的 Binder对象，此处是 AMS，if命中
    JavaBBinderHolder* jbh = (JavaBBinderHolder*)
        env->GetLongField(obj, gBinderOffsets.mObject);
    return jbh != NULL ? jbh->get(env, obj) : NULL; // 返回 JavaBBinder对象
}

```

3-2-1-1. JavaBBinderHolder.get

frameworks/base/core/jni/android_util_Binder.cpp\$JavaBBinderHolder.cpp

```

// 317
sp<JavaBBinder> get(JNIEnv* env, jobject obj)

// 320
sp<JavaBBinder> b = mBinder.promote(); // 将弱指针升级为强指针，首次进来返回空指针
if (b == NULL) {
    b = new JavaBBinder(env, obj); // 创建一个 JavaBBinder 对象并返回
    记住：writeStrongBinder的参数是 JavaBBinder对象。
}

```

3-2-2. (Parcel.cpp)parcel->writeStrongBinder

frameworks/native/libs/binder/Parcel.cpp

```
// 872
status_t Parcel::writeStrongBinder(const sp<IBinder>& val)
{
    return flatten_binder(ProcessState::self(), val, this);
}
```

3-2-2-1. flatten_binder

frameworks/native/libs/binder/Parcel.cpp

```
// 205
status_t flatten_binder(const sp<ProcessState>& /*proc*/,
    const sp<IBinder>& binder, Parcel* out)

// 208
flat_binder_object obj;

// 212 当前进程有 Binder，所以本地 Binder不为空
IBinder *local = binder->localBinder();

// 224 Binder对象扁平化，转换成 flat_binder_object对象
obj.type = BINDER_TYPE_BINDER;
obj.binder = reinterpret_cast<uintptr_t>(local->getWeakRefs());
obj.cookie = reinterpret_cast<uintptr_t>(local);

// 234
return finish_flatten_binder(binder, obj, out);
```

3-2-2-2. finishflattenbinder

frameworks/native/libs/binder/Parcel.cpp

```
// 199
inline static status_t finish_flatten_binder(
    const sp<IBinder>& /*binder*/, const flat_binder_object& flat, Parcel* out)
{
    // 将 flat_binder_object写入 out
    return out->writeObject(flat, false);
}
```

7.3 SMP.addService--2

frameworks/base/core/java/android/os/ServiceManagerNative.java\$ServiceManagerProxy.java

```
// 142
public void addService(String name, IBinder service, boolean allowIsolated)
    throws RemoteException {

// 150 mRemote为 BinderProxy对象
mRemote.transact(ADD_SERVICE_TRANSACTION, data, reply, 0);
```

7.3.1 BinderProxy.transact

frameworks/base/core/java/android/os/Binder.java\$BinderProxy.java

```
// 501
public boolean transact(int code, Parcel data, Parcel reply, int flags) throws RemoteException {

// 503
return transactNative(code, data, reply, flags);
```

7.3.2 android.os.BinderProxy.transact

frameworks/base/core/jni/android_util_Binder.cpp

```

// 1083
static jboolean android_os_BinderProxy_transact(JNIEnv* env, jobject obj,
        jint code, jobject dataObj, jobject replyObj, jint flags) // throws RemoteException

// 1091 获取 data对象
Parcel* data = parcelForJavaObject(env, dataObj);

// 1095 获取 reply对象
Parcel* reply = parcelForJavaObject(env, replyObj);

// 1100 获取 BpBinder 对象
IBinder* target = (IBinder*)
        env->GetLongField(obj, gBinderProxyOffsets.mObject);

// 1124
status_t err = target->transact(code, *data, reply, flags);

```

7.3.3 BpBinder::transact

frameworks/native/libs/binder/BpBinder.cpp

```

// 159
status_t BpBinder::transact(
        uint32_t code, const Parcel& data, Parcel* reply, uint32_t flags)

// 164
status_t status = IPThreadState::self()->transact(
        mHandle, code, data, reply, flags);

```

7.3.4 IPThreadState::transact

frameworks/native/libs/binder/IPThreadState.cpp

```

// 548
status_t IPThreadState::transact(int32_t handle,
        uint32_t code, const Parcel& data,
        Parcel* reply, uint32_t flags)

// 552 数据错误检查
status_t err = data.errorCheck();

// 554
flags |= TF_ACCEPT_FDS;
    TF_ACCEPT_FDS = 0x10: 允许回复中包含文件描述符
    TF_ONE_WAY: 当前业务是异步的, 不需要等待
    TF_ROOT_OBJECT: 所包含的内容是根对象
    TF_STATUS_CODE: 所包含的内容是 32-bit 的状态值

// 566 整理数据, 并把结果存入 mOut 中。(在 talkWithDriver方法中才会将命令真正发送给 Binder驱动)
err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, NULL);

// 574
if ((flags & TF_ONE_WAY) == 0) { // 不是异步, if命中
    if (reply) { // 不为空
        err = waitForResponse(reply); // 等待回应事件
    }
}

```

3-4-1. writeTransactionData

frameworks/native/libs/binder/IPThreadState.cpp

```

// 904
status_t IPThreadState::writeTransactionData(int32_t cmd, uint32_t binderFlags,
        int32_t handle, uint32_t code, const Parcel& data, status_t* statusBuffer)

// 934
mOut.writeInt32(cmd); // mOut写入命令为 BC_TRANSACTION
mOut.write(&tr, sizeof(tr)); // 写入 binder_transaction_data数据

```

7.3.5 IPThreadState::waitForResponse--1

frameworks/native/libs/binder/IPThreadState.cpp

```
// 712
status_t IPThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)
```

```
// 717 循环等待结果
```

```
while (1) {
    if ((err=talkWithDriver()) < NO_ERROR) break;
```

3-5-1.talkWithDriver

frameworks/native/libs/binder/IPThreadState.cpp

```
// 803
```

```
status_t IPThreadState::talkWithDriver(bool doReceive)
```

```
// 812 读的 buffer是否为空。现在读为 null
```

```
const bool needRead = mIn.dataPosition() >= mIn.dataSize();
```

```
// 817
```

```
const size_t outAvail = (!doReceive || needRead) ? mOut.dataSize() : 0; // 读的时候不能写 mOut
```

```
bwr.write_size = outAvail;
```

```
bwr.write_buffer = (uintptr_t)mOut.data(); // 在 bwr中填写需要 write的大小和内容
```

```
if (doReceive && needRead) {
    bwr.read_size = mIn.dataCapacity();
    bwr.read_buffer = (uintptr_t)mIn.data();
} else { // needRead为 null, 走 else
    bwr.read_size = 0;
    bwr.read_buffer = 0;
}
```

```
// 851
```

```
do { // while循环条件不会成立, 只执行一次
```

```
    /* 856 写入命令 BC_TRANSACTION */
```

```
    if (ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr) >= 0)
```

```
} while (err == -EINTR);
```

3-5-2.binderioctlwrite_read--1

kernel/drivers/staging/android/binder.c

```
// 3136
```

```
static int binder_ioctl_write_read(struct file *filp,
    unsigned int cmd, unsigned long arg,
    struct binder_thread *thread)
```

```
// 3161 通过这个函数写入用户的数据
```

```
ret = binder_thread_write(proc, thread,
    bwr.write_buffer,
    bwr.write_size,
    &bwr.write_consumed);
```

3-5-2-1.binderthreadwrite

kernel/drivers/staging/android/binder.c

```
// 2250
```

```
static int binder_thread_write(struct binder_proc *proc,
    struct binder_thread *thread,
    binder_uintptr_t binder_buffer, size_t size,
    binder_size_t *consumed)
```

```
// 2442
```

```
case BC_TRANSACTION:
```

```
    binder_transaction(proc, thread, &tr,
```

```
        cmd == BC_REPLY, 0); // 此处 cmd == BC_TRANSACTION, 第四个参数为 false
```

3-5-2-2. binder_transaction

kernel/drivers/staging/android/binder.c

```
// 1829
static void binder_transaction(struct binder_proc *proc,
                              struct binder_thread *thread,
                              struct binder_transaction_data *tr, int reply,
                              binder_size_t extra_buffers_size)

// 1861 此处 reply为 false (cmd == BC_TRANSACTION)
if (reply) {

} else {
    if (tr->target.handle) { // 不命中 if, handle不为 0, 才会命中 if

    } else { // 此处走 else
        /* 获取目标对象的 target_node, 目标是 service_manager, 所以可以直接使用全局变量 binder_context_mgr_node */
        target_node = context->binder_context_mgr_node;
    }
    /* 1919 target_proc为 service_manager进程 */
    target_proc = target_node->proc;
}

// 1954 找到 service_manager进程的 todo队列
target_list = &target_proc->todo;
target_wait = &target_proc->wait;

// 1960 生成一个 binder_transaction 变量 (即变量 t), 用于描述本次要进行的 transaction (最后将其加入 target_thread->todo)。
// 这样当目标对象被唤醒时, 它就可以从这个队列中取出需要做的工作。
t = kzalloc(sizeof(*t), GFP_KERNEL);

// 1967 生成一个binder_work变量 (即变量 tcomplete), 用于说明当前调用者线程有一宗未完成的 transaction (它最后会被添加到本线程的 todo队列中)
tcomplete = kzalloc(sizeof(*tcomplete), GFP_KERNEL);

// 1996 给 transaction结构体赋值, 即变量 t
if (!reply && !(tr->flags & TF_ONE_WAY)) // 非 oneway的通信方式, 把当前 thread保存到 transaction的 from字段
    t->from = thread;
else
    t->from = NULL;
t->sender_euid = task_euid(proc->tsk);
t->to_proc = target_proc; // 此次通信目标进程为 service_manager进程
t->to_thread = target_thread;
t->code = tr->code; // 此次通信 code = ADD_SERVICE_TRANSACTION
t->flags = tr->flags; // 此次通信 flags = 0
t->priority = task_nice(current);

// 2009 从 service_manager进程中分配 buffer (为完成本条 transaction申请内存, 从 binder_mmap开辟的空间中申请内存)
t->buffer = binder_alloc_buf(target_proc, tr->data_size,
                             tr->offsets_size, extra_buffers_size,
                             !reply && (t->flags & TF_ONE_WAY));

// 2028 分别拷贝用户空间的 binder_transaction_data中 ptr.buffer和 ptr.offsets到内核
if (copy_from_user(t->buffer->data, (const void __user *) (uintptr_t)
                  tr->data.ptr.buffer, tr->data_size)) {
}
if (copy_from_user(offp, (const void __user *) (uintptr_t)
                  tr->data.ptr.offsets, tr->offsets_size)) {
}

// 2059
for (; offp < off_end; offp++) {

    // 2075
    case BINDER_TYPE_BINDER:
    case BINDER_TYPE_WEAK_BINDER: {
```

```

    struct flat_binder_object *fp;

    fp = to_flat_binder_object(hdr);
    /* 创建 binder_ref, service_manager的 binder引用对象---见后面小节 */
    ret = binder_translate_binder(fp, t, thread);
    if (ret < 0) {
        return_error = BR_FAILED_REPLY;
        goto err_translate_failed;
    }
} break;
}

// 2187
} else if (!(t->flags & TF_ONE_WAY)) {

    // 2191
    thread->transaction_stack = t; // 记录本次 transaction,以备后期查询 (service_manager通过这个知道是谁调用的,从而返回数据)
}

// 2201
t->work.type = BINDER_WORK_TRANSACTION; // 设置 t的类型为 BINDER_WORK_TRANSACTION
list_add_tail(&t->work.entry, target_list); // 将 t加入目标的处理队列中
tcomplete->type = BINDER_WORK_TRANSACTION_COMPLETE; // 设置 binder_work的类型为 BINDER_WORK_TRANSACTION_COMPLETE
list_add_tail(&tcomplete->entry, &thread->todo); // 当前线程有一个未完成的操作
if (target_wait)
    wake_up_interruptible(target_wait); // 唤醒目标,即 service_manager

```

3-5-2-2-1. binder *translate* binder

kernel/drivers/staging/android/binder.c

```

// 1564
static int binder_translate_binder(struct flat_binder_object *fp,
                                struct binder_transaction *t,
                                struct binder_thread *thread)

// 1592 创建一个 binder_ref
ref = binder_get_ref_for_node(target_proc, node);

// 1596 改变类型为 BINDER_TYPE_HANDLE
if (fp->hdr.type == BINDER_TYPE_BINDER)
    fp->hdr.type = BINDER_TYPE_HANDLE;

```

3-5-2. binder *ioctl* *write* *read*--2--*servicemanager*已被唤醒

kernel/drivers/staging/android/binder.c

```

// 3136
static int binder_ioctl_write_read(struct file *filp,
                                unsigned int cmd, unsigned long arg,
                                struct binder_thread *thread)

// 3174
ret = binder_thread_read(proc, thread, bwr.read_buffer,
                        bwr.read_size,
                        &bwr.read_consumed,
                        filp->f_flags & O_NONBLOCK);

```

3-5-2-3. binder *thread* *read*

kernel/drivers/staging/android/binder.c

```

// 2652
static int binder_thread_read(struct binder_proc *proc,
                            struct binder_thread *thread,
                            binder_uintptr_t binder_buffer, size_t size,
                            binder_size_t *consumed, int non_block)

// 2664 如果 consumed==0, 则写入一个 BR_NOOP

```

```

if (*consumed == 0) {
    if (put_user(BR_NOOP, (uint32_t __user *)ptr))

// 2739 前面把一个 binder_work 添加到 thread->todo 队列中, 所以 w 不为空, 类型为 BINDER_WORK_TRANSACTION_COMPLETE
if (!list_empty(&thread->todo)) {
    w = list_first_entry(&thread->todo, struct binder_work,
                        entry);

// 2760 写入命令 BR_TRANSACTION_COMPLETE
case BINDER_WORK_TRANSACTION_COMPLETE: {
    cmd = BR_TRANSACTION_COMPLETE;
    if (put_user(cmd, (uint32_t __user *)ptr))

```

7.3.5 IPCThreadState::waitForResponse--2

frameworks/native/libs/binder/IPCThreadState.cpp

```

// 712
status_t IPCThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)

// 786 处理 BR_NOOP 命令, 什么也没干
default:
    err = executeCommand(cmd);

// 718 while 循环, 继续执行 talkWithDriver 方法
if ((err=talkWithDriver()) < NO_ERROR) break;

// 812 因为 mIn 中有数据, 所以 needRead 为 false, 导致 bwr.write_size 和 bwr.read_size 都为 0, 直接返回
const bool needRead = mIn.dataPosition() >= mIn.dataSize();

// 731 处理 BR_TRANSACTION_COMPLETE 命令
case BR_TRANSACTION_COMPLETE:
    if (!reply && !acquireResult) goto finish; // 当前为同步, 不会进入 if, 继续 while 循环

// 718 再次执行 talkWithDriver 方法, 这个时候 bwr.write_size==0, bwr.read_size 还是大于 0, 所以直接执行驱动中 binder_thread_read
if ((err=talkWithDriver()) < NO_ERROR) break;

```

7.3.6 binderthreadread--Client 线程进入等待

kernel/drivers/staging/android/binder.c

```

// 2652
static int binder_thread_read(struct binder_proc *proc,
                             struct binder_thread *thread,
                             binder_uintptr_t binder_buffer, size_t size,
                             binder_size_t *consumed, int non_block)

// 2664 放入 BR_NOOP 命令
if (*consumed == 0) {
    if (put_user(BR_NOOP, (uint32_t __user *)ptr))

// 2671 此时 wait_for_proc_work 为 false
wait_for_proc_work = thread->transaction_stack == NULL &&
    list_empty(&thread->todo);

// 2717
if (non_block) { // 是阻塞模式的, 所以 if 不会命中
} else // 进入等待, 直到 service_manager 来唤醒

```

```
ret = wait_event_freezable(thread->wait, binder_has_thread_work(thread));
```

7.3.7 binderthreadread--service_manager开始处理消息

kernel/drivers/staging/android/binder.c

```
// 2652
static int binder_thread_read(struct binder_proc *proc,
                             struct binder_thread *thread,
                             binder_uintptr_t binder_buffer, size_t size,
                             binder_size_t *consumed, int non_block)

// 2757 主要是把用户的请求复制到 service_manager中并对各种队列进行调整
case BINDER_WORK_TRANSACTION: {
    t = container_of(w, struct binder_transaction, work);

// 2898 设置命令为 BR_TRANSACTION
cmd = BR_TRANSACTION;
```

7.3.8 binder_loop

frameworks/native/cmds/servicemanager/binder.c

```
// 372
void binder_loop(struct binder_state *bs, binder_handler func)

// 397 对 getService请求进行解析
res = binder_parse(bs, 0, (uintptr_t) readbuf, bwr.read_consumed, func);
```

7.3.9 binder_parse

frameworks/native/cmds/servicemanager/binder.c

```
// 204
int binder_parse(struct binder_state *bs, struct binder_io *bio,
                uintptr_t ptr, size_t size, binder_handler func)

// 230
case BR_TRANSACTION: {

    // 237
    if (func) {

        // 243 为 reply初始化
        bio_init(&reply, rdata, sizeof(rdata), 4);

        // 245
        res = func(bs, txn, &msg, &reply); // 由 svcmgr_handler 处理请求
        binder_send_reply(bs, &reply, txn->data.ptr.buffer, res); // 将 reply发给 binder驱动
    }
}
```

3-9-1. service_manager

frameworks/native/cmds/servicemanager/service_manager.c

```
// 251
int svcmgr_handler(struct binder_state *bs,
                  struct binder_transaction_data *txn,
                  struct binder_io *msg,
                  struct binder_io *reply)

// 309
case SVC_MGR_ADD_SERVICE:

    // 316 注册指定服务
    if (do_add_service(bs, s, len, handle, txn->sender_euid,
                      allow_isolated, txn->sender_pid))
```

3-9-1-1. doaddservice

frameworks/native/cmds/servicemanager/service_manager.c


```

// 201
int do_add_service(struct binder_state *bs,
                  const uint16_t *s, size_t len,
                  uint32_t handle, uid_t uid, int allow_isolated,
                  pid_t spid)

// 214
if (!svc_can_register(s, len, spid, uid)) {

// 220
si = find_svc(s, len);
if (si) {
    if (si->handle) {
        svcinfo_death(bs, si); // 服务已注册时，释放相应的服务
    }
    si->handle = handle; // 重新放入新的
} else {
    si = malloc(sizeof(*si) + (len + 1) * sizeof(uint16_t));
    if (!si) { // 内存不足，无法分配足够内存
        return -1;
    }
    si->handle = handle;
    si->len = len;
    memcpy(si->name, s, (len + 1) * sizeof(uint16_t)); // 内存拷贝服务信息
    si->name[len] = '\0';
    si->death.func = (void*) svcinfo_death;
    si->death.ptr = si;
    si->allow_isolated = allow_isolated;
    si->next = svclist; // svclist保存所有已注册的服务
    svclist = si;
}

/* 以 BC_ACQUIRE命令，handle为目标的信息，通过 ioctl1发送给 binder驱动，binder_ref强引用加 1操作 */
binder_acquire(bs, handle);
/* 以 BC_REQUEST_DEATH_NOTIFICATION命令的信息，通过 ioctl1发送给 binder驱动，主要用于清理内存等收尾工作 */
binder_link_to_death(bs, handle, &si->death);

```

3-9-2. bindersendreply

frameworks/native/cmds/servicemanager/binder.c

```

// 170
void binder_send_reply(struct binder_state *bs,
                      struct binder_io *reply,
                      binder_uintptr_t buffer_to_free,
                      int status)

// 182
data.cmd_free = BC_FREE_BUFFER; // free buffer命令
data.buffer = buffer_to_free;
data.cmd_reply = BC_REPLY; // reply命令
data.txn.target.ptr = 0;
data.txn.cookie = 0;
data.txn.code = 0;
if (status) { // status == 0
} else {
    data.txn.flags = 0;
    data.txn.data_size = reply->data - reply->data0;
    data.txn.offsets_size = ((char*) reply->offs) - ((char*) reply->offs0);
    data.txn.data.ptr.buffer = (uintptr_t)reply->data0;
    data.txn.data.ptr.offsets = (uintptr_t)reply->offs0;
}
binder_write(bs, &data, sizeof(data)); // 向 Binder驱动通信

```

3-9-2-1. binderthreadwrite

kernel/drivers/staging/android/binder.c

```
// 2250
static int binder_thread_write(struct binder_proc *proc,
    struct binder_thread *thread,
    binder_uintptr_t binder_buffer, size_t size,
    binder_size_t *consumed)
```

```
binder_transaction(proc, thread, &tr,
    cmd == BC_REPLY, 0);
```

附：本内容由享学课堂移动开发技术团队独家提供
<https://ke.qq.com/course/341933?tuin=9f978cfe&taid=9303882710988717>，未经享学课堂授权不得转载、摘编或利用其它方式使用上述作品。已经享学课堂授权使用作品的，应在授权范围内使用，并注明“来源：享学课堂”。违反上述声明者，将追究其相关法律责任。

第八节 Binder面试题全解析

Binder是一种进程间通信机制，做 **Android** 开发肯定离不开跟 **Binder** 打交道。

在面试中**Binder**也是经常被问的一个点，那么本篇文章就以问答的方式，带你了解一下关于**Binder** 的重要知识点。

8.1 Binder有什么优势

性能方面

- 共享内存 0次数据拷贝
- **Binder** 1次数据拷贝
- **Socket**/管道/消息队列 2次数据拷贝

稳定性方面

- **Binder**：基于C/S架构，客户端（**Client**）有什么需求就丢给服务端（**Server**）去完成，架构清晰、职责明确又相互独立，自然稳定性更好
- 共享内存：虽然无需拷贝，但是控制复杂，难以使用
- 从稳定性的角度讲，**Binder**机制是优于内存共享的。

安全性方面

- 传统的**IPC**没有任何安全措施，安全依赖上层协议来确保。
- 传统的**IPC**方法无法获得对方可靠的进程用户**ID**/进程**UI**（**UID/PID**），从而无法鉴别对方身份。
- 传统的**IPC**只能由用户在数据包中填入**UID/PID**，容易被恶意程序利用。
- 传统的**IPC**访问接入点是开放的，无法阻止恶意程序通过猜测接收方地址获得连接。
- **Binder**既支持实名**Binder**，又支持匿名**Binder**，安全性高。

8.2 Binder是如何做到一次拷贝的

主要是因为**Linux**是使用的虚拟内存寻址方式，它有如下特性：

- 用户空间的虚拟内存地址是映射到物理内存中的

- 对虚拟内存的读写实际上是对物理内存的读写，这个过程就是内存映射
- 这个内存映射过程是通过系统调用mmap()来实现的
- **Binder**借助了内存映射的方法，在内核空间和接收方用户空间的数据缓存区之间做了一层内存映射，就相当于直接拷贝到了接收方用户空间的数据缓存区，从而减少了一次数据拷贝

8.3 MMAP的内存映射原理了解吗

MMAP内存映射的实现过程，总的来说可以分为三个阶段：

（一）进程启动映射过程，并在虚拟地址空间中为映射创建虚拟映射区域

1. 进程在用户空间调用库函数mmap，原型：`void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset);`
2. 在当前进程的虚拟地址空间中，寻找一段空闲的满足要求的连续的虚拟地址
3. 为此虚拟区分配一个vm_area_struct结构，接着对这个结构的各个域进行了初始化
4. 将新建的虚拟区结构（vm_area_struct）插入进程的虚拟地址区域链表或树中

（二）调用内核空间的系统调用函数mmap（不同于用户空间函数），实现文件物理地址和进程虚拟地址的一一映射关系

1. 为映射分配了新的虚拟地址区域后，通过待映射的文件指针，在文件描述符表中找到对应的文件描述符，通过文件描述符，链接到内核“已打开文件集”中该文件的文件结构体（`struct file`），每个文件结构体维护着和这个已打开文件相关各项信息。
2. 通过该文件的文件结构体，链接到file_operations模块，调用内核函数mmap，其原型为：`int mmap(struct file *filp, struct vm_area_struct *vma)`，不同于用户空间库函数。
3. 内核mmap函数通过虚拟文件系统inode模块定位到文件磁盘物理地址。
4. 通过remap_pfn_range函数建立页表，即实现了文件地址和虚拟地址区域的映射关系。此时，这片虚拟地址并没有任何数据关联到主存中。

（三）进程发起对这片映射空间的访问，引发缺页异常，实现文件内容到物理内存（主存）的拷贝

注：前两个阶段仅在于创建虚拟区间并完成地址映射，但是并没有将任何文件数据的拷贝至主存。真正的文件读取是当进程发起读或写操作时。

进程的读或写操作访问虚拟地址空间这一段映射地址，通过查询页表，发现这一段地址并不在物理页面上。因为目前只建立了地址映射，真正的硬盘数据还没有拷贝到内存中，因此引发缺页异常。

1. 缺页异常进行一系列判断，确定无非法操作后，内核发起请求调页过程。
2. 调页过程先在交换缓存空间（`swap cache`）中寻找需要访问的内存页，如果没有则调用npage函数把所缺的页从磁盘装入到主存中。
3. 之后进程即可对这片主存进行读或者写的操作，如果写操作改变了其内容，一定时间后系统会自动回写脏页面到对应磁盘地址，也即完成了写入到文件的过程。

注：修改过的脏页面并不会立即更新回文件中，而是有一段时间的延迟，可以调用msync()来强制同步，这样所写的内容就能立即保存到文件里了。

8.4 Binder机制是如何跨进程的

1. Binder驱动

- 在内核空间创建一块接收缓存区，
- 实现地址映射：将内核缓存区、接收进程用户空间映射到同一接收缓存区

2. 发送进程通过系统调用（`copy_from_user`）将数据发送到内核缓存区。由于内核缓存区和接收进程用户空间存在映射关系，故相当于也发送了接收进程的用户空间，实现了跨进程通信。

8.5 说说四大组件的通信机制

1. activity

- （1）一个Activity通常就是一个单独的屏幕（窗口）。
- （2）Activity之间通过Intent进行通信。
- （3）android应用中每一个Activity都必须要在AndroidManifest.xml配置文件中声明，否则系统将不识别也不执行该Activity。

2. service

- （1）service用于在后台完成用户指定的操作。service分为两种：
 - **started**（启动）：当应用程序组件（如activity）调用startService()方法启动服务时，服务处于started状态。
 - **bound**（绑定）：当应用程序组件调用bindService()方法绑定到服务时，服务处于bound状态。
- （2）startService()与bindService()区别：
 - **started service**（启动服务）是由其他组件调用startService()方法启动的，这导致服务的onStartCommand()方法被调用。当服务是started状态时，其生命周期与启动它的组件无关，并且可以在后台无限期运行，即使启动服务的组件已经被销毁。因此，服务需要在完成任务后调用stopSelf()方法停止，或者由其他组件调用stopService()方法停止。
 - 使用bindService()方法启用服务，调用者与服务绑定在了一起，调用者一旦退出，服务也就终止，大有“不求同时生，必须同时死”的特点。
- （3）开发人员需要在应用程序配置文件中声明全部的service，使用<service></service>标签。

（4）Service通常位于后台运行，它一般不需要与用户交互，因此Service组件没有图形用户界面。Service组件需要继承Service基类。Service组件通常用于为其他组件提供后台服务或监控其他组件的运行状态。

3.content provider

(1) android平台提供了Content Provider使一个应用程序的指定数据集提供给其他应用程序。其他应用可以通过ContentResolver类从该内容提供者中获取或存入数据。

(2) 只有需要在多个应用程序间共享数据是才需要内容提供者。例如，通讯录数据被多个应用程序使用，且必须存储在一个内容提供者中。它的好处是统一数据访问方式。

(3) ContentProvider实现数据共享。ContentProvider用于保存和获取数据，并使其对所有应用程序可见。这是不同应用程序间共享数据的唯一方式，因为android没有提供所有应用共同访问的公共存储区。

(4) 开发人员不会直接使用ContentProvider类的对象，大多数是通过ContentResolver对象实现对ContentProvider的操作。

(5) ContentProvider使用URI来唯一标识其数据集，这里的URI以content://作为前缀，表示该数据由ContentProvider来管理。

4.broadcast receiver

(1) 你的应用可以使用它对外部事件进行过滤，只对感兴趣的外部事件(如当电话呼入时，或者数据网络可用时)进行接收并做出响应。广播接收器没有用户界面。然而，它们可以启动一个activity或service来响应它们收到的信息，或者用NotificationManager来通知用户。通知可以用很多种方式来吸引用户的注意力，例如闪动背灯、震动、播放声音等。一般来说是在状态栏上放一个持久的图标，用户可以打开它并获取消息。

(2) 广播接收者的注册有两种方法，分别是程序动态注册和AndroidManifest文件中进行静态注册。

(3) 动态注册广播接收器特点是当用来注册的Activity关掉后，广播也就失效了。静态注册无需担忧广播接收器是否被关闭，只要设备是开启状态，广播接收器也是打开着的。也就是说哪怕app本身未启动，该app订阅的广播在触发时也会对它起作用。

8.6 为什么Intent不能传递大数据

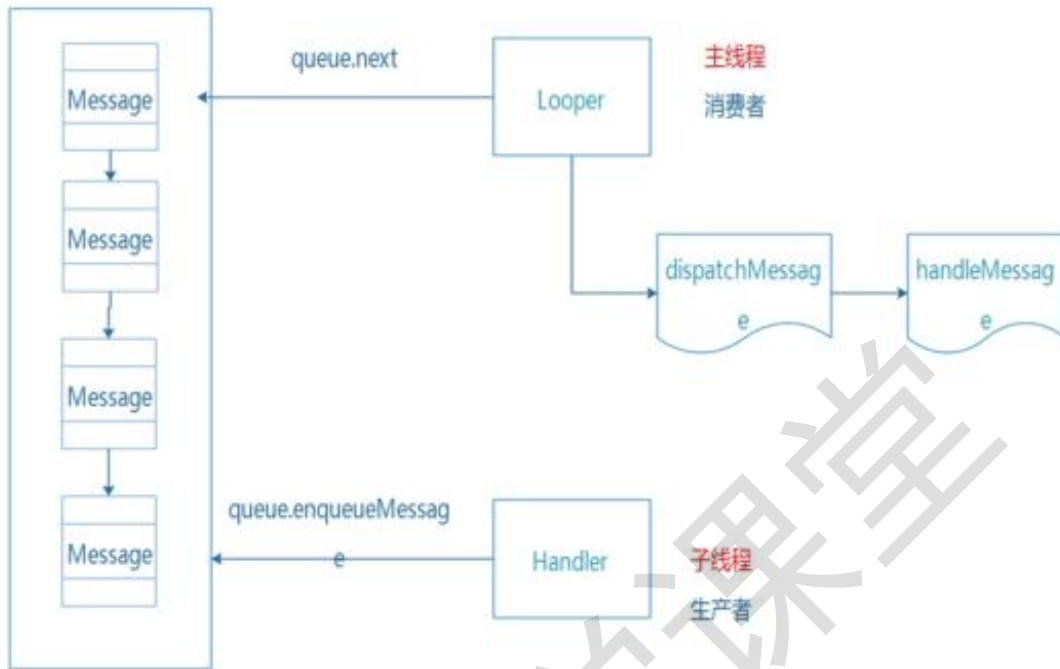
Intent携带信息的大小其实是受Binder限制。数据以Parcel对象的形式存放在Binder传递缓存中。如果数据或返回值比传递buffer大，则此次传递调用失败并抛出TransactionTooLargeException异常。

Binder传递缓存有一个限定大小，通常是1Mb。但同一个进程中所有的传输共享缓存空间。多个地方在进行传输时，即时它们各自传输的数据不超出大小限制，

TransactionTooLargeException异常也可能被抛出。在使用Intent传递数据时，1Mb并不是安全上限。因为Binder中可能正在处理其它的传输工作。不同的机型和系统版本，这个上限值也可能不同。

第三章 Handler解析

在android开发中，经常会在子线程中进行一些操作，当操作完毕后会通过handler发送一些数据给主线程，通知主线程做相应的操作。**探索其背后的原理：**子线程 handler 主线程 其实构成了线程模型中的经典问题 生产者-消费者模型。生产者-消费者模型：生产者和消费者在同一时间段内共用同一个存储空间，生产者往存储空间中添加数据，消费者从存储空间中取走数据。



好处： - 保证数据生产消费的顺序（通过MessageQueue, 先进先出） - 不管是生产者（子线程）还是消费者（主线程）都只依赖缓冲区（handler），生产者消费者之间不会相互持有，使他们之间没有任何耦合

第一节 源码分析

- Handler
 - Handler机制的相关类
 - 创建Looper
 - 创建MessageQueue以及Looper与当前线程的绑定
 - Looper.loop()
 - 创建Handler
 - 创建Message
 - Message和Handler的绑定
 - Handler发送消息
 - Handler处理消息

1.1 Handler机制的相关类

Handler：发送和接收消息 Looper：用于轮询消息队列，一个线程只能有一个Looper Message：消息实体 MessageQueue：消息队列用于存储消息和管理消息

1.2 创建Looper

创建Looper的方法是调用Looper.prepare() 方法

在ActivityThread中的main方法中为我们prepare了

```
public static void main(String[] args) {
```

```

Trace.traceBegin(Trace.TRACE_TAG_ACTIVITY_MANAGER, "ActivityThreadMain");
//其他代码省略...
Looper.prepareMainLooper(); //初始化Looper以及MessageQueue

ActivityThread thread = new ActivityThread();
thread.attach(false);

if (sMainThreadHandler == null) {
    sMainThreadHandler = thread.getHandler();
}

if (false) {
    Looper.myLooper().setMessageLogging(new
        LogPrinter(Log.DEBUG, "ActivityThread"));
}

// End of event ActivityThreadMain.
Trace.traceEnd(Trace.TRACE_TAG_ACTIVITY_MANAGER);
Looper.loop(); //开始轮循操作

throw new RuntimeException("Main thread loop unexpectedly exited");
}

Looper.prepareMainLooper();
public static void prepareMainLooper() {
    prepare(false); //消息队列不可以quit
    synchronized (Looper.class) {
        if (sMainLooper != null) {
            throw new IllegalStateException("The main Looper has already been prepared.");
        }
        sMainLooper = myLooper();
    }
}

prepare有两个重载的方法，主要看 prepare(boolean quitAllowed) quitAllowed的作用是在创建MessageQueue时标识消息队列是否可以销毁，主线程不可被销毁 下面有介绍
public static void prepare() {
    prepare(true); //消息队列可以quit
}
//quitAllowed 主要
private static void prepare(boolean quitAllowed) {
    if (sThreadLocal.get() != null) { //不为空表示当前线程已经创建了Looper
        throw new RuntimeException("Only one Looper may be created per thread");
        //每个线程只能创建一个Looper
    }
    sThreadLocal.set(new Looper(quitAllowed)); //创建Looper并设置给sThreadLocal，这样get
    的时候就不会为null了
}

创建MessageQueue以及Looper与当前线程的绑定
private Looper(boolean quitAllowed) {
    mQueue = new MessageQueue(quitAllowed); //创建了MessageQueue
    mThread = Thread.currentThread(); //当前线程的绑定
}

```

MessageQueue的构造方法

```
MessageQueue(boolean quitAllowed) {  
    //mQuitAllowed决定队列是否可以销毁 主线程的队列不可以被销毁需要传入false, 在MessageQueue的quit()  
    //方法就不贴源码了  
    mQuitAllowed = quitAllowed;  
    mPtr = nativeInit();  
}
```

1.3 Looper.loop()

同时是在main方法中 Looper.prepareMainLooper() 后Looper.loop(); 开始轮询

```
public static void loop() {  
    final Looper me = myLooper(); //里面调用了sThreadLocal.get() 获得刚才创建的Looper对象  
    if (me == null) {  
        throw new RuntimeException("No Looper; Looper.prepare() wasn't called on this thread.  
read.");  
    } //如果Looper为空则会抛出异常  
    final MessageQueue queue = me.mQueue;  
  
    // Make sure the identity of this thread is that of the local process,  
    // and keep track of what that identity token actually is.  
    Binder.clearCallingIdentity();  
    final long ident = Binder.clearCallingIdentity();  
  
    for (;;) {  
        //这是一个死循环, 从消息队列不断的取消息  
        Message msg = queue.next(); // might block  
        if (msg == null) {  
            //由于刚创建MessageQueue就开始轮询, 队列里是没有消息的, 等到Handler sendMessage enqueueMessage后  
            //队列里才有消息  
            // No message indicates that the message queue is quitting.  
            return;  
        }  
  
        // This must be in a local variable, in case a UI event sets the logger  
        Printer logging = me.mLogging;  
        if (logging != null) {  
            logging.println(">>>>> Dispatching to " + msg.target + " " +  
                msg.callback + ": " + msg.what);  
        }  
  
        msg.target.dispatchMessage(msg); //msg.target就是绑定的Handler, 详见后面Message的部分, Handler开始  
        //后面代码省略.....  
  
        msg.recycleUnchecked();  
    }  
}
```


1.4 创建Handler

最常见的创建handler

```
Handler handler=new Handler() {  
    @Override  
    public void handleMessage(Message msg) {  
        super.handleMessage(msg);  
    }  
};
```

在内部调用 this(null, false);

```
public Handler(Callback callback, boolean async) {  
    //前面省略  
    mLooper = Looper.myLooper(); //获取Looper, **注意不是创建Looper**!  
    if (mLooper == null) {  
        throw new RuntimeException(  
            "Can't create handler inside thread that has not called Looper.prepare()");  
    }  
    mQueue = mLooper.mQueue; //创建消息队列MessageQueue  
    mCallback = callback; //初始化了回调接口  
    mAsynchronous = async;  
}  
  
Looper.myLooper();  
//这是Handler中定义的ThreadLocal ThreadLocal主要解多线程并发的问题  
// sThreadLocal.get() will return null unless you've called prepare().  
static final ThreadLocal<Looper> sThreadLocal = new ThreadLocal<Looper>();  
  
public static @Nullable Looper myLooper() {  
    return sThreadLocal.get();  
}
```

sThreadLocal.get() will return null unless you've called prepare(). 这句话告诉我们get可能返回null 除非先调用prepare()方法创建Looper。在前面已经介绍了

1.5 创建Message

可以直接new Message 但是有更好的方式 Message.obtain。因为可以检查是否有可以复用的Message, 用过复用避免过多的创建、销毁Message对象达到优化内存和性能的目的

```
public static Message obtain(Handler h) {  
    Message m = obtain(); //调用重载的obtain方法  
    m.target = h; //并绑定的创建Message对象的handler  
  
    return m;  
}  
  
public static Message obtain() {  
    synchronized (sPoolSync) { //sPoolSync是一个Object对象, 用来同步保证线程安全  
        if (sPool != null) { //sPool是就是handler dispatchMessage 后 通过recycleUnchecked  
            回收用以复用的Message  
            Message m = sPool;  
            sPool = m.next;  
            m.next = null;  
        }
```

```

        m.flags = 0; // clear in-use flag
        sPoolSize--;
        return m;
    }
}
return new Message();
}

```

1.6 Message和Handler的绑定

创建Message的时候可以通过 Message.obtain(Handler h) 这个构造方法绑定。当然可以在 Handler 中的 enqueueMessage() 也绑定了，所有发送Message的方法都会调用此方法入队，所以在创建Message的时候是可以不绑定的

```

private boolean enqueueMessage(MessageQueue queue, Message msg, long uptimeMillis) {
    msg.target = this; //绑定
    if (mAsynchronous) {
        msg.setAsynchronous(true);
    }
    return queue.enqueueMessage(msg, uptimeMillis);
}

```

1.7 Handler发送消息

Handler发送消息的重载方法很多，但是主要只有2种。sendMessage(Message) sendMessage方法通过一系列重载方法的调用，sendMessage调用sendMessageDelayed，继续调用sendMessageAtTime，继续调用enqueueMessage，继续调用messageQueue的enqueueMessage方法，将消息保存在了消息队列中，而最终由Looper取出，交给Handler的dispatchMessage进行处理

我们可以看到在dispatchMessage方法中，message中callback是一个Runnable对象，如果callback不为空，则直接调用callback的run方法，否则判断mCallback是否为空，mCallback在Handler构造方法中初始化，在主线程通过直接通过无参的构造方法new出来的为null，所以会直接执行后面的handleMessage()方法。

```

public void dispatchMessage(Message msg) {
    if (msg.callback != null) {//callback在message的构造方法中初始化或者使用handler.post(Runnable)时候才不为空
        handleCallback(msg);
    } else {
        if (mCallback != null) {//mCallback是一个Callback对象，通过无参的构造方法创建出来的handler，该属性为null，此段不
            if (mCallback.handleMessage(msg)) {
                return;
            }
        }
        handleMessage(msg);//最终执行handleMessage方法
    }
}

private static void handleCallback(Message message) {
    message.callback.run();
}

```

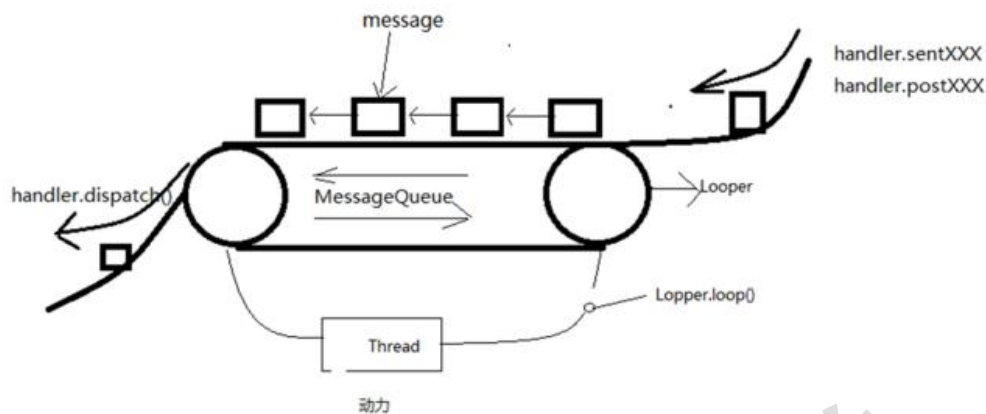
1.8 Handler处理消息

在handleMessage(Message)方法中，我们可以拿到message对象，根据不同的需求进行处理，整个Handler机制的流程就结束了。

1.9小结

handler.sendMessage 发送消息到消息队列MessageQueue，然后looper调用自己的loop()函数带动MessageQueue从而轮询

messageQueue里面的每个Message，当Message达到了可以执行的时间的时候开始执行，执行后就会调用message绑定的Handler来处理消息。大致的过程如下图所示



handler机制就是一个传送带的运转机制。

- 1) MessageQueue就像履带。
- 2) Thread就像背后的动力，就是我们通信都是基于线程而来的。
- 3) 传送带的滚动需要一个开关给电机通电，那么就相当于我们的loop函数，而这个loop里面的for循环就会带着不断的滚动，去轮询messageQueue
- 4) Message就是 我们的货物了。

第二节 难点问题

2.1线程同步问题

Handler是用于线程间通信的，但是它产生的根本并不只是用于UI处理，而更多的是handler是整个app通信的框架，大家可以在ActivityThread里面感受到，整个App都是用它来进行线程间的协调。Handler既然这么重要，那么它的线程安全就至关重要了，那么它是如何保证自己的线程安全呢？

Handler机制里面最主要的类MessageQueue，这个类就是所有消息的存储仓库，在这个仓库中，我们如何的管理好消息，这个就是一个关键点了。消息管理就2点：1) 消息入库（enqueueMessage），2) 消息出库（next），所以这两个接口是确保线程安全的主要档口。

enqueueMessage 源码如下：

```
boolean enqueueMessage(Message msg, long when) {
    if (msg.target == null) {
        throw new IllegalArgumentException("Message must have a target.");
    }
    if (msg.isInUse()) {
        throw new IllegalStateException(msg + " This message is already in use.");
    }

    // 锁开始的地方
    synchronized (this) {
        if (mQuitting) {
            IllegalStateException e = new IllegalStateException(
                msg.target + " sending message to a Handler on a dead thread");
            Log.w(TAG, e.getMessage(), e);
            msg.recycle();
            return false;
        }
    }
}
```

```

    }

    msg.markInUse();
    msg.when = when;
    Message p = mMessages;
    boolean needWake;
    if (p == null || when == 0 || when < p.when) {
        // New head, wake up the event queue if blocked.
        msg.next = p;
        mMessages = msg;
        needWake = mBlocked;
    } else {
        // Inserted within the middle of the queue. Usually we don't have to wake
        // up the event queue unless there is a barrier at the head of the queue
        // and the message is the earliest asynchronous message in the queue.
        needWake = mBlocked && p.target == null && msg.isAsynchronous();
        Message prev;
        for (;;) {
            prev = p;
            p = p.next;
            if (p == null || when < p.when) {
                break;
            }
            if (needWake && p.isAsynchronous()) {
                needWake = false;
            }
        }
        msg.next = p; // invariant: p == prev.next
        prev.next = msg;
    }

    // We can assume mPtr != 0 because mQuitting is false.
    if (needWake) {
        nativeWake(mPtr);
    }
}

//锁结束的地方

```

synchronized锁是一个内置锁，也就是由系统控制锁的lock unlock时机的。在多线程的课程中我们有详细分析过，有问题的同学可以去研究一下。

synchronized (this)

这个锁，说明的是对所有调用同一个MessageQueue对象的线程来说，他们都是互斥的，然而，在我们的Handler里面，一个线程是对应着一个唯一的Looper对象，而Looper中又只有一个唯一的MessageQueue（这个在上文中也有介绍）。所以，我们主线程就只有一个MessageQueue对象，也就是说，所有的子线程向主线程发送消息的时候，主线程一次都只会处理一个消息，其他的都需要等待，那么这个时候消息队列就不会出现混乱。

另外，在看next函数

```

Message next() {

    ....

    for (;;) {
        ....

        nativePollOnce(ptr, nextPollTimeoutMillis);
    }
}

```

```

synchronized (this) {
    // Try to retrieve the next message. Return if found.
    ...
    return msg;
}
} else {
    // No more messages.
    nextPollTimeoutMillis = -1;
}

...
} //synchronized 结束之处

// Run the idle handlers.
// We only ever reach this code block during the first iteration.
for (int i = 0; i < pendingIdleHandlerCount; i++) {
    final IdleHandler idler = mPendingIdleHandlers[i];
    mPendingIdleHandlers[i] = null; // release the reference to the handler

    boolean keep = false;
    try {
        keep = idler.queueIdle();
    } catch (Throwable t) {
        Log.wtf(TAG, "IdleHandler threw exception", t);
    }

    if (!keep) {
        synchronized (this) {
            mIdleHandlers.remove(idler);
        }
    }
}

// Reset the idle handler count to 0 so we do not run them again.
pendingIdleHandlerCount = 0;

// While calling an idle handler, a new message could have been delivered
// so go back and look again for a pending message without waiting.
nextPollTimeoutMillis = 0;
}
}

```

next函数很多同学会有疑问：我从线程里面取消息，而且每次都是队列的头部取，那么它加锁是不是没有意义呢？答案是否定的，我们必须要在next里面加锁，因为，这样由于synchronized (this) 作用范围是所有 this正在访问的代码块都会有保护作用，也就是它可以保证 next函数和 enqueueMessage函数能够实现互斥。这样才能真正的保证多线程访问的时候messagequeue的有序进行。

小结：这个地方是面试官经常问的点，而且他们会基于这个点来拓展问你多线程，所以，这个地方请大家重视。

2. 2消息机制之同步屏障

同步屏障的概念，在Android开发中非常容易被人忽略，因为这个概念在我们普通的开发中太少了，很容易被忽略。

大家经过上面的学习应该知道，线程的消息都是放到同一个MessageQueue里面，取消息的时候是互斥取消息，而且**只能从头部取消息**，而**添加消息是按照消息的执行的先后顺序进行的排序**，那么问题来了，同一个时间范围内的消息，如果它是需要立刻执行的，那我们怎么办，按照常规的办法，我们需要等到队列轮询到我自己的时候才能执行哦，那岂不是黄花菜都凉了。所以，我们需要给紧急需要执行的消息一个绿色通道，这个绿色通道就是同步屏障的概念。

同步屏障是什么？

屏障的意思即为阻碍，顾名思义，**同步屏障就是阻碍同步消息，只让异步消息通过**。如何开启同步屏障呢？如下而已：

MessageQueue#postSyncBarrier()

我们看看它的源码

```
•
/**
 *
 @hide
 */
public int postSyncBarrier() {
    return postSyncBarrier(SystemClock.uptimeMillis());
}
private int postSyncBarrier(long when) {
    // Enqueue a new sync barrier token
    synchronized (this) {
        final int token = mNextBarrierToken++;
        //从消息池中获取Message
        final Message msg = Message.obtain();
        msg.markInUse();

        //就是这里!!! 初始化Message对象的时候，并没有给target赋值，因此 target==null
        msg.when = when;
        msg.arg1 = token;

        Message prev = null;
        Message p = mMessages;

        if (when != 0) {
            while (p != null && p.when <= when) {
                //如果开启同步屏障的时间（假设记为T）T不为0，且当前的同步消息里有时间小于T，则prev
                //也不为null
                prev = p;
                p = p.next;
            }
        }
        //根据prev是不是为null，将 msg 按照时间顺序插入到 消息队列（链表）的合适位置
        if (prev != null) { // invariant: p == prev.next
            msg.next = p;
            prev.next = msg;
        } else {
            msg.next = p;
            mMessages = msg;
        }
        return token;
    }
}
```

可以看到，Message 对象初始化的时候并没有给 target 赋值，因此，target == null的 来源就找到了。上面消息的插入也做了相应的注释。这样，一条target == null 的消息就进入了消息队列。

那么，开启同步屏障后，所谓的异步消息又是如何被处理的呢？

如果对消息机制有所了解的话，应该知道消息的最终处理是在消息轮询器`Looper#loop()`中，而`loop()`循环中会调用`MessageQueue#next()`从消息队列中进行取消息。

`//MessageQueue.java`

`Message next()`

`.....//省略一些代码`

`int pendingIdleHandlerCount = -1; // -1 only during first iteration`

`// 1. 如果nextPollTimeoutMillis=-1，一直阻塞不会超时。`

`// 2. 如果nextPollTimeoutMillis=0，不会阻塞，立即返回。`

`// 3. 如果nextPollTimeoutMillis>0，最长阻塞nextPollTimeoutMillis毫秒(超时)`

`// 如果期间有程序唤醒会立即返回。`

`int nextPollTimeoutMillis = 0;`

`//next()也是一个无限循环`

`for (;;) {`

`if (nextPollTimeoutMillis != 0) {`

`Binder.flushPendingCommands();`

`}`

`nativePollOnce(ptr, nextPollTimeoutMillis);`

`synchronized (this) {`

`//获取系统开机到现在的时间`

`final long now = SystemClock.uptimeMillis();`

`Message prevMsg = null;`

`Message msg = mMessages; //当前链表的头结点`

`//关键!!!`

`//如果target==null，那么它就是屏障，需要循环遍历，一直往后找到第一个异步的消息`

`if (msg != null && msg.target == null) {`

`// Stalled by a barrier. Find the next asynchronous message in the queue.`

`do {`

`prevMsg = msg;`

`msg = msg.next;`

`} while (msg != null && !msg.isAsynchronous());`

`}`

`if (msg != null) {`

`//如果有消息需要处理，先判断时间有没有到，如果没到的话设置一下阻塞时间，`

`//场景如常用的postDelay`

`if (now < msg.when) {`

`//计算出离执行时间还有多久赋值给nextPollTimeoutMillis，`

`//表示nativePollOnce方法要等待nextPollTimeoutMillis时长后返回`

`nextPollTimeoutMillis = (int) Math.min(msg.when - now, Integer.MAX_VALU`

E);

`} else {`

`// 获取到消息`

`mBlocked = false;`

`//链表操作，获取msg并且删除该节点`

`if (prevMsg != null)`

`prevMsg.next = msg.next;`

`} else {`

`mMessages = msg.next;`

`}`

`msg.next = null;`

`msg.markInUse();`

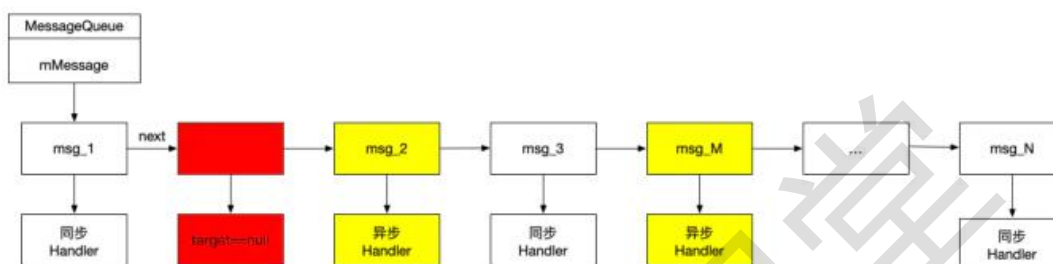
```

        //返回拿到的消息
        return msg;
    }
} else {
    //没有消息，nextPollTimeoutMillis复位
    nextPollTimeoutMillis = -1;
}
.....//省略
}

```

从上面可以看出，当消息队列开启同步屏障的时候（即标识为`msg.target == null`），消息机制在处理消息的时候，优先处理异步消息。这样，同步屏障就起到了一种过滤和优先级的作用。

下面用示意图简单说明：



如上图所示，在消息队列中有同步消息和异步消息（黄色部分）以及一道墙——同步屏障（红色部分）。有了同步屏障的存在，`msg2` 和 `msgM` 这两个异步消息可以被优先处理，而后面的 `msg_3` 等同步消息则不会被处理。那么这些同步消息什么时候可以被处理呢？那就需要先移除这个同步屏障，即调用`removeSyncBarrier()`。

同步消息的应用场景

似乎在日常的应用开发中，很少会用到同步屏障。那么，同步屏障在系统源码中有哪些使用场景呢？Android 系统中的 UI 更新相关的消息即为异步消息，需要优先处理。

比如，在 View 更新时，`draw`、`requestLayout`、`invalidate` 等很多地方都调用了`ViewRootImpl#scheduleTraversals()`，如下：

```

//ViewRootImpl.java
void scheduleTraversals() {
    if (!mTraversalScheduled) {
        mTraversalScheduled = true;
        //开启同步屏障
        mTraversalBarrier = mHandler.getLooper().getQueue().postSyncBarrier();
        //发送异步消息
        mChoreographer.postCallback(
            Choreographer.CALLBACK_TRAVERSAL, mTraversalRunnable, null);
        if (!mUnbufferedInputDispatch) {
            scheduleConsumeBatchedInput();
        }
        notifyRendererOfFramePending();
        pokeDrawLockIfNeeded();
    }
}

```

```

postCallback() 最终走到了ChoreographerpostCallbackDelayedInternal():
private void postCallbackDelayedInternal(int callbackType,
    Object action, Object token, long delayMillis) {
    if (DEBUG_FRAMES) {
        Log.d(TAG, "PostCallback: type=" + callbackType + ", action=" + action + ",
            token=" + token + " = " + delayMillis);
    }
}

```



```

    synchronized (mLock) {
        final long now = SystemClock.uptimeMillis();
        final long dueTime = now + delayMillis;
        mCallbackQueues[callbackType].addCallbackLocked(dueTime, action, token);

        if (dueTime <= now) {
            scheduleFrameLocked(now);
        } else {
            Message msg = mHandler.obtainMessage(MSG_DO_SCHEDULE_CALLBACK, action);
            msg.arg1 = callbackType;
            msg.setAsynchronous(true); //异步消息
            mHandler.sendMessageAtTime(msg, dueTime);
        }
    }
}

```

这里就开启了同步屏障，并发送异步消息，由于 UI 更新相关的消息是优先级最高的，这样系统就会优先处理这些异步消息。

最后，当要移除同步屏障的时候需要调用ViewRootImpl#unscheduleTraversals()。

```

void unscheduleTraversals() {
    if (mTraversalScheduled) {
        mTraversalScheduled = false;
        //移除同步屏障
        mHandler.getLooper().getQueue().removeSyncBarrier(mTraversalBarrier);
        mChoreographer.removeCallbacks(
            Choreographer.CALLBACK_TRAVERSAL, mTraversalRunnable, null);
    }
}

```

2.3小结

同步屏障的设置可以方便地处理那些优先级较高的异步消息。当我们调用Handler.getLooper().getQueue().postSyncBarrier() 并设置消息的setAsynchronous(true)时，target 即为 null，也就开启了同步屏障。当在消息轮询器 Looper 在loop()中循环处理消息时，如若开启了同步屏障，会优先处理其中的异步消息，而阻碍同步消息。

第三节Handler常问面试题

请大家研究一下，课程中会重点讲解。

1. 一个线程有几个 Handler?
2. 一个线程有几个 Looper? 如何保证?
3. Handler内存泄漏原因? 为什么其他的内部类没有说过有这个问题?
4. 为何主线程可以new Handler? 如果想要在子线程中新 Handler 要做什么准备?
5. 子线程中维护的Looper，消息队列无消息的时候的处理方案是什么? 有什么用?
6. 既然可以存在多个 Handler 往 MessageQueue 中添加数据（发消息时各个 Handler 可能处于不同线程），那它内部是如何确保线程安全的? 取消息呢?
7. 我们使用 Message 时应该如何创建它?
8. Looper死循环为什么不会导致应用卡死

腾讯享学课堂B站Hanler解析教学地址:

https://www.bilibili.com/video/BV1zM4y1u7j8?from=search&seid=319986098539213223&spm_id_from=333.337.0.0

3.1 HandlerThread是什么? 为什么它会存在?

作为一个Android开发，Handler机制是一定要了解的。在我面试过程中，发现很多人对Handler和Looper机制非常了解，对答如流，但是却不知道何为HandlerThread

HandlerThread是Thread的子类，严格意义上来说就是一个线程，只是它在自己的线程里面帮我们创建了Looper

HandlerThread 存在的意义如下:

- 1) 方便使用: a. 方便初始化, b. 方便获取线程looper

2) 保证了线程安全

我们一般在Thread里面 线程Looper进行初始化的代码里面, 必须要对Looper.prepare(), 同时要调用Looper.loop();

```
@Override
public void run() {
    Looper.prepare();
    Looper.loop();
}
```

而我们要使用子线程中的Looper的方式是怎样的呢? 看下面的代码

```
Thread thread = new Thread(new Runnable() {
    Looper looper;
    @Override
    public void run() {
        // Log.d(TAG, "click2: " + Thread.currentThread().getName());
        Looper.prepare();
        looper = Looper.myLooper();
        Looper.loop();
    }

    public Looper getLooper() {
        return looper;
    }
});
thread.start();
Handler handler = new Handler(thread.getLooper());
```

上面这段代码有没有问题呢?

1) 在初始化子线程的handler的时候, 我们无法将子线程的looper传值给Handler, 解决办法有如下办法:

- 可以将Handler的初始化放到 Thread里面进行
- 可以创建一个独立的类继承Thread, 然后, 通过类的对象获取。

这两种办法都可以, 但是, 这个工作 HandlerThread帮我们完成了

2) 依据多线程的工作原理, 我们在上面的代码中, 调用 thread.getLooper() 的时候, 此时的looper可能还没有初始化, 此时是不是可能会挂掉呢?

以上问题

HandlerThread 已经帮我们完美的解决了, 这就是 handlerThread存在的必要性了。

我们再看 HandlerThread源码

```
public void run() {
    mTid = Process.myTid();
    Looper.prepare();
    synchronized (this) {
        mLooper = Looper.myLooper();
        notifyAll(); //此时唤醒其他等待的锁, 但是
    }
    Process.setThreadPriority(mPriority);
    onLooperPrepared();
    Looper.loop();
    mTid = -1;
}
```

它的优点就在于它的多线程操作, 可以帮我们保证使用Thread的handler时一定是安全的。

3.2简述下 Handler 机制的总体原理？

1. Looper 准备和开启轮循：

Looper#prepare() 初始化线程独有的 Looper 以及 MessageQueue Looper#loop() 开启死循环读取 MessageQueue 中下一个满足执行时间的 Message 尚无 Message 的话，调用 Native 侧的 pollOnce() 进入无限等待 存在 Message，但执行时间 when 尚未满足的话，调用 pollOnce() 时传入剩余时长参数进入有限等待

1. Message 发送、入队和出队：

Native 侧如果处于无限等待的话：任意线程向 Handler 发送 Message 或 Runnable 后，Message 将按照 when 条件的先后，被插入 Handler 持有的 Looper 实例所对应的 MessageQueue 中适当的位置。MessageQueue 发现有合适的 Message 插入后将调用 Native 侧的 wake() 唤醒无限等待的线程。这将促使 MessageQueue 的读取继续进入下一次循环，此刻 Queue 中已有满足条件的 Message 则出队返回给 Looper Native 侧如果处于有限等待的话：在等待指定时长后 epoll_wait 将返回。线程继续读取 MessageQueue，此刻因为时长条件将满足将其出队 Looper 处理 Message 的实现：

1. Looper 得到 Message 后回调 Message 的 callback 属性即 Runnable，或依据 target 属性即 Handler，去执行 Handler 的回调。

存在 mCallback 属性的话回调 Handler\$Callback 反之，回调 handleMessage()

3.3 Looper 存在哪？如何可以保证线程独有？

Looper 实例被管理在静态属性 sThreadLocal 中 ThreadLocal 内部通过 ThreadLocalMap 持有 Looper，key 为 ThreadLocal 实例本身，value 即为 Looper 实例 每个 Thread 都有一个自己的 ThreadLocalMap，这样可以保证每个线程对应一个独立的 Looper 实例，进而保证 myLooper() 可以获得线程独有的 Looper 彩蛋：一个 App 拥有几个 Looper 实例？几个 ThreadLocal 实例？几个 MessageQueue 实例？几个 Message 实例？几个 Handler 实例

一个线程只有一个 Looper 实例 一个 Looper 实例只对应着一个 MessageQueue 实例 一个 MessageQueue 实例可对应多个 Message 实例，其从 Message 静态池里获取，存在 50 的上限 一个线程可以拥有多个 Handler 实例，其Handler 只是发送和执行任务逻辑的入口和出口 ThreadLocal 实例是静态的，整个进程共用一个实例。每个 Looper 存放的 ThreadLocalMap 均弱引用它作为 key

3.4如何理解 ThreadLocal 的作用？

首先要明确并非不是用来切换线程的，只是为了让每个线程方便获取自己的 Looper 实例，见 Looper#myLooper() 后续可供 Handler 初始化时指定其所属的 Looper 线程 也可用来线程判断自己是否是主线程

3.5主线程 Main Looper 和一般 Looper 的异同？

区别：

Main Looper 不可 quit 主线程需要不断读取系统消息和用书输入，是进程的入口，只可被系统直接终止。进而其 Looper 在创建的时候设置了不可 quit 的标志，而其他线程的 Looper 则可以也必须手动 quit

Main Looper 实例还被静态缓存 为了便于每个线程获得主线程 Looper 实例，见 Looper#getMainLooper()，Main Looper 实例还作为 sMainLooper 属性缓存到了 Looper 类中。

相同点：

都是通过 Looper#prepare() 间接调用 Looper 构造函数创建的实例 都被静态实例 ThreadLocal 管理，方便每个线程获取自己的 Looper 实例 彩蛋：主线程为什么不用初始化 Looper？

App 的入口并非 MainActivity，也不是 Application，而是 ActivityThread。

其为了 Application、ContentProvider、Activity 等组件的运行，必须先启动不停接受输入的 Looper 机制，所以在 main() 执行的最后将调用 prepareMainLooper() 创建 Looper 并调用 loop() 轮循。

不需要我们调用，也不可能由我们调用。

可以说如果主线程没有创建 Looper 的话，我们的组件也不可能运行得到！

3.6 Handler 或者说 Looper 如何切换线程？

Handler 创建的时候指定了其所属线程的 Looper，进而持有了 Looper 独有的 MessageQueue
Looper#loop() 会持续读取 MessageQueue 中合适的 Message，没有 Message 的时候进入等待
当向 Handler 发送 Message 或 Runnable 后，会向持有的 MessageQueue 中插入 Message
Message 抵达并满足条件后会唤醒 MessageQueue 所属的线程，并将 Message 返回给 Looper
Looper 接着回调 Message 所指向的 Handler Callback 或 Runnable，达到线程切换的目的

简言之，向 Handler 发送 Message 其实是向 Handler 所属线程的独有 MessageQueue 插入 Message。而线程独有的 Looper 又会持续读取该 MessageQueue。所以向其他线程的 Handler 发送完 Message，该线程的 Looper 将自动响应。

3.7 Looper 的 loop() 死循环为什么不卡死？

为了让主线程持续处理用户的输入，loop() 是死循环，持续调用 MessageQueue#next() 读取合适的 Message。

但当没有 Message 的时候，会调用 pollOnce() 并通过 Linux 的 epoll 机制进入等待并释放资源。同时 eventFd 会监听 Message 抵达的写入事件并进行唤醒。

这样可以空闲时释放资源、不卡死线程，同时能持续接收输入的目的。

彩蛋1: loop() 后的处理为什么不可执行

因为 loop() 是死循环，直到 quit 前后面的处理无法得到执行，所以避免将处理放在 loop() 的后面。

彩蛋2: Looper 等待的时候线程到底是什么状态？

调用 Linux 的 epoll 机制进入等待，事实上 Java 侧打印该线程的状态，你会发现线程处于 Runnable 状态，只不过 CPU 资源被暂时释放。

3.8 Looper 的等待是如何能够准确唤醒的？

读取合适 Message 的 MessageQueue#next() 会因为 Message 尚无或执行条件尚未满足进行两种等的等待：

无限等待

尚无 Message（队列中没有 Message 或建立了同步屏障但尚无异步 Message）的时候，调用 Native 侧的 pollOnce() 会传入参数 -1。

Linux 执行 epoll_wait() 将进入无限等待，其等待合适的 Message 插入后调用 Native 侧的 wake() 向唤醒 fd 写入事件触发唤醒 MessageQueue 读取的下一循环

有限等待

有限等待的场合将下一个 Message 剩余时长作为参数交给 epoll_wait()，epoll 将等待一段时间之后自动返回，接着回到 MessageQueue 读取的下一循环

3.9 Message 如何获取？为什么这么设计？

享元设计模式：通过 Message 的静态方法 obtain() 获取，因为该方法不是无脑地 new，而是从单链表池子里获取实例，并在 recycle() 后将其放回池子

好处在于复用 Message 实例，满足频繁使用 Message 的场景，更加高效

当然，缓存池存在上限 50，因为没必要无限制地缓存，这本身也是一种浪费

需要留意缓存池是静态的，也就是整个进程共用一个缓存池

3.10 MessageQueue 如何管理 Message？

MessageQueue 通过单链表管理 Message，不同于进程共用的 Message Pool，其是线程独有的 通过 Message 的执行时刻 when 对 Message 进行排队和出队 MessageQueue 除了管理 Message，还要管理空闲 Handler 和 同步屏障

3.11 理解 Message 和 MessageQueue 的异同？

相同点：都是通过单链表来管理 Message 实例；

Message 通过 obtain() 和 recycle() 向单链表获取插入节点

MessageQueue 通过 enqueueMessage() 和 next() 向单链表获取和插入节点

区别：

Message 单链表是静态的，供进程使用的缓存池 MessageQueue 单链表非静态，只供 Looper 线程使用

3.12 Message 的执行时刻如何管理？

发送的 Message 都是按照执行时刻 when 属性的先后管理在 MessageQueue 里 延时 Message 的 when 等于调用的当前时刻和 delay 之和 非延时 Message 的 when 等于当前时刻 (delay 为 0) 插队 Message 的 when 固定为 0，便于插入队列的 head 之后 MessageQueue 会根据读取的时刻和 when 进行比较 将 when 已抵达的出队，尚未抵达的计算出当前时刻和目标 when 的插值，交由 Native 等待对应的时长，时间到了自动唤醒继续进行 Message 的读取 事实上，无论上述哪种 Message 都不能保证在其对应的 when 时刻执行，往往都会延迟一些！因为必须等当前执行的 Message 处理完了才有机会读取队列的下一个 Message。

比如发送了非延时 Message，when 即为发送的时刻，可它们不会立即执行。都要等主线程现有的任务 (Message) 走完才能有机会出队，而当这些任务执行完 when 的时刻已经过了。假使队列的前面还有其他 Message 的话，延迟会更加明显！

彩蛋：. onCreate() 里向 Handler 发送大量 Message 会导致主线程卡顿吗？

不会，发送的大量 Message 并非立即执行，只是先放到队列当中而已。

onCreate() 以及之后同步调用的 onStart() 和 onResume() 处理，本质上也是 Message。等这个 Message 执行完之后，才会进行读取 Message 的下次循环，这时候才能回调 onCreate 里发送的 Message。

需要说明的是，如果发送的是 FrontOfQueue 将 Message 插入队首也不会立即先执行，因为 onStart 和 onResume 是 onCreate 之后同步调用的，本质上是同一个 Message 的作业周期

3.13 Handler、Message 和 Runnable 的关系如何理解？

作为使用 Handler 机制的入口，Handler 是发送 Message 或 Runnable 的起点 发送的 Runnable 本质上也是 Message，只不过作为 callback 属性被持有 Handler 作为 target 属性被持有在 Message 中，在 Message 执行条件满足的时候供 Looper 回调 事实上，Handler 只是供 App 使用 Handler 机制的 API，实质来说，Message 是更为重要的载体。

3.14 IdleHandler 空闲 Message 了解过吗？有什么用？

适用于期望空闲时候执行，但不影响主线程操作的任务

系统应用：

Activity destroy 回调就放在了 IdleHandler 中 ActivityThread 中 GCHandler 使用了 IdleHandler，在空闲的时候执行 GC 操作 App 应用：

发送一个返回 true 的 IdleHandler，在里面让某个 View 不停闪烁，这样当用户发呆时就可以诱导用户点击这个 View 将某部分初始化放在 IdleHandler 里不影响 Activity 的启动 彩蛋问题：

add/remove IdleHandler 的方法，是否需要成对使用？ 不需要，回调返回 false 也可以移除

当 mIdleHandlers 一直不为空时，为什么不会进入死循环？ 执行过 IdleHandler 之后会将计数重置为 0，确保下一次循环不重复执行

是否可以将一些不重要的启动服务，搬移到 IdleHandler 中去处理？ 最好不要，回调时机不太可控，需要搭配 remove 谨慎使用

IdleHandler 的 queueIdle() 运行在那个线程？ 取决于 IdleHandler add 到的 MessageQueue 所处的线程

3.15 异步 Message 或同步屏障了解过吗？怎么用？什么原理？

异步 Message：设置了 isAsync 属性的 Message 实例

可以用异步 Handler 发送 也可以调用 Message#setAsynchronous() 直接设置为异步 Message 同步屏障：在 MessageQueue 的某

个位置放一个 target 属性为 null 的 Message，确保此后的非异步 Message 无法执行，只能执行异步 Message

原理：当 MessageQueue 轮循 Message 时候发现建立了同步屏障的时候，会去跳过其他 Message，读取下个 async 的 Message 并执行，屏障移除之前同步 Message 都会被阻塞

应用：比如屏幕刷新 Choreographer 就使用到了同步屏障，确保屏幕刷新事件不会因为队列负荷影响屏幕及时刷新。

注意：同步屏障的添加或移除 API 并未对外公开，App 需要使用的话需要依赖反射机制

3.16 Looper 和 MessageQueue、Message 及 Handler 的关系？

Message 是承载任务的载体，在 Handler 机制中贯穿始终 Handler 则是对外公开的 API，负责发送 Message 和处理任务的回调，是 Message 的生产者 MessageQueue 负责管理待处理 Message 的入队和出队，是 Message 的容器 Looper 负责轮循 MessageQueue，保持线程持续运行任务，是 Message 的消费者 彩蛋：如何保证 MessageQueue 并发访问安全？

任何线程都可以通过 Handler 生产 Message 并放入 MessageQueue 中，可 Queue 所属的 Looper 在持续地读取并尝试消费 Message。如何保证两者不产生死锁？

Looper 在消费 Message 之前要先拿到 MessageQueue 的锁，**只不过没有 Message 或 Message 尚未满足条件的进行等待前会事先释放锁**，具体在于 nativePollOnce() 的调用在 synchronized 方法块的外侧。

Message 入队前也需先拿到 MessageQueue 的锁，而这时 Looper 线程正在等待且不持有锁，可以确保 Message 的成功入队。入队后执行唤醒后释放锁，Native 收到 event 写入后恢复 MessageQueue 的读取并可以拿到锁，成功出队。

这样一种在没有 Message 可以消费时执行等待同时不占着锁的机制，避免了生产和消费的死锁。

```
boolean enqueueMessage(Message msg, long when) {
    synchronized (this) {
        ...
    }
    return true;
}

Message next() {
    ...
    for (;;) {
        nativePollOnce(ptr, nextPollTimeoutMillis);
        synchronized (this) {
            ...
        }
        ...
    }
}

Message next() { ... for (;;) { nativePollOnce(ptr, nextPollTimeoutMillis); synchronized (this)
{ ... } ... } }
```

3.17 Native 侧的 NativeMessageQueue 和 Looper 的作用？

NativeMessageQueue 负责连接 Java 侧的 MessageQueue，进行后续的 wait 和 wake，后续将创建 wake 的FD，并通过 epoll 机制等待或唤醒。但并不参与管理 Java 的 Message

Native 侧也需要 Looper 机制，等待和唤醒的需求是同样的，所以将这部分实现都封装到了 JNI 的NativeMessageQueue 和 Native 的 Looper 中，供 Java 和 Native 一起使用

3.18 Native 侧如何使用 Looper？

Looper Native 部分承担了 Java 侧 Looper 的等待和唤醒，除此之外还提供了 Message、MessageHandler 或 WeakMessageHandler、LooperCallback 或 SimpleLooperCallback 等 API

这些部分可供 Looper 被 Native 侧直接调用，比如 InputFlinger 广泛使用了 Looper

主要方法是调用 Looper 构造函数或 prepare 创建 Looper，然后通过 poll 开始轮询，接着 sendMessage 或 addEventFd，等待 Looper 的唤醒。使用过程和 Java 的调用思路类似

3.19 Handler 为什么可能导致内存泄露？如何避免？

持有 Activity 实例的内名内部类或内部类的生命周期应当和 Activity 保持一致，否则产生内存泄露的风险。如果 Handler 使用不当，将造成不一致，表现为：匿名内部类或内部类写法的 Handler、Handler\$Callback、Runnable，或者 Activity 结束时仍有活跃的 Thread 线程或 Looper 子线程。具体在于：异步任务仍然活跃或通过发送的 Message 尚未处理完毕，将使得内部类实例的生命周期被错误地延长。造成本该回收的 Activity 实例被别的 Thread 或 Main Looper 占据而无法及时回收（活跃的 Thread 或静态属性 sMainLooper 是 GC Root 对象）建议的做法：无论是 Handler、Handler\$Callback 还是 Runnable，尽量采用静态内部类 + 弱引用的写法，确保尽管发生不当引用的时候也可以因为弱引用能清楚持有关系。另外在 Activity 销毁的时候及时地终止 Thread、停止子线程的 Looper 或清空 Message，确保彻底切断 Activity 经由 Message 抵达 GC Root 的引用源头（Message 清空后会其于 Handler 的引用关系，Thread 的终止将结束其 GC Root 的源头）注意：静态的 sThreadLocal 实例不持有存放 Looper 实例的 ThreadLocalMap，而是由 Thread 持有。从这个角度上来讲，Looper 会被活跃的 GC Root Thread 持有，进而也可能导致内存泄露。

彩蛋：网传的 Handler\$Callback 方案能否解决内存泄露？

不能。

Callback 采用内部类或匿名内部类写法的话，默认持有 Activity 的引用，而 Callback 被 Handler 持有。这最终将导致 Message -> Handler -> Callback -> Activity 的链条仍然存在。

3.20 Handler 在系统当中的应用

特别广泛，比如：

Activity 生命周期的管理 屏幕刷新 HandlerThread、IntentService AsyncTask 等。主要利用 Handler 的切换线程、主线程异步 Message 的重要特性。注意：Binder 线程非主线程，但很多操作比如生命周期的管理都要回到主线程，所以很多 Binder 调用过来后都要通过 Handler 切换回主线程执行后续任务，比如 ActivityThread\$H 就是 extends Handler。

3.21 Android 为什么不允许并发访问 UI？

Android 中 UI 非线程安全，并发访问的话会造成数据和显示错乱。

但此限制的检查始于 ViewRootImpl#checkThread()，其会在刷新等多个访问 UI 的时机被调用，去检查当前线程，非主线程的话抛出异常。

而 ViewRootImpl 的创建在 onResume() 之后，也就是说如果在 onResume() 执行前启动线程访问 UI 的话是不会报错的，这点需要注意！

彩蛋：onCreate() 里子线程更新 UI 有问题吗？为什么？

不会。

因为异常的检测处理在 ViewRootImpl 中，该实例的创建和检测在 onResume() 之后进行。

让我们共同维护这几十个问题，彻底吃透 Handler 机制！

如果对 Message 创建和复用的细节感兴趣，可以查阅《Handler 的 Message 实例怎么创建，为什么不是直接 new？》。如果并不确定 Message 有多少种、已经背后执行的细节，或者对空闲 Handler、同步屏障等特殊用法不了解的话，建议阅读《万字复盘 Handler 中各式 Message 的使用和原理》。如果对 Looper quit 的理由和背后原理好奇的话，可以参考《Looper 需要手动 quit，那主线程 Looper 呢？》。如果对于 Handler 内存泄漏的问题存在疑惑，想要一次性搞清楚的话，可以试着看看这篇文章：《重新理解为什么 Handler 可能导致内存泄露？》。

本章知识点解析由享学课堂提供，面试题解析由作者 TechMerger 提供， 原文链接：

<https://blog.csdn.net/allisonchen/article/details/120618884>

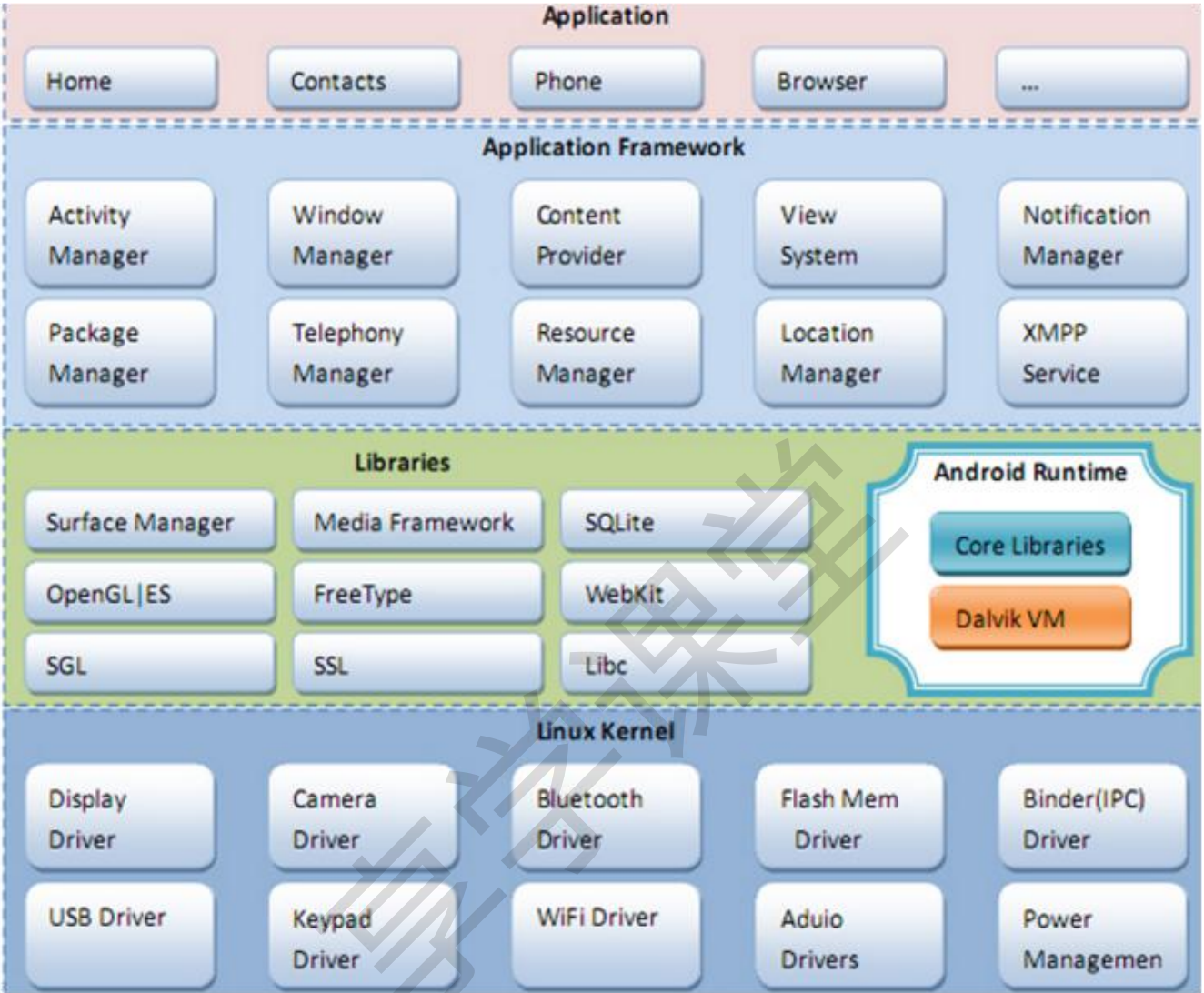
第四章 AMS 解析

第一节 引言

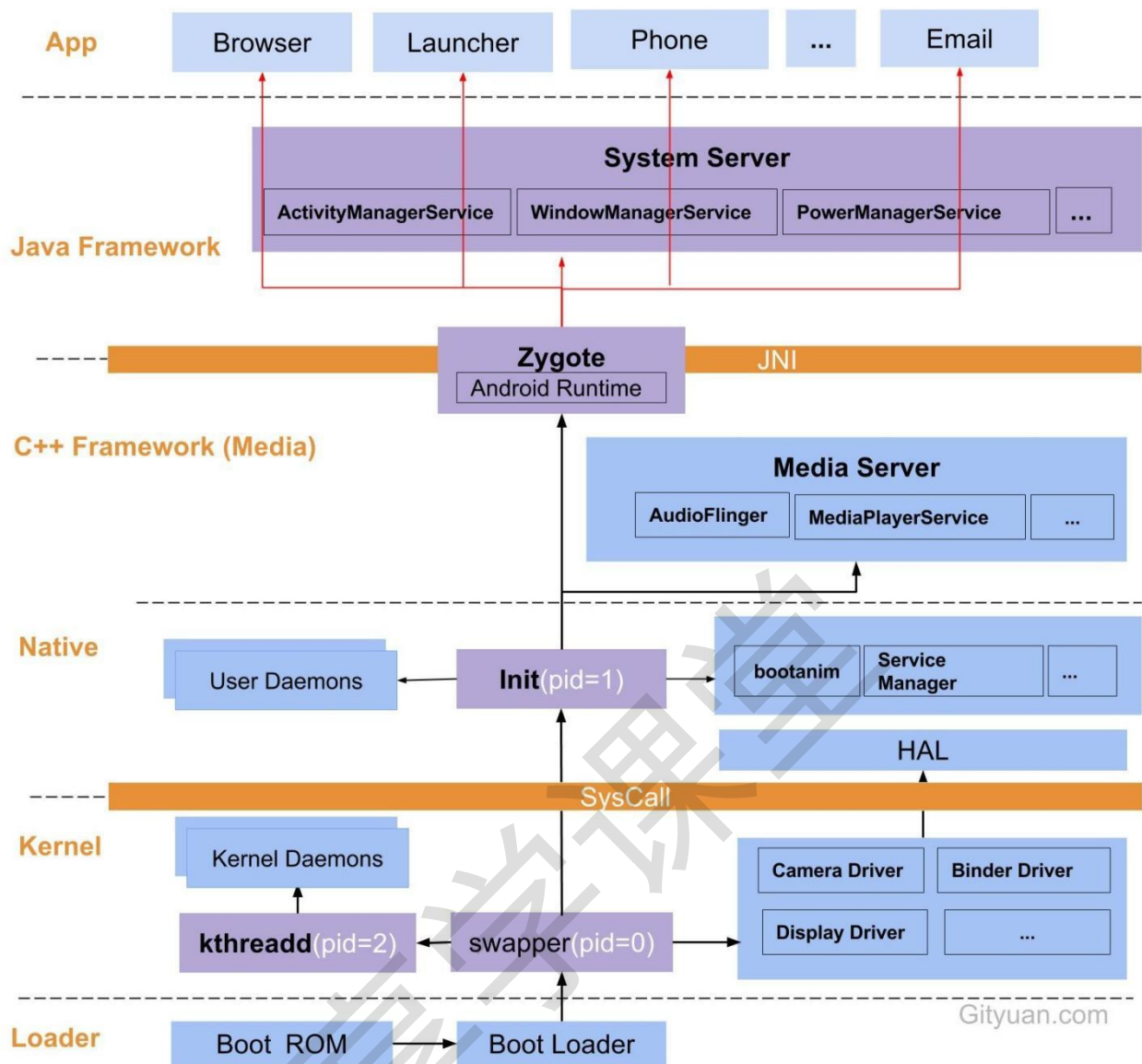
Android 系统非常庞大、错综复杂，其底层是采用 Linux 作为基底，上层采用包含虚拟机的 Java 层以及 Native 层，通过系统调用 (Syscall) 连通系统的内核空间与用户空间。用户空间主要采用 C++ 和 Java 代码，通过 JNI 技术打通用户空间的 Java 层和 Native 层

(C++/C)，从而融为一体。

Google官方提供了一张经典的四层架构图，从下往上依次分为Linux内核、系统库和Android运行时环境、框架层以及应用层这4层架构，其中每一层都包含大量的子模块或子系统



第二节 Android架构



图解，Android系统启动过程由上图从下往上的一个过程：Loader -> Kernel -> Native -> Framework -> App，接下来简要说说每个过程：

• Loader层

Boot ROM：当手机处于关机状态时，长按Power键开机，引导芯片开始从固化在ROM里的预设出代码开始执行，然后加载引导程序到RAM；

Boot Loader：这是启动Android系统之前的引导程序，主要是检查RAM，初始化硬件参数等功能。

• Kernel层

Kernel层是指Android内核层，到这里才刚刚开始进入Android系统。

启动Kernel的swapper进程(pid=0)：该进程又称为idle进程，系统初始化过程Kernel由无到有开创的第一个进程，用于初始化进程管理、内存管理，加载Display, Camera Driver, Binder Driver等相关工作；

启动kthreadd进程(pid=2)：是Linux系统的内核进程，会创建内核工作线程kworkder，软中断线程

ksoftirqd, thermal等内核守护进程。kthreadd进程是所有内核进程的鼻祖。

- Native层

这里的Native层主要包括init孵化来的用户空间的守护进程、HAL层以及开机动画等。启动init进程(pid=1), 是

Linux系统的用户进程, init进程是所有用户进程的鼻祖。

init进程会孵化出ueventd、logd、healthd、installd、adbd、lmkd等用户守护进程; init进程还启动servicemanager(binder服务管家)、bootanim(开机动画)等重要服务

init进程孵化出Zygote进程, Zygote进程是Android系统的第一个Java进程(即虚拟机进程), Zygote是所有Java进程的父进程, Zygote进程本身是由init进程孵化而来的。

- Framework层

- Zygote进程, 是由init进程通过解析init.rc文件后fork生成的, Zygote进程主要包含: 加载

- ZygoteInit类, 注册Zygote Socket服务端套接字;

- 加载虚拟机;

- preloadClasses;

- preloadResources.

System Server进程, 是由Zygote进程fork而来, System Server是Zygote孵化的第一个进程, System Server负责启动和管理整个Java framework, 包含ActivityManager, PowerManager等服务。

Media Server进程, 是由init进程fork而来, 负责启动和管理整个C++ framework, 包含AudioFlinger, Camera Service, 等服务。

- App层

Zygote进程孵化出的第一个App进程是Launcher, 这是用户看到的桌面App;

Zygote进程还会创建Browser, Phone, Email等App进程, 每个App至少运行在一个进程上。所有的App进程都是由Zygote进程fork生成的。

- Syscall && JNI

Native与Kernel之间有一层系统调用(SysCall)层, 见Linux系统调用(Syscall)原理;

Java层与Native(C/C++)层之间的纽带JNI, 见Android JNI原理分析。

第三节 通信方式

无论是Android系统, 还是各种Linux衍生系统, 各个组件、模块往往运行在各种不同的进程和线程内, 这里就必然涉及进程/线程之间的通信。对于IPC(Inter-Process Communication, 进程间通信), Linux现有管道、消息队列、共享内存、套接字、信号量、信号这些IPC机制, Android额外还有Binder IPC机制, Android OS中的Zygote进程的IPC采用的是Socket机制, 在上层system server、media server以及上层App之间更多的是采用Binder IPC方式来完成跨进程间的通信。对于Android上层架构中, 很多时候是在同一个进程的线程之间需要相互通信, 例如同一个进程的主线程与工作线程之间的通信, 往往采用的Handler消息机制。

想深入了解Android内核层架构, 必须先深入理解Linux现有的IPC机制; 对于Android上层架构, 则最常用的通信方式是Binder、Socket、Handler, 当然也有少量其他的IPC方式, 比如杀进程Process.killProcess() 采用的是signal方式。下面说说Binder、Socket、Handler:

20 为何Android要采用Binder作为IPC机制

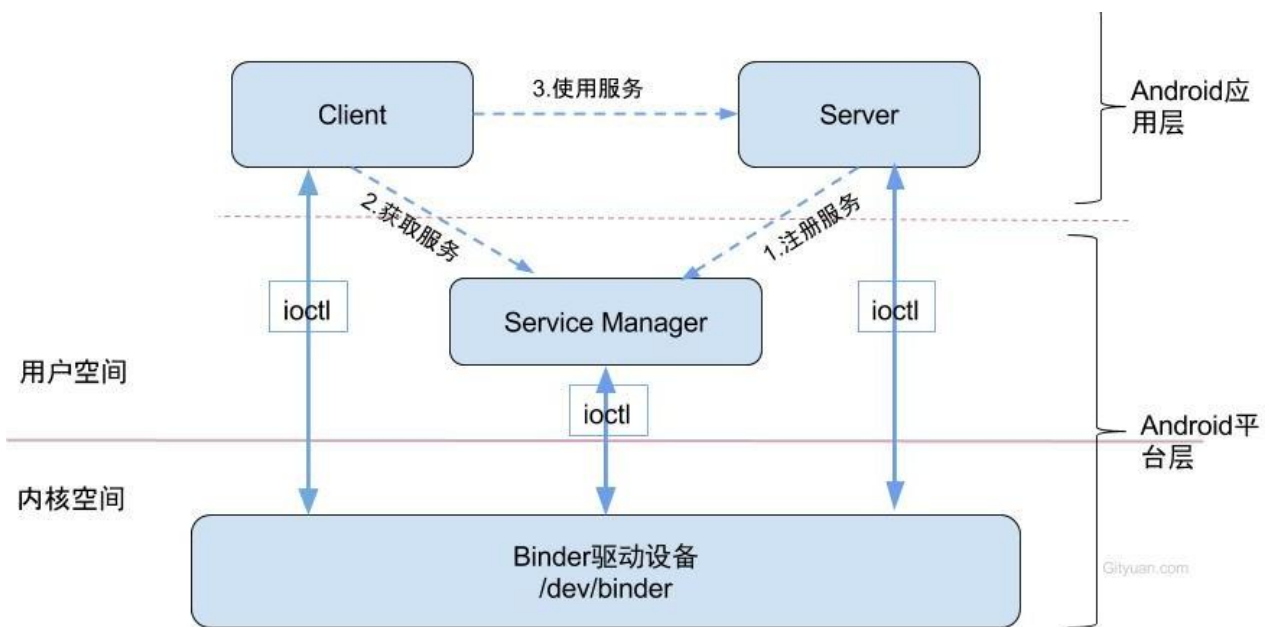
- 管道: 在创建时分配一个page大小的内存, 缓存区大小比较有限;

- 消息队列：信息复制两次，额外的CPU消耗；不合适频繁或信息量大的通信；
- 共享内存：无须复制，共享缓冲区直接付附加到进程虚拟地址空间，速度快；但进程间的同步问题操作系统无法实现，必须各进程利用同步工具解决；
- 套接字：作为更通用的接口，传输效率低，主要用于不通机器或跨网络的通信；
- 信号量：常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。因此，主要作为进程间以及同一进程内不同线程之间的同步手段。
- 信号：不适用于信息交换，更适用于进程中断控制，比如非法内存访问，杀死某个进程等；

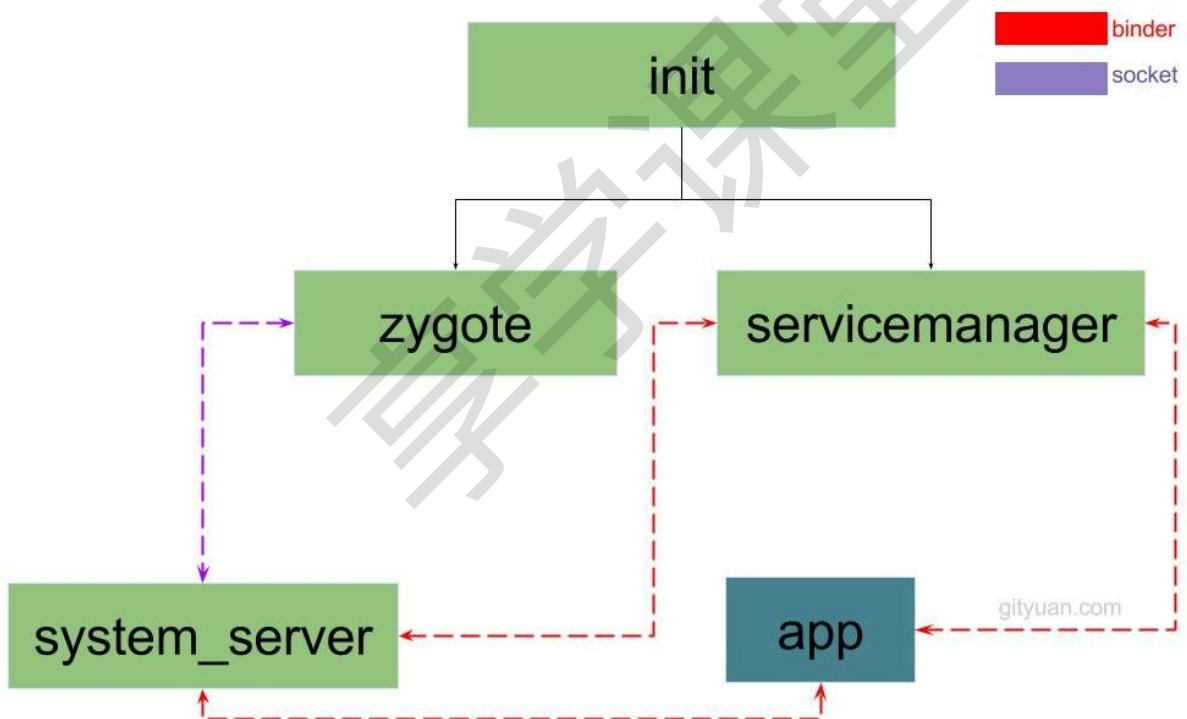
21 5个角度来分析Binder

- 从性能的角度 数据拷贝次数：Binder数据拷贝只需要一次，而管道、消息队列、Socket都需要2次，但共享内存方式一次内存拷贝都不需要；从性能角度看，Binder性能仅次于共享内存
- 从稳定性的角度
 - Binder是基于C/S架构的 C/S 相对独立，稳定性较好
 - 共享内存实现方式复杂，没有客户与服务端之别， 需要充分考虑到访问临界资源的并发同步问题，否则可能会出现死锁等问题
- 从安全的角度
 - 传统Linux IPC的接收方无法获得对方进程可靠的UID/PID，从而无法鉴别对方身份
 - 传统IPC只能由用户在数据包里填入UID/PID
 - 可靠的身份标记只有由IPC机制本身在内核中添加
 - 传统IPC访问接入点是开放的，无法建立私有通道
 - Android为每个安装好的应用程序分配了自己的UID，故进程的UID是鉴别进程身份的重要标志， 前面提到C/S架构，Android系统中对外只暴露Client端，Client端将任务发送给Server端， Server端会根据权限控制策略，判断UID/PID是否满足访问权限，目前权限控制很多时候是通过 弹出权限询问对话框，让用户选择是否运行
 - Android的UID权鉴是如何做的？
- 从语言层面的角度
 - Linux是基于C语言(面向过程的语言)，而Android是基于Java语言(面向对象的语句)
 - Binder恰恰也符合面向对象的思想 Binder模糊了进程边界，淡化了进程间通信过程，整个系统仿佛运行于同一个面向对象的程序之中
 - Android OS中的Zygote进程的IPC采用的是Socket（套接字）机制，Android中的Kill Process采用的signal（信号）机制等等。而Binder更多则用在system_server进程与上层App层的IPC交互。
- 从公司战略的角度 总所周知，Linux内核是开源的系统，所开放源代码许可协议GPL保护，该协议具有“病毒式感染”的能力，怎么理解这句话呢？受GPL保护的Linux Kernel是运行在内核空间，对于上层的任何类库、服务、应用等运行在用户空间，一旦进行SysCall（系统调用），调用到底层Kernel，那么也必须遵循GPL协议。 而Android之父 Andy Rubin对于GPL显然是不能接受的，为此，Google巧妙地将GPL协议控制在内核空间，将用户空间的协议采用Apache-2.0协议（允许基于Android的开发商不向社区反馈源码）， 同时在GPL协议与Apache-2.0之间的Lib库中采用BSD证授权方法，有效隔断了GPL的传染性，仍有较大争议，但至少目前缓解Android，让GPL止步于内核空间，这是Google在GPL Linux下 开源与商业化共存的一个成功典范。

Binder IPC原理



第四节 系统启动系列



序号	进程启动	概述
1	init进程	Linux系统中用户空间的第一个进程，Init.main
2	zygote进程	所有 App进程的父进程，ZygoteInit.main
3	system_server进程	系统各大服务的载体
4	servicemanager进程	binder服务的大管家，守护进程循环运行在binder_loop
5	app进程	通过Process.start启动App进程，ActivityThread.main

第五节 AMS

3.1 AMS启动

在system_server进程中启动

SystemServer.java #startBootstrapServices()

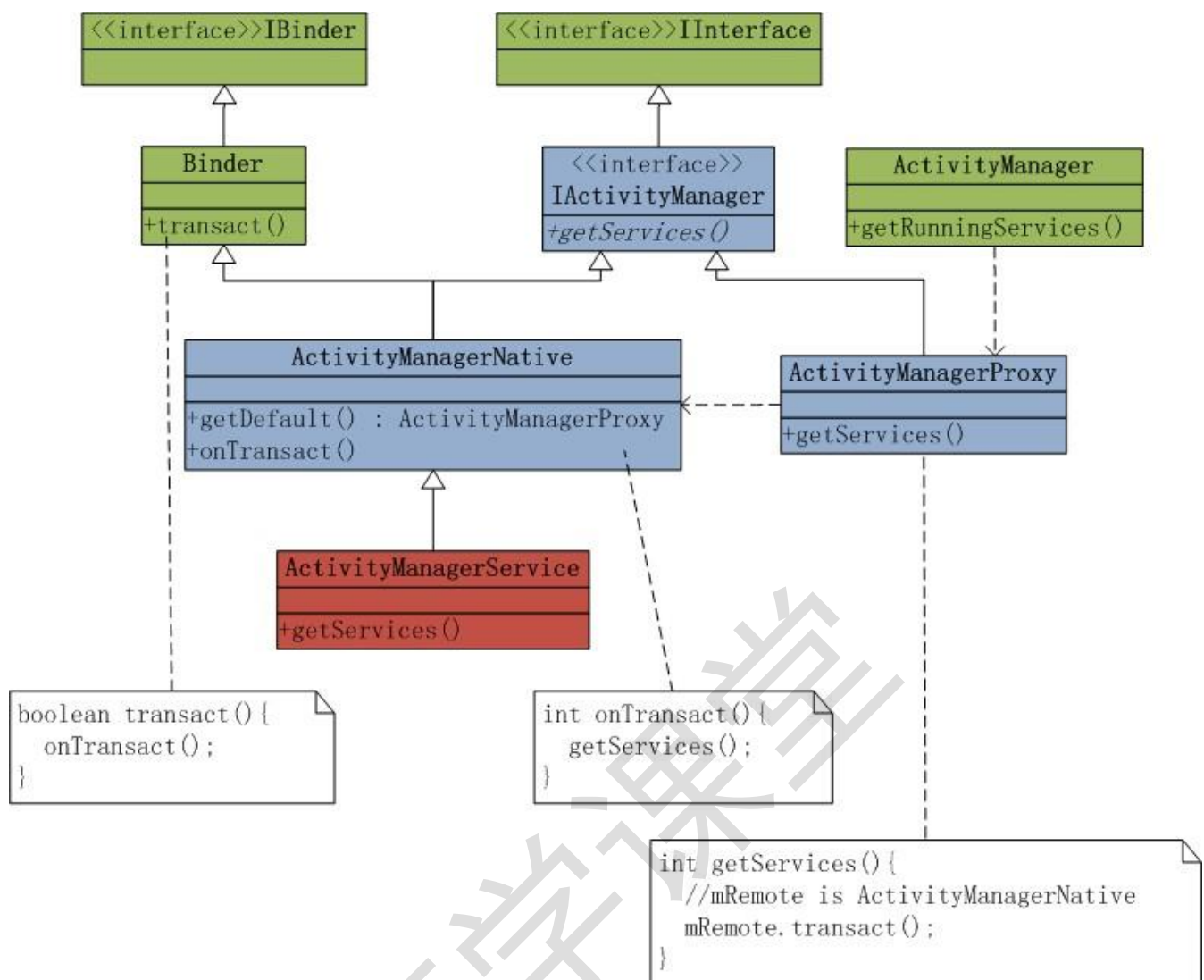
创建AMS实例对象，创建Andoid Runtime，ActivityThread和Context对象； setSystemProcess：注册AMS、meminfo、cpuinfo等服务到ServiceManager； installSystemProviderss，加载SettingsProvider；

启动SystemUIService，再调用一系列服务的systemReady()方法； 发布Binder服务

服务名	类名	功能
activity	ActivityManagerService	AMS
procstats	ProcessStatsService	进程统计
meminfo	MemBinder	内存
gfxinfo	GraphicsBinder	图像信息
dbinfo	DbBinder	数据库
cpuinfo	CpuBinder	CPU
permission	PermissionController	权限
processinfo	ProcessInfoService	进程服务
usagestats	UsageStatsService	应用的使用情况

想要查看这些服务的信息，可通过dumpsys <服务名>命令。比如查看CPU信息命令dumpsys cpuinfo

3.2 AMS类图结构



在这张图中，绿色的部分是在SDK中开放给应用程序开发人员的接口，蓝色的部分是一个典型的Proxy模式，红色的部分是底层的服务实现，是真正的动作执行者。这里的一个核心思想是Proxy模式，我们接下来对此模式加以介绍

Proxy模式

Proxy模式，也称代理模式，是经典设计模式中的一种结构型模式，其定义是为其他对象提供一种代理以控制对这个对象的访问，简单的说就是在访问和被访问对象中间加上的一个间接层，以隔离访问者和被访问者的实现细节。

结合上面的类结构图，其中ActivityManager是一个客户端，为了隔离它与ActivityManagerService，有效降低甚至消除二者的耦合度，在这中间使用了ActivityManagerProxy代理类，所有对ActivityManagerService的访问都转换成对代理类的访问，这样ActivityManager就与ActivityManagerService解耦了。这就是代理模式的典型应用场景。

景。

为了让代理类与被代理类保持一致的接口，从而实现更加灵活的类结构，或者说完美的屏蔽实现细节，通常的做法是让代理类与被代理类实现一个公共的接口，这样对调用者来说，无法知道被调用的是代理类还是直接是被代理类，因为二者的接口是相同的。

这个思路在上面的类结构图里也有落实，IActivityManager接口类就是起的这个作用。

以上就是代理模式的思路，有时我们也称代理类为本地代理（Local Proxy），被代理类为远端代理（Remote Proxy）。

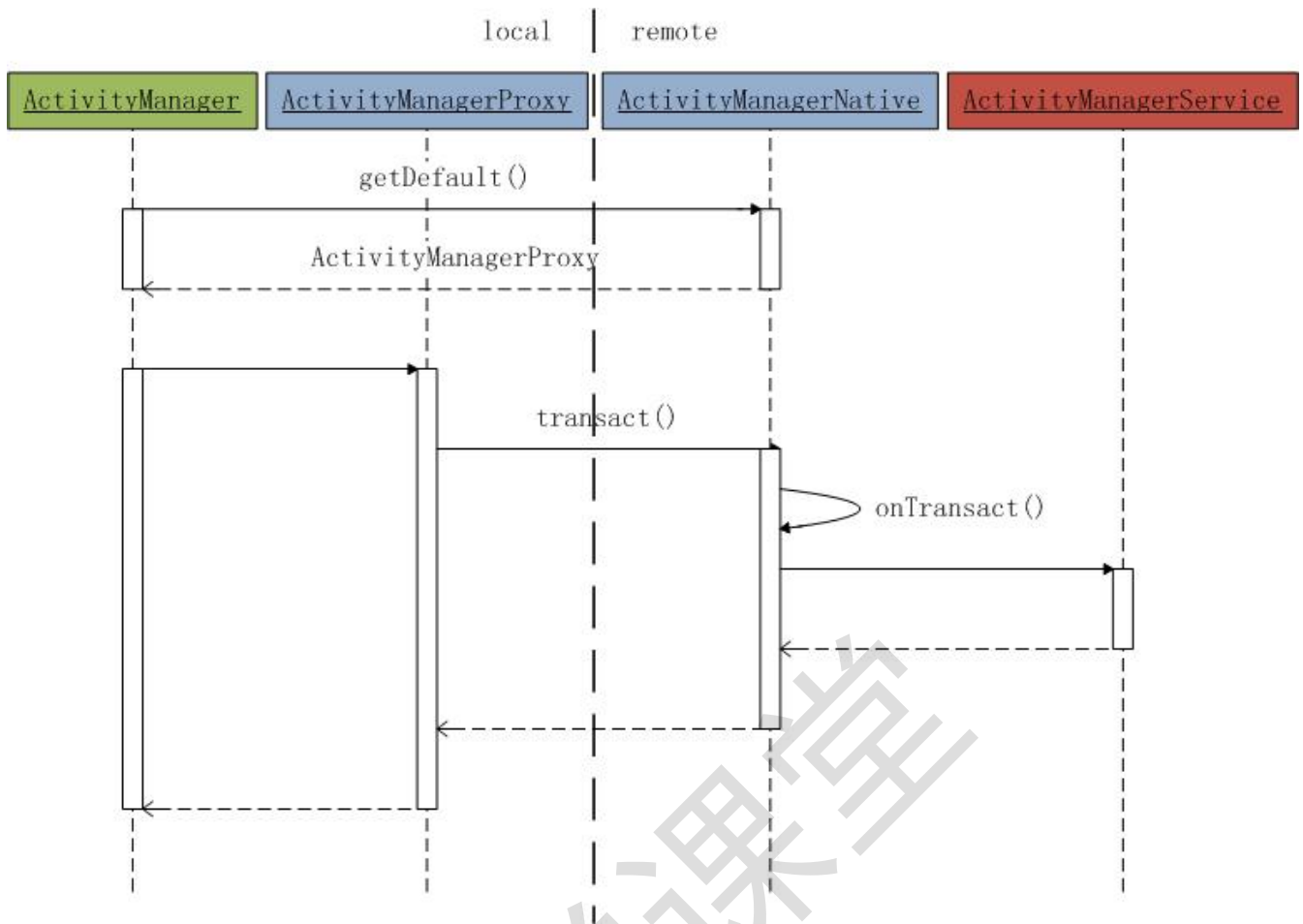
本地代理与远端代理的Binder 我们再来看一下Binder类的作用，Binder的含义可能译为粘合剂更为贴切，即将两侧的东西粘贴起来。在操作系统中，Binder的一大作用就是连接本地代理和远端代理。Binder中最重要的一个函数是：

```
public final boolean transact(int code, Parcel data, Parcel reply,
    int flags) throws RemoteException {
    .....
    boolean r = onTransact(code, data, reply, flags);
    if (reply != null) {
        reply.setDataPosition(0);
    }
    return r;
}
```

它的作用就在于通过code来表示请求的命令标识，通过data和reply进行数据传递，只要远端代理能实现onTransact()函数，即可做出正确的动作，远端的执行接口被完全屏蔽了。

当然，Binder的实现还是很复杂的，不仅是类型转换，还要透过Binder驱动进入KERNEL层来完成进程通信，这些内容不在本文的范围之内，故此处不再深入解析相应的机制。此处我们只要知道Binder的transact()函数实现就可以了。

到此为止，我们对ActivityManager的静态类结构就分析完了，但这还不足以搞清在系统运行中的调用过程，因此，我们以下图的序列图为基础，结合源码探索一下ActivityManager运行时的机制。



我们以ActivityManager的getRunningServices()函数为例，对上述序列图进行解析。

```

public List<RunningServiceInfo> getRunningServices(int maxNum)
    throws SecurityException {
    try {
        return (List<RunningServiceInfo>)ActivityManagerNative.getDefault()
            .getServices(maxNum, 0);
    } catch (RemoteException e) {
        // System dead, we will be dead too soon!
        return null;
    }
}

```

可以看到，调用被委托到了ActivityManagerNative.getDefault()。

```

static public IActivityManager asInterface(IBinder obj)

```



```

{
    .....

    return new ActivityManagerProxy(obj);

}

static public IActivityManager getDefault()
{
    .....

    IBinder b = ServiceManager.getService("activity");
    gDefault = asInterface(b);
    return gDefault;

}

```

从上述简化后的源码可以看到，getDefault() 函数返回的是一个ActivityManagerProxy对象的引用，也就是说，ActivityManager得到了一个本地代理。

因为在IActivityManager接口中已经定义了getServices() 函数，所以我们来看这个本地代理对该函数的实现。

```

public List getServices(int maxNum, int flags) throws RemoteException {
    Parcel data = Parcel.obtain();
    Parcel reply = Parcel.obtain();

    .....

    mRemote.transact(GET_SERVICES_TRANSACTION, data, reply, 0);

    .....
}

```

从这个代码版段我们看到，调用远端代理的transact() 函数，而这个mRemote就是ActivityManagerNative的Binder接口。

接下来我们看一下ActivityManagerNative的代码，因为该类是继承于Binder类的，所以transact的机制此前我们已经展示了代码，对于该类而言，重要的是对onTransact() 函数的实现。

```

public boolean onTransact(int code, Parcel data, Parcel reply, int flags)
    throws RemoteException {

```

```

switch (code) {
case GET_SERVICES_TRANSACTION: {
.....

    List list = getServices(maxNum, fl);

    .....

    return true;
}

.....

}

return super.onTransact(code, data, reply, flags);

}

```

在onTransact()函数内，虽然代码特别多，但就是一个switch语句，根据不同的code命令进行不同的处理，比如 对于 GET_SERVICES_TRANSACTION命令，只是调用了getServices()函数。而该函数的实现是在ActivityManagerService类中，它是ActivityManagerNative的子类，对于该函数的实现细节，不在本文中详细分析。

Activity启动 在经过前文的学习以后，我们一起来整理一下Activity的启动机制。就从Activity的startActivity()函数开始吧。

startActivity()函数调用了startActivityForResult()函数，该函数有源码如下：

```

public void startActivityForResult(Intent intent, int requestCode) {
.....

    Instrumentation.ActivityResult ar =
        mInstrumentation.execStartActivity(
            this, mMainThread.getApplicationThread(), mToken, this,
            intent, requestCode);
    .....

}

```

可见，功能被委托给Instrumentation对象来执行了。这个类的功能是辅助Activity的监控和测试，在此我们不详细描述，我们来看它的execStartActivity()函数。

ivityManagerNative.getDefault(), 没错, 利用ActivityManagerProxy, 通过Binder的transact机制, 并沿此线索继续追踪源码, 在startActivity调用, mWindowManager是WindowManagerService.

ty启动的底层实现机制, 也加深了对Proxy模式和Bin

的`ActivityManagerNative.getDefault()`，没
以透过`ActivityManagerProxy`，通过`Binder`的`transact()`函数，并沿此线索继续追踪源 码，在`start`
的语句调用，`mWindowManager`是`WindowManager`。
`Activity`启动的底层实现机制，也加深了对`Proxy`

的语句调用，mWindowManager是WindowManager的静态成员变量，也是Activity启动的底层实现机制，也加深了对Proxy的理解。

11-11-11 11:11:11

[illegible]

```
at android.support.constraint.ConstraintLayout.requestLayout(ConstraintLayout.java:3172)
at android.view.View.requestLayout(View.java:19781)
at android.widget.TextView.checkForRelayout(TextView.java:7368) at
android.widget.TextView.setText(TextView.java:4480)
at android.widget.TextView.setText(TextView.java:4337) at
android.widget.TextView.setText(TextView.java:4312)
at com.example.amsdemo.MainActivity$3.run(MainActivity.java:46)
```

```
ViewRootImpl.java#checkThread() L 6889
-> #requestLayout() L 1046
-> #scheduleTraversals() L 1222
-> #TraversalRunnable L 6334
-> #doTraversal() L 1245
ActivityThread.java#handleResumeActivity() L3456
-> #performResumeActivity() L 3385
Activity.java#performResume() L 6774
Instrumentation.java#callActivityOnResume() L 1267
//performResumeActivity方法确实是回调onResume方法的入口
ActivityThread.java#handleResumeActivity() L 3560
Activity.java#makeVisible() L 5128
WindowManagerImpl.java#addView() L 91
WindowManagerGlobal.java#addView() L 263
-> addView() L 331
```

android中进程的层次如下（重要性由高到低）：

1、前端进程。顾名思义，前端进程就是目前显示在屏幕上和用户交互的进程，在系统中前端进程数量很少，而这种进程是对用户体验的影响最大，只有系统的内存稀少到不足以维持和用户的基本交互时才会销毁前端进程。因此这种进程重要性是最高的。

2、可见进程。可见进程也拥有一个可视化的界面，只是目前不是最上层界面（最上层界面在前端进程里面），可见进程一般调用了OnPause()，可见进程比前端进程重要性低，但是在交互方面影响还是很大，因为用户可能随时切换过去，所以系统不会轻易销毁它。

3、服务进程。一个服务进程就是一个Service，它调用了startService()，就是UNIX中说的守护进程，对用户不可见，但是保证了一些重要的事件被监听或者维持着某些状态，比如网络数据传输、后台音乐播放，这类进程在内存不足且为了保证前端交互的顺利进行的时候被销毁。

4、后台进程。这里叫后台进程可能会和一般意义上的后台进程混淆，要说明的是，android里的后台进程是调用了OnStop()的，可以理解成用户暂时没有和这个进程交互的愿望，所以这里后台进程有点“待销毁”的意思。

5、空进程。这是一种系统缓存机制，其实就是个进程的外壳，当有新进程创建的时候，这个空进程可以加快进程创建速度，当系统内存不足的时候，首先销毁空进程。

本章知识点解析由享学课堂提供。

第六节 AMS 面试题解析

6.1 ActivityManagerService是什么？什么时候初始化的？有什么作用？

ActivityManagerService 主要负责系统中四大组件的启动、切换、调度及应用进程的管理和调度等工作，其职责

与操作系统中的进程管理和调度模块类似。ActivityManagerService进行初始化的时机很明确，就是在SystemServer进程开启的时候，就会初始化

ActivityManagerService。（系统启动流程）

如果打开一个App的话，需要AMS去通知zygote进程，

所有的Activity的生命周期AMS来控制

6.2 ActivityThread是什么？ApplicationThread是什么？他们的区别

ActivityThread

在Android中它就代表了Android的主线程,它是创建完新进程之后,main函数被加载，然后执行一个loop的循环

使当前线程进入消息循环，并且作为主线程。

ApplicationThread

ApplicationThread是ActivityThread的内部类，

是一个Binder对象。在此处它是作为IApplicationThread对象

的server端等待client端的请求然后进行处理，最大的client就是AMS。

6.3 Instrumentation是什么？和ActivityThread是什么关系？

AMS与ActivityThread之间诸如Activity的创建、暂停等的交互工作实际上是由Instrumentation具体操作的。每

个Activity都持有一个Instrumentation对象的一个引用，

整个进程中是只有一个Instrumentation。

mInstrumentation的初始化在ActivityThread::handleBindApplication函数。

可以用来独立地控制某个组件的生命周期。

Activity`的`startActivity`方法。`startActivity`会调用`mInstrumentation.execStartActivity();`

mInstrumentation 掉用 AMS ,AMS 通过 socket 通信告知 Zygote 进程 fork 子进程。

6.4 ActivityManagerService和zygote进程通信是如何实现的。

应用启动时,Launcher进程请求AMS。 AMS发送创建应用进程请求，Zygote进程接受请求并fork应用进程
客户端发送请求

调用 Process.start() 方法新建进程

连接调用的是 `ZygoteState.connect()` 方法，`ZygoteState` 是 `ZygoteProcess` 的内部类。`ZygoteState`里用的

`LocalSocket`

```
public static ZygoteState connect(LocalSocketAddress address) throws IOException {
    DataInputStream zygoteInputStream = null;
    BufferedWriter zygoteWriter = null;
    final LocalSocket zygoteSocket = new LocalSocket();
    try {
        zygoteSocket.connect(address);
        zygoteInputStream = new DataInputStream(zygoteSocket.getInputStream());
        zygoteWriter = new BufferedWriter(new OutputStreamWriter(
            zygoteSocket.getOutputStream(), 256));
    } catch (IOException ex) {
        try {
            zygoteSocket.close();
        } catch (IOException ignore) {}
        throw ex;
    }
    return new ZygoteState(zygoteSocket, zygoteInputStream, zygoteWriter,
        Arrays.asList(abiListString.split(",")));
}
```

`Zygote` 处理客户端请求

`Zygote` 服务端接收到参数之后调用 `ZygoteConnection.processOneCommand()` 处理参数，并 `fork` 进程最后通过 `findStaticMain()` 找到 `ActivityThread` 类的 `main()` 方法并执行，子进程就启动了

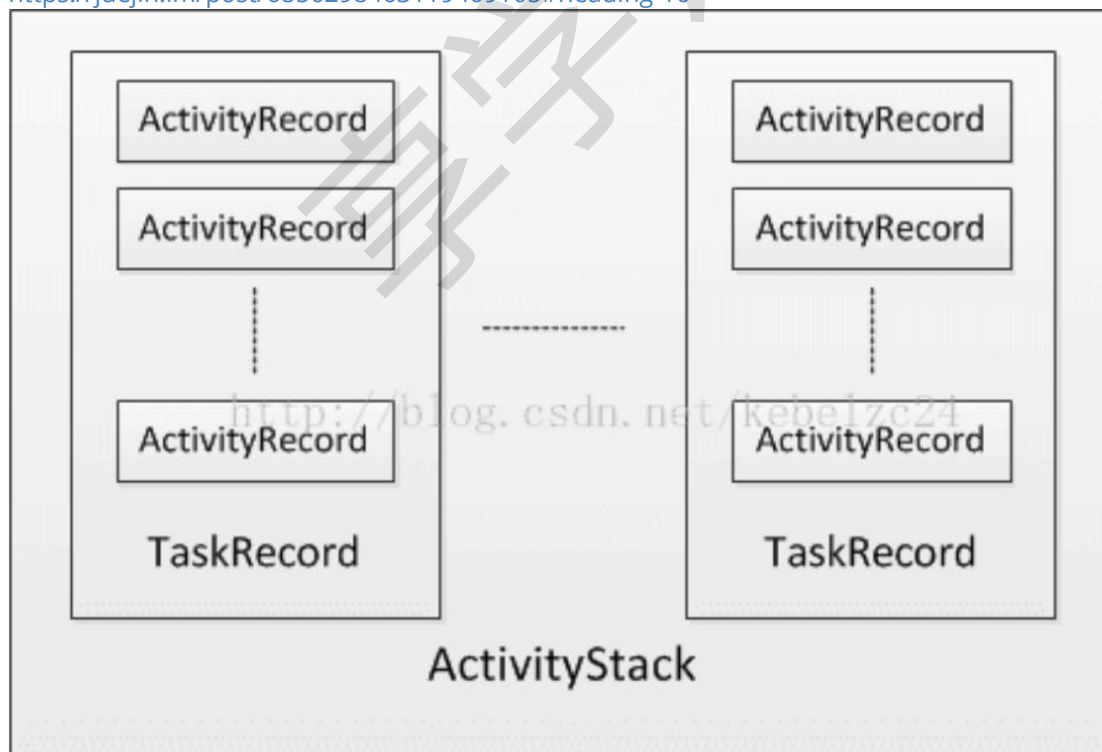
6.5 ActivityRecord、TaskRecord、ActivityStack，

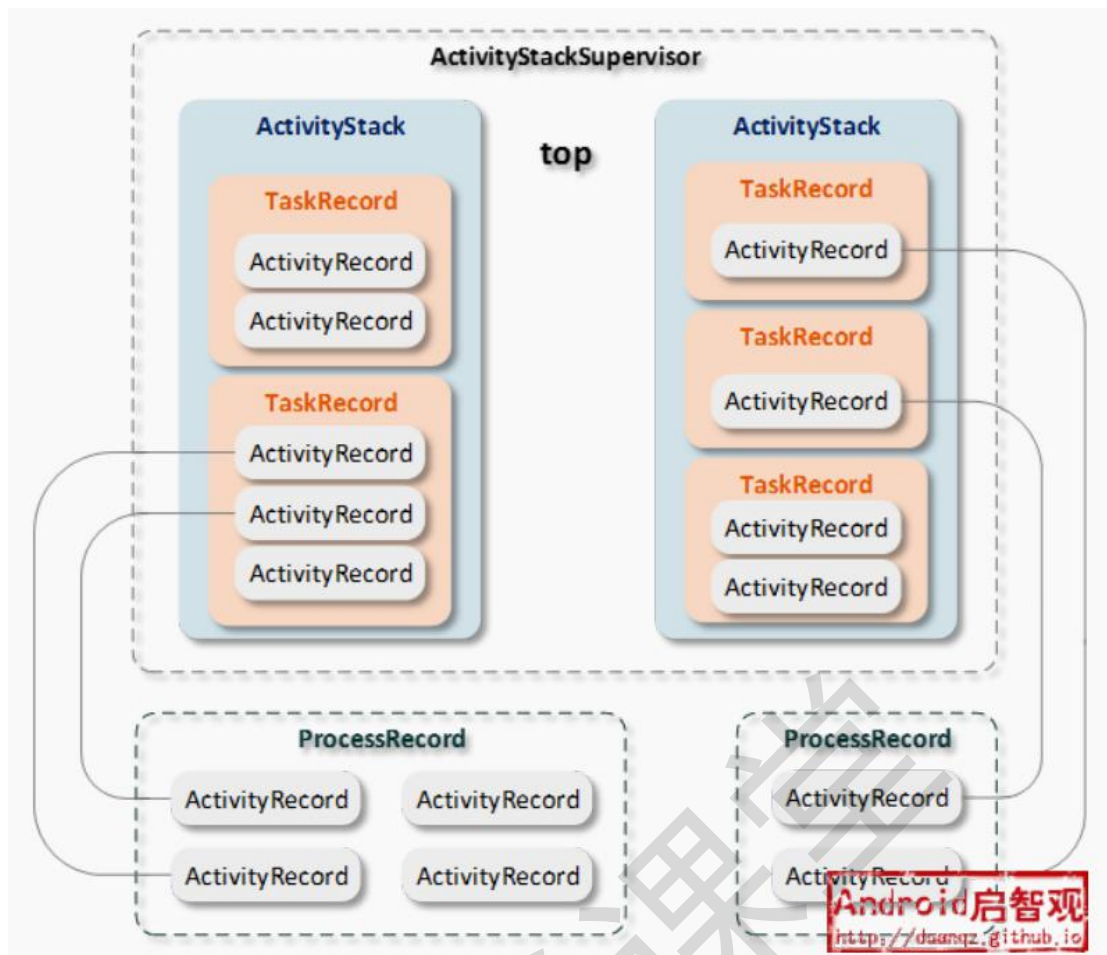
ActivityStackSupervisor, ProcessRecord

<https://duanqz.github.io/2016-02-01-Activity-Maintenance#activityrecord>

<https://www.jianshu.com/p/94816e52cd77>

<https://juejin.im/post/6856298463119409165#heading-10>





ActivityRecord

Activity管理的最小单位，它对应着一个用户界面

ActivityRecord是应用层Activity组件在AMS中的代表，每一个在应用中启动的Activity，在AMS中都有一个ActivityRecord实例来与之对应，这个ActivityRecord伴随着Activity的启动而创建，也伴随着Activity的终止而销毁。

TaskRecord

TaskRecord即任务栈，

每一个TaskRecord都可能存在一个或多个ActivityRecord，栈顶的ActivityRecord表示当前可见的界面。

一个App是可能有多个TaskRecord存在的

一般情况下,启动App的第一个activity时，AMS为其创建一个TaskRecord任务栈

特殊情况,启动singleTask的Activity，而且为该Activity指定了和包名不同的taskAffinity，也会为该activity创建

一个新的TaskRecord

ActivityStack

ActivityStack,ActivityStack是系统中用于管理TaskRecord的,内部维护了一个ArrayList。

ActivityStackSupervisor内部有两个不同的ActivityStack对象：mHomeStack、mFocusedStack，用来管理不同

的任务。我们启动的App对应的TaskRecord由非Launcher ActivityStack管理，它是在系统启动第一个app时创建的。

ActivityStackSupervisor

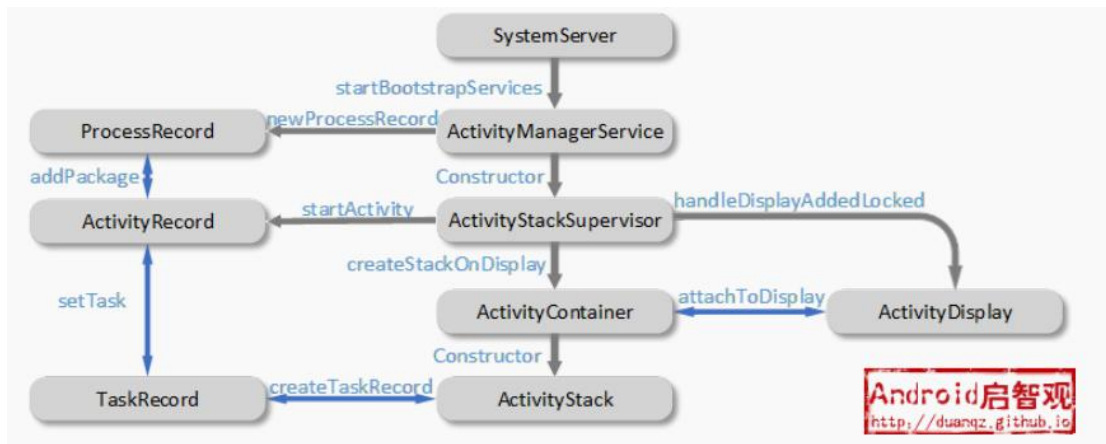
ActivityStackSupervisor管理着多个ActivityStack，但当前只会有一个获取焦点(Focused)的ActivityStack;

AMS对象只会存在一个，在初始化的时候，会创建一个唯一的ActivityStackSupervisor对象

ProcessRecord

ProcessRecord记录着属于一个进程的所有ActivityRecord，运行在不同TaskRecord中的ActivityRecord可能是属于同一个 ProcessRecord。

关系



AMS运行在SystemServer进程中。SystemServer进程启动时，会通过SystemServer.startBootstrapServices()来创建一个AMS的对象；

AMS通过ActivityStackSupervisor来管理Activity。AMS对象只会存在一个，在初始化的时候，会创建一个唯一的ActivityStackSupervisor对象；

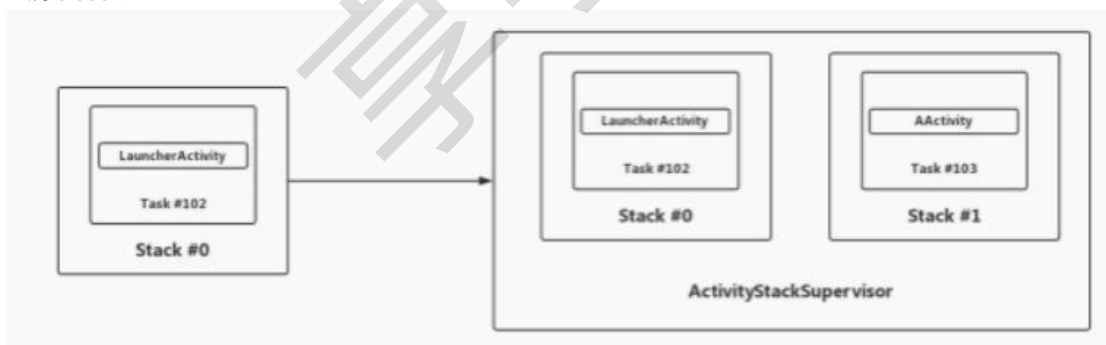
ActivityStackSupervisor中维护了显示设备的信息。当有新的显示设备添加时，会创建一个新的ActivityDisplay对象；

ActivityStack与显示设备的绑定。ActivityStack的创建时在Launcher启动时候进行的，AMS还未有非Launcher的ActivityStack。后面的App启动时就会创建Launcher的ActivityStack，通过ActivityStackSupervisor来创建ActivityRecord

在ActivityStack上创建TaskRecord

每一个ActivityRecord都需要找到自己的宿主TaskRecord

从桌面启动



从桌面点击图标启动一个Activity，

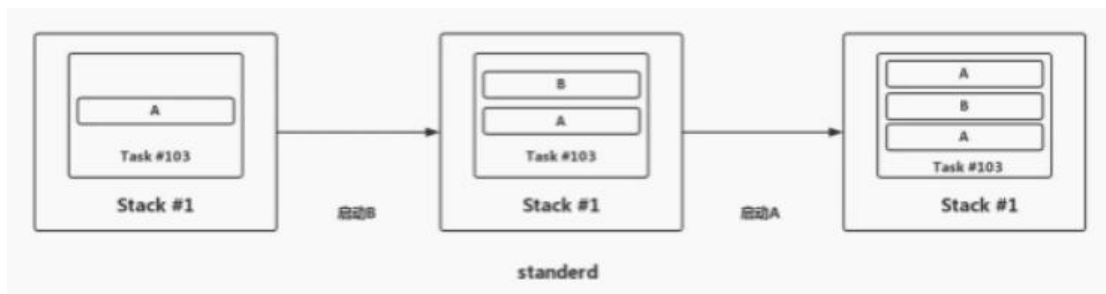
会启动ActivityStackSupervisor中的mFocusedStack，mFocusedStack负责

管理的是非Launcher相关的任务。同时也会创建一个新的ActivityRecord和TaskRecord，ActivityRecord放到TaskRecord中，TaskRecord则放进mFocusedStack中。

四种启动模式

standerd

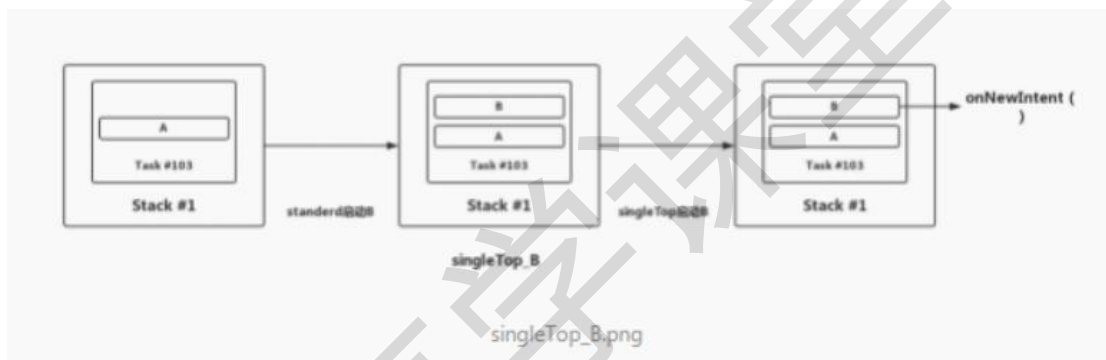
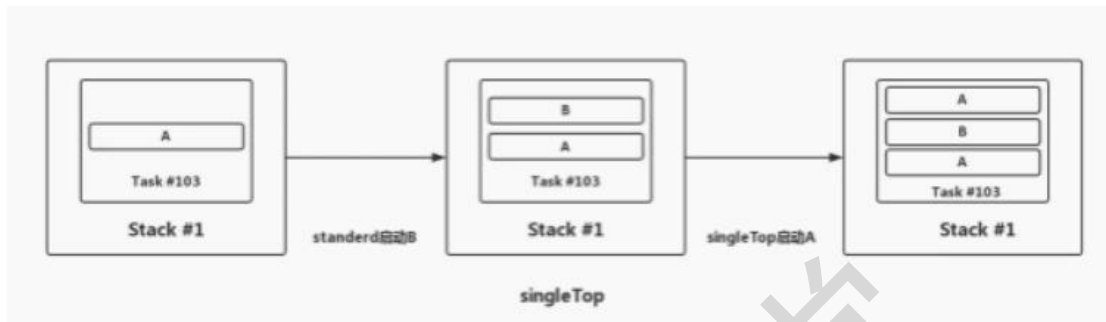
默认模式，每次启动Activity都会创建一个新的Activity实例。



singleTop

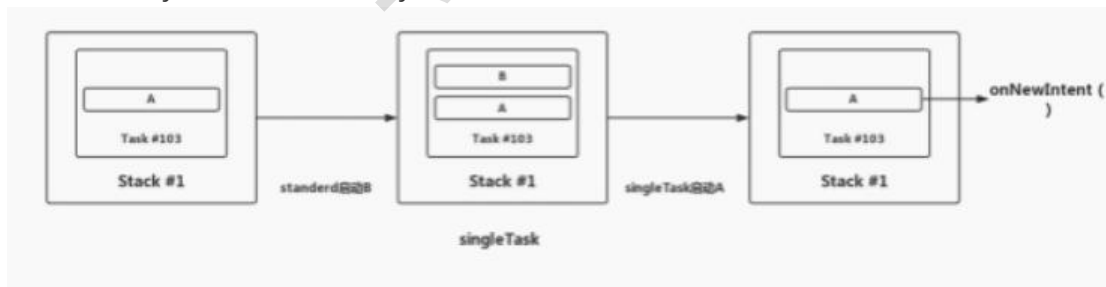
如果要启动的Activity已经在栈顶，则不会重新创建Activity，只会调用该Activity的onNewIntent()方法。

如果要启动的Activity不在栈顶，则会重新创建该Activity的实例。



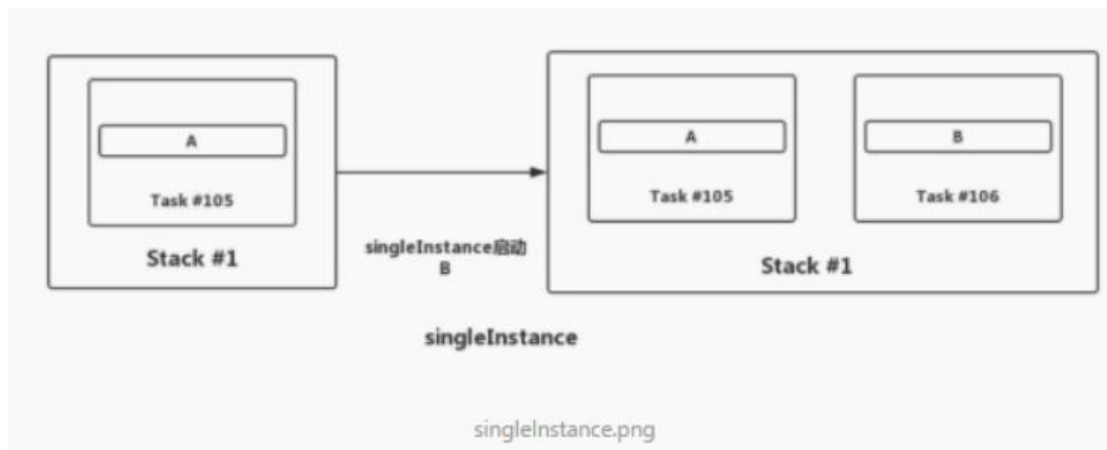
singleTask

如果要启动的Activity已经存在于它想要归属的栈中，那么不会创建该Activity实例，将栈中位于该Activity上的所有的Activity出栈，同时该Activity的onNewIntent()方法会被调用。



singleInstance

要创建在一个新栈，然后创建该Activity实例并压入新栈中，新栈中只会存在这一个Activity实例。



6.6 ActivityManager、ActivityManagerService、

ActivityManagerNative、ActivityManagerProxy的关系

<https://www.cnblogs.com/mingfeng002/p/10650364.html>

Activity 的 `startActivity` 方法。 `startActivity` 会调用 `mInstrumentation.execStartActivity()`; `execStartActivity` 通过 ActivityManager 的 `getService` 。代码层面:

ActivityManager.getRunningServices里通过ActivityManagerNative.getDefault得到此代理对象

ActivityManagerProxy , ActivityManagerProxy代理类是ActivityManagerNative的内部类。

ActivityManagerNative

是个抽象类，真正发挥作用的是它的子类ActivityManagerService。

介绍:

ActivityManager

ActivityManager官方介绍：是与系统所有正在运行着的Activity进行交互，对系统所有运行中的Activity相关信

息 (Task , Memory , Service , App) 进行管理和维护。

ActivityManagerNative、 ActivityManagerProxy

IActivityManager继承了Interface接口。而ActivityManagerNative和ActivityManagerPorxy实现了这个IActivityManager接口

ActivityManagerProxy代理类是ActivityManagerNative的内部类；

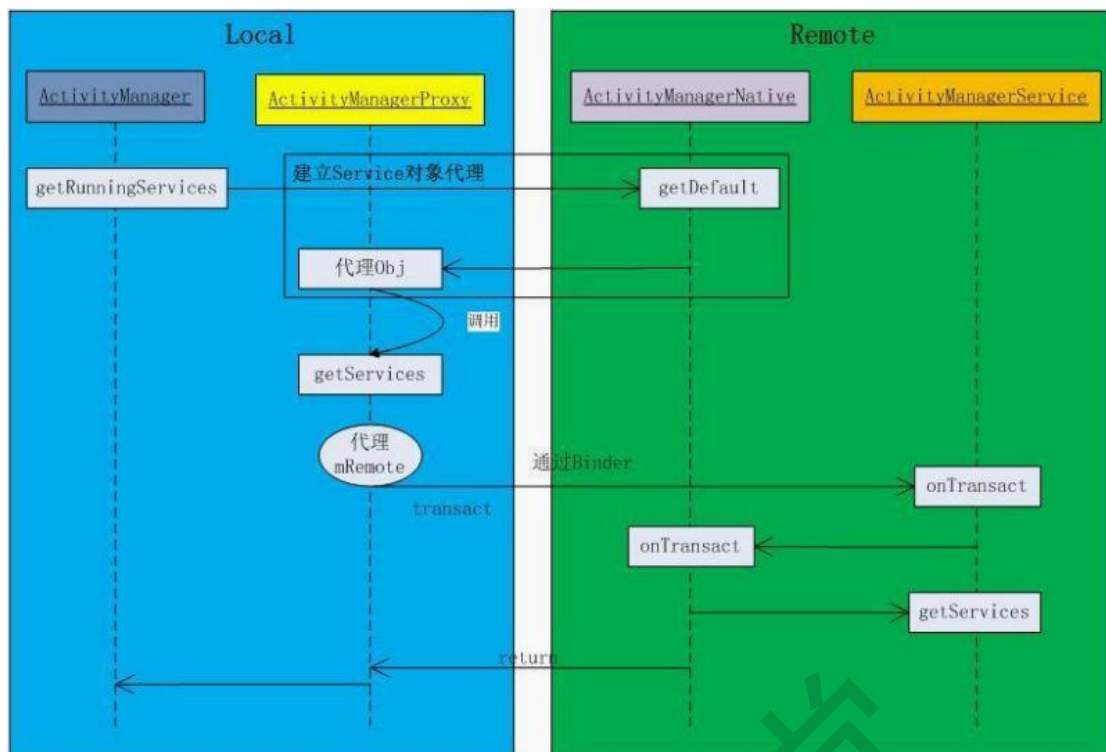
ActivityManagerNative是个抽象类，真正发挥作用的是它的子类ActivityManagerService

ActivityManager持有的是这个ActivityManagerPorxy代理对象，这样，只需要操作这个代理对象就能操作其业

务实现的方法。那么真正实现其也业务的则是ActivityManagerService。

ActivityManagerNative这个类，他继承了Binder而Binder实现了IBinder接口。其子类则是

ActivityManagerService。



7.手写实现简化版AMS

AMS与Binder相关，其中要明白下面几个类的职责：

IBinder：跨进程通信的Base接口，它声明了跨进程通信需要实现的一系列抽象方法，实现了这个接口就说明可

以进行跨进程通信，所有的Binder实体都必须实现IBinder接口。

IInterface：这也是一个Base接口，用来表示Server提供了哪些能力，是Client和Server通信的协议，Client和Server都要实现此接口。

Binder：IBinder的子类，Java层提供服务的Server进程持有一个Binder对象从而完成跨进程间通信。

BinderProxy：在Binder.java这个文件中还定义了一个BinderProxy类，这个类表示Binder代理对象它同样实现

了IBinder接口。Client中拿到的实际上是这个代理对象。

1、首先定义IActivityManager接口(继承IInterface)：

```

public interface IActivityManager extends IInterface {
    //binder描述符
    String DESCRIPTOR = "android.app.IActivityManager";
    //方法编号
    int TRANSACTION_startActivity = IBinder.FIRST_CALL_TRANSACTION + 0;
    //声明一个启动activity的方法，为了简化，这里只传入intent参数
    int startActivity(Intent intent) throws RemoteException;
}
  
```

2、实现ActivityManagerService侧的本地Binder对象基类：

```

public abstract class ActivityManagerNative extends Binder implements IActivityManager {
    public static IActivityManager asInterface(IBinder obj) {
        if (obj == null) {
            return null;
        }
        IActivityManager in = (IActivityManager)
            obj.queryLocalInterface(IActivityManager.DESRIPTOR);
        if (in != null) {
            return in;
        }
        //代理对象，见下面的代码
        return new ActivityManagerProxy(obj);
    }
    @Override
    public IBinder asBinder() {
        return this;
    }
    @Override
  
```

```
protected boolean onTransact(int code, Parcel data, Parcel reply, int flags) throws
RemoteException {
    switch (code) {
        // 获取binder描述符
        case INTERFACE_TRANSACTION:
            reply.writeString(IActivityManager.DESRIPTOR);
            return true; // 启动activity, 从data中反序列化出intent参数后, 直接调用子类startActivity方法启动
            activity.
        case IActivityManager.TRANSACTION_startActivity:
            data.enforceInterface(IActivityManager.DESRIPTOR);
            Intent intent = Intent.CREATOR.createFromParcel(data);
            int result = this.startActivity(intent);
            reply.writeNoException();
            reply.writeInt(result);
            return true;
    }
    return super.onTransact(code, data, reply, flags);
}
}
```

3、实现Client侧的代理对象：

```
public class ActivityManagerProxy implements IActivityManager {
    private IBinder mRemote;
    @Override
    public int startActivity(Intent intent) throws RemoteException {
        Parcel data = Parcel.obtain();
        Parcel reply = Parcel.obtain();
        int result;
        try {
            // 将intent参数序列化, 写入data中
            intent.writeToParcel(data, 0);
            // 调用BinderProxy对象的transact方法, 交由Binder驱动处理。
            mRemote.transact(IActivityManager.TRANSACTION_startActivity, data, reply, 0);
            reply.readException();
            // 等待server执行结束后, 读取执行结果
            result = reply.readInt();
        } finally {
            data.recycle();
            reply.recycle();
        }
        return result;
    }
}
```

4、实现Binder本地对象 (IActivityManager接口)：

```
public class ActivityManagerService extends ActivityManagerNative {
    @Override
    public int startActivity(Intent intent) throws RemoteException {
        // 启动activity
        return 0;
    }
}
```

第五章 WMS解析

第一节Activity与Window相关概念

- Activity只负责生命周期和事件处理
- Window只控制视图
- 一个Activity包含一个Window, 如果Activity没有Window, 那就相当于Service
- AMS统一调度所有应用程序的Activity
- WMS控制所有Window的显示与隐藏以及要显示的位置

1.1 Window

“Window”表明它是和窗口相关的, “窗口”是一个抽象的概念, 从用户的角度来讲, 它是一个“界面”; 从SurfaceFlinger的角度来看, 它是一个Layer, 承载着和界面有关的数据和属性; 从WMS角度来看, 它是一个WindowState, 用于管理和界面有关的状态。

- 表示一个窗口的概念, 是所有View的直接管理者, 任何视图都通过Window呈现(点击事件由Window->DecorView->View; Activity的setContentView底层通过Window完成)
- Window是一个抽象类, 具体实现是PhoneWindow
- 创建Window需要通过WindowManager创建

- WindowManager是外界访问Window的入口
- Window具体实现位于WindowManagerService中
- WindowManager和WindowManagerService的交互是通过IPC完成
- 定义窗口样式和行为的抽象基类，用于作为顶层的view加到WindowManager中，其实现类是PhoneWindow。
- 每个Window都需要指定一个Type（应用窗口、子窗口、系统窗口）。Activity对应的窗口是应用窗口；PopupWindow，ContextMenu，OptionsMenu是常用的子窗口；像Toast和系统警告提示框（如ANR）就是系统窗口，还有很多应用的悬浮框也属于系统窗口类型。

1.2WindowManager

用来在应用与window之间的管理接口，管理窗口顺序，消息等。

1.3WindowManagerService

简称Wms，WindowManagerService管理窗口的创建、更新和删除，显示顺序等，是WindowManager这个管理接口的真正的实现类。它运行在System_server进程，作为服务端，客户端（应用程序）通过IPC调用和它进行交互。

1.4Token

这里提到的Token主要是指窗口令牌（Window Token），是一种特殊的Binder令牌，Wms用它唯一标识系统中的一个窗口。

1.5Window的type

- 应用窗口：层级范围是1~99
- 子窗口：层级范围是1000~1999
- 系统窗口：层级范围是2000~2999
- 各级别type值在WindowManager中的定义分别为：
- 应用窗口（1~99）

//第一个应用窗口

```
public static final int FIRST_APPLICATION_WINDOW = 1;
```

//所有程序窗口的base窗口，其他应用程序窗口都显示在它上面

```
public static final int TYPE_BASE_APPLICATION = 1;
```

//所有Activity的窗口，只能配合Activity在当前APP使用

```
public static final int TYPE_APPLICATION = 2;
```

//目标应用窗口未启动之前的那个窗口

```
public static final int TYPE_APPLICATION_STARTING = 3;
```

//最后一个应用窗口

```
public static final int LAST_APPLICATION_WINDOW = 99;
```

- 子窗口（1000~1999）

//第一个子窗口

```
public static final int FIRST_SUB_WINDOW = 1000;
```

// 面板窗口，显示于宿主窗口的上层，只能配合Activity在当前APP使用

```
public static final int TYPE_APPLICATION_PANEL = FIRST_SUB_WINDOW;
```

// 媒体窗口（例如视频），显示于宿主窗口下层

```
public static final int TYPE_APPLICATION_MEDIA = FIRST_SUB_WINDOW+1;
```

// 应用程序窗口的子面板，只能配合Activity在当前APP使用 (PopupWindow默认就是这个Type)

pe)

```
public static final int TYPE_APPLICATION_SUB_PANEL = FIRST_SUB_WINDOW+2;
//对话框窗口, 只能配合Activity在当前APP使用
public static final int TYPE_APPLICATION_ATTACHED_DIALOG = FIRST_SUB_WINDOW+3;
//
public static final int TYPE_APPLICATION_MEDIA_OVERLAY = FIRST_SUB_WINDOW+4;
//最后一个子窗口
public static final int LAST_SUB_WINDOW = 1999;
• 系统窗口 (2000~2999)
  //系统窗口, 非应用程序创建
  public static final int FIRST_SYSTEM_WINDOW = 2000;
  //状态栏, 只能有一个状态栏, 位于屏幕顶端, 其他窗口都位于它下方
  public static final int TYPE_STATUS_BAR = FIRST_SYSTEM_WINDOW;
  //搜索栏, 只能有一个搜索栏, 位于屏幕上方
  public static final int TYPE_SEARCH_BAR = FIRST_SYSTEM_WINDOW+1;
  //
  //电话窗口, 它用于电话交互 (特别是呼入), 置于所有应用程序之上, 状态栏之下,
  属于悬浮窗 (并且给一个Activity的话按下HOME键会出现看不到桌面上的图标异常情况)
  public static final int TYPE_PHONE = FIRST_SYSTEM_WINDOW+2;
  //
  //系统警告提示窗口, 出现在应用程序窗口之上, 属于悬浮窗, 但是会被禁止
  public static final int TYPE_SYSTEM_ALERT = FIRST_SYSTEM_WINDOW+3;
  //
  //信息窗口, 用于显示Toast, 不属于悬浮窗, 但有悬浮窗的功能, 缺点是在Android
  2.3上无法接收点击事件
  public static final int TYPE_TOAST = FIRST_SYSTEM_WINDOW+5;
  //
  public static final int TYPE_KEYGUARD = FIRST_SYSTEM_WINDOW+4;
  //锁屏窗口
  public static final int TYPE_KEYGUARD = FIRST_SYSTEM_WINDOW+4;
  //系统顶层窗口, 显示在其他一切内容之上, 此窗口不能获得输入焦点, 否则影响锁
  屏
  public static final int TYPE_SYSTEM_OVERLAY = FIRST_SYSTEM_WINDOW+6;
  //电话优先, 当锁屏时显示, 此窗口不能获得输入焦点, 否则影响锁屏
  public static final int TYPE_PRIORITY_PHONE = FIRST_SYSTEM_WINDOW+7;
  //系统对话框窗口
  public static final int TYPE_SYSTEM_DIALOG = FIRST_SYSTEM_WINDOW+8;
  //锁屏时显示的对话框
  public static final int TYPE_KEYGUARD_DIALOG = FIRST_SYSTEM_WINDOW+9;
  //系统内部错误提示, 显示在任何窗口之上
  public static final int TYPE_SYSTEM_ERROR = FIRST_SYSTEM_WINDOW+10;
  //内部输入法窗口, 显示于普通UI之上, 应用程序可重新布局以免被此窗口覆盖
  public static final int TYPE_INPUT_METHOD = FIRST_SYSTEM_WINDOW+11;
  //内部输入法对话框, 显示于当前输入法窗口之上
  public static final int TYPE_INPUT_METHOD_DIALOG = FIRST_SYSTEM_WINDOW+12;
  //墙纸窗口
  public static final int TYPE_WALLPAPER = FIRST_SYSTEM_WINDOW+13;
  //状态栏的滑动面板
  public static final int TYPE_STATUS_BAR_PANEL = FIRST_SYSTEM_WINDOW+14;
  //安全系统覆盖窗口, 这些窗口必须不带输入焦点, 否则会干扰键盘
  public static final int TYPE_SECURE_SYSTEM_OVERLAY = FIRST_SYSTEM_WINDOW+15;
  //最后一个系统窗口
  public static final int LAST_SYSTEM_WINDOW = 2999;
```


2. 窗口flags显示属性在WindowManager中也有定义:

```
//窗口特征标记
public int flags;
//当该window对用户可见的时候, 允许锁屏
public static final int FLAG_ALLOW_LOCK_WHILE_SCREEN_ON = 0x00000001;
//窗口后面的所有内容都变暗
public static final int FLAG_DIM_BEHIND = 0x00000002;
//Flag: 窗口后面的所有内容都变模糊
public static final int FLAG_BLUR_BEHIND = 0x00000004;
//窗口不能获得焦点
public static final int FLAG_NOT_FOCUSABLE = 0x00000008;
//窗口不接受触摸屏事件
public static final int FLAG_NOT_TOUCHABLE = 0x00000010;
//即使在该window在可获得焦点情况下, 允许该窗口之外的点击事件传递到当前窗口
//后面的窗口去
public static final int FLAG_NOT_TOUCH_MODAL = 0x00000020;
//当手机处于睡眠状态时, 如果屏幕被按下, 那么该window将第一个收到触摸事件
public static final int FLAG_TOUCHABLE_WHEN_WAKING = 0x00000040;
//当该window对用户可见时, 屏幕出于常亮状态
public static final int FLAG_KEEP_SCREEN_ON = 0x00000080;
//: 让window占满整个手机屏幕, 不留任何边界
public static final int FLAG_LAYOUT_IN_SCREEN = 0x00000100;
//允许窗口超出整个手机屏幕
public static final int FLAG_LAYOUT_NO_LIMITS = 0x00000200;
//window全屏显示
public static final int FLAG_FULLSCREEN = 0x00000400;
//恢复window非全屏显示
public static final int FLAG_FORCE_NOT_FULLSCREEN = 0x00000800;
//开启窗口抖动
public static final int FLAG_DITHER = 0x00001000;
//安全内容窗口, 该窗口显示时不允许截屏
public static final int FLAG_SECURE = 0x00002000;
//锁屏时显示该窗口
public static final int FLAG_SHOW_WHEN_LOCKED = 0x00080000;
//系统的墙纸显示在该窗口之后
public static final int FLAG_SHOW_WALLPAPER = 0x00100000;
//当window被显示的时候, 系统将把它当做一个用户活动事件, 以点亮手机屏幕
public static final int FLAG_TURN_SCREEN_ON = 0x00200000;
//该窗口显示, 消失键盘
public static final int FLAG_DISMISS_KEYGUARD = 0x00400000;
//当该window在可以接受触摸屏情况下, 让因在该window之外, 而发送到后面的window的触摸屏可以支持split touch
public static final int FLAG_SPLIT_TOUCH = 0x00800000;
//对该window进行硬件加速, 该flag必须在Activity或Dialog的Content View之前进行设置
public static final int FLAG_HARDWARE_ACCELERATED = 0x01000000;
//让window占满整个手机屏幕, 不留任何边界
public static final int FLAG_LAYOUT_IN_OVERSCAN = 0x02000000;
//透明状态栏
public static final int FLAG_TRANSLUCENT_STATUS = 0x04000000;
//透明导航栏
public static final int FLAG_TRANSLUCENT_NAVIGATION = 0x08000000;
```

第二节 Android窗口管理服务WindowManagerService 计算Activity窗口大小的过程分析

在Android系统中，Activity窗口的大小是由WindowManagerService服务来计算的。WindowManagerService服务会根据屏幕及其装饰区的大小来决定Activity窗口的大小。一个Activity窗口只有知道自己的大小之后，才能对它里面的UI元素进行测量、布局以及绘制。本文将详细分析WindowManagerService服务计算Activity窗口大小的过程。

一般来说，Activity窗口的大小等于整个屏幕的大小，但是它并不占据着整块屏幕。为了理解这一点，我们首先分析一下Activity窗口的区域是如何划分的。

我们知道，Activity窗口的上方一般会有一个状态栏，用来显示3G信号、电量使用等图标，如图1所示。

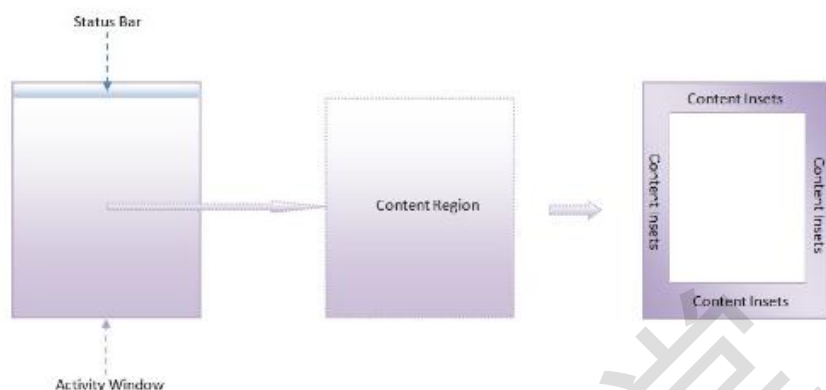


图1 Activity窗口的Content区域示意图

从Activity窗口剔除掉状态栏所占用的区域之后，所得到的区域就称为内容区域（Content Region）。顾名思义，内容区域就是用来显示Activity窗口的内容的。我们再抽象一下，假设Activity窗口的四周都有一块类似状态栏的区域，那么将这些区域剔除之后，得到中间的那一块区域就称为内容区域，而被剔除出来的区域所组成的区域就称为内容边衬区域（Content Insets）。Activity窗口的内容边衬区域可以用一个四元组（content-left, content-top, content-right, content-bottom）来描述，其中，content-left、content-right、content-top、content-bottom分别用来描述内容区域与窗口区域的左右上下边界距离。

我们还知道，Activity窗口有时候需要显示输入法窗口，如图2所示。

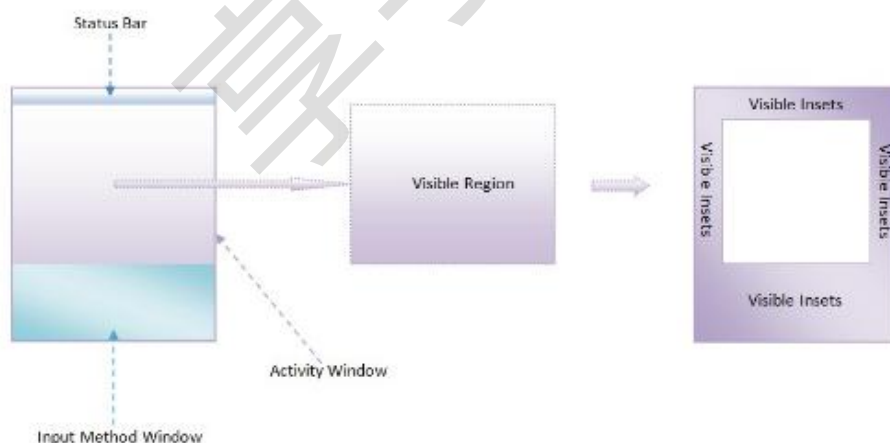


图2 Activity窗口的Visible区域示意图

这时候Activity窗口的内容区域的大小有可能没有发生变化，这取决于它的Soft Input Mode。我们假设Activity窗口的内容区域没有发生变化，但是它在底部的一些区域被输入法窗口遮挡了，即它在底部的一些内容是不可见的。从Activity窗口剔除掉状态栏和输入法窗口所占用的区域之后，所得到的区域就称为可见区域（Visible Region）。同样，我们再抽象一下，假设Activity窗口的四周都有一块类似状态栏和输入法窗口的区域，那么将这些区域剔除之后，得到中间的那一块区域就称为可见区域，而被剔除出来的区域所组成的区域就称为可见边衬区域（Visible Insets）。Activity窗口的可见边衬区域可以用一个四元组（visible-left, visible-top, visible-right, visible-bottom）来描述，其中，visible-left、visible-right、visible-top、visible-bottom分别用来描述可见区域与窗口区域的左右上下边界距离。

在大多数情况下，Activity窗口的内容区域和可见区域的大小是一致的，而状态栏和输入法窗口所占用的区域又称为屏幕装饰区。理解了这些概念之后，我们就可以推断，WindowManagerService服务实际上就是需要根据屏幕以及可能出现的状态栏和输入法窗口的大小来计算出Activity窗口的整体大小及其内容区域边衬和可见区域边衬的大小。有了

这三个数据之后，Activity窗口就可以对它里面的UI元素进行测量、布局以及绘制等操作了。

从前面Android应用程序窗口（Activity）的绘图表面（Surface）的创建过程分析一文可以知道，应用程序进程是从ViewRoot类的成员函数performTraversals开始，向WindowManagerService服务请求计算一个Activity窗口的大小的，因此，接下来我们就从ViewRoot类的成员函数performTraversals开始分析一个Activity窗口大小的计算过程，如图3所示。

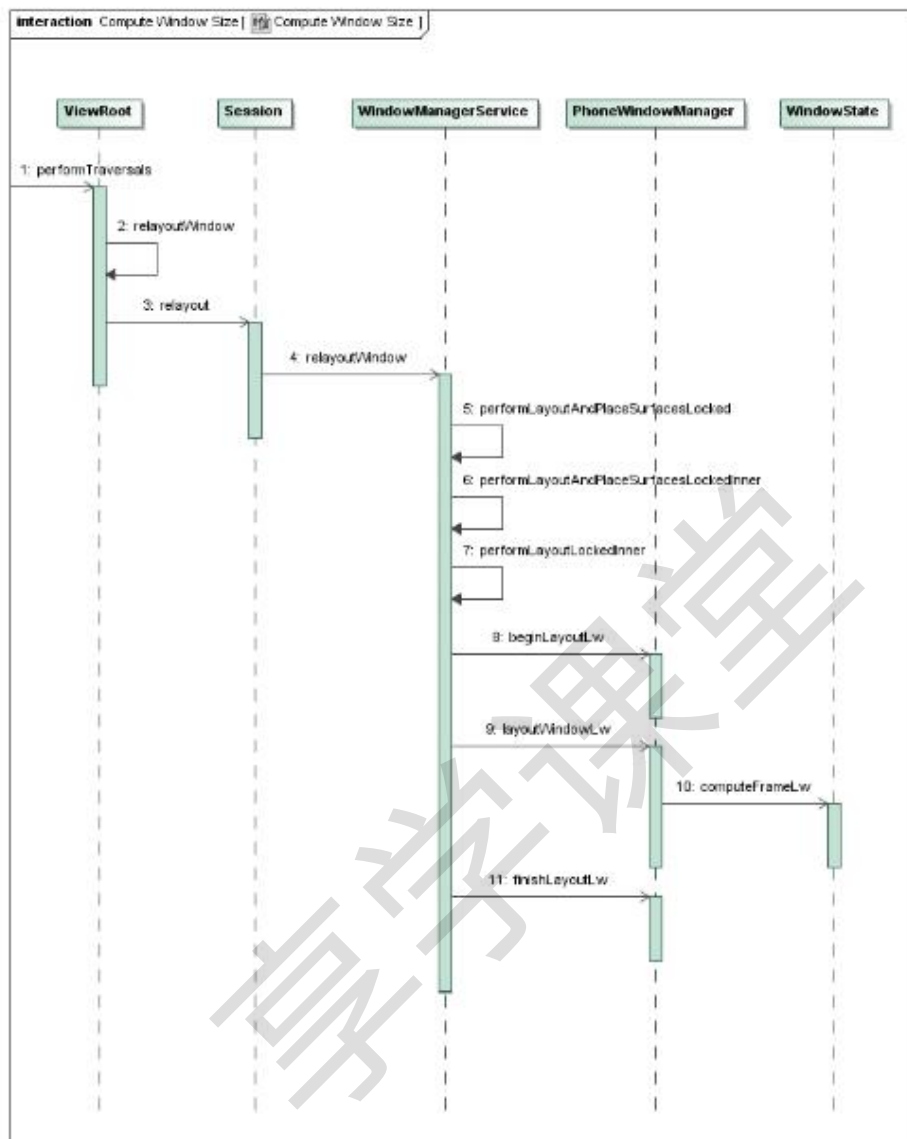


图3 Activity窗口大小的计算过程

这个过程可以分为11个步骤，接下来我们就详细分析每一个步骤。

Step 1. ViewRoot.performTraversals

这个函数定义在文件frameworks/base/core/java/android/view/ViewRoot.java中，它的实现很复杂，一共有600+行，不过大部分代码都是用来计算Activity窗口的大小的，我们分段来阅读：

```
public final class ViewRoot extends Handler implements ViewParent,
    View.AttachInfo.Callbacks {
    .....

```

```
private void performTraversals() {
    .....

    final View host = mView;
    .....

```

```

int desiredWindowWidth;
int desiredWindowHeight;
int childWidthMeasureSpec;
int childHeightMeasureSpec;
.....

Rect frame = mWinFrame;
if (mFirst) {
    .....

    DisplayMetrics packageMetrics =
        mView.getContext().getResources().getDisplayMetrics();
    desiredWindowWidth = packageMetrics.widthPixels;
    desiredWindowHeight = packageMetrics.heightPixels;
} else {
    desiredWindowWidth = frame.width();
    desiredWindowHeight = frame.height();
    if (desiredWindowWidth != mWidth || desiredWindowHeight != mHeight) {
        .....
        windowResizesToFitContent = true;
    }
}
}

```

这段代码用来获得Activity窗口的当前宽度desiredWindowWidth和当前高度desiredWindowHeight。注意，Activity窗口当前的宽度和高度是保存ViewRoot类的成员变量mWinFrame中的。ViewRoot类的另外两个成员变量mWidth和mHeight也是用来描述Activity窗口当前的宽度和高度的，但是它们的值是由应用程序进程上一次主动请求WindowManagerService服务计算得到的，并且会一直保持不变到应用程序进程下一次再请求WindowManagerService服务来重新计算为止。Activity窗口的当前宽度和高度有时候是被WindowManagerService服务主动请求应用程序进程修改的，修改后的值就会保存在ViewRoot类的成员变量mWinFrame中，它们可能会与ViewRoot类的成员变量mWidth和mHeight的值不同。

如果Activity窗口是第一次被请求执行测量、布局和绘制操作，即ViewRoot类的成员变量mFirst的值等于true，那么它的当前宽度desiredWindowWidth和当前高度desiredWindowHeight就等于屏幕的宽度和高度，否则的话，它的当前宽度desiredWindowWidth和当前高度desiredWindowHeight就等于保存在ViewRoot类的成员变量mWinFrame中的宽度和高度值。

如果Activity窗口不是第一次被请求执行测量、布局和绘制操作，并且Activity窗口主动上一次请求WindowManagerService服务计算得到的宽度mWidth和高度mHeight不等于Activity窗口的当前宽度desiredWindowWidth和当前高度desiredWindowHeight，那么就说明Activity窗口的大小发生了变化，这时候变量windowResizesToFitContent的值就会被标记为true，以便接下来可以对Activity窗口的大小变化进行处理。

我们继续往下阅读代码：

```

boolean insetsChanged = false;

if (mLayoutRequested) {
    .....

    if (mFirst) {
        host.fitSystemWindows(mAttachInfo.mContentInsets);
        .....
    } else {
        if (!mAttachInfo.mContentInsets.equals(mPendingContentInsets)) {
            mAttachInfo.mContentInsets.set(mPendingContentInsets);
            host.fitSystemWindows(mAttachInfo.mContentInsets);
            insetsChanged = true;
            .....
        }
        if (!mAttachInfo.mVisibleInsets.equals(mPendingVisibleInsets)) {
            mAttachInfo.mVisibleInsets.set(mPendingVisibleInsets);
            .....
        }
        if (lp.width == ViewGroup.LayoutParams.WRAP_CONTENT
            || lp.height == ViewGroup.LayoutParams.WRAP_CONTENT) {
            windowResizesToFitContent = true;

            DisplayMetrics packageMetrics =
                mView.getContext().getResources().getDisplayMetrics();
            desiredWindowWidth = packageMetrics.widthPixels;
            desiredWindowHeight = packageMetrics.heightPixels;
        }
    }
}
}

```

```

        childWidthMeasureSpec = getRootMeasureSpec(desiredWindowWidth, lp.width);
        childHeightMeasureSpec = getRootMeasureSpec(desiredWindowHeight, lp.height);
        .....

        host.measure(childWidthMeasureSpec, childHeightMeasureSpec);

        .....
    }

```

这段代码用来在Activity窗口主动请求WindowManagerService服务计算大小之前，对它的顶层视图进行一次测量操作。在分析这段代码之前，我们首先解释一下ViewRoot类的成员变量mAttachInfo和mPendingContentInsets、mPendingVisibleInsets。ViewRoot类的成员变量mAttachInfo指向的一个AttachInfo对象，这个AttachInfo对象用来描述Activity窗口的属性，例如，这个AttachInfo对象的成员变量mContentInsets和mVisibleInsets分别用来描述Activity窗口上一次主动请求WindowManagerService服务计算得到的内容边衬大小和可见边衬大小，即Activity窗口的当前内容边衬大小和可见边衬大小。ViewRoot类的成员变量mPendingContentInsets和mPendingVisibleInsets也是用来描述Activity窗口的内容边衬大小和可见边衬大小的，不过它们是由WindowManagerService服务主动请求Activity窗口设置的，但是尚未生效。

我们分两种情况分析这段代码。

第一种情况是Activity窗口是第一次被请求执行测量、布局和绘制操作，即ViewRoot类的成员变量mFirst的值等于true，那么这段代码在测量Activity窗口的顶层视图host的大小之前，首先会调用这个顶层视图host的成员函数fitSystemWindows来设置它的四个内边距（mPaddingLeft，mPaddingTop，mPaddingRight，mPaddingBottom）的大小设置为Activity窗口的初始化内容边衬大小。这样做的目的是可以在Activity窗口的四周留下足够的区域来放置可能会出现出现的系统窗口，也就是状态栏和输入法窗口。

第二种情况是Activity窗口不是第一次被请求执行测量、布局和绘制操作，即ViewRoot类的成员变量mFirst的值等于false，那么这段代码就会检查Activity窗口是否被WindowManagerService服务主动请求设置了一个新的内容边衬大小mPendingContentInsets和一个新的可见边衬大小mPendingVisibleInsets。如果是的话，那么就会分别将它们保存在ViewRoot类的成员变量mAttachInfo所指向的一个AttachInfo对象的成员变量mContentInsets和成员变量mVisibleInsets中。注意，如果Activity窗口被WindowManagerService服务主动请求设置了一个新的内容边衬大小mPendingContentInsets，那么这段代码同时还需要同步调用Activity窗口的顶层视图host的成员函数fitSystemWindows来将它的四个内边距（mPaddingLeft，mPaddingTop，mPaddingRight，mPaddingBottom）的大小设置为新的内容边衬大小，并且将变量insetsChanged的值设置为true，表明Activity窗口的内容边衬大小发生了变化。

在第二种情况下，如果Activity窗口的宽度被设置为ViewGroup.LayoutParams.WRAP_CONTENT或者高度被设置为ViewGroup.LayoutParams.WRAP_CONTENT，那么就意味着Activity窗口的大小要等于内容区域的大小。但是由于Activity窗口的大小是需要覆盖整个屏幕的，因此，这时候就会Activity窗口的当前宽度desiredWindowWidth和当前高度desiredWindowHeight设置为屏幕的宽度和高度。也就是说，如果我们将Activity窗口的宽度和高度设置为ViewGroup.LayoutParams.WRAP_CONTENT，实际上就意味着它的宽度和高度等于屏幕的宽度和高度。这种情况也意味着Activity窗口的大小发生了变化，因此，就将变量windowResizesToFitContent的值设置为true。

经过上面的一系列处理之后，这段代码就会调用ViewRoot类的成员函数getRootMeasureSpec来根据Activity窗口的当前宽度和宽度测量规范以及高度和高度测量规范来计算得到它的顶层视图host的宽度测量规范childWidthMeasureSpec和高度测量规范childHeightMeasureSpec。有了这两个规范之后，就可以调用Activity窗口的顶层视图host的成员函数measure来执行大小测量的工作了。这个大小测量的过程可以参考前面Android应用程序窗口（Activity）的测量（Measure）、布局（Layout）和绘制（Draw）过程分析一文。

我们继续往下阅读代码：

```

boolean windowShouldResize = mLayoutRequested && windowResizesToFitContent
    && ((mWidth != host.mMeasuredWidth || mHeight != host.mMeasuredHeight)
        || (lp.width == ViewGroup.LayoutParams.WRAP_CONTENT &&
            frame.width() < desiredWindowWidth && frame.width() != mWidth)
        || (lp.height == ViewGroup.LayoutParams.WRAP_CONTENT &&
            frame.height() < desiredWindowHeight && frame.height() != mHeight));

final boolean computesInternalInsets =
    attachInfo.mTreeObserver.hasComputeInternalInsetsListeners();

```

这段代码主要是做两件事情。

第一件事情是检查是否需要处理Activity窗口的大小变化事件。如果满足以下条件，那么就需要处理，即将变量windowShouldResize的值设置为true：

1. ViewRoot类的成员变量mLayoutRequest的值等于true，这说明应用程序进程正在请求对Activity窗口执行一次测量、布局和绘制操作；

2. 变量windowResizesToFitContent的值等于true，这说明前面检测到了Activity窗口的大小发生了变化；

3. 前面我们已经Activity窗口的顶层视图host的大小重新进行了测量。如果测量出来的宽度host.mMeasuredWidth和高度host.mMeasuredHeight和Activity窗口的当前宽度mWidth和高度mHeight一样，那么即使条件1和条件2能满足，那么也是可以认为是Activity窗口的大小是没有发生变化的。换句话说，只有当测量出来的大小和当前大小不一致时，才认为Activity窗口大小发生了变化。另一方面，如果测量出来的大小和当前大小一致，但是Activity窗口的大小被

要求设置成WRAP_CONTENT，即设置成和屏幕的宽度desiredWindowWidth和高度desiredWindowHeight一致，但是WindowManagerService服务请求Activity窗口设置的宽度frame.width()和高度frame.height()与它们不一致，而且与Activity窗口上一次请求WindowManagerService服务计算的宽度mWidth和高度mHeight也不一致，那么也是认为Activity窗口大小发生了变化的。

第二件事情是检查Activity窗口是否需要指定有额外的内容边衬区域和可见边衬区域。如果有的话，那么变量attachInfo所指向的一个AttachInfo对象的成员变量mTreeObserver所描述的一个TreeObserver对象的成员函数hasComputeInternalInsetsListener的返回值ComputeInternalInsets就会等于true。Activity窗口指定额外的内容边衬区域和可见边衬区域是为了放置一些额外的东西。

我们继续往下阅读代码：

```
if (mFirst || windowShouldResize || insetsChanged
    || viewVisibilityChanged || params != null) {

    if (viewVisibility == View.VISIBLE) {
        // If this window is giving internal insets to the window
        // manager, and it is being added or changing its visibility,
        // then we want to first give the window manager "fake"
        // insets to cause it to effectively ignore the content of
        // the window during layout. This avoids it briefly causing
        // other windows to resize/move based on the raw frame of the
        // window, waiting until we can finish laying out this window
        // and get back to the window manager with the ultimately
        // computed insets.
        insetsPending = computesInternalInsets
            && (mFirst || viewVisibilityChanged);

        .....
    }
}
```

这段代码以及接下来的两段代码都是在满足下面的条件之一的情况下执行的：1. Activity窗口是第一次执行测量、布局和绘制操作，即ViewRoot类的成员变量mFirst的值等于true。2. 前面得到的变量windowShouldResize的值等于true，即Activity窗口的大小的确是发生了变化。3. 前面得到的变量insetsChanged的值等于true，即Activity窗口的内容区域边衬发生了变化。4. Activity窗口的可见性发生了变化，即变量viewVisibilityChanged的值等于true。5. Activity窗口的属性发生了变化，即变量params指向了一个WindowManager.LayoutParams对象。

在满足上述条件之一，并且Activity窗口处于可见状态，即变量viewVisibility的值等于View.VISIBLE，那么就需要检查接下来请求WindowManagerService服务计算大小时，是否要告诉WindowManagerService服务它指定了额外的内容区域边衬和可见区域边衬，但是这些额外的内容区域边衬和可见区域边衬又还有待确定。这种情况发生在Activity窗口第一次执行测量、布局和绘制操作或者由不可见变化可见时。因此，当前面得到的变量computesInternalInsets等于true时，即Activity窗口指定了额外的内容区域边衬和可见区域边衬，那么就需要检查ViewRoot类的成员变量mFirst或者变量viewVisibilityChanged的值是否等于true。如果这些条件能满足，那么变量insetsPending的值就会等于true，表示Activity窗口有额外的内容区域边衬和可见区域边衬等待指定。

我们继续往下阅读代码：

```
boolean contentInsetsChanged = false;
boolean visibleInsetsChanged;
.....
try {
    .....
    relayLayoutResult = relayLayoutWindow(params, viewVisibility, insetsPending);

    contentInsetsChanged = !mPendingContentInsets.equals(
        mAttachInfo.mContentInsets);
    visibleInsetsChanged = !mPendingVisibleInsets.equals(
        mAttachInfo.mVisibleInsets);
    if (contentInsetsChanged) {
        mAttachInfo.mContentInsets.set(mPendingContentInsets);
        host.fitSystemWindows(mAttachInfo.mContentInsets);
        .....
    }
    if (visibleInsetsChanged) {
        mAttachInfo.mVisibleInsets.set(mPendingVisibleInsets);
        .....
    }

    .....
} catch (RemoteException e) {
}
```

.....

```
attachInfo.mWindowLeft = frame.left;
attachInfo.mWindowTop = frame.top;

// !!FIXME!! This next section handles the case where we did not get the
// window size we asked for. We should avoid this by getting a maximum size from
// the window session beforehand.
mWidth = frame.width();
mHeight = frame.height();
```

这段代码主要就是调用ViewRoot类的另外一个成员函数relayoutWindow来请求WindowManagerService服务计算Activity窗口的大小以及内容区域边衬大小和可见区域边衬大小。计算完毕之后，Activity窗口的大小就会保存在ViewRoot类的成员变量mWinFrame中，而Activity窗口的内容区域边衬大小和可见区域边衬大小分别保存在ViewRoot类的成员变量mPendingContentInsets和mPendingVisibleInsets中。如果这次计算得到的Activity窗口的内容区域边衬大小mPendingContentInsets和可见区域边衬大小mPendingVisibleInsets与上一次计算得到的不一致，即与ViewRoot类的成员变量mAttachInfo所指向的一个AttachInfo对象的成员变量mContentInsets和mVisibleInsets所描述的大小不一致，那么变量contentInsetsChanged和visibleInsetsChanged的值就会等于true，表示Activity窗口的内容区域边衬大小和可见区域边衬大小发生了变化。

由于变量frame和ViewRoot类的成员变量mWinFrame引用的是同一个Rect对象，因此，这时候变量frame描述的也是Activity窗口请求WindowManagerService服务计算之后得到的大小。这段代码分别将计算得到的Activity窗口的左上角坐标保存在变量attachInfo所指向的一个AttachInfo对象的成员变量mWindowLeft和mWindowTop中，并且将计算得到的Activity窗口的宽度和高度保存在ViewRoot类的成员变量mWidth和mHeight中。

我们继续往下阅读代码：

```
boolean focusChangedDueToTouchMode = ensureTouchModeLocally(
    (relayoutResult&WindowManagerImpl.RELAYOUT_IN_TOUCH_MODE) != 0);
if (focusChangedDueToTouchMode || mWidth != host.mMeasuredWidth
    || mHeight != host.mMeasuredHeight || contentInsetsChanged) {
    childWidthMeasureSpec = getRootMeasureSpec(mWidth, lp.width);
    childHeightMeasureSpec = getRootMeasureSpec(mHeight, lp.height);
    .....

    // Ask host how big it wants to be
    host.measure(childWidthMeasureSpec, childHeightMeasureSpec);

    // Implementation of weights from WindowManager.LayoutParams
    // We just grow the dimensions as needed and re-measure if
    // needs be
    int width = host.mMeasuredWidth;
    int height = host.mMeasuredHeight;
    boolean measureAgain = false;

    if (lp.horizontalWeight > 0.0f) {
        width += (int) ((mWidth - width) * lp.horizontalWeight);
        childWidthMeasureSpec = MeasureSpec.makeMeasureSpec(width,
            MeasureSpec.EXACTLY);
        measureAgain = true;
    }
    if (lp.verticalWeight > 0.0f) {
        height += (int) ((mHeight - height) * lp.verticalWeight);
        childHeightMeasureSpec = MeasureSpec.makeMeasureSpec(height,
            MeasureSpec.EXACTLY);
        measureAgain = true;
    }

    if (measureAgain) {
        .....
        host.measure(childWidthMeasureSpec, childHeightMeasureSpec);
    }

    mLayoutRequested = true;
}
}
```

这段代码用来检查是否需要重新测量Activity窗口的大小。如果满足以下条件之一，那么就需要重新测量： 1. Activity窗口的触摸模式发生了变化，并且由此引发了Activity窗口当前获得焦点的控件发生了变化，即变量focusChangedDueToTouchMode的值等于true。这个检查是通过调用ViewRoot类的成员函数ensureTouchModeLocally来

实现的。

2. Activity窗口前面测量出来的宽度host.mMeasuredWidth和高度host.mMeasuredHeight不等于WindowManagerService服务计算出来的宽度mWidth和高度mHeight。

3. Activity窗口的内容区域边衬大小和可见区域边衬大小发生了变化，即前面得到的变量contentInsetsChanged的值等于true。

重新计算了一次之后，如果Activity窗口的属性lp表明需要对测量出来的宽度width和高度height进行扩展，即变量lp所指向的一个WindowManager.LayoutParams对象的成员变量horizontalWeight和verticalWeight的值大于0.0，那么就需要对Activity窗口的顶层视图host的最大可用空间进行扩展后再进行一次测量工作。

我们继续往下阅读最后一段代码：

```
final boolean didLayout = mLayoutRequested;
.....
if (didLayout) {
    .....
    host.layout(0, 0, host.mMeasuredWidth, host.mMeasuredHeight);
    .....
}

if (computesInternalInsets) {
    ViewTreeObserver.InternalInsetsInfo insets = attachInfo.mGivenInternalInsets;
    final Rect givenContent = attachInfo.mGivenInternalInsets.contentInsets;
    final Rect givenVisible = attachInfo.mGivenInternalInsets.visibleInsets;
    givenContent.left = givenContent.top = givenContent.right
        = givenContent.bottom = givenVisible.left = givenVisible.top
        = givenVisible.right = givenVisible.bottom = 0;
    attachInfo.mTreeObserver.dispatchOnComputeInternalInsets(insets);
    Rect contentInsets = insets.contentInsets;
    Rect visibleInsets = insets.visibleInsets;
    if (mTranslator != null) {
        contentInsets = mTranslator.getTranslatedContentInsets(contentInsets);
        visibleInsets = mTranslator.getTranslatedVisibleInsets(visibleInsets);
    }
    if (insetsPending || !mLastGivenInsets.equals(insets)) {
        mLastGivenInsets.set(insets);
        try {
            sWindowSession.setInsets(mWindow, insets.mTouchableInsets,
                contentInsets, visibleInsets);
        } catch (RemoteException e) {}
    }
}
.....
}

.....
}
```

经过前面漫长的操作后，Activity窗口的大小测量工作终于尘埃落定，这时候就可以对Activity窗口的内容进行布局了，前提是ViewRoot类的成员变量mLayoutRequest的值等于true。对Activity窗口的内容进行布局是通过调用它的顶层视图host的成员函数layout来实现的，这个过程可以参考前面 Android应用程序窗口（Activity）的测量（Measure）、布局（Layout）和绘制（Draw）过程分析一文。从前面的描述可以知道，当变量computesInternalInsets的值等于true时，就表示Activity窗口指定有额外的内容区域边衬和可见区域边衬，这时候就是时候把它们告诉给WindowManagerService服务了，以便WindowManagerService服务下次可以知道Activity窗口的真实布局。Activity窗口额外指定的内容区域边衬大小和可见区域边衬大小是通过调用变量attachInfo所指向的一个AttachInfo对象的成员变量mTreeObserver所描述的一个TreeObserver对象的成员函数dispatchOnComputeInternalInsets来计算的。计算完成之后，就会保存在变量attachInfo所指向的一个AttachInfo对象的成员变量mGivenInternalInsets中，并且会通过ViewRoot类的静态成员变量sWindowSession所指向一个Binder代理对象来设置到WindowManagerService服务中去。

注意，如果ViewRoot类的成员变量mTranslator指向了一个Translator对象，那么就说明Activity窗口是运行兼容模式中，这时候就需要将前面计算得到的内容区域边衬大小和可见区域边衬大小转化到兼容模式下，然后才可以保存在变量attachInfo所指向的一个AttachInfo对象的成员变量mGivenInternalInsets中，以及设置到WindowManagerService服务中去。

另外，只有前面得到的变量insetsPending的值等于true，即Activity窗口正在等待告诉WindowManagerService服务它有额外指定的内容区域边衬和可见区域边衬，或者Activity窗口额外指定的内容区域边衬和可见区域边衬发生了变化，即Activity窗口上一次额外指定的内容区域边衬和可见区域边衬mLastGivenInsets不等于当前这次指定的内容区域

边衬和可见区域边衬insets, Activity窗口额外指定的内容区域边衬和可见区域边衬才会被设置到WindowManagerService服务中去。

ViewRoot类的成员函数再接下来的工作就是绘制Activity窗口的UI了, 这个过程同样可以参考前面Android应用程序窗口(Activity)的测量(Measure)、布局(Layout)和绘制(Draw)过程分析一文。

接下来, 我们继续分析ViewRoot类的成员函数relayoutWindow的实现, 以便可以了解它是如何请求WindowManagerService服务计算Activity窗口的大小的。

```
Step 2. ViewRoot.relayoutWindow
public final class ViewRoot extends Handler implements ViewParent,
    View.AttachInfo.Callbacks {
    .....

    private int relayoutWindow(WindowManager.LayoutParams params, int viewVisibility,
        boolean insetsPending) throws RemoteException {

        float appScale = mAttachInfo.mApplicationScale;
        .....

        int relayoutResult = sWindowSession.relayout(
            mWindow, params,
            (int) (mView.mMeasuredWidth * appScale + 0.5f),
            (int) (mView.mMeasuredHeight * appScale + 0.5f),
            viewVisibility, insetsPending, mWinFrame,
            mPendingContentInsets, mPendingVisibleInsets,
            mPendingConfiguration, mSurface);
        .....

        if (mTranslator != null) {
            mTranslator.translateRectInScreenToAppWinFrame(mWinFrame);
            mTranslator.translateRectInScreenToAppWindow(mPendingContentInsets);
            mTranslator.translateRectInScreenToAppWindow(mPendingVisibleInsets);
        }
        return relayoutResult;
    }
    .....
}
```

这个函数定义在文件frameworks/base/core/java/android/view/ViewRoot.java中。从前面Android应用程序窗口(Activity)与WindowManagerService服务的连接过程分析一文可以知道, ViewRoot类的静态成员变量sWindowSession是一个Binder代理对象, 它引用了运行在WindowManagerService服务这一侧的一个Session对象, ViewRoot类的成员函数relayoutWindow通过调用这个Session对象的成员函数relayout来请求WindowManagerService服务计算Activity窗口的大小, 其中, 传递给WindowManagerService服务的参数包括:

1. ViewRoot类的成员变量mWindow, 用来标志要计算的是哪一个Activity窗口的大小。
2. Activity窗口的顶层视图经过测量后得到的宽度和高度。注意, 传递给WindowManagerService服务的宽度和高度是已经考虑了Activity窗口所设置的缩放因子了的。
3. Activity窗口的可见状态, 即参数viewVisibility。
4. Activity窗口是否有额外的内容区域边衬和可见区域边衬等待告诉给WindowManagerService服务, 即参数insetsPending。
5. ViewRoot类的成员变量mWinFrame, 这是一个输出参数, 用来保存WindowManagerService服务计算后得到的Activity窗口的大小。
6. ViewRoot类的成员变量mPendingContentInsets, 这是一个输出参数, 用来保存WindowManagerService服务计算后得到的Activity窗口的内容区域边衬大小。
7. ViewRoot类的成员变量mPendingVisibleInsets, 这是一个输出参数, 用来保存WindowManagerService服务计算后得到的Activity窗口的可见区域边衬大小。
8. ViewRoot类的成员变量mPendingConfiguration, 这是一个输出参数, 用来保存WindowManagerService服务返回来的Activity窗口的配置信息。
9. ViewRoot类的成员变量mSurface, 这是一个输出参数, 用来保存WindowManagerService服务返回来的Activity窗口的绘图表面。

得到了Activity窗口的大小以及内容区域边衬大小和可见区域边衬大小之后, 如果Activity窗口是运行在兼容模式中, 即ViewRoot类的成员变量mTranslator指向了一个Translator对象, 那么就需要调用它的成员函数

translateRectInScreenToAppWindow来对它们进行转换。

接下来，我们继续分析Session类的成员函数relayout，以便可以了解WindowManagerService服务是如何计算一个Activity窗口的大小的。

Step 3. Session.relayout

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class Session extends IWindowSession.Stub
        implements IBinder.DeathRecipient {
        .....

        public int relayout(IWindow window, WindowManager.LayoutParams attrs,
            int requestedWidth, int requestedHeight, int viewFlags,
            boolean insetsPending, Rect outFrame, Rect outContentInsets,
            Rect outVisibleInsets, Configuration outConfig, Surface outSurface) {
            //Log.d(TAG, ">>>>> ENTERED relayout from " + Binder.getCallingPid());
            int res = relayoutWindow(this, window, attrs,
                requestedWidth, requestedHeight, viewFlags, insetsPending,
                outFrame, outContentInsets, outVisibleInsets, outConfig, outSurface);
            //Log.d(TAG, "<<<<< EXITING relayout to " + Binder.getCallingPid());
            return res;
        }

        .....
    }

    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。Session类的成员函数relayout的实现很简单，它只是调用了WindowManagerService类的成员函数relayoutWindow来进一步计算参数window所描述的一个Activity窗口的大小，接下来我们就继续分析WindowManagerService类的成员函数relayoutWindow的实现。

Step 4. WindowManagerService.relayoutWindow

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int relayoutWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int requestedWidth,
        int requestedHeight, int viewVisibility, boolean insetsPending,
        Rect outFrame, Rect outContentInsets, Rect outVisibleInsets,
        Configuration outConfig, Surface outSurface) {
        .....

        synchronized(mWindowMap) {
            WindowState win = windowForClientLocked(session, client, false);
            .....

            win.mRequestedWidth = requestedWidth;
            win.mRequestedHeight = requestedHeight;
            .....

            final boolean scaledWindow =
                ((win.mAttrs.flags & WindowManager.LayoutParams.FLAG_SCALED) != 0);

            if (scaledWindow) {
                // requested{Width|Height} Surface's physical size
                // attrs.{width|height} Size on screen
                win.mHScale = (attrs.width != requestedWidth) ?
                    (attrs.width / (float)requestedWidth) : 1.0f;
                win.mVScale = (attrs.height != requestedHeight) ?
                    (attrs.height / (float)requestedHeight) : 1.0f;
            } else {
                win.mHScale = win.mVScale = 1;
            }
        }
    }
}
```



```

.....

win.mGivenInsetsPending = insetsPending;

.....

performLayoutAndPlaceSurfacesLocked();

.....

outFrame.set(win.mFrame);
outContentInsets.set(win.mContentInsets);
outVisibleInsets.set(win.mVisibleInsets);

.....
}

return (inTouchMode ? WindowManagerImpl.RELAYOUT_IN_TOUCH_MODE : 0)
       | (displayed ? WindowManagerImpl.RELAYOUT_FIRST_TIME : 0);
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

参数client是一个Binder代理对象，它引用了运行在应用程序进程这一侧中的一个W对象，用来标志一个Activity窗口。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，在应用程序进程这一侧的每一个W对象，在WindowManagerService服务这一侧都有一个对应的WindowState对象，用来描述一个Activity窗口的状态。因此，WindowManagerService类的成员函数relayoutWindow首先通过调用另外一个成员函数windowForClientLocked来获得与参数client所对应的一个WindowState对象win，以便接下来可以对它进行操作。

本文我们只关注WindowManagerService类的成员函数relayoutWindow中与窗口大小计算有关的逻辑，计算过程如下所示：

1. 参数requestedWidth和requestedHeight描述的是应用程序进程请求设置Activity窗口中的宽度和高度，它们会被记录在WindowState对象win的成员变量mRequestedWidth和mRequestedHeight中。
2. WindowState对象win的成员变量mAttr，它指向的是一个WindowManager.LayoutParams对象，用来描述Activity窗口的布局参数。其中，这个WindowManager.LayoutParams对象的成员变量width和height是用来描述Activity窗口的宽度和高度的。当这个WindowManager.LayoutParams对象的成员变量flags的WindowManager.LayoutParams.FLAG_SCALED位不等于0的时候，就说明需要给Activity窗口的大小设置缩放因子。缩放因子分为两个维度，分别是宽度缩放因子和高度缩放因子，保存在WindowState对象win的成员变量HScale和VScale中，计算方法分别是用应用程序进程请求设置Activity窗口中的宽度和高度除以Activity窗口在布局参数中所设置的宽度和高度。
3. 参数insetsPending用来描述Activity窗口是否有额外的内容区域边衬和可见区域边衬未设置，它被记录在WindowState对象win的成员变量mGivenInsetsPending中。
4. 调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来计算Activity窗口的大小。计算完成之后，参数client所描述的Activity窗口的大小、内容区域边衬大小和可见区域边衬大小就会分别保存在WindowState对象win的成员变量mFrame、mContentInsets和mVisibleInsets中。
5. 将WindowState对象win的成员变量mFrame、mContentInsets和mVisibleInsets的值分别拷贝到参数出数outFrame、outContentInsets和outVisibleInsets中，以便可以返回给应用程序进程。

经过上述五个操作后，Activity窗口的大小计算过程就完成了，接下来我们继续分析WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked的实现，以便可以详细了解Activity窗口的大小计算过程。

Step 5. WindowManagerService.performLayoutAndPlaceSurfacesLocked

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
.....

private final void performLayoutAndPlaceSurfacesLocked() {

```

```

        if (mInLayout) {
            .....
            return;
        }

        .....

        boolean recoveringMemory = false;
        if (mForceRemoves != null) {
            recoveringMemory = true;
            // Wait a little it for things to settle down, and off we go.
            for (int i=0; i<mForceRemoves.size(); i++) {
                WindowState ws = mForceRemoves.get(i);
                Slog.i(TAG, "Force removing: " + ws);
                removeWindowInnerLocked(ws.mSession, ws);
            }
            mForceRemoves = null;
            .....
        }

        mInLayout = true;
        try {
            performLayoutAndPlaceSurfacesLockedInner(recoveringMemory);

            int i = mPendingRemove.size()-1;
            if (i >= 0) {
                while (i >= 0) {
                    WindowState w = mPendingRemove.get(i);
                    removeWindowInnerLocked(w.mSession, w);
                    i--;
                }
                mPendingRemove.clear();

                mInLayout = false;
                assignLayersLocked();
                mLayoutNeeded = true;
                performLayoutAndPlaceSurfacesLocked();
            } else {
                mInLayout = false;
                .....
            }
            .....
        } catch (RuntimeException e) {
            mInLayout = false;
            .....
        }
    }

    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。从WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked的名称可以推断出，它执行的操作绝非是计算窗口大小这么简单。计算窗口大小只是其中的一个小小功能点，它主要的功能是用来刷新系统的UI。在我们这个情景中，为什么需要刷新系统的UI呢？Activity窗口在其属性发生了变化，例如，可见性、大小发生了变化，又或者它新增、删除了子视图，都需要重新计算大小，而这些变化都是要求WindowManagerService服务重新刷新系统的UI的。事实上，刷新系统的UI是WindowManagerService服务的主要任务，在新增和删除了窗口、窗口动画显示过程、窗口切换过程中，WindowManagerService服务都需要不断地刷新系统的UI。

WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked主要是通过调用另外一个成员函数performLayoutAndPlaceSurfacesLockedInner来刷新系统的UI的，而在刷新的过程中，就会对系统中的各个窗口的大小进行计算。

在调用成员函数performLayoutAndPlaceSurfacesLockedInner来刷新系统UI的前后，WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked还会执行以下两个操作：

1. 调用前，检查系统中是否存在强制删除的窗口。有内存不足的情况下，有一些窗口就会被回收，即要从系统中删除，这些窗口会保存在WindowManagerService类的成员变量mForceRemoves所描述的一个ArrayList中。如果存在这些窗口，那么

WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked就会调用另外一个成员函数removeWindowInnerLocked来删除它们，以便可以回收它们所占用的内存。

2. 调用后，检查系统中是否有窗口需要移除。如果有的话，那么WindowManagerService类的成员变量mPendingRemove所描述的一个ArrayList的大小就会大于0。这种情况下，WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked就会调用另外一个成员函数removeWindowInnerLocked来移除这些窗口。注意，WindowManagerService类的成员函数removeWindowInnerLocked只是用来移除窗口，但是并没有回收这些窗口所占用的内存。等到合适的时候，例如，内存不足时，才会考虑回收这些窗口所占用的内存。移除一个窗口的操作也是很复杂的，除了要将窗口从WindowManagerService类的相关成员变量中移除之外，还要考虑重新调整输入法窗口和壁纸窗口，因为被移除的窗口可能要求显示壁纸和输入法窗口，当它被移除之后，就要将壁纸窗口和输入法窗口调整到合适的Z轴位置上去，以便可以交给下一个需要显示壁纸和输入法窗口的窗口使用。此外，在移除了窗口之后，WindowManagerService服务还需要重新计算现存的其它窗口的Z轴位置，以便可以正确地反映系统当前的UI状态，这是通过调用WindowManagerService类的成员函数assignLayersLocked来实现的。重新计算了现存的其它窗口的Z轴位置之后，又需要再次刷新系统的UI，即要对WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked进行递归调用，并且在调用前，将WindowManagerService类的成员变量mLayoutNeeded的值设置为true。由此可见，系统UI的刷新过程是非常复杂的。

注意，为了防止在刷新系统UI的过程中被重复调用，WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked在刷新系统UI之前，即调用成员函数performLayoutAndPlaceSurfacesLockedInner之前，会将WindowManagerService类的成员变量mInLayout的值设置为true，并且在调用之后，重新将这个成员变量的值设置为false。这样，WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked就可以在一开始的时候检查成员变量mInLayout的值是否等于true，如果等于的话，那么就说明WindowManagerService服务正在刷新系统UI的过程中，于是就不用往下执行了。

接下来，我们就继续分析WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner的实现，以便可以了解Activity窗口的大小计算过程。

```
Step 6. WindowManagerService.performLayoutAndPlaceSurfacesLockedInner
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final void performLayoutAndPlaceSurfacesLockedInner(
        boolean recoveringMemory) {
        .....

        Surface.openTransaction();
        .....

        try {
            .....
            int repeats = 0;
            int changes = 0;

            do {
                repeats++;
                if (repeats > 6) {
                    .....
                    break;
                }

                // FIRST LOOP: Perform a layout, if needed.
                if (repeats < 4) {
                    changes = performLayoutLockedInner();
                    if (changes != 0) {
                        continue;
                    }
                } else {
                    Slog.w(TAG, "Layout repeat skipped after too many iterations");
                    changes = 0;
                }
            }
        }
```

```

        // SECOND LOOP: Execute animations and update visibility of windows.
        .....

    } while (changes != 0);

    // THIRD LOOP: Update the surfaces of all windows.

    .....
} catch (RuntimeException e) {
    .....
}

.....

Surface.closeTransaction();

.....

// Destroy the surface of any windows that are no longer visible.
.....

// Time to remove any exiting tokens?
.....

// Time to remove any exiting applications?
.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner是一个巨无霸的函数，它一共有1200+行代码，承载了WindowManagerService服务的核心功能。对于这样一个巨无霸函数，要逐行地分析它的实现是很困难的，因为要理解各种上下文信息，才可以清楚地知道它的执行过程。这里我们就大概地分析它的实现框架，以后再逐步地分析它的具体实现：

1. 在一个最多执行7次的while循环中，做两件事情：第一件事情是计算各个窗品的大小，这是通过调用另外一个成员函数performLayoutLockedInner来实现的；第二件事情是执行窗口的动画，主要是处理窗口的启动窗口显示动画和窗口切换过程中的动画，以及更新各个窗口的可见性。注意，每一次while循环执行之后，如果发现系统中的各个窗口的相应布局属性不再发生变化，那么就不行执行下一次的while循环了，即该while循环可能不用执行7次就结束了。窗口的动画显示过程和窗口的可见性更新过程是相当复杂的，它们也是WindowManagerService服务最为核的地方，在后面的文章中，我们再详细分析。
2. 经过第1点的操作之后，接下来就可以将各个窗口的属性，例如，大小、位置等属性，通知SurfaceFlinger服务了，也就是让SurfaceFlinger服务更新它里面的各个Layer的属性值，以便可以对这些Layer执行可见性计算、合成等操作，最后渲染到硬件帧缓冲区中去。SurfaceFlinger服务计算系统中各个窗口，即各个Layer的可见性，以便将它们合成、渲染到硬件帧缓冲区的过程可以参考前面Android系统Surface机制的SurfaceFlinger服务渲染应用程序UI的过程分析一文。注意，各个窗口的属性更新操作是被包含在SurfaceFlinger服务的一个事务中的，即一个Transaction中，这样做是为了避免每更新一个窗口的一个属性就触发SurfaceFlinger服务重新计算各个Layer的可见性，以及对各个Layer进行合并和渲染的操作。启动SurfaceFlinger服务的一个事务可以通过调用Surface类的静态成员函数openTransaction来实现，而关闭SurfaceFlinger服务的一个事务可以通过调用Surface类的静态成员函数closeTransaction来实现。
3. 经过第1点和第2点的操作之后，一次系统UI的刷新过程就完成了，这时候就会将系统中的那些不会再显示的窗口的绘图表面销毁掉，并且将那些已经完成退出了的窗口令牌，即将我们在前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文中所提到的WindowToken移除掉，以及将那些已经退出了的Activity窗口令牌，即将我们在前面Android应用程序窗口（Activity）与WindowManagerService

服务的连接过程分析一文中所提到的AppWindowToken也移除掉。这一步实际执行的是窗口清理操作。

上述三个操作是WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner的实现关键所在，理解了这三个操作，基本也就可以理解WindowManagerService服务刷新系统UI的过程了。

接下来，我们继续分析WindowManagerService类的成员函数performLayoutLockedInner的实现，以便可以继续了解Activity窗口的大小计算过程。

```
Step 7. WindowManagerService.performLayoutLockedInner
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    final WindowManagerPolicy mPolicy = PolicyManager.makeNewWindowManager();
    .....

    /**
     * Z-ordered (bottom-most first) list of all Window objects.
     */
    final ArrayList<WindowState> mWindows = new ArrayList<WindowState>();
    .....

    private final int performLayoutLockedInner() {
        .....

        final int dw = mDisplay.getWidth();
        final int dh = mDisplay.getHeight();

        final int N = mWindows.size();
        int i;

        .....

        mPolicy.beginLayoutLw(dw, dh);

        int seq = mLayoutSeq+1;
        if (seq < 0) seq = 0;
        mLayoutSeq = seq;

        // First perform layout of any root windows (not attached
        // to another window).
        int topAttached = -1;
        for (i = N-1; i >= 0; i--) {
            WindowState win = mWindows.get(i);
            .....

            final AppWindowToken atoken = win.mAppToken;
            final boolean gone = win.mViewVisibility == View.GONE
                || !win.mRelayoutCalled
                || win.mRootToken.hidden
                || (atoken != null && atoken.hiddenRequested)
                || win.mAttachedHidden
                || win.mExiting || win.mDestroying;
            .....

            if (!gone || !win.mHaveFrame) {
                if (!win.mLayoutAttached) {
                    mPolicy.layoutWindowLw(win, win.mAttrs, null);
                    win.mLayoutSeq = seq;
                    .....
                } else {
                    if (topAttached < 0) topAttached = i;
                }
            }
        }
        .....

        for (i = topAttached; i >= 0; i--) {
            WindowState win = mWindows.get(i);
```

```

        // If this view is GONE, then skip it -- keep the current
        // frame, and let the caller know so they can ignore it
        // if they want. (We do the normal layout for INVISIBLE
        // windows, since that means "perform layout as normal,
        // just don't display").
        if (win.mLayoutAttached) {
            .....

            if ((win.mViewVisibility != View.GONE && win.mRelayoutCalled)
                || !win.mHaveFrame) {
                mPolicy.layoutWindowLw(win, win.mAttrs, win.mAttachedWindow);
                win.mLayoutSeq = seq;
                .....
            }
        }
    }
}

.....

return mPolicy.finishLayoutLw();
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

在分析WindowManagerService类的成员函数performLayoutLockedInner的实现之前，我们首先介绍WindowManagerService类的两个成员变量mPolicy和mWindows：

1. mPolicy指向的是一个窗口管理策略类，它是通过调用PolicyManager类的静态成员函数makeNewWindowManager来初始化的，在Phone平台中，它指向的是便是一个PhoneWindowManager对象，主要是用来制定窗口的大小计算策略。
2. mWindows指向的是一个类型为WindowState的ArrayList，它里面保存的就是系统中的所有窗口，这些窗口是按照Z轴位置从小到大的顺序保存在这个ArrayList中的，也就是说，第i个窗口位于第i-1个窗口的上面，其中， $i > 0$ 。

理解了这两个成员变量的含义之后，我们就分析WindowManagerService类的成员函数performLayoutLockedInner的执行过程，主要是分三个阶段：

1. 准备阶段：调用PhoneWindowManager类的成员函数beginLayoutLw来设置屏幕的大小。屏幕的大小可以通过调用WindowManagerService类的成员变量mDisplay所描述的一个Display对象的成员函数getWidth和getHeight来获得。
2. 计算阶段：调用PhoneWindowManager类的成员函数layoutWindowLw来计算各个窗口的大小、内容区域边衬大小以及可见区域边衬大小。
3. 结束阶段：调用PhoneWindowManager类的成员函数finishLayoutLw来执行一些清理工作。

按照父子关系来划分，系统中的窗口可以分为父窗口和子窗口两种。如果一个WindowState对象的成员变量mLayoutAttached的值等于false，那么它所描述的窗口就可以作为一个父窗口，否则的话，它所描述的窗口就是一个子窗口。由于子窗口的大小计算是依赖于其父窗口的，因此，在计算各个窗口的大小的过程中，即在上述的第2阶段中，按照以下方式进行：

1. 先计算父窗口的大小。一般来说，能够作为父窗口的，是那些Activity窗口。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，如果一个窗口是Activity窗口，那么用来描述它的一个WindowState对象的成员变量mAppToken就不等于null，并且指向的是一个AppWindowToken对象。这个AppWindowToken对象主要是用来描述一个Activity，即与ActivityManagerService服务中的一个ActivityRecord对象对应。一个Activity窗口只有在两种情况下才会被计算大小：第一种情况是窗口不是处于不可见状态的；第二种情况是窗口从来还没有被计算过大小，即用来描述该Activity窗口的WindowState对象的成员变量mHaveFrame的值等于false，这种情况一般发生在窗口刚刚被添加到WindowManagerService的过程中。一个Activity窗口的不可见状态由它本身的状态、它所在的窗口结构树状态以及它所属的Activity的状态有关，也就是说，如果一个Activity窗口本身是可见的，但是由于它的父窗口、它所在的窗口组的根窗口或者它所属的Activity是不可见的，那么该Activity窗口也是不可见的。一个Activity窗口的不可见状态由以下因素决定：

- 1). 它本身处于不可见状态, 即对应的WindowState对象的成员变量mViewVisibility的值等于View.GONE;
- 2). 它本身处于正在退出的状态, 即对应的WindowState对象的成员变量mExiting的值等于true;
- 3). 它本身处于正在销毁的状态, 即对应的WindowState对象的成员变量mDestroying的值等于true;
- 4). 它的父窗口处于不可见状态, 即对应的WindowState对象的成员变量mAttachedHidden的值等于true;
- 5). 它所在窗口结构树中的根窗口处于不可见状态, 即对应的WindowState对象的成员变量mRootToken所描述的一个WindowToken对象的成员变量hidden的值等于true;
- 6). 它所属的Activity处于不可见状态, 即对应的WindowState对象的成员变量mAppToken所描述的一个AppWindowToken对象的成员变量hiddenRequested的值等于true。

除了上述六个因素之外, 如果一个Activity窗口没有被它所运行在的应用程序进程主动请求WindowManagerService服务对它进行布局, 即对应的WindowState对象的成员变量mRelayoutCalled的值等于false, 那么此时也是不需要计算Activity窗口的大小的。

一个Activity窗口的大小一旦确定是需要计算大小之后, PhoneWindowManager类的成员函数layoutWindowLw就被调用来计算它的大小。

1. 接着计算子窗口的大小。前面在计算父窗口的大小过程中, 会记录位于系统最上面的一个子窗口在mWindows所描述的一个ArrayList的位置topAttached, 接下来就可以从这个位置开始向下计算每一个子窗口的大小。一个子窗口在以下两种情况下, 才会被计算大小:

- 1). 它本身处于可见状态, 即对应的WindowState对象的成员变量mViewVisibility的值不等于View.GONE, 并且它所运行在的应用程序进程主动请求WindowManagerService服务对它进行布局, 即对应的WindowState对象的成员变量mRelayoutCalled的值等于true。
- 2). 它从来还没有被计算过大小, 即用来描述该子窗口的WindowState对象的成员变量mHaveFrame的值等于false, 这种情况一般发生在子窗口刚刚被添加到WindowManagerService的过程中。

接下来, 我们就分别分析PhoneWindowManager类的成员函数beginLayoutLw、layoutWindowLw和finishLayoutLw的实现, 以便可以了解Activity窗口的大小计算过程。

```
Step 8. PhoneWindowManager.beginLayoutLw
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    WindowState mStatusBar = null;
    .....

    // The current size of the screen.
    int mW, mH;
    // During layout, the current screen borders with all outer decoration
    // (status bar, input method dock) accounted for.
    int mCurLeft, mCurTop, mCurRight, mCurBottom;
    // During layout, the frame in which content should be displayed
    // to the user, accounting for all screen decoration except for any
    // space they deem as available for other content. This is usually
    // the same as mCur*, but may be larger if the screen decor has supplied
    // content insets.
    int mContentLeft, mContentTop, mContentRight, mContentBottom;
    // During layout, the current screen borders along with input method
    // windows are placed.
    int mDockLeft, mDockTop, mDockRight, mDockBottom;
    // During layout, the layer at which the doc window is placed.
    int mDockLayer;

    static final Rect mTmpParentFrame = new Rect();
    static final Rect mTmpDisplayFrame = new Rect();
    static final Rect mTmpContentFrame = new Rect();
    static final Rect mTmpVisibleFrame = new Rect();
    .....

    public void beginLayoutLw(int displayWidth, int displayHeight) {
        mW = displayWidth;
        mH = displayHeight;
        mDockLeft = mContentLeft = mCurLeft = 0;
        mDockTop = mContentTop = mCurTop = 0;
        mDockRight = mContentRight = mCurRight = displayWidth;
        mDockBottom = mContentBottom = mCurBottom = displayHeight;
        mDockLayer = 0x10000000;
```

```

// decide where the status bar goes ahead of time
if (mStatusBar != null) {
    final Rect pf = mTmpParentFrame;
    final Rect df = mTmpDisplayFrame;
    final Rect vf = mTmpVisibleFrame;
    pf.left = df.left = vf.left = 0;
    pf.top = df.top = vf.top = 0;
    pf.right = df.right = vf.right = displayWidth;
    pf.bottom = df.bottom = vf.bottom = displayHeight;

    mStatusBar.computeFrameLw(pf, df, vf, vf);
    if (mStatusBar.isVisibleLw()) {
        // If the status bar is hidden, we don't want to cause
        // windows behind it to scroll.
        mDockTop = mContentTop = mCurTop = mStatusBar.getFrameLw().bottom;
        .....
    }
}
}
.....
}

```

这个函数定义在文件frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java中。

在分析PhoneWindowManager类的成员函数beginLayoutLw的实现之前，我们首先介绍PhoneWindowManager类的五组成员变量。

第一组成员变量是mW和mH，它们分别用来描述当前这轮窗口大小计算过程的屏幕宽度和高度。

第二组成员变量是mCurLeft、mCurTop、mCurRight和mCurBottom，它们组成一个四元组（mCurLeft, mCurTop, mCurRight, mCurBottom），用来描述当前这轮窗口大小计算过程的屏幕装饰区，它对应于前面所提到的Activity窗口的可见区域边衬。

第三组成员变量是mContentLeft、mContentTop、mContentRight和mContentBottom，它们组成一个四元组（mContentLeft, mContentTop, mContentRight, mContentBottom），也是用来描述当前这轮窗口大小计算过程的屏幕装饰区，不过它对应的是前面所提到的Activity窗口的内容区域边衬。

第四组成员变量是mDockLeft、mDockTop、mDockRight、mDockBottom和mDockLayer，其中，前四个成员变量组成一个四元组（mDockLeft, mDockTop, mDockRight, mDockBottom），用来描述当前这轮窗口大小计算过程中的输入法窗口所占据的位置，后一个成员变量mDockLayer用来描述输入法窗口的Z轴位置。

第五组成员变量是mTmpParentFrame、mTmpDisplayFrame、mTmpContentFrame和mTmpVisibleFrame，它们是一组临时Rect区域，用来作为参数传递给具体的窗口计算大小的，避免每次都创建一组新的Rect区域来作为参数传递窗口。

除了这五组成员变量之外，PhoneWindowManager类还有一个成员变量mStatusBar，它的类型为WindowState，用来描述系统的状态栏。

理解了这些成员变量的含义之后，PhoneWindowManager类的成员函数beginLayoutLw的实现就容易理解了，它主要做了以下两件事情：

1. 初始化前面所提到的四组成员变量，其中，mW和mH设置为参数displayWidth和displayHeight所指定的屏幕宽度和高度，并且使得（mCurLeft, mCurTop, mCurRight, mCurBottom）、（mContentLeft, mContentTop, mContentRight, mContentBottom）和（mDockLeft, mDockTop, mDockRight, mDockBottom）这三个区域的大小等于屏幕的大小。
2. 计算状态栏的大小。状态栏的大小一经确定，并且它是可见的，那么就会修改成员变量mCurLeft、mContentLeft和mDockLeft的值为状态栏的所占据的区域的下边界位置，这样就可以将（mCurLeft, mCurTop, mCurRight, mCurBottom）、（mContentLeft, mContentTop, mContentRight, mContentBottom）和（mDockLeft, mDockTop, mDockRight, mDockBottom）这三个区域限制为剔除状态栏区域之后所得到的屏幕区域。

还有另外一个地方需要注意的是，输入法窗口的Z轴被初始化为0x10000000，这个值是相当大的了，可以保证输入法窗口作为顶层窗口出现。

这一步执行完成之后，返回到前面的Step 7中，即WindowManagerService类的成员函数performLayoutLockedInner，接下来就会调用PhoneWindowManager类的成员函数layoutWindowLw来计算系统中各个可见窗口的大小。

Step 9. PhoneWindowManager.layoutWindowLw

```

public class PhoneWindowManager implements WindowManagerPolicy {
    .....
}

```



```

public void layoutWindowLw(WindowState win, WindowManager.LayoutParams attrs,
    WindowState attached) {
    // we've already done the status bar
    if (win == mStatusBar) {
        return;
    }
    .....

    final int fl = attrs.flags;
    final int sim = attrs.softInputMode;

    final Rect pf = mTmpParentFrame;
    final Rect df = mTmpDisplayFrame;
    final Rect cf = mTmpContentFrame;
    final Rect vf = mTmpVisibleFrame;

    if (attrs.type == TYPE_INPUT_METHOD) {
        pf.left = df.left = cf.left = vf.left = mDockLeft;
        pf.top = df.top = cf.top = vf.top = mDockTop;
        pf.right = df.right = cf.right = vf.right = mDockRight;
        pf.bottom = df.bottom = cf.bottom = vf.bottom = mDockBottom;
        // IM dock windows always go to the bottom of the screen.
        attrs.gravity = Gravity.BOTTOM;
        mDockLayer = win.getSurfaceLayer();
    } else {
        if ((fl &
            (FLAG_LAYOUT_IN_SCREEN | FLAG_FULLSCREEN | FLAG_LAYOUT_INSET_DECOR))
            == (FLAG_LAYOUT_IN_SCREEN | FLAG_LAYOUT_INSET_DECOR)) {
            // This is the case for a normal activity window: we want it
            // to cover all of the screen space, and it can take care of
            // moving its contents to account for screen decorations that
            // intrude into that space.
            if (attached != null) {
                // If this window is attached to another, our display
                // frame is the same as the one we are attached to.
                setAttachedWindowFrames(win, fl, sim, attached, true, pf, df, cf, vf);
            } else {
                pf.left = df.left = 0;
                pf.top = df.top = 0;
                pf.right = df.right = mW;
                pf.bottom = df.bottom = mH;
                if ((sim & SOFT_INPUT_MASK_ADJUST) != SOFT_INPUT_ADJUST_RESIZE) {
                    cf.left = mDockLeft;
                    cf.top = mDockTop;
                    cf.right = mDockRight;
                    cf.bottom = mDockBottom;
                } else {
                    cf.left = mContentLeft;
                    cf.top = mContentTop;
                    cf.right = mContentRight;
                    cf.bottom = mContentBottom;
                }
                vf.left = mCurLeft;
                vf.top = mCurTop;
                vf.right = mCurRight;
                vf.bottom = mCurBottom;
            }
        }
        .....
    }

    win.computeFrameLw(pf, df, cf, vf);

    // Dock windows carve out the bottom of the screen, so normal windows
    // can't appear underneath them.
    if (attrs.type == TYPE_INPUT_METHOD && !win.getGivenInsetsPendingLw()) {
        int top = win.getContentFrameLw().top;
        top += win.getGivenContentInsetsLw().top;
    }
}

```

```

        if (mContentBottom > top) {
            mContentBottom = top;
        }
        top = win.getVisibleFrameLw().top;
        top += win.getGivenVisibleInsetsLw().top;
        if (mCurBottom > top) {
            mCurBottom = top;
        }
        .....
    }
}

.....
}

```

这个函数定义在文件frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java中。第一个参数win描述的是当前要计算大小的窗口，第二个参数attrs描述的是窗口win的布局参数，第三个参数attached描述的是窗口win的父窗口，如果它的值等于null，就表示窗口win没有父窗口。

PhoneWindowManager类的成员函数layoutWindowLw会根据窗口win的是子窗口还是全屏窗口及其软键盘显示模式来决定它的大小如何计算。这里我们只关注输入法窗口和非全屏的Activity窗口的大小计算方式，其它类型的窗口大小计算方式是差不多的。

从前面的Step 8可以知道，系统的状态栏大小已经计算过了，因此，PhoneWindowManager类的成员函数layoutWindowLw如果发现参数win描述的正好是状态栏的话，它就什么也不做就返回了。

在计算一个窗口的大小的时候，我们需要四个参数。第一个参数是父窗口的大小pf，第二个参数是屏幕的大小df，第三个参数是内容区域边衬大小cf，第四个参数是可见区域边衬大小vf。

如果参数win描述的是输入法窗口，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUTMETHOD，那么上述四个用来计算窗口大小的区域pf、df、cf和vf就等于PhoneWindowManager类的成员变量mDockLeft、mDockTop、mDockRight和mDockBottom所组成的区域的大小。

如果参数win描述的是一个非全屏的Activity窗口，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_LAYOUT_IN_SCREEN位和FLAG_LAYOUT_INSET_DECOR位等于1，那么PhoneWindowManager类的成员函数layoutWindowLw就会继续检查参数attached的值是否不等于null。如果不等于null的话，那么就说明参数win所描述的一个非全屏的Activity窗口附加在其它窗口上，即它具有一个父窗口，这时候就会调用另外一个成员函数setAttachedWindowFrames来计算它的大小。

接下来我们就只关注参数win描述的是一个非全屏的、并且没有附加到其它窗口的Activity窗口的大小计算过程。

首先，父窗口大小pf和屏幕大小df都会被设置为整个屏幕区域的大小。

其次，可见区域边衬大小vf被设置为PhoneWindowManager类的成员变量mCurLeft、mCurTop、mCurRight和mCurBottom所组成的区域的大小。

第三，内容区域边衬大小cf的计算相对复杂一些，需要考虑窗口win的软键盘显示模式sim的值。如果变量sim的SOFT_INPUT_ADJUST_RESIZE位等于1，那么就意味着窗口win在出向输入法窗口的时候，它的内容要重新进行排布，避免被输入法窗口挡住，因此，这时候窗口win的内容区域大小就会等于PhoneWindowManager类的成员变量mContentLeft、mContentTop、mContentRight和mContentBottom所组成的区域的大小。另一方面，如果变量sim的SOFT_INPUT_ADJUST_RESIZE位等于0，那么就意味着窗口win在出向输入法窗口的时候，它的内容不需要重新进行排布，这时候它的内容区域大小就会等于PhoneWindowManager类的成员变量mDockLeft、mDockTop、mDockRight和mDockBottom所组成的区域的大小。注意，PhoneWindowManager类的成员变量mDockLeft、mDockTop、mDockRight和mDockBottom所组成的区域的大小并不是等于输入法窗口的大小的，而是包含了输入法窗口所占据的区域的大小，这就意味着输入法窗口与窗口win会有重叠的部分，或者说输入法窗口覆盖了窗口win的一部分。

得到了用来计算窗口win四个参数pf、df、cf和vf之后，就可以调用参数win所描述的一个WindowState对象的成员函数computeFrameLw来计算窗口win的具体大小了。计算的结果便得到了窗口win的大小，以及它的内容区域边衬大小和可见区域边衬大小。注意，窗口经过计算后得到的内容区域边衬大小和可见区域边衬大小并不一定是等于参数cf和vf所指定的大小的。

计算完成窗口win的大小之后，PhoneWindowManager类的成员函数layoutWindowLw还会检查窗口win是否是一个输入法窗口，并且它是否指定了额外的内容区域边衬和可见区域边衬。如果这两个条件都成立的话，那么就需要相应地调整PhoneWindowManager类的成员变量mContentBottom和mCurBottom的值，以便使得PhoneWindowManager类的成员变量是mContentLeft、mContentTop、mContentRight和mContentBottom所围成的内容区域和成员变量mCurLeft、mCurTop、mCurRight和mCurBottom所围成的可见区域不会覆盖到输入法窗口额外指定的内容区域边衬和可见区域边衬。

接下来，我们就继续分析WindowState类的成员函数computeFrameLw的实现，以便可以了解Activity窗口的大小计算的具体过程。

```

Step 10. WindowState.computeFrameLw
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {

```

```

.....

boolean mHaveFrame;
.....

// "Real" frame that the application sees.
final Rect mFrame = new Rect();
.....

final Rect mContainingFrame = new Rect();
final Rect mDisplayFrame = new Rect();
final Rect mContentFrame = new Rect();
final Rect mVisibleFrame = new Rect();

public void computeFrameLw(Rect pf, Rect df, Rect cf, Rect vf) {
    mHaveFrame = true;

    final Rect container = mContainingFrame;
    container.set(pf);

    final Rect display = mDisplayFrame;
    display.set(df);

    if ((mAttrs.flags & FLAG_COMPATIBLE_WINDOW) != 0) {
        container.intersect(mCompatibleScreenFrame);
        if ((mAttrs.flags & FLAG_LAYOUT_NO_LIMITS) == 0) {
            display.intersect(mCompatibleScreenFrame);
        }
    }

    final int pw = container.right - container.left;
    final int ph = container.bottom - container.top;

    int w, h;
    if ((mAttrs.flags & mAttrs.FLAG_SCALED) != 0) {
        w = mAttrs.width < 0 ? pw : mAttrs.width;
        h = mAttrs.height < 0 ? ph : mAttrs.height;
    } else {
        w = mAttrs.width == mAttrs.MATCH_PARENT ? pw : mRequestedWidth;
        h = mAttrs.height == mAttrs.MATCH_PARENT ? ph : mRequestedHeight;
    }

    final Rect content = mContentFrame;
    content.set(cf);

    final Rect visible = mVisibleFrame;
    visible.set(vf);

    final Rect frame = mFrame;
    final int fw = frame.width();
    final int fh = frame.height();

    .....

    Gravity.apply(mAttrs.gravity, w, h, container,
        (int) (mAttrs.x + mAttrs.horizontalMargin * pw),
        (int) (mAttrs.y + mAttrs.verticalMargin * ph), frame);

    .....

    // Now make sure the window fits in the overall display.
    Gravity.applyDisplay(mAttrs.gravity, df, frame);

    // Make sure the content and visible frames are inside of the
    // final window frame.
    if (content.left < frame.left) content.left = frame.left;
    if (content.top < frame.top) content.top = frame.top;
    if (content.right > frame.right) content.right = frame.right;

```

```

        if (content.bottom > frame.bottom) content.bottom = frame.bottom;
        if (visible.left < frame.left) visible.left = frame.left;
        if (visible.top < frame.top) visible.top = frame.top;
        if (visible.right > frame.right) visible.right = frame.right;
        if (visible.bottom > frame.bottom) visible.bottom = frame.bottom;

        final Rect contentInsets = mContentInsets;
        contentInsets.left = content.left-frame.left;
        contentInsets.top = content.top-frame.top;
        contentInsets.right = frame.right-content.right;
        contentInsets.bottom = frame.bottom-content.bottom;

        final Rect visibleInsets = mVisibleInsets;
        visibleInsets.left = visible.left-frame.left;
        visibleInsets.top = visible.top-frame.top;
        visibleInsets.right = frame.right-visible.right;
        visibleInsets.bottom = frame.bottom-visible.bottom;

        if (mIsWallpaper && (fw != frame.width() || fh != frame.height())) {
            updateWallpaperOffsetLocked(this, mDisplay.getWidth(),
                mDisplay.getHeight(), false);
        }

        .....
    }

    .....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowState类的成员变量mHaveFrame用来描述一个窗口的大小是否计算过了。当WindowState类的成员函数computeFrameLw被调用的时候，就说明一个相应的窗口的大小得到计算了，因此，WindowState类的成员函数computeFrameLw一开始就会将成员变量mHaveFrame的值设置为true。

回忆一下，在前面的Step 9中提到，参数pf描述的是父窗口的大小，参数df描述的是屏幕的大小，参数cf描述的内容区域大小，参数vf描述的是可见区域大小，接下来我们就分析WindowState类的成员函数computeFrameLw是如何利用这些参数来计算一个窗口的大小的。

WindowState类的成员变量mContainingFrame和mDisplayFrame描述的是当前正在处理的窗口的父窗口和屏幕的大小，它们刚好就分别等于参数pf和df的大小，因此，函数就直接将参数pf和df的值分别保存在WindowState类的成员变量mContainingFrame和mDisplayFrame中。如果当前正在处理的窗口运行在兼容模式，即WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_COMPATIBLEWINDOW位等于1，那么就需要将其父窗口的大小限制mContainingFrame在兼容模式下的屏幕区域中。兼容模式下的屏幕区域保存在WindowManagerService类的成员变量mCompatibleScreenFrame中，将父窗口的大小mContainingFrame与它执行一个相交操作，就可以将父窗口的大小限制兼容模式下的屏幕区域中。在当前正在处理的窗口运行在兼容模式的情况下，如果它的大小被限制在了兼容模式下的屏幕区域之中，即WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_LAYOUT_NO_LIMITS位等于0，那么同样需要将屏幕大小mDisplayFrame限制在兼容模式下的屏幕区域mCompatibleScreenFrame，这也是通过执行一个相交操作来完成的。

WindowState类的成员变量mContentFrame和mVisibleFrame描述的是当前正在处理的窗口的内容区域和可见区域大小，它们刚好就分别等于参数cf和vf的大小，因此，函数就直接将参数cf和vf的值分别保存在WindowState类的成员变量mContainingFrame和mDisplayFrame中。现在，就剩下窗口的大小还没有计算。一旦窗口大小确定下来之后，就可以继续计算窗口的内容区域边衬和可见区域边衬大小了。接下来我们就继续分析窗口大小的计算过程。

WindowState类的成员变量mFrame描述的就是当前正在处理的窗口的大小，我们的目标就是计算它的值。一个窗口的大小是受以下因素影响的：

1. 是否指定了缩放因子。如果一个窗口的大小被指定了缩放因子，即WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_SCALED位等于1，那么该窗口的大小就是在它的布局参数中指定的，即是由WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量width和height所指定的。但是，如果在布局参数中指定的窗口宽度或者高度小于0，那么就会使用其父窗口的大小来作为当前窗口的大小。当前窗口的父窗口的宽度和高度分别保存在变量pw和ph中。
2. 是否指定了等于父窗口的大小。如果一个窗口的大小被指定为其父窗口的大小，即WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成

员变量width和height的值等于mAttrs.MATCH_PARENT，那么该窗口的大小就会等于其父窗口的大小，即等于变量pw和ph所描述的宽度和高度。另一方面，如果一个窗口的大小没有指定为其父窗口的大小的话，那么它的大小就会等于应用程序进程请求WindowManagerService所设置的大小，即等于WindowState类的成员变量mRequestedWidth和mRequestedHeight所描述的宽度和高度。

经过上述2个操作之后，我们就初步地得到了窗口的宽度w和高度h，但是，它们还不是最终的窗口大小，还要进一步地根据窗口的Gravity属性来作调整，这个调整分两步进行：

1. 根据窗口的Gravity值，以及位置、初始大小和父窗口大小，来计算窗口的大小，并且保存在变量frame中，即保存在WindowState类的成员变量mFrame中，这是通过调用Gravity类的静态成员函数apply来实现的。其中，窗口的初始大小保存在变量w和h中，父窗口大小保存在变量container中，即WindowState类的成员变量mContainingFrame中，位置保存在WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量x和y中。注意，如果窗口指定了相对父窗口的margin值，那么还需要相应的调整其位置值，即要在指定的位置值的基础上，再加上相对父窗口的margin值。一个窗口相对父窗口的margin是通过一个百分比来表示的，用这个百分比乘以父窗口的大小就可以得到绝对值。这个百分比又分为在水平方向和垂直方向两个值，分别保存在WindowState类的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量horizontalMargin和verticalMargin中。
2. 前面计算得到的窗口大小没有考虑在屏幕的大小，因此，接下来还需要继续调用Gravity类的静态成员函数applyDisplay来将前面计算得到的窗口大小限制在屏幕区域df中，即限制在WindowState类的成员变量mDisplayFrame所描述的区域中。

经过上述2个操作之后，窗口的最终大小就保存在变量frame中了，即WindowState类的成员变量mFrame中，接下来就可以计算窗口的内容区域边衬和可见区域边衬大小了。

内容区域边衬和可见区域边衬大小的计算很简单的，只要将窗口的大小frame，即WindowState类的成员变量mFrame所描述的区域，分别减去变量content和visible，即WindowState类的成员变量mContentFrame和mVisibleFrame所描述的区域，就可以得到窗口的内容区域边衬和可见区域边衬大小，它们分别保存在WindowState类的成员变量mContentInsets和mVisibleInsets中。注意，在计算窗口的内容区域边衬和可见区域边衬大小之前，首先要保证窗口的内容区域和可见区域包含在整个窗口区域中，这一点是由中间的8个if语句来保证的。

窗口上一次的大小保存在变量fw和fh中。如果当前正在处理的窗口是一个壁纸窗口，即WindowState类的成员变量mIsWallpaper的值等于true，并且该窗口的大小发生了变化，即变量fw和fh的所描述的窗口大小不等于变量frame描述的窗口大小，那么就需要调用WindowManagerService类的成员函数updateWallpaperOffsetLocked来更新壁纸的位置。在后面的文章中，我们再详细描述系统的壁纸窗口的位置是如何计算的。

这一步执行完成之后，一个窗口的大小就计算完成了。从计算的过程可以知道，整个窗口大小保存在WindowState类的成员变量mFrame中，而窗口的内容区域边衬大小和可见区域边衬大小分别保存在WindowState类的成员变量mContentInsets和mVisibleInsets中。这些值最终会通过前面的Step 4返回给应用程序进程。

返回到前面的Step 7中，即WindowManagerService类的成员函数performLayoutLockedInner，接下来就会调用PhoneWindowManager类的成员函数finishLayoutLw来结束当前这轮窗口大小的计算工作。

```
Step 11. PhoneWindowManager.finishLayoutLw
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    public int finishLayoutLw() {
        return 0;
    }

    .....
}
```

这个函数定义在文件frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java中。PhoneWindowManager类的成员函数finishLayoutLw是设计来结束一轮窗口大小的计算过程中，不过目前它什么也不做，只是一个空实现。

至此，我们就分析完成Activity窗口的大小计算过程了。从这个计算过程中，我们就可以知道一个Activity窗口除了有一个整体大小之外，还有一个内容区域边衬大小和一个可见区域边衬大小。此外，我们还知道，一个Activity窗口的内容区域边衬大小和可见区域边衬大小是可能会受到与其所关联的输入法窗口的影响的，因为输入法窗口会叠加在该Activity窗口上面，这就涉及到了系统中的窗口的组织方式。在接下来的一篇文章中，我们还将继续分析WindowManagerService服务是如何组织系统中的窗口的。敬请关注！

原文链接：<https://blog.csdn.net/Luoshengyang/article/details/8479101>

第三节Android窗口管理服务WindowManagerService对窗口的组织方式分析

我们知道，在Android系统中，Activity是以堆栈的形式组织在ActivityManagerService服务中的。与Activity类似，Android系统中的窗口也是以堆栈的形式组织在WindowManagerService服务中的，其中，Z轴位置较低的窗口位于Z轴位置较高的窗口的下面。在本文中，我们就详细分析WindowManagerService服务是如何以堆栈的形式来组织窗口的。

《Android系统源代码情景分析》一书正在进击的程序员网 (<http://0xcc0xcd.com>) 中连载，点击进入！

从前面Android应用程序启动过程源代码分析一文可以知道，应用程序进程中的每一个Activity组件在Activity管理服务ActivityManagerService中都对应有一个ActivityRecord对象。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文又可以知道，Activity管理服务ActivityManagerService中每一个ActivityRecord对象在Window管理服务WindowManagerService中都对应有一个AppWindowToken对象。

此外，在输入法管理服务InputMethodManagerService中，每一个输入法窗口都对应有一个Binder对象，这个Binder对象在Window管理服务WindowManagerService又对应有一个WindowToken对象。

与输入法窗口类似，在壁纸管理服务WallpaperManagerService中，每一个壁纸窗口都对应有一个Binder对象，这个Binder对象在Window管理服务WindowManagerService也对应有一个WindowToken对象。

在Window管理服务WindowManagerService中，无论是AppWindowToken对象，还是WindowToken对象，它们都是用来描述一组有着相同令牌的窗口的，每一个窗口都是通过一个WindowState对象来描述的。例如，一个Activity组件窗口可能有一个启动窗口（Starting Window），还有若干个子窗口，那么这些窗口就会组成一组，并且都是以Activity组件在Window管理服务WindowManagerService中所对应的AppWindowToken对象为令牌的。从抽象的角度来看，就是在Window管理服务WindowManagerService中，每一个令牌（AppWindowToken或者WindowToken）都是用来描述一组窗口（WindowState）的，并且每一个窗口的子窗口也是与它同属于一个组，即都有着相同的令牌。

上述的窗口组织方式如图1所示：

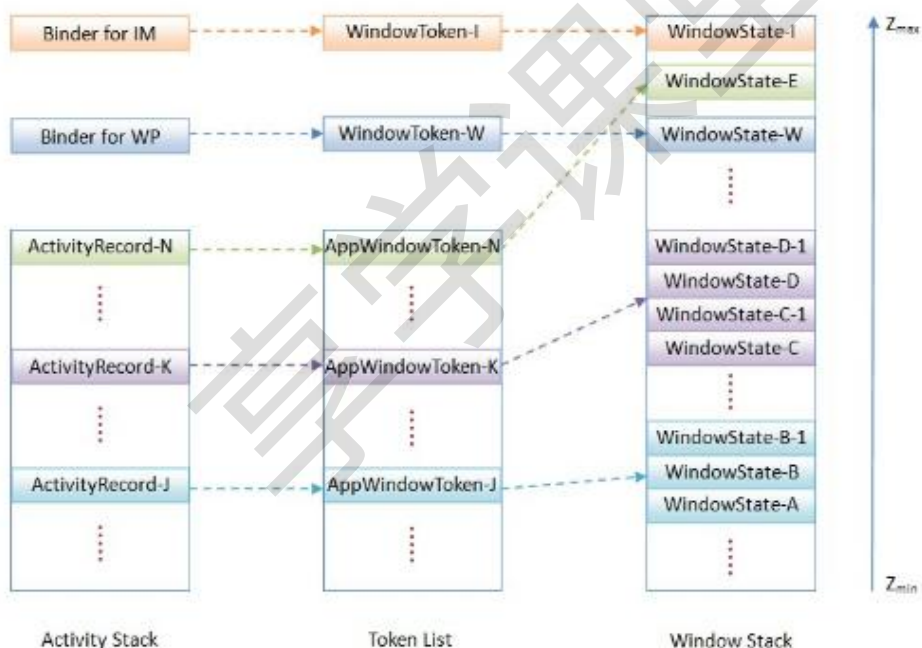


图1 窗口在WindowManagerService服务中的组织方式

其中，Activity Stack是在ActivityManagerService服务中创建的，Token List和Window Stack是在WindowManagerService中创建的，而Binder for IM和Binder for WP分别是在InputMethodManagerService服务和WallpaperManagerService服务中创建的，用来描述一个输入法窗口和一个壁纸窗口。

图1中的对象的对应关系如下所示：

1. ActivityRecord-J对应于AppWindowToken-J，后者描述的一组窗口是{WindowState-A, WindowState-B, WindowState-B-1}，其中， WindowState-B-1是WindowState-B的子窗口。
2. ActivityRecord-K对应于AppWindowToken-K，后者描述的一组窗口是{WindowState-C, WindowState-C-1, WindowState-D, WindowState-D-1}，其中， WindowState-C-1是WindowState-C的子窗口， WindowState-D-1是WindowState-D的子窗口。
3. ActivityRecord-N对应于AppWindowToken-N，后者描述的一组窗口是{WindowState-E}，其中， WindowState-E是系统当前激活的Activity窗口。
4. Binder for IM对应于WindowToken-I，后者描述的一组窗口是{WindowState-I}，其

中， WindowState-I是WindowState-E的输入法窗口。

5. Binder for WP对应于WindowToken-W，后者描述的一组窗口是{WindowState-W}，其中， WindowState-W是WindowState-E的壁纸窗口。

从图1还可以知道， Window Stack中的WindowState是按照它们所描述的窗口的Z轴位置从低到高排列的。

以上就是WindowManagerService服务组织系统中的窗口的抽象模型，接下来我们将分析AppWindowToken、WindowToken和WindowState的一些增加、移动和删除等操作，以便可以对这个抽象模型有一个更深刻的认识。

1. 增加AppWindowToken

从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，一个Activity组件在启动的过程中，ActivityManagerService服务会调用调用WindowManagerService类的成员函数addAppToken来为它增加一个AppWindowToken，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    /**
     * Mapping from a token IBinder to a WindowToken object.
     */
    final HashMap<IBinder, WindowToken> mTokenMap =
        new HashMap<IBinder, WindowToken>();

    /**
     * The same tokens as mTokenMap, stored in a list for efficient iteration
     * over them.
     */
    final ArrayList<WindowToken> mTokenList = new ArrayList<WindowToken>();
    .....

    /**
     * Z-ordered (bottom-most first) list of all application tokens, for
     * controlling the ordering of windows in different applications. This
     * contains WindowToken objects.
     */
    final ArrayList<AppWindowToken> mAppTokens = new ArrayList<AppWindowToken>();
    .....

    public void addAppToken(int addPos, IApplicationToken token,
        int groupId, int requestedOrientation, boolean fullscreen) {
        .....

        synchronized(mWindowMap) {
            AppWindowToken wtoken = findAppWindowToken(token.asBinder());
            if (wtoken != null) {
                .....
                return;
            }
            wtoken = new AppWindowToken(token);
            .....
            mAppTokens.add(addPos, wtoken);
            .....
            mTokenMap.put(token.asBinder(), wtoken);
            mTokenList.add(wtoken);
            .....
        }
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类有三个成员变量mTokenMap、mTokenList和mAppTokens，它们都是用来描述系统中的窗口的。

成员变量mTokenMap指向的是一个HashMap，它里面保存的是一系列的WindowToken对象，每一个WindowToken对象都是用来描述一个窗口的，并且是以描述这些窗口的一个Binder对象的IBinder接口为键值的。例如，对于Activity组件类型的窗口来说，它们分别是以用来描述它们的一个ActivityRecord对象的IBinder接口保存在成员变量mTokenMap所指向的一个HashMap中的。

成员变量mTokenList指向的是一个ArrayList，它里面保存的也是一系列WindowToken对象，这些WindowToken对象与

保存在成员变量mTokenMap所指向的一个HashMap中的WindowToken对象是一样的。成员变量mTokenMap和成员变量mTokenList的区别就在于，前者在给定一个IBinder接口的情况下，可以迅速指出是否存在一个对应的WindowToken对象，而后者可以迅速遍历WindowManagerService服务中的WindowToken对象。

成员变量mAppTokens指向的也是一个ArrayList，不过它里面保存的是一系列AppWindowToken对象，每一个AppWindowToken对象都是用来描述一个Activity组件窗口的，而这些AppWindowToken对象是以它们描述的窗口的Z轴坐标由小到大保存在这个ArrayList中的，这样我们就可以通过这个ArrayList来从上到下或者从下到上地遍历系统中的所有Activity组件窗口。由于这些AppWindowToken对象所描述的Activity组件窗口也是一个窗口，并且AppWindowToken类是从WindowToken继承下来的，因此，这些AppWindowToken对象还会同时被保存在成员变量mTokenMap所指向的一个HashMap和成员变量mTokenList所指向的一个ArrayList中。

理解了WindowManagerService类的这三个成员变量的含义之后，它的成员函数addAppToken的实现就好理解了，其中，参数token指向的便是用来描述正在启动的Activity组件所对应的一个ActivityRecord对象，而参数addPos用来描述该Activity组件在堆栈中的位置，这个位置同时也是接下来要创建的AppWindowToken对象在WindowManagerService类的mTokenList所描述的一个ArrayList中的位置。

WindowManagerService类的成员函数addAppToken首先调用另外一个成员函数findAppWindowToken来在成员变量mTokenMap所描述的一个HashMap检查是否已经存在一个AppWindowToken。如果已经存在的话，那么WindowManagerService类的成员函数addAppToken就什么也不做就返回了，否则的话，就会使用参数token来创建一个AppWindowToken对象，并且会将该AppWindowToken对象分别保存在WindowManagerService类的成员变量mTokenMap、mTokenList和mAppTokens中。

1. 删除AppWindowToken

删除AppWindowToken是通过调用WindowManagerService类的成员函数removeAppTokensLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private void removeAppTokensLocked(List<IBinder> tokens) {
        // XXX This should be done more efficiently!
        // (take advantage of the fact that both lists should be
        // ordered in the same way.)
        int N = tokens.size();
        for (int i=0; i<N; i++) {
            IBinder token = tokens.get(i);
            final AppWindowToken wtoken = findAppWindowToken(token);
            if (!mAppTokens.remove(wtoken)) {
                .....
                i--;
                N--;
            }
        }
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的成员函数removeAppTokensLocked可以同时删除一组AppWindowToken对象。

参数tokens所描述的是一个IBinder接口列表，与这些IBinder接口所对应的AppWindowToken对象就是接下来要删除的。WindowManagerService类的成员函数removeAppTokensLocked通过一个for循环来依次调用另外一个成员函数findAppWindowToken，以便可以找到保存在列表tokens中的每一个IBinder接口所对应的AppWindowToken对象，然后将该AppWindowToken对象从WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList中删除。

注意，WindowManagerService类的成员函数removeAppTokensLocked是在内部使用的，它只是把一个AppWindowToken对象从成员变量mAppTokens中删除，而没有从另外两个成员变量mTokenMap和mTokenList中删除。

3. 移动AppWindowToken至指定位置

移动AppWindowToken至指定位置是通过调用WindowManagerService类的成员函数moveAppToken来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void moveAppToken(int index, IBinder token) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "moveAppToken()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        synchronized(mWindowMap) {
            .....
            final AppWindowToken wtoken = findAppWindowToken(token);
```



```

        if (wtoken == null || !mAppTokens.remove(wtoken)) {
            .....
            return;
        }
        mAppTokens.add(index, wtoken);
        .....

        final long origId = Binder.clearCallingIdentity();
        .....
        if (tmpRemoveAppWindowsLocked(wtoken)) {
            .....
            reAddAppWindowsLocked(findWindowOffsetLocked(index), wtoken);
            .....
            updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES);
            mLayoutNeeded = true;
            performLayoutAndPlaceSurfacesLocked();
        }
        Binder.restoreCallingIdentity(origId);
    }
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。参数token描述的是要移动的AppWindowToken对象所对应的一个IBinder接口，而参数index描述的是该AppWindowToken对象要移动到的位置。注意，移动一个AppWindowToken对象到指定的位置是需要android.Manifest.permission.MANAGE_APP_TOKENS权限的。

WindowManagerService类的成员函数moveAppToken首先找到与参数token所对应的AppWindowToken对象，并且将该AppWindowToken对象从WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList中移除，这样做的目的是为了接下来可以将该AppWindowToken对象移动至该ArrayList中的指定位置上，即参数index所描述的位置上。

注意，上述操作只是将参数token所对应的AppWindowToken对象移动到了WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList的指定位置上，接下来还需要同时将与该AppWindowToken对象所对应的WindowState对象移动至WindowManagerService服务内部的一个WindowState堆栈合适位置上去。

移动对应的WindowState对象的操作同样也是分两步执行的：第一步先调用WindowManagerService类的成员函数tmpRemoveAppWindowsLocked来将这些WindowState对象从原来的WindowState堆栈位置移除；第二步再调用WindowManagerService类的成员函数reAddAppWindowsLocked来将这些WindowState对象插入到WindowState堆栈的合适位置去。

对应的WindowState对象被移动到的合适位置是通过调用WindowManagerService类的成员函数findWindowOffsetLocked来获得的，它的实现如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    /**
     * Z-ordered (bottom-most first) list of all Window objects.
     */
    final ArrayList<WindowState> mWindows = new ArrayList<WindowState>();
    .....

    private int findWindowOffsetLocked(int tokenPos) {
        final int NW = mWindows.size();

        if (tokenPos >= mAppTokens.size()) {
            int i = NW;
            while (i > 0) {
                i--;
                WindowState win = mWindows.get(i);
                if (win.getAppToken() != null) {
                    return i+1;
                }
            }
        }

        while (tokenPos > 0) {
            // Find the first app token below the new position that has
            // a window displayed.

```

```

        final AppWindowToken wtoken = mAppTokens.get(tokenPos-1);
        .....
        if (wtoken.sendingToBottom) {
            .....
            tokenPos--;
            continue;
        }
        int i = wtoken.windows.size();
        while (i > 0) {
            i--;
            WindowState win = wtoken.windows.get(i);
            int j = win.mChildWindows.size();
            while (j > 0) {
                j--;
                WindowState cwin = win.mChildWindows.get(j);
                if (cwin.mSubLayer >= 0) {
                    for (int pos=NW-1; pos>=0; pos--) {
                        if (mWindows.get(pos) == cwin) {
                            .....
                            return pos+1;
                        }
                    }
                }
            }
            for (int pos=NW-1; pos>=0; pos--) {
                if (mWindows.get(pos) == win) {
                    .....
                    return pos+1;
                }
            }
        }
        tokenPos--;
    }

    return 0;
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。参数tokenPos描述的是一个AppWindowToken对象在WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList的位置，WindowManagerService类的成员函数findWindowOffsetLocked的目标就要找到与该AppWindowToken对象所对应的WindowState对象在WindowManagerService服务内部的一个WindowState堆栈的起始偏移位置。有了这个起始偏移位置之后，我们就可以将对应的所有WindowState对象有序地插入到该WindowState堆栈中去。WindowManagerService服务内部的WindowState堆栈是通过WindowManagerService类的成员变量mWindows来描述的。接下来我们就分两种情况来分析这个起始偏移位置的计算过程。

第一种情况是参数tokenPos的值大于WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList的大小。这是一种异常情况，一般来说，参数tokenPos是指向mAppTokens列表的某一个位置的，不过这时候意味着它所描述的AppWindowToken对象的Z轴位置要大于mAppTokens列表的最上面的一个AppWindowToken对象的Z轴位置的。这也就是说，与参数tokenPos所描述的AppWindowToken对象所对应的WindowState对象的要位于与mAppTokens列表的最上面的一个AppWindowToken对象所对应的任一个WindowState对象的上面。因此，就需要找到与mAppTokens列表的最上面的一个AppWindowToken对象所对应的Z轴位置最大的一个WindowState对象在WindowState堆栈中的位置i，然后就可以知道与参数tokenPos所描述的AppWindowToken对象所对应的WindowState对象在WindowState堆栈的起始偏移位置为i+1。

如何找到mAppTokens列表的最上面的一个AppWindowToken对象所对应的Z轴位置最大的一个WindowState对象在WindowState堆栈中的位置i呢？从图1可以可得到一个结论：WindowManagerService服务内部中的所有WindowState对象都是按照Z轴从位置从小到大排列在WindowState堆栈中的，并且在mAppTokens列表中，位于上面的一个AppWindowToken对象所对应的那些WindowState对象的Z轴位置是一定大于位于下面的一个AppWindowToken对象所对应的那些WindowState对象的Z轴位置的。因此，我们只要从WindowState堆栈的顶端开始往下遍历，找到这样的—一个WindowState对象，它是属于一个AppWindowToken对象的，即它的成员函数getAppToken的返回值不等于null，那么它在WindowState堆栈中的位置就是我们要找到的位置i。有了这个位置i之后，将它的值加上1，就可以得到参数t所描述的AppWindowToken对象所对应的WindowState对象在WindowState堆栈的起始偏移位置了。

第二种情况是参数tokenPos的值小于WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList的大小。根据前面得到的推论，我们只要在mAppTokens列表中找到一个AppWindowToken对象，它满足以下三个条件：

- A. 它在mAppTokens列表中的位置小于tokenPos；
- B. 它在WindowState堆栈中对应WindowState对象；

C. 它不是将要置于WindowState堆栈的底部。

如果一个AppWindowToken对象在WindowState堆栈中对应WindowState对象，那么这些WindowState对象也会同时按照Z轴从小到大的顺序保存它的成员变量windows所描述的一个ArrayList中，这意味着如果一个AppWindowToken对象满足条件B，那么它的成员变量windows所描述的一个ArrayList的大小就大于0。

如果一个AppWindowToken对象不是将要置于WindowState堆栈的底部，那么它的成员变量sendingToBottom的值就不等于true，这也意味这个AppWindowToken对象满足条件C。

如果能找到满足上述条件的一个AppWindowToken对象wtoken，那么我们只要找到与它所对应的Z轴位置最大的WindowState对象在WindowManagerService服务内部的WindowState堆栈中的位置i，那么将它的值加1，就可以得到与参数tokenPos所描述的AppWindowToken对象所对应的WindowState对象在WindowState堆栈的起始偏移位置了。

那么如何找到与这个AppWindowToken对象wtoken对应的Z轴位置最大的WindowState对象在WindowManagerService服务内部的WindowState堆栈中的位置i呢？从前面的图1可以知道，一个AppWindowToken对象所对应的WindowState对象可以划分为两种类型：第一种类型是父窗口类型的；第二种是子窗口类型的。如果一个WindowState对象所描述的窗口是父窗口，那么它的子窗口就保存在它的成员变量mChildWindows所描述的一个ArrayList中，并且这些子窗口是按照Z轴位置从小到大的顺序排列的，同时，该WindowState对象也会保存在与它所对应的一个AppWindowToken对象的成员变量windows所描述的一个ArrayList中。

有了上述结论，并且假设存在一个能够满足上述三个条件的AppWindowToken对象wtoken，那么就可以从上到下遍历保存在它的成员变量windows所描述的一个ArrayList中的每一个WindowState对象win：

I. 如果WindowState对象win所描述的一个窗口具有子窗口，那么就继续从上到下遍历这些子窗口，即从上到下遍历WindowState对象win的成员变量mChildWindows所描述的一个ArrayList。如果能找到一个WindowState对象cwin，它的成员变量mSubLayer的值大于等于0，那么该WindowState对象cwin在WindowManagerService服务内部的WindowState堆栈中的位置就是我们得到的位置i。注意，如果WindowState对象cwin的成员变量mSubLayer的值小于0，那么它虽然是一个子窗口，但是它却是位于父窗口的后面的，即它的Z轴位置是小于父窗口的Z轴位置的。

II. 如果WindowState对象win所描述的一个窗口不具有子窗口，即它的成员变量mChildWindows所描述的一个ArrayList的大小等于0，那么它在WindowManagerService服务内部的WindowState堆栈中的位置就是我们得到的位置i。

得到了位置i之后，将它的值加1，那么就可以得到与参数tokenPos所描述的AppWindowToken对象所对应的WindowState对象在WindowState堆栈的起始偏移位置了。

回到WindowManagerService类的成员函数moveAppToken中，调整好参数token所描述的AppWindowToken对象所对应的WindowState对象在WindowState堆栈中的位置之后，即调用了成员函数reAddAppWindowsLocked之后，这时候系统中的窗口的布局就会发生了变化，即系统中的窗口的Z轴位置关系发生了变化，那么接下来就需要调用成员函数updateFocusedWindowLocked来重新计算系统中的窗口的Z轴位置，并且调用成员函数performLayoutAndPlaceSurfacesLocked来重新布局系统中的窗口。

1. 移动AppWindowToken至顶端

移动AppWindowToken至顶端是通过调用WindowManagerService类的成员函数moveAppTokensToTop来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void moveAppTokensToTop(List<IBinder> tokens) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "moveAppTokensToTop()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        final long origId = Binder.clearCallingIdentity();
        synchronized(mWindowMap) {
            removeAppTokensLocked(tokens);
            final int N = tokens.size();
            for (int i=0; i<N; i++) {
                AppWindowToken wt = findAppWindowToken(tokens.get(i));
                if (wt != null) {
                    mAppTokens.add(wt);
                    if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
                        mToTopApps.remove(wt);
                        mToBottomApps.remove(wt);
                        mToTopApps.add(wt);
                        wt.sendingToBottom = false;
                        wt.sendingToTop = true;
                    }
                }
            }
        }

        if (mNextAppTransition == WindowManagerPolicy.TRANSIT_UNSET) {
            moveAppWindowsLocked(tokens, mAppTokens.size());
        }
    }
}
```

```

    }
}
Binder.restoreCallingIdentity(origId);
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数moveAppTokensToTop可以同时将一组AppWindowToken移至顶端，同时需要调用者具有android.Manifest.permission.MANAGE_APP_TOKENS权限。

参数tokens所描述的是一个IBinder接口列表，与这些IBinder接口所对应的AppWindowToken对象就是接下来要移至顶端的。在将保存在参数tokens中的IBinder接口所对应的AppWindowToken对象移至顶端之前，WindowManagerService类的成员函数首先会调用前面所描述的成员函数removeAppTokensLocked来删除这些AppWindowToken对象，然后再依次将它们添加到WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList的末尾去。

注意，WindowManagerService类的成员变量mNextAppTransition用来描述系统当前是否正在切换Activity窗口。如果是的话，那么它的值就不等于WindowManagerPolicy.TRANSIT_UNSET，这时候就需要：

A. 将所有要移至顶端的AppWindowToken对象都保存在WindowManagerService类的另外一个成员变量mToTopApps所描述的一个ArrayList中去，并且将这些AppWindowToken对象的成员变量sendingToTop的值设置为true。

B. 将所有要移至顶端的AppWindowToken对象所对应WindowState对象都移至WindowManagerService服务内部的一个WindowState堆栈的顶端去，这是通过调用另外一个成员函数moveAppWindowsLocked来实现的。

执行完成上述两个操作之后，与要移至顶端的AppWindowToken对象所对应的窗口就会位于窗口堆栈的最上面了。

1. 移动AppWindowToken至底端

移动AppWindowToken至顶端是通过调用WindowManagerService类的成员函数moveAppTokensToBottom来实现的，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void moveAppTokensToBottom(List<IBinder> tokens) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "moveAppTokensToBottom()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        final long origId = Binder.clearCallingIdentity();
        synchronized(mWindowMap) {
            removeAppTokensLocked(tokens);
            final int N = tokens.size();
            int pos = 0;
            for (int i=0; i<N; i++) {
                AppWindowToken wt = findAppWindowToken(tokens.get(i));
                if (wt != null) {
                    mAppTokens.add(pos, wt);
                    if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
                        mToTopApps.remove(wt);
                        mToBottomApps.remove(wt);
                        mToBottomApps.add(i, wt);
                        wt.sendingToTop = false;
                        wt.sendingToBottom = true;
                    }
                    pos++;
                }
            }

            if (mNextAppTransition == WindowManagerPolicy.TRANSIT_UNSET) {
                moveAppWindowsLocked(tokens, 0);
            }
        }
        Binder.restoreCallingIdentity(origId);
    }
    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的成员函数moveAppTokensToBottom可以同时将一组AppWindowToken移至底端。将一组AppWindowToken移至底端与将一组AppWindowToken移至顶端的实现是类似的，只不过是移动的方向相反而已。因此，WindowManagerService类的成员函数moveAppTokensToBottom的实现可以参考前面所分析的成员函数moveAppTokensToTop的实现，这里不再详述。

1. 增加WindowToken

从图1可以知道，如果一个WindowState对象不是与一个AppWindowToken对象对应的，那么它就必须要与一个WindowToken对象对应。例如，用来描述输入法窗口和壁纸窗口的WindowState对象对应的就是WindowToken对象，而不是AppWindowToken对象，因为它们不是Activity类型的窗口。

输入法窗口和壁纸窗口分别是由输入法管理服务InputMethodManagerService和壁纸管理服务WallpaperManagerService调用WindowManagerService类的成员函数addWindowToken来增加对应的WindowToken对象的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void addWindowToken(IBinder token, int type) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "addWindowToken()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        synchronized(mWindowMap) {
            WindowToken wtoken = mTokenMap.get(token);
            if (wtoken != null) {
                Slog.w(TAG, "Attempted to add existing input method token: " + token);
                return;
            }
            wtoken = new WindowToken(token, type, true);
            mTokenMap.put(token, wtoken);
            mTokenList.add(wtoken);
            if (type == TYPE_WALLPAPER) {
                mWallpaperTokens.add(wtoken);
            }
        }
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。调用WindowManagerService类的成员函数addWindowToken需要具有android.Manifest.permission.MANAGE_APP_TOKENS权限。

对于输入法窗口和壁纸窗口来说，参数token指向的是与它们所关联的一个Binder对象的IBinder接口，而参数type描述的是要在WindowManagerService服务内部增加WindowToken对象的窗口的类型。

WindowManagerService类的成员函数addWindowToken首先检查在成员变量mTokenMap所描述的一个HashMap检查是否已经存在一个WindowToken对象与参数token对应。如果已经存在的话，那么WindowManagerService类的成员函数addWindowToken就什么也不做就返回了，否则的话，就会使用参数token来创建一个WindowToken对象，并且会将该WindowToken对象分别保存在WindowManagerService类的成员变量mTokenMap和mTokenList中。

这里有两个地方需要注意：

A. 由于这里增加的是WindowToken对象，而不是AppWindowToken对象，因此，与增加AppWindowToken不同，这里不需要将新创建的WindowToken对象保存在WindowManagerService类的成员变量mAppTokens中。

B. 如果参数type的值等于TYPE_WALLPAPER，那么就意味着新创建的WindowToken对象是用来描述壁纸窗口的，这时候还需要将新创建的WindowToken对象保存在WindowManagerService类的成员变量mWallpaperTokens所描述的一个ArrayList中，以方便管理壁纸窗口。

对于非输入法窗口、非壁纸窗口以及非Activity窗口来说，它们所对应的WindowToken对象是在它们增加到WindowManagerService服务的时候创建的。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，增加一个窗口WindowManagerService服务最终是通过调用WindowManagerService类的成员函数addWindow来实现的，接下来我们就主要分析与创建WindowToken相关的逻辑，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int addWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int viewVisibility,
        Rect outContentInsets, InputChannel outInputChannel) {
        .....
    }
}
```

```

synchronized(mWindowMap) {
    .....

    boolean addToken = false;
    WindowToken token = mTokenMap.get(attrs.token);
    if (token == null) {
        if (attrs.type >= FIRST_APPLICATION_WINDOW
            && attrs.type <= LAST_APPLICATION_WINDOW) {
            .....
            return WindowManagerImpl.ADD_BAD_APP_TOKEN;
        }
        if (attrs.type == TYPE_INPUT_METHOD) {
            .....
            return WindowManagerImpl.ADD_BAD_APP_TOKEN;
        }
        if (attrs.type == TYPE_WALLPAPER) {
            .....
            return WindowManagerImpl.ADD_BAD_APP_TOKEN;
        }
        token = new WindowToken(attrs.token, -1, false);
        addToken = true;
    }

    .....

    if (addToken) {
        mTokenMap.put(attrs.token, token);
        mTokenList.add(token);
    }

    .....
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。如果参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量token所指向的一个IBinder接口在WindowManagerService类的成员变量mTokenMap所描述的一个HashMap中没有一个对应的WindowToken对象，并且该WindowManager.LayoutParams对象的成员变量type的值不等于TYPE_INPUT_METHOD、TYPE_WALLPAPER，以及不在FIRST_APPLICATION_WINDOW和LAST_APPLICATION_WINDOW，那么就意味着这时候要增加的窗口就既不是输入法窗口，也不是壁纸窗口和Activity窗口，因此，就需要以参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量token所指向的一个IBinder接口为参数来创建一个WindowToken对象，并且将该WindowToken对象保存在WindowManagerService类的成员变量mTokenMap和mTokenList中。

1. 删除WindowToken

删除WindowToken是通过调用WindowManagerService类的成员函数removeWindowToken来实现的，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void removeWindowToken(IBinder token) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "removeWindowToken()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        final long origId = Binder.clearCallingIdentity();
        synchronized(mWindowMap) {
            WindowToken wtoken = mTokenMap.remove(token);
            mTokenList.remove(wtoken);
            if (wtoken != null) {
                boolean delayed = false;
                if (!wtoken.hidden) {
                    wtoken.hidden = true;

```

```

        final int N = wtoken.windows.size();
        boolean changed = false;

        for (int i=0; i<N; i++) {
            WindowState win = wtoken.windows.get(i);

            if (win.isAnimating()) {
                delayed = true;
            }

            if (win.isVisibleNow()) {
                applyAnimationLocked(win,
                    WindowManagerPolicy.TRANSIT_EXIT, false);
                changed = true;
            }
        }

        if (changed) {
            mLayoutNeeded = true;
            performLayoutAndPlaceSurfacesLocked();
            updateFocusedWindowLocked(UPDATE_FOCUS_NORMAL);
        }

        if (delayed) {
            mExitingTokens.add(wtoken);
        } else if (wtoken.windowType == TYPE_WALLPAPER) {
            mWallpaperTokens.remove(wtoken);
        }
    }
    .....
} else {
    Slog.w(TAG, "Attempted to remove non-existing token: " + token);
}
}
Binder.restoreCallingIdentity(origId);
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。调用WindowManagerService类的成员函数removeWindowToken需要具有android.Manifest.permission.MANAGE_APP_TOKENS权限。

WindowManagerService类的成员函数removeWindowToken首先找到与参数token所描述的Binder接口所对应的WindowToken对象，接着再将该WindowToken对象从WindowManagerService类的成员变量mTokenMap和mTokenList中删除。

删除了一个WindowToken对象之后，如果该WindowToken对象不是处于不可见的状态，即它的成员变量hidden的值不等于false，那么就意味着它所描述窗口也有可能是可见的，那么WindowManagerService类的成员函数removeWindowToken就需要作以下两个检查：

- A. 如果该WindowToken对象所描述的窗口的其中一个处于动画显示过程，即用来描述该窗口的一个WindowState对象的成员函数isAnimating的返回值等于true，那么就需要该WindowToken对象的状态设置为正在退出状态，即将它保存在WindowManagerService类的成员变量mExitingTokens所描述的一个ArrayList中。
- B. 如果该WindowToken对象所描述的窗口是可见的，即用来描述该窗口的一个WindowState对象的成员函数isVisibleNow的返回值等于true，那么就需要调用WindowManagerService类的成员函数applyAnimationLocked来给它应用一个退出动画，该退出动画是通过调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来实现的。当一个窗口退出了之后，系统当前获得焦点的窗口可能会发生变化，这时候就需要调用WindowManagerService类的成员函数updateFocusedWindowLocked来重新调整系统当前获得焦点的窗口。

注意，如果正在删除的WindowToken对象是用来描述壁纸窗口的，那么还需要将该WindowToken对象从WindowManagerService类的成员变量mWallpaperTokens所描述的一个ArrayList中删除。

1. 增加WindowState

从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，增加一个窗口WindowManagerService服务最终是通过调用WindowManagerService类的成员函数addWindow来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
```

```

        implements Watchdog.Monitor {
        .....

    /**
     * Mapping from an IWindow IBinder to the server's Window object.
     * This is also used as the lock for all of our state.
     */
    final HashMap<IBinder, WindowState> mWindowMap = new HashMap<IBinder, WindowState>();
    .....

    /**
     * Z-ordered (bottom-most first) list of all Window objects.
     */
    final ArrayList<WindowState> mWindows = new ArrayList<WindowState>();
    .....

    public int addWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int viewVisibility,
        Rect outContentInsets, InputChannel outInputChannel) {
        .....

        WindowState win = null;

        synchronized(mWindowMap) {
            .....

            win = new WindowState(session, client, token,
                attachedWindow, attrs, viewVisibility);
            .....

            mWindowMap.put(client.asBinder(), win);
            .....

            if (attrs.type == TYPE_INPUT_METHOD) {
                mInputMethodWindow = win;
                addInputMethodWindowToListLocked(win);
                .....
            } else if (attrs.type == TYPE_INPUT_METHOD_DIALOG) {
                mInputMethodDialogs.add(win);
                addWindowToListInOrderLocked(win, true);
                adjustInputMethodDialogsLocked();
                .....
            } else {
                addWindowToListInOrderLocked(win, true);
                if (attrs.type == TYPE_WALLPAPER) {
                    .....
                    adjustWallpaperWindowsLocked();
                } else if ((attrs.flags & FLAG_SHOW_WALLPAPER) != 0) {
                    adjustWallpaperWindowsLocked();
                }
            }
        }
        .....
    }
    .....
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类有两个成员变量mWindowMap和mWindows是用来保存系统中的WindowState对象。其中，成员变量mWindowMap指向的是一个HashMap，它的关键字是一个IBinder接口，一般这个IBinder接口指向的是一个Binder代理对象，引用了运行在应用程序进程这一侧的一个类型为W的Binder本地对象，用来描述一个窗口；成员变量mWindows指向的是一个ArrayList，保存在它里面的WindowState对象是按照其Z轴位置从小到大的顺序排列的。成员变量mWindowMap和mWindows的区别在于，前者给在定一个IBinder接口的情况下，可以快速找到与对应的WindowState对象，而后者用来从上到下或者下到上遍历系统的WindowState对象。由于系统中的WindowState对象是按照其Z轴位置从小到大的顺序排列在成员变量mWindows中的，因此，成员变量mWindows所指向的ArrayList就是我们在前面图1中所说

的Window Stack。

理解了WindowManagerService类有两个成员变量mWindowMap和mWindows的作用之后，WindowManagerService类的成员函数addWindow增加一个WindowState对象的过程就容易理解了。

参数client是一个Binder代理对象，引用了运行在应用程序进程这一侧的一个类型为W的Binder本地对象，用来描述要增加到WindowManagerService服务中的一个窗口。WindowManagerService类的成员函数addWindow首先创建一个WindowState对象win，接着再以参数client所描述的一个Binder代理对象的IBinder接口为关键字，将WindowState对象win保存在WindowManagerService类的成员变量mWindowMap中，最后还会根据要增加到WindowManagerService服务中的窗口的类型来调用不同的成员函数将WindowState对象win增加到WindowManagerService类的成员变量mWindows中：

A. 如果要增加的是输入法窗口，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUTMETHOD，那么就会调用成员函数addInputMethodWindowToListLocked来将WindowState对象win增加到WindowManagerService类的成员变量mWindows中去，并且会将WindowState对象win保存在WindowManagerService类的成员变量mInputMethodWindow中。

B. 如果要增加的是输入法对话框，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUTMETHOD_DIALOG，那么就会调用成员函数addWindowToListInOrderLocked来将WindowState对象win增加到WindowManagerService类的成员变量mWindows中去，并且会将WindowState对象win保存在WindowManagerService类的成员变量mInputMethodDialogs中，以及调用成员函数adjustInputMethodDialogsLocked来调整刚才所添加的输入法窗口在窗口堆栈中的位置，使得它位于系统当前需要输入法窗口的窗口的上面。

C. 如果要增加的是壁纸窗口，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_WALLPAPER，那么就会调用成员函数addWindowToListInOrderLocked来将WindowState对象win增加到WindowManagerService类的成员变量mWindows中去，并且会调用成员函数adjustWallpaperWindowsLocked来调整刚才所添加的壁纸窗口在窗口堆栈中的位置，使得它位于系统当前需要壁纸窗口的窗口的下面。

D. 如果要增加的既不是输入法窗口，也不是输入法对话框和壁纸窗口，那么就只会调用成员函数addWindowToListInOrderLocked来将WindowState对象win增加到WindowManagerService类的成员变量mWindows中去，但是如果增加的窗口需要显示壁纸，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_SHOW_WALLPAPER位等于1，那么还会继续调用成员函数adjustWallpaperWindowsLocked来调整系统中的壁纸窗口在窗口堆栈中的位置，使得它位于刚才所添加的窗口的下面。

在后面的两篇文章中，我们再详细分析WindowManagerService类的成员函数addInputMethodWindowToListLocked、adjustInputMethodDialogsLocked和adjustWallpaperWindowsLocked的实现，其中，前两者是与输入法窗口相关的，而后者是与壁纸窗口相关的。本文主要关注WindowManagerService类的成员函数addWindowToListInOrderLocked的实现，它会将一个指定的WindowState对象增加到窗口堆栈中的合适位置上去。

1. 增加WindowState到窗口堆栈

从前面的分析可以知道，将一个WindowState对象增加到WindowManagerService服务内部中的窗口堆栈，即WindowManagerService类的成员变量mWindows，是通过调用WindowManagerService类的成员函数addWindowToListInOrderLocked来实现的。

WindowManagerService类的成员函数addWindowToListInOrderLocked的实现比较复杂，我们先列出它的框架，然后再详细分析它的实现，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private void addWindowToListInOrderLocked(WindowState win, boolean addToToken) {
        final IWindow client = win.mClient;
        final WindowToken token = win.mToken;
        final ArrayList<WindowState> localmWindows = mWindows;

        final int N = localmWindows.size();
        final WindowState attached = win.mAttachedWindow;
        int i;
        if (attached == null) {
            //CASE 1: 要增加的窗口win没有附加在其它窗口上
            int tokenWindowsPos = token.windows.size();
            if (token.appWindowToken != null) {
                //CASE 1.1: 要增加的窗口win是一个Activity窗口
                int index = tokenWindowsPos-1;
                if (index >= 0) {
                    //CASE 1.1.1: 用来要增加的窗口win的令牌token已存在其它窗口
                    .....
                } else {
                    //CASE 1.1.2: 用来要增加的窗口win的令牌token尚未存在任何窗口
                    .....
                }
            } else {
                //CASE 1.2: 要增加的窗口win不是一个Activity窗口
                .....
            }
        }
    }
```

```

        if (addToToken) {
            token.windows.add(tokenWindowsPos, win);
        }
    } else {
        //CASE 2: 要增加的窗口win附加在窗口attached上
        .....
    }

    if (win.mAppToken != null && addToToken) {
        win.mAppToken.allAppWindows.add(win);
    }
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。我们首先分析一下WindowManagerService类的成员函数addWindowToListInOrderLocked的几个本地变量的含义：

A. token。本地变量token指向的是参数win所描述的一个WindowState对象的成员变量mToken所指向一个WindowToken对象，这个WindowToken对象用来描述WindowState对象win所对应的窗口令牌。

B. localmWindows。本地变量localmWindows指向的是WindowManagerService类的成员变量mWindows所描述的一个ArrayList，即一个窗口堆栈，WindowManagerService类的成员函数addWindowToListInOrderLocked的目标就是要将参数win所描述的一个WindowState对象增加到该窗口堆栈的合适位置上去。

C. attached。本地变量attached指向的是参数win所描述的一个WindowState对象的成员变量mAttachedWindow 所指向的一个WindowState对象，如果它的值不等于null，那么就意味参数win所描述的窗口要附加在本地变量attached所描述的窗口上。

D. tokenWindowsPos。本地变量tokenWindowsPos用来描述与窗口令牌token所对应的窗口的数量。

E. token.appWindowToken。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，如果一个WindowToken对象的成员变量appWindowToken的值不等于null，那么就意味着该WindowToken对象的实际类型为是AppWindowToken，即它所描述的是一个Activity窗口令牌，这种类型的令牌的特点是在ActivityManagerService服务的Activity组件堆栈中对应有一个ActivityRecord对象，如图1所示。

F. index。本地变量index的值等于tokenWindowsPos-1，如果它的值大于等于0，那么就意味着窗口令牌token已经存在其它窗口，否则的话，就意味着窗口令牌token尚未存在任何窗口。

从这些本地变量的含义，我们就可以分情况来将参数win所描述的一个WindowState对象增加到WindowManagerService服务内部的窗口堆栈的合适位置上去：

CASE 1: 要增加的窗口win没有附加在其它窗口上

——CASE 1.1: 要增加的窗口win是一个Activity窗口

——CASE 1.1.1: 用来要增加的窗口win的令牌token已存在其它窗口。这时候意味着窗口win需要保存在其它已经存在的窗口的附近，因此，我们只要找到这些已经存在的窗口在窗口堆栈中的位置，那么再根据其它属性，就可以将窗口win保存在已经存在的窗口的上面或者下面。

——CASE 1.1.2: 用来要增加的窗口win的令牌token尚未存在任何窗口。虽然这时候窗口win在窗口堆栈中没有位置可以参考，但是它毕竟是一个Activity窗口，我们可以通过与它所对应的AppWindowToken对象在AppTokenList（即WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList）中的位置来获得它窗口堆栈中的位置。回忆我们在前面第3节分析移动AppWindowToken至指定位置的操作时得到的结论：WindowManagerService服务内部中的所有WindowState对象都是按照Z轴从位置从小到大排列在WindowState堆栈中的，并且在mAppTokens列表中，位于上面的一个AppWindowToken对象所对应的那些WindowState对象的Z轴位置是一定大于位于下面的一个AppWindowToken对象所对应的那些WindowState对象的Z轴位置的。因此，我们只要找到用来描述窗口win的一个AppWindowToken对象（token.appWindowToken）的上一个或者下一个AppWindowToken对象所对应的窗口在窗口堆栈中的位置，那么就可以将这个位置为参考，得到窗口win在窗口堆栈中的位置。

——CASE 1.2: 要增加的窗口win不是一个Activity窗口。这时候既然要增加的窗口也没有附加在其它窗口上，那么就意味着要增加的窗口win在窗口堆栈中没有位置可以参考，因此，我们就需要根据它的Z轴位置来决定它在窗口堆栈的位置。

CASE 2: 要增加的窗口win附加在窗口attached上。这时候就意味着要增加的窗口win要保存在窗口attached的上面，即窗口在窗口堆栈的位置要以窗口attached在窗口堆栈的位置为参考。

从上面的分析就可以知道，CASE 1.1.1、CASE 1.1.2和CASE 2都有一个共同特点，即要增加的窗口win在窗口堆栈的位置有一个参考值，而在CASE 1.2中，要增加的窗口win在窗口堆栈的位置没有参考值，需要通过其Z轴位置来确定。

在分析上述四种情况之前，我们还需要再说明一下WindowManagerService类的成员函数addWindowToListInOrderLocked的参数addToToken的含义。参数addToToken是一个布尔变量，如果它的值等于true，那么就说明需要将参数win所描述的一个WindowState对象添加用来描述它的窗口令牌token的成员变量windows所描述的一个ArrayList中去。注意，窗口令牌token的成员变量windows所描述的一个ArrayList里面所保存的WindowState对象是按照Z轴位置从小到大的顺序来排列的，因此，在将WindowState对象win保存到ArrayList之前，首先要按照它的Z轴位置计算得到它在这个ArrayList中的位置tokenWindowsPos。另一方面，在参数addToToken的值等于true，并

且参数win所描述的是一个Activity窗口，即它的成员变量mAppToken不等于null的情况下，还需要将参数win所描述的一个WindowState对象保存在用来描述它的窗口令牌，即一个AppWindowToken对象成员变量allAppWindows所描述的一个ArrayList中去，以便可以知道一个AppWindowToken对象对应的Activity窗口都有哪些。

接下来，我们就分别分析这四种情况是如何将窗口win增加窗口堆栈中去的。

CASE 1.1.1对应的代码为：

```
if (win.mAttrs.type == TYPE_BASE_APPLICATION) {
    // Base windows go behind everything else.
    placeWindowBefore(token.windows.get(0), win);
    tokenWindowsPos = 0;
} else {
    AppWindowToken atoken = win.mAppToken;
    if (atoken != null &&
        token.windows.get(index) == atoken.startingWindow) {
        placeWindowBefore(token.windows.get(index), win);
        tokenWindowsPos--;
    } else {
        int newIdx = findIdxBasedOnAppTokens(win);
        if(newIdx != -1) {
            //there is a window above this one associated with the same
            //apptoken note that the window could be a floating window
            //that was created later or a window at the top of the list of
            //windows associated with this token.
            .....
            localmWindows.add(newIdx+1, win);
            mWindowsChanged = true;
        }
    }
}
```

这段代码又分为三种情况来将参数win所描述的一个WindowState对象添加到窗口堆栈中：

A. 参数win描述的窗口的类型为TYPE_BASEAPPLICATION。在一个令牌对应的所有窗口中，类型为TYPE_BASEAPPLICATION的窗口位于其它类型的窗口的下面。因此，这段代码就会调用WindowManagerService类的成员函数placeWindowBefore来将参数win所描述的一个WindowState对象保存窗口堆栈中，并且它是位于令牌token的窗口列表的第0个位置的WindowState对象的下面。这时候变量tokenWindowsPos的值会被设置为0，表示参数win所描述的一个WindowState对象要保存窗口令牌token的窗口列表的第0个位置上。

B. 参数win描述的一个WindowState对象的成员变量mAppToken的值不等于null，这意味着参数win描述的是一个Activity窗口，这时候如果窗口令牌atoken（与token描述的是同一个窗口令牌）的窗口列表的第index个位置（即最上面的一个位置）的WindowState对象描述的是一个Activity启动窗口，即与窗口令牌atoken的成员变量startingWindow描述的是同一个窗口，那么就说明窗口令牌atoken的窗口列表的第index个位置的WindowState对象描述的是窗口win的启动窗口。由于一个窗口的启动窗口总是位于它的上面，因此，这段代码就会调用WindowManagerService类的成员函数placeWindowBefore来将参数win所描述的一个WindowState对象保存窗口堆栈中，并且它是位于令牌atoken的窗口列表的第index个位置的WindowState对象的下面。这时候变量tokenWindowsPos的值减少1，即相当于是等于index，表示参数win所描述的一个WindowState对象要插入在窗口令牌token的窗口列表的第index个位置上。

C. 参数win所描述的窗口的类型既不是TYPE_BASEAPPLICATION，而且它也没有启动窗口，那么这时候就需要将它保存在窗口令牌token的窗口列表的最上面一个窗口的上面。窗口令牌token的窗口列表的最上面一个窗口在窗口堆栈中的位置newIdx是通过调用WindowManagerService类的成员函数findIdxBaseOnAppTokens来获得的，这时候参数win所描述的一个WindowState对象就应该保存在窗口堆栈，即变量localmWindows所描述的一个ArrayList的第newIdx+1个位置上。

CASE 1.1.2对应的代码为：

```
// Figure out where the window should go, based on the
// order of applications.
final int NA = mAppTokens.size();
WindowState pos = null;
for (i=NA-1; i>=0; i--) {
    AppWindowToken t = mAppTokens.get(i);
    if (t == token) {
        i--;
        break;
    }

    // We haven't reached the token yet; if this token
    // is not going to the bottom and has windows, we can
    // use it as an anchor for when we do reach the token.
    if (!t.sendingToBottom && t.windows.size() > 0) {
        pos = t.windows.get(0);
    }
}
```

```

    }
    // We now know the index into the apps. If we found
    // an app window above, that gives us the position; else
    // we need to look some more.
    if (pos != null) {
        // Move behind any windows attached to this one.
        WindowToken atoken = mTokenMap.get(pos.mClient.asBinder());
        if (atoken != null) {
            final int NC = atoken.windows.size();
            if (NC > 0) {
                WindowState bottom = atoken.windows.get(0);
                if (bottom.mSubLayer < 0) {
                    pos = bottom;
                }
            }
        }
        placeWindowBefore(pos, win);
    } else {
        // Continue looking down until we find the first
        // token that has windows.
        while (i >= 0) {
            AppWindowToken t = mAppTokens.get(i);
            final int NW = t.windows.size();
            if (NW > 0) {
                pos = t.windows.get(NW-1);
                break;
            }
            i--;
        }
        if (pos != null) {
            // Move in front of any windows attached to this
            // one.
            WindowToken atoken = mTokenMap.get(pos.mClient.asBinder());
            if (atoken != null) {
                final int NC = atoken.windows.size();
                if (NC > 0) {
                    WindowState top = atoken.windows.get(NC-1);
                    if (top.mSubLayer >= 0) {
                        pos = top;
                    }
                }
            }
        }
        placeWindowAfter(pos, win);
        placeWindowAfter(pos, win);
    } else {
        // Just search for the start of this layer.
        final int myLayer = win.mBaseLayer;
        for (i=0; i<N; i++) {
            WindowState w = localmWindows.get(i);
            if (w.mBaseLayer > myLayer) {
                break;
            }
        }
        .....
        localmWindows.add(i, win);
        mWindowsChanged = true;
    }
}
}

```

这段代码要能冠军WindowManagerService服务内部的一个AppWindowToken列表mAppTokens来在窗口堆栈中找到一个参数位置来保存参数win所描述的一个WindowState对象。最上面的一个for循环执行完成之后，我们假设变量pos的值不等于null，这时候它与变量i以及变量token的关系如图2所示：

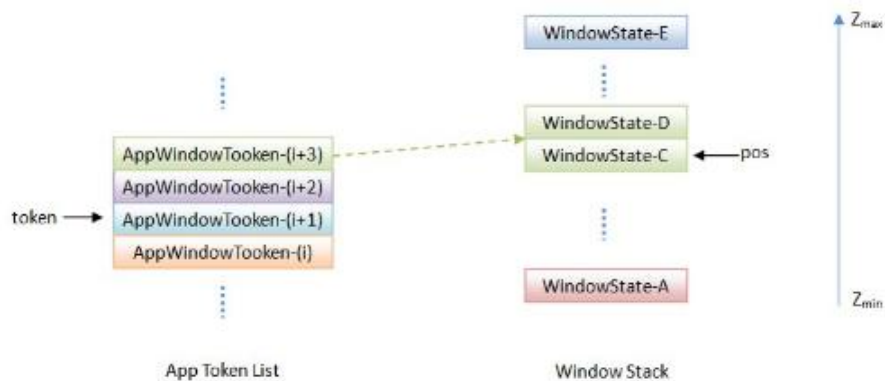


图2 窗口win位于窗口C的下面

这时候位于令牌token上面的令牌在窗口堆栈中对应有WindowState对象。注意，这时候第i+2个令牌在窗口堆栈中不对应有WindowState对象，而第i+3个令牌在窗口堆栈中对应有C和D两个WindowState对象，并且这两个WindowState对象所描述的窗口都不是即将要切换到窗口堆栈的底部的。由于第i+3个令牌位于令牌token的上面，并且这两个令牌之间的其它令牌在窗口堆栈中不对应有WindowState对象，因此，这时候参数win所描述的WindowState对象在窗口堆栈中的位置应该以第i+3个令牌所对应的Z轴位置最小的WindowState对象在窗口堆栈中的位置为参考，即以WindowState对象C在窗口堆栈中的位置为参考，而WindowState对象C也正好是变量pos所指向的WindowState对象。

接下来，上述代码会继续检查WindowState对象C是否附加有SubLayer值小于0的窗口。如果有的话，那么就会将变量pos指向SubLayer值最小的那个WindowState对象，这是因为该WindowState对象是在WindowState对象C的最下面的，并且它与WindowState对象C是同属一个令牌的。最后，上述代码就会调用WindowManagerService类的成员函数placeWindowBefore来将参数win所描述的一个WindowState对象保存窗口堆栈中由变量pos所指向的那个WindowState对象的下面。

假设最上面的一个for循环执行完成之后，变量pos的值等于null，那么就说明位于令牌token上面的令牌在窗口堆栈中都没有对应有WindowState对象，或者说它们所对应的WindowState对象都是即将要切换到窗口堆栈的底部去的，这时候就需要通过位于令牌token上面的令牌在窗口堆栈中找到一个参考位置来保存参数win所描述的WindowState对象，这是通过中间的while循环来实现的。

中间的while循环执行完成之后，假设变量pos的值不等于null，这时候它与变量i以及变量token的关系如图3所示：

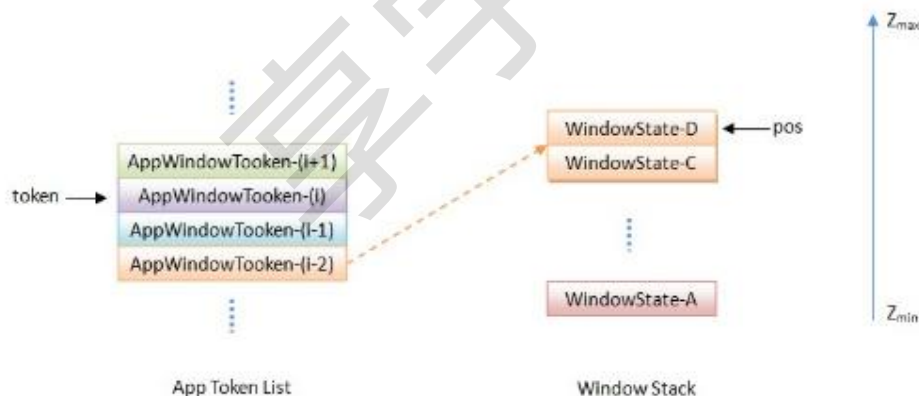


图3 窗口win位于窗口D的上面

这时候位于令牌token上面的令牌在窗口堆栈中没有对应有WindowState对象。注意，这时候第i-1个令牌在窗口堆栈中不对应有WindowState对象，而第i-2个令牌在窗口堆栈中对应有C和D两个WindowState对象。由于第i-2个令牌位于令牌token的下面，并且这两个令牌之间的其它令牌在窗口堆栈中不对应有WindowState对象，因此，这时候参数win所描述的WindowState对象在窗口堆栈中的位置应该以第i-2个令牌所对应的Z轴位置最大的WindowState对象在窗口堆栈中的位置为参考，即以WindowState对象D在窗口堆栈中的位置为参考，而WindowState对象D也正好是变量pos所指向的WindowState对象。

接下来，上述代码会继续检查WindowState对象D是否附加有SubLayer值大于等于0的窗口。如果有的话，那么就会将变量pos指向SubLayer值最大的那个WindowState对象，这是因为该WindowState对象是在WindowState对象D的最上面的，并且它与WindowState对象D是同属一个令牌的。最后，上述代码就会调用WindowManagerService类的成员函数placeWindowAfter来将参数win所描述的一个WindowState对象保存窗口堆栈中由变量pos所指向的那个WindowState对象的上面。

假设中间的while循环执行完成之后，变量pos的值等于null，这时候就说明在窗口堆栈中实在是找不到参考位置来保存参数win所描述的WindowState对象了，因此，就只能通过参数win所描述的WindowState对象的Z轴位置，即它的成员变量mBaseLayer的值来在窗口堆栈中找到一个合适的位置了，如最下面的for循环所示。由于窗口堆栈中的

WindowState对象是按照它们的Z轴位置由小到大的顺序来排列的，因此，最下面的for循环只要从下到上找到一个Z轴位置比参数win所描述的WindowState对象的Z轴位置大的一个WindowState对象在窗口堆栈中的位置i，那么就可以将参数win所描述的WindowState对象插入在窗口堆栈的第i个位置上了。

CASE 1.2对应的代码为：

```
// Figure out where window should go, based on layer.
final int myLayer = win.mBaseLayer;
for (i=N-1; i>=0; i--) {
    if (localmWindows.get(i).mBaseLayer <= myLayer) {
        i++;
        break;
    }
}
if (i < 0) i = 0;
.....
localmWindows.add(i, win);
mWindowsChanged = true;
```

由于这时候在窗口堆栈中是没有参考位置来保存参数win所描述的WindowState对象的，因此，这段代码就只能通过参数win所描述的WindowState对象的Z轴位置，即它的成员变量mBaseLayer的值来在窗口堆栈中找到一个合适的位置了，如这段代码中的for循环所示。由于窗口堆栈中的WindowState对象是按照它们的Z轴位置由小到大的顺序来排列的，因此，这段代码中的for循环只要从上到下找到一个WindowState对象，它的Z轴位置小于或者等于参数win所描述的WindowState对象的Z轴位置，那么该WindowState对象在窗口堆栈中的位置i就可以用插入参数win所描述的WindowState对象了。 CASE 2对应的代码为：

```
// Figure out this window's ordering relative to the window
// it is attached to.
final int NA = token.windows.size();
final int sublayer = win.mSubLayer;
int largestSublayer = Integer.MIN_VALUE;
WindowState windowWithLargestSublayer = null;
for (i=0; i<NA; i++) {
    WindowState w = token.windows.get(i);
    final int wSublayer = w.mSubLayer;
    if (wSublayer >= largestSublayer) {
        largestSublayer = wSublayer;
        windowWithLargestSublayer = w;
    }
    if (sublayer < 0) {
        // For negative sublayers, we go below all windows
        // in the same sublayer.
        if (wSublayer >= sublayer) {
            if (addToToken) {
                token.windows.add(i, win);
            }
            placeWindowBefore(
                wSublayer >= 0 ? attached : w, win);
            break;
        }
    } else {
        // For positive sublayers, we go above all windows
        // in the same sublayer.
        if (wSublayer > sublayer) {
            if (addToToken) {
                token.windows.add(i, win);
            }
            placeWindowBefore(w, win);
            break;
        }
    }
}
if (i >= NA) {
    if (addToToken) {
        token.windows.add(win);
    }
    if (sublayer < 0) {
        placeWindowBefore(attached, win);
    } else {
        placeWindowAfter(largestSublayer >= 0
            ? windowWithLargestSublayer
```

```

        : attached,
        win);
    }
}

```

这段代码要将参数win所描述的WindowState对象附加在变量attached所描述的WindowState对象的上面或者下面，取决于它的成员变量mSubLayer的值是大于0还是小于0。我们分四种情况来考虑。第一种情况是参数win所描述的WindowState对象的成员变量mSubLayer的值小于0，并且这时候在附加在窗口attached的WindowState对象中，存在一个WindowState对象，它的成员变量mSubLayer的值大于等于参数win所描述的WindowState对象的成员变量mSubLayer的值，如图4和图5所示：

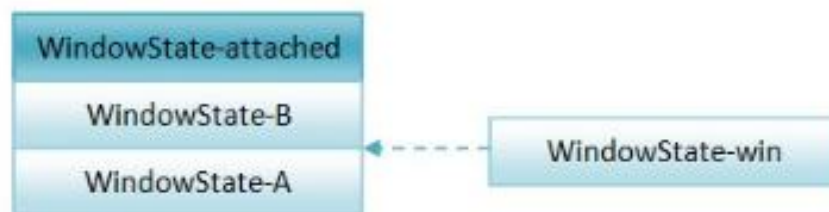


图4 窗口win插入到窗口B的下面



图5 窗口win插入在窗口attached的下面

在图4和图5中，WindowState对象A和B均是附加在WindowState对象attached中。

在图4中，WindowState对象A和B的成员变量mSubLayer的值均小于0，而WindowState对象win的成员变量mSubLayer的值比WindowState对象A的大，但是比WindowState对象B的小，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象B的下面，这是通过调用WindowManagerService类的成员函数placeWindowBefore来实现的。

在图5中，WindowState对象A和B的成员变量mSubLayer的值均大于0，由于WindowState对象win的成员变量mSubLayer的值小于0，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象attached的下面，这是通过调用WindowManagerService类的成员函数placeWindowBefore来实现的。

第二种情况是参数win所描述的WindowState对象的成员变量mSubLayer的值大于0，并且这时候在附加在窗口attached的WindowState对象中，存在一个WindowState对象，它的成员变量mSubLayer的值大于参数win所描述的WindowState对象的成员变量mSubLayer的值，如图6所示：

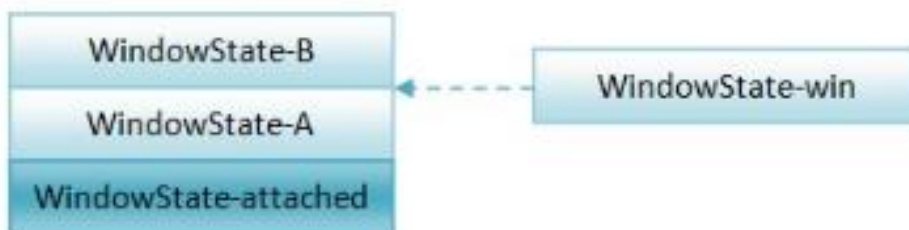


图6 窗口win插入在窗口B的下面

在图6中，WindowState对象A和B均是附加在WindowState对象attached中。其中，WindowState对象A和B的成员变量mSubLayer的值均大于0，而WindowState对象win的成员变量mSubLayer的值比WindowState对象A的大，但是比WindowState对象B的小，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象B的下面，这是通过调

用WindowManagerService类的成员函数placeWindowBefore来实现的。

第三种情况是参数win所描述的WindowState对象的成员变量mSubLayer的值小于0，但是在附加在窗口attached的WindowState对象中，找不到一个WindowState对象，它的成员变量mSubLayer的值比WindowState对象的成员变量mSubLayer的值大，如图7所示：

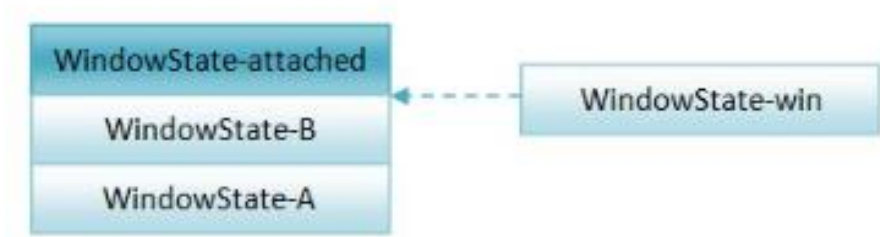


图7 窗口win插入在窗口attached的下面

在图7中，WindowState对象A和B均是附加在WindowState对象attached中。其中，WindowState对象A和B以及win的成员变量mSubLayer的值均小于0，但是WindowState对象win的成员变量mSubLayer的值比WindowState对象A和B的都要大，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象attached的下面，这是通过调用WindowManagerService类的成员函数placeWindowBefore来实现的。

第四种情况是参数win所描述的WindowState对象的成员变量mSubLayer的值大于等于0，但是在附加在窗口attached的WindowState对象中，找不到一个WindowState对象，它的成员变量mSubLayer的值比WindowState对象的成员变量mSubLayer的值大，如图8和图9所示：

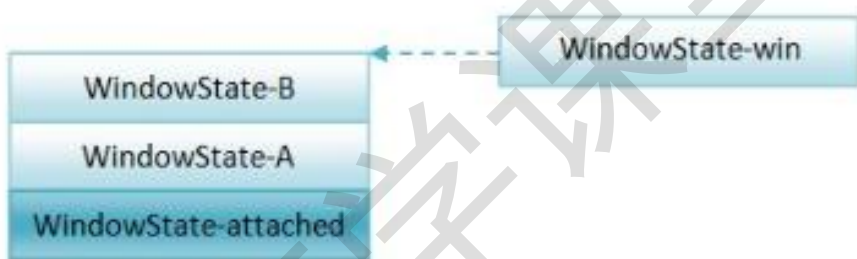


图8 窗口win插入在窗口B的上面

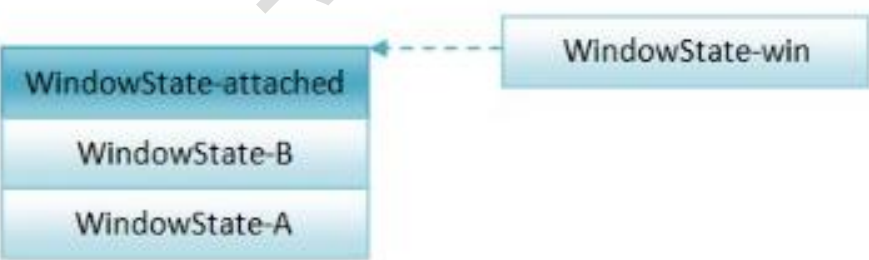


图9 窗口win插入在窗口attached的上面

在图8和图9中，WindowState对象A和B均是附加在WindowState对象attached中。

在图8中，WindowState对象A和B的成员变量mSubLayer的值均大于0，并且WindowState对象win的成员变量mSubLayer的值比WindowState对象A和B的都要大，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象B的上面，这是通过调用WindowManagerService类的成员函数placeWindowAfter来实现的。

在图9中，WindowState对象A和B的成员变量mSubLayer的值均小于等于0，而WindowState对象win的成员变量mSubLayer的值大于0，这时候WindowState对象win在窗口堆栈中就应该位于WindowState对象attached的上面，这是通过调用WindowManagerService类的成员函数placeWindowAfter来实现的。

注意，在这四种情况中，如果参数addToToken的值等于true，那么都需要将参数win所描述的WindowState对象增加到与它所对应的窗口令牌token的窗口列表windows中去。

10. 删除WindowState

删除WindowState是通过调用WindowManagerService类的成员函数tmpRemoveWindowLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private int tmpRemoveWindowLocked(int interestingPos, WindowState win) {
        int wpos = mWindows.indexOf(win);
        if (wpos >= 0) {
            if (wpos < interestingPos) interestingPos--;
            .....
            mWindows.remove(wpos);
            mWindowsChanged = true;
            int NC = win.mChildWindows.size();
            while (NC > 0) {
                NC--;
                WindowState cw = win.mChildWindows.get(NC);
                int cpos = mWindows.indexOf(cw);
                if (cpos >= 0) {
                    if (cpos < interestingPos) interestingPos--;
                    .....
                    mWindows.remove(cpos);
                }
            }
        }
        return interestingPos;
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的成员函数tmpRemoveWindowLocked将参数win所描述的窗口及其子窗口从WindowManagerService服务内部的窗口堆栈中删除，即从WindowManagerService类的成员变量mWindows所描述的一个ArrayList中删除。

如果每一个被删除的窗口在窗口堆栈中的位置比参数interestingPos的值小，那么WindowManagerService类的成员函数tmpRemoveWindowLocked还会将参数interestingPos的值减少1，这相当于是计算当删除参数win所描述的窗口及其子窗口之后，原来位于窗口堆栈中第interestingPos个位置的窗口现在位于窗口堆栈的位置，这个位置最终会作为WindowManagerService类的成员函数tmpRemoveWindowLocked的返回值。

1. 在指定位置增加WindowState

在指定位置增加WindowState是通过调用WindowManagerService类的成员函数reAddWindowLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final int reAddWindowLocked(int index, WindowState win) {
        final int NCW = win.mChildWindows.size();
        boolean added = false;
        for (int j=0; j<NCW; j++) {
            WindowState cwin = win.mChildWindows.get(j);
            if (!added && cwin.mSubLayer >= 0) {
                .....
                mWindows.add(index, win);
                index++;
                added = true;
            }
            .....
            mWindows.add(index, cwin);
            index++;
        }
        if (!added) {
            .....
            mWindows.add(index, win);
            index++;
        }
        mWindowsChanged = true;
        return index;
    }
}
```

```
.....  
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

参数win描述的即为要增加的WindowState对象，而参数index描述的即为要将参数win所描述的WindowState对象及其子WindowState对象要增加到窗口堆栈中的起始位置。

由于参数win所描述的WindowState对象的子WindowState对象的成员变量mSubLayer的值可能会小于0，也可能大于0。大于0的子WindowState对象位于参数win所描述的WindowState对象的上面，而小于0的子WindowState对象位于参数win所描述的WindowState对象的下面。因此，WindowManagerService类的成员函数reAddWindowLocked先增加那些小于0的子WindowState对象，接着再增加参数win所描述的WindowState对象，最后增加那些大于0的子WindowState对象。

假设WindowManagerService类的成员函数reAddWindowLocked一共在窗口堆栈中增加了N个WindowState对象，那么它的返回值就等于index + N，这样调用者就可以知道参数win所描述的WindowState对象及其子WindowState对象在窗口堆栈中的最高位置是多少。

基于第9、第10和第11这三操作，可以组合成很多其它的WindowState操作，如接下来的第12、第13、第14和第15个操作所示。

1. 将一个WindowState对象及其所有子WindowState对象增加到窗口堆栈中

将一个WindowState对象及其所有子WindowState对象增加到窗口堆栈中是通过调用WindowManagerService类的成员函数reAddWindowToListInOrderLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub  
    implements Watchdog.Monitor {  
    .....  
  
    private void reAddWindowToListInOrderLocked(WindowState win) {  
        addWindowToListInOrderLocked(win, false);  
        // This is a hack to get all of the child windows added as well  
        // at the right position. Child windows should be rare and  
        // this case should be rare, so it shouldn't be that big a deal.  
        int wpos = mWindows.indexOf(win);  
        if (wpos >= 0) {  
            .....  
            mWindows.remove(wpos);  
            mWindowsChanged = true;  
            reAddWindowLocked(wpos, win);  
        }  
    }  
    .....  
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

为了得到参数win所描述的WindowState对象的子WindowState对象在窗口堆栈中的起始位置，WindowManagerService类的成员函数reAddWindowToListInOrderLocked首先将参数win所描述的WindowState对象增加到窗口堆栈中，这是通过调用前面所分析的成员函数addWindowToListInOrderLocked来实现的，目的是为了获得它在窗口堆栈的位置。有了这个位置之后，WindowManagerService类的成员函数reAddWindowToListInOrderLocked就可以调用前面所分析的成员函数reAddWindowLocked来将WindowState对象及其所有子WindowState对象增加到窗口堆栈中去了，不过在调用之前，要先将参数win所描述的WindowState对象从窗口中堆栈删除。

1. 将一个WindowToken对象对应的所有WindowState对象及其子WindowState对象增加到窗口堆栈的指定位置上

将一个WindowToken对象对应的所有WindowState对象都增加到窗口堆栈中是通过调用WindowManagerService类的成员函数reAddAppWindowsLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub  
    implements Watchdog.Monitor {  
    .....  
  
    private final int reAddAppWindowsLocked(int index, WindowToken token) {  
        final int NW = token.windows.size();  
        for (int i=0; i<NW; i++) {  
            index = reAddWindowLocked(index, token.windows.get(i));  
        }  
        return index;  
    }  
    .....  
}
```

```
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

与参数token所描述的WindowToken对象所对应的WindowState对象保存在它的成员变量windows所描述的一个ArrayList中。通过遍历这个ArrayList，就可以将与参数token所描述的WindowToken对象所对应的WindowState对象及其子WindowState对象都增加到窗口堆栈的指定的起始位置上去，这是通过调用前面所分析的成员函数reAddWindowLocked来实现的。

参数index描述的便是最初指定的起始位置，每一次调用WindowManagerService类的成员函数reAddWindowLocked之后，它的值都便会被更新为下一个WindowState对象及其子WindowState对象要增加到窗口堆栈中的位置。

最后，WindowManagerService类的成员函数reAddAppWindowsLocked将与参数token所描述的WindowToken对象所对应的WindowState对象在窗口堆栈中的最高位置加1后的得到结果返回给调用者。

1. 将一个AppWindowToken对象所对应的WindowState对象及其子 WindowState对象移动到窗口堆栈的指定位置上

将一个AppWindowToken对象所对应的WindowState对象及其子 WindowState对象移动到窗口堆栈的指定位置上是通过调用WindowManagerService类的成员函数moveAppWindowsLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private void moveAppWindowsLocked(AppWindowToken wtoken, int tokenPos,
        boolean updateFocusAndLayout) {
        // First remove all of the windows from the list.
        tmpRemoveAppWindowsLocked(wtoken);

        // Where to start adding?
        int pos = findWindowOffsetLocked(tokenPos);

        // And now add them back at the correct place.
        pos = reAddAppWindowsLocked(pos, wtoken);

        if (updateFocusAndLayout) {
            if (!updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES)) {
                assignLayersLocked();
            }
            mLayoutNeeded = true;
            performLayoutAndPlaceSurfacesLocked();
        }
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

参数wtoken描述的是要移动其所对应的WindowState对象的一个AppWindowToken对象，而参数tokenPos描述的是该AppWindowToken对象在WindowManagerService服务内部的AppWindowToken列表中的新位置。

WindowManagerService类的成员函数moveAppWindowsLocked首先调用前面所分析的成员函数tmpRemoveAppWindowsLocked来移除所有与参数wtoken所描述的AppWindowToken对象所对应的WindowState对象，接着再调用也是前面所分析的成员函数findWindowOffsetLocked来获得与参数wtoken所描述的AppWindowToken对象所对应的WindowState对象在窗口堆栈中的起始位置。有了这个起始位置之后，就可以也是前面所分析的成员函数reAddAppWindowsLocked来将与参数wtoken所描述的AppWindowToken对象所对应的WindowState对象及其子WindowState对象移动到窗口堆栈上去了。

最后，如果参数updateFocusAndLayout的值等于true，那么WindowManagerService类的成员函数moveAppWindowsLocked还会更新系统当前获得焦点的窗口，以及重新计算系统中的所有窗口的Z轴位置以及重新布局系统中的所有窗口，这三个操作分别是通过调用WindowManagerService类的成员函数updateFocusedWindowLocked、assignLayersLocked和performLayoutAndPlaceSurfacesLocked来实现的。

1. 将一组AppWindowToken对象所对应的WindowState对象及其子 WindowState对象移动到窗口堆栈的指定位置上

将一组AppWindowToken对象所对应的WindowState对象及其子WindowState对象移动到窗口堆栈的指定位置上是通过调用WindowManagerService类的另外一个版本的成员函数moveAppWindowsLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private void moveAppWindowsLocked(List<IBinder> tokens, int tokenPos) {
        // First remove all of the windows from the list.
```

```

        final int N = tokens.size();
        int i;
        for (i=0; i<N; i++) {
            WindowToken token = mTokenMap.get(tokens.get(i));
            if (token != null) {
                tmpRemoveAppWindowsLocked(token);
            }
        }

        // Where to start adding?
        int pos = findWindowOffsetLocked(tokenPos);

        // And now add them back at the correct place.
        for (i=0; i<N; i++) {
            WindowToken token = mTokenMap.get(tokens.get(i));
            if (token != null) {
                pos = reAddAppWindowsLocked(pos, token);
            }
        }

        if (!updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES)) {
            assignLayersLocked();
        }
        mLayoutNeeded = true;
        performLayoutAndPlaceSurfacesLocked();

        //dump();
    }

    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

这个操作与前面分析的第14个操作是类似，区别只在于前者是批量地移动一组AppWindowToken对象所对应的WindowState对象及其子 WindowState对象，而后者是只移动一个AppWindowToken对象所对应的WindowState对象及其子WindowState对象，此外，前者总是会调用WindowManagerService类的成员函数updateFocusedWindowLocked、assignLayersLocked和performLayoutAndPlaceSurfacesLocked来更新系统当前获得焦点的窗口、以及重新计算每一个窗口的Z轴位置，并且对这些窗口进行重新布局。

至此，我们就分析完成WindowManagerService服务组织系统中的窗口的方式了。从分析的过程中，可以得到以下结论：

1. WindowManagerService服务维护有一个AppWindowToken堆栈和一个WindowState堆栈，它们与ActivityManagerService服务维护的Activity堆栈是有关相同的Z轴位置关系的
2. ActivityManagerService服务中的每一个ActivityRecord对象在WindowManagerService服务中都对应有一个AppWindowToken对象，而WindowManagerService服务中的每一个AppWindowToken对象都对应有一组WindowState对象。
3. 在WindowState堆栈中，AppWindowToken堆栈中的第i+1个AppWindowToken对象所对应的WindowState对象都位于第i个AppWindowToken对象所对应的WindowState对象的上面。
4. 一个WindowState对象可以附加在另外一个WindowState对象上面，此外，一个WindowState对象还可以有子WindowState对象，它们都是与同一个AppWindowToken对象或者WindowToken对象所对应的。
5. WindowManagerService服务有两个特殊的WindowToken，它们分别用来描述系统中的输入法窗口令牌和壁纸窗口令牌，其中，输入法窗口位于需要输入法的窗口的上面，而壁纸窗口位于需要壁纸的窗口的下面。

最后，我们可以将WindowManagerService服务中的AppWindowToken理解成一个Activity组件令牌，而将它所对应的WindowState对象理解成一个Activity窗口。有了这些概念之后，就为学习WindowManagerService服务的各种实现打下坚实的基础。在接下来的两篇文章中，我们就会在本文的基础上，继续分析WindowManagerService服务是如何管理系统中的输入法窗口和壁纸窗口的，敬请关注！

原文链接：<https://blog.csdn.net/Luoshengyang/article/details/8498908>

第四节 Android窗口管理服务WindowManagerService对输入法窗口（Input Method Window）的管理分析

在Android系统中，输入法窗口是一种特殊类型的窗口，它总是位于需要使用输入法的窗口的上面。也就是说，一旦WindowManagerService服务检测到焦点窗口需要使用输入法，那么它就会调整输入法窗口在窗口堆栈中的位置，使得输入法窗口位于在焦点窗口的上面，这样用户可以通过输入法窗口来录入字母或者文字。本文将详细分析WindowManagerService服务是如何管理系统中的输入法窗口的。

《Android系统源代码情景分析》一书正在进击的程序员网 (<http://0xcc0xcd.com>) 中连载，点击进入！

在Android系统中，除了输入法窗口之外，还有一种窗口称为输入法对话框，它们总是位于输入窗口的上面。Activity窗口、输入法窗口和输入法对话框的位置关系如图1所示：

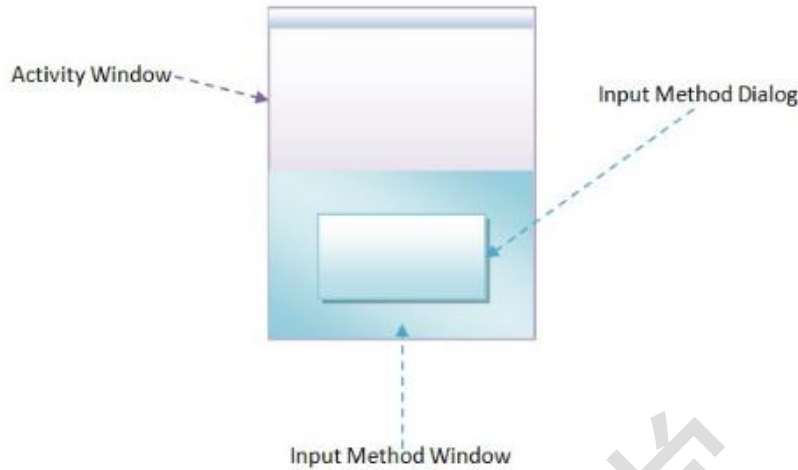


图1 Activity窗口、输入法窗口和输入法对话框的位置关系

在前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文中提到，WindowManagerService服务是使用堆栈来组织系统中的窗口的，因此，如果我们在窗口堆栈中观察Activity窗口、输入法窗口和输入法对话框，它们的位置关系就如图2所示：

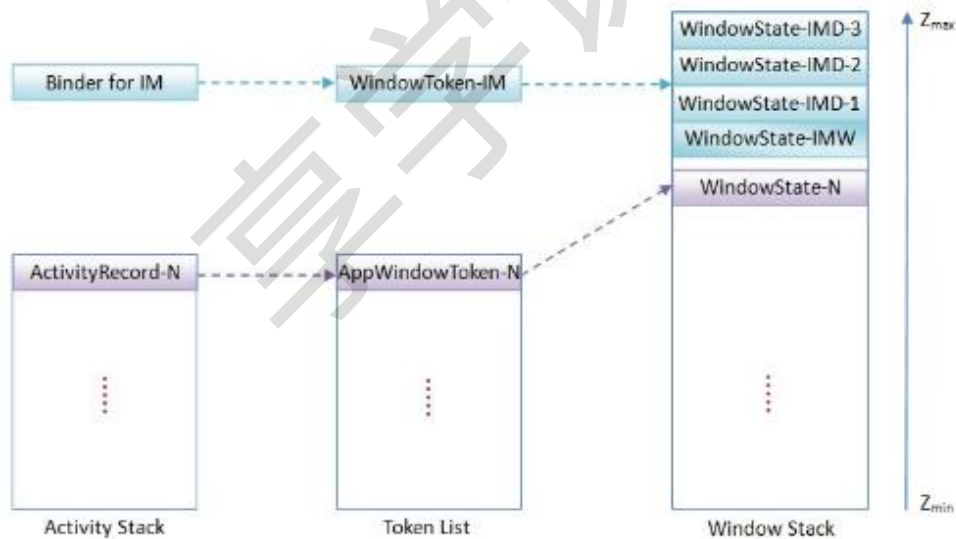


图2 Activity窗口、输入法窗口和输入法对话框在窗口堆栈中的位置关系

图2中的对象的关系如下所示：

1. 在ActivityManagerService服务内部的Activity组件堆栈顶端的ActivityRecord对象N描述的是系统当前激活的Activity组件
2. ActivityRecord对象N在WindowManagerService服务内部的窗口令牌列表顶端对应有一个AppWindowToken对象N
3. AppWindowToken对象N在WindowManagerService服务内部的窗口堆栈中对应有一个WindowState对象N，用来描述系统当前激活的Activity组件窗口。
4. WindowState对象N上面有一个WindowState对象IMW，用来描述系统中的输入法窗口。
5. WindowState对象IMW上面有三个WindowState对象IMD-1、IMD-2和IMD-3，它们用来描述系统中的输入法对话框。
6. 系统中的输入法窗口以及输入法对话框在WindowManagerService服务内部中对应的窗口

令牌是由WindowToken对象IM来描述的。

7. WindowToken对象IM在InputMethodManagerService服务中对应有一个Binder对象。

总的来说，就是图2描述了系统当前激活的Activity窗口上面显示输入法窗口，而输入法窗口上面又有一系列的输入法对话框的情景。WindowManagerService服务的职能之一就是要时刻关注系统中是否有窗口需要使用输入法。

WindowManagerService服务一旦发现有窗口需要使用输入法，那么就会调整输入法窗口以及输入法对话框在窗口堆栈中的位置，使得它们放置在需要使用输入法的窗口的上面。

接下来，我们就首先分析两个需要调整输入法窗口以及输入法对话框在窗口堆栈中的位置的情景，然后再分析它们是如何在窗口堆栈中进行调整的。

第一个需要调整输入法窗口以及输入法对话框在窗口堆栈中的位置的情景是增加一个窗口到WindowManagerService服务去的时候。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，增加一个窗口到WindowManagerService服务最终是通过调用WindowManagerService类的成员函数addWindow来实现的。

接下来我们就主要分析这个函数中与输入法窗口以及输入法对话框调整相关的逻辑，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    WindowState mInputMethodWindow = null;
    final ArrayList<WindowState> mInputMethodDialogs = new ArrayList<WindowState>();
    .....

    public int addWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int viewVisibility,
        Rect outContentInsets, InputChannel outInputChannel) {
        .....

        synchronized(mWindowMap) {
            .....

            WindowToken token = mTokenMap.get(attrs.token);
            if (token == null) {
                .....
                if (attrs.type == TYPE_INPUT_METHOD) {
                    .....
                    return WindowManagerImpl.ADD_BAD_APP_TOKEN;
                }
                .....
            }
            .....

            win = new WindowState(session, client, token,
                attachedWindow, attrs, viewVisibility);
            .....

            boolean imMayMove = true;

            if (attrs.type == TYPE_INPUT_METHOD) {
                mInputMethodWindow = win;
                addInputMethodWindowToListLocked(win);
                imMayMove = false;
            } else if (attrs.type == TYPE_INPUT_METHOD_DIALOG) {
                mInputMethodDialogs.add(win);
                addWindowToListInOrderLocked(win, true);
                adjustInputMethodDialogsLocked();
                imMayMove = false;
            }
            .....

            boolean focusChanged = false;
            if (win.canReceiveKeys()) {
                focusChanged = updateFocusedWindowLocked(UPDATE_FOCUS_WILL_ASSIGN_LAYERS);
                if (focusChanged) {
                    imMayMove = false;
                }
            }

            if (imMayMove) {
                moveInputMethodWindowsIfNeededLocked(false);
            }
        }
    }
}
```

```

        .....
    }

    .....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

如果当前增加到WindowManagerService服务来的是一个输入法窗口，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUT_METHOD，那么就要求与该输入法窗口所对应的类型为WindowToken的窗口令牌已经存在，否则的话，WindowManagerService类的成员函数addWindow就会直接返回一个错误码WindowManagerImpl.ADD_BAD_APP_TOKEN给调用者。这个类型为WindowToken的窗口令牌是InputMethodManagerService服务请求WindowManagerService服务创建的，即调用WindowManagerService类的成员函数addWindowToken来创建的，具体可以参考前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文。

如果当前增加到WindowManagerService服务来的是一个输入法窗口，那么就会将前面为它所创建的一个WindowState对象win保存在WindowManagerService类的成员变量mInputMethodWindow中，接着还会调用WindowManagerService类的成员函数addInputMethodWindowToListLocked来将该WindowState对象插入到窗口堆栈的合适位置去。

如果当前增加到WindowManagerService服务来的是一个输入法对话框，即参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUT_METHOD_DIALOG，那么就会将前面为它所创建的一个WindowState对象win添加到WindowManagerService类的成员变量mInputMethodDialogs所描述的一个ArrayList中去，并且先后调用WindowManagerService类的成员函数addWindowToListInOrderLocked和adjustInputMethodDialogsLocked来将该WindowState对象插入到窗口堆栈的合适位置去。

在上述两种情况中，由于用来描述输入法窗口或者输入法对话框的WindowState对象已经被插入到了窗口堆栈中的合适位置，因此，接下来就不再需要考虑移动该输入法窗口或者输入法对话框了，这时候变量imMayMove的值就会被设置为false。

另一方面，如果当前增加到WindowManagerService服务来的既不是一个输入法窗口，也不是一个输入法对话框，并且该窗口需要接收键盘事件，即前面所创建的WindowState对象win的成员函数canReceiveKeys的返回值为true，那么就可能会导致系统当前获得焦点的窗口发生变化，这时候就需要调用WindowManagerService类的成员函数updateFocusedWindowLocked来重新计算系统当前获得焦点的窗口。如果系统当前获得焦点的窗口发生了变化，那么WindowManagerService类的成员函数updateFocusedWindowLocked的返回值focusChanged就会等于true，同时系统的输入法窗口和输入法对话框在窗口堆栈中的位置也会得到调整，即它们会位于系统当前获得焦点的窗口的上面，因此，这时候变量imMayMove的值也会被设置为false，表示接下来不再需要考虑移动系统中的输入法窗口或者输入法对话框在窗口堆栈中的位置。

最后，如果变量imMayMove的值保持为初始值，即保持为true，那么就说明当前增加的窗口可能会引发系统的输入法窗口和输入法对话框在窗口堆栈中的位置发生变化，因此，这时候就需要调用WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked来作检测，并且在发生变化的情况下，将系统的输入法窗口和输入法对话框移动到窗口堆栈的合适位置上去。

从上面的分析就可以知道，在增加一个窗口的过程中，可能需要调用WindowManagerService类的成员函数addInputMethodWindowToListLocked、addWindowToListInOrderLocked、adjustInputMethodDialogsLocked和moveInputMethodWindowsIfNeededLocked来移动系统的输入法窗口和输入法对话框，其中，WindowManagerService类的成员函数addWindowToListInOrderLocked在前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文已经分析过了，本文只要关注其余三个成员函数的实现。

第二个需要调整输入法窗口以及输入法对话框在窗口堆栈中的位置的情景是一个应用程序进程请求WindowManagerService服务重新布局一个窗口的时候。从前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文可以知道，应用程序进程请求WindowManagerService服务重新布局一个窗口最终是通过调用WindowManagerService类的成员函数relayWindow来实现的。接下来我们就主要分析这个函数中与输入法窗口以及输入法对话框调整相关的逻辑，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int relayWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int requestedWidth,
        int requestedHeight, int viewVisibility, boolean insetsPending,
        Rect outFrame, Rect outContentInsets, Rect outVisibleInsets,
        Configuration outConfig, Surface outSurface) {
        .....

        synchronized(mWindowMap) {
            WindowState win = windowForClientLocked(session, client, false);
            .....

            int attrChanges = 0;

```

```

int flagChanges = 0;
if (attrs != null) {
    flagChanges = win.mAttrs.flags ^ attrs.flags;
    attrChanges = win.mAttrs.copyFrom(attrs);
}
.....

boolean imMayMove = (flagChanges & (
    WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM |
    WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE)) != 0;

boolean focusMayChange = win.mViewVisibility != viewVisibility
    || ((flagChanges & WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE) != 0)
    || (!win.mRelayoutCalled);
.....

if (viewVisibility == View.VISIBLE &&
    (win.mAppToken == null || !win.mAppToken.clientHidden)) {
displayed = !win.isVisibleLw();
.....

if ((attrChanges & WindowManager.LayoutParams.FORMAT_CHANGED) != 0) {
    // To change the format, we need to re-build the surface.
    win.destroySurfaceLocked();
    displayed = true;
}
.....

if (win.mAttrs.type == TYPE_INPUT_METHOD
    && mInputMethodWindow == null) {
    mInputMethodWindow = win;
    imMayMove = true;
}

if (displayed) {
    focusMayChange = true;
}

.....
} else {
    .....

    if (win.mSurface != null) {
        .....
        // If we are not currently running the exit animation, we
        // need to see about starting one.
        if (!win.mExiting || win.mSurfacePendingDestroy) {
            .....

            if (!win.mSurfacePendingDestroy && win.isWinVisibleLw() &&
                applyAnimationLocked(win, transit, false)) {
                focusMayChange = true;
                win.mExiting = true;
            } else if (win.isAnimating()) {
                // Currently in a hide animation... turn this into
                // an exit.
                win.mExiting = true;
            } else if (win == mWallpaperTarget) {
                // If the wallpaper is currently behind this
                // window, we need to change both of them inside
                // of a transaction to avoid artifacts.
                win.mExiting = true;
                win.mAnimating = true;
            } else {
                if (mInputMethodWindow == win) {
                    mInputMethodWindow = null;
                }
                win.destroySurfaceLocked();
            }
        }
    }
}
}

```



```

    }

    .....
}

if (focusMayChange) {
    .....
    if (updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES)) {
        imMayMove = false;
    }
    .....
}

// updateFocusedWindowLocked() already assigned layers so we only need to
// reassign them at this point if the IM window state gets shuffled
boolean assignLayers = false;

if (imMayMove) {
    if (moveInputMethodWindowsIfNeededLocked(false) || displayed) {
        // Little hack here -- we -should- be able to rely on the
        // function to return true if the IME has moved and needs
        // its layer recomputed. However, if the IME was hidden
        // and isn't actually moved in the list, its layer may be
        // out of data so we make sure to recompute it.
        assignLayers = true;
    }
}
}
.....

if (assignLayers) {
    assignLayersLocked();
}

.....
}

.....

return (inTouchMode ? WindowManagerImpl.RELAYOUT_IN_TOUCH_MODE : 0)
    | (displayed ? WindowManagerImpl.RELAYOUT_FIRST_TIME : 0);
}

.....
}
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

应用程序进程在请求WindowManagerService服务重新布局一个窗口的时候，这个窗口的一些布局参数可能会发生变化，而这些变化可能会同时引发系统的输入法窗口以及输入法对话框在窗口堆栈中的位置发生变化。如果系统的输入法窗口以及输入法对话框在窗口堆栈中的位置发生了变化，那么就需要调整它们在窗口堆栈中的位置。

WindowManagerService类的成员函数relayoutWindow首先调用根据参数session和client来调用另外一个成员函数windowForClientLocked，以便可以获得用来描述要重新布局的窗口的一个WindowState对象win。

WindowState对象win的成员变量mAttrs指向的是一个WindowManager.LayoutParams对象，该WindowManager.LayoutParams对象的成员变量flags描述的是窗口上一次所设置的布局属性标志位，而参数attrs所描述的一个WindowManager.LayoutParams对象的成员变量flags描述的是窗口当前被设置的布局属性标志位。WindowManagerService类的成员函数relayoutWindow通过对这两个标志位执行一个异或操作，就可以知道窗口的哪些布局属性标志位发生了变化，这些变化就记录在变量flagChanges中。

WindowManagerService类的成员函数relayoutWindow在对WindowState对象win所描述的窗口进行布局之前，还要将参数attrs指的是一个WindowManager.LayoutParams对象的内容拷贝到 WindowState对象win的成员变量mAttrs指向的是一个WindowManager.LayoutParams对象中去。在拷贝的过程中，如果发现这两个WindowManager.LayoutParams对象所描述的窗口布局属性有发生变化，那么这些变化就会记录在变量attrChanges中。

在窗口的布局属性标志中，位WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE表示窗口是否可以获得焦点，另外一个位WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM是用来反转WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位的作用的。一个窗口是否可以获得焦点意味着它是否需要与输入法窗口交互，即如果一个窗口是可以获得焦点的，那么就意味着它需要与输入法窗口交互，否则就不需要。当一个窗口的WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位等于1，那么就表示窗口不可以获得焦点，即不需要与输入法窗口交互，但是如果该窗口的WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM位也等于1，那么就表示窗口仍然是需要与输入法窗口交互的。另一方面，如果一个窗口的WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM位等于1，但是

该窗口的WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位等于0，那么就表示窗口仍然是不可以与输入法窗口交互的。因此，当前面得到的变量flagChanges的WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位或者WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM位发生了变化时，都意味着对WindowState对象win所描述的窗口进行重新布局会影响系统中的输入法窗口以及输入法对话框，即该窗口可能会由需要显示输入法窗口以及输入法对话框，到不需要显示输入法窗口以及输入法对话框，反之亦然。最后得到的变量imMayMove的值等于true就表示要移动系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置。

一个窗口由不可获得焦点到可以获得焦点，或者由可获得焦点到不可获得焦点，即窗口布局属性标志中的WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位发生了变化，那么就意味着要重新计算系统当前获得焦点的窗口。从前面分析增加窗口到WindowManagerService服务的情景可以知道，当系统当前获得焦点的窗口发生变化时，也意味着需要系统中的移动输入法窗口以及输入法对话框在窗口堆栈中的位置。除了窗口布局属性标志中的WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE位变化会引发系统当前获得焦点的窗口发生变化之外，还有另外两个因素会引发系统当前获得焦点的窗口发生变化。第一个因素是窗口的可见性发生变化。WindowState对象win的成员变量mViewVisibility描述的是窗口上一次布局时的可见性，而参数viewVisibility描述的是窗口当前的可见性，当它们的值不相等时，就意味着窗口的可见性发生了变化。第二个因素是窗口是第一次被应用程序进程请求WindowManagerService服务布局，这时候WindowState对象win的成员变量mRelayoutCalled的值就会等于false。最后得到的变量focusMayChange等于true，就表示需要重新计算系统当前获得焦点的窗口。

WindowState对象win所描述的窗口在此次重新布局中是否会引起移动系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置，还取决于它的可见性以及它的绘图表面属性等信息，接下来我们就按照 WindowState对象win所描述的窗口当前是可见还是不可见来分别分析。

我们首先分析WindowState对象win所描述的窗口在此次重新布局中是可见的情景，即参数viewVisibility的值等于View.VISIBLE。注意，如果WindowState对象win所描述的是一个Activity窗口，而该Activity组件是不可见的，那么即使参数viewVisibility的值等于View.VISIBLE，那么WindowState对象win所描述的窗口在此次重新布局中也是认为不可见的。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，当WindowState对象win的成员变量mAppToken的值不等于null时，那么该WindowState对象win描述的是一个Activity窗口，而当该成员变量所指向的一个AppWindowToken对象的成员变量clientHidden的值等于false时，就表示对应的Activity组件是可见的。

WindowState对象win所描述的窗口在上一次布局时的可见性可以调用它的成员函数isVisibleLw来获得。如果WindowState对象win所描述的窗口在上一次布局时是不可见的，那么现在就需要将它设置为可见的，即要将它显示出来，这时候变量displayed的值就会等于true。另一方面，如果WindowState对象win所描述的窗口的绘图表面的像素格式发生了变化，即变量attrChanges的WindowManager.LayoutParams.FORMAT_CHANGED位等于1，那么这时候就需要调用WindowState对象win的成员函数destroySurfaceLocked来销毁它所描述的窗口的绘图表面，以便接下来可以为它重新创建一个新的绘图表面，这时候也会将变量displayed的值设置为true，表示接下来是要显示WindowState对象win所描述的窗口的。如果最终得到的变量displayed的值设置为true，那么就相当于说明WindowState对象win所描述的窗口经历一个由不可见到可见的状态变化，因此就可能会导致系统当前获得焦点的窗口发生变化，这时候就会将变量focusMayChange的值设置为true。

如果WindowState对象win描述的是一个输入法窗口，即它的成员变量mAttrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_INPUT_METHOD，并且系统中的输入法窗口尚未设置，即WindowManagerService类的成员变量mInputMethodWindow的值等于null，那么就说明接下来要显示的其实是输入法窗口，这情况会导致需要移动系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置，因此，这时候除了需要将WindowState对象win保存在WindowManagerService类的成员变量mInputMethodWindow之外，还需要将变量imMayMove的值设置为true。

我们接下来再分析WindowState对象win所描述的窗口在此次重新布局中是不可见的情景。一个窗口变得不可见了，就意味着可能要销毁它的绘图表面，取决于它的绘图表面是否存在，以及它的退出动画是否已经显示结束。WindowState对象win所描述的窗口的绘图表面保存在它的成员变量mSurface中，因此，当WindowState对象win的成员变量mSurface不等于null的时候，就意味着可能会销毁它所描述的绘图表面。

如果WindowState对象win的成员变量mExiting等于false时，那么就说明该WindowState对象win所描述的窗口的退出动画可能尚未开始，也可能已经结束。另一方面，如果WindowState对象win的成员变量mSurfacePendingDestroy的值等于true，那么就说明该WindowState对象win所描述的窗口的绘图表面正在等待销毁。这两种情况都需要进一步确定接下来是要开始WindowState对象win所描述的窗口的退出动画，还是要销毁WindowState对象win所描述的窗口的绘图表面。

如果WindowState对象win的成员变量mSurfacePendingDestroy的值等于false，那么同时也意味着它所描述的窗口还未开始显示退出动画，因而它的绘图表面就没有进入正在等待销毁的状态。在这种情况下，如果WindowState对象win所描述的窗口是可见的，即它的成员函数isVisibleLw的返回值等于true，那么就意味着要开始该窗口的退出动画了，这是通过调用WindowManagerService类的成员函数applyAnimationLocked来实现的。WindowState对象win描述的窗口开始退出动画之后，就意味着要重新计算系统当前获得焦点的窗口，因此，这时候就会将变量focusMayChange的值设置为true，同时还会将WindowState对象win的成员变量mExiting的值设置为true，表示它描述的窗口正在退出的过程中。

如果WindowState对象win所描述的窗口正处于退出动画的过程中，即它的成员函数isAnimating的返回值等于true，那么这时候需要确保WindowState对象win的成员变量mExiting的值为true。

如果WindowState对象win所描述的窗口已经结束退出动画，但是它仍然是壁纸窗口的目标，即WindowManagerService类的成员变量mWallpaperTarget的值不等于null，并且它的值就等于WindowState对象win，那么这时候就需要等待壁纸窗口也退出之后，才销毁WindowState对象win所描述的窗口，因此，这时候就需要将WindowState对象win的成员变量mExiting和mAnimating的值设置为true，即假装它所描述的窗口还处于正在退出的过程，这样做是为了等待壁纸窗口退出完成。

如果WindowState对象win所描述的窗口已经结束退出动画，并且它不是壁纸窗口的目标，那么这时候就需要调用它的

成员函数destroySurfaceLocked来销毁它的绘图表面了。在销毁WindowState对象win所描述的窗口之前，还会判断它是否就是系统当前的输入法窗口，即WindowManagerService类的成员变量mInputMethodWindow的值是否等于win。如果等于的话，那么就说明系统当前的输入法窗口被销毁了，因此，就需要将WindowManagerService类的成员变量mInputMethodWindow的值设置为null。

经过上面的一系列操作之后，如果最终得到的变量focusMayChange的值等于true，那么就说明需要重新计算系统当前获得焦点的窗口了，这是通过调用WindowManagerService类的成员函数updateFocusedWindowLocked来实现的。一旦WindowManagerService类的成员函数updateFocusedWindowLocked的返回值为true，那么就说明当前获得焦点的窗口发生了变化，并且系统中的输入法窗口以及输入法对话框也移动到窗口堆栈中的正确位置了，因此，这时候就会将变量imMayMove的值设置为false。

经过上面的一系列操作之后，如果最终得到的变量imMayMove的值等于true，那么就说明有可能需要移动系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置，这是通过调用WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked来实现的。一旦系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置发生了移动，那么WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked的返回值就等于true，这时候就需要将变量assignLayers的值设置为true，表示要重新计算系统中的窗口的Z轴位置，以便可以同步到SurfaceFlinger服务中去。注意，如果系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置没有发生变化，但是前面得到的变量displayed的值等于true，那么也是需要将变量assignLayers的值设置为true的，因为这个变量displayed的值等于true意味着WindowState对象win所描述的窗口经历了从不可见到可见的状态变化，因此也需要重新计算系统中的窗口的Z轴位置。

经过上面的一系列操作之后，如果最终得到的变量assignLayers的值等于true，那么就需要调用WindowManagerService类的成员函数assignLayersLocked来执行重新计算系统中的窗口的Z轴位置的操作了。在后面的文章中，我们再详细分析WindowManagerService服务是如何计算系统中的窗口的Z轴位置的。

从上面的分析就可以知道，在布局一个窗口的过程中，可能需要调用WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked来移动系统的输入法窗口和输入法对话框。再结合前面增加窗口的情景，我们就可以知道，在WindowManagerService类中，与输入法窗口以及输入法对话框相关的成员函数有addInputMethodWindowToListLocked、adjustInputMethodDialogsLocked和moveInputMethodWindowsIfNeededLocked，它们的作用如下所示：

A. 成员函数addInputMethodWindowToListLocked用来将输入法窗口插入到窗口堆栈的合适位置，即插入到需要显示输入法窗口的窗口的上面。

B. 成员函数adjustInputMethodDialogsLocked用来移动输入法对话框到窗口堆栈的合适位置，即移动到输入法窗口的上面。

C. 成员函数moveInputMethodWindowsIfNeededLocked用来检查是否需要移动输入法窗口以及输入法对话框。如果需要的话，那么就将它们移动到窗口堆栈的合适位置去，即将输入法窗口移动到需要显示输入法窗口的窗口的上面，而将输入法对话框移动到输入法窗口的上面。

在分析这三个成员函数的实现之前，我们首先分析WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked和moveInputMethodDialogsLocked，它们是两个基本的操作，其中：

D. 成员函数findDesiredInputMethodWindowIndexLocked用来查找输入法窗口在窗口堆栈的正确位置，这个位置刚好就是在需要显示输入法窗口的窗口在窗口堆栈中的上一个位置。

E. 成员函数moveInputMethodDialogsLocked用来将移动输入法对话框移动到输入法窗口的上面去。

接下来我们开始分析上述五个函数的实现。

1. 计算输入法窗口在窗口堆栈中的位置

输入法窗口在窗口堆栈中的位置是通过调用WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked来获得的，它首先找到需要显示输入法的窗口在窗口堆栈中的位置，然后再将这个位置加1，就可以得到输入法窗口在窗口堆栈中的位置。

WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中，它的实现比较长，我们分段来阅读：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    int findDesiredInputMethodWindowIndexLocked(boolean willMove) {
        final ArrayList<WindowState> localmWindows = mWindows;
        final int N = localmWindows.size();
        WindowState w = null;
        int i = N;
        while (i > 0) {
            i--;
            w = localmWindows.get(i);
            .....

            if (canBeImeTarget(w)) {
                .....

                // Yet more tricksyness! If this window is a "starting"
```

```

// window, we do actually want to be on top of it, but
// it is not -really- where input will go. So if the caller
// is not actually looking to move the IME, look down below
// for a real window to target...
if (!willMove
    && w.mAttrs.type == WindowManager.LayoutParams.TYPE_APPLICATION_STARTING
    && i > 0) {
    WindowState wb = localmWindows.get(i-1);
    while (i > 1 && wb.mAppToken == w.mAppToken && !canBeImeTarget(wb)) {
        i--;
        wb = localmWindows.get(i-1);
    }
    if (wb.mAppToken == w.mAppToken && canBeImeTarget(wb)) {
        i--;
        w = wb;
    }
}
break;
}
}
}

```

```
mUpcomingInputMethodTarget = w;
```

这段代码从上到下遍历WindowManagerService服务内部的窗口堆栈，即WindowManagerService类的成员变量mWindows所描述的一个ArrayList。如果发现有一个窗口是可见的，并且需要显示输入法窗口，那么整个查找过程就会结束。检查一个窗口是否可见以及需要显示输入法窗口是通过调用WindowManagerService类的成员函数canBeImeTarget来实现的。最后得到的需要显示输入法的窗口就使用WindowState对象w中，这个WindowState对象w接下来还会保存在WindowManagerService类的成员变量mUpcomingInputMethodTarget中，表示它即将要成为输入法窗口的目标窗口。参数willMove表示调用者计算输入法窗口在窗口堆栈中的位置的目的。如果它的值等于true，那么就说明调用者获得了输入法窗口在窗口堆栈中的位置之后，接下来就会将输入法窗口移动到需要显示输入法窗口的窗口的上面去，否则的话，就说明调用者只是为了知道输入法窗口在窗口堆栈中的位置，而不打算移动输入法窗口。

在从上到下查找需要显示输入法的窗口的过程中，如果找到一个WindowState对象w，它所描述的窗口需要显示输入法窗口，但是这个窗口其实是一个Activity窗口的启动窗口，即该WindowState对象w的成员变量mAttrs所描述的一个WindowManager.LayoutParams对象的成员变量type的值等于WindowManager.LayoutParams.TYPE_APPLICATION_STARTING，那么由于调用WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked的目的不是用来移动输入法窗口，而是用来查找输入法窗口在窗口堆栈中的确切位置，因此就不能前面所找到的启动窗口看作是一个需要输入法的窗口，因为这个启动窗口只是Activity窗口在显示过程中出现的一个临时窗口。在这种情况下，这段代码就会继续沿着窗口堆栈往下查找另外一个窗口，该窗口一方面是显示需要输入法的窗口的，另一方面要与前面所找到的启动窗口对应的是同一个窗口令牌的。如果能找到这样的一个窗口，那么就会将用来描述它的一个WindowState对象wb保存在变量w中。如果找不到这样的一个窗口，那么这段代码就会继续沿着窗口堆栈往下查找另外一个需要显示输入法的窗口。

我们继续往下阅读代码：

```

if (willMove && w != null) {
    final WindowState curTarget = mInputMethodTarget;
    if (curTarget != null && curTarget.mAppToken != null) {

        // Now some fun for dealing with window animations that
        // modify the Z order. We need to look at all windows below
        // the current target that are in this app, finding the highest
        // visible one in layering.
        AppWindowToken token = curTarget.mAppToken;
        WindowState highestTarget = null;
        int highestPos = 0;
        if (token.animating || token.animation != null) {
            int pos = 0;
            pos = localmWindows.indexOf(curTarget);
            while (pos >= 0) {
                WindowState win = localmWindows.get(pos);
                if (win.mAppToken != token) {
                    break;
                }
                if (!win.mRemoved) {
                    if (highestTarget == null || win.mAnimLayer >
                        highestTarget.mAnimLayer) {
                        highestTarget = win;
                        highestPos = pos;
                    }
                }
            }
            pos--;
        }
    }
}

```

```

    }
}

if (highestTarget != null) {
    .....

    if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
        // If we are currently setting up for an animation,
        // hold everything until we can find out what will happen.
        mInputMethodTargetWaitingAnim = true;
        mInputMethodTarget = highestTarget;
        return highestPos + 1;
    } else if (highestTarget.isAnimating() &&
        highestTarget.mAnimLayer > w.mAnimLayer) {
        // If the window we are currently targeting is involved
        // with an animation, and it is on top of the next target
        // we will be over, then hold off on moving until
        // that is done.
        mInputMethodTarget = highestTarget;
        return highestPos + 1;
    }
}
}
}
}

```

这段代码用来处理一种特殊情况，即参数willMove的值等于true，并且前面找到了一个需要显示输入法的窗口w，但是当前输入法窗口已经存在一个目标窗口，并且该目标窗口正在切换的过程中。在这种情况下，调用WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked的函数就需要等到当前输入法窗口的目标窗口的切换过程结束之后，再将输入法窗口移动到窗口w的上面去，换句话说，就是要保持输入法窗口在它当前的目标窗口的上面，直到它当前的目标窗口的切换过程结束为止。这样WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked就需要找到当前输入法窗口的目标窗口在窗口堆栈中的位置，然后再将该位置加1后返回给调用者。当WindowManagerService类的成员变量mInputMethodTarget的值不等于null，并且它描述的是一个Activity窗口时，即它的成员变量mAppToken的值不等于null时，那么就说明当前输入法窗口已经存在一个目标窗口，而这个目标窗口就是使用WindowManagerService类的成员变量mInputMethodTarget所指向的一个WindowState对象来描述的。接下来这段代码就检查该目标窗口是否正在切换的过程中，即是否正在显示切换动画。如果是的话，那么WindowState对象curTarget的成员变量animating的值就会等于true，或者另外一个成员变量animation的值不等于null，这时候就需要在与该目标窗口所对应的窗口令牌token所描述的一组窗口中，找到一个Z轴位置最大的并且不是已经被移除的窗口。WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked的调用者最后就是需要将输入法窗口移动到这个Z轴位置最大的并且不是已经被移除的窗口的上面的。

一个窗口的Z轴位置是记录在用描述它的一个WindowState对象的成员变量mAnimLayer中的，而它是否是已经被移除是记录在这个WindowState对象的成员变量mRemoved中的，因此，如果在窗口令牌token所描述的一组WindowState对象中，能找到一个WindowState对象，它的成员变量mAnimLayer的值最大，并且它的成员变量mRemoved不等于true，那么这段代码就会将它保存在变量highestTarget中，并且将它描述的窗口在窗口堆栈中的位置保存在变量highestPos中。

经过前面的一系列计算之后，如果变量highestTarget的值不等于null，那么就说明我们碰到前面所说的特殊的情况，这时候又要分为两种情况来讨论。

第一种情况是当前输入法窗口的目标窗口即将要进入到切换过程，但是这个切换过程尚未开始，即WindowManagerService类的成员变量mNextAppTransition的值不等于WindowManagerPolicy.TRANSIT_UNSET。这时候就需要将WindowManagerService类的成员变量mInputMethodTargetWaitingAnim的值设置为true，表示当前输入法窗口的目标窗口正在等待进入切换动画中，并且需要将WindowManagerService类的成员变量mInputMethodTarget修正为变量highestTarget所描述的一个WindowState对象，因为这个WindowState对象才是真正用来描述当前输入法窗口的目标窗口的。

第二种情况是当前输入法窗口的目标窗口已经处于切换的过程了，即变量highestTarget所描述的一个WindowState对象的成员函数isAnimating的返回值为true，并且该目标窗口的Z轴位置大于前面所找到的需要显示输入法的窗口的窗口的Z轴，即变量highestTarget所描述的一个WindowState对象的成员变量mAnimLayer的值大于变量w所描述的一个WindowState对象的成员变量mAnimLayer的值。这时候就需要将WindowState对象highestTarget所描述的窗口维持为当前输入法窗口的目标窗口，即将WindowManagerService类的成员变量mInputMethodTarget设置为变量highestTarget，直到WindowState对象highestTarget所描述的窗口的切换过程结束为止。

上述两种情况最后都需要将WindowState对象highestTarget所描述的窗口在窗口堆栈中的位置highestPos加1，然后再返回给调用者，以便调用者接下来可以输入法窗口移动到窗口堆栈的第(highestPos+1)个位置上。

如果我们没有碰到前面所说的特殊的情况，那么WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked就会继续往下执行：

```

if (w != null) {
    if (willMove) {
        .....
        mInputMethodTarget = w;
        if (w.mAppToken != null) {
            setInputMethodAnimLayerAdjustment(w.mAppToken.animLayerAdjustment);

```

```

    } else {
        setInputMethodAnimLayerAdjustment(0);
    }
}
return i+1;
}

```

如果变量w的值不等于null，那么就说明WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked在前面找到了一个需要显示输入法窗口的窗口。这个窗口是使用WindowState对象w来描述的，并且它在窗口堆栈中的位置记录在变量i中。这时候WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked就会执行以下三个操作： A. 将WindowState对象w保存在WindowManagerService类的成员变量mInputMethodTarget中，以便WindowManagerService服务可以知道当前输入法窗口的目标窗口是什么。

B. 检查WindowState对象w描述的窗口是否是Activity窗口，即检查WindowState对象w的成员变量mAppToken的值是否不等于null。如果WindowState对象w描述的窗口是Activity窗口的话，那么就需要根据WindowState对象w的成员变量mAppToken所描述的一个AppWindowToken对象的成员变量animLayerAdjustment来调整系统中的输入法窗口以及输入法对话框的Z轴位置，即在系统中的输入法窗口以及输入法对话框的现有Z轴位置的基础上再增加一个调整量，这个调整量就保存在WindowState对象w的成员变量mAppToken所描述的一个AppWindowToken对象的成员变量animLayerAdjustment中。这个调整的过程是通过调用WindowManagerService类的成员函数setInputMethodAnimLayerAdjustment来实现的。如果WindowState对象w描述的窗口不是Activity窗口，那么就不需要调整系统中的输入法窗口以及输入法对话框的Z轴位置，但是仍然需要调用WindowManagerService类的成员函数setInputMethodAnimLayerAdjustment来将系统中的输入法窗口以及输入法对话框的Z轴位置调整量设置为0，即将WindowManagerService类的成员变量mInputMethodAnimLayerAdjustment的值设置为0。

C. 将变量i的值加1之后返回给调用者，以便调用者可以将系统中的输入法窗口移动到窗口堆栈中的第(i+1)个位置上。

如果变量w的值等于null，那么就说明WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked在前面没有找到需要显示输入法窗口的窗口，我们继续往下阅读它的代码，以便可以了解它是如何处理这种情况的：

```

if (willMove) {
    .....
    mInputMethodTarget = null;
    setInputMethodAnimLayerAdjustment(0);
}
return -1;
}
.....
}

```

WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked对在前面没有找到需要显示输入法窗口的窗口的情况的处理很简单。它判断参数willMove的值是否等于true。如果等于true的话，那么就会将WindowManagerService类的成员变量mInputMethodTarget的值设置为null，并且调用WindowManagerService类的成员函数setInputMethodAnimLayerAdjustment来将系统中的输入法窗口以及输入法对话框的Z轴位置调整量设置为0。这实际上是用来通知WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked的调用者，系统当前没有需要显示输入法窗口的窗口。最后，WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked返回一个-1值给调用者，也是表明系统当前没有需要显示输入法窗口的窗口。

2. 移动输入法对话框移动到输入法窗口的上面

系统中的输入法对话框是需要位于输入法窗口的上面的，因此，我们就需要有一个函数来将输入法对话框移动到输入法窗口的上面去。这个函数就是WindowManagerService类的成员函数moveInputMethodDialogsLocked，它的实现如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    void moveInputMethodDialogsLocked(int pos) {
        ArrayList<WindowState> dialogs = mInputMethodDialogs;

        final int N = dialogs.size();
        .....
        for (int i=0; i<N; i++) {
            pos = tmpRemoveWindowLocked(pos, dialogs.get(i));
        }
        .....

        if (pos >= 0) {
            final AppWindowToken targetAppToken = mInputMethodTarget.mAppToken;
            if (pos < mWindows.size()) {

```

```

        WindowState wp = mWindows.get(pos);
        if (wp == mInputMethodWindow) {
            pos++;
        }
    }
    .....
    for (int i=0; i<N; i++) {
        WindowState win = dialogs.get(i);
        win.mTargetAppToken = targetAppToken;
        pos = reAddWindowLocked(pos, win);
    }
    .....
    return;
}
for (int i=0; i<N; i++) {
    WindowState win = dialogs.get(i);
    win.mTargetAppToken = null;
    reAddWindowToListInOrderLocked(win);
    .....
}
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

在调用WindowManagerService类的成员函数moveInputMethodDialogsLocked之前，必须要保证系统中的输入法窗口已经被移动到窗口堆栈的正确位置，即已经被移动到需要显示输入法窗口的窗口的上面。这时候参数pos描述的或者是输入法窗口在窗口堆栈中的位置，或者是输入法窗口在窗口堆栈的位置的上一个位置，即输入法对话框在窗口堆栈中的起始位置。参数pos的值还可以小于0，这时候就表示系统当前没有需要显示输入法窗口的窗口。

在移动输入法对话框到输入法窗口的上面之前，首先要将输入法对话框从窗口堆栈中移除，以便接下来可以重新将它们插入到窗口堆栈中。系统中的输入法对话框都保存在WindowManagerService类的成员变量mInputMethodDialogs所描述的一个ArrayList中，通过调用WindowManagerService类的成员函数来tmpRemoveWindowLocked来移除保存在这个ArrayList中的每一个WindowState对象，就可以将系统中的输入法对话框从窗口堆栈中移除。注意，将一个WindowState对象从窗口堆栈中移除之后，可能会影响参数pos的值。例如，如果参数pos的值大于被移除的WindowState对象原来在窗口堆栈中的位置值，那么在该WindowState对象被移除之后，参数pos的值就要相应地减少1，这样它才能正确地反映输入法窗口在窗口堆栈中的位置，或者输入法对话框在窗口堆栈中的起始位置。WindowManagerService类的成员函数来tmpRemoveWindowLocked在将一个WindowState对象从窗口堆栈中移除的过程中，会正确处理好参数pos的值，这一点可以参考前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文。

接下来，我们就分为两种情况来分析输入法对话框在窗口是如何移动到输入法窗口的上面去的。

第一种情况是参数pos的值大于等于0，这表明系统当前存在一个需要显示输入法窗口的窗口，这个窗口是通过WindowManagerService类的成员变量mInputMethodTarget所指向的一个WindowState对象来描述的。

前面提到，参数pos描述的或者是输入法窗口在窗口堆栈中的位置，或者是输入法对话框在窗口堆栈中的起始位置，我们首先要将它统一描述为输入法对话框在窗口堆栈中的起始位置。这时候就需要检查保存在窗口堆栈的第pos个位置的WindowState对象wp，是否就是WindowManagerService类的成员变量mInputMethodWindow所指向的那个WindowState对象。如果是的话，那么就说明参数pos描述的或者是输入法窗口在窗口堆栈中的位置，这时候将它的值增加1，就可以让它表示为输入法对话框在窗口堆栈中的起始位置。

得到了输入法对话框在窗口堆栈中的起始位置pos之后，接下来只需要调用WindowManagerService类的成员函数reAddWindowLocked来依次地将保存在WindowManagerService类的成员变量mInputMethodDialogs所描述的一个ArrayList中的第i个WindowState对象保存在窗口堆栈中的第(pos+i)个位置即可，这样就可以将输入法对话框都移动到输入法窗口的上面去了。

注意，在移动的过程中，用来描述每一个输入法对话框的每一个WindowState对象的成员变量mTargetAppToken的值设置为WindowManagerService类的成员变量mInputMethodTarget所描述的一个WindowState对象的成员变量mAppToken的值，以便可以将输入法对话框和输入法窗口的目标窗口设置为同一个窗口。

第二种情况是参数pos的值小于0，这表明系统当前不存在一个需要显示输入法窗口的窗口。这时候就需要根据输入法窗口自身的属性来将它们移动到窗口堆栈的合适的位置上去，这是通过调用WindowManagerService类的成员函数reAddWindowToListInOrderLocked来实现的。WindowManagerService类的成员函数reAddWindowToListInOrderLocked的实现可以参考前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文，这里不再详细。

注意，在移动的过程中，用来描述每一个输入法对话框的每一个WindowState对象的成员变量mTargetAppToken的值会被设置为null，这是因为系统当前不存在一个需要显示输入法窗口的窗口，即这时候每一个输入法对话框都没有目标窗口。

理解了WindowManagerService类的成员函数findDesiredInputMethodWindowIndexLocked和moveInputMethodDialogsLocked的实现之后，对WindowManagerService类的另外三个成员函数addInputMethodWindowToListLocked、adjustInputMethodDialogsLocked和moveInputMethodWindowsIfNeededLocked

的实现就很有帮助，接下来我们就继续分析这三个成员函数的实现。

1. 插入输入法窗口到需要显示输入法窗口的窗口上面

插入输入法窗口到窗口堆栈的合适位置，使得它位于需要显示输入法窗口的窗口上面，这是通过调用 WindowManagerService 类的成员函数 addInputMethodWindowToListLocked 来实现的，它的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    void addInputMethodWindowToListLocked(WindowState win) {
        int pos = findDesiredInputMethodWindowIndexLocked(true);
        if (pos >= 0) {
            win.mTargetAppToken = mInputMethodTarget.mAppToken;
            .....
            mWindows.add(pos, win);
            mWindowsChanged = true;
            moveInputMethodDialogsLocked(pos+1);
            return;
        }
        win.mTargetAppToken = null;
        addWindowToListInOrderLocked(win, true);
        moveInputMethodDialogsLocked(pos);
    }
    .....
}
```

这个函数定义在文件 frameworks/base/services/java/com/android/server/WindowManagerService.java 中。参数 win 描述的是要添加到窗口堆栈中去的输入法窗口。

WindowManagerService 类的成员函数 addInputMethodWindowToListLocked 首先调用另外一个成员函数 findDesiredInputMethodWindowIndexLocked 来计算输入法窗口在窗口堆栈中的位置，并且保存在变量 pos 中。

如果变量 pos 的值大于等于 0，那么就说明 WindowManagerService 类的成员函数 findDesiredInputMethodWindowIndexLocked 在窗口堆栈中找到了一个合适的位置来放置输入法窗口，于是接下来就会参数 win 所描述的输入法窗口插入在 WindowManagerService 类的成员变量 mWindows 所描述的窗口堆栈的第 pos 个位置上。由于系统中的输入法对话框要保持输入在输入法窗口的上面，因此，WindowManagerService 类的成员函数 addInputMethodWindowToListLocked 还需要继续调用另外一个成员函数 moveInputMethodDialogsLocked 来将系统中的输入法对话框在窗口堆栈中的起始位置设置为 (pos+1)。

还有一个地方需要注意的是，前面在调用 WindowManagerService 类的成员函数 addInputMethodWindowToListLocked 来计算输入法窗口在窗口堆栈中的位置的时候，已经将用来描述需要显示输入法窗口的 Activity 窗口的一个 WindowState 对象保存了 WindowManagerService 类的成员变量 mInputMethodTarget 中，因此，这里就需要这个 WindowState 对象的成员变量 mAppToken 所指向的一个 AppWindowToken 对象保存在用来描述输入法窗口的 WindowState 对象的 win 的成员变量 mTargetAppToken 中，以便 WindowManagerService 服务可以知道当前输入法窗口的目标窗口是什么。

如果变量 pos 的值小于 0，那么就说明 WindowManagerService 类的成员函数 findDesiredInputMethodWindowIndexLocked 没有找到一个需要输入法的窗口，因此，这时候就需要调用另外一个成员函数 addWindowToListInOrderLocked 来将参数 win 所描述的输入法窗口插入到窗口堆栈中去。WindowManagerService 类的成员函数 addWindowToListInOrderLocked 会根据目标窗口所对应的窗口令牌在窗口令牌列表中的位置以及是否在窗口堆栈中存在其它窗口等信息在窗口堆栈中找到一个合适的前位置来放置目标窗口，它的具体实现可以参考前面 Android 窗口管理服务 WindowManagerService 组织窗口的方式分析一文。将参数 win 所描述的输入法窗口插入到窗口堆栈中去之后，WindowManagerService 类的成员函数 addInputMethodWindowToListLocked 还需要继续调用另外一个成员函数 moveInputMethodDialogsLocked 来调整系统中的输入法对话框。

注意，在调用 WindowManagerService 类的成员函数 moveInputMethodDialogsLocked 的时候，传递进去的参数 pos 的值等于 -1，这时候 WindowManagerService 类的成员函数 moveInputMethodDialogsLocked 就不是直接调整输入法对话框在窗口堆栈中的位置的，而是调用另外一个成员函数 reAddWindowToListInOrderLocked 来调整的。

还有另外一个地方需要注意的是，由于前面在调用 WindowManagerService 类的成员函数 findDesiredInputMethodWindowIndexLocked 的时候，没有找到一个需要输入法的窗口，因此，这里就需要将参数 win 所描述的一个 WindowState 对象的成员变量 mTargetAppToken 的值设置为 null，以便 WindowManagerService 服务可以知道当前输入法窗口的没有目标窗口。

1. 调整输入法对话框在窗口堆栈的位置

一旦系统中存在需要显示输入法窗口的窗口，那么就需要系统中的输入法对话框在窗口堆栈中的位置，使得它们放置在输入法窗口的上面，这是通过调用 WindowManagerService 类的成员函数 adjustInputMethodDialogsLocked 来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    void adjustInputMethodDialogsLocked() {
```



```

        moveInputMethodDialogsLocked(findDesiredInputMethodWindowIndexLocked(true));
    }

    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数adjustInputMethodDialogsLocked的实现很简单，它首先调用成员函数findDesiredInputMethodWindowIndexLocked来找到输入法窗口在窗口堆栈中的位置，然后再调用成员函数moveInputMethodDialogsLocked来将输入法对话框保存在这个位置之上。

1. 调整输入法窗口在窗口堆栈的位置

当系统中的窗口布局发生了变化之后，例如，当前获得焦点的窗口发生了变化，或者新增了一个窗口，那么都可能需要调整输入法窗口在窗口堆栈中的位置，以便它可以居于需要显示输入法窗口的窗口的上面，这是通过调用WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked来实现的，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    boolean moveInputMethodWindowsIfNeededLocked(boolean needAssignLayers) {
        final WindowState imWin = mInputMethodWindow;
        final int DN = mInputMethodDialogs.size();
        if (imWin == null && DN == 0) {
            return false;
        }

        int imPos = findDesiredInputMethodWindowIndexLocked(true);
        if (imPos >= 0) {
            // In this case, the input method windows are to be placed
            // immediately above the window they are targeting.

            // First check to see if the input method windows are already
            // located here, and contiguous.
            final int N = mWindows.size();
            WindowState firstImWin = imPos < N
                ? mWindows.get(imPos) : null;

            // Figure out the actual input method window that should be
            // at the bottom of their stack.
            WindowState baseImWin = imWin != null
                ? imWin : mInputMethodDialogs.get(0);
            if (baseImWin.mChildWindows.size() > 0) {
                WindowState cw = baseImWin.mChildWindows.get(0);
                if (cw.mSubLayer < 0) baseImWin = cw;
            }

            if (firstImWin == baseImWin) {
                // The windows haven't moved... but are they still contiguous?
                // First find the top IM window.
                int pos = imPos+1;
                while (pos < N) {
                    if (!(mWindows.get(pos)).mIsImWindow) {
                        break;
                    }
                    pos++;
                }
                pos++;
                // Now there should be no more input method windows above.
                while (pos < N) {
                    if ((mWindows.get(pos)).mIsImWindow) {
                        break;
                    }
                    pos++;
                }
                if (pos >= N) {
                    // All is good!
                    return false;
                }
            }
        }
    }
}

```

```

        if (imWin != null) {
            .....
            imPos = tmpRemoveWindowLocked(imPos, imWin);
            .....
            imWin.mTargetAppToken = mInputMethodTarget.mAppToken;
            reAddWindowLocked(imPos, imWin);
            .....
            if (DN > 0) moveInputMethodDialogsLocked(imPos+1);
        } else {
            moveInputMethodDialogsLocked(imPos);
        }

    } else {
        // In this case, the input method windows go in a fixed layer,
        // because they aren't currently associated with a focus window.

        if (imWin != null) {
            .....
            tmpRemoveWindowLocked(0, imWin);
            imWin.mTargetAppToken = null;
            reAddWindowToListInOrderLocked(imWin);
            .....
            if (DN > 0) moveInputMethodDialogsLocked(-1);
        } else {
            moveInputMethodDialogsLocked(-1);
        }

    }

    if (needAssignLayers) {
        assignLayersLocked();
    }

    return true;
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked首先检查系统中是否存在输入法窗口和输入法对话框，即检查WindowManagerService类的成员变量mInputMethodWindow的值是否等于null，并且WindowManagerService类的成员变量mInputMethodDialogs所描述的一个ArrayList的大小是否等于0。如果输入法窗口和输入法对话框都不存在的话，那么就不用调整它们在窗口堆栈中的位置了，否则的话，WindowManagerService类的成员变量mInputMethodWindow所指向的一个WindowState对象就会保存在变量imWin中，以便接下来可以通过它来描述系统中的输入法窗口。

在输入法窗口或者输入法对话框存在的情况下，WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked接下来就会继续调用另外一个成员函数findDesiredInputMethodWindowIndexLocked来找到输入法窗口在窗口堆栈中的位置，并且保存在变量imPos中。注意，变量imPos的值可能大于等于0，也可能等于-1。当变量imPos的值大于等于0的时候，就说明系统当前存在一个窗口需要显示输入法窗口，而当变量imPos的值等于-1的时候，就说明系统当前不存在一个窗口需要显示输入法窗口，或者系统中不存在输入法窗口。接下来我们分两种情况来分析WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked的实现。

第一种情况是变量imPos的值可能大于等于0。这时候可能需要调整输入法窗口在窗口堆栈中的位置，也可能不需要调整输入法窗口在窗口堆栈中的位置，取决于输入法窗口的位置是否已经在窗口堆栈的第imPos个位置上，以及是否所有与输入法相关的窗口都连续在放置在窗口堆栈中。

变量firstImWin描述的是当前位于窗口堆栈中Z轴位置最小的与输入法相关的窗口，它是通过变量imPos来获得的。另外一个变量baseImWin描述的是Z轴位置最小的与输入法相关的窗口。如果这两个变量描述的是同一个窗口，那么就说明输入法窗口的位置已经在窗口堆栈的第imPos个位置上，因此，就有可能不需要调整输入法窗口在窗口堆栈中的位置了。接下来我们就描述如何找到这个Z轴位置最小的与输入法相关的窗口。

如果变量imWin的值不等于null，即WindowManagerService类的成员变量mInputMethodWindow的值不等于null，那么它所描述的窗口就是Z轴位置最小的与输入法相关的窗口，否则的话，Z轴位置最小的与输入法相关的窗口就是位于WindowManagerService类的成员变量mInputMethodDialogs所描述的一个ArrayList的第0个位置上的输入法对话框。这一步得到的Z轴位置最小的与输入法相关的窗口就保存在变量baseImWin中。

如果变量baseImWin所描述的窗口有子窗口，即它所指向的一个WindowState对象的成员变量mChildWindows所描述的一个ArrayList的大小大于0。这时候如果用来描述第一个子窗口的WindowState对象的成员变量mSubLayer的值小于0，

那么就说明变量baseImWin所描述的窗口在所有与输入法相关的窗口中的Z轴位置还不是最小的，因为在它的下面还存在着Z轴位置更小的子窗口。在这种情况下，变量baseImWin就会指向这个Z轴位置最小的子窗口。

经过上面的一系列计算之后，如果变量firstImWin和变量baseImWin描述的是同一个窗口，那么还需要继续判断所有与输入法相关的窗口都连续在放置在窗口堆栈中。判断的方法如下所示：

- (1). 从窗口堆栈的第(imPos + 1)个位置开始往上查找一个非输入法相关的窗口。
- (2). 如果第(1)步能在窗口堆栈中大于等于(imPos+1)的位置pos上找到一个非输入法窗口，那么再继续从第pos个位置开始往上查找一个与输入法相关的窗口。
- (3). 如果第(2)步能在窗口堆栈中找到一个与输入法相关的窗口，那么就说明所有与输入法相关的窗口不是连续在放置在窗口堆栈中的，因为在它们中间有一个非输入法相关的窗口，否则的话，就说明所有与输入法相关的窗口都是连续在放置在窗口堆栈中的。

在所有与输入法相关的窗口都是连续在放置在窗口堆栈中的情况下，WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked就会直接返回一个false值给调用者，表明不需要调整系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置。

在所有与输入法相关的窗口不是连续在放置在窗口堆栈中的情况下，就需要重新调整系统中的输入法窗口以及输入法对话框在窗口堆栈中的位置。这里又需要分两个情景来讨论。

第一个情景是变量imWin的值不等于null，这时候说明系统中存在一个输入法窗口，因此，就需要调整这个输入法窗口在窗口堆栈中的位置。调整的方法很简单：

- (1). 调用WindowManagerService类的成员函数tmpRemoveWindowLocked来从窗口堆栈中移除变量imWin所描述的输入法窗口。在移除的过程中，会同时计算输入法窗口在窗口堆栈中的新位置，这个位置还是保存在变量imPos中。
- (2). 调用WindowManagerService类的成员函数reAddWindowLocked重新将变量imWin所描述的输入法窗口插入到窗口堆栈的第imPos个位置中。在插入之前，还会将变量imWin所描述的一个WindowState对象的成员变量mTargetAppToken与WindowManagerService类的成员变量mInputMethodTarget所描述的一个WindowState对象的成员变量mAppToken指向同一个AppWindowToken对象，这样WindowManagerService服务就可以知道imWin所描述的输入法窗口的目标窗口是什么。
- (3). 如果系统中还存在输入法对话框，那么就调用WindowManagerService类的成员函数moveInputMethodDialogsLocked来将它们放置在第(imPos+1)个位置上，目的是将它们放置在输入法窗口的上面。

第二个情景是变量imWin的值等于null，这时候说明系统中不存在输入法窗口。在这个情景下，系统中肯定会存在输入法对话框，否则的话，WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked在前面就会返回了。因此，WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked接下来就会直接调用成员函数moveInputMethodDialogsLocked来将系统中的输入法对话框放置在在第imPos个位置上。

第二种情况是变量imPos的值等于-1。这时候说明系统中不存在需要显示输入法窗口的窗口。这里同样也需要分两个情景来分析。

第一个情景是变量imWin的值不等于null，这时候说明系统中存在一个输入法窗口，因此，就需要调整这个输入法窗口在窗口堆栈中的位置。调整的方法与前面第一种情况的第一个情景是类似的。不过由于事先不知道输入法窗口在窗口堆栈中的位置，因此，这里就会调用WindowManagerService类的成员函数reAddWindowToListInOrderLocked和moveInputMethodDialogsLocked来间接地调整输入法窗口和输入法对话框在窗口堆栈中的位置。注意，在调用WindowManagerService类的成员函数moveInputMethodDialogsLocked的时候，传进去的参数为-1。另外一个地方需要注意的是，在WindowManagerService类的成员函数reAddWindowToListInOrderLocked来间接地调整输入法窗口在窗口堆栈中的位置之前，会将量imWin所描述的一个WindowState对象的成员变量mTargetAppToken的值设置为null，这样WindowManagerService服务就可以知道imWin所描述的输入法窗口没有目标窗口。

第二情景是变量imWin的值等于null，这时候系统中不存在输入法窗口。这个情景与前面第一种情况的第二个情景也是类似的。由于系统中不存在输入法窗口，因此只需要调用WindowManagerService类的成员函数moveInputMethodDialogsLocked来间接地输入法对话框在窗口堆栈中的位置即可，即以参数-1来调用WindowManagerService类的成员函数moveInputMethodDialogsLocked。

至此，我们就分析完成WindowManagerService服务对输入法窗口的基本操作了。从分析的过程中，我们可以得到以下两个结论：

- A. 系统中与输入法相关的窗口有两种，一种是输入法窗口，另一种是输入法对话框。
- B. 当Z轴位置最大的窗口需要使用输入法时，输入法窗口就会位于它的上面，而输入法对话框又会位于输入法窗口的上面。

在WindowManagerService服务中，还有一种类型的窗口与输入法窗口类似，它总是与Activity窗口粘在一起。不过，这种类型的窗口是位于Activity窗口的下面，刚好与输入法窗口相反，它就是壁纸窗口(Wallpaper)。在接下来的一篇文章中，我们就将继续分析WindowManagerService服务是如何管理系统中的壁纸窗口的。敬请关注！

----- 原文链接: <https://blog.csdn.net/Luoshengyang/article/details/8526644>

第五节 Android窗口管理服务WindowManagerService对壁纸窗口(Wallpaper Window)的管理分析

在Android系统中，壁纸窗口和输入法窗口一样，都是一种特殊类型的窗口，而且它们都是喜欢和一个普通的Activity窗口纠缠在一起。大家可以充分地想象这样的一个3W场景：输入法窗口在上面，壁纸窗口在下面，Activity

窗口夹在它们的中间。在前面一篇文章中，我们已经分析过输入法窗口是如何压在Activity窗口上面的了。在这篇文章中，我们就将继续分析壁纸窗口是如何贴在Activity窗口下面的。

一个Activity窗口如果需要显示壁纸，那么它必须满足以下两个条件：

1. 背景是半透明的，例如，它在AndroidManifest.xml文件中的android:theme属性设置为Theme.Translucent：

```
<activity android:name=".WallpaperActivity"
    android:theme="@android:style/Theme.Translucent">
    .....
</activity>
```

1. 窗口属性中的WindowManager.LayoutParams.FLAG_SHOW_WALLPAPER位设置为1：

```
public class WallpaperActivity extends Activity {
    .....

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        getWindow().addFlags(WindowManager.LayoutParams.FLAG_SHOW_WALLPAPER);
    }

    .....
}
```

满足了以上两个条件之后，Activity窗口和壁纸窗口的位置关系就如图1所示：

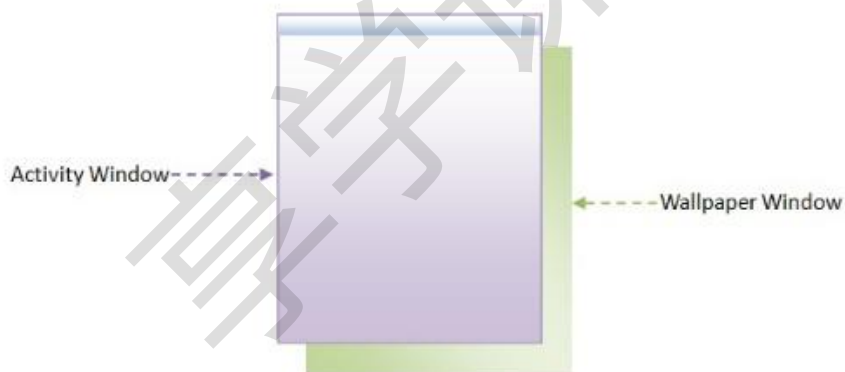


图1 Activity窗口和Wallpaper窗口的位置关系

在前面Android窗口管理服务WindowManagerService组织窗口的方式分析一文中提到，WindowManagerService服务是使用堆栈来组织系统中的窗口的，因此，如果我们在窗口堆栈中观察Activity窗口和壁纸窗口，它们的位置关系就如图2所示：

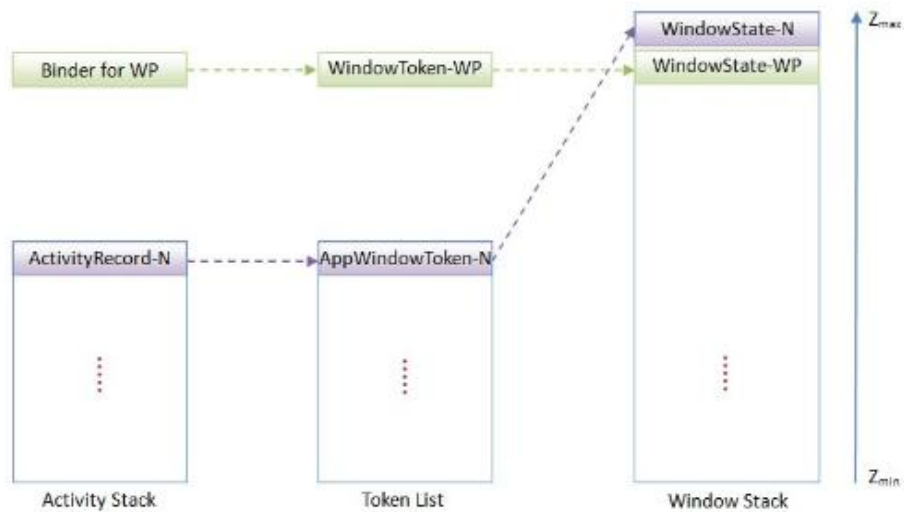


图2 Activity窗口和Wallpaper窗口在窗口堆栈中的位置关系

图2中的对象的关系如下所示：

1. 在ActivityManagerService服务内部的Activity组件堆栈顶端的ActivityRecord对象N描述的是系统当前激活的Activity组件。
2. ActivityRecord对象N在WindowManagerService服务内部的窗口令牌列表顶端对应有一个AppWindowToken对象N。
3. AppWindowToken对象N在WindowManagerService服务内部的窗口堆栈中对应有一个WindowState对象N，用来描述系统当前激活的Activity组件窗口。
4. WindowState对象N下面有一个WindowState对象WP，用来描述系统中的壁纸窗口。
5. 系统中的壁纸窗口在WindowManagerService服务内部中对应的窗口令牌是由WindowToken对象WP来描述的。
6. WindowToken对象WP在WallpaperManagerService服务中对应有一个Binder对象。

总的来说，就是图2描述了系统当前激活的Activity窗口需要显示壁纸的情景。WindowManagerService服务的职能之一就是时刻关注系统中是否有窗口需要显示壁纸。WindowManagerService服务一旦发现有窗口需要显示壁纸，那么就会调整壁纸窗口在窗口堆栈中的位置，使得它放置在需要显示壁纸的窗口的下面。此外，需要显示壁纸的窗口还可以设置壁纸窗口在X轴和Y轴上的偏移位置，以便可以将壁纸窗口的某一部分指定为它的背景。

接下来，我们就首先分析两个需要调整壁纸窗口在窗口堆栈中的位置的情景，然后再分析壁纸窗口在X轴和Y轴上的偏移位置的调整过程，最后分析壁纸窗口在窗口堆栈中的位置调整过程。

一. 调整壁纸窗口在窗口堆栈中的位置的情景

第一个需要调整壁纸窗口在窗口堆栈中的位置的情景是增加一个窗口到WindowManagerService服务去的时候。从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，增加一个窗口到WindowManagerService服务最终是通过调用WindowManagerService类的成员函数addWindow来实现的。接下来我们就主要分析这个函数中与壁纸窗口调整相关的逻辑，如下所示：

```
public class WindowManagerService extends WindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int addWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int viewVisibility,
        Rect outContentInsets, InputChannel outInputChannel) {
        .....

        synchronized(mWindowMap) {
            .....

            WindowToken token = mTokenMap.get(attrs.token);
            if (token == null) {
                .....

                if (attrs.type == TYPE_WALLPAPER) {
                    .....
                    return WindowManagerImpl.ADD_BAD_APP_TOKEN;
                }
            }
        }
    }
}
```

```

        .....
    }
    .....

    win = new WindowState(session, client, token,
        attachedWindow, attrs, viewVisibility);
    .....

    if (attrs.type == TYPE_INPUT_METHOD) {
        .....
    } else if (attrs.type == TYPE_INPUT_METHOD_DIALOG) {
        .....
    } else {
        addWindowToListInOrderLocked(win, true);
        if (attrs.type == TYPE_WALLPAPER) {
            .....
            adjustWallpaperWindowsLocked();
        } else if ((attrs.flags & FLAG_SHOW_WALLPAPER) != 0) {
            adjustWallpaperWindowsLocked();
        }
    }
    .....

    assignLayersLocked();

    .....
}
.....
}
.....
}
}

```

这个函数定义在文件 `frameworks/base/services/java/com/android/server/WindowManagerService.java` 中。如果当前增加到 `WindowManagerService` 服务来的是一个壁纸窗口，即参数 `attrs` 所描述的一个 `WindowManager.LayoutParams` 对象的成员变量 `type` 的值等于 `TYPE_WALLPAPER`，那么就要求与该壁纸窗口所对应的类型为 `WindowToken` 的窗口令牌已经存在，否则的话，`WindowManagerService` 类的成员函数 `addWindow` 就会直接返回一个错误码

`WindowManagerImpl.ADLBADAPPTOKEN` 给调用者。这个类型为 `WindowToken` 的窗口令牌是 `WallpaperManagerService` 服务请求 `WindowManagerService` 服务创建的，即调用 `WindowManagerService` 类的成员函数 `addWindowToken` 来创建的，具体可以参考前面 Android 窗口管理服务 `WindowManagerService` 组织窗口的方式分析一文。

如果当前增加到 `WindowManagerService` 服务来的既不是一个输入法窗口，也不是一个输入法对话框，那么 `WindowManagerService` 类的成员函数 `addWindow` 就会调用另外一个成员函数 `addWindowToListInOrderLocked` 来将前面为它所创建的一个 `WindowState` 对象 `win` 增加到窗口堆栈的合适位置上去。

如果前面增加到窗口堆栈中的窗口是一个壁纸窗口，即参数 `attrs` 所描述的一个 `WindowManager.LayoutParams` 对象的成员变量 `type` 的值等于 `TYPE_WALLPAPER`，或者是一个需要显示壁纸的窗口，即参数 `attrs` 所描述的一个 `WindowManager.LayoutParams` 对象的成员变量 `flags` 的值的 `FLAG_SHOW_WALLPAPER` 位等于 1，那么就说明需要调整壁纸窗口在窗口堆栈中的位置，使得它位于需要显示壁纸的窗口的下面，这是通过调用 `WindowManagerService` 类的成员函数 `adjustWallpaperWindowsLocked` 来实现的。

最后，由于增加了一个窗口到窗口堆栈中，以及窗口堆栈的窗口位置发生了变化，因此，就需要重新各个窗口的 Z 轴位置，这是通过调用 `WindowManagerService` 类的成员函数 `assignLayersLocked` 来实现的。

在这个情景中，主要涉及到了 `WindowManagerService` 类的三个成员函数 `addWindowToListInOrderLocked`、`adjustWallpaperWindowsLocked` 和 `assignLayersLocked`，其中，成员函数 `addWindowToListInOrderLocked` 的实现可以参考前面前面 Android 窗口管理服务 `WindowManagerService` 组织窗口的方式分析一文，成员函数 `assignLayersLocked` 的实现在接下来的文章中再分析，本文主要是关注成员函数 `adjustWallpaperWindowsLocked` 的实现。

第二个需要调整壁纸窗口在窗口堆栈中的位置的情景是一个应用程序进程请求 `WindowManagerService` 服务重新布局一个窗口的时候。从前面 Android 窗口管理服务 `WindowManagerService` 计算 Activity 窗口大小的过程分析一文可以知道，应用程序进程请求 `WindowManagerService` 服务重新布局一个窗口最终是通过调用 `WindowManagerService` 类的成员函数 `relayoutWindow` 来实现的。接下来我们就主要分析这个函数中与壁纸窗口调整相关的逻辑，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int relayoutWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int requestedWidth,
        int requestedHeight, int viewVisibility, boolean insetsPending,
        Rect outFrame, Rect outContentInsets, Rect outVisibleInsets,

```

```

        Configuration outConfig, Surface outSurface) {
    boolean displayed = false;
    .....

    synchronized(mWindowMap) {
        WindowState win = windowForClientLocked(session, client, false);
        .....

        int attrChanges = 0;
        .....
        if (attrs != null) {
            .....
            attrChanges = win.mAttrs.copyFrom(attrs);
        }
        .....

        boolean wallpaperMayMove = win.mViewVisibility != viewVisibility
            && (win.mAttrs.flags & FLAG_SHOW_WALLPAPER) != 0;
        .....

        if (viewVisibility == View.VISIBLE &&
            (win.mAppToken == null || !win.mAppToken.clientHidden)) {
            displayed = !win.isVisibleLw();
            .....

            if ((attrChanges & WindowManager.LayoutParams.FORMAT_CHANGED) != 0) {
                // To change the format, we need to re-build the surface.
                win.destroySurfaceLocked();
                displayed = true;
            }
            .....
        }
        .....

        boolean assignLayers = false;
        .....

        if (wallpaperMayMove) {
            if ((adjustWallpaperWindowsLocked() & ADJUST_WALLPAPER_LAYERS_CHANGED) != 0) {
                assignLayers = true;
            }
        }
        .....

        if (assignLayers) {
            assignLayersLocked();
        }
        .....

        performLayoutAndPlaceSurfacesLocked();
        if (displayed && win.mIsWallpaper) {
            updateWallpaperOffsetLocked(win, mDisplay.getWidth(),
                mDisplay.getHeight(), false);
        }
        .....

    }

    .....

    return (inTouchMode ? WindowManagerImpl.RELAYOUT_IN_TOUCH_MODE : 0)
        | (displayed ? WindowManagerImpl.RELAYOUT_FIRST_TIME : 0);
}

.....
}

```

这个函数定义在文件 `frameworks/base/services/java/com/android/server/WindowManagerService.java` 中。

应用程序进程在请求 `WindowManagerService` 服务重新布局一个窗口的时候，这个窗口的一些布局参数可能会发生变

化，而这些变化可能会引发系统的壁纸窗口在窗口堆栈中的位置发生变化。如果系统的壁纸窗口在窗口堆栈中的位置发生了变化，那么就需要调整它们在窗口堆栈中的位置。

WindowManagerService类的成员函数relayoutWindow首先调用根据参数session和client来调用另外一个成员函数windowForClientLocked，以便可以获得用来描述要重新布局的窗口的一个WindowState对象win。

WindowState对象win的成员变量mViewVisibility描述的是窗口上一次布局时的可见性，而参数viewVisibility描述的是窗口当前的可见性，当它们的值不相等时，就意味着窗口的可见性发生了变化。在窗口的可见性发生了变化的情况下，如果正在请求重新布局的是一个需要显示壁纸的窗口，即WindowState对象win的成员变量mAttrs所指向的是一个WindowManager.LayoutParams对象的成员变量flags的FLAG_SHOW_WALLPAPER位等于1，那么就说明可能需要调整壁纸窗口在窗口堆栈中的位置，以便它可以位于WindowState对象win所描述的窗口的下面，这时候变量wallpaperMayMove的值就会等于true。

WindowManagerService类的成员函数relayoutWindow执行了一系列的其它操作之后，接下来就会判断变量wallpaperMayMove的值是否等于true。如果等于true的话，那么就会调用另外一个成员函数adjustWallpaperWindowsLocked来调整壁纸窗口在窗口堆栈中的位置，以便它可以位于需要显示壁纸的窗口的下面。WindowManagerService类的成员函数adjustWallpaperWindowsLocked的返回值是一个整数，当它的ADJUST_WALLPAPER_LAYERS_CHANGED位等于1的时候，就说明壁纸窗口在窗口堆栈的位置发生了变化，于是就会将变量assignLayers的值设置为true，以便接下来可以调用WindowManagerService类的成员函数assignLayersLocked来重新计算系统中各个窗口的Z轴位置。

变量displayed用来描述WindowState对象win所描述的窗口在当前布局中是由不可见变为可见的。在满足以下的条件之下，WindowState对象win所描述的窗口是由不可见变为可见的：

1. 参数viewVisibility的值等于View.VISIBLE，即应用程序进程请求显示WindowState对象win所描述的窗口。
2. WindowState对象win描述的是一个Activity窗口，即它的成员变量mAppToken不等于null，并且它所指向的AppWindowToken对象的成员变量clientHidden的值等于false，即WindowState对象win的窗口所对应的Activity组件当前是可见的。注意，如果WindowState对象win描述的不是一个Activity窗口，即它的成员变量mAppToken等于null，那么就可以忽略条件2。
3. WindowState对象win所描述的窗口上一次是不可见的，即调用WindowState对象win的成员函数isVisibleLw的返回值等于false。

此外，在满足条件1和条件2的情况下，如果WindowState对象win所描述的窗口的像素格式发生了变化，那么就需要将该窗口的绘图表面销毁掉，然后再重新创建一个，这时候也会认为该窗口由不可见变为可见。

参数attrs所指向的一个WindowManager.LayoutParams对象是用来保存WindowState对象win所描述的窗口在当前布局中所使用的布局参数的，而WindowState对象win的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象是用来保存WindowState对象win所描述的窗口在上一次布局所使用的布局参数的。在将参数attrs所指向的一个WindowManager.LayoutParams对象的内容拷贝到WindowState对象win的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的过程中，如果某些布局参数发生了变化，那么就会记录在变量attrChanges中。当变量attrChanges的WindowManager.LayoutParams.FORMAT_CHANGED位等于1时，就说明WindowState对象win所描述的窗口的像素格式发生了变化，因此，WindowManagerService类的成员函数relayoutWindow就会调用WindowState对象win的成员函数destroySurfaceLocked来销毁该窗口的绘图表面，并且将变量displayed的值设置为true。

WindowManagerService类的成员函数relayoutWindow调用另外一个成员函数performLayoutAndPlaceSurfacesLocked来对WindowState对象win所描述的窗口进行了布局之后，如果发现变量displayed的值等于true，并且WindowState对象win描述的是一个壁纸窗口，即它的成员变量mIsWallpaper的值等于true，那么还需要调用另外一个成员函数updateWallpaperOffsetLocked来重新计算该壁纸窗口在X轴和Y轴上的偏移位置，以便可以将它的某一部分区域指定在需要显示壁纸的窗口的背景。

在这个情景中，主要涉及到了WindowManagerService类的四个成员函数adjustWallpaperWindowsLocked、updateWallpaperOffsetLocked、performLayoutAndPlaceSurfacesLocked和assignLayersLocked，其中，成员函数performLayoutAndPlaceSurfacesLocked的实现框架可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文，成员函数assignLayersLocked的实现如上所述在接下来的一篇文章中再分析，本文主要是关注成员函数adjustWallpaperWindowsLocked和updateWallpaperOffsetLocked的实现。

从上面的分析就可以知道，在布局一个窗口的过程中，可能需要调用WindowManagerService类的成员函数updateWallpaperOffsetLocked和adjustWallpaperWindowsLocked来调整壁纸窗口在X轴和Y轴上的偏移位置和在窗口堆栈中的位置。接下来我们就分别分析壁纸窗口在X轴和Y轴上的偏移位置和在窗口堆栈中的位置的调整过程。

二. 调整壁纸窗口在X轴和Y轴上的偏移位置

壁纸窗口的大小是可以大于屏幕大小的。在这种情况下，需要显示壁纸的Activity窗口就需要指定壁纸在X轴和Y轴上的偏移位置，以便可以将壁纸的某一部分作为窗口的背景。

假设壁纸窗口的大小为（WallpaperWidth, WallpaperHeight），屏幕的大小为（DisplayWidth, DisplayHeight），并且壁纸在X轴和Y轴上的偏移位置为WallpaperX和WallpaperY，其中，WallpaperWidth > DisplayWidth, WallpaperHeight > DisplayHeight, 0.0 <= WallpaperX <= 1.0, 0.0 <= WallpaperY <= 1.0，如图3所示：

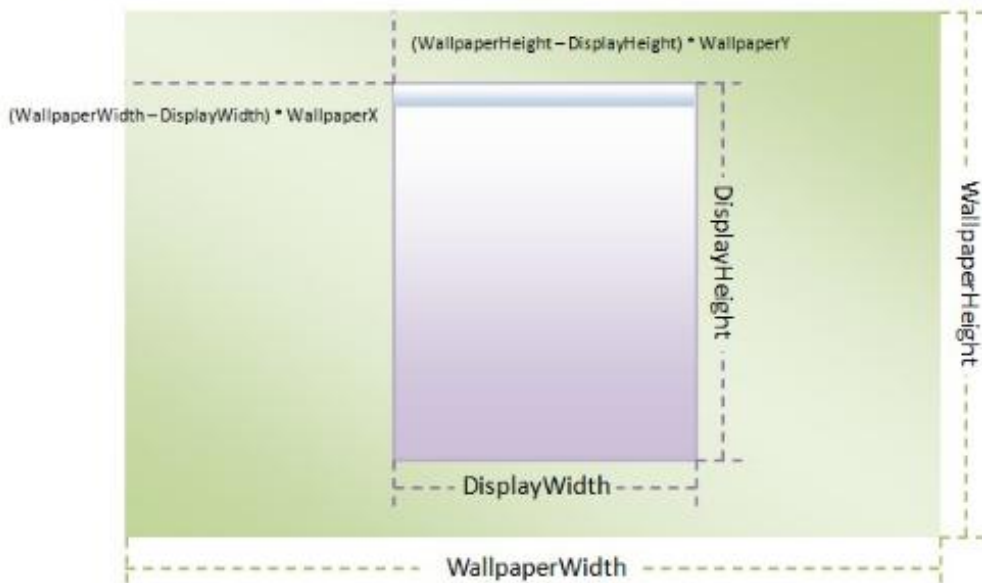


图3 指定壁纸窗口在X轴和Y轴上的偏移位置

这时候壁纸窗口在X轴和Y轴上的偏移位置的绝对值XOffset和YOffset就分别等于 $(\text{WallpaperWidth} - \text{DisplayWidth}) * \text{WallpaperX}$ 和 $(\text{WallpaperHeight} - \text{DisplayHeight}) * \text{WallpaperY}$ 。这意味着：

1. 当 $\text{WallpaperX} = \text{WallpaperY} = 0.0$ 时，取壁纸窗口的左上角区域作为窗口背景。
2. 当 $\text{WallpaperX} = \text{WallpaperY} = 0.5$ 时，取壁纸窗口的中间区域作为窗口背景。
3. 当 $\text{WallpaperX} = \text{WallpaperY} = 1.0$ 时，取壁纸窗口的右下角区域作为窗口背景。

除了使用 WallpaperX 和 WallpaperY 来描述壁纸窗口在X轴和Y轴上的偏移位置之外，`WindowManagerService` 服务还使用 WallpaperXStep 和 WallpaperYStep 来描述壁纸窗口跨越了多少个虚拟屏幕。例如，假设一个Activity窗口在X轴上有3个虚拟屏幕，即它的实际宽度是屏幕宽度的3倍，而在Y轴上有一个屏幕，即它的实际高度刚好等于屏幕高度，并且壁纸窗口的宽度也刚好是屏幕宽度的3倍，而高度也刚好是等于屏幕高度，那么 WallpaperXStep 和 WallpaperYStep 的值就可以分别指定为0.5和0，这意味着：

1. 第1个虚拟屏幕取壁纸窗口的左边三分之一的区域作为窗口背景，相当于是将壁纸窗口在X轴和Y轴上的偏移位置 WallpaperX 和 WallpaperY 的值分别设置为0.0和0.0。
2. 第2个虚拟屏幕取壁纸窗口的中间三分之一的区域作为窗口背景，相当于是将壁纸窗口在X轴和Y轴上的偏移位置 WallpaperX 和 WallpaperY 的值分别设置为0.5和0.0。
3. 第3个虚拟屏幕取壁纸窗口的右边三分之一的区域作为窗口背景，相当于是将壁纸窗口在X轴和Y轴上的偏移位置 WallpaperX 和 WallpaperY 的值分别设置为1.0和0.0。

一般地，如果一个Activity窗口在X轴上有N个虚拟屏幕，而在Y轴上有M个虚拟屏幕，那么它就会将壁纸窗口的 WallpaperXStep 和 WallpaperYStep 值分别设置为 $1.0 / (N - 1)$ 和 $1.0 / (M - 1)$ 。对于 `WindowManagerService` 服务来说，它并不关心壁纸窗口的 WallpaperXStep 和 WallpaperYStep 值，而只关心壁纸窗口的 WallpaperX 和 WallpaperY 值，因为通过后两者，它就可以知道怎么显示壁纸窗口了。壁纸窗口的 WallpaperXStep 和 WallpaperYStep 值是用来传递给提供壁纸的服务的。提供壁纸的服务一旦知道壁纸窗口的 WallpaperXStep 和 WallpaperYStep 值是多少，就可以知道当前需要显示壁纸的窗口有多少个虚拟屏幕。

上面提到的与壁纸窗口在X轴和Y轴上的偏移位置相关的六个状态 WallpaperX 、 WallpaperY 、 WallpaperXStep 、 WallpaperYStep 、 XOffset 和 YOffset 由 `WindowManagerService` 服务来统一维护，它们分别对应于 `WindowState` 类的六个成员变量 `mWallpaperX`、`mWallpaperY`、`mWallpaperXStep`、`mWallpaperYStep`、`mXOffset` 和 `mYOffset`，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    float mLastWallpaperX = -1;
    float mLastWallpaperY = -1;
    float mLastWallpaperXStep = -1;
    float mLastWallpaperYStep = -1;
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {
        .....

        // If a window showing a wallpaper: the requested offset for the
        // wallpaper; if a wallpaper window: the currently applied offset.
```

```

float mWallpaperX = -1;
float mWallpaperY = -1;

// If a window showing a wallpaper: what fraction of the offset
// range corresponds to a full virtual screen.
float mWallpaperXStep = -1;
float mWallpaperYStep = -1;

// Wallpaper windows: pixels offset based on above variables.
int mXOffset;
int mYOffset;

.....
}

.....
}

```

这段代码定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。此外，WindowManagerService类还使用四个成员变量mLastWallpaperX、mLastWallpaperY、mLastWallpaperXStep和mLastWallpaperYStep来记录壁纸窗口上一次所使用的WallpaperX、WallpaperY、WallpaperXStep和WallpaperYStep值。

在Android系统中，提供壁纸功能的组件叫做WallpaperService，它是一个Service组件，是由壁纸管理服务WallpaperManagerService负责启动的。WallpaperService有两个内部类BaseIWindow和Engine，其中，BaseIWindow是一个实现了IWindow接口的Binder本地对象类，用来和WindowManagerService服务通信，而Engine是一个真正用来实现壁纸功能的类。当一个Activity窗口需要指定壁纸窗口的某一部分区域作为它的背景时，它就会通过WallpaperManager类来通知WallpaperService设置壁纸窗口在X轴和Y轴上的偏移位置，这个过程如图4所示：

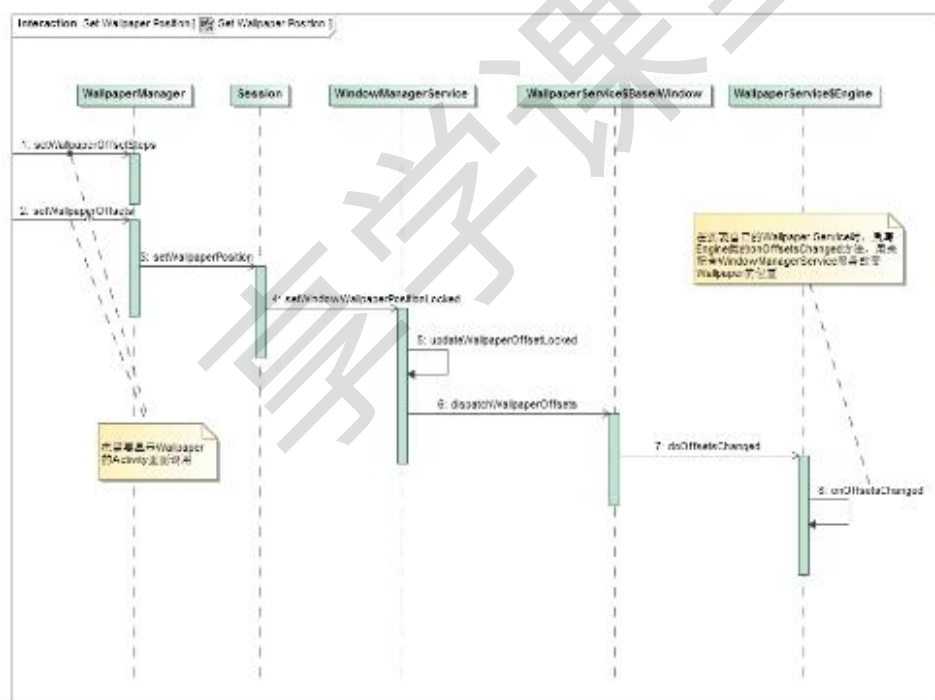


图4 壁纸窗口在X轴和Y轴上的偏移位置的设置过程

这个过程大概如下所示：

Step 1. 需要显示壁纸的Activity组件调用WallpaperManager类的成员函数setWallpaperOffsetSteps来设置壁纸窗口的WallpaperXStep和WallpaperYStep值。

Step 2. 需要显示壁纸的Activity组件调用WallpaperManager类的成员函数setWallpaperOffsets来设置壁纸窗口的WallpaperX和WallpaperY值。

Step 3. 一个类型为Session的Binder代理对象的成员函数setWallpaperPosition会被调用来通知WindowManagerService服务来重新计算壁纸窗口在X轴和Y轴上的偏移位置，传递的参数包括在Step 1和Step 2中所设置的WallpaperXStep、WallpaperYStep、WallpaperX和WallpaperY四个值。

Step 4. WindowManagerService类的成员函数setWindowWallpaperPositionLocked会被调用来保存从前面Step 3传递过来的WallpaperXStep、WallpaperYStep、WallpaperX和WallpaperY值。

Step 5. WindowManagerService类的成员函数updateWallpaperOffsetLocked会被调用来计算壁纸窗口在X轴和Y轴上的偏移位置的绝对值XOffset和YOffset，是根据壁纸窗口的大小（WallpaperWidth, WallpaperHeight）、屏幕的大小（DisplayWidth, DisplayHeight），以及保存在前面Step 4中的WallpaperX和WallpaperY来计算的。

Step 6. 在WallpaperService类内部的一个BaseIWindow对象的成员函数dispatchWallpaperOffsets会被调用来通知WallpaperService服务，壁纸窗口在X轴和Y轴上的偏移位置发生改变，传递过来的参数包括壁纸窗口的XOffset、YOffset、WallpaperXStep和WallpaperYStep值。

Step 7. 在WallpaperService类内部的一个Engine对象的成员函数doOffsetsChanged会被调用来处理壁纸窗口在X轴和Y轴上的偏移位置变化事件。

Step 8. Engine类的成员函数doOffsetsChanged会调用另外一个成员函数onOffsetsChanged来分发壁纸窗口在X轴和Y轴上的偏移位置变化事件。Engine类的成员函数onOffsetsChanged一般是由其子类来重写的，以便子类可以实现自己的壁纸效果。

本文不打算详细这八个步骤，而主要关注Step 3、Step 4和Step 5这三步是如何计算壁纸窗口在X轴和Y轴上的偏移位置的，即主要关注Session类的成员函数setWallpaperPosition，以及WindowManagerService类的成员函数setWindowWallpaperPositionLocked和updateWallpaperOffsetLocked的实现。

Session类的成员函数setWallpaperPosition的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class Session extends IWindowSession.Stub
        implements IBinder.DeathRecipient {
        .....

        public void setWallpaperPosition(IBinder window, float x, float y, float xStep, float yStep) {
            synchronized(mWindowMap) {
                long ident = Binder.clearCallingIdentity();
                try {
                    setWindowWallpaperPositionLocked(
                        windowForClientLocked(this, window, true),
                        x, y, xStep, yStep);
                } finally {
                    Binder.restoreCallingIdentity(ident);
                }
            }
        }
    }
}
.....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

Session类的成员函数setWallpaperPosition首先调用WindowManagerService类的成员函数windowForClientLocked来找到与参数window所对应的一个WindowState对象，这个WindowState对象描述的是要改变壁纸窗口位置的窗口，接着再调用WindowManagerService类的另外一个成员函数setWindowWallpaperPositionLocked来执行设置壁纸窗口在X轴和Y轴的偏移位置的操作。

WindowManagerService类的成员函数setWindowWallpaperPositionLocked的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void setWindowWallpaperPositionLocked(WindowState window, float x, float y,
        float xStep, float yStep) {
        if (window.mWallpaperX != x || window.mWallpaperY != y) {
            window.mWallpaperX = x;
            window.mWallpaperY = y;
            window.mWallpaperXStep = xStep;
            window.mWallpaperYStep = yStep;
            if (updateWallpaperOffsetLocked(window, true)) {
                performLayoutAndPlaceSurfacesLocked();
            }
        }
    }
}
.....
```

```
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数setWindowWallpaperPositionLocked首先检查参数window所描述的WindowState对象上一次所设置的壁纸窗口的偏移位置与参数x和y所描述的偏移位置是否不一样。如果不一样的话，那么就会分别将参数x、y、xStep和yStep分别保存在参数window所描述的WindowState对象的成员变量mWallpaperX、mWallpaperY、mWallpaperXStep和mWallpaperYStep中，并且调用WindowManagerService类的成员函数updateWallpaperOffsetLocked来更新系统中的壁纸窗口的偏移位置。

如果WindowManagerService类的成员函数updateWallpaperOffsetLocked的返回值等于true，那么就说明它更新了系统中的壁纸窗口的偏移位置，因此，就需要调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来刷新系统的UI。

接下来我们继续分析WindowManagerService类的成员函数updateWallpaperOffsetLocked的实现，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    final ArrayList<WindowToken> mWallpaperTokens = new ArrayList<WindowToken>();

    // If non-null, this is the currently visible window that is associated
    // with the wallpaper.
    WindowState mWallpaperTarget = null;
    .....

    boolean updateWallpaperOffsetLocked(WindowState changingTarget, boolean sync) {
        final int dw = mDisplay.getWidth();
        final int dh = mDisplay.getHeight();

        boolean changed = false;

        WindowState target = mWallpaperTarget;
        if (target != null) {
            if (target.mWallpaperX >= 0) {
                mLastWallpaperX = target.mWallpaperX;
            } else if (changingTarget.mWallpaperX >= 0) {
                mLastWallpaperX = changingTarget.mWallpaperX;
            }
            if (target.mWallpaperY >= 0) {
                mLastWallpaperY = target.mWallpaperY;
            } else if (changingTarget.mWallpaperY >= 0) {
                mLastWallpaperY = changingTarget.mWallpaperY;
            }
        }

        int curTokenIndex = mWallpaperTokens.size();
        while (curTokenIndex > 0) {
            curTokenIndex--;
            WindowToken token = mWallpaperTokens.get(curTokenIndex);
            int curWallpaperIndex = token.windows.size();
            while (curWallpaperIndex > 0) {
                curWallpaperIndex--;
                WindowState wallpaper = token.windows.get(curWallpaperIndex);
                if (updateWallpaperOffsetLocked(wallpaper, dw, dh, sync)) {
                    wallpaper.computeShownFrameLocked();
                    changed = true;
                    // We only want to be synchronous with one wallpaper.
                    sync = false;
                }
            }
        }

        return changed;
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

当WindowManagerService类的成员变量mWallpaperTarget的值不等于null时，它所指向的一个WindowState对象描述的是系统当前可见的并且需要显示壁纸的窗口。在这种情况下，要将这个WindowState对象当前正在使用的壁纸窗口在X轴和Y轴上的偏移位置分别保存在WindowManagerService类的成员变量mLastWallpaperX和mLastWallpaperY，以便接下来可以用来计算壁纸窗口在X轴和Y轴上的偏移位置的绝对值。

注意，如果WindowManagerService类的成员变量mWallpaperTarget所指向的一个WindowState对象的成员变量mWallpaperX（mWallpaperY）的值小于0，那么就说明这个WindowState对象所描述的窗口还没有设置过壁纸窗口在X轴上（Y轴上）的偏移位置，这时候就需要将参数changingTarget所指向的一个WindowState对象的成员变量mWallpaperX（mWallpaperY）的值保存在WindowManagerService类的成员变量mLastWallpaperX（mLastWallpaperY）中，前提也是它的值大于等于0，即它描述的是一个有效的偏移值。

WindowManagerService类的成员变量mWallpaperTokens保存的是一系列与壁纸相关的窗口令牌，与这些窗口令牌所对应的窗口就是系统当前所设置的壁纸窗口。WindowManagerService类的成员函数updateWallpaperOffsetLocked依次调用另外一个四参数版本的成员函数updateWallpaperOffsetLocked来更新系统当前所设置的每一个壁纸窗口在X轴和Y轴上的偏移位置。

注意，WindowManagerService类的四个参数版本的成员函数updateWallpaperOffsetLocked的最后一个参数sync是一个布尔值，用来表示在更新壁纸窗口在X轴和Y轴上的偏移位置的时候，是否需要同步等待提供壁纸窗口的服务处理完成壁纸窗口在X轴和Y轴上的偏移位置变化事件。参数sync本身也是由两个参数版本的成员函数updateWallpaperOffsetLocked的调用者传进来的，它的值即使等于true，两个参数版本的成员函数updateWallpaperOffsetLocked也只会同步等待提供第一个壁纸窗口的服务处理完成壁纸窗口在X轴和Y轴上的偏移位置变化事件。

WindowManagerService类的四个参数版本的成员函数updateWallpaperOffsetLocked的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    boolean updateWallpaperOffsetLocked(WindowState wallpaperWin, int dw, int dh,
        boolean sync) {
        boolean changed = false;
        boolean rawChanged = false;
        float wpx = mLastWallpaperX >= 0 ? mLastWallpaperX : 0.5f;
        float wpxs = mLastWallpaperXStep >= 0 ? mLastWallpaperXStep : -1.0f;
        int availw = wallpaperWin.mFrame.right-wallpaperWin.mFrame.left-dw;
        int offset = availw > 0 ? -(int)(availw*wpx+.5f) : 0;
        changed = wallpaperWin.mXOffset != offset;
        if (changed) {
            .....
            wallpaperWin.mXOffset = offset;
        }
        if (wallpaperWin.mWallpaperX != wpx || wallpaperWin.mWallpaperXStep != wpxs) {
            wallpaperWin.mWallpaperX = wpx;
            wallpaperWin.mWallpaperXStep = wpxs;
            rawChanged = true;
        }

        float wpy = mLastWallpaperY >= 0 ? mLastWallpaperY : 0.5f;
        float wpys = mLastWallpaperYStep >= 0 ? mLastWallpaperYStep : -1.0f;
        int availh = wallpaperWin.mFrame.bottom-wallpaperWin.mFrame.top-dh;
        offset = availh > 0 ? -(int)(availh*wpy+.5f) : 0;
        if (wallpaperWin.mYOffset != offset) {
            .....
            changed = true;
            wallpaperWin.mYOffset = offset;
        }
        if (wallpaperWin.mWallpaperY != wpy || wallpaperWin.mWallpaperYStep != wpys) {
            wallpaperWin.mWallpaperY = wpy;
            wallpaperWin.mWallpaperYStep = wpys;
            rawChanged = true;
        }

        if (rawChanged) {
            try {
                .....
                if (sync) {
                    mWaitingOnWallpaper = wallpaperWin;
                }
                wallpaperWin.mClient.dispatchWallpaperOffsets(
                    wallpaperWin.mWallpaperX, wallpaperWin.mWallpaperY,
                    wallpaperWin.mWallpaperXStep, wallpaperWin.mWallpaperYStep, sync);
                if (sync) {
```

```

        if (mWaitingOnWallpaper != null) {
            long start = SystemClock.uptimeMillis();
            if ((mLastWallpaperTimeoutTime+WALLPAPER_TIMEOUT_RECOVERY)
                < start) {
                try {
                    .....
                    mWindowMap.wait(WALLPAPER_TIMEOUT);
                } catch (InterruptedException e) {
                }
                .....
                if ((start+WALLPAPER_TIMEOUT)
                    < SystemClock.uptimeMillis()) {
                    .....
                    mLastWallpaperTimeoutTime = start;
                }
            }
            mWaitingOnWallpaper = null;
        }
    } catch (RemoteException e) {
    }
}

return changed;
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的四个参数版本的成员函数updateWallpaperOffsetLocked首先计算参数wallpaper所描述的壁纸窗口在X轴和Y轴上的偏移位置，接着再向提供该壁纸窗口的服务发向一个壁纸窗口在X轴和Y轴上的偏移位置变化事件通知。

参数wallpaper所描述的壁纸窗口在X轴和Y轴上的偏移位置的计算过程是一样的，这里我们只要结合前面的图3来分析壁纸窗口在X轴上的偏移位置的计算过程。

在图3中，壁纸窗口的WallpaperX、WallpaperXStep、WallpaperWidth和DisplayWidth值分别等于这里的mLastWallpaperX、mLastWallpaperXStep、wallpaperWin.mFrame.right - wallpaperWin.mFrame.left和dw。有了这些值之后，就可以计算得到参数wallpaper所描述的壁纸窗口在X轴上的偏移位置的绝对值XOffset了。

如果计算得到的XOffset、WallpaperX、WallpaperXStep的值与原来保存在参数wallpaper所指向的一个WindowState对象的成员变量mXOffset、mWallpaperX、mWallpaperXStep的值不相等，那么就会将计算得到的XOffset、WallpaperX、WallpaperXStep的值分别保存在这个WindowState对象的成员变量mXOffset、mWallpaperX、mWallpaperXStep，并且相应地将变量changed和rawChanged的值设置为true，表示参数wallpaper所描述的壁纸窗口在X轴上的偏移位置发生了变化。

有四个地方需要注意：

1. 当mLastWallpaperX的值小于0的时候，那么就说明系统中的壁纸窗口还没有被设置一个有效的X轴偏移位置，这时候计算壁纸窗口在X轴上的偏移位置所采用的WallpaperX值就会取为0.5，即默认将壁纸窗口的中间区域指定为需要显示壁纸的窗口的背景。
2. 当mLastWallpaperXStep的值小于0的时候，那么就说明需要显示壁纸的窗口还没有告诉WindowManagerService服务它有多少个虚拟屏幕，这时候就会将壁纸窗口的WallpaperXStep值设置为-1.0，用来告诉提供壁纸窗口的服务，需要显示壁纸的窗口没有指定虚拟屏幕的个数。
3. 当壁纸窗口的宽度小于等于屏幕宽度的时候，即变量availw的值小于等于0的时候，那么就说明不需要设置壁纸窗口在X轴上的偏移位置，也就是说，这时候壁纸窗口在X轴上的偏移位置始终保持为0。
4. 当壁纸窗口的宽度大于屏幕宽度的时候，即变量availw的值大于0的时候，壁纸窗口在X轴上的偏移值等于availw * wps，加上0.5是为了向上取整，向上取整后需要取反，因为负数才能正确表达出壁纸窗口相对屏幕的偏移。

计算完成参数wallpaper所描述的壁纸窗口在X轴和Y轴上的偏移位置之后，如果变量rawChanged的值等于true，那么就说明参数wallpaper所描述的壁纸窗口在X轴和Y轴上的偏移位置发生了变化，这时候就需要向提供该壁纸窗口的服务发送一个事件通知，这是通过调用参数wallpaperWin所指向的一个WindowState对象的成员变量mClient所描述的一个实现了IWindow接口的Binder代理对象的成员函数dispatchWallpaperOffsets来实现的，同时传递给壁纸窗口的服务的参数有壁纸窗口当前所使用的WallpaperX、WallpaperY、WallpaperXStep和WallpaperYStep值，以及另外一个同步参数sync。

当参数sync的值等于true的时候，就表示WindowManagerService服务需要等待提供壁纸窗口wallpaperWin的服务处理完成前面所发送的偏移位置变化事件通知，等待的最长时间为WALLPAPER_TIMEOUT。如果提供壁纸窗口wallpaperWin的服务不能在WALLPAPER_TIMEOUT时间内向WindowManagerService服务发送一个事件处理完成通知，那么WindowManagerService服务就会将这次事件通知发送时间start保存在WindowManagerService类的成员变量mLastWallpaperTimeoutTime中。

如果上一次发送的壁纸窗口偏移位置变化事件通知发生了超时，那么在上次发送这个事件通知起的WALLPAPER_TIMEOUT_RECOVERY时间内，是不允许再次发送壁纸窗口偏移位置变化事件通知的。这是因为在上一次事件通知超时的情况下，在短时间内再次发送相同的事件通知也是非常有可能是超时的，因此，就不允许短时间内重复发送相同的事件通知，避免出现雪崩现象。

关于互联网的雪崩现象，可以举一个常见的例子来说明。假设现在有一个Web页面正在现场直播一项非常热门的体育赛事，这时候就会有海量的用户访问这个页面。一旦访问量快要达到Web服务器的承受能力的时候，Web页面的打开速度就会越来越慢。Web页面打开速度变慢的时候，用户就会下意识地不断按F5刷新。越是不断地按F5刷新，Web页面的请求量就越大，而当请求量大于Web服务器的承受能力的时候，Web服务器就会宕机了，这个就是雪崩现象。为了避免雪崩现象，就需要在请求量快要达到Web服务器的承受能力的时候，避免用户发送更多的访问请求，以使得Web服务器有喘息的机会。

废话少说了，当WindowManagerService服务在等待壁纸窗口wallpaper所属的服务处理它的偏移位置变化事件通知时，会将该壁纸窗口wallpaper保存在WindowManagerService类的成员变量mWaitingOnWallpaper中，用来表示WindowManagerService服务正在处于等待壁纸服务处理完成一个壁纸窗口偏移位置变化事件通知。一旦壁纸服务处理完成该事件通知，WindowManagerService类的成员变量mWaitingOnWallpaper的值就会被设置为null。

壁纸服务处理壁纸窗口在X轴和Y轴上的偏移位置变化事件通知的过程就如图4的Step 6至Step 8所示。

至此，我们就分析完成壁纸窗口在X轴和Y轴上的偏移位置的调整过程了，接下来我们就继续分析壁纸窗口在窗口堆栈中的位置调整过程。

三. 调整壁纸窗口在窗口堆栈中的位置

调整壁纸窗口在窗口堆栈中的位置实际上就是将壁纸窗口放置在需要显示壁纸的窗口的下面，这是通过调用WindowManagerService类的成员函数adjustWallpaperWindowsLocked来实现的。

WindowManagerService类的成员函数adjustWallpaperWindowsLocked的实现框架如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    int adjustWallpaperWindowsLocked() {
        int changed = 0;

        final int dw = mDisplay.getWidth();
        final int dh = mDisplay.getHeight();

        // First find top-most window that has asked to be on top of the
        // wallpaper; all wallpapers go behind it.
        final ArrayList<WindowState> localmWindows = mWindows;
        int N = localmWindows.size();
        WindowState w = null;
        WindowState foundW = null;
        int foundI = 0;
        WindowState topCurW = null;
        int topCurI = 0;
        int i = N;
        //Label #1:
        while (i > 0) {
            //从上到下遍历窗口堆栈，查找需要显示壁纸的窗口foundW，foundI为窗口foundW在窗口堆栈中
            //的位置如果没有找到需要显示壁纸的窗口，并且系统中存在壁纸窗口，那么topCurW就指向Z轴
            //位置最大的壁纸窗口，topCurI为窗口topCurW在窗口堆栈中的位置，这时候foundW一定等于
            //null。
            .....
        }

        //Label #2:
        if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
            //如果系统当前正在窗口切换的过程中，并且系统当前存在一个需要显示壁纸的Activity窗口，
            //那么就认为当前正在执行的窗口切换涉及到了这个需要显示壁纸的Activity窗口，
            //因此，就暂时不要调整壁纸窗口的位置了，等到窗口切换过程完成了再说。
            //系统当前存在一个需要显示壁纸的Activity窗口，意味着mWallpaperTarget不等于null，
            //或者foundW不等于null。
            .....
        }

        if (mWallpaperTarget != foundW) {
```

```

//上一次显示壁纸的窗口和接下来要显示壁纸的窗口发生了变化

mLowerWallpaperTarget = null;
mUpperWallpaperTarget = null;

WindowState oldW = mWallpaperTarget;
mWallpaperTarget = foundW;

// Now what is happening... if the current and new targets are
// animating, then we are in our super special mode!
if (foundW != null && oldW != null) {
    boolean oldAnim = oldW.mAnimation != null
        || (oldW.mAppToken != null && oldW.mAppToken.animation != null);
    boolean foundAnim = foundW.mAnimation != null
        || (foundW.mAppToken != null && foundW.mAppToken.animation != null);
    .....

    //Label #3:
    if (foundAnim && oldAnim) {
        //上一次显示壁纸的窗口oldW和接下来要显示壁纸的窗口foundW正在显示动画的
        //过程中, 那么就将Z轴位置较高的窗口保存在mUpperWallpaperTarget中, 而将
        //Z轴位置较低的窗口保存在mLowerWallpaperTarget中, 并且将变量foundW指向
        //Z轴位置较高的窗口, 这样就可以在这两个窗口的动画显示过程中都能看到壁
        //纸窗口.
        .....
    }
}
} else if (mLowerWallpaperTarget != null) {
    //Label #4:
    //检查mUpperWallpaperTarget和mLowerWallpaperTarget所指向的窗口的动画显示过程
    //是否已经结束, 如果已经结束, 那么就将mUpperWallpaperTarget和
    //mLowerWallpaperTarget的值置null。
    // Is it time to stop animating?
    .....
}

boolean visible = foundW != null;
//Label #5:
if (visible) {
    //前面找到了一个需要显示壁纸的窗口foundW, 并且存在其它窗口与它关联, 这些关联的
    //窗口包括:
    //1. 在与该窗口所对应的窗口令牌的其它窗口
    //2. 该窗口所设置的启动窗口
    //3. 附加在该窗口的其它窗口
    //在上述这些关联的窗口中, 如果存在一些Z轴位置比窗口foundW小, 那么就需要将壁纸
    //窗口放在Z轴位置最小的那个窗口下面, 即将变量foundW指向Z轴位置最小的那个窗口。
    .....
}

//让变量foundW指向前面找到的需要显示壁纸的窗口的下一个窗口,
//这时候变量foundI记录的仍是需要显示壁纸的窗口在窗口堆栈中的位置,
//接下来会根据这两个变量来调整壁纸窗口在窗口堆栈中的位置
if (foundW == null && topCurW != null) {
    //前面提到, 如果没有找到需要显示壁纸的窗口, 并且系统中存在壁纸窗口, 那么foundW一
    //定等于null, 并且topCurW一定不等于null, 这时候就不需要调整壁纸窗口在窗口堆栈中的
    //位置。为了与其它情况统一处理, 这时候假设位于壁纸窗口上面的那个窗口就是需要显示
    //壁纸的窗口。因此, 就会将foundI的值设置为(topCurI+1), 而将foundW的值设置为
    //topCurW。
    // There is no wallpaper target, so it goes at the bottom.
    // We will assume it is the same place as last time, if known.
    foundW = topCurW;
    foundI = topCurI+1;
} else {
    //前面找到了需要显示壁纸的窗口, 因此, 就将它的下一个窗口保存在foundW中, 变量foundI
    //的值不需要修改。
    // Okay i is the position immediately above the wallpaper. Look at
    // what is below it for later.
    foundW = foundI > 0 ? localmWindows.get(foundI-1) : null;
}
}

```



```

//如果前面找到的需要显示壁纸的窗口是可见的，并且当前正在显示壁纸的窗口设置了壁纸窗口
//在X轴和Y轴上的偏移位置，那么就将用来描述壁纸窗口在X轴和Y轴上的偏移位置的WallpaperX、
//WallpaperY、WallpaperXStep和WallpaperYStep值记录在mLastWallpaperX、
//mLastWallpaperXStep、mLastWallpaperY和mLastWallpaperYStep中。
if (visible) {
    if (mWallpaperTarget.mWallpaperX >= 0) {
        mLastWallpaperX = mWallpaperTarget.mWallpaperX;
        mLastWallpaperXStep = mWallpaperTarget.mWallpaperXStep;
    }
    if (mWallpaperTarget.mWallpaperY >= 0) {
        mLastWallpaperY = mWallpaperTarget.mWallpaperY;
        mLastWallpaperYStep = mWallpaperTarget.mWallpaperYStep;
    }
}

//Label #6:
// Start stepping backwards from here, ensuring that our wallpaper windows
// are correctly placed.
int curTokenIndex = mWallpaperTokens.size();
while (curTokenIndex > 0) {
    //一切准备就绪，开始调整系统中的壁纸窗口在窗口堆栈的位置，算法如下所示。
    //对于从Z轴位置从高到低的每一个壁纸窗口wallpaper:
    //1. 如果它与变量foundW指向的不是同一个壁纸窗口，那么就说明它在窗口堆栈中
    //的位置不对，这时候就需要将它调整到窗口堆栈中的第foundI个位置上。
    //2. 如果它与变量foundW指向的是同一个壁纸窗口，那么就说明它在窗口堆栈中的
    //位置是正确的，这时候就不需要对它进行调整，不过要让变量foundI的值减1，并且将
    //在窗口堆栈第(foundI - 1)个位置的窗口记录在变量foundW中。
    //注意，变量foundW一开始就指向Z轴位置最高的壁纸窗口，而变量foundI记录的是
    //位于Z轴位置最高的壁纸窗口上面的那个窗口在窗口堆栈中的位置。
    //上述算法实际上是用状态机的方法将系统中的所有壁纸窗口（假设数量为N）按照Z轴
    //位置从高到底的顺序放置在窗口堆栈中的第(foundI - 1)、(foundI - 2)、
    //(foundI - 3)、.....、(foundI - N)个位置上。
    .....
}

return changed;
}

.....

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的成员函数adjustWallpaperWindowsLocked是按照以下流程来调整壁纸窗口在窗口堆栈中的位置的：

1. 通过一个while循环来从上到下地遍历窗口堆栈，找到需要显示壁纸的窗口foundW，其中，foundI为窗口foundW在窗口堆栈中的位置。如果没有找到需要显示壁纸的窗口，并且系统中存在壁纸窗口，那么topCurW就指向Z轴位置最大的壁纸窗口，其中，topCurI为窗口topCurW在窗口堆栈中的位置。在这种情况下，变量foundW的值一定等于null的。
2. 如果WindowManagerService类的成员变量mNextAppTransition的值不等于WindowManagerPolicy.TRANSIT_UNSET，那么就说明系统当前正在窗口切换的过程中。在这种情况下，如果系统当前存在一个需要显示壁纸的Activity窗口，即WindowManagerService类的成员变量mWallpaperTarget的值不等于null，或者前面得到的变量foundW的值不等于null，那么就认为当前正在执行的窗口切换操作涉及到了这个需要显示壁纸的Activity窗口。这时候就不需要调整壁纸窗口的位置，要等到窗口切换过程完成了之后再调整。
3. 如果WindowManagerService类的成员变量mWallpaperTarget和前面得到的变量foundW指向的不是同一个WindowState对象，那么就说明上一次显示壁纸的窗口和接下来要显示壁纸的窗口发生了变化。在这种情况下，就会使用变量oldW来描述上一次显示壁纸的窗口，而接下来要显示壁纸的窗口通过WindowManagerService类的成员变量mWallpaperTarget以及变量foundW来描述。这时候如果检查发现上一次显示壁纸的窗口和接下来要显示壁纸的窗口都处于显示动画的过程中，那么就会将Z轴位置较高的窗口保存在WindowManagerService类的成员变量mUpperWallpaperTarget中，而将Z轴位置较低的窗口保存在WindowManagerService类的成员变量mLowerWallpaperTarget中，并且将变量foundW指向Z轴位置较高的窗口。这样就能保证在这两个窗口的动画显示过程中

都能看到壁纸窗口，实际上就是保证在两个窗口的切换过程中看到壁纸窗口。

4. 如果WindowManagerService类的成员变量mWallpaperTarget和前面得到的变量foundW指向的是同一个WindowState对象，并且WindowManagerService类的成员变量mLowerWallpaperTarget的值不等于null，那么就说明需要检查系统的窗口切换过程完成了没有。如果已经完成，那么就需要将WindowManagerService类的成员变量mUpperWallpaperTarget和mLowerWallpaperTarget的值设置为null。由此可以，WindowManagerService类的成员变量mUpperWallpaperTarget和mLowerWallpaperTarget的作用就是用来记录两个处于切换状态的需要显示壁纸的窗口。
5. 如果变量foundW的值不等于null，那么就说明前面找到了一个接下来要显示壁纸的窗口。在这种情况下，需要做两件事情。第一件事情是判断接下来要显示壁纸的窗口是否是可见的。如果是的话，那么就会将变量visible的值设置为true。第二件事情是在与接下来要显示壁纸的窗口相关联的窗口中，即与变量foundW所描述的窗口相关联的窗口中，找到一个Z轴位置最小的窗口，因为壁纸窗口最终是要放置在这个Z轴位置最小的窗口的下面，而不是最初找到的那个窗口的下面。与变量foundW所描述的窗口相关联的窗口包括：A. 与变量foundW所描述的窗口具有相同窗口令牌的其它窗口；B. 与变量foundW所描述的窗口附加在同一个窗口的其它窗口；C. 为变量foundW所描述的窗口所设置的启动窗口；D. 附加在变量foundW所描述的窗口上的其它窗口。一旦找到这样的窗口，那么就会让重新让变量foundW指向它。
6. 再次重新调整变量foundW的值，让它指向位于前面所找到的需要显示壁纸的窗口的下面的一个窗口。注意，这个窗口有可能就是壁纸窗口。这时候变量foundI记录的然是前面所找到的需要显示壁纸的窗口在窗口堆栈中的位置。这样做的目的是为了接下来可以方便地调整壁纸窗口在窗口堆栈中的位置。但是如果变量foundW的值等于null，那么就说明前面根本没有找到需要显示壁纸的窗口。在这种情况下，如果变量topCurW的值不等于null，那么就说明系统中存在壁纸窗口。这种情况其实就不需要调整壁纸窗口在窗口堆栈中的位置了，但是为了接下来的逻辑可以统一处理，就假定位于壁纸窗口上面的那个窗口是需要显示壁纸的窗口。因此，就会将变量foundI的值设置为(topCurI+1)，而将变量foundW的值设置为topCurW。
7. 如果前面所找到的需要显示壁纸的窗口是可见的，即变量visible的值等于true，并且当前正在显示壁纸的窗口设置了壁纸窗口在X轴和Y轴上的有效偏移位置，即WindowManagerService类的成员变量mWallpaperTarget所指向的一个WindowState对象的成员变量mWallpaperX和mWallpaperY的值大于等于0，那么就将用来描述壁纸窗口在X轴和Y轴上的偏移位置的WallpaperX、WallpaperY、WallpaperXStep和WallpaperYStep值记录在WindowManagerService类的成员变量mLastWallpaperX、mLastWallpaperY、mLastWallpaperXStep和mLastWallpaperYStep中。
8. 经过上面的一系列操作之后，现在一切准备就绪，因此就可以按照以下的算法来调整系统中的壁纸窗口在窗口堆栈的位置。对于从Z轴位置从高到低的每一个壁纸窗口wallpaper：(1). 如果它与变量foundW指向的不是同一个壁纸窗口，那么就说明它在窗口堆栈中的位置不对，这时候就需要将它调整到窗口堆栈中的第foundI个位置上；(2). 如果它与变量foundW指向的是同一个壁纸窗口，那么就说明它在窗口堆栈中的位置是正确的，这时候就不需要对它进行调整，不过要让变量foundI的值减1，并且将在窗口堆栈第(foundI - 1)个位置的窗口记录在变量foundW中；(3). 重复执行第(1)和第(2)步的操作，直到系统所有的壁纸窗口都检查完成为止。注意，在上述算法中，变量foundW一开始就指向Z轴位置最高的壁纸窗口，而变量foundI记录的是位于Z轴位置最高的壁纸窗口上面的那个窗口在窗口堆栈中的位置。每当Z轴位置最高的壁纸窗口在窗口堆栈中的位置调整完成之后，变量foundW就会指向Z轴位置次高的壁纸窗口，而变量foundI的值也会相应地减少1。这个算法其实就是用状态机的方法来将系统中的所有壁纸窗口（假设数量为N）按照Z轴位置从高到底的顺序放置在窗口堆栈中的第(foundI - 1)、(foundI - 2)、(foundI - 3)、.....、(foundI - N)个位置上。

上述流程可能还是比较抽象，接下来我们就通过在标号为Label #1、Label #2、Label #3、Label #4、Label #5和Label #6处所忽略的代码来详细分析壁纸窗口在窗口堆栈中的位置的调整过程。

标号为Label #1的代码如下所示：

```
while (i > 0) {  
    i--;
```

```

w = localmWindows.get(i);
if ((w.mAttrs.type == WindowManager.LayoutParams.TYPE_WALLPAPER)) {
    if (topCurW == null) {
        topCurW = w;
        topCurI = i;
    }
    continue;
}
topCurW = null;
if (w.mAppToken != null) {
    // If this window's app token is hidden and not animating,
    // it is of no interest to us.
    if (w.mAppToken.hidden && w.mAppToken.animation == null) {
        .....
        topCurW = null;
        continue;
    }
}
.....

if ((w.mAttrs.flags&FLAG_SHOW_WALLPAPER) != 0 && w.isReadyForDisplay()
    && (mWallpaperTarget == w
        || (!w.mDrawPending && !w.mCommitDrawPending))) {
    .....

    foundW = w;
    foundI = i;
    if (w == mWallpaperTarget && ((w.mAppToken != null
        && w.mAppToken.animation != null)
        || w.mAnimation != null)) {
        // The current wallpaper target is animating, so we'll
        // look behind it for another possible target and figure
        // out what is going on below.
        .....
        continue;
    }
    break;
}
}
}

```

这段代码从上到下遍历保存在窗口堆栈中的窗口，目的是要找到一个Z轴位置最大的并且需要显示壁纸的窗口。一个窗口如果需要显示壁纸，那么用来描述它的一个WindowState对象w的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的值的FLAG_SHOW_WALLPAPER位就不等于0。一个需要显示壁纸的窗口只有准备就绪显示并且UI也已经绘制完成之后，WindowManagerService服务才会将壁纸窗口放置在它的下面。一个需要显示壁纸的窗口如果已经准备就绪显示，那么用来描述它的一个WindowState对象w的成员函数isReadyForDisplay的返回值等于true。另一方面，如果一个窗口的UI还没有绘制，那么用来描述它的一个WindowState对象w的成员变量mDrawPending的值就会等于true。一个窗口的UI虽然绘制好了，但是还没有提交给SurfaceFlinger服务处理，即用来描述它的一个WindowState对象w的成员变量mCommitDrawPending的值等于true，那么它的UI也是认为还没有绘制完成的。

在遍历的过程中，如果发现一个窗口w刚好就是当前正在显示壁纸的窗口mWallpaperTarget，那么就会继续检查该窗口是否正处于显示动画的过程中。如果是的话，那么就需要跳过该窗口，因为我们的目标是要找到另外一个接下来要显示壁纸的窗口。对于Activity窗口和非Activity窗口来说，判断它们是否是正处于显示动画的过程中的方法是不一样的。对于一个处于显示动画过程的Activity窗口来说，用来描述它的一个WindowState对象w的成员变量mAppToken的值不等于null，并且指向了一个AppWindowToken对象，并且这个AppWindowToken对象的成员变量animation的值不等于null。对于一个处于显示动画过程的非Activity窗口来说，用来描述它的一个WindowState对象w的成员变量mAnimation的值不等于null。这就是说，AppWindowToken类的成员变量animation和WindowState类的成员变量mAnimation都是用来描述一个动画对象的。

在遍历的过程中，有两种类型的窗口是需要跳过的。第一种类型的窗口是壁纸窗口，即用来描述它的一个WindowState对象w的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量type的值等于WindowManager.LayoutParams.TYPE_WALLPAPER。第二种类型的窗口是Activity窗口，但是与之所对应的Activity组件处于不可见状态，这意味着这种类型的窗口也是不可见的。前面提到，对于Activity窗口来说，用来描述它的一个WindowState对象w的成员变量mAppToken的值是不等于null的，并且指向了一个AppWindowToken对象。当这个AppWindowToken对象的成员变量hidden的值等于true的时候，就意味着对应的Activity组件是不可见的。有时候一个AppWindowToken对象的成员变量hidden的值虽然等于true，但是如果这个AppWindowToken对象的成员变量animation的值不等于null，那么隐含着对应的Activity组件其实还是可见的，因为它还处于显示动画的过程中。

遍历完成之后，有可能找到了接下来要显示壁纸的窗口，也有可能找不到接下来要显示壁纸的窗口。

如果找到了接下来要显示壁纸的窗口，那么变量foundW的值就不等于null，并且指向了这个接下来要显示壁纸的窗

口，另外一个变量foundI记录的是该窗口在窗口堆栈中位置。这时候变量topCurW的值一定等于null，但是变量topCurI的值却不一定等于0，它有可能指向了Z轴位置最大的那个壁纸窗口。

假设foundW的值不等于null，并且变量topCurI的值等于0.，那么窗口堆栈的状态就如图5所示：

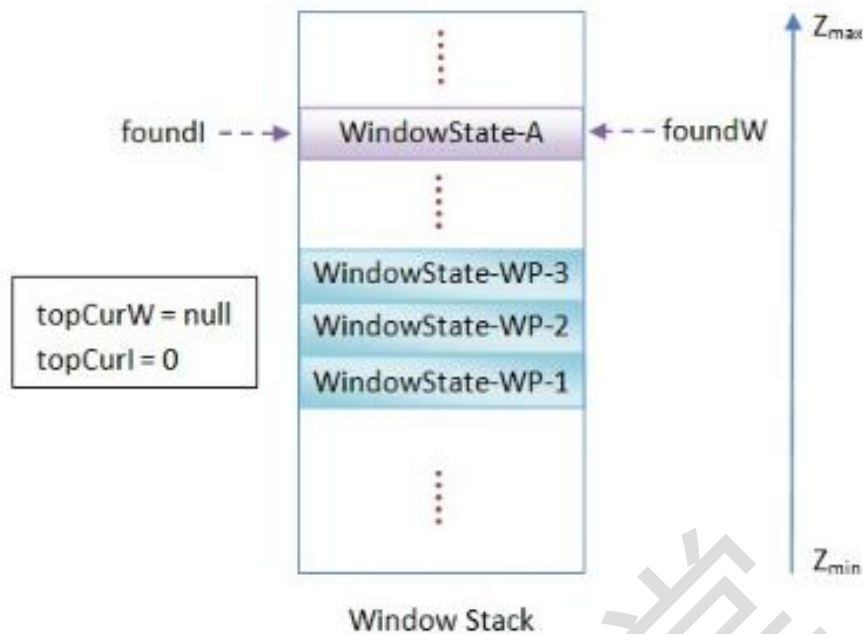


图5 $\text{foundW} \neq \text{null} \ \& \ \text{topCurI} == 0$

假设foundW的值不等于null，并且变量topCurI的值大于0.，那么窗口堆栈的状态就如图6所示：

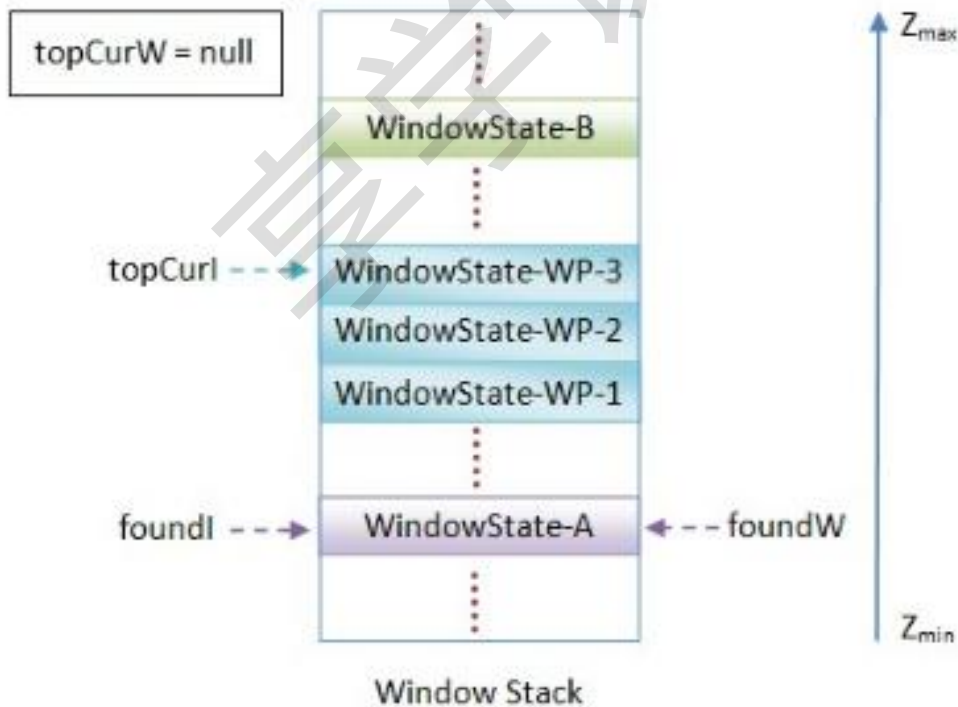


图6 $\text{foundW} \neq \text{null} \ \& \ \text{topCurI} \neq 0$

如果没有找到接下来要显示壁纸的窗口，那么变量foundW的值就等于null，并且另外一个变量foundI的值等于0。这时候变量topCurW的值始终等于null，而变量topCurI的值可能不等于0，取决于系统中是否存在壁纸窗口。

为了方便描述，我们假设系统中是存在壁纸窗口，那么这时候topCurI的值就不等于0，并且它记录的是Z轴位置最大的那个壁纸窗口在窗口堆栈中的位置，如图7所示：

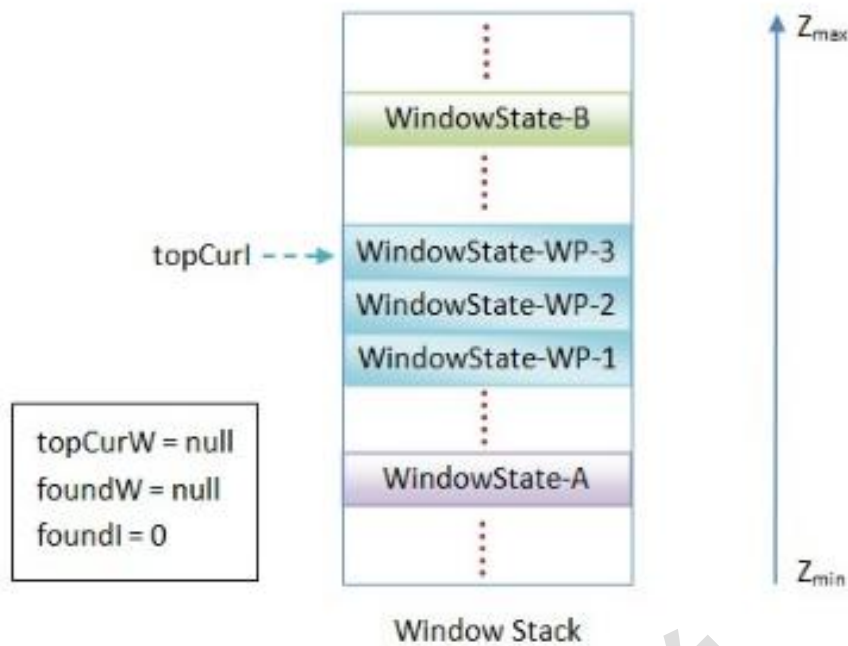


图7 foundW == null && topCurI != 0

标号为Label #2的代码如下所示:

```
if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
    // If we are currently waiting for an app transition, and either
    // the current target or the next target are involved with it,
    // then hold off on doing anything with the wallpaper.
    // Note that we are checking here for just whether the target
    // is part of an app token... which is potentially overly aggressive
    // (the app token may not be involved in the transition), but good
    // enough (we'll just wait until whatever transition is pending
    // executes).
    if (mWallpaperTarget != null && mWallpaperTarget.mAppToken != null) {
        .....
        return 0;
    }
    if (foundW != null && foundW.mAppToken != null) {
        .....
        return 0;
    }
}
```

WindowManagerService类的成员变量mNextAppTransition的值不等于WindowManagerPolicy.TRANSIT_UNSET意味着系统当前正在窗口切换的过程中。这里说的窗口切换其实就是由Activity组件切换引起的，即切换的是Activity窗口。如果正在切换的Activity窗口是需要显示壁纸的，那么WindowManagerService类的成员函数adjustWallpaperWindowsLocked就要等到切换过程结束后，才能调整重新调整壁纸窗口在窗口堆栈中的位置。这里本来是要判断正在发生切换的Activity窗口是否是当前壁纸窗口的目标窗口或者前面所找到的接下来要显示壁纸的窗口的，但是却没有这样做。这段代码采取了一种比较激进的方法，即主要发现当前壁纸窗口的目标窗口是一个Activity窗口，或者前面所找到的接下来要显示壁纸的窗口是一个Activity窗口，那么就认为当前正在执行的窗口切换过程涉及到了壁纸窗口，因此，就要等到切换过程结束后，再来重新调整壁纸窗口在窗口堆栈中的位置。

WindowManagerService类的成员变量mWallpaperTarget描述的就是当前壁纸窗口的目标窗口，当它的值不等于null时，并且它所指向的一个WindowState对象的成员变量mAppToken的值不等于null，那么就说明当前壁纸窗口的目标窗口是一个Activity窗口。同样，如果前面得到的变量foundW的值不等于null，并且它所指向的一个WindowState对象的成员变量mAppToken的值不等于null，那么就说明前面所找到的接下来要显示壁纸的窗口是一个Activity窗口。

标号为Label #3的代码如下所示:

```
if (foundAnim && oldAnim) {
    int oldI = localmWindows.indexOf(oldW);
    .....

    if (oldI >= 0) {
        .....
    }
}
```

```

// Set the new target correctly.
if (foundW.mAppToken != null && foundW.mAppToken.hiddenRequested) {
    .....
    mWallpaperTarget = oldW;
}

// Now set the upper and lower wallpaper targets
// correctly, and make sure that we are positioning
// the wallpaper below the lower.
if (foundI > oldI) {
    // The new target is on top of the old one.
    .....
    mUpperWallpaperTarget = foundW;
    mLowerWallpaperTarget = oldW;
    foundW = oldW;
    foundI = oldI;
} else {
    // The new target is below the old one.
    .....
    mUpperWallpaperTarget = oldW;
    mLowerWallpaperTarget = foundW;
}
}
}

```

当变量foundAnim和oldAnim的值均等于true的时候，就说明当前正在显示壁纸的窗口oldW和接下来要显示壁纸的窗口foundW均处于显示动画的过程中，那么就分别将它们记录在WindowManagerService类的成员变量mLowerWallpaperTarget和mUpperWallpaperTarget中，其中，前者用来描述Z轴位置较低的窗口，而后者用来描述Z轴位置较高的窗口。变量foundI和oldI记录的分别是窗口foundW和oldW在窗口堆栈中的位置。因此，当变量foundI的值大于变量oldI的值的时候，窗口foundW就是Z轴位置较高的窗口，而窗口oldW就是Z轴位置较低的窗口。相反，当变量foundI的值小于等于变量oldI的值的时候，窗口oldW就是Z轴位置较高的窗口，而窗口foundW就是Z轴位置较低的窗口。

这里有三个地方是需要注意的：

1. 当前正在显示壁纸的窗口oldW其实就是WindowManagerService类的成员变量mWallpaperTarget所描述的那个窗口。
2. 变量foundW和foundI记录的始终都是Z轴位置较低的那个窗口及其在窗口堆栈的位置，因此，当变量foundI的值大于变量oldI的值的时候，要将变量foundW和foundI的值分别设置为oldW和oldI，这样做的目的是为了接下来可以将壁纸窗口放置在Z轴位置较低的窗口的下面，以便可以在两个窗口的动画显示过程中看到壁纸。
3. 如果前面找到的接下来要显示壁纸的窗口是一个Activity窗口，即变量foundW所描述的一个WindowState对象的成员变量mAppToken的值不等于null，并且它所指向的一个AppWindowToken对象的成员变量hiddenRequested的值等于true，那么就说明与窗口foundW所对应的一个Activity组件已经被请求隐藏起来了。在这种情况下，当前正在显示壁纸的窗口就会仍然被当作是接下来壁纸窗口的目标窗口。由于此前我们已经将WindowManagerService类的成员变量mWallpaperTarget的值设置了为foundW，因此，这时候就需要将它的值修改为oldW。

这段代码执行完成之后，窗口堆栈的状态就如图8所示：

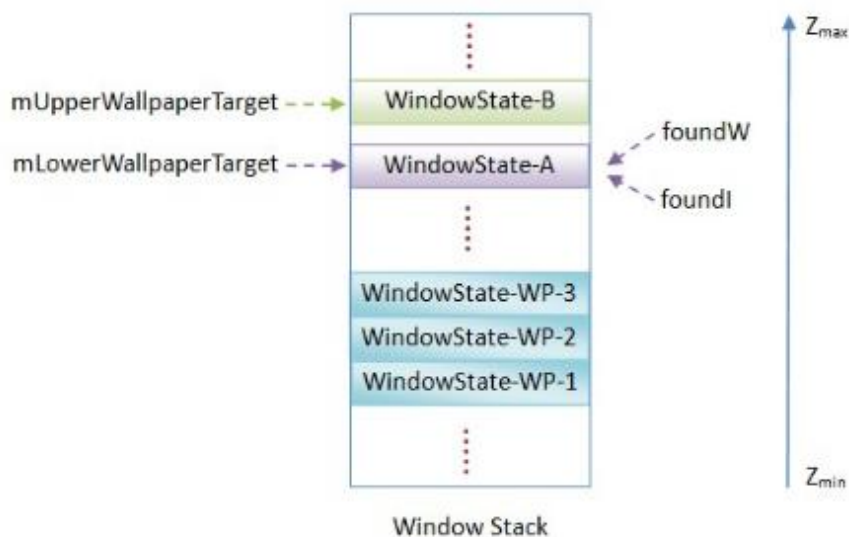


图8 mUpperWallpaperTarget、mLowerWallpaperTarget、foundW和foundI的关系

标号为Label #4的代码如下所示：

```
// Is it time to stop animating?
boolean lowerAnimating = mLowerWallpaperTarget.mAnimation != null
    || (mLowerWallpaperTarget.mAppToken != null
        && mLowerWallpaperTarget.mAppToken.animation != null);
boolean upperAnimating = mUpperWallpaperTarget.mAnimation != null
    || (mUpperWallpaperTarget.mAppToken != null
        && mUpperWallpaperTarget.mAppToken.animation != null);
if (!lowerAnimating || !upperAnimating) {
    .....
    mLowerWallpaperTarget = null;
    mUpperWallpaperTarget = null;
}
```

这段代码检查WindowManagerService类的成员变量mLowerWallpaperTarget和mUpperWallpaperTarget所描述的两个窗口的动画是否已经显示结束。如果已经显示结束，那么就会将这两个成员变量的值设置为null。注意，如果一个窗口的动画已经显示结束，那么用来描述它的一个WindowState对象的成员变量mAnimation的值就会等于null。另外，如果一个Activity窗口的动画已经显示结束，那么用来描述它的WindowState对象的成员变量mAppWindowToken所指向的一个AppWindowToken对象的成员变量animation的值也会等于null。

标号为Label #5的代码如下所示：

```
boolean visible = foundW != null;
if (visible) {
    // The window is visible to the compositor... but is it visible
    // to the user? That is what the wallpaper cares about.
    visible = isWallpaperVisible(foundW);
    .....

    // If the wallpaper target is animating, we may need to copy
    // its layer adjustment. Only do this if we are not transferring
    // between two wallpaper targets.
    mWallpaperAnimLayerAdjustment =
        (mLowerWallpaperTarget == null && foundW.mAppToken != null)
        ? foundW.mAppToken.animLayerAdjustment : 0;

    final int maxLayer = mPolicy.getMaxWallpaperLayer()

* TYPE_LAYER_MULTIPLIER

+ TYPE_LAYER_OFFSET;

    // Now w is the window we are supposed to be behind... but we
    // need to be sure to also be behind any of its attached windows,
    // AND any starting window associated with it, AND below the
    // maximum layer the policy allows for wallpapers.
    while (foundI > 0) {
        WindowState wb = localmWindows.get(foundI-1);
        if (wb.mBaseLayer < maxLayer &&
            wb.mAttachedWindow != foundW &&
            wb.mAttachedWindow != foundW.mAttachedWindow &&
            (wb.mAttrs.type != TYPE_APPLICATION_STARTING ||
             wb.mToken != foundW.mToken)) {
            // This window is not related to the previous one in any
            // interesting way, so stop here.
            break;
        }
        foundW = wb;
        foundI--;
    }
}
```

当变量foundW的值不等于null时，就说明前面找到了一个接下来要显示壁纸的窗口。在这种情况下，需要做三件事：

1. 判断窗口foundW是否是可见的，这是通过调用WindowManagerService类的成员函数isWallpaperVisible来实现的。如果可见，那么变量visible的值就会等于true，否则就会等于false。后面在调整壁纸窗口在窗口堆栈中的位置时，会根据变量visible的值来决定要显示壁纸窗口还是隐藏壁纸窗口。
2. 检查窗口foundW是否是一个Activity窗口。如果是的话，那么就会将用来描述它的一个

WindowState对象的成员变量mAppToken所指向的一个AppWindowToken对象的成员变量animLayerAdjustment的值保存在WindowManagerService类的成员变量mWallpaperAnimLayerAdjustment中。在计算壁纸窗口的Z轴位置的时候，需要使用到WindowManagerService类的成员变量mWallpaperAnimLayerAdjustment，用来调整壁纸窗口的Z轴位置。在后面一篇文章分析窗口的Z轴位置的计算方法时，我们再详细分析壁纸窗口的Z轴位置是如何计算的。注意，如果这时候系统的壁纸窗口有两个目标窗口，即WindowManagerService类的成员变量mLowerWallpaperTarget的值不等于null，那么就说明壁纸窗口的目标窗口正在显示动画的过程中。在这种情况下，就不需要调整壁纸窗口的Z轴位置，即会将WindowManagerService类的成员变量mLowerWallpaperTarget的值设置为0。等到壁纸窗口的目标窗口结束动画显示过程之后，再来调整它的Z轴位置。

3. 检查窗口foundW的下面是否存在一些关联的窗口。如果存在的话，就需要将壁纸窗口放置在这些关联的窗口中Z轴位置最低的窗口的下面。这段代码通过一个while循环从窗口foundW的下面一个窗口开始往下检查，直到找到一个没有关联的窗口为止。在检查的过程中，每碰到一个关联的窗口，那么就让变量foundW指向它，并且将变量foundI的值减少1。这样最终得到的变量foundW和foundI就是用来描述与窗口foundW有联的、Z轴位置最低的窗口及其在窗口堆栈中的位置。

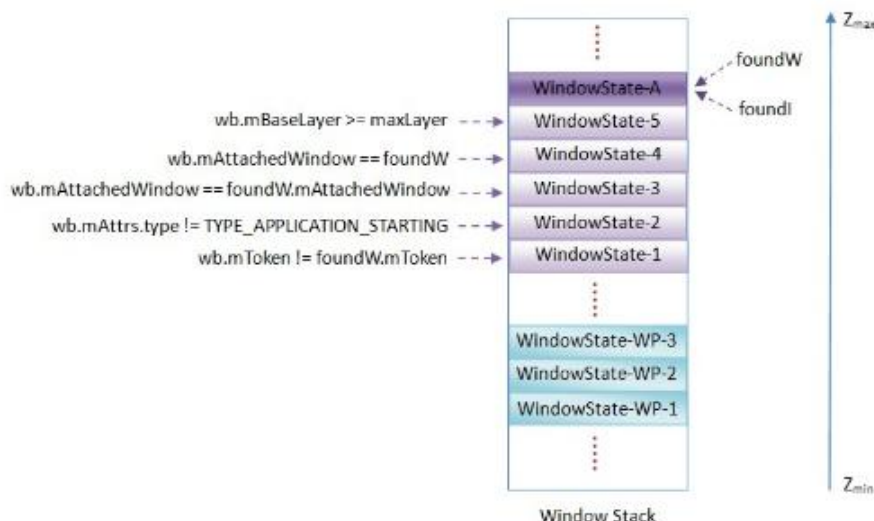
前面提到，窗口foundW所关联的窗口四种，即对于一个窗口wb来，如果它满足以下四个条件，那么它就与窗口foundW有关联：

- A. 窗口wb与窗口foundW对应的是同一个窗口令牌，即分别用来描述窗口wb和窗口foundW的两个WindowState对象的成员变量mToken指向的是同一个WindowToken对象。
- B. 窗口wb附加在窗口foundW上，即用来描述窗口wb的一个WindowState对象的成员变量mAttachedWindow与变量foundW指向的是同一个WindowState对象。
- C. 窗口wb与窗口foundW附加在同一个窗口上，即分别用来描述窗口wb和窗口foundW的两个WindowState对象的成员变量mAttachedWindow指向的是同一个WindowState对象。
- D. 窗口wb是窗口foundW的启动窗口，即用来描述窗口wb的一个WindowState对象的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量type的值等于TYPE_APPLICATION_STARTING。

此外，WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象会规定系统中的壁纸窗口的Z轴位置不能大于某一个值，也就是说，壁纸窗口的Z轴位置有一个最大值限制。这个限制值可以通过调用WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数getMaxWallpaperLayer来获得。获得了这个限制值之后，还需要乘以一个窗口类型因子TYPE_LAYER_MULTIPLIER，最后再加一个窗口类型偏移值TYPE_LAYER_OFFSET，就可以得到壁纸窗口的最大Z轴位置限制值maxLayer。这时候如果在窗口foundW的下面找到一个窗口wb，它的Z轴位置大于等于maxLayer，即用来描述它的一个WindowState对象的成员变量mBaseLayer的值大于maxLayer，那么也会认为窗口wb是与窗口foundW有关联的。

我们通过图9和图10来说明查找与窗口foundW关联的、Z轴位置最小的窗口的过程：

图9 查找与窗口foundW关联的窗口之前



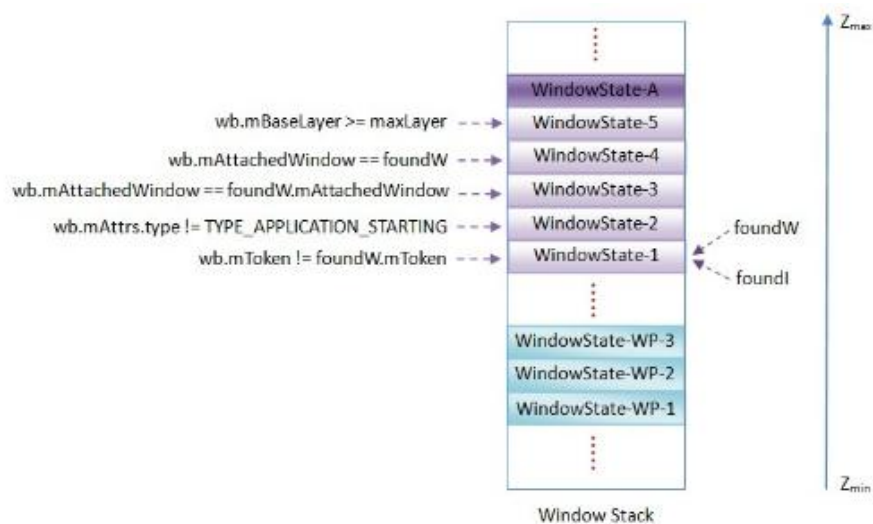


图10 查找与窗口foundW关联的窗口之后

标号为Label #6的代码如下所示:

```
// Start stepping backwards from here, ensuring that our wallpaper windows
// are correctly placed.
int curTokenIndex = mWallpaperTokens.size();
while (curTokenIndex > 0) {
    curTokenIndex--;
    WindowToken token = mWallpaperTokens.get(curTokenIndex);
    if (token.hidden == visible) {
        changed |= ADJUST_WALLPAPER_VISIBILITY_CHANGED;
        token.hidden = !visible;
        // Need to do a layout to ensure the wallpaper now has the
        // correct size.
        mLayoutNeeded = true;
    }

    int curWallpaperIndex = token.windows.size();
    while (curWallpaperIndex > 0) {
        curWallpaperIndex--;
        WindowState wallpaper = token.windows.get(curWallpaperIndex);

        if (visible) {
            updateWallpaperOffsetLocked(wallpaper, dw, dh, false);
        }

        // First, make sure the client has the current visibility
        // state.
        if (wallpaper.mWallpaperVisible != visible) {
            wallpaper.mWallpaperVisible = visible;
            try {
                .....
                wallpaper.mClient.dispatchAppVisibility(visible);
            } catch (RemoteException e) {
            }
        }

        wallpaper.mAnimLayer = wallpaper.mLayer + mWallpaperAnimLayerAdjustment;
        .....

        // First, if this window is at the current index, then all
        // is well.
        if (wallpaper == foundW) {
            foundI--;
            foundW = foundI > 0
                ? localmWindows.get(foundI-1) : null;
            continue;
        }

        // The window didn't match... the current wallpaper window,
```

```

// wherever it is, is in the wrong place, so make sure it is
// not in the list.
int oldIndex = localmWindows.indexOf(wallpaper);
if (oldIndex >= 0) {
    i.....
    localmWindows.remove(oldIndex);
    mWindowsChanged = true;
    if (oldIndex < foundI) {
        foundI--;
    }
}

// Now stick it in.
.....

localmWindows.add(foundI, wallpaper);
mWindowsChanged = true;
changed |= ADJUST_WALLPAPER_LAYERS_CHANGED;
}
}

```

这段代码就是用来调整系统中的壁纸窗口在窗口堆栈中的位置的，目标就是要将它们放置在前面所找到的接下来要显示壁纸的窗口的下面。 WindowManagerService类的成员变量mWallpaperTokens保存的是一系列WindowToken对象，它们描述的是系统中的壁纸窗口令牌。这些WindowToken对象都有一个成员变量windows，里面保存的是一系列WindowState对象，它们描述的是系统中的壁纸窗口。这段代码就目标就要通过两个嵌套的while循环来将这些WindowState对象调整到前面所找到的接下来要显示壁纸的窗口的下面去。

在调整壁纸窗口在窗口堆栈中的位置的过程中，还会做以下四件事情：

1. 设置壁纸窗口令牌的可见性。也就是说，如果一个用来描述壁纸窗口令牌的WindowToken对象token的成员变量hidden的值不等于前面得到的变量visible的值，那么就说明该壁纸窗口令牌的可见性发生了变化。由于WindowToken类的成员变量hidden是用来表示壁纸窗口令牌的不可见状态的，而变量visible是用来表示接下来要显示壁纸的窗口是可见的，因此，当一个壁纸窗口令牌的可见性发生变化时，就要将用来描述它的WindowToken对象token的成员变量hidden的值设置为!visible。壁纸窗口令牌的可见性发生了变化之后，需要重新刷新系统的UI，因此，就需要将WindowManagerService类的成员变量mLayoutNeeded 的值设置为true，并且将函数返回值changed的ADJUST_WALLPAPERVISIBILITY_CHANGED位设置为1。
2. 在前面所找到的接下来要显示壁纸的窗口是可见的情况下，即在变量visible的值等于true的情况下，重新计算每一个壁纸窗口wallpaper在X轴和Y轴上的偏移位置，这是通过调用WindowManagerService类的成员函数updateWallpaperOffsetLocked来实现的。
3. 如果一个壁纸窗口之前是不可见的，现在变得可见了，或者之前是可见的，现在变得不可见了，具体就表现在用来描述该壁纸窗口的一个WindowState对象的成员变量mWallpaperVisible的值不等于变量visible的值，那么就需要该WindowState对象的成员变量mWallpaperVisible的值设置为visible，并且向提供该壁纸窗口的服务发送一个可见性变化事件通知。
4. 调整每一个壁纸窗口的Z轴位置。一个壁纸窗口的Z轴位置保存在用来描述它的一个WindowState对象的成员变量mLayer中，用这个成员变量的值加上前面已经计算好的壁纸窗口的Z轴位置调整值，即保存在WindowManagerService类的成员变量mWallpaperAnimLayerAdjustment中的值，就可以得到一个壁纸窗口的最终Z轴位置值，并且保存WindowState对象的成员变量mAnimLayer中。

前面在分析WindowManagerService类的成员函数adjustWallpaperWindowsLocked的实现框架时提到，在调整系统中的壁纸窗口在窗口堆栈中的位置之前，变量foundW描述的应该是Z轴位置最大的壁纸窗口，而变量foundI记录的是需要显示壁纸的窗口在窗口堆栈中的位置，如图11所示：

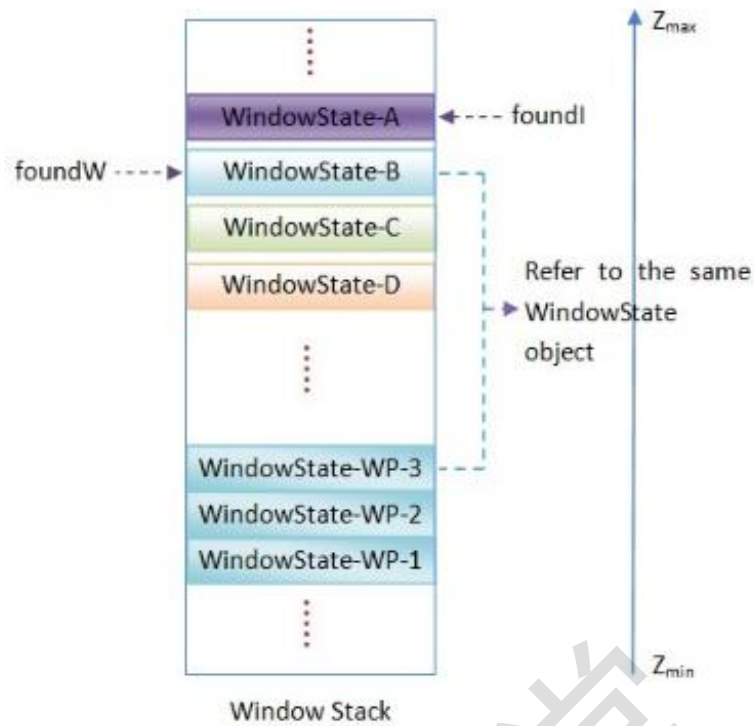


图11 调整壁纸窗口前的窗口堆栈状态

在图11中，接下来需要显示壁纸的是窗口A，在它下面依次是窗口B、C和D，并且系统中存在着三个壁纸窗口，它们的编号分别为1、2和3。假设窗口B和编号为3的壁纸窗口是同一个窗口，那么就说明编号为3的壁纸窗口已经在窗口堆栈中的正确位置了，因此，就不需要调整它在窗口堆栈中的位置了。这时候窗口堆栈中的状态如图12所示：

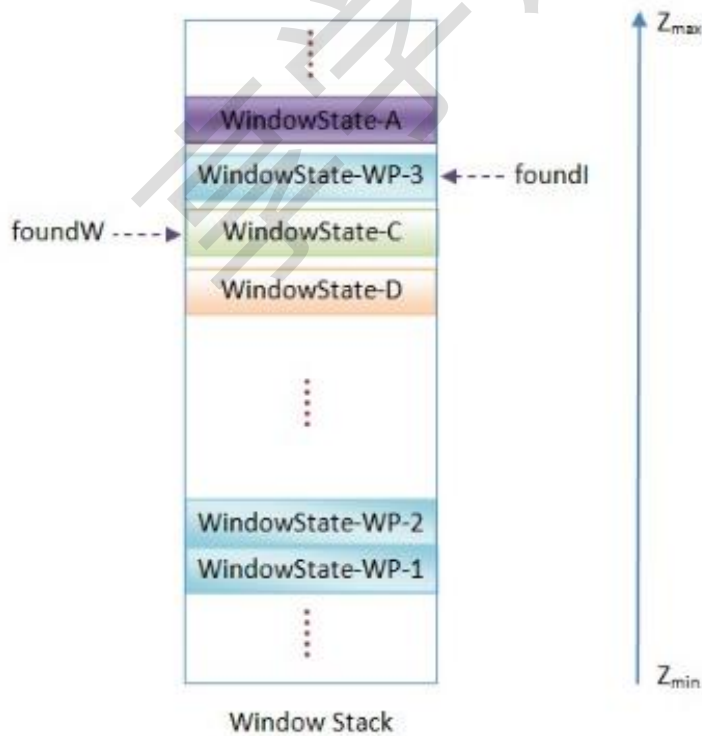


图12 处理完成编号为3的壁纸窗口后的窗口堆栈状态

在图12中，假设窗口C和编号为2的壁纸窗口不是同一个窗口，那么就需要将编号为2的壁纸窗口放置在窗口C的位置上，如图13所示：

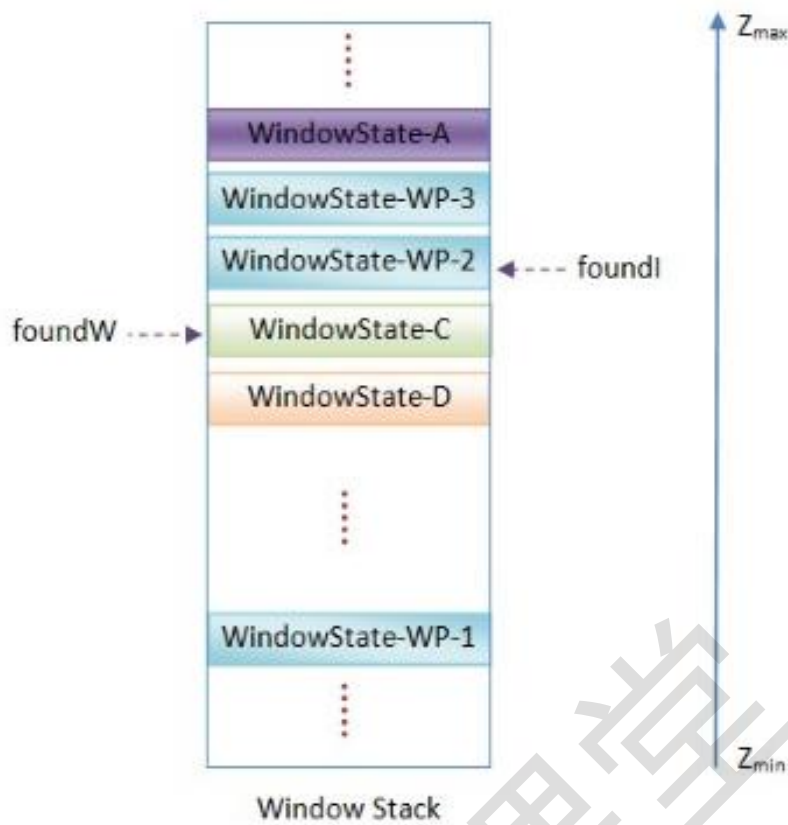


图13 处理完成编号为2的壁纸窗口后的窗口堆栈状态

在图13中，假设窗口C和编号为1的壁纸窗口也不是同一个窗口，那么就需要将编号为1的壁纸窗口放置在窗口C的位置上，如图14所示：

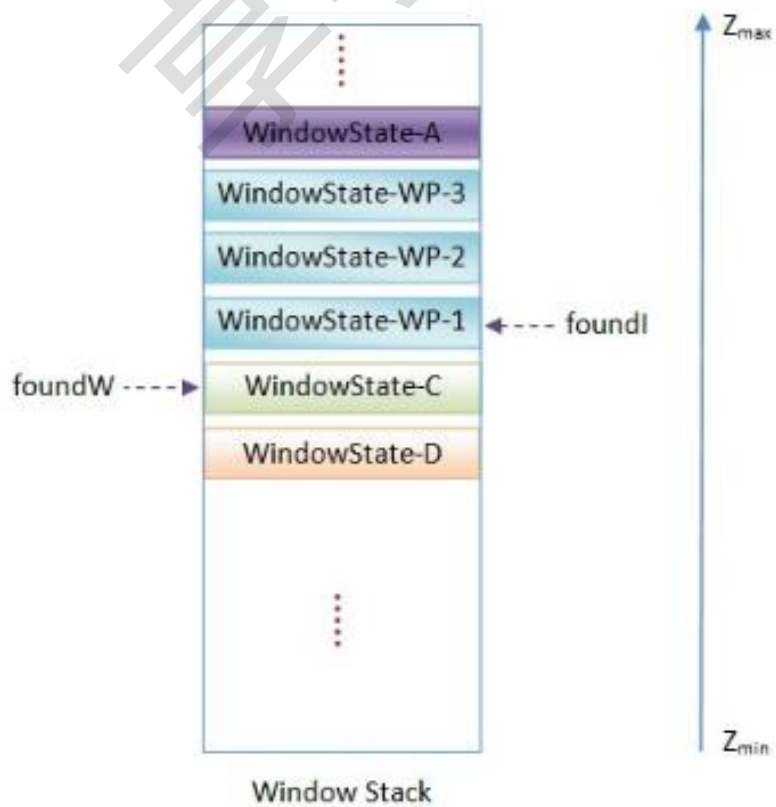


图14 处理完成编号为1的壁纸窗口的窗口堆栈状态

处理完成编号为1的壁纸窗口之后，系统中所有的壁纸窗口都调整到窗口A的下面去了，这样在下一次在刷新系统UI时，就可以将系统中的壁纸窗口作为窗口A的背景了。

至此，我们就分析完成壁纸窗口在窗口堆栈中的位置调整过程了，WindowManagerService服务对壁纸窗口的管理也分析完成了。结合前面Android窗口管理服务WindowManagerService对窗口的组织方式分析和Android窗口管理服务WindowManagerService对输入法窗口（Input Method Window）的管理分析这两篇文章，我们就可以对WindowManagerService服务在内部所维护的窗口堆栈有一个清晰的认识了。

当系统中的所有窗口都在窗口堆栈排列好之后，WindowManagerService服务就可以计算每一个窗口的Z轴坐标了，以便可以传递给SurfaceFlinger服务做可见性计算，从而正确地将系统的UI渲染出来。在接下来的一篇文章中，我们将继续分析WindowManagerService服务计算窗口的Z轴坐标的过程，敬请关注！

原文链接：<https://blog.csdn.net/Luoshengyang/article/details/8550820>

第六节Android窗口管理服务WindowManagerService计算窗口Z轴位置的过程分析

通过前面几篇文章的学习，我们知道了在Android系统中，无论是普通的Activity窗口，还是特殊的输入法窗口和壁纸窗口，它们都是被WindowManagerService服务组织在一个窗口堆栈中的，其中，Z轴位置较大的窗口排列在Z轴位置较小的窗口的上面。有了这个窗口堆栈之后，WindowManagerService服务就可以按照一定的规则计算每一个窗口的Z轴位置了，本文就详细分析这个计算过程。

基于窗口堆栈来计算窗口的Z轴位置是比较有意思的。按照一般的理解，应该是先计算好窗口的Z轴位置，然后再按照Z轴位置的大小来将各个窗口排列在堆栈中。但是，事实上，窗口是按照其它规则排列在堆栈中。这些规则与窗口的类型、创建顺序和运行状态等有关。例如，状态栏窗口总是位于堆栈的顶端，输入法窗口总是位于需要输入法的窗口的上面，而壁纸窗口总是位于需要显示壁纸的窗口的下面。又如，当一个Activity组件从后台激活到前台时，与它所对应的窗口就会被相应地移动到窗口堆栈的上面去。

从前面Android应用程序与SurfaceFlinger服务的关系概述和学习计划和Android系统Surface机制的SurfaceFlinger服务简要介绍和学习计划这两个系列的文章可以知道，窗口的UI最终是需要通过SurfaceFlinger服务来统一渲染的，而SurfaceFlinger服务在渲染窗口的UI之前，需要计算基于各个窗口的Z轴位置来计算它们的可见区域。因此，WindowManagerService服务计算好每一个窗口的Z轴位置之后，还需要将它们设置到SurfaceFlinger服务中去，以便SurfaceFlinger服务可以正确地渲染每一个窗口的UI。

上述窗口的Z轴位置计算和设置过程如图1所示：

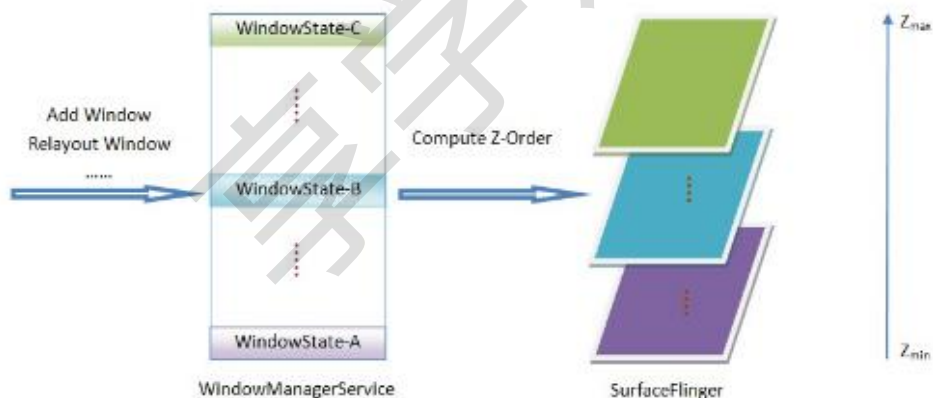


图1 窗口Z轴位置的计算和设置过程

接下来，我们就首先分析两个需要重新计算窗口Z轴位置的情景，接着再分析窗口的Z轴位置的计算过程，最后分析WindowManagerService服务将窗口的Z轴设置到SurfaceFlinger服务中去的过程。

一、需要重新计算窗口Z轴位置的情景

这里主要分析两个需要重新计算窗口Z轴位置的情景：应用程序增加一个窗口到WindowManagerService服务和应用程序请求WindowManagerService服务重新布局一个窗口。

从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，应用程序请求增加一个窗口到WindowManagerService服务的时候，最终会调用到WindowManagerService类的成员函数addWindow。接下来我们就主要分析这个函数与重新计算窗口Z轴位置相关的逻辑，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int addWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int viewVisibility,
        Rect outContentInsets, InputChannel outInputChannel) {
```

```

.....

synchronized(mWindowMap) {
    .....

    WindowToken token = mTokenMap.get(attrs.token);
    .....

    win = new WindowState(session, client, token,
        attachedWindow, attrs, viewVisibility);
    .....

    if (attrs.type == TYPE_INPUT_METHOD) {
        mInputMethodWindow = win;
        addInputMethodWindowToListLocked(win);
        .....
    } else if (attrs.type == TYPE_INPUT_METHOD_DIALOG) {
        mInputMethodDialogs.add(win);
        addWindowToListInOrderLocked(win, true);
        adjustInputMethodDialogsLocked();
        .....
    } else {
        addWindowToListInOrderLocked(win, true);
        if (attrs.type == TYPE_WALLPAPER) {
            .....
            adjustWallpaperWindowsLocked();
        } else if ((attrs.flags&FLAG_SHOW_WALLPAPER) != 0) {
            adjustWallpaperWindowsLocked();
        }
    }
    .....

    assignLayersLocked();

    .....
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数addWindow的具体实现可以参考Android窗口管理服务WindowManagerService对壁纸窗口（Wallpaper Window）的管理分析和Android窗口管理服务WindowManagerService对输入法窗口（Input Method Window）的管理分析这两篇文章。我们注意到，WindowManagerService类的成员函数addWindow会根据当前正在添加的窗口的类型来调用不同的成员函数来向窗口堆栈的合适位置插入一个WindowState对象，即：

1. 如果添加的是一个输入法窗口，那么就调用成员函数addInputMethodWindowToListLocked将它放置在需要显示输入法的窗口的上面去；
2. 如果添加的是一个输入法对话框，那么就先调用成员函数addWindowToListInOrderLocked来将它插入到窗口堆栈中，接着再调用成员函数adjustInputMethodDialogsLocked来将它放置在输入法窗口的上面；
3. 如果添加的是一个普通窗口，那么就直接调用成员函数addWindowToListInOrderLocked来将它插入到窗口堆栈中；
4. 如果添加的是一个普通窗口，并且这个窗口需要显示壁纸，那么就先调用成员函数addWindowToListInOrderLocked来将它插入到窗口堆栈中，接着再调用成员函数adjustWallpaperWindowsLocked来将壁纸窗口放置在它的下面。
5. 如果添加的是一个壁纸窗口，那么就先调用成员函数addWindowToListInOrderLocked来将它插入到窗口堆栈中，接着再调用成员函数adjustWallpaperWindowsLocked来将它放置在需要显示壁纸的窗口的下面。

无论如何，WindowManagerService类的成员函数addWindow最终都会调用成员函数assignLayersLocked来重新计算系统中所有窗口的Z轴位置，这是因为前面往窗口堆栈增加了一个新的窗口。

从前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文可以知道，应用程序进程

请求WindowManagerService服务重新布局一个窗口的时候，最终会调用到WindowManagerService类的成员函数relayoutWindow。接下来我们就主要分析这个函数与重新计算窗口Z轴位置相关的逻辑，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int relayoutWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int requestedWidth,
        int requestedHeight, int viewVisibility, boolean insetsPending,
        Rect outFrame, Rect outContentInsets, Rect outVisibleInsets,
        Configuration outConfig, Surface outSurface) {
        .....

        synchronized(mWindowMap) {
            WindowState win = windowForClientLocked(session, client, false);
            .....

            int attrChanges = 0;
            int flagChanges = 0;
            if (attrs != null) {
                flagChanges = win.mAttrs.flags ^ attrs.flags;
                attrChanges = win.mAttrs.copyFrom(attrs);
            }
            .....

            boolean imMayMove = (flagChanges & (
                WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM |
                WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE)) != 0;

            boolean focusMayChange = win.mViewVisibility != viewVisibility
                || ((flagChanges & WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE) != 0)
                || (!win.mRelayoutCalled);

            boolean wallpaperMayMove = win.mViewVisibility != viewVisibility
                && (win.mAttrs.flags & FLAG_SHOW_WALLPAPER) != 0;
            .....

            if (focusMayChange) {
                .....
                if (updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES)) {
                    imMayMove = false;
                }
                .....
            }

            // updateFocusedWindowLocked() already assigned layers so we only need to
            // reassign them at this point if the IM window state gets shuffled
            boolean assignLayers = false;

            if (imMayMove) {
                if (moveInputMethodWindowsIfNeededLocked(false) || displayed) {
                    // Little hack here -- we -should- be able to rely on the
                    // function to return true if the IME has moved and needs
                    // its layer recomputed. However, if the IME was hidden
                    // and isn't actually moved in the list, its layer may be
                    // out of data so we make sure to recompute it.
                    assignLayers = true;
                }
            }

            if (wallpaperMayMove) {
                if ((adjustWallpaperWindowsLocked() & ADJUST_WALLPAPER_LAYERS_CHANGED) != 0) {
                    assignLayers = true;
                }
            }
            .....
        }
    }
}
```

```

        if (assignLayers) {
            assignLayersLocked();
        }

        .....
    }

    .....

    return (inTouchMode ? WindowManagerImpl.RELAYOUT_IN_TOUCH_MODE : 0)
        | (displayed ? WindowManagerImpl.RELAYOUT_FIRST_TIME : 0);
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数relayoutWindow具体实现可以参考Android窗口管理服务WindowManagerService对壁纸窗口（Wallpaper Window）的管理分析和Android窗口管理服务WindowManagerService对输入法窗口（Input Method Window）的管理分析这两篇文章，与窗口Z轴位置计算相关的逻辑大概是这样的：

1. 如果系统当前获得焦点的窗口可能发生了变化，那么就会调用成员函数updateFocusedWindowLocked来重新计算系统当前应该获得焦点的窗口。如果系统当前获得焦点的窗口真的发生了变化，即窗口堆栈的窗口排列发生了变化，那么在调用成员函数updateFocusedWindowLocked的时候，就会调用成员函数assignLayersLocked来重新计算系统中所有窗口的Z轴位置。
2. 如果系统中的输入法窗口可能需要移动，那么就会调用成员函数moveInputMethodWindowsIfNeededLocked来检查是否真的需要移动输入法窗口。如果需要移动，那么成员函数moveInputMethodWindowsIfNeededLocked的返回值就会等于true，这时候就说明输入法窗口在窗口堆栈中的位置发生了变化，因此，就会将变量assignLayers的值设置为true，表示接下来需要重新计算系统中所有窗口的Z轴位置。
3. 如果当前正在请求调整其布局的窗口是由不可见变化可见的，即变量displayed的值等于true，那么接下来也是需要重新计算系统中所有窗口的Z轴位置的，因此，就会将assignLayers的值设置为true。
4. 如果系统中的壁纸窗口可能需要移动，那么就会调用成员函数adjustWallpaperWindowsLocked来检查是否真的需要移动壁纸窗口。如果需要移动，那么成员函数adjustWallpaperWindowsLocked的返回值的ADJUST_WALLPAPERLAYERS_CHANGED位就会等于1，这时候就说明壁纸窗口在窗口堆栈中的位置发生了变化，因此，就会将变量assignLayers的值设置为true，表示接下来需要重新计算系统中所有窗口的Z轴位置。

经过上述的一系列操作后，如果得到的变量assignLayers的值设置等于true，那么WindowManagerService类的成员函数relayoutWindow就会调用成员函数assignLayersLocked来重新计算系统中所有窗口的Z轴位置。

二. 计算系统中所有窗口的Z轴位置

从前面第一部分的内容可以知道，一旦窗口堆栈中的窗口发生了变化，那么WindowManagerService类的成员函数assignLayersLocked就会调用来计算系统中所有窗口的Z轴位置。

窗口的Z轴位置除了与它在窗口堆栈中的位置有关之外，还与窗口的类型有关。窗口的类型在创建的时候就已经是确定了的，WindowManagerService服务在为它创建一个WindowState对象的时候，就会根据它的类型得到一个BaseLayer值，这个BaseLayer值在计算它的Z轴位置的时候会用到。

接下来我们就通过WindowState类的构造函数来分析一个窗口的BaseLayer值是如何确定的，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    /** How much to multiply the policy's type layer, to reserve room
     * for multiple windows of the same type and Z-ordering adjustment
     * with TYPE_LAYER_OFFSET. */
    static final int TYPE_LAYER_MULTIPLIER = 10000;

    /** Offset from TYPE_LAYER_MULTIPLIER for moving a group of windows above
     * or below others in the same layer. */
    static final int TYPE_LAYER_OFFSET = 1000;

```



```

.....

private final class WindowState implements WindowManagerPolicy.WindowState {
    .....

    final int mBaseLayer;
    final int mSubLayer;
    .....

    WindowState(Session s, IWindow c, WindowToken token,
        WindowState attachedWindow, WindowManager.LayoutParams a,
        int viewVisibility) {
        .....

        if ((mAttrs.type >= FIRST_SUB_WINDOW &&
            mAttrs.type <= LAST_SUB_WINDOW)) {
            // The multiplier here is to reserve space for multiple
            // windows in the same type layer.
            mBaseLayer = mPolicy.windowTypeToLayerLw(
                attachedWindow.mAttrs.type) * TYPE_LAYER_MULTIPLIER
                + TYPE_LAYER_OFFSET;
            mSubLayer = mPolicy.subWindowTypeToLayerLw(a.type);
            .....
        } else {
            // The multiplier here is to reserve space for multiple
            // windows in the same type layer.
            mBaseLayer = mPolicy.windowTypeToLayerLw(a.type)
                * TYPE_LAYER_MULTIPLIER
                + TYPE_LAYER_OFFSET;
            mSubLayer = 0;
            .....
        }
        .....
    }
    .....
}
.....
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

一个窗口除了有一个BaseLayer值之外，还有一个SubLayer值，分别保存在一个对应的WindowState对象的成员变量mBaseLayer和mSubLayer。SubLayer值是用来描述一个窗口是否是另外一个窗口的子窗口的。

假设一个窗口是另外一个窗口的子窗口，那么参数attachedWindow所描述的窗口就是父窗口，这时候子窗口的BaseLayer值就等于父窗口的BaseLayer值，而SubLayer值要么大于0，要么小于0，这与它自己的具体类型有关。

假设一个窗口不是另外一个窗口的子窗口，那么这个窗口的BaseLayer值就与它自己的具体类型有关，而SubLayer值就等于0。

现在的关键就是要根据窗口的类型来计算它的BaseLayer值和SubLayer值，它们分别是通过调用WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数windowTypeToLayerLw和subWindowTypeToLayerLw来计算得到的。这里有两个地方是需要注意的。

第一个地方是PhoneWindowManager对象的成员函数windowTypeToLayerLw的返回值并且不是一个窗口的最终的BaseLayer值，而是要将它的返回值乘以一个常量TYPE_LAYER_MULTIPLIER，再加上另外一个常量TYPE_LAYER_OFFSET之后，才得到最终的BaseLayer值。这是因为在Android系统中，相同类型的窗口的Z轴位置都是有着相同的值域，而不同类型的窗口的Z轴位置都是处于两个不相交的值域。例如，假设有两种不同类型的窗口，它们的Z轴位置的值域分别为[a, b]和[c, d]，那么[a, b]和[c, d]的交集一定等于空。又由于每一种类型的窗口的数量是不确定的，因此，WindowManagerService服务就需要为每一种类型的窗口都预留一个范围足够大的值域，以便可以满足要求。

WindowManagerService服务是如何为类型相同的窗口的Z轴位置预留一个范围足够大的值域的呢？我们假设类型为t的窗口的Z轴位置的值域为[a, b]，并且以t为参数调用PhoneWindowManager对象的成员函数windowTypeToLayerLw的返回值为T，那么a的值就等于 $T * TYPE_LAYER_MULTIPLIER + TYPE_LAYER_OFFSET$ ，而b的值就等于 $(T - 1) * TYPE_LAYER_MULTIPLIER + TYPE_LAYER_OFFSET - 1$ ，即从 $T * TYPE_LAYER_MULTIPLIER + TYPE_LAYER_OFFSET$ 开始，一共预留了TYPE_LAYER_MULTIPLIER个值作为类型为t窗口的Z轴位置。由于TYPE_LAYER_MULTIPLIER的值定义为10000，而TYPE_LAYER_OFFSET的值定义为1000，因此，每一种类型的窗口都预留有一个足够大的值域来作为Z轴位置。

第二个地方是窗口的SubLayer值并不直接参与窗口的Z轴位置的计算，但是会影响到窗口在窗口堆栈的位置。接下来我们就会看到，窗口在窗口堆栈的位置是会影响到它的Z轴位置的计算的，因此，窗口的SubLayer间接地参与了窗口

的Z轴位置的计算。

窗口的SubLayer值是如何影响到窗口在窗口堆栈的位置的呢？在前面Android窗口管理服务WindowManagerService对窗口的组织方式分析一文中，在分析WindowManagerService类的成员函数addWindowToListInOrderLocked的实现时提到，如果一个窗口是另外一个窗口的子窗口，那么当它的SubLayer值小于0的时候，它就会位于父窗口的下面，否则的话，就会位于父窗口的上面。

在继续分析WindowManagerService类的成员函数assignLayersLocked之前，我们首先分析PhoneWindowManager类的成员函数windowTypeToLayerLw和subWindowTypeToLayerLw的实现，以便可以了解一个窗口的BaseLayer值和SubLayer值是如何确定的。

PhoneWindowManager类的成员函数windowTypeToLayerLw的实现如下所示：

```
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    public int windowTypeToLayerLw(int type) {
        if (type >= FIRST_APPLICATION_WINDOW && type <= LAST_APPLICATION_WINDOW) {
            return APPLICATION_LAYER;
        }
        switch (type) {
            case TYPE_STATUS_BAR:
                return STATUS_BAR_LAYER;
            case TYPE_STATUS_BAR_PANEL:
                return STATUS_BAR_PANEL_LAYER;
            case TYPE_SYSTEM_DIALOG:
                return SYSTEM_DIALOG_LAYER;
            case TYPE_SEARCH_BAR:
                return SEARCH_BAR_LAYER;
            case TYPE_PHONE:
                return PHONE_LAYER;
            case TYPE_KEYGUARD:
                return KEYGUARD_LAYER;
            case TYPE_KEYGUARD_DIALOG:
                return KEYGUARD_DIALOG_LAYER;
            case TYPE_SYSTEM_ALERT:
                return SYSTEM_ALERT_LAYER;
            case TYPE_SYSTEM_ERROR:
                return SYSTEM_ERROR_LAYER;
            case TYPE_INPUT_METHOD:
                return INPUT_METHOD_LAYER;
            case TYPE_INPUT_METHOD_DIALOG:
                return INPUT_METHOD_DIALOG_LAYER;
            case TYPE_SYSTEM_OVERLAY:
                return SYSTEM_OVERLAY_LAYER;
            case TYPE_SECURE_SYSTEM_OVERLAY:
                return SECURE_SYSTEM_OVERLAY_LAYER;
            case TYPE_PRIORITY_PHONE:
                return PRIORITY_PHONE_LAYER;
            case TYPE_TOAST:
                return TOAST_LAYER;
            case TYPE_WALLPAPER:
                return WALLPAPER_LAYER;
        }
        Log.e(TAG, "Unknown window type: " + type);
        return APPLICATION_LAYER;
    }
    .....
}
```

这个函数定义在frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java文件中。从这里就可以看出，每一种窗口类型type都对应有一个BaseLayer值，即每一个TYPE.XXX值都对应有一个XXXLAYER值，其中，TYPE.XXX值定义在WindowManager.LayoutParams类中，而XXXLAYER值就定义在PhoneWindowManager类中，它们的对应关系如图2所示：

Window Type	Layer Value	Description
TYPE_STATUS_BAR(2000)	STATUS_BAR_LAYER(8)	状态栏
TYPE_STATUS_BAR_PANEL(2014)	STATUS_BAR_PANEL_LAYER(5)	从状态栏滑出来的面板
TYPE_SYSTEM_DIALOG(2008)	SYSTEM_DIALOG_LAYER(6)	从状态栏滑出来的面板
TYPE_SEARCH_BAR(2001)	SEARCH_BAR_LAYER(4)	搜索栏
TYPE_PHONE(2002)	PHONE_LAYER(3)	拨号窗口
TYPE_KEYGUARD(2004)	KEYGUARD_LAYER(14)	锁屏窗口
TYPE_KEYGUARD_DIALOG(2009)	KEYGUARD_DIALOG_LAYER(15)	锁屏窗口显示的对话框
TYPE_SYSTEM_ALERT(2003)	SYSTEM_ALERT_LAYER(10)	系统警告窗口
TYPE_SYSTEM_ERROR(2010)	SYSTEM_ERROR_LAYER(11)	系统错误窗口
TYPE_INPUT_METHOD(2011)	INPUT_METHOD_LAYER(12)	系统输入法窗口
TYPE_INPUT_METHOD_DIALOG(2012)	INPUT_METHOD_DIALOG_LAYER(13)	系统输入法对话框，位于系统输入法窗口之上
TYPE_SYSTEM_OVERLAY(2006)	SYSTEM_OVERLAY_LAYER(16)	重叠窗口
TYPE_SECURE_SYSTEM_OVERLAY(2015)	SECURE_SYSTEM_OVERLAY_LAYER(17)	重叠窗口，但是只有系统可以创建，应用程序不可以创建
TYPE_PRIORITY_PHONE(2007)	PRIORITY_PHONE_LAYER(9)	拨号窗口，可以位于锁屏窗口之上
TYPE_TOAST(2005)	TOAST_LAYER(7)	短暂停留的通知窗口
TYPE_WALLPAPER(2013)	WALLPAPER_LAYER(2)	壁纸窗口

图2 窗口类型与窗口BaseLayer值的对应关系

注意，如果参数type的值小于FIRSTAPPLICATIONWINDOW，或者大于LASTAPPLICATIONWINDOW，或者不是图2列出来的其中一个值，那么PhoneWindowManager类的成员函数windowTypeToLayerLw就会返回一个APPLICATION_LAYER（2）值给调用者。

PhoneWindowManager类的成员函数subWindowTypeToLayerLw的实现如下所示：

```
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    public int subWindowTypeToLayerLw(int type) {
        switch (type) {
            case TYPE_APPLICATION_PANEL:
            case TYPE_APPLICATION_ATTACHED_DIALOG:
                return APPLICATION_PANEL_SUBLAYER;
            case TYPE_APPLICATION_MEDIA:
                return APPLICATION_MEDIA_SUBLAYER;
            case TYPE_APPLICATION_MEDIA_OVERLAY:
                return APPLICATION_MEDIA_OVERLAY_SUBLAYER;
            case TYPE_APPLICATION_SUB_PANEL:
                return APPLICATION_SUB_PANEL_SUBLAYER;
        }
        Log.e(TAG, "Unknown sub-window type: " + type);
        return 0;
    }

    .....
}
```

这个函数定义在frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java文件中。从这里就可以看出，只有类型为TYPEAPPLICATIONPANEL、TYPEAPPLICATIONMEDIA、TYPEAPPLICATIONMEDIAOVERLAY和TYPEAPPLICATIONSUBPANEL的窗口才对应有一个SubLayer值，它们的对应关系如图3所示：

Window Type	Sub Layer Value	Description
TYPE_APPLICATION_PANEL(1000)	APPLICATION_PANEL_SUBLAYER(1)	附加在其它窗口之上的窗口
TYPE_APPLICATION_ATTACHED_DIALOG(1003)	APPLICATION_PANEL_SUBLAYER(1)	同TYPE_APPLICATION_PANEL, 但是它是顶层窗口出现的, 不附加在任何一个窗口之上
TYPE_APPLICATION_MEDIA	APPLICATION_MEDIA_SUBLAYER(-2)	多媒体窗口
TYPE_APPLICATION_MEDIA_OVERLAY	APPLICATION_MEDIA_OVERLAY_SUBLAYER(-1)	显示在多媒体窗口上面的重叠窗口
TYPE_APPLICATION_SUB_PANEL	APPLICATION_SUB_PANEL_SUBLAYER(2)	附加在其它窗口的窗口, 它同时可以附加在类型为TYPE_APPLICATION_PANEL的窗口之上

图3 窗口类型与窗口SubLayer值的对应关系

在图3中, TYPEXXX值定义在WindowManager.LayoutParams类中, 而XXXLAYER值就定义在PhoneWindowManager类中。注意, 有两种特殊的多媒体窗口TYPEAPPLICATIONMEDIA和TYPEAPPLICATIONMEDIAOVERLAY, 它们是用来显示多媒体的, 例如, 用来显示视频, 并且它们都是附加在应用程序窗口之上的, 但是由于它们的SubLayer值为负数, 因此它们实际上是位于宿主窗口之下的。类型为TYPEAPPLICATIONMEDIA的窗口有一个魔术, 它会在宿主窗口里面挖一个洞, 以便可以将自己显示出来, 而类型为TYPEAPPLICATIONMEDIAOVERLAY背景一般都是透明的, 位于类型为TYPEAPPLICATIONMEDIA的窗口, 可以用来显示视频的字幕之类的东西。实际上, 类型为TYPEAPPLICATIONMEDIA和TYPEAPPLICATIONMEDIA_OVERLAY的窗口也称为SurfaceView。SurfaceView很特殊, 它与普通的View的最大区别就在于它们有独立的绘图表面, 于是它们就可以在一个独立的子线程里面进行UI渲染。

理解了窗口的BaseLayer值和SubLayer值的计算过程之外, 接下来我们就可以分析WindowManagerService类的成员函数assignLayersLocked的实现了, 如下所示:

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    /** How much to increment the layer for each window, to reserve room
     * for effect surfaces between them.
     */
    static final int WINDOW_LAYER_MULTIPLIER = 5;
    .....

    private final void assignLayersLocked() {
        int N = mWindows.size();
        int curBaseLayer = 0;
        int curLayer = 0;
        int i;

        for (i=0; i<N; i++) {
            WindowState w = mWindows.get(i);
            if (w.mBaseLayer == curBaseLayer || w.mIsImWindow
                || (i > 0 && w.mIsWallpaper)) {
                curLayer += WINDOW_LAYER_MULTIPLIER;
                w.mLayer = curLayer;
            } else {
                curBaseLayer = curLayer = w.mBaseLayer;
                w.mLayer = curLayer;
            }
            if (w.mTargetAppToken != null) {
                w.mAnimLayer = w.mLayer + w.mTargetAppToken.animLayerAdjustment;
            } else if (w.mAppToken != null) {
                w.mAnimLayer = w.mLayer + w.mAppToken.animLayerAdjustment;
            } else {
                w.mAnimLayer = w.mLayer;
            }
            if (w.mIsImWindow) {
                w.mAnimLayer += mInputMethodAnimLayerAdjustment;
            } else if (w.mIsWallpaper) {
                w.mAnimLayer += mWallpaperAnimLayerAdjustment;
            }
        }
    }
}
```

```

    }
    .....
}
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

注意，在调用WindowManagerService类的成员函数assignLayersLocked之前，系统中的所有窗口在窗口堆栈中的位置都是已经排列好了的，这时候WindowManagerService类的成员函数assignLayersLocked就从下往上遍历窗口堆栈，以连排列在一起的类型相同的窗口为单位来计算每一个窗口的Z位置，即：

1. 每次遇到一个窗口，它的BaseLayer值与上一次计算的窗口的BaseLayer值不相等，就开始一个新的计算单元。
2. 在每一个计算单元中，第一个窗口的Z轴位置就等于它的BaseLayer值，而之后的每一个窗口的Z轴位置都比前一个窗口的Z轴位置大WINDOW_LAYER_MULTIPLIER。

这个窗口的Z轴位置计算方法有三个地方是需要注意的。

第一个地方是从第2点可以看出，每一个窗口的Z轴位置值都不是连续的，这样就在每两个窗口之间都保留了一定的位置来插入其它窗口。

第二个地方是由于系统中所有类型相同的窗口不一定是排列在一起的，因此，就有可能出现有些类型相同的窗口具有相同的Z轴位置。WindowManagerService服务并不关心两个不同窗口的Z轴位置是否相同，但是SurfaceFlinger服务就需要关心了，因为SurfaceFlinger服务需要是按照Z轴从大到小的顺序来计算窗口的可见性。那么SurfaceFlinger服务是如何确定两个Z轴位置相同的窗口的次序的呢？从前面Android应用程序与SurfaceFlinger服务的关系概述和学习计划和Android系统Surface机制的SurfaceFlinger服务简要介绍和学习计划这两个系列的文章可以知道，每一个窗口在SurfaceFlinger服务都对应有一个Layer对象，而每一个Layer对象都有一个sequence值，其中，先创建的Layer对象的sequence值大于后创建的Layer对象的sequence值。这样，SurfaceFlinger服务在计算对于两个Z轴位置相同的窗口的可见性的时候，就会比较它们所对应的Layer对象的sequence值，其中，sequence值大的窗口的可见性会先于sequence值小的窗口得到计算，即先计算后创建的窗口的可见性，再计算先创建的窗口的可见性。

第三个地方是有两种特殊的窗口，即输入法窗口和壁纸窗口，当它们不是窗口堆栈底部的第一个窗口时，它们所在的计算单元不是以窗口类型来划分的，而靠近在哪个窗口，就与哪个窗口在同一个计算单元中。当输入法窗口是窗口堆栈底部的第一个窗口时，它的Z轴位置就等于WINDOW_LAYER_MULTIPLIER，而当壁纸窗口是窗口堆栈底部的第一个窗口时，它的Z轴位置就等于它的BaseLayer值。

前面计算得到的窗口的Z轴位置保存在WindowState类的成员变量mLayer中。事实上，保存在WindowState类的成员变量mLayer中的Z轴位置还不是窗口的最终Z轴位置，因为还没有考虑到窗口与窗口令牌之间的关系。每一个窗口令牌都可以设置一个Z轴调整值，而每一个窗口要加上它所对应的窗口令牌所设置的Z轴调整值之后，才能得到最终的Z轴位置。注意，只有类型为AppWindowToken的窗口令牌才可以设置Z轴调整值，这个Z轴调整值就保存在AppWindowToken类的成员变量animLayerAdjustment中。

有时候，一个窗口会有一个目标窗口。例如，输入法窗口的目标窗口是系统当前需要显示输入法的窗口。在这种情况下，我们要使用目标窗口所对应的窗口令牌所设置的Z轴调整值来调整窗口的Z轴位置。

那么，WindowManagerService服务是如何知道一个窗口所对应的窗口令牌的类型是AppWindowToken，或者一个窗口有没有目标窗口的呢？当用来描述一个窗口的WindowState对象成员变量mAppToken的值不等于null的时候，那么就说明该窗口所对应的窗口令牌的类型是AppWindowToken，而当用来描述一个窗口的WindowState对象成员变量mTargetAppToken的值不等于null的时候，那么就说明该窗口有一个目标窗口。

经过上面的调整之后，窗口的Z轴位置就保存在WindowState类的成员变量mAnimLayer中。对于非输入法窗口和非壁纸窗口来说，这时候保存在用来描述它们的WindowState对象的成员变量mAnimLayer中的Z轴位置就是它们最终的Z轴位置了，但是对于输入法窗口和壁纸窗口来说，还需要继续判断它们的目标窗口是否需要调整它们的Z轴位置。

从前面Android窗口管理服务WindowManagerService对壁纸窗口（Wallpaper Window）的管理分析和Android窗口管理服务WindowManagerService对输入法窗口（Input Method Window）的管理分析这两篇文章知道，如果一个窗口要调整它所关联的输入法窗口和壁纸窗口的Z轴位置，那么要调整的值就会保存在WindowManagerService类的成员变量mInputMethodAnimLayerAdjustment和mWallpaperAnimLayerAdjustment中，因此，只要将WindowManagerService类的成员变量mInputMethodAnimLayerAdjustment和mWallpaperAnimLayerAdjustment的值分别增加到前面所计算得到的输入法窗口和壁纸窗口的Z轴位置上去，就可以得到输入法窗口和壁纸窗口的最终Z轴位置，并且保存到用来对应的WindowState对象的成员变量mAnimLayer中。

从上面的计算过程就可以知道，系统中所有类型的窗口的最终Z轴位置都保存在WindowState类的成员变量mAnimLayer中。

三. 设置窗口的Z轴位置到SurfaceFlinger服务中去

WindowManagerService服务在刷新系统的UI的时候，就会将系统中已经计算好了的窗口Z轴位置设置到SurfaceFlinger服务中去，以便SurfaceFlinger服务可以对系统中的窗口进行可见性计算以及合成和渲染等操作。

从前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文可以知道，刷新系统UI是通过调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner来实现的，接下来我们就分析这个成员函数与设置窗口的Z轴位置到SurfaceFlinger服务中去相关的逻辑。

为了方便描述设置窗口的Z轴位置到SurfaceFlinger服务中去的过程，我们先列出WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner的实现架构，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final void performLayoutAndPlaceSurfacesLockedInner(
        boolean recoveringMemory) {
        .....

        Surface.openTransaction();
        .....

        try {
            .....
            int repeats = 0;
            int changes = 0;

            do {
                repeats++;
                if (repeats > 6) {
                    .....
                    break;
                }

                // FIRST LOOP: Perform a layout, if needed.
                if (repeats < 4) {
                    changes = performLayoutLockedInner();
                    if (changes != 0) {
                        continue;
                    }
                } else {
                    Slog.w(TAG, "Layout repeat skipped after too many iterations");
                    changes = 0;
                }

                // SECOND LOOP: Execute animations and update visibility of windows.
                .....

            } while (changes != 0);

            // THIRD LOOP: Update the surfaces of all windows.
            .....

            //更新窗口的绘图表面的操作包括：
            //1. 设置窗口的大小
            //2. 设置窗口在X轴和Y轴上的位置
            //3. 设置窗口在Z轴上的位置
            //4. 设置窗口的Alpha通道
            //5. 设置窗口的变换矩阵
            .....

        } catch (RuntimeException e) {
            .....
        }

        .....

        Surface.closeTransaction();

        .....

        // Destroy the surface of any windows that are no longer visible.
        .....

        // Time to remove any exiting tokens?
        .....

        // Time to remove any exiting applications?
        .....
    }
}
```

```

}

.....

}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。在前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文中，我们已经分析过WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner的实现架构了，其中，设置窗口的Z轴位置到SurfaceFlinger服务中去是在更新窗口的绘图表面的操作中进行的，即是在THIRD LOOP中进行的，同时设置的还包括窗口的大小、X轴和Y轴位置、Alpha通道和变换矩阵，这些代码如下所示：

//更新窗口的绘图表面的操作包括： //1. 设置窗口的大小 //2. 设置窗口在X轴和Y轴上的位置 //3. 设置窗口在Z轴上的位置 //4. 设置窗口的Alpha通道值 //5. 设置窗口的变换矩阵 final int N = mWindows.size();

```

for (i=N-1; i>=0; i--) {
    WindowState w = mWindows.get(i);
    .....

    if (w.mSurface != null) {
        .....

        w.computeShownFrameLocked();
        .....

        boolean resize;
        int width, height;
        if ((w.mAttrs.flags & w.mAttrs.FLAG_SCALED) != 0) {
            resize = w.mLastRequestedWidth != w.mRequestedWidth ||
                w.mLastRequestedHeight != w.mRequestedHeight;
            // for a scaled surface, we just want to use
            // the requested size.
            width = w.mRequestedWidth;
            height = w.mRequestedHeight;
            w.mLastRequestedWidth = width;
            w.mLastRequestedHeight = height;
            w.mLastShownFrame.set(w.mShownFrame);
            try {
                .....
                w.mSurfaceX = w.mShownFrame.left;
                w.mSurfaceY = w.mShownFrame.top;
                w.mSurface.setPosition(w.mShownFrame.left, w.mShownFrame.top);
            } catch (RuntimeException e) {
                .....
                if (!recoveringMemory) {
                    reclaimSomeSurfaceMemoryLocked(w, "position");
                }
            }
        } else {
            resize = !w.mLastShownFrame.equals(w.mShownFrame);
            width = w.mShownFrame.width();
            height = w.mShownFrame.height();
            w.mLastShownFrame.set(w.mShownFrame);
        }

        if (resize) {
            if (width < 1) width = 1;
            if (height < 1) height = 1;
            if (w.mSurface != null) {
                try {
                    .....
                    w.mSurfaceResized = true;
                    w.mSurfaceW = width;
                    w.mSurfaceH = height;
                    w.mSurface.setSize(width, height);
                    w.mSurfaceX = w.mShownFrame.left;
                    w.mSurfaceY = w.mShownFrame.top;
                    w.mSurface.setPosition(w.mShownFrame.left,
                        w.mShownFrame.top);
                } catch (RuntimeException e) {

```

```

        .....
        if (!recoveringMemory) {
            reclaimSomeSurfaceMemoryLocked(w, "size");
        }
    }
}

.....

if (w.mAttachedHidden || !w.isReadyForDisplay()) {
    if (!w.mLastHidden) {
        w.mLastHidden = true;
        .....

        if (w.mSurface != null) {
            w.mSurfaceShown = false;
            try {
                w.mSurface.hide();
            } catch (RuntimeException e) {
                .....
            }
        }
    }
}

.....
} else if (w.mLastLayer != w.mAnimLayer
    || w.mLastAlpha != w.mShownAlpha
    || w.mLastDsDx != w.mDsDx
    || w.mLastDtDx != w.mDtDx
    || w.mLastDsDy != w.mDsDy
    || w.mLastDtDy != w.mDtDy
    || w.mLastHScale != w.mHScale
    || w.mLastVScale != w.mVScale
    || w.mLastHidden) {
    .....
    w.mLastAlpha = w.mShownAlpha;
    w.mLastLayer = w.mAnimLayer;
    w.mLastDsDx = w.mDsDx;
    w.mLastDtDx = w.mDtDx;
    w.mLastDsDy = w.mDsDy;
    w.mLastDtDy = w.mDtDy;
    w.mLastHScale = w.mHScale;
    w.mLastVScale = w.mVScale;
    .....
    if (w.mSurface != null) {
        try {
            w.mSurfaceAlpha = w.mShownAlpha;
            w.mSurface.setAlpha(w.mShownAlpha);
            w.mSurfaceLayer = w.mAnimLayer;
            w.mSurface.setLayer(w.mAnimLayer);
            w.mSurface.setMatrix(
                w.mDsDx*w.mHScale, w.mDtDx*w.mVScale,
                w.mDsDy*w.mHScale, w.mDtDy*w.mVScale);
        } catch (RuntimeException e) {
            .....
            if (!recoveringMemory) {
                reclaimSomeSurfaceMemoryLocked(w, "update");
            }
        }
    }
}

if (w.mLastHidden && !w.mDrawPending
    && !w.mCommitDrawPending
    && !w.mReadyToShow) {
    .....
    if (showSurfaceRobustlyLocked(w)) {
        w.mHasDrawn = true;
        w.mLastHidden = false;
    }
}

```



```

    }
    .....
}
.....
}

```

这段代码通过一个for循环来遍历保存在窗口堆栈的每一个WindowState对象，以便可以对系统中的每一个窗口的绘图表面进行更新。注意，只有那些成员变量mSurface的值不等于null的WindowState对象，它们所描述的窗口才具有绘图表面，因此需要对它们进行更新。在更新WindowState对象w所描述的窗口的绘图表面之前，首先要调用它的成员函数computeShownFrameLocked来确定该窗口实际要显示的大小、位置、Alpha通道和变换矩阵等信息，其中：

1. 窗口实际要显示的大小和X轴、Y轴位置保存在WindowState对象w的成员变量mShownFrame中。
2. 窗口实际要显示的Alpha通道保存在WindowState对象w的成员变量mShownAlpha中。
3. 窗口实际要显示的Z轴位置保存在WindowState对象w的成员变量mAnimLayer中。
4. 窗口实际要使用的变换矩阵保存在WindowState对象w的成员变量mDsDx、mDtDx、mDsDy和mDtDy中。

有了上述信息之后，我们就可以将WindowState对象w所描述的窗口实际要显示的大小、位置、Alpha通道和变换矩阵等信息设置到SurfaceFlinger服务中去了。

我们首先分析WindowState对象w所描述的窗口实际要显示的大小、X轴和Y轴位置的设置过程，接着再分析WindowState对象w所描述的窗口实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵的设置过程。

在调用WindowState对象w的成员函数computeShownFrameLocked来计算它所描述的窗口的大小的时候，是没有考虑该窗口的大小是否设置有缩放因子的。

当WindowState对象w所描述的窗口的大小设置有缩放因子的时候，那么WindowState对象w的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_SCALED位就会等于1，这时候WindowState对象w所描述的窗口实际要显示的大小是保存在它的成员变量mRequestedWidth和mRequestedHeight中的。在这种情况下，这段代码就会执行以下操作：

1. 计算WindowState对象w所描述的窗口实际要显示的大小是否发生了变化。如果发生了变化，那么就将变量resize的值设置为true。注意，WindowState对象w所描述的窗口上一次实际要显示的大小保存在成员变量mLastRequestedWidth和mLastRequestedHeight中，因此，当这两个成员变量与其它两个成员变量mRequestedWidth和mRequestedHeight的值不相等于时，就说明WindowState对象w所描述的窗口实际要显示的大小是否发生了变化。
2. 将WindowState对象w所描述的窗口实际要显示的大小分别更新到成员变量mLastRequestedWidth和mLastRequestedHeight中，以及变量width和height中。
3. 将WindowState对象w的成员变量mShownFrame的值保存在另外一个成员变量mLastShownFrame中，以便可以记录WindowState对象w所描述的窗口上一次实际要显示的大小和X轴、Y轴位置。
4. 将WindowState对象w所描述的窗口的X轴和Y轴位置分别保存到成员变量mSurfaceX和mSurfaceY中，并且调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setPosition来将这两个位置值设置到SurfaceFlinger服务中去。
5. 在设置WindowState对象w所描述的窗口的X轴和Y轴位置到SurfaceFlinger服务中去的过程中，如果出现了异常，那么就说明系统内存资源不足。在这种情况下，如果参数recoveringMemory的值等于false，那么就说明WindowManagerService服务目前不是处于内存资源的回收过程中，于是就会调用WindowManagerService类的成员函数reclaimSomeSurfaceMemoryLocked来执行回收系统内存资源的操作。

当WindowState对象w所描述的窗口的大小没有设置有缩放因子的时候，那么WindowState对象w的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_SCALED位就会等于0，这时候WindowState对象w所描述的窗口实际要显示的大小是保存在它的成员变量mShownFrame中的。在这种情况下，这段代码就会执行以下操作：

1. 计算WindowState对象w所描述的窗口实际要显示的大小是否发生了变化。如果发生了变化，那么就将变量resize的值设置为true。注意，这时候只要比较WindowState对象w的成员变量mLastShownFrame和mShownFrame所描述的两个矩形区域的大小是否相等，就可以知道WindowState对象w所描述的窗口实际要显示的大小是否发生了变化，因为WindowState对象w的成员变量mLastShownFrame保存的是窗口上一次实际要显示的大小。
2. 将WindowState对象w所描述的窗口实际要显示的大小分别保存在变量width和height中。

3. 将WindowState对象w的成员变量mShownFrame的值保存在另外一个成员变量mLastShownFrame中，以便可以记录WindowState对象w所描述的窗口上一次实际要显示的大小和X轴、Y轴位置。

执行完成以上操作之后，WindowState对象w所描述的窗口实际要显示的X轴和Y轴位置就保存在成员变量mShownFrame所描述的一个Rect对象的成员变量left和top中，而实际要显示的大小就显示在变量width和height中。这时候如果变量resize的值等于true，那么就说明WindowState对象w所描述的窗口的大小发生了变化。在这种情况下，就需要执行以下操作：

1. 重新设置WindowState对象w所描述的窗口的大小到SurfaceFlinger服务中去，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setSize来实现的。注意，如果前面计算得到WindowState对象w所描述的窗口的宽度width和高度height的值小于1，那么就需要将它们设置为1，因为一个窗口的宽度和高度值是不能小于1的。
2. 重新设置WindowState对象w所描述的窗口在X轴和Y轴上的位置到SurfaceFlinger服务中去，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setPosition来实现的。注意，在设置之前，还会将WindowState对象w所描述的窗口在X轴和Y轴上的位置保存在成员变量mSurfaceX和mSurfaceY中。
3. 在设置WindowState对象w所描述的窗口的大小以及在X轴和Y轴上的位置到SurfaceFlinger服务中去的过程中，如果出现了异常，那么同样需要判断参数recoveringMemory的值来决定是否需要WindowManagerService类的成员函数reclaimSomeSurfaceMemoryLocked来回收系统内存资源。

设置好WindowState对象w所描述的窗口实际要显示的大小、X轴和Y轴位置到SurfaceFlinger服务中去之后，接下来就要继续设置它实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵了，不过只有当WindowState对象w所描述的窗口当前是处于可见状态、并且这些值没有发生变化的情况下才需要这样做。

当WindowState对象w的成员函数isReadyForDisplay的返回值等于false时，就说明WindowState对象w所描述的窗口当前是处于不可见状态的。还有另外一种情况，即当WindowState对象w所描述的窗口是附加在另外一个窗口之上、并且这个被附加的窗口是不可见时，即WindowState对象w的成员变量mAttachedHidden的值等于true时，也是说明WindowState对象w所描述的窗口当前是处于不可见状态的。

在WindowState对象w所描述的窗口当前是处于不可见状态的情况下，如果该窗口在上一次系统UI刷新时是处于可见状态的，即WindowState对象w的成员变量mLastHidden的值等于true，那么这时候就需要将WindowState对象w所描述的窗口隐藏起来，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数hide来实现的。注意，在调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数hide来隐藏窗口之前，需要分别将WindowState对象w的成员变量mLastHidden和mSurfaceShown的值设置为true和false，以便可以正确描述窗口的不可见状态。

在WindowState对象w所描述的窗口当前是处于可见状态的情况下，如果该窗口实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵发生了变化，那么就需要将新的值设置到SurfaceFlinger服务中去，其中：

1. WindowState对象w的成员变量mLastLayer与mAnimLayer的值不相等说明它描述的窗口的Z轴位置发生了变化。
2. WindowState对象w的成员变量mLastAlpha与mShownAlpha的值不相等说明它描述的窗口的Alpha通道发生了变化。
3. WindowState对象w的成员变量mLastDsDx、mLastDtDx、mLastDsDy、mLastDtDy、mLastHScale、mLastVScale与成员变量mDsDx、mDtDx、mDsDy、mDtDy、mHScale、mVScale的值不相等说明它描述的窗口的变换矩阵发生了变化。

在WindowState对象w所描述的窗口当前是处于可见状态的情况下，如果该窗口在上一次系统UI刷新时是处于可见状态的，即WindowState对象w的成员变量mLastHidden的值等于true，那么也是需要重新设置WindowState对象w所描述的窗口实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵到SurfaceFlinger服务中去的。

无论如何，当需要重新设置WindowState对象w所描述的窗口实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵到SurfaceFlinger服务中去时，就需要执行以下操作：

1. 重新设置WindowState对象w所描述的窗口的Alpha通道到SurfaceFlinger服务中去，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setAlpha来实现的。在设置之前，还会将WindowState对象w的成员变量mShownAlpha的值同时保存在成员变量mLastAlpha和mSurfaceAlpha中，以便可以记录WindowState对象w所描述的窗口上一次所使用的Alpha通道。
2. 重新设置WindowState对象w所描述的窗口的Z轴位置到SurfaceFlinger服务中去，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setLayer来实现的。在设置之前，还会将WindowState对象w的成员变量mAnimLayer的值同时保存在成员变量mLastLayer和mSurfaceLayer中，以便可以记录WindowState对象w所描述的窗口上一次所使用的Z轴位置。

3. 重新设置WindowState对象w所描述的窗口的变换矩阵到SurfaceFlinger服务中去，这是通过调用WindowState对象w的成员变量mSurface所指向的一个Surface对象的成员函数setMatrix来实现的。在设置之前，还会将WindowState对象w的成员变量成员变量mDsDx、mDtDx、mDsDy、mDtDy、mHScale、mVScale的值分别保存在成员变量mLastDsDx、mLastDtDx、mLastDsDy、mLastDtDy、mLastHScale、mLastVScale中，以便可以记录WindowState对象w所描述的窗口上一次所使用的变换矩阵。注意，WindowState对象的成员变量mHScale和mVScale描述的窗口在宽度和高度上的缩放因子，因此，在设置窗口的变换矩阵时，需要乘以这些因子才可以得到正确的变换矩阵参数。
4. 在设置WindowState对象w所描述的窗口的Alpha通道、Z轴位置以及实际要使用的变换矩阵到SurfaceFlinger服务的过程中，如果出现了异常，那么同样需要判断参数recoveringMemory的值来决定是否需要WindowManagerService类的成员函数reclaimSomeSurfaceMemoryLocked来回收系统内存资源。

将WindowState对象w所描述的窗口实际要显示的Alpha通道、Z轴位置以及实际要使用的变换矩阵设置到SurfaceFlinger服务之后，如果WindowState对象w所描述的窗口满足以下条件：

1. 上一次处于不可见状态，即WindowState对象w的成员变量mLastHidden的值等于true；
2. UI已经绘制完成，即WindowState对象w的成员变量mDrawPending和mCommitDrawPending值等于false；
3. 不是处于等待同一个窗口令牌的其它窗口的完成UI绘制的状态，即WindowState对象w的成员变量mReadyToShow的值等于false；

那么就说明现在就是时候要将WindowState对象w所描述的窗口显示出来了，这是通过调用WindowManagerService类的成员函数showSurfaceRobustlyLocked来实现的。如果WindowManagerService类的成员函数showSurfaceRobustlyLocked的返回值等于true，那么就说明WindowManagerService服务已经成功地通知SurfaceFlinger服务将WindowState对象w所描述的窗口显示出来，于是就会分别将WindowState对象w的成员变量mHasDrawn和mLastHidden的值设置为true和false，以便可以表示WindowState对象w所描述的窗口的UI已经绘制完成，并且已经显示出来。

WindowManagerService类的成员函数showSurfaceRobustlyLocked的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    boolean showSurfaceRobustlyLocked(WindowState win) {
        try {
            if (win.mSurface != null) {
                win.mSurfaceShown = true;
                win.mSurface.show();
                .....
            }
            return true;
        } catch (RuntimeException e) {
            .....
        }

        reclaimSomeSurfaceMemoryLocked(win, "show");

        return false;
    }

    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowManagerService类的成员函数showSurfaceRobustlyLocked用来通知SurfaceFlinger服务将参数win所描述的窗口显示出来，这是通过调用WindowState对象win的成员变量mSurface所指向的一个Surface对象的成员函数show来实现的。注意，在通知SurfaceFlinger服务将WindowState对象win所描述的窗口显示出来之前，还会将它的成员变量mSurfaceShown的值设置为true。

如果在通知SurfaceFlinger服务将WindowState对象win所描述的窗口显示出来的过程出现了异常，那么WindowManagerService类的成员函数showSurfaceRobustlyLocked就会调用另外一个成员函数reclaimSomeSurfaceMemoryLocked来回收系统内存资源。

从上面分析可以知道，一个窗口的显示和隐藏，以及大小、X轴和Y轴位置、Z轴位置、Alpha通道和变换矩阵设置，是通过调用Java层的Surface类的成员函数show、hide、setSize、setPosition、setLayer、setAlpha和setMatrix来实现的，它们都是一些JNI方法，定义在文件frameworks/base/core/java/android/view/Surface.java中，如下所示：

```

public class Surface implements Parcelable {
    .....

    private int mSurfaceControl;
    .....

    /**
     * set surface parameters.
     * needs to be inside open/closeTransaction block
     */
    public native void setLayer(int zorder);
    public native void setPosition(int x, int y);
    public native void setSize(int w, int h);

    public native void hide();
    public native void show();
    .....
    public native void setAlpha(float alpha);
    public native void setMatrix(float dsdx, float dtdx,
                                float dsdy, float dtdy);
    .....
}

```

这些JNI方法是由C++层中的函数Surface.show、Surface.hide、Surface.setSize、Surface.setPosition、Surface.setLayer、Surface.setAlpha和Surface.setMatrix来实现的，如下所示：

```

static void Surface_setLayer(
    JNIEnv* env, jobject clazz, jint zorder)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->setLayer(zorder);
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_setPosition(
    JNIEnv* env, jobject clazz, jint x, jint y)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->setPosition(x, y);
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_setSize(
    JNIEnv* env, jobject clazz, jint w, jint h)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->setSize(w, h);
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_hide(
    JNIEnv* env, jobject clazz)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->hide();
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_show(
    JNIEnv* env, jobject clazz)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));

```

```

    if (surface == 0) return;
    status_t err = surface->show();
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_setAlpha(
    JNIEnv* env, jobject clazz, jfloat alpha)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->setAlpha(alpha);
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

static void Surface_setMatrix(
    JNIEnv* env, jobject clazz,
    jfloat dsdx, jfloat dtdx, jfloat dsdy, jfloat dtdy)
{
    const sp<SurfaceControl>& surface(getSurfaceControl(env, clazz));
    if (surface == 0) return;
    status_t err = surface->setMatrix(dsdx, dtdx, dsdy, dtdy);
    if (err<0 && err!=NO_INIT)
        doThrow(env, "java/lang/IllegalArgumentException", NULL);
}

```

这些JNI方法定义在文件frameworks/base/core/jni/androidviewSurface.cpp中。这些JNI都有一个共同的特点，即先调用函数getSurfaceControl来获得与参数clazz所描述的一个Java层的Surface对象所对应的一个SurfaceControl对象。有了这个SurfaceControl对象之后，就可以分别调用它的成员函数show、hide、setSize、setPosition、setLayer、setAlpha和setMatrix来通知SurfaceFlinger服务来显示和隐藏一个窗口，以及设置一个窗口大小、X轴和Y轴位置、Z轴位置、Alpha通道和变换矩阵。

从前面Android应用程序窗口（Activity）的绘图表面（Surface）的创建过程分析一文可以知道，每一个Activity窗口在Java层都对应有两个Surface对象，其中一个位于应用程序进程这一侧，而另外一个位于WindowManagerService服务这一侧。每一个位于应用程序进程这一侧的Java层的Surface对象在C++层中都对应有一个Surface对象，而每一个位于WindowManagerService服务这一侧的Java层的Surface对象在C++层中都对应有一个SurfaceControl对象，这个C++层的SurfaceControl对象的地址就保存在Java层的Surface对象的成员变量mSurfaceControl中。

从上面的分析可以知道，我们目前正在操作的是正在位于WindowManagerService服务这一侧的Java层的Surface对象，因此，通过调用函数getSurfaceControl就可以在C++层中获得一个对应的SurfaceControl对象，而有了这个SurfaceControl对象之后，就可以用来通知SurfaceFlinger服务更新一个窗口的属性，这一点可以参考前面Android应用程序与SurfaceFlinger服务的关系概述和学习计划和Android系统Surface机制的SurfaceFlinger服务简要介绍和学习计划两个系列的文章。

至此，WindowManagerService服务计算窗口Z轴位置的过程就分析完成了，这个过程如下所示：

1. WindowManagerService服务将窗口排列在一个窗口堆栈中；
2. WindowManagerService服务根据窗口类型以及窗口在窗口堆栈的位置来计算得窗口的Z轴位置；
3. WindowManagerService服务通过Java层的Surface类的成员函数setLayer来将窗口的Z轴位置设置到SurfaceFlinger服务中去；
4. Java层的Surface类的成员函数setLayer又是通过调用C++层的SurfaceControl类的成员函数setLayer来将窗口的Z轴位置设置到SurfaceFlinger服务中去的；

通过这篇文章以及前面三篇文章（窗口组织、输入法窗口、壁纸窗口）的学习，我们对WindowManagerService服务对窗口的管理就有一个比较深刻的认识了，在接下来的文章中，我们还将继续分析ActivityWindowManager服务和WindowManagerService服务是如何协作来完成Activity窗口的显示过程的，敬请关注！

原文链接：<https://blog.csdn.net/Luoshengyang/article/details/8570428>

第七节Android窗口管理服务WindowManagerService显示Activity组件的启动窗口（Starting Window）的过程分析

在Android系统中，Activity组件在启动之后，并且在它的窗口显示出来之前，可以显示一个启动窗口。这个启动窗口可以看作是Activity组件的预览窗口，是由WindowManagerService服务统一管理的，即由WindowManagerService服务负责启动和结束。在本文中，我们就详细分析WindowManagerService服务启动和结束Activity组件的启动窗口的过程。

Activity组件的启动窗口是由ActivityManagerService服务来决定是否要显示的。如果需要显示，那么ActivityManagerService服务就会通知WindowManagerService服务来为正在启动的Activity组件显示一个启动窗口，而WindowManagerService服务又是通过窗口管理策略类PhoneWindowManager来创建这个启动窗口的。这个过程如图1所示。

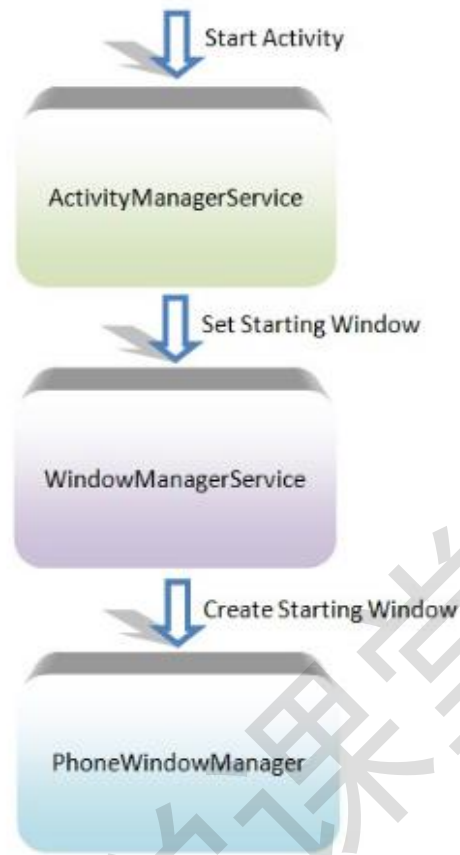


图1 Activity窗口的启动窗口的创建过程

窗口管理策略类PhoneWindowManager创建完成Activity组件的启动窗口之后，就会请求WindowManagerService服务将该启动窗口显示出来。当Activity组件启动完成，并且它的窗口也显示出来的时候，WindowManagerService服务就会结束显示它的启动窗口。

注意，Activity组件的启动窗口是由ActivityManagerService服务来控制是否显示的，也就是说，Android应用程序是无法决定是否要要Activity组件显示启动窗口的。接下来，我们就分别分析Activity组件的启动窗口的显示和结束过程。

一. Activity组件的启动窗口的显示过程

从前面Android应用程序启动过程源代码分析一文可以知道，Activity组件在启动的过程中，会调用ActivityStack类的成员函数startActivityLocked。注意，在调用ActivityStack类的成员函数startActivityLocked的时候，Activity组件还处于启动的过程，即它的窗口尚未显示出来，不过这时候ActivityManagerService服务会检查是否需要为正在启动的Activity组件显示一个启动窗口。如果需要的话，那么ActivityManagerService服务就会请求WindowManagerService服务为正在启动的Activity组件设置一个启动窗口。这个过程如图2所示。

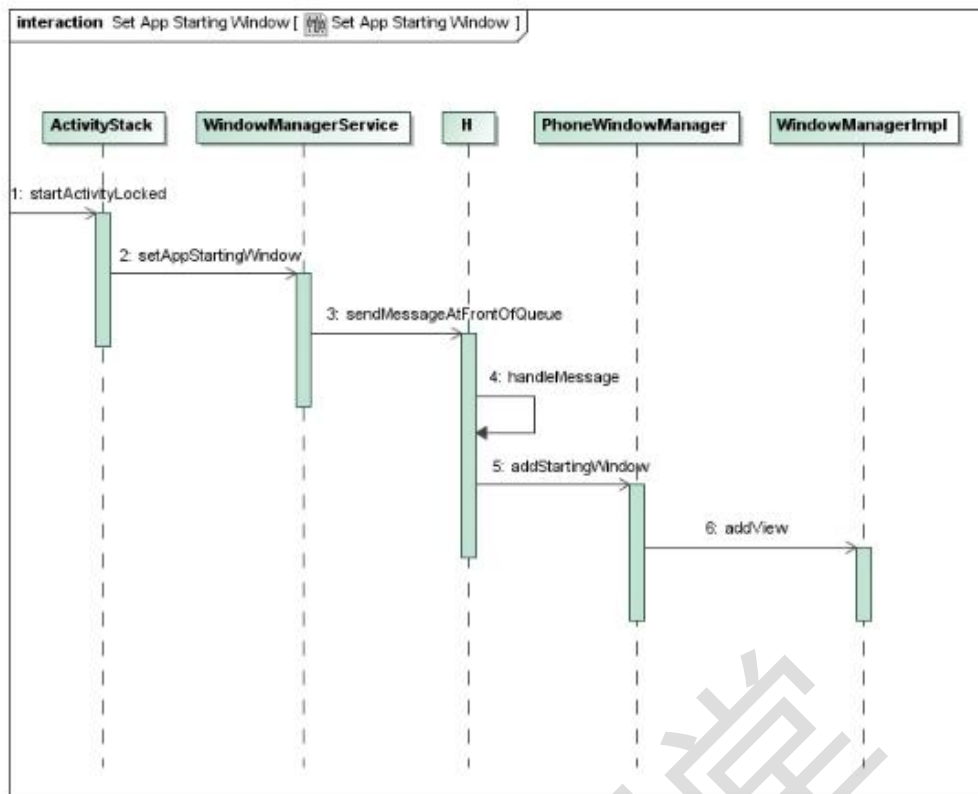


图2 Activity组件的启动窗口的显示过程

这个过程可以分为6个步骤，接下来我们就详细分析每一个步骤。

Step 1. ActivityStack.startActivityLocked

```

public class ActivityStack {
    .....

    // Set to false to disable the preview that is shown while a new activity
    // is being started.
    static final boolean SHOW_APP_STARTING_PREVIEW = true;
    .....

    private final void startActivityLocked(ActivityRecord r, boolean newTask,
        boolean doResume) {
        final int NH = mHistory.size();
        .....

        int addPos = -1;
        .....

        // Place a new activity at top of stack, so it is next to interact
        // with the user.
        if (addPos < 0) {
            addPos = NH;
        }
        .....

        // Slot the activity into the history stack and proceed
        mHistory.add(addPos, r);
        .....

        if (NH > 0) {
            // We want to show the starting preview window if we are
            // switching to a new task, or the next activity's process is
            // not currently running.
            boolean showStartingIcon = newTask;
            ProcessRecord proc = r.app;
  
```

```

        if (proc == null) {
            proc = mService.mProcessNames.get(r.processName, r.info.applicationInfo.uid);
        }
        if (proc == null || proc.thread == null) {
            showStartingIcon = true;
        }
        .....

        mService.mWindowManager.addAppToken(
            addPos, r, r.task.taskId, r.info.screenOrientation, r.fullscreen);
        boolean doShow = true;
        if (newTask) {
            // Even though this activity is starting fresh, we still need
            // to reset it to make sure we apply affinities to move any
            // existing activities from other tasks in to it.
            // If the caller has requested that the target task be
            // reset, then do so.
            if ((r.intent.getFlags()
                & Intent.FLAG_ACTIVITY_RESET_TASK_IF_NEEDED) != 0) {
                resetTaskIfNeededLocked(r, r);
                doShow = topRunningNonDelayedActivityLocked(null) == r;
            }
        }
        if (SHOW_APP_STARTING_PREVIEW && doShow) {
            // Figure out if we are transitioning from another activity that is
            // "has the same starting icon" as the next one. This allows the
            // window manager to keep the previous window it had previously
            // created, if it still had one.
            ActivityRecord prev = mResumedActivity;
            if (prev != null) {
                // We don't want to reuse the previous starting preview if:
                // (1) The current activity is in a different task.
                if (prev.task != r.task) prev = null;
                // (2) The current activity is already displayed.
                else if (prev.nowVisible) prev = null;
            }
            mService.mWindowManager.setAppStartingWindow(
                r, r.packageName, r.theme, r.nonLocalizedLabel,
                r.labelRes, r.icon, prev, showStartingIcon);
        }
        } else {
            // If this is the first activity, don't do any fancy animations,
            // because there is nothing for it to animate on top of.
            mService.mWindowManager.addAppToken(addPos, r, r.task.taskId,
                r.info.screenOrientation, r.fullscreen);
        }
        .....

        if (doResume) {
            resumeTopActivityLocked(null);
        }
    }
    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/am/ActivityStack.java中。

参数r描述的就是正在启动的Activity组件，而参数newTask和doResume描述的是是否要将该Activity组件放在一个新的任务中启动，以及是否要马上将该Activity组件启动起来。

ActivityStack类的成员变量mHistory指向的是一个ArrayList，它描述的便是系统的Activity组件堆栈。ActivityStack类的成员函数startActivityLocked首先找到正在启动的Activity组件r在系统的Activity组件堆栈中的位置addPos，然后再将正在启动的Activity组件r保存在这个位置上。

变量NH记录的是将正在启动的Activity组件r插入到系统的Activity组件堆栈中之前系统中已经启动了的Activity组件的个数。如果变量NH的值大于0，那么就说明系统需要执行一个Activity组件切换操作，即需要在系统当前激活的Activity组件和正在启动的Activity组件r之间执行一个切换操作，使得正在启动的Activity组件r成为系统接下来要激活的Activity组件。在切换的过程，需要显示切换动画，即给系统当前激活的Activity组件显示一个退出动画，而给正在启动的Activity组件r显示一个启动动画，以及需要为正在启动的Activity组件r显示一个启动窗口。另一方

面，如果变量NH的值等于0，那么系统就不需要执行Activity组件切换操作，或者为正在启动的Activity组件r显示一个启动窗口，这时候只需要为正在启动的Activity组件r创建一个窗口令牌即可。

ActivityStack类的成员变量mService指向的是一个ActivityManagerService对象，这个ActivityManagerService对象就是系统的Activity组件管理服务，它的成员变量mWindowManager指向的是一个WindowManagerService对象，这个WindowManagerService对象也就是系统的Window管理服务。通过调用WindowManagerService类的成员函数addAppToken就可以为正在启动的Activity组件r创建一个窗口令牌，这个过程可以参考前面Android窗口管理服务WindowManagerService对窗口的组织方式分析一文。

在变量NH的值大于0的情况下，ActivityStack类的成员函数startActivityLocked首先检查用来运行Activity组件r的进程是否已经启动起来了。如果已经启动起来，那么用来描述这个进程的ProcessRecord对象proc的值就不等于null，并且这个ProcessRecord对象proc的成员变量thread的值也不等于null。如果用来运行Activity组件r的进程还没有启动起来，或者Activity组件r需要运行在一个新的任务中，那么变量showStartingIcon的值就会等于true，用来描述在系统当前处于激活状态的Activity组件没有启动窗口的情况下，要为Activity组件r创建一个新的启动窗口，否则的话，就会将系统当前处于激活状态的Activity组件的启动窗口复用为Activity组件r的启动窗口。

系统当前处于激活状态的Activity组件是通过ActivityStack类的成员变量mResumedActivity来描述的，它的启动窗口可以复用为Activity组件r的启动窗口还需要满足两个额外的条件：

1. Activity组件mResumedActivity与Activity组件r运行在同一个任务中，即它们的成员变量task指向的是同一个TaskRecord对象
2. Activity组件mResumedActivity当前是不可见的，即它的成员变量nowVisible的值等于false。

这两个条件意味着Activity组件mResumedActivity与Activity组件r运行在同一个任务中，并且Activity组件mResumedActivity的窗口还没有显示出来就需要切换到Activity组件r去。

ActivityStack类的静态成员变量SHOWAPPSTARTING_PREVIEW是用描述系统是否可以为正在启动的Activity组件显示启动窗口，只有在它的值等于true，以及正在启动的Activity组件的窗口接下来是要显示出来的情况下，即变量doShow的值等于true，ActivityManagerService服务才会请求WindowManagerService服务为正在启动的Activity组件设置启动窗口。

一般来说，一个正在启动的Activity组件的窗口接下来是需要显示的，但是正在启动的Activity组件可能会设置一个标志位，用来通知ActivityManagerService服务在它启动的时候，对它运行在的任务进行重置。一个任务被重置之后，可能会导致其它的Activity组件转移到这个任务中来，并且位于这个任务的顶端。在这种情况下，系统接下来要显示的窗口就不是正在启动的Activity组件的窗口的了，而是位于正在启动的Activity组件所运行在的任务的顶端的那个Activity组件的窗口。正在启动的Activity组件所运行在的任务同时也是一个前台任务，即它顶端的Activity组件就是系统Activity组件堆栈顶端的Activity组件。

调用参数r所指向一个ActivityRecord对象的成员变量intent所描述的一个Intent对象的成员函数getFlags就可以获得正在启动的Activity组件的标志值，当这个标志值的FLAGACTIVITYRESET_TASKIF_NEEDED位等于1的时候，就说明正在启动的Activity组件通知ActivityManagerService服务对它运行在的任务进行重置。重置一个任务是通过调用ActivityStack类的成员函数resetTaskIfNeededLocked来实现的。重置了正在启动的Activity组件所运行在的任务之后，再调用ActivityStack类的成员函数topRunningNonDelayedActivityLocked来检查位于系统Activity组件堆栈顶端的Activity组件是否就是正在启动的Activity组件，就可以知道正在启动的Activity组件的窗口接下来是否需要显示的。如果需要显示的话，那么变量doShow的值就等于true。

ActivityManagerService服务请求WindowManagerService服务为正在启动的Activity组件设置启动窗口是通过调用WindowManagerService类的成员函数setAppStartingWindow来实现的。注意，ActivityManagerService服务在请求WindowManagerService服务为正在启动的Activity组件设置启动窗口之前，同样会调用WindowManagerService类的成员函数addAppToken来创建窗口令牌。

ActivityManagerService服务请求WindowManagerService服务为正在启动的Activity组件设置启动窗口之后，如果参数doResume的值等于true，那么就会调用ActivityStack类的成员函数resumeTopActivityLocked继续执行启动参数r所描述的一个Activity组件的操作，这个过程可以参考前面Android应用程序启动过程源代码分析一文。

接下来，我们就继续分析WindowManagerService类的成员函数setAppStartingWindow的实现，以便可以了解WindowManagerService服务是如何为正在启动的Activity组件设置启动窗口的。

Step 2. WindowManagerService.setAppStartingWindow

WindowManagerService类的成员函数setAppStartingWindow定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中，它的实现比较长，我们分段来阅读：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....
```

```
public void setAppStartingWindow(IBinder token, String pkg,
    int theme, CharSequence nonLocalizedLabel, int labelRes, int icon,
    IBinder transferFrom, boolean createIfNeeded) {
    .....
```

```

synchronized(mWindowMap) {
    .....

    AppWindowToken wtoken = findAppWindowToken(token);
    .....

    // If the display is frozen, we won't do anything until the
    // actual window is displayed so there is no reason to put in
    // the starting window.
    if (mDisplayFrozen || !mPolicy.isScreenOn()) {
        return;
    }

    if (wtoken.startingData != null) {
        return;
    }
}

```

参数token描述的是要设置启动窗口的Activity组件，而参数transferFrom描述的是要将启动窗口转移给Activity组件token的Activity组件。从Step 1可以知道，这两个Activity组件是运行在同一个任务中的，并且参数token描述的Activity组件Activity组件是正在启动的Activity组件，而参数transferFrom描述的Activity组件是系统当前激活的Activity组件。这段代码首先调用WindowManagerService类的成员函数findAppWindowToken来获得与参数token对应的一个类型为AppWindowToken的窗口令牌wtoken。如果这个AppWindowToken对象的成员变量startingData的值不等于null，那么就说明参数token所描述的Activity组件已经设置过启动窗口了，因此，WindowManagerService类的成员函数setAppStartingWindow就不用往下处理了。

这段代码还会检查系统屏幕当前是否处于冻结状态，即WindowManagerService类的成员变量mDisplayFrozen的值是否等于true，或者系统屏幕当前是否处于黑屏状态，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值是否等于false。如果是处于上述两种状态的话，那么WindowManagerService类的成员函数setAppStartingWindow就不用往下处理的。因为在这两种状态下，为token所描述的Activity组件设置的启动窗口是无法显示的。

我们接着往下阅读代码：

```

if (transferFrom != null) {
    AppWindowToken ttoken = findAppWindowToken(transferFrom);
    if (ttoken != null) {
        WindowState startingWindow = ttoken.startingWindow;
        if (startingWindow != null) {
            if (mStartingIconInTransition) {
                // In this case, the starting icon has already
                // been displayed, so start letting windows get
                // shown immediately without any more transitions.
                mSkipAppTransitionAnimation = true;
            }
        }
        .....

        final long origId = Binder.clearCallingIdentity();

        // Transfer the starting window over to the new
        // token.
        wtoken.startingData = ttoken.startingData;
        wtoken.startingView = ttoken.startingView;
        wtoken.startingWindow = startingWindow;
        ttoken.startingData = null;
        ttoken.startingView = null;
        ttoken.startingWindow = null;
        ttoken.startingMoved = true;
        startingWindow.mToken = wtoken;
        startingWindow.mRootToken = wtoken;
        startingWindow.mAppToken = wtoken;
        .....
        mWindows.remove(startingWindow);
        mWindowsChanged = true;
        ttoken.windows.remove(startingWindow);
        ttoken.allAppWindows.remove(startingWindow);
        addWindowToListInOrderLocked(startingWindow, true);

        // Propagate other interesting state between the
        // tokens. If the old token is displayed, we should
        // immediately force the new one to be displayed. If
        // it is animating, we need to move that animation to
        // the new one.
    }
}

```

```

        if (ttoken.allDrawn) {
            wtoken.allDrawn = true;
        }
        if (ttoken.firstWindowDrawn) {
            wtoken.firstWindowDrawn = true;
        }
        if (!ttoken.hidden) {
            wtoken.hidden = false;
            wtoken.hiddenRequested = false;
            wtoken.willBeHidden = false;
        }
        if (wtoken.clientHidden != ttoken.clientHidden) {
            wtoken.clientHidden = ttoken.clientHidden;
            wtoken.sendAppVisibilityToClients();
        }
        if (ttoken.animation != null) {
            wtoken.animation = ttoken.animation;
            wtoken.animating = ttoken.animating;
            wtoken.animLayerAdjustment = ttoken.animLayerAdjustment;
            ttoken.animation = null;
            ttoken.animLayerAdjustment = 0;
            wtoken.updateLayers();
            ttoken.updateLayers();
        }

        updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES);
        mLayoutNeeded = true;
        performLayoutAndPlaceSurfacesLocked();
        Binder.restoreCallingIdentity(origId);
        return;
    } else if (ttoken.startingData != null) {
        // The previous app was getting ready to show a
        // starting window, but hasn't yet done so. Steal it!
        .....
        wtoken.startingData = ttoken.startingData;
        ttoken.startingData = null;
        ttoken.startingMoved = true;
        Message m = mH.obtainMessage(H.ADD_STARTING, wtoken);
        // Note: we really want to do sendMessageAtFrontOfQueue() because we
        // want to process the message ASAP, before any other queued
        // messages.
        mH.sendMessageAtFrontOfQueue(m);
        return;
    }
}
}
}

```

如果参数transferFrom的值不等于null，那么就需要检查它所描述的Activity组件是否设置有启动窗口。如果设置有的话，那么就需要将它的启动窗口设置为参数token所描述的Activity组件的启动窗口。参数transferFrom所描述的Activity组件所设置的启动窗口保存在与它所对应的一个类型为AppWindowToken的窗口令牌的成员变量startingWindow或者startingData中，因此，这段代码首先调用WindowManagerService类的成员函数findAppWindowToken来获得与参数transferFrom对应的一个AppWindowToken对象ttoken。如果AppWindowToken对象ttoken的成员变量startingWindow的值不等于null，那么就说明参数transferFrom所描述的Activity组件的启动窗口已经创建出来了。另一方面，如果AppWindowToken对象ttoken的成员变量startingData的值不等于null，那么就说明用来描述参数transferFrom所描述的Activity组件的启动窗口的相关数据已经准备好了，但是这个启动窗口还未创建出来。接下来我们就分别分析这两种情况。

我们首先分析AppWindowToken对象ttoken的成员变量startingWindow的值不等于null的情况。

这时候如果WindowManagerService类的成员变量mStartingIconInTransition的值等于true，那么就说明参数transferFrom所描述的Activity组件所设置的启动窗口已经在启动的过程中了。在这种情况下，就需要跳过参数token所描述的Activity组件和参数transferFrom所描述的Activity组件的切换过程，即将WindowManagerService类的成员变量mSkipAppTransitionAnimation的值设置为true，这是因为接下来除了要将参数transferFrom所描述的Activity组件的启动窗口转移给参数token所描述的Activity组件之外，还需要将参数transferFrom所描述的Activity组件的窗口状态转移给参数token所描述的Activity组件的窗口。

将参数transferFrom所描述的Activity组件的启动窗口转移给参数token所描述的Activity组件需要执行以下几个操作：

1. 将AppWindowToken对象ttoken的成员变量startingData、startingView和startingWindow的值设置到AppWindowToken对象wtoken的对应成员变量中去，其中，成员变量startingData指向的是一个StartingData对象，它描述的是用来创建启动窗口的相关数据，成员变量startingView指向的是一个View对象，它描述的是启动窗口的顶层

视图，成员变量startingWindow指向的是一个WindowState对象，它描述的就是启动窗口。 2. 将AppWindowToken对象ttoken的成员变量startingData、startingView和startingWindow的值设置为null，这是因为参数transferFrom所描述的Activity组件的启动窗口已经转移给参数token所描述的Activity组件了。 3. 将原来属于参数transferFrom所描述的Activity组件的启动窗口startingWindow的成员变量mToken、mRootToken和mAppToken的值设置为wtoken，因为这个启动窗口现在已经属于参数token所描述的Activity组件了。

将参数transferFrom所描述的Activity组件的窗口状态转移给参数token所描述的Activity组件的窗口需要执行几个操作：

1. 将启动窗口startingWindow从窗口堆栈中删除，即从WindowManagerService类的成员变量mWindows所描述的一个ArrayList中删除。startingWindow从属于窗口令牌ttoken的窗口列表中删除，即从AppWindowToken对象ttoken的成员变量windows和allAppWindows所描述的两个ArrayList中删除。

1. 调用WindowManagerService类的成员函数addWindowToListInOrderLocked重新将启动窗口startingWindow插入到窗口堆栈中去。注意，因为这时候启动窗口startingWindow已经被设置为参数token所描述的Activity组件了，因此，在重新将它插入到窗口堆栈中去的时候，它就会位于参数token所描述的Activity组件的窗口的上面，这一点可以参考前面Android窗口管理服务WindowManagerService对窗口的组织方式分析一文。pWindowToken对象ttoken的成员变量allDrawn和firstWindowDrawn的值等于true，那么就说明与AppWindowToken对象ttoken对应的所有窗口或者第一个窗口已经绘制好了，这时候也需要分别将AppWindowToken对象wtoken的成员变量allDrawn和firstWindowDrawn的值设置为true，以便可以迫使那些与AppWindowToken对象wtoken对应的窗口接下来可以马上显示出来。

1. 如果AppWindowToken对象ttoken的成员变量hidden的值等于false，那么就说明参数transferFrom所描述的Activity组件是处于可见状态的，这时候就需要将AppWindowToken对象wtoken的成员变量hidden、hiddenRequested和willBeHidden的值也设置为false，以便表示参数token所描述的Activity组件也是处于可见状态的。WindowToken类的成员变量clientHidden描述的是对应的Activity组件在应用程序进程这一侧的可见状态。如果AppWindowToken对象wtoken和ttoken的成员变量clientHidden的值不相等，那么就需要将AppWindowToken对象ttoken的成员变量clientHidden的值设置给AppWindowToken对象wtoken的成员变量clientHidden，并且调用AppWindowToken对象wtoken的成员函数sendAppVisibilityToClients来通知相应的应用程序进程，运行在它里面的参数token所描述的Activity组件的可见状态。

1. 如果AppWindowToken对象ttoken的成员变量animation的值不等于null，那么就说明参数transferFrom所描述的Activity组件的窗口正在显示动画，那么就需要将该动画转移给参数token所描述的Activity组件的窗口，即将AppWindowToken对象ttoken的成员变量animation、animating和animLayerAdjustment的值设置到AppWindowToken对象wtoken的对应成员变量，并且将AppWindowToken对象ttoken的成员变量animation和animLayerAdjustment的值设置为null和0。最后还需要重新计算与AppWindowToken对象ttoken和wtoken所对应的窗口的Z轴位置。于前面的操作导致窗口堆栈的窗口发生了变化，因此就需要调用WindowManagerService类的成员函数updateFocusedWindowLocked来重新计算系统当前可获得焦点的窗口，以及调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来刷新系统的UI。

我们接着分析AppWindowToken对象ttoken的成员变量startingData的值不等于null的情况。

这时候由于WindowManagerService服务还没有参数transferFrom所描述的Activity组件创建启动窗口，因此，这段代码只需要将用创建这个启动窗口的相关数据转移给参数token所描述的Activity组件就可以了，即将AppWindowToken对象ttoken的成员变量startingData的值设置给AppWindowToken对象wtoken的成员变量startingData，并且将AppWindowToken对象ttoken的成员变量startingData的值设置为null。

由于这时候参数token所描述的Activity组件的启动窗口还没有创建出来，因此，接下来就会向WindowManagerService服务所运行在线程的消息队列的头部插入一个类型ADD_STARTING的消息。当这个消息被处理的时候，WindowManagerService服务就会为参数token所描述的Activity组件创建一个启动窗口。

WindowManagerService类的成员变量mH指向的是一个类型为H的对象。H是WindowManagerService的一个内部类，它是从Handler类继承下来的，因此，调用它的成员函数sendMessageAtFrontOfQueue就可以往一个线程的消息队列的头部插入一个消息。又由于 WindowManagerService类的成员变量mH所指向的一个H对象是在WindowManagerService服务所运行在的线程中创建的，因此，调用它的成员函数sendMessageAtFrontOfQueue发送的消息是保存在WindowManagerService服务所运行在的线程的消息队列中的。

如果参数transferFrom所描述的Activity组件没有启动窗口或者启动窗口数据转移给参数token所描述的Activity组件，那么接下来就可能需要为参数token所描述的Activity组件创建一个新的启动窗口，如最后一段代码所示：

```
// There is no existing starting window, and the caller doesn't
// want us to create one, so that's it!
if (!createIfNeeded) {
    return;
}

// If this is a translucent or wallpaper window, then don't
// show a starting window -- the current effect (a full-screen
// opaque starting window that fades away to the real contents
// when it is ready) does not work for this.
if (theme != 0) {
    AttributeCache.Entry ent = AttributeCache.instance().get(pkg, theme,
        com.android.internal.R.styleable.Window);
    if (ent.array.getBoolean(
        com.android.internal.R.styleable.Window_windowIsTranslucent, false)) {
        return;
    }
    if (ent.array.getBoolean(
        com.android.internal.R.styleable.Window_windowIsFloating, false)) {
        return;
    }
    if (ent.array.getBoolean(
        com.android.internal.R.styleable.Window_windowShowWallpaper, false)) {
        return;
    }
}

mStartingIconInTransition = true;
wtoken.startingData = new StartingData(
    pkg, theme, nonLocalizedLabel,
    labelRes, icon);
Message m = mH.obtainMessage(H.ADD_STARTING, wtoken);
// Note: we really want to do sendMessageAtFrontOfQueue() because we
// want to process the message ASAP, before any other queued
// messages.
mH.sendMessageAtFrontOfQueue(m);
}
}

.....
}
```

如果参数createIfNeeded的值等于false，那么就说明不可以为参数token所描述的Activity组件创建一个新的启动窗口，因此，这时候WindowManagerService类的成员函数setAppStartingWindow就直接返回而不往下处理了。

另一方面，如果参数token所描述的Activity组件的窗口设置有一个主题，即参数theme的值不等于0，那么该窗口就有可能是：

1. 背景是半透明的；
2. 浮动窗口，即是一个壁纸窗口或者一个输入法窗口；
3. 需要显示壁纸（背景也是半透明的）。

由于浮动窗口和背景半透明的窗口是不可以显示启动窗口的，因此，在上述三种情况下，WindowManagerService类的成员函数setAppStartingWindow也是直接返回而不往下处理了。

通过了上面的检查之后，这段代码就可以为参数token所描述的Activity组件创建一个启动窗口了，不过这个启动窗口不是马上就创建的，而通过一个类型为ADD_STARTING的消息来驱动创建的。这个类型为ADD_STARTING的消息是需要发送到WindowManagerService服务所运行在的线程的消息队列的头部去的。在发送这个类型为ADD_STARTING的消息之前，这段代码首先会创建一个StartingData对象，并且保存在AppWindowToken对象wtoken的成员变量startingData中，用来封装创建启动窗口所需要的数据。

如上所述，通过调用WindowManagerService类的成员变量mH的成员函数sendMessageAtFrontOfQueue可以向WindowManagerService服务所运行在的线程的消息队列的头部发送一个类型为ADD_STARTING的消息。注意，在发送这个消息之前，这段代码还会将WindowManagerService类的成员变量mStartingIconInTransition的值设置为true，以便

可以表示WindowManagerService服务正在为正在启动的Activity组件创建启动窗口。

接下来，我们就继续分析定义在WindowManagerService内部的H类的成员函数sendMessageToFrontOfQueue的实现，以便可以了解Activity组件的启动窗口的创建过程。

Step 3. H.sendMessageToFrontOfQueue

H类的成员函数sendMessageToFrontOfQueue是从父类Handler继承下来的，因此，这一步调用的实际上是Handler类的成员函数sendMessageToFrontOfQueue。Handler类的成员函数sendMessageToFrontOfQueue用来向消息队列头部的插入一个新的消息，以便这个消息可以下一次消息循环中就能得到处理。在前面Android应用程序线程消息循环模型分析一文中，我们已经分析过消息队列发送消息的过程了，这里不再详述。

从上面的调用过程可以知道，这一步所发送的消息的类型为ADD_STARTING，并且是向WindowManagerService服务所运行在的线程的消息队列发送的。当这个消息得到处理的时候，H类的成员函数handleMessage就会被调用，因此，接下来我们就继续分析H类的成员函数handleMessage的实现，以便可以了解Activity组件的启动窗口的创建过程。

Step 4. H.handleMessage

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class H extends Handler {
        .....

        public void handleMessage(Message msg) {
            switch (msg.what) {
                .....

            case ADD_STARTING: {
                final AppWindowToken wtoken = (AppWindowToken)msg.obj;
                final StartingData sd = wtoken.startingData;

                if (sd == null) {
                    // Animation has been canceled... do nothing.
                    return;
                }

                .....

                View view = null;
                try {
                    view = mPolicy.addStartingWindow(
                        wtoken.token, sd.pkg,
                        sd.theme, sd.nonLocalizedLabel, sd.labelRes,
                        sd.icon);
                } catch (Exception e) {
                    .....
                }

                if (view != null) {
                    boolean abort = false;

                    synchronized(mWindowMap) {
                        if (wtoken.removed || wtoken.startingData == null) {
                            // If the window was successfully added, then
                            // we need to remove it.
                            if (wtoken.startingWindow != null) {
                                .....
                                wtoken.startingWindow = null;
                                wtoken.startingData = null;
                                abort = true;
                            }
                        } else {
                            wtoken.startingView = view;
                        }
                        .....
                    }

                    if (abort) {
                        try {
                            mPolicy.removeStartingWindow(wtoken.token, view);
                        } catch (Exception e) {
```

```

        .....
    }
}
} break;
.....
}
}
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。参数msg指向一个Message对象，从前面的Step 2可以知道，它的成员变量what的值等于ADD_STARTING，而成员变量obj指向的是一个AppWindowToken对象。这个AppWindowToken对象描述的就是要创建启动窗口的Activity组件。H类的成员函数handleMessage首先获得该AppWindowToken对象，并且保存在变量wtoken中。

有了AppWindowToken对象wtoken，H类的成员函数handleMessage就可以通过它的成员变量startingData来获得一个StartingData对象，并且保存在变量sd中。StartingData对象sd里面保存了创建启动窗口所需要的参数，因此，H类的成员函数handleMessage就可以通过这些参数来调用外部类WindowManagerService的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数addStartingWindow来为AppWindowToken对象wtoken所描述的Activity组件创建启动窗口。

如果能够成功地为AppWindowToken对象wtoken所描述的Activity组件创建启动窗口，那么PhoneWindowManager类的成员函数addStartingWindow就返回该Activity组件的启动窗口的顶层视图。H类的成员函数handleMessage获得这个视图之后，就会将它保存在变量view中。

由于在创建这个启动窗口的过程中，AppWindowToken对象wtoken所描述的Activity组件可能已经被移除，即AppWindowToken对象wtoken的成员变量removed的值等于true，或者它的启动窗口已经被转移给另外一个Activity组件了，即AppWindowToken对象wtoken的成员变量startingData的值等于null。在这两种情况下，如果AppWindowToken对象wtoken的成员变量startingWindow的值不等于null，那么就说明前面不仅成功地为AppWindowToken对象wtoken所描述的Activity组件创建了启动窗口，并且这个启动窗口也已经成功地增加到WindowManagerService服务中去了，因此，就需要将该启动窗口从WindowManagerService服务中删除，这是通过调用外部类WindowManagerService的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数removeStartingWindow来实现的。注意，在删除之前，还会将AppWindowToken对象wtoken的成员变量startingWindow和startingData的值均设置为null，以表示它所描述的Activity组件没有一个关联的启动窗口。

另一方面，如果AppWindowToken对象wtoken所描述的Activity组件没有被移除，并且它的启动窗口了没有转移给另外一个Activity组件，那么H类的成员函数handleMessage就会将前面得到的启动窗口的顶层视图保存在AppWindowToken对象wtoken的成员变量startingView中。注意，这时候AppWindowToken对象wtoken的成员变量startingWindow会指向一个WindowState对象，这个WindowState对象是由PhoneWindowManager类的成员函数addStartingWindow请求WindowManagerService服务创建的。

接下来，我们就继续分析PhoneWindowManager类的成员函数addStartingWindow的实现，以便可以了解它是如何为一个Activity组件创建一个启动窗口，并且将这个启动窗口增加到WindowManagerService服务中去的。

```

Step 5. PhoneWindowManager.addStartingWindow
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    static final boolean SHOW_STARTING_ANIMATIONS = true;
    .....

    public View addStartingWindow(IBinder appToken, String packageName,
                                int theme, CharSequence nonLocalizedLabel,
                                int labelRes, int icon) {
        if (!SHOW_STARTING_ANIMATIONS) {
            return null;
        }
        if (packageName == null) {
            return null;
        }

        try {
            Context context = mContext;
            boolean setTheme = false;
            .....
            if (theme != 0 || labelRes != 0) {
                try {
                    context = context.createPackageContext(packageName, 0);

```

```

        if (theme != 0) {
            context.setTheme(theme);
            setTheme = true;
        }
    } catch (PackageManager.NameNotFoundException e) {
        // Ignore
    }
}

if (!setTheme) {
    context.setTheme(com.android.internal.R.style.Theme);
}

Window win = PolicyManager.makeNewWindow(context);
if (win.getWindowStyle().getBoolean(
    com.android.internal.R.styleable.Window_windowDisablePreview, false)) {
    return null;
}

Resources r = context.getResources();
win.setTitle(r.getText(labelRes, nonLocalizedLabel));

win.setType(
    WindowManager.LayoutParams.TYPE_APPLICATION_STARTING);
// Force the window flags: this is a fake window, so it is not really
// touchable or focusable by the user. We also add in the ALT_FOCUSABLE_IM
// flag because we do know that the next window will take input
// focus, so we want to get the IME window up on top of us right away.
win.setFlags(
    WindowManager.LayoutParams.FLAG_NOT_TOUCHABLE|
    WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE|
    WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM,
    WindowManager.LayoutParams.FLAG_NOT_TOUCHABLE|
    WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE|
    WindowManager.LayoutParams.FLAG_ALT_FOCUSABLE_IM);

win.setLayout(WindowManager.LayoutParams.MATCH_PARENT,
    WindowManager.LayoutParams.MATCH_PARENT);

final WindowManager.LayoutParams params = win.getAttributes();
params.token = appToken;
params.packageName = packageName;
params.windowAnimations = win.getWindowStyle().getResourceId(
    com.android.internal.R.styleable.Window_windowAnimationStyle, 0);
params.setTitle("Starting " + packageName);

WindowManagerImpl wm = (WindowManagerImpl)
    context.getSystemService(Context.WINDOW_SERVICE);
View view = win.getDecorView();

if (win.isFloating()) {
    // Whoops, there is no way to display an animation/preview
    // of such a thing! After all that work... let's skip it.
    // (Note that we must do this here because it is in
    // getDecorView() where the theme is evaluated... maybe
    // we should peek the floating attribute from the theme
    // earlier.)
    return null;
}

.....

wm.addView(view, params);

// Only return the view if it was successfully added to the
// window manager... which we can tell by it having a parent.
return view.getParent() != null ? view : null;
} catch (WindowManagerImpl.BadTokenException e) {
    // ignore
    .....
} catch (RuntimeException e) {

```



```

        // don't crash if something else bad happens, for example a
        // failure loading resources because we are loading from an app
        // on external storage that has been unmounted.
        .....
    }

    return null;
}

.....
}

```

这个函数定义在文件frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java中。PhoneWindowManager类有一个静态成员变量SHOW_STARTING_ANIMATIONS。如果它的值等于false，那么就禁止显示Activity组件的启动窗口。此外，如果参数packageName的值等于null，那么禁止显示Activity组件的启动窗口，即调用者需要指定要显示启动窗口的Activity组件的包名称。

如果参数theme和labelRes的值不等于0，那么就说明调用者指定了启动窗口的主题和标题，这时候就需要创建一个代表了要显示启动窗口的Activity组件的运行上下文，以便可以从里面获得指定的主题和标题。否则的话，启动窗口的运行上下文就是通过PhoneWindowManager类的成员变量mContext来描述的。PhoneWindowManager类的成员变量mContext是在其构造函数中初始化的，它描述的WindowManagerService服务的运行上下文。注意，如果调用者没有指定启动窗口的主题，那么默认使用的主题就为com.android.internal.R.style.Theme。

初始化好启动窗口的运行上下文之后，即初始化好变量context所指向的一个Context对象之后，PhoneWindowManager类的成员函数addStartingWindow接下来就可以以它来参数来调用PolicyManager类的成员函数makeNewWindow来创建一个窗口了，并且将这个窗口保存在变量win中。从前面Android应用程序窗口（Activity）的窗口对象（Window）的创建过程分析一文可以知道，PolicyManager类的成员函数makeNewWindow创建的是一个类型为PhoneWindow的窗口。注意，如果这个类型为PhoneWindow的窗口的com.android.internal.R.styleable.Window_windowDisablePreview属性的值等于true，那么就说明不允许为参数appToken所描述的Activity组件显示启动窗口。

PolicyManager类的成员函数makeNewWindow接下来还会继续设置前面所创建的窗口win的以下属性：

1. 窗口类型：设置为WindowManager.LayoutParams.TYPE_APPLICATION_STARTING，即设置为启动窗口类型；
2. 窗口标题：由参数labelRes、nonLocalizedLabel，以及窗口的运行上下文context来确定；
3. 窗口标志：分别将indowManager.LayoutParams.FLAG_NOT_TOUCHABLE、WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE和WindowManager.LayoutParams.FLAG_ALLOW_FOCUSABLE_IM位设置为1，即不可接受触摸事件和不可获得焦点，但是可以接受输入法窗口；
4. 窗口大小：设置为WindowManager.LayoutParams.MATCH_PARENT，即与父窗口一样大，但是由于这是一个顶层窗口，因此实际上是指与屏幕一样大；
5. 布局参数：包括窗口所对应的窗口令牌（token）和包名（packageName），以及窗口所使用的动画类型（windowAnimations）和标题（title）。

从第5点可以看出，一个Activity组件的启动窗口和它本身的窗口都是对应同一个窗口令牌的，因此，它们在窗口堆栈中就属于同一组窗口。

设置好窗口win的属性之后，接下来调用它的成员函数getDecorView就可以将它的顶层视图创建出来，并且保存在变量view中。从前面Android应用程序窗口（Activity）的视图对象（View）的创建过程分析一文可以知道，这个顶层视图的类型为DecorView。

从前面Android应用程序窗口（Activity）实现框架简要介绍和学习计划一文可以知道，每一个进程都有一个本地窗口管理服务，这个本地窗口管理服务是由WindowManagerImpl类来实现的，负责维护进程内的所有窗口的视图对象。通过调用WindowManagerImpl类的成员函数addView就可以将一个窗口的视图增加到本地窗口管理服务中去，以便这个视图可以接受本地窗口管理服务的管理。同时，WindowManagerImpl类的成员函数addView还会请求WindowManagerService服务为其所增加的窗口视图创建一个WindowState对象，以便WindowManagerService服务可以维护对应的窗口的运行状态。

注意，如果前面所创建的窗口win是一个浮动的窗口，即它的成员函数isFloating的值等于true，那么窗口win就不能作为启动窗口来使用。这里之所以要在创建了窗口win的顶层视图之后，再来判断窗口win是否是一个浮动窗口，是因为一个窗口只有在创建了顶层视图之后，才能确定是否是浮动窗口。

如果能够成功地将前面所创建的启动窗口win的顶层视图view增加到本地窗口管理服务中，那么顶层视图view就会有一个父视图，即它的成员函数getParent的返回值不等于null，这时候PhoneWindowManager类的成员函数addStartingWindow就会将顶层视图view返回给调用者，表示已经成功地为参数appToken所描述的Activity组件创建了一个启动窗口。从前面Android应用程序窗口（Activity）的视图对象（View）的创建过程分析一文可以知道，一个窗口的顶层视图的父视图就是通过一个ViewRoot对象来描述的，这个ViewRoot对象负责渲染它的子视图的UI，以及和WindowManagerService服务通信。

从上面的描述就可以知道，一个Activity组件的启动窗口在创建完成之后，就会通过调用WindowManagerImpl类的成员函数addView来将其增加到WindowManagerService服务中去，因此，接下来我们就继续分析WindowManagerImpl类的

成员函数addView的实现。

Step 6. WindowManagerImpl.addView

这个函数定义在文件frameworks/base/core/java/android/view/WindowManagerImpl.java中，它的具体实现可以参考前面Android应用程序窗口（Activity）的视图对象（View）的创建过程分析一文。总的来说，这一步最终会通过调用一个实现了IWindowSession接口的Binder代理对象的成员函数add向WindowManagerService服务发送一个Binder进程间通信请求，这个Binder进程间通信请求最终是由WindowManagerService类的成员函数addWindow来处理的，即为指定的窗口创建一个WindowState对象，以便WindowManagerService服务可以维护它的状态。这个过程又可以参考前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文。

这一步执行完成之后，一个新创建的Activity组件的启动窗口就增加到WindowManagerService服务中去了，这样，WindowManagerService服务就可以下次刷新系统UI时，将该启动窗口显示出来。

接下来我们继续分析Activity组件的启动窗口结束显示的过程。

二. Activity组件的启动窗口的结束过程

Activity组件启动完成之后，它的窗口就会显示出来，这时候WindowManagerService服务就需要将它的启动窗口结束掉。我们知道，在WindowManagerService服务中，每一个窗口都对应有一个WindowState对象。每当WindowManagerService服务需要显示一个窗口的时候，就会调用一个对应的WindowState对象的成员函数performShowLocked。WindowState类的成员函数performShowLocked在执行的过程中，就会检查当前正在处理的WindowState对象所描述的窗口是否设置有启动窗口。如果有的话，那么就会将它结束掉。这个过程如图3所示。

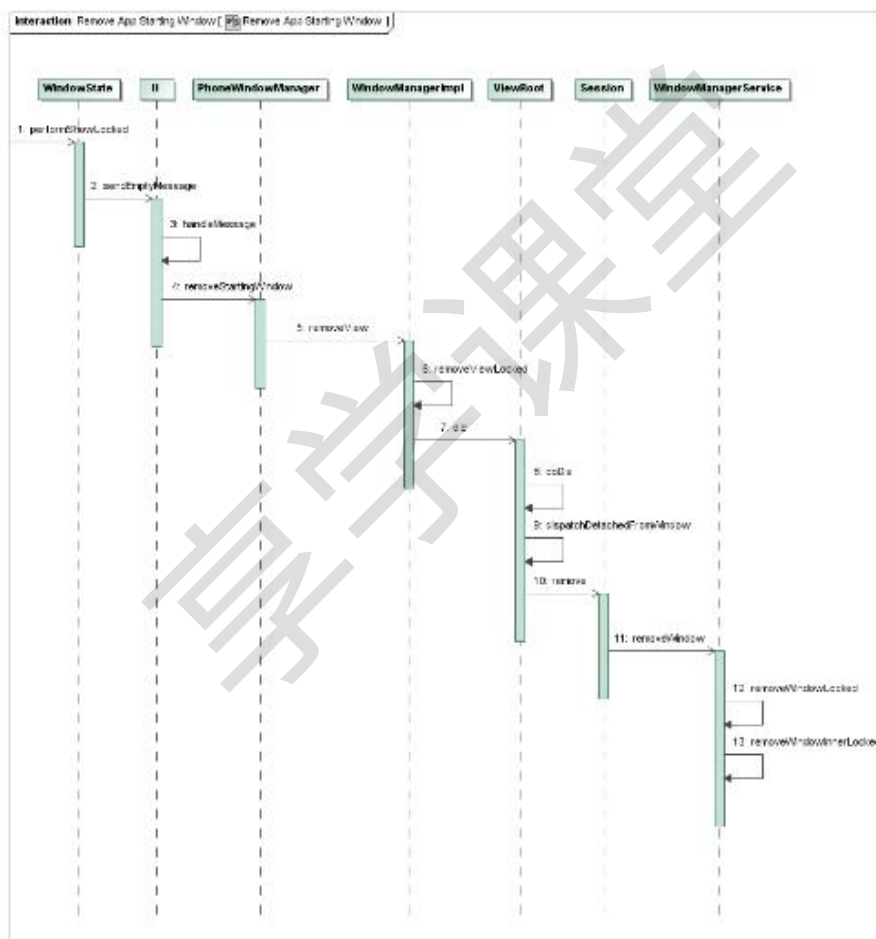


图3 Activity组件的启动窗口的结束过程

这个过程可以分为13个步骤，接下来我们就详细分析每一个步骤。

Step 1. WindowState.performShowLocked

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {
        .....

        // This must be called while inside a transaction.
        boolean performShowLocked() {
```

```

.....

if (mReadyToShow && isReadyForDisplay()) {
    .....

    if (!showSurfaceRobustlyLocked(this)) {
        return false;
    }
    mLastAlpha = -1;
    mHasDrawn = true;
    mLastHidden = false;
    mReadyToShow = false;
    enableScreenIfNeededLocked();

    applyEnterAnimationLocked(this);

    int i = mChildWindows.size();
    while (i > 0) {
        i--;
        WindowState c = mChildWindows.get(i);
        if (c.mAttachedHidden) {
            c.mAttachedHidden = false;
            if (c.mSurface != null) {
                c.performShowLocked();
                // It hadn't been shown, which means layout not
                // performed on it, so now we want to make sure to
                // do a layout. If called from within the transaction
                // loop, this will cause it to restart with a new
                // layout.
                mLayoutNeeded = true;
            }
        }
    }
}

if (mAttrs.type != TYPE_APPLICATION_STARTING
    && mAppToken != null) {
    mAppToken.firstWindowDrawn = true;

    if (mAppToken.startingData != null) {
        .....
        // If this initial window is animating, stop it -- we
        // will do an animation to reveal it from behind the
        // starting window, so there is no need for it to also
        // be doing its own stuff.
        if (mAnimation != null) {
            mAnimation = null;
            // Make sure we clean up the animation.
            mAnimating = true;
        }
        mFinishedStarting.add(mAppToken);
        mH.sendMessage(H.FINISHED_STARTING);
    }
    mAppToken.updateReportedVisibilityLocked();
}
}
return true;
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。一个窗口只有在准备就绪显示之后，才能调用WindowState类的成员函数performShowLocked来显示它。一个窗口在什么情况下才是准备就绪显示的呢？一是用来描述它的一个WindowState对象的成员变量mReadyToShow的值等于true，二是调用该WindowState对象的成员函数isReadyForDisplay的返回值也等于true。WindowState类的成员函数isReadyForDisplay主要验证一个窗口中是否是处于可见状态、并且不是处于销毁的状态。一个窗口必须要处于可见状

态，并且不是正在被销毁，那么它才是准备就绪显示的。此外，一个窗口如果是一个子窗口，那么只有当它以及它的父窗口都是可见的，那么它才是处于可见状态的。

通过了上面的检查之后，`WindowState`类的成员函数`performShowLocked`就会调用`WindowManagerService`类的成员函数`showSurfaceRoot`，以便以后在显示窗口之前，都可以更新窗口的`Alpha`通道值。2. 将对应的`WindowState`对象的成员变量`mHasDrawn`的值设置为`true`，以便以后在显示窗口之前，都可以更新窗口的`Alpha`通道值。

public class WindowManagerService extends IWindowManager.Stub

implements Watchdog.Monitor {

.....

private final class H extends Handler {

.....

public void handleMessage(Message msg) {

switch (msg.what) {

.....

case FINISHED_STARTING: {

IBinder token = null;

View view = null;

while (true) {

synchronized (mWindowMap) {

final int N = mFinishedStarting.size();

if (N <= 0) {

break;

}

AppWindowToken wtoken = mFinishedStarting.remove(N-1);

.....

if (wtoken.startingWindow == null) {

continue;

}

view = wtoken.startingView;

token = wtoken.token;

wtoken.startingData = null;

wtoken.startingView = null;

wtoken.startingWindow = null;

}

try {

mPolicy.removeStartingWindow(token, view);

} catch (Exception e) {

.....

}

}

} break;

.....

}

}

}

.....

}

这个函数定义在文件`frameworks/base/services/java/com/android/server/WindowManagerService.java`中。从前面的Step 1可以知道，这时候用来描述需要结束启动窗口的Activity组件的AppWindowToken对象都都保存在WindowManagerService类的成员变量mFinishedStarting所描述的一个ArrayList中，因此，通过遍历保存在该ArrayList中的每一个AppWindowToken对象，就可以结束对应的启动窗口。

从前面第一部分的内容可以知道，如果一个Activity组件设置有启动窗口，那么这个启动窗口的顶层视图就会保存在用来描述该Activity组件的一个AppWindowToken对象的成员变量startingView中。获得了一个启动窗口的顶层视图之后，就可以调用WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数removeStartingWindow来通知WindowManagerService服务删除该启动窗口，从而可以结束显示该启动窗口。

注意，在调用PhoneWindowManager类的成员函数removeStartingWindow来通知WindowManagerService服务删除一个启动窗口之前，还需要将来描述该启动窗口的宿主Activity组件的一个AppWindowToken对象的成员变量startingData、startingView和startingWindow的值设置为null，因为这三个成员变量保存的数据都是与启动窗口相关的。

接下来，我们就继续分析PhoneWindowManager类的成员函数removeStartingWindow的实现，以便可以了解Activity组件的启动窗口的结束过程。

Step 4. PhoneWindowManager.removeStartingWindow

```
public class PhoneWindowManager implements WindowManagerPolicy {
    .....

    public void removeStartingWindow(IBinder appToken, View window) {
        .....

        if (window != null) {
            WindowManagerImpl wm = (WindowManagerImpl) mContext.getSystemService(Context.WINDOW_SERVICE);
            wm.removeView(window);
        }
    }

    .....
}
```

这个函数定义在文件frameworks/base/policy/src/com/android/internal/policy/impl/PhoneWindowManager.java中。PhoneWindowManager类的成员函数removeStartingWindow首先获得一个本地窗口管理服务。从前面第一部分的Step 5可以知道，这个本地窗口管理服务是由WindowManagerImpl类来实现的。获得了本地窗口管理服务之后，就可以调用它的成员函数removeView来请求WindowManagerService服务删除参数window所描述的一个启动窗口。

Step 5. WindowManagerImpl.removeView

```
public class WindowManagerImpl implements WindowManager {
    .....

    public void removeView(View view) {
        synchronized (this) {
            int index = findViewLocked(view, true);
            View curView = removeViewLocked(index);
            if (curView == view) {
                return;
            }
        }
        .....
    }

    .....

    private View[] mViews;
    private ViewRoot[] mRoots;

    .....
}
```

这个函数定义在文件frameworks/base/core/java/android/view/WindowManagerImpl.java中。从前面Android应用程序窗口（Activity）的视图对象（View）的创建过程分析一文可以知道，在应用程序进程中，每一个窗口的顶层视图都对应有一个ViewRoot对象，它们的对应关系是由WindowManagerImpl类来维护的。当前正在处理的进程即为WindowManagerService服务所运行在的进程，也就是System进程，它与普通的应用程序进程一样，也可以创建窗口，即它在内部也会通过WindowManagerImpl类管理窗口的顶层视图及其所对应的ViewRoot对象。

WindowManagerImpl类是如何管理进程中的窗口顶层视图与ViewRoot对象的对应关系的呢？在WindowManagerImpl类中，有两个成员变量mViews和mRoots，它们分别指向两个大小相等的View数组和ViewRoot数组，用来保存进程中的窗口顶层视图对象和ViewRoot对象，其中，第mViews[i]个窗口顶层视图与第mRoots[i]个ViewRoot对象是一一对应的。

WindowManagerImpl类的成员函数removeView首先调用另外一个成员函数findViewLocked来找到参数view所描述的一个启动窗口的顶层视图在数组mViews中的位置index，然后再以这个位置为参数来调用另外一个成员函数removeViewLocked，以便可以删除该启动窗口的顶层视图。

Step 6. WindowManagerImpl.removeViewLocked

```
public class WindowManagerImpl implements WindowManager {
    .....

    View removeViewLocked(int index) {
        ViewRoot root = mRoots[index];
        View view = root.getView();
        .....
    }
}
```

```

        root.die(false);
        .....

        return view;
    }

    .....
}

```

这个函数定义在文件frameworks/base/core/java/android/view/WindowManagerImpl.java中。从前面的调用过程可以知道，WindowManagerImpl类的成员函数removeViewLocked要删除的是在数组mViews中第index个位置的视图，同时，与这个即将要被删除的视图所对应的ViewRoot对象保存在数组mRoots中的第index个位置上。有了这个ViewRoot对象之后，就可以调用它的成员函数die来删除指定的窗口了。

Step 7. ViewRoot.die

```

public final class ViewRoot extends Handler implements ViewParent,
    View.AttachInfo.Callbacks {
    .....

    public void die(boolean immediate) {
        if (immediate) {
            doDie();
        } else {
            sendEmptyMessage(DIE);
        }
    }

    .....
}

```

这个函数定义在frameworks/base/core/java/android/view/ViewRoot.java文件中。当参数immediate的值等于true的时候，ViewRoot类的成员函数die直接调用另外一个成员函数doDie来删除与当前正在处理的ViewRoot对象所对应的一个View对象，否则的话，ViewRoot类的成员函数die就会先向当前线程的消息队列发送一个类型为DIE消息，然后等到该消息被处理的时候，再调用ViewRoot类的成员函数doDie来删除与当前正在处理的ViewRoot对象所对应的一个View对象。

因此，无论如何，ViewRoot类的成员函数die最终都会调用到另外一个成员函数doDie来删除与当前正在处理的ViewRoot对象所对应的一个View对象，接下来我们就继续分析它的实现。

Step 8. ViewRoot.doDie

```

public final class ViewRoot extends Handler implements ViewParent,
    View.AttachInfo.Callbacks {
    .....

    void doDie() {
        .....

        synchronized (this) {
            .....

            if (mAdded) {
                mAdded = false;
                dispatchDetachedFromWindow();
            }
        }
    }

    .....
}

```

这个函数定义在frameworks/base/core/java/android/view/ViewRoot.java文件中。ViewRoot类有一个成员变量mAdded，当它的值等于true的时候，就表示当前正在处理的ViewRoot对象有一个关联的View对象，因此，这时候就可以调用另外一个成员函数dispatchDetachedFromWindow来删除这个View对象。由于删除了这个View对象之后，当前正在处理的ViewRoot对象就不再关联有View对象了，因此，ViewRoot类的成员函数doDie在调用另外一个成员函数dispatchDetachedFromWindow之前，还会将成员变量mAdded的值设置为false。

Step 9. ViewRoot.dispatchDetachedFromWindow

```

public final class ViewRoot extends Handler implements ViewParent,
    View.AttachInfo.Callbacks {
    .....
}

```

```

static IWindowSession sWindowSession;
.....

final W mWindow;
.....

void dispatchDetachedFromWindow() {
    .....

    try {
        sWindowSession.remove(mWindow);
    } catch (RemoteException e) {
    }

    .....
}

.....
}

```

这个函数定义在frameworks/base/core/java/android/view/ViewRoot.java文件中。

从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文可以知道，每一个与UI相关的应用程序进程，都与WindowManagerService服务建立有一个连接，这个连接是通过一个实现了IWindowSession接口的Binder代理对象来描述的，并且这个Binder代理对象就保存在ViewRoot类的静态成员变量sWindowSession中，它引用了运行在WindowManagerService服务这一侧的一个类型为Session的Binder本地对象。

注意，由于当前进程即为WindowManagerService服务所运行在的进程，因此，这时候ViewRoot类的静态成员变量sWindowSession保存的其实不是一个实现了IWindowSession接口的Binder代理对象，而是一个实现了IWindowSession接口的类型为Session的Binder本地对象。这是因为Binder驱动发现Client和服务是位于同一个进程的时候，就会将Service的本地接口直接返回给Client，而不会将Service的代理对象返回给Client，这样就可以避免在同一进程中执行Binder进程间调用也会经过Binder驱动来中转。有关Binder进程间通信的内容，可以参考前面Android进程间通信（IPC）机制Binder简要介绍和学习计划这个系列的文章。

从前面Android应用程序窗口（Activity）与WindowManagerService服务的连接过程分析一文还可以知道，进程中的每一个窗口都有一个对应的W对象，这个W对象就保存在ViewRoot类的成员变量mWindow中。有了这个W对象之后，ViewRoot类的成员函数dispatchDetachedFromWindow就可以调用静态成员变量sWindowSession所描述的一个Session对象的成员函数remove来通知WindowManagerService服务删除一个对应的WindowState对象。从前面的调用过程可以知道，这个WindowState对象描述的是一个Activity组件的启动窗口，因此，WindowManagerService服务删除了这个WindowState对象之后，就相当于是一个Activity组件的启动窗口结束掉了。

接下来，我们就继续分析Session类的成员函数remove的实现，以便可以了解Activity组件的启动窗口的结束过程。

```

Step 10. Session.remove
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class Session extends IWindowSession.Stub
        implements IBinder.DeathRecipient {
        .....

        public void remove(IWindow window) {
            removeWindow(this, window);
        }

        .....
    }

    .....
}

```

这个函数定义在frameworks/base/services/java/com/android/server/WindowManagerService.java文件中。

Session类的成员函数remove的实现很简单，它只是调用外部类WindowManagerService的成员函数removeWindow来执行参数window所描述的一个窗口的删除操作。

Step 11. WindowManagerService.removeWindow

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

```

```

public void removeWindow(Session session, IWindow client) {
    synchronized(mWindowMap) {
        WindowState win = windowForClientLocked(session, client, false);
        if (win == null) {
            return;
        }
        removeWindowLocked(session, win);
    }
}

.....
}

```

这个函数定义在frameworks/base/services/java/com/android/server/WindowManagerService.java文件中。WindowManagerService类的成员函数removeWindow首先调用成员函数windowForClientLocked来找到与参数client所对应的一个WindowState对象，接着再调用成员函数removeWindowLocked来删除这个WindowState对象，以便可以结束掉这个WindowState对象所描述的一个启动窗口。

Step 12. WindowManagerService.removeWindowLocked

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void removeWindowLocked(Session session, WindowState win) {
        .....

        // Visibility of the removed window. Will be used later to update orientation later on.
        boolean wasVisible = false;
        // First, see if we need to run an animation. If we do, we have
        // to hold off on removing the window until the animation is done.
        // If the display is frozen, just remove immediately, since the
        // animation wouldn't be seen.
        if (win.mSurface != null && !mDisplayFrozen && mPolicy.isScreenOn()) {
            // If we are not currently running the exit animation, we
            // need to see about starting one.
            if (wasVisible=win.isWinVisibleLw()) {

                int transit = WindowManagerPolicy.TRANSIT_EXIT;
                if (win.getAttrs().type == TYPE_APPLICATION_STARTING) {
                    transit = WindowManagerPolicy.TRANSIT_PREVIEW_DONE;
                }
                // Try starting an animation.
                if (applyAnimationLocked(win, transit, false)) {
                    win.mExiting = true;
                }
            }
            if (win.mExiting || win.isAnimating()) {
                // The exit animation is running... wait for it!
                .....
                win.mExiting = true;
                win.mRemoveOnExit = true;
                mLayoutNeeded = true;
                updateFocusedWindowLocked(UPDATE_FOCUS_WILL_PLACE_SURFACES);
                performLayoutAndPlaceSurfacesLocked();
                if (win.mAppToken != null) {
                    win.mAppToken.updateReportedVisibilityLocked();
                }
                .....
                return;
            }
        }

        removeWindowInnerLocked(session, win);
        .....

        updateFocusedWindowLocked(UPDATE_FOCUS_NORMAL);
        .....
    }
}

```



```
.....  
}
```

这个函数定义在frameworks/base/services/java/com/android/server/WindowManagerService.java文件中。
WindowManagerService类的成员函数removeWindowLocked在删除参数win所描述的一个窗口之前，首先检查是否需要对该窗口设置一个退出动画。只要满足以下四个条件，那么就需要对参数win所描述的一个窗口设置退出动画：

1. 参数win所描述的一个窗口具有绘图表面，即它的成员变量mSurface的值不等于null；
2. 系统屏幕当前没有被冻结，即WindowManagerService类的成员变量mDisplayFrozen的值等于false；
3. 系统屏幕当前是点亮的，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值为true；
4. 参数win所描述的一个窗口当前是可见的，即它的成员函数isWinVisibleLw的返回值等于true。

对参数win所描述的一个窗口设置退出动画是通过调用WindowManagerService类的成员函数applyAnimationLocked来实现的。注意，如果参数win描述的是一个启动窗口，那么退出动画的类型就为WindowManagerPolicy.TRANSIT_PREVIEW_DONE，否则的话，退出动画的类型就为WindowManagerPolicy.TRANSIT_EXIT。

一旦参数win所描述的一个窗口正处于退出动画或者其它动画状态，即它的成员变量mExiting的值等于true或者成员函数isAnimating的返回值等于true，那么WindowManagerService服务就要等它的动画显示完成之后，再删除它，这是通过将它成员变量mExiting和mRemoveOnExit的值设置为true来完成的。由于这时候还需要显示参数win所描述的一个窗口的退出动画或者其它动画，因此，WindowManagerService类的成员函数removeWindowLocked在返回之前，还需要执行以下操作：

1. 调用WindowManagerService类的成员函数updateFocusedWindowLocked来重新计算系统当前需要获得焦点的窗口；
2. 调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来重新布局和刷新系统的UI；
3. 如果参数win所描述的一个与Activity组件相关的窗口，即它的成员变量mAppToken的值不等于null，那么就会调用这个成员变量所指向的一个AppWindowToken对象的成员函数updateReportedVisibilityLocked来向ActivityManagerService服务报告该Activity组件的可见性。

如果不需要对参数win所描述的一个窗口设置退出动画，那么WindowManagerService类的成员函数removeWindowLocked就会直接调用成员函数removeWindowInnerLocked来删除该窗口，并且在删除了该窗口之后，调用成员函数updateFocusedWindowLocked来重新计算系统当前需要获得焦点的窗口以及重新布局和刷新系统的UI。

接下来，我们就继续分析WindowManagerService类的成员函数removeWindowLocked的实现，以便可以了解Activity组件的启动窗口的结束过程。

```
Step 13. WindowManagerService.removeWindowLocked  
public class WindowManagerService extends IWindowManager.Stub  
    implements Watchdog.Monitor {  
    .....  
  
    private void removeWindowInnerLocked(Session session, WindowState win) {  
        win.mRemoved = true;  
  
        if (mInputMethodTarget == win) {  
            moveInputMethodWindowsIfNeededLocked(false);  
        }  
        .....  
  
        mPolicy.removeWindowLw(win);  
        win.removeLocked();  
  
        mWindowMap.remove(win.mClient.asBinder());  
        mWindows.remove(win);  
        mWindowsChanged = true;  
        .....  
  
        if (mInputMethodWindow == win) {  
            mInputMethodWindow = null;  
        } else if (win.mAttrs.type == TYPE_INPUT_METHOD_DIALOG) {  
            mInputMethodDialogs.remove(win);  
        }  
  
        final WindowToken token = win.mToken;
```

```

final AppWindowToken atoken = win.mAppToken;
token.windows.remove(win);
if (atoken != null) {
    atoken.allAppWindows.remove(win);
}
.....

if (token.windows.size() == 0) {
    if (!token.explicit) {
        mTokenMap.remove(token.token);
        mTokenList.remove(token);
    } else if (atoken != null) {
        atoken.firstWindowDrawn = false;
    }
}

if (atoken != null) {
    if (atoken.startingWindow == win) {
        atoken.startingWindow = null;
    } else if (atoken.allAppWindows.size() == 0 && atoken.startingData != null) {
        // If this is the last window and we had requested a starting
        // transition window, well there is no point now.
        atoken.startingData = null;
    } else if (atoken.allAppWindows.size() == 1 && atoken.startingView != null) {
        // If this is the last window except for a starting transition
        // window, we need to get rid of the starting transition.
        .....
        Message m = mHandler.obtainMessage(H.REMOVE_STARTING, atoken);
        mHandler.sendMessage(m);
    }
}

if (win.mAttrs.type == TYPE_WALLPAPER) {
    .....
    adjustWallpaperWindowsLocked();
} else if ((win.mAttrs.flags & FLAG_SHOW_WALLPAPER) != 0) {
    adjustWallpaperWindowsLocked();
}

if (!mInLayout) {
    assignLayersLocked();
    mLayoutNeeded = true;
    performLayoutAndPlaceSurfacesLocked();
    if (win.mAppToken != null) {
        win.mAppToken.updateReportedVisibilityLocked();
    }
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。由于参数win所描述的一个窗口马上就要被删除了，因此，WindowManagerService类的成员函数removeWindowLocked首先就将它的成员变量mRemoved的值设置为true。此外，如果参数win所描述的窗口是系统输入法的目标窗口，那么还需要调用WindowManagerService类的成员函数moveInputMethodWindowsIfNeededLocked来重新移动系统输入法窗口到其它可能需要输入法的窗口的上面去。

执行完成以上两个操作之后，WindowManagerService类的成员函数removeWindowLocked接下来就可以对参数win所描述的一个窗口进行清理了，包括：

1. 调用WindowManagerService类的成员变量mPolicy的成员函数removeWindowLw来通知窗口管理策略类PhoneWindowManager，参数win所描述的一个窗口被删除了；
2. 调用参数win所指向的一个WindowState对象的成员函数removeLocked来执行自身的清理工作；
3. 将参数win所指向的一个WindowState对象从WindowManagerService类的成员变量mWindowMap和mWindows中删除，即将参数win所描述的一个窗口从窗口堆栈中删除。

执行完成以上三个清理工作之后，窗口堆栈就发生了变化了，因此，就需要将WindowManagerService类的成员变量mWindowsChanged的值设置为true。

接下来，WindowManagerService类的成员函数removeWindowLocked还会检查前面被删除的窗口是否是一个输入法窗口或者一个输入法对话框。如果是一个输入法窗口，那么就会将WindowManagerService类的成员变量mInputMethodWindow的值设置为true；如果是一个输入法对话框，那么就会将它从WindowManagerService类的成员变量mInputMethodDialogs所描述的一个输入法对话框列表中删除。

WindowManagerService类的成员函数removeWindowLocked的任务还没有完成，它还需要继续从参数win所描述的一个窗口从它的窗口令牌的窗口列表中删除。参数win所描述的一个窗口的窗口令牌保存在它的成员变量mToken中，这个成员变量指向的是一个WindowToken对象。这个WindowToken对象有一个成员变量windows，它指向的是一个ArrayList中。这个ArrayList即为参数win所描述的一个窗口从它的窗口令牌的窗口列表，因此，将参数win所描述的一个窗口从这个窗口列表中删除即可。

如果参数win描述的一个是与Activity组件有关的窗口，那么它的成员变量mAppToken就会指向一个AppWindowToken对象。这个AppWindowToken对象的成员变量allAppWindows所指向的一个ArrayList也会保存有参数win所描述的窗口。因此，这时候也需要将参数win所描述的一个窗口从这个ArrayList中删除。

参数win所描述的一个窗口被删除了以后，与它所对应的窗口令牌的窗口数量就会减少1。如果一个窗口令牌的窗口数量减少1之后变成0，那么就需要考虑将这个窗口令牌从WindowManagerService服务的窗口令牌列表中删除了，即从WindowManagerService类的成员变量mTokenMap和mTokenList中删除，前提是这个窗口令牌不是显式地被增加到WindowManagerService服务中去的，即用来描述这个窗口令牌的一个WindowToken对象的成员变量explicit的值等于false。

另一方面，如果参数win描述的一个是与Activity组件有关的窗口，并且当它被删除之后，与该Activity组件有关的窗口的数量变为0，那么就需要将用来描述该Activity组件的一个AppWindowToken对象的成员变量firstWindowDrawn的值设置为false，以表示该Activity组件的第一个窗口还没有被显示出来，事实上也是表示目前没有窗口与该Activity组件对应。

当参数win描述的一个是与Activity组件有关的窗口的时候，WindowManagerService类的成员函数removeWindowLocked还需要检查该Activity组件是否设置有启动窗口。如果该Activity组件设置有启动窗口的话，那么就需要对它的相应成员变量进行清理。这些检查以及清理工作包括：

1. 如果参数win所描述的窗口即为一个Activity组件的窗口，即它的值等于用来描述与它的宿主Activity组件的一个AppWindowToken对象的成员变量startingWindow的值，那么就需要将AppWindowToken对象的成员变量startingWindow的值设置为null，以便可以表示它所描述的Activity组件的启动窗口已经结束了；
2. 如果删除了参数win所描述的窗口之后，它的宿主Activity组件的窗口数量为0，但是该Activity组件又正在准备显示启动窗口，即用来描述该Activity组件的一个AppWindowToken对象的成员变量startingData的值不等于null，那么就说明这个启动窗口接下来也没有必要显示了，因此，就需要将该AppWindowToken对象的成员变量startingData的值设置为null；
3. 如果删除了参数win所描述的窗口之后，它的宿主Activity组件的窗口数量为1，并且用来描述该Activity组件的一个AppWindowToken对象的成员变量startingView的值不等于null，那么就说明该Activity组件剩下的最后一个窗口即为它的启动窗口，这时候就需要请求WindowManagerService服务结束掉这个启动窗口，因为已经没有必要显示了。

当一个Activity组件剩下的窗口只有一个，并且用来描述该Activity组件的一个AppWindowToken对象的成员变量startingView的值不等于null时，我们是如何知道这个剩下的窗口就是该Activity组件的启动窗口的呢？从前面第一个部分的内容可以知道，当一个Activity组件的启动窗口被创建出来之后，它的顶层视图就会保存在用来描述该Activity组件的一个AppWindowToken对象的成员变量startingView中。因此，如果Activity组件满足上述两个条件，我们就可以判断出它所剩下的一个窗口即为它的启动窗口。注意，在这种情况下，WindowManagerService类的成员函数removeWindowLocked不是马上删除这个启动窗口的，而是通过向WindowManagerService服务所运行在的线程发送一个类型为REMOVE_STARTING的消息，等到该消息被处理时再来删除这个启动窗口。

清理了窗口win的宿主Activity组件的启动窗口相关的数据之后，WindowManagerService类的成员函数removeWindowLocked又继续检查窗口win是否是一个壁纸窗口或者一个显示壁纸的窗口。如果是的话，那么就需要调用WindowManagerService类的成员函数adjustWallpaperWindowsLocked来重新调整系统中的壁纸窗口在窗口堆栈中的位置，即将它们移动到下一个可能需要显示壁纸窗口的其它窗口的下面去。

WindowManagerService类的成员函数removeWindowLocked的最后一个任务是检查WindowManagerService服务当前是否正处于重新布局窗口的状态，即判断WindowManagerService类的成员变量mInLayout的值是否等于true。如果不等于true的话，那么就需要调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来重新布局窗口，实际上就是刷新系统的UI。

注意，WindowManagerService类的成员函数removeWindowLocked在重新布局系统中的窗口之前，还需要调用另外一个成员函数assignLayersLocked来重新计算系统中的所有窗口的Z轴位置了。此外，WindowManagerService类的成员函数removeWindowLocked在重新布局了系统中的窗口之后，如果发现前面被删除的窗口win是一个与Activity组件相关的窗口，即它的成员变量mAppToken的值不等于null，那么还会调用这个成员变量所指向的一个AppWindowToken对象的成员函数updateReportedVisibilityLocked来向ActivityManagerService服务报告该Activity组件的可见性。

这一步执行完成之后，一个的Activity组件的启动窗口结束掉了。至此，我们就分析完成Activity组件的启动窗口的启动过程和结束过程了。事实上，一个Activity组件在启动的过程中，除了可能需要显示启动窗口之外，还需要与系

统当前激活的Activity组件执行一个切换操作，然后才可以将自己的窗口显示出来。在接下来的一篇文章中，我们将继续分析Activity组件的切换过程，敬请关注！

原文链接：<https://blog.csdn.net/Luoshengyang/article/details/8577789>

第八节Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析

在Android系统中，同一时刻只有一个Activity组件是处于激活状态的，因此，当ActivityManagerService服务激活了一个新的Activity组件时，它就需要通知WindowManagerService服务将该Activity组件的窗口显示出来，这会涉及到将焦点和屏幕等资源从前一个激活的Activity组件切换到后一个激活的Activity组件的过程，本文就详细分析这个过程。

《Android系统源代码情景分析》一书正在进击的程序员网（<http://0xcc0xcd.com>）中连载，点击进入！

Activity窗口的切换操作是在新激活的Activity组件的启动过程进行的。具体来说，就是在前一个激活的Activity组件进入到Paused状态并且新激活的Activity组件进入到Resumed状态之后，将前一个激活的Activity组件的窗口设置为不可见，以及将新激活的Activity组件的窗口设置为可见。整个切换过程是需要ActivityManagerService服务和WindowManagerService服务的协作之下进行的，如图1所示。

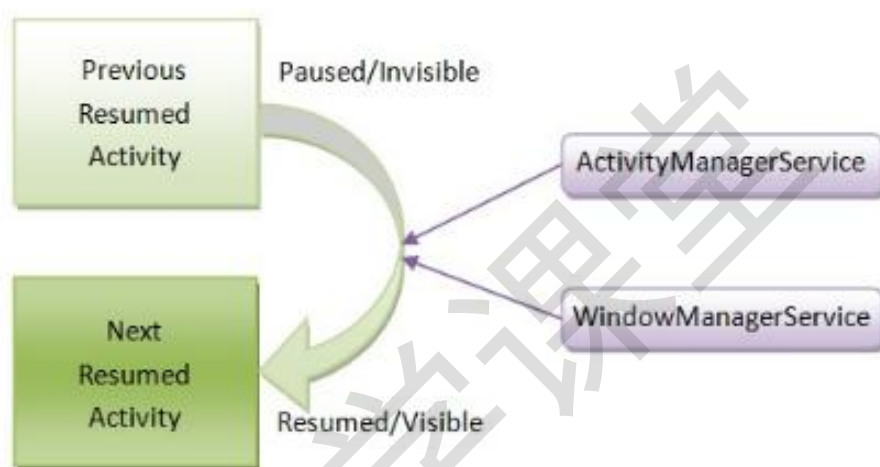


图1 Activity窗口的切换操作示意图

WindowManagerService服务在执行Activity窗口的切换操作的时候，会给参与切换操作的Activity组件的设置一个动画，以便可以向用户展现一个Activity组件切换效果，从而提高用户体验。事实上，一个Activity窗口在由不可见状态切换到可见状态的过程中，除了会被设置一个Activity组件切换动画之外，还有被设置一个窗口进入动画，此外，如果该Activity窗口是附加在另外一个窗口之上的，并且这个被附加的窗口正在显示一个动画，那么这个动画也同时会被设置给到该Activity窗口的显示过程中去。本文主要是关注Activity窗口的切换操作，在接下来的一篇文章中分析窗口的动画框架时，我们再详细分析上述三种动画是如何作用在窗口的显示过程中的。

从前面Android应用程序启动过程源代码分析一文可以知道，ActivityManagerService服务在启动一个Activity组件的过程中，会调用到ActivityStack类的成员函数startActivityLocked。ActivityStack类的成员函数startActivityLocked首先会给正在启动的Activity组件准备一个切换操作，接着再调用其它的成员函数来通知前一个激活的Activity组件进入到Paused状态。等到前一个激活的Activity组件进入到Paused状态之后，ActivityManagerService服务就会检查用来运行正在启动的Activity组件的进程启动起来了没有。如果这个进程还没有启动，那么ActivityManagerService服务就会将该进程启动起来，然后再调用ActivityStack类的成员函数realStartActivityLocked来将正在启动的Activity组件加载起来，并且将它的状态设置为Resumed，最后通知WindowManagerService服务执行前面所准备的切换操作。

接下来，我们就从ActivityStack类的成员函数startActivityLocked开始分析Activity窗口的切换过程，如图2所示。

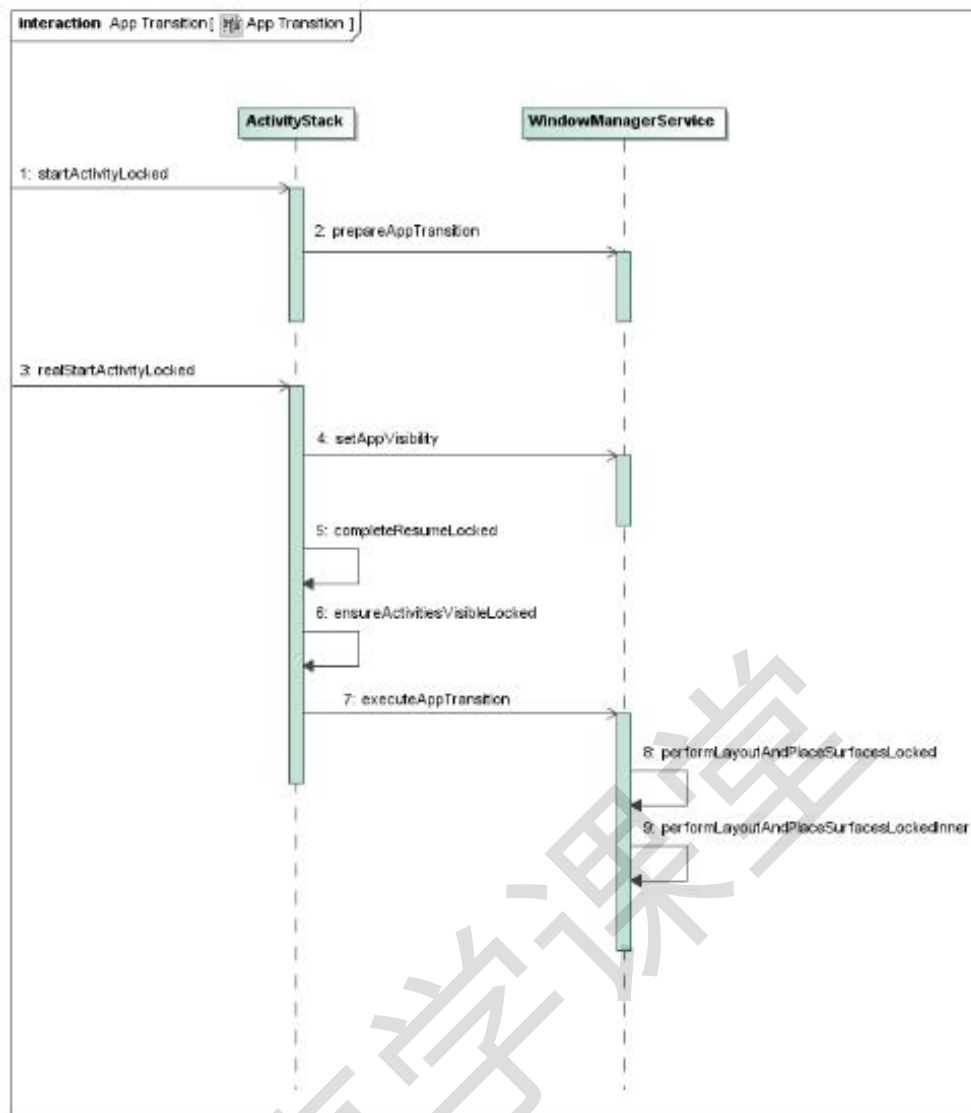


图2 Activity窗口的切换过程

这个过程可以分为9个步骤，接下来我们就详细分析每一个步骤。

Step 1. ActivityStack.startActivityLocked

```

public class ActivityStack {
    .....

    private final void startActivityLocked(ActivityRecord r, boolean newTask,
        boolean doResume) {
        final int NH = mHistory.size();

        int addPos = -1;

        if (!newTask) {
            // If starting in an existing task, find where that is...
            .....
        }

        // Place a new activity at top of stack, so it is next to interact
        // with the user.
        if (addPos < 0) {
            addPos = NH;
        }
        .....

        // Slot the activity into the history stack and proceed
        mHistory.add(addPos, r);
    }
}
  
```

```

.....

if (NH > 0) {
    .....

    if ((r.intent.getFlags() & Intent.FLAG_ACTIVITY_NO_ANIMATION) != 0) {
        mService.mWindowManager.prepareAppTransition(WindowManagerPolicy.TRANSIT_NONE);
        .....
    } else if ((r.intent.getFlags() & Intent.FLAG_ACTIVITY_CLEAR_WHEN_TASK_RESET) != 0) {
        mService.mWindowManager.prepareAppTransition(
            WindowManagerPolicy.TRANSIT_TASK_OPEN);
        .....
    } else {
        mService.mWindowManager.prepareAppTransition(newTask
            ? WindowManagerPolicy.TRANSIT_TASK_OPEN
            : WindowManagerPolicy.TRANSIT_ACTIVITY_OPEN);
        .....
    }

    .....
}

.....

if (doResume) {
    resumeTopActivityLocked(null);
}
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/am/ActivityStack.java中。

参数r描述的是正在启动的Activity组件，ActivityStack类的成员函数startActivityLocked首先找到它在Activity组件堆栈中的位置addPos，即找到它在ActivityStack类的成员变量mHistory所描述的一个ArrayList中的位置，然后将正在启动的Activity组件保存在该位置中。

变量NH描述的是将参数r描述的是正在启动的Activity组件保存在Activity组件堆栈前系统已经启动了的Activity组件的个数。只有在这个变量的值大于0的情况下，系统才需要执行一个Activity组件切换操作。也就是说，如果参数r描述的是正在启动的Activity组件是系统中第一个启动的Activity组件，那么就不需要执行一个Activity组件切换操作了。

注意，即使参数r描述的是正在启动的Activity组件不是系统中第一个启动的Activity组件，那么系统也可能不需要执行一个Activity组件切换操作，因为用来启动参数r所描述的一个Activity组件的一个Intent对象的成员函数getFlags返回的一个标志值的Intent.FLAGACTIVITYNOANIMATION位可能会不等于0，这意味着正在启动的Activity组件不需要显示切换动画。在这种情况下，ActivityManagerService服务就会通知WindowManagerService服务不需要准备一个Activity组件切换操作，这是通过以WindowManagerPolicy.TRANSIT_NONE为参数来调用ActivityStack类的成员变量mService所指向的一个ActivityManagerService对象的成员变量mWindowManager所描述的一个WindowManagerService对象的成员函数prepareAppTransition来实现的。

另一方面，如果参数r描述的是正在启动的Activity组件不是系统中第一个启动的Activity组件，并且系统需要执行一个Activity组件切换操作，即需要WindowManagerService服务在显示正在启动的Activity组件的窗口时应用一个切换动画，那么这个动画的类型也是有讲究的。具体来说，如果参数r描述的是Activity组件是需要在一个新的任务中启动的，即参数newTask的值等于true，那么切换动画的类型就指定为WindowManagerPolicy.TRANSIT_TASK_OPEN，否则的话，切换动画的类型就指定为WindowManagerPolicy.TRANSIT_ACTIVITY_OPEN。

此外，如果用来启动参数r所描述的一个Activity组件的一个Intent对象的成员函数getFlags返回的一个标志值的Intent.FLAGACTIVITYCLEARWHENTASKRESET位不等于0，那么也会将切换动画的类型设置为WindowManagerPolicy.TRANSIT_TASK_OPEN。Intent.FLAGACTIVITYCLEARWHENTASKRESET这个标志位等于1意味着当参数r所描述的一个Activity组件所在的任务下次作为前台任务来运行时，参数r所描述的Activity组件以及它上面的并且属于同一个任务的其它Activity组件都会被结束掉，以使得位于参数r所描述的Activity组件的前面一个Activity组件可以显示出来。例如，如果我们正在使用一个Email Activity组件来查看Email，这时候又需要启动另外一个Pictrue Activity来查看该Email附件的一张图片。这时候Email Activity和Pictrue Activity就是在同一个任务中的。突然间，我们因为其它原历，需要按下Home键回到Launcher中去完成其它任务。完成其它任务之后，再点击Launcher上面的Email Activity图标来想重新浏览之前正在查看的Email，但是又不想看到的是该Email附件的那张图片。在这种情况下，之前在启动Pictrue Activity来查看Email的附件图片时，就可以将用来启动Pictrue Activity的一个Intent对象的标志值的Intent.FLAGACTIVITYCLEARWHENTASK_RESET位设置为1。

无论如何，最终指定的切换动画的类型都是通过调用ActivityStack类的成员变量mService所指向的一个ActivityManagerService对象的成员变量mWindowManager所描述的一个WindowManagerService对象的成员函数prepareAppTransition来通知WindowManagerService服务的。

ActivityStack类的成员函数startActivityLocked通知WindowManagerService服务准备好一个Activity组件切换操作之后，如果参数doResume的值等于true，那么它就会继续调用另外一个成员函数resumeTopActivityLocked来继续执行启动参数r所描述的一个Activity组件的操作。

接下来，我们就首先分析WindowManagerService类的成员函数prepareAppTransition的实现，以便可以了解WindowManagerService服务是如何准备一个Activity组件切换操作的，然后再回过头来分析ActivityStack类的成员函数resumeTopActivityLocked是如何继续执行启动参数r所描述的一个Activity组件的操作的。

```
Step 2. WindowManagerService.prepareAppTransition
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    // State management of app transitions. When we are preparing for a
    // transition, mNextAppTransition will be the kind of transition to
    // perform or TRANSIT_NONE if we are not waiting. If we are waiting,
    // mOpeningApps and mClosingApps are the lists of tokens that will be
    // made visible or hidden at the next transition.
    int mNextAppTransition = WindowManagerPolicy.TRANSIT_UNSET;
    .....
    boolean mAppTransitionReady = false;
    .....
    boolean mAppTransitionTimeout = false;
    boolean mStartingIconInTransition = false;
    boolean mSkipAppTransitionAnimation = false;

    public void prepareAppTransition(int transit) {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "prepareAppTransition()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        synchronized(mWindowMap) {
            .....
            if (!mDisplayFrozen && mPolicy.isScreenOn()) {
                if (mNextAppTransition == WindowManagerPolicy.TRANSIT_UNSET
                    || mNextAppTransition == WindowManagerPolicy.TRANSIT_NONE) {
                    mNextAppTransition = transit;
                } else if (transit == WindowManagerPolicy.TRANSIT_TASK_OPEN
                    && mNextAppTransition == WindowManagerPolicy.TRANSIT_TASK_CLOSE) {
                    // Opening a new task always supersedes a close for the anim.
                    mNextAppTransition = transit;
                } else if (transit == WindowManagerPolicy.TRANSIT_ACTIVITY_OPEN
                    && mNextAppTransition == WindowManagerPolicy.TRANSIT_ACTIVITY_CLOSE) {
                    // Opening a new activity always supersedes a close for the anim.
                    mNextAppTransition = transit;
                }
                mAppTransitionReady = false;
                mAppTransitionTimeout = false;
                mStartingIconInTransition = false;
                mSkipAppTransitionAnimation = false;
                mH.removeMessages(H.APP_TRANSITION_TIMEOUT);
                mH.sendMessageDelayed(mH.obtainMessage(H.APP_TRANSITION_TIMEOUT),
                    5000);
            }
        }
    }
    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。调用WindowManagerService类的成员函数prepareAppTransition来通知WindowManagerService服务准备一个Activity组件切换操作是需要具有android.Manifest.permission.MANAGE_APP_TOKENS，否则的话，WindowManagerService类的成员函数prepareAppTransition就会抛出一个类型为SecurityException的异常。

WindowManagerService类的成员变量mNextAppTransition描述的就是WindowManagerService服务接下来要执行一个Activity组件切换操作的类型，也就是要执行的一个Activity组件切换动画的类型。WindowManagerService类的成员函数prepareAppTransition按照以下规则来设置WindowManagerService服务接下来要执行的Activity组件切换动画的

类型:

1. 当 WindowManagerService类的成员变量mNextAppTransition的值等于 WindowManagerPolicy.TRANSITUNSET或者WindowManagerPolicy.TRANSITNONE的时候, 就说明WindowManagerService服务接下来没有Activity组件切换动画等待执行的, 这时候参数transit所描述的Activity组件切换动画就可以作为WindowManagerService服务接下来要执行的Activity组件切换动画。
2. 当WindowManagerService类的成员变量mNextAppTransition的值等于 WindowManagerPolicy.TRANSITTASKCLOSE, 那么就说明WindowManagerService服务接下来要执行一个关闭Activity组件任务的切换动画等待执行的。在这种情况下, 如果参数transit所描述的是一个打开Activity组件任务的切换动画, 即它的值等于 WindowManagerPolicy.TRANSITTASKOPEN, 那么就需要将WindowManagerService服务接下来要执行的Activity组件切换动画为打开Activity组件任务类型的。这是因为打开Activity组件任务的切换动画的优先级高于关闭Activity组件任务的切换动画。
3. 当WindowManagerService类的成员变量mNextAppTransition的值等于 WindowManagerPolicy.TRANSITACTIVITYCLOSE, 那么就说明WindowManagerService服务接下来要执行一个关闭Activity组件的切换动画等待执行的。在这种情况下, 如果参数transit所描述的是一个打开Activity组件的切换动画, 即它的值等于 WindowManagerPolicy.TRANSITACTIVITYOPEN, 那么就需要将WindowManagerService服务接下来要执行的Activity组件切换动画为打开Activity组件类型的。这是因为打开Activity组件的切换动画的优先级高于关闭Activity组件的切换动画。

设置好WindowManagerService服务接下来要执行的Activity组件切换动画的类型之后, WindowManagerService类的成员函数prepareAppTransition还会将其余四个成员变量mAppTransitionReady、mAppTransitionTimeout、mStartingIconInTransition和mSkipAppTransitionAnimation的值设置为false, 其中:

1. mAppTransitionReady表示WindowManagerService服务可以开始执行一个Activity组件的切换动画了没有?
2. mAppTransitionTimeout表示WindowManagerService服务正在执行的Activity组件切换动画是否已经超时
3. mStartingIconInTransition表示WindowManagerService服务开始显示正在启动的Activity组件的启动窗口了没有?
4. mSkipAppTransitionAnimation表示WindowManagerService服务是否需要不执行Activity组件的切换动画?

最后, WindowManagerService类的成员函数prepareAppTransition还会调用成员变量mH所描述的一个H对象的成员函数sendMessageDelayed来向WindowManagerService服务所运行在的线程发送一个类型为APP_TRANSITION_TIMEOUT的消息。这个消息将在5秒后被执行, 是用来强制前面所设置的Activity组件切换动画要在5秒之内执行完成的, 否则的话, WindowManagerService服务就会认为该切换动画执行超时了。

这一步执行完成之后, WindowManagerService服务接下来要执行的Activity组件切换操作或者说切换动画就准备完成了。注意, 这时候只是准备好Activity组件切换动画, 但是这个切换动画还不能执行, 要等到前一个激活的Activity组件进入到Paused状态并且接下来正在启动的Activity组件进入到Resumed状态之后才能执行。

回到前面的Step 1中, 即ActivityStack类的成员函数startActivityLocked, 接下来它就会调用另外一个成员函数resumeTopActivityLocked来继续启动指定的Activity组件。从前面Android应用程序启动过程源代码分析一文可以知道, ActivityStack类的成员函数resumeTopActivityLocked以及接下来要调用的其它成员函数就是执行以下两个操作:

1. 通知当前处于激活状态的Activity组件所运行在的应用程序进程, 它所运行的一个Activity组件要由Resumed状态进入到Paused状态了。
2. 检查用来运行当前正在启动的Activity组件的应用程序进程是否已经启动起来了。如果已经启动起来, 那么就会直接通知该应用程序进程将正在启动的Activity组件加载起来, 否则的话, 就会先将该应用程序进程启动起来, 然后再通知它将正在启动的Activity组件加载起来。

在第2步中, 通知相应的应用程序进程将正在启动的Activity组件加载起来是通过调用ActivityStack类的成员函数realStartActivityLocked来实现的, 接下来我们就继续分析这个成员函数的实现, 以便可以继续了解Activity组件的切换操作的执行过程。

```
Step 3. ActivityStack.realStartActivityLocked
public class ActivityStack {
    .....

    final boolean realStartActivityLocked(ActivityRecord r,
        ProcessRecord app, boolean andResume, boolean checkConfig)
```



```

        throws RemoteException {
        .....

        mService.mWindowManager.setAppVisibility(r, true);
        .....

        try {
            .....

            app.thread.scheduleLaunchActivity(new Intent(r.intent), r,
                System.identityHashCode(r),
                r.info, r.icicle, results, newIntents, !andResume,
                mService.isNextTransitionForward());
            .....

        } catch (RemoteException e) {
            .....
        }

        .....

        if (andResume) {
            // As part of the process of launching, ActivityThread also performs
            // a resume.
            r.state = ActivityState.RESUMED;
            .....
            mResumedActivity = r;
            .....
            completeResumeLocked(r);
            .....
        }

        .....

        return true;
    }

    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/am/ActivityStack.java中。
ActivityStack类的成员函数realStartActivityLocked主要执行以下三个操作：

1. 通知WindowManagerService服务将参数r所描述的Activity组件的可见性设置为true，这是通过调用ActivityStack类的成员变量mService所指向的一个ActivityManagerService对象的成员变量mWindowManager所描述的一个WindowManagerService对象的成员函数setAppVisibility来实现的。
2. 通知参数r所描述的Activity组件所运行在的应用程序进程将它加载起来，这是通过调用参数app所指向的一个ProcessRecord对象的成员变量thread所描述的一个类型为ApplictionThread的Binder代理对象的成员函数scheduleLaunchActivity来实现的。
3. 当参数andResume的值等于true的时候，就表示在执行第2步时，参数r所描述的Activity组件所运行在的应用程序进程已经将它状态设置为Resumed了，即已经调用过它的成员函数onResume了。在这种情况下，ActivityManagerService服务也需要将该Activity组件的状态设置为Resumed了，即将r所指向的一个ActivityRecord对象的成员变量state的值设置为ActivityState.RESUMED，并且将ActivityStack类的成员变量mResumedActivity的值设置为r，以便表示当前激活的Activity组件为参数r所描述的Activity组件。最后，ActivityManagerService服务还需要调用ActivityStack类的成员函数completeResumeLocked来通知WindowManagerService服务执行在前面Step 2所准备好的Activity组件切换操作。

接下来，我们首先分析WindowManagerService类的成员函数setAppVisibility的实现，以便可以了解WindowManagerService服务是如何设置一个Activity组件的可见性的，接着再分析ActivityStack类的成员函数completeResumeLocked的实现，以便可以了解ActivityManagerService服务是如何通知WindowManagerService服务执行前面所准备好的一个Activity组件切换操作的。

Step 4. WindowManagerService.setAppVisibility
public class WindowManagerService extends IWindowManager.Stub

```

implements Watchdog.Monitor {
.....

public void setAppVisibility(IBinder token, boolean visible) {
    if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
        "setAppVisibility()")) {
        throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
    }

    AppWindowToken wtoken;

    synchronized(mWindowMap) {
        wtoken = findAppWindowToken(token);
        .....

        // If we are preparing an app transition, then delay changing
        // the visibility of this token until we execute that transition.
        if (!mDisplayFrozen && mPolicy.isScreenOn()
            && mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
            // Already in requested state, don't do anything more.
            if (wtoken.hiddenRequested != visible) {
                return;
            }
            wtoken.hiddenRequested = !visible;
            .....

            wtoken.setDummyAnimation();
            mOpeningApps.remove(wtoken);
            mClosingApps.remove(wtoken);
            wtoken.waitingToShow = wtoken.waitingToHide = false;
            wtoken.inPendingTransaction = true;
            if (visible) {
                mOpeningApps.add(wtoken);
                wtoken.startingDisplayed = false;
                wtoken.startingMoved = false;

                // If the token is currently hidden (should be the
                // common case), then we need to set up to wait for
                // its windows to be ready.
                if (wtoken.hidden) {
                    wtoken.allDrawn = false;
                    wtoken.waitingToShow = true;

                    if (wtoken.clientHidden) {
                        // In the case where we are making an app visible
                        // but holding off for a transition, we still need
                        // to tell the client to make its windows visible so
                        // they get drawn. Otherwise, we will wait on
                        // performing the transition until all windows have
                        // been drawn, they never will be, and we are sad.
                        wtoken.clientHidden = false;
                        wtoken.sendAppVisibilityToClients();
                    }
                }
            } else {
                mClosingApps.add(wtoken);

                // If the token is currently visible (should be the
                // common case), then set up to wait for it to be hidden.
                if (!wtoken.hidden) {
                    wtoken.waitingToHide = true;
                }
            }
            return;
        }
    }

    .....
    setTokenVisibilityLocked(wtoken, null, visible, WindowManagerPolicy.TRANSIT_UNSET, true);
    wtoken.updateReportedVisibilityLocked();
    .....
}

```

```

    }
}

.....

}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。调用WindowManagerService类的成员函数setAppVisibility来设置Activity组件的可见性需要具有android.Manifest.permission.MANAGE_APP_TOKENS权限，否则的话，WindowManagerService类的成员函数setAppVisibility就会抛出一个类型为SecurityException的异常。

从前面Android窗口管理服务WindowManagerService对窗口的组织方式分析一文可以知道，每一个Activity组件在WindowManagerService服务内部都对应有一个AppWindowToken对象，用来描述该Activity组件的窗口在WindowManagerService服务中的状态。因此，WindowManagerService类的成员函数setAppVisibility就先通过调用成员函数findAppWindowToken来找到与参数token所描述的一个Activity组件所对应的一个AppWindowToken对象wtoken，以便接下来可以设置它的状态。

注意，WindowManagerService类的成员函数setAppVisibility在修改参数token所描述的一个Activity组件的可见性的时候，需要考虑WindowManagerService服务接下来是否需要执行一个Activity组件操作，即WindowManagerService类的成员变量mNextAppTransition的值是否等于WindowManagerPolicy.TRANSIT_UNSET。如果等于的话，那么参数token所描述的Activity组件就是正在等待执行切换操作的Activity组件，这时候修改的可见性就会复杂一些，否则的话，只要简单地执行以下两个操作即可：

1. 调用WindowManagerService类的成员函数setTokenVisibilityLocked来将参数token所描述的Activity组件的可见性设置为参数visible所描述的值；
2. 调用AppWindowToken对象wtoken的成员函数updateReportedVisibilityLocked来向ActivityManagerService服务报告参数token所描述的Activity组件的可见性。

我们假设WindowManagerService服务接下来需要执行一个Activity组件操作，即WindowManagerService类的成员变量mNextAppTransition的值等于WindowManagerPolicy.TRANSIT_UNSET，这时候还需要满足三个额外的条件，即：

1. 屏幕当前不是处于冻结状态，即WindowManagerService类的成员变量mDisplayFrozen的值等于false；
2. 屏幕当前是点亮的，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值等于true；
3. 参数token所描述的一个Activity组件的可见性已经不等于所要设置的可见性，即前面所找到的AppWindowToken对象wtoken的成员变量hiddenRequested的值不等于参数visible的值。

那么WindowManagerService类的成员函数setAppVisibility接下来才会开始修改参数token所描述的一个Activity组件的可见性，即：

1. 将AppWindowToken对象wtoken的成员变量hiddenRequested的值设置为参数visible的相反值。也就是说，如果参数token所描述的Activity组件是可见的，那么就onAppWindowToken对象wtoken的成员变量hiddenRequested的值设置为false，否则的话，就设置为true。
2. 调用AppWindowToken对象wtoken的成员函数setDummyAnimation来给参数token所描述的Activity组件设置一个哑动画。注意，要等到执行参数token所描述的Activity组件的切换操作时，WindowManagerService服务才会给该Activity组件设置一个合适的切换动画。
3. 分别将AppWindowToken对象wtoken从WindowManagerService类的成员变量mOpeningApps和mClosingApps所描述的两个ArrayList中删除。注意，WindowManagerService类的成员变量mOpeningApps和mClosingApps保存的分别是系统当前正在打开和关闭的Activity组件，后面会根据参数visible的值来决定参数token所描述的Activity组件是正在打开的还是正在关闭的，以便可以把它放在WindowManagerService类的成员变量mOpeningApps或者mClosingApps中。
4. 将AppWindowToken对象wtoken的成员变量waitingToShow和waitingToHide的值都初始化为false，表示参数token所描述的Activity组件既不是正在等待显示的，也不是正在等待隐藏的，这两个成员变量的值同样是要根据参数visible的值来设置的。
5. 将AppWindowToken对象wtoken的成员变量inPendingTransaction的值设置为true，表示参数token所描述的Activity组件正在等待切换。

接下来的操作取决于参数visible的值是true还是false。

假设参数visible的值等于true，那么就表示要将参数token所描述的Activity组件设置可见，这时候WindowManagerService类的成员函数setAppVisibility会继续执行以下操作：

1. 将AppWindowToken对象wtoken添加WindowManagerService类的成员变量mOpeningApps所描述的一个ArrayList中，表示参数token所描述的Activity组件是正在打开的。
2. 将AppWindowToken对象wtoken的成员变量startingDisplayed和startingMoved的值都设置为false，表示参数token所描述的Activity组件的启动窗口还没有显示出来，以及也没有被转移给其它的Activity组件。
3. 如果AppWindowToken对象wtoken的成员变量hidden的值等于true，那么就意味着参数token所描述的Activity组件当前是不可见的。由于在这种情况下，参数token所描述的Activity组件正在等待打开，因此，该Activity组件的窗口一定是还没有绘制出来，并且正在等待绘制以及显示出来，这时候就需要将AppWindowToken对象wtoken的成员变量allDrawn和waitingToShow的值分别设置为false和true。
4. 如果AppWindowToken对象wtoken的成员变量hidden的值等于true，并且另外一个成员变量clientHidden的值也等于true，那么就说明在应用程序进程这一侧看来，参数token所描述的Activity组件是不可见的，这时候就需要让该应用程序进程认为参数token所描述的Activity组件是可见的，以便该应用程序进程可以将参数token所描述的Activity组件的窗口绘制出来，这样WindowManagerService服务接下来才可以将该Activity组件的窗口显示出来。通知应用程序进程将参数token所描述的Activity组件设置为true是通过调用AppWindowToken对象wtoken的成员函数sendAppVisibilityToClients来实现的，同时在通知之前，也会将AppWindowToken对象wtoken的成员变量clientHidden设置为false。

假设参数visible的值等于false，那么就表示要将参数token所描述的Activity组件设置不可见，这时候WindowManagerService类的成员函数setAppVisibility会继续执行以下操作：

1. 将AppWindowToken对象wtoken添加WindowManagerService类的成员变量mClosingApps所描述的一个ArrayList中，表示参数token所描述的Activity组件是正在关闭的。
2. 如果AppWindowToken对象wtoken的成员变量hidden的值等于false，那么就意味着参数token所描述的Activity组件当前是可见的。由于在这种情况下，参数token所描述的Activity组件正在等待关闭，因此，该Activity组件的窗口接下来的状态应该等待隐藏不见的，这时候就需要将AppWindowToken对象wtoken的成员变量waitingToHide的值设置为true。

这一步执行完成之后，参数token所描述的Activity组件的可见性就设置好了，回到前面的Step 3中，即ActivityStack类的成员函数realStartActivityLocked中，接下来就会通知参数token所描述的Activity组件所运行在的应用程序进程将它加载起来，并且最后调用ActivityStack类的成员函数completeResumeLocked来通知WindowManagerService服务执行在前面Step 2所准备好的Activity组件切换操作。

接下来，我们就继续分析ActivityStack类的成员函数completeResumeLocked的实现。

```

Step 5. ActivityStack.completeResumeLocked
public class ActivityStack {
    .....

    private final void completeResumeLocked(ActivityRecord next) {
        .....

        // schedule an idle timeout in case the app doesn't do it for us.
        Message msg = mHandler.obtainMessage(IDLE_TIMEOUT_MSG);
        msg.obj = next;
        mHandler.sendMessageDelayed(msg, IDLE_TIMEOUT);
        .....

        if (mMainStack) {
            mService.setFocusedActivityLocked(next);
        }
        .....

        ensureActivitiesVisibleLocked(null, 0);
        mService.mWindowManager.executeAppTransition();

        .....
    }

    .....
}

```

}

这个函数定义在文件frameworks/base/services/java/com/android/server/am/ActivityStack.java中。
ActivityStack类的成员函数completeResumeLocked主要是执行以下三个操作：

1. 向ActivityManagerService服务所运行在线程发送一个类型为IDLE_TIMEOUTMSG的消息，这个消息将在IDLE_TIMEOUT毫秒后处理。这个类型为IDLE_TIMEOUTMSG实际是用来监控WindowManagerService服务能否在IDLE_TIMEOUT毫秒之内，完成参数next所描述的Activity组件的切换操作，并且将它的窗口显示出来。如果能够到的话，WindowManagerService服务就会通知ActivityManagerService服务，然后ActivityManagerService服务就会执行一些清理工作，例如，将那些已经处于Stopped状态的Activity组件清理掉。如果不能做到的话，那么ActivityStack类的成员函数completeResumeLocked也需要保证在IDLE_TIMEOUT毫秒之后，ActivityManagerService服务能够执行上述的清理工作。
2. 如果当前正在处理的ActivityStack对象描述的是系统当前所使用的Activity组件堆栈，即ActivityStack类的成员变量mMainStack的值等于true，那么就会调用成员变量mService所指向的一个ActivityManagerService对象的成员函数setFocusedActivityLocked来将参数next所描述的Activity组件设置为系统当前获得焦点的Activity组件。
3. 调用ActivityStack类的成员函数ensureActivitiesVisibleLocked来从上下到检查哪些Activity组件是需要设置为可见的，哪些Activity组件是需要设置为不可见的。
4. 调用成员变量mService所指向的一个ActivityManagerService对象的成员变量mWindowManager所描述的一个WindowManagerService对象的成员函数executeAppTransition来通知WindowManagerService服务执行在前面Step 2所准备好的Activity组件切换操作。

我们主要关注第3点和第4点的操作，因此，接下来我们先分析ActivityStack类的成员函数ensureActivitiesVisibleLocked的实现，然后再分析WindowManagerService类的成员函数executeAppTransition的实现。

```
Step 6. ActivityStack.ensureActivitiesVisibleLocked
public class ActivityStack {
    .....

    final void ensureActivitiesVisibleLocked(ActivityRecord starting,
        int configChanges) {
        ActivityRecord r = topRunningActivityLocked(null);
        if (r != null) {
            ensureActivitiesVisibleLocked(r, starting, null, configChanges);
        }
    }

    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/am/ActivityStack.java。ActivityStack类有两个重载版本的成员函数ensureActivitiesVisibleLocked，其中，两个参数版本的成员函数是通过调用四个版本的成员函数来实现的，因此，接下来我们主要分析ActivityStack类四个参数版本的成员函数ensureActivitiesVisibleLocked的实现。

ActivityStack类四个参数版本的成员函数ensureActivitiesVisibleLocked主要就是从上下到检查哪些Activity组件是需要设置为可见的，哪些Activity组件是需要设置为不可见的，我们分段来阅读：

```
public class ActivityStack {
    .....

    final void ensureActivitiesVisibleLocked(ActivityRecord top,
        ActivityRecord starting, String onlyThisProcess, int configChanges) {
        .....

        // If the top activity is not fullscreen, then we need to
        // make sure any activities under it are now visible.
        final int count = mHistory.size();
        int i = count-1;
        while (mHistory.get(i) != top) {
            i--;
        }
    }
}
```

参数top描述的是位于Activity组件堆栈顶端的Activity组件，而参数starting描述的是ActivityManagerService服务当前正在启动的Activity组件。另外，参数onlyThisProcess描述的是是否只对运行在进程名称等于它的Activity组件进行处理，如果它的值等于null，那么就表示对所有的Activity组件都进行处理。这段代码首先找到参数top所描述的一个Activity组件在Activity组件堆栈的位置i，以便接下来可以从这个位置开始向下检查哪些Activity组件是需要设置为可见的，哪些Activity组件是需要设置为不可见的。

我们继续往下阅读代码：

```
ActivityRecord r;
boolean behindFullscreen = false;
for (; i>=0; i--) {
    r = (ActivityRecord)mHistory.get(i);
    .....
    if (r.finishing) {
        continue;
    }

    .....

    if (r.app == null || r.app.thread == null) {
        if (onlyThisProcess == null
            || onlyThisProcess.equals(r.processName)) {
            // This activity needs to be visible, but isn't even
            // running... get it started, but don't resume it
            // at this point.
            .....
            if (!r.visible) {
                .....
                mService.mWindowManager.setAppVisibility(r, true);
            }
            if (r != starting) {
                startSpecificActivityLocked(r, false, false);
            }
        }

        } else if (r.visible) {
            // If this activity is already visible, then there is nothing
            // else to do here.
            .....

        } else if (onlyThisProcess == null) {
            // This activity is not currently visible, but is running.
            // Tell it to become visible.
            r.visible = true;
            if (r.state != ActivityState.RESUMED && r != starting) {
                // If this activity is paused, tell it
                // to now show its window.
                .....
                try {
                    mService.mWindowManager.setAppVisibility(r, true);
                    r.app.thread.scheduleWindowVisibility(r, true);
                    .....
                } catch (Exception e) {
                    .....
                }
            }
        }
    }

    .....

    if (r.fullscreen) {
        // At this point, nothing else needs to be shown
        .....
        behindFullscreen = true;
        i--;
        break;
    }
}
```

这段码的功能是从上到下找到第一个全屏显示的Activity组件，并且将该Activity组件以及位于该Activity组件上面的其它Activity组件的可见性设置为true。对这些Activity组件的可见性设置逻辑如下所示： 1. 如果一个Activity组件处于正在结束的状态，即用来描述该Activity组件的一个ActivityRecord对象r的成员变量finishing的值等于

true, 那么跳过对该Activity组件的处理。 2. 如果一个Activity组件所运行在的进程还没有启动起来, 即用来描述该Activity组件的一个ActivityRecord对象r的成员变量app的值等于null, 或者该成员变量app的值不等于null, 但是它所指的一个ProcessRecord对象的成员变量thread的值等于null, 并且该Activity组件是需要处理的, 即参数onlyThisProcess的值等于null, 或者它的值不等于null, 但是等于该Activity组件所运行在的进程的名称, 那么就会做两个检查。第一个检查是看该Activity组件是否已经处于可见状态, 即ActivityRecord对象r的成员变量visible的值是否等于true。如果不等于的话, 那么就会通知WindowManagerService服务将该Activity组件的可见性设置为true。第二个检查是看该Activity组件是否就是参数starting所描述的Activity组件。如果不是的话, 那么就需要调用ActivityStack类的成员函数startSpecificActivityLocked来将该Activity组件所运行的进程启动起来。 3. 如果一个Activity组件已经是可见的, 即用来描述该Activity组件的一个ActivityRecord对象r的成员变量visible的值等于true, 那么就什么也不用做。 4. 如果一个Activity组件是不可见的, 即用来描述该Activity组件的一个ActivityRecord对象r的成员变量visible的值等于true, 并且该Activity组件是需要处理的, 即参数onlyThisProcess的值等于null, 那么就需要做两件事情。第一件事情就将ActivityRecord对象r的成员变量visible的值设置为true, 以便表示该Activity组件是可见的。第二件事情是检查该Activity组件是否是不处于Resumed状态, 并且它不是参数starting所描述的Activity组件。如果检查通过的话, 那么就会先通知WindowManagerService服务将该Activity组件的可见性设置为true, 然后再向该Activity组件所运行在的进程发送一个通知, 该Activity组件可见变为可见了。注意, 参数starting所描述的Activity组件在前面的Step 3中已经被设置为可见的了, 所以这里不需要重复将它设置为可见。

这段代码是如何知道一个Activity组件是不是全屏显示的呢? 这是通过检查一个对应的ActivityRecord对象的成员变量fullscreen来判断的。如果这个成员变量的值等于true, 那么就说明对应的Activity组件是全屏显示的。

我们继续往下阅读最后一段代码:

```
// Now for any activities that aren't visible to the user, make
// sure they no longer are keeping the screen frozen.
while (i >= 0) {
    r = (ActivityRecord)mHistory.get(i);
    .....
    if (!r.finishing) {
        if (behindFullscreen) {
            if (r.visible) {
                .....
                r.visible = false;
                try {
                    mService.mWindowManager.setAppVisibility(r, false);
                    if ((r.state == ActivityState.STOPPING
                        || r.state == ActivityState.STOPPED)
                        && r.app != null && r.app.thread != null) {
                        .....
                        r.app.thread.scheduleWindowVisibility(r, false);
                    }
                } catch (Exception e) {
                    .....
                }
            }
            .....
        } else if (r.fullscreen) {
            .....
            behindFullscreen = true;
        }
    }
    i--;
}
}

.....
}
```

这段代码是将在前面找到的第一个全屏显示的Activity组件的下面的所有Activity组件的可见性设置为false。注意, 只有那些当前不是处于正在结束状态的、并且是处于可见状态的Activity组件才需要处理, 即那些对应的ActivityRecord对象的成员变量finishing和visible的值分别等于false和true的值Activity组件才进行处理。处理的逻辑如下所示:

1. 将对应的ActivityRecord对象的成员变量visible的值设置为false。
2. 通知WindowManagerService服务将该Activity组件的可见性设置为false。
3. 向相应的进程发送一个通知, 该Activity组件可见变为不可见了。

这一步执行完成之后, 回到前面的Step 5中, 即ActivityStack类的成员函数completeResumeLocked中, 接下来它就会继续调用WindowManagerService类的成员函数executeAppTransition来执行在前面Step 2所准备好的Activity组件切换操作。

```

Step 7. WindowManagerService.executeAppTransition
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public void executeAppTransition() {
        if (!checkCallingPermission(android.Manifest.permission.MANAGE_APP_TOKENS,
            "executeAppTransition()")) {
            throw new SecurityException("Requires MANAGE_APP_TOKENS permission");
        }

        synchronized(mWindowMap) {
            .....
            if (mNextAppTransition != WindowManagerPolicy.TRANSIT_UNSET) {
                mAppTransitionReady = true;
                final long origId = Binder.clearCallingIdentity();
                performLayoutAndPlaceSurfacesLocked();
                Binder.restoreCallingIdentity(origId);
            }
        }
    }
    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。调用WindowManagerService类的成员函数executeAppTransition来执行之前准备好的Activity组件切换操作需要具有android.Manifest.permission.MANAGE_APP_TOKENS权限，否则的话，WindowManagerService类的成员函数setAppVisibility就会抛出一个类型为SecurityException的异常。

注意，只有当之前已经准备好了一个Activity组件切换操作的情况下，WindowManagerService类的成员函数executeAppTransition才可以调用另外一个成员函数performLayoutAndPlaceSurfacesLocked来执行该Activity组件切换操作。从前面的Step 2可以知道，如果WindowManagerService类的成员变量mNextAppTransition的值不等于WindowManagerPolicy.TRANSIT_UNSET，那么就说明之前已经准备好了一个Activity组件切换操作。

在调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来执行一个Activity组件切换操作之前，WindowManagerService类的成员函数executeAppTransition还会将成员变量mAppTransitionReady的值设置为true，以便可以表示WindowManagerService服务已经就绪准备就绪执行一个Activity组件切换操作了。

接下来，我们就继续分析WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked的实现，以便可以了解WindowManagerService服务是如何执行一个Activity组件切换操作的。

Step 8. WindowManagerService.performLayoutAndPlaceSurfacesLocked

这一步可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文的Step 5，它主要就是做三件事情：

1. 检查是否有窗口资源等待回收。如果有的话，那么就调用WindowManagerService类的成员函数removeWindowInnerLocked来将它们从系统中移除，以便可以回收它们所占用的资源。
2. 调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner来刷新系统UI，其中就包含了执行在前面Step 2中所准备的Activity组件切换操作。
3. 在第2步的执行过程中，如果又有一些窗口要被移除，那么又会继续调用WindowManagerService类的成员函数removeWindowInnerLocked来将它们从系统中移除。移除完成之后，又会再次递归调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLocked来刷新系统UI。

我们主要关注第2步，即WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner的实现，以便可以了解Activity组件的切换过程。

Step 9.

```

WindowManagerService.performLayoutAndPlaceSurfacesLockedInner

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final void performLayoutAndPlaceSurfacesLockedInner(
        boolean recoveringMemory) {
        .....
    }
}

```



```

Surface.beginTransaction();
.....

try {
    .....
    int repeats = 0;
    int changes = 0;

    do {
        repeats++;
        if (repeats > 6) {
            .....
            break;
        }

        // 1. 计算各个窗口的大小，以便让各个窗口可以对自己的UI元素进行布局
        // FIRST LOOP: Perform a layout, if needed.
        if (repeats < 4) {
            changes = performLayoutLockedInner();
            if (changes != 0) {
                continue;
            }
        } else {
            Slog.w(TAG, "Layout repeat skipped after too many iterations");
            changes = 0;
        }

        // 2. 计算各个窗口接下来要执行的动画
        // Update animations of all applications, including those
        // associated with exiting/removed apps
        .....

        // 3. 执行各个窗口的动画
        // SECOND LOOP: Execute animations and update visibility of windows.
        .....

        // 4. 检查当前是否需要执行Activity组件切换操作
        // If we are ready to perform an app transition, check through
        // all of the app tokens to be shown and see if they are ready
        // to go.
        if (mAppTransitionReady) {
            .....
        }

        .....

    } while (changes != 0);

    // 5. 更新各个窗口的绘图表面
    // THIRD LOOP: Update the surfaces of all windows.
    .....
} catch (RuntimeException e) {
    .....
}

.....

Surface.closeTransaction();

.....

// 6. 销毁那些需要销毁的绘图表面
// Destroy the surface of any windows that are no longer visible.
.....

// 7. 删除那些正在退出的类型为WindowToken的窗口令牌
// Time to remove any exiting tokens?
.....

// 8. 删除那些正在退出的类型为AppWindowToken的窗口令牌

```

```

// Time to remove any exiting applications?
.....

// 9. 检查是否需要再一次刷新系统UI
if (needRelayout) {
    requestAnimationLocked(0);
} else if (animating) {
    requestAnimationLocked(currentTime+(1000/60)-SystemClock.uptimeMillis());
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner是整个WindowManagerService服务的核心，它的主要任务就是刷新系统的UI，执行的操作包括：

1. 计算各个窗口的大小，以便让各个窗口可以对自己的UI元素进行布局。这一步主要是通过调用WindowManagerService类的成员函数performLayoutLockedInner来实现的，具体可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文。
2. 计算各个窗口接下来要执行的动画。
3. 执行各个窗口的动画。
4. 检查当前是否需要执行Activity组件切换操作，即检查WindowManagerService类的成员变量mAppTransitionReady的值是否等于true。如果等于的话，那么接下来就会执行这个Activity组件切换操作，实际上就是给正在切换的Activity组件应用一个切换动画。
5. 更新各个窗口的绘图表面。
6. 销毁那些需要销毁的绘图表面。
7. 删除那些正在退出的类型为WindowToken的窗口令牌。
8. 删除那些正在退出的类型为AppWindowToken的窗口令牌。
9. 检查是否需要再一次刷新系统UI。前面的操作可能会导致窗口堆栈或者窗口布局发生变化，例如，前面第6步销毁的绘图表面可能是当前正在显示的输入法窗口或者壁纸窗口的绘图表面，或者前面第3步表明窗口的动画还没有结束，这时候都会要求WindowManagerService服务马上再次刷新系统UI，以便可以反映出窗口堆栈或者窗口布局的变化，例如，可以持续地将窗口的动画显示完毕。如果需要再一次刷新系统UI，那么有可能是需要马上执行的，也有可能是过一会再执行的。如果是窗口堆栈或者窗口布局发生了变化的情况，那么变量needRelayout的值就会等于true，这时候就会以0为参数来调用WindowManagerService类的成员函数requestAnimationLocked，以便可以马上再次刷新系统UI。如果是窗口的动画还没有结束，那么变量animating的值就会等于true，这时候就会以一个非0参数来调用WindowManagerService类的成员函数requestAnimationLocked，请求过一段时间后再刷新系统UI。这个非0参数的值大约就等于1000/60，其中，分子表示1000毫秒，也就是说，在1/60秒后再刷新系统UI，意思就是以60帧每秒（fps）的速度来显示窗口的动画。

注意，在上述9个操作中，第1步到第4步是放在一个do...while循环中执行的，这是因为第2步到第4步的操作会导致导致窗口堆栈或者窗口布局发生变化，例如，有些窗口本来是是不可见的，现在变成可见了，这时候就会要求重新计算各个窗口的大小，以及让各个窗口重新布局自己的UI元素，即重新执行第1步的操作。然而，这些操作不能无休止地执行，否则的话，系统的UI就会死在那里不动了，因此，这个do...while循环最多只能执行7次。此外，在这最多7次的循环中，只有前4次循环才会重新各个窗口的大小。这些都是为了尽早地结束上述的do...while的，以便系统的UI可以尽快地刷新出来。

另外一个还有一个地方是需要注意的，即第1步到第5步的操作是放在一个事务中执行，即在Surface.openTransaction和Surface.closeTransaction之间执行，这意味着当Surface.closeTransaction执行完成之后，WindowManagerService服务才会通知SurfaceFlinger服务将系统的UI渲染到帧缓冲区（FB）中去，也就是说，在Surface.closeTransaction执行之后，我们才会看到系统的新UI。实际上，前面第1步到第5步的操作都只是修改了各个窗口的绘制表面的状态，这些修改都只是反映在图形缓冲区中，而不是反映在帧缓冲区中的。

在本文中，我们主要关注第4步的操作，即与Activity组件切换相关的操作，我们分段来阅读这部分代码：

```

// If we are ready to perform an app transition, check through
// all of the app tokens to be shown and see if they are ready
// to go.

```

```

if (mAppTransitionReady) {
    int NN = mOpeningApps.size();
    boolean goodToGo = true;
    .....
    if (!mDisplayFrozen && !mAppTransitionTimeout) {
        // If the display isn't frozen, wait to do anything until
        // all of the apps are ready. Otherwise just go because
        // we'll unfreeze the display when everyone is ready.
        for (i=0; i<NN && goodToGo; i++) {
            AppWindowToken wtoken = mOpeningApps.get(i);
            .....
            if (!wtoken.allDrawn && !wtoken.startingDisplayed
                && !wtoken.startingMoved) {
                goodToGo = false;
            }
        }
    }
}

```

从前面的Step 7可以知道，只有当WindowManagerService类的成员变量mAppTransitionReady的值等于true的时候，才说明WindowManagerService服务可以执行一个Activity组件切换操作。不过在执行这个切换操作之前，要求所有正在打开的Activity组件的UI都已经绘制完毕。因为如果这些正在打开的Activity组件的UI还没有绘制完成的话，就无法给它们应用一个切换动画。

从前面的Step 4可以知道，系统当前所有正在打开的Activity组件都保存在WindowManagerService类的成员变量mOpeningApps所描述的一个ArrayList中，因此，这段代码就通过遍历这个ArrayList来检查每一个正在打开的Activity组件的UI是否已经绘制完成，即检查对应的AppWindowToken对象的成员变量allDrawn的值是否不等于true。如果不等于true的话，就说明还没有绘制完成。此外，还要求这些正在打开的Activity组件的启动窗口已经显示结束，或者已经转移给其它的Activity组件，即要求查对应的AppWindowToken对象的成员变量startingDisplayed和startingMoved的值均等于true。只要其中的一个正在打开的Activity组件不能满足上述条件，那么变量goodToGo的值就会等于false，表示这时候还不能执行Activity组件操作。

注意，上述检查是在屏幕不是处于冻结状态或者前面准备好的Activity组件切换操作尚未超时的情况下进行，即在WindowManagerService类的成员变量mDisplayFrozen和mAppTransitionTimeout的值均等于false的情况下进行的，这意味着：

1. 如果屏幕处于冻结状态，就要马上执行前面准备好的Activity组件切换操作，因为等到屏幕解冻的时候，上述的三个条件都会得到满足。
2. 如果者前面准备好的Activity组件切换操作已经超时，那么就说明不需要再等待了，而应该立即执行该Activity组件切换操作。

我们假设上述检查都能通过，即最终得到的变量goodToGo的值等于true，我们继续往下阅读代码：

```

if (goodToGo) {
    .....
    int transit = mNextAppTransition;
    if (mSkipAppTransitionAnimation) {
        transit = WindowManagerPolicy.TRANSIT_UNSET;
    }
    mNextAppTransition = WindowManagerPolicy.TRANSIT_UNSET;
    mAppTransitionReady = false;
    mAppTransitionRunning = true;
    mAppTransitionTimeout = false;
    mStartingIconInTransition = false;
    mSkipAppTransitionAnimation = false;

    mH.removeMessages(H.APP_TRANSITION_TIMEOUT);
}

```

由于接下来就要执行Activity组件切换操作了，因此，这段代码就先修改与WindowManagerService服务的Activity组件切换操作相关的状态。首先是将WindowManagerService类的成员变量mNextAppTransition的值保存在变量transit中，并且将该成员变量的值重置为WindowManagerPolicy.TRANSIT_UNSET，以便WindowManagerService服务接下来可以准备一个新的Activity组件切换操作。注意，如果WindowManagerService类的成员变量mSkipAppTransitionAnimation的值等于true，那么就意味着要跳过此次Activity组件切换操作，即将前面得到的变量transit的值设置为WindowManagerPolicy.TRANSIT_UNSET。一般来说，如果一个Activity组件还在显示启动窗口的过程中，又有另外一个Activity组件被启动，并且这个Activity组件也要求显示启动窗口，那么当前正在显示启动窗口的Activity组件就会跳过之前为它所准备的切换操作，这是为了让后面那个启动的Activity组件尽快地显示出来。

接着还会分别设置WindowManagerService类的以下五个成员变量的值，其中：

1. mAppTransitionReady的值被重置为false，表示WindowManagerService服务接下来可以接受执行下一个Activity组件切换操作；
2. mAppTransitionRunning的值被设置为true，表示WindowManagerService服务目前正处于显示Activity组件切换动画的过程中；

3. `mAppTransitionTimeout`的值被设置为`false`，这是由于前面准备好的`Activity`组件切换操作已经得到执行了，所以就不存在超时问题了，同时后面还会通过调用`WindowManagerService`类的成员变量`mH`所指向一个`H`对象的成员函数`removeMessages`来删除之前发送到`WindowManagerService`服务所运行在的线程的消息队列中的一个类型为`APP_TRANSITION_TIMEOUT`的消息；
4. `mStartingIconInTransition`的值被重置为`false`，因为此时那些正在打开的`Activity`组件的启动窗口都已经结束显示；
5. `mSkipAppTransitionAnimation`的值被重置为`false`，因为前面已经对跳过切换动画的情况进行过处理了。

我们继续向下阅读代码：

```
// If there are applications waiting to come to the
// top of the stack, now is the time to move their windows.
// (Note that we don't do apps going to the bottom
// here -- we want to keep their windows in the old
// Z-order until the animation completes.)
if (mToTopApps.size() > 0) {
    NN = mAppTokens.size();
    for (i=0; i<NN; i++) {
        AppWindowToken wtoken = mAppTokens.get(i);
        if (wtoken.sendingToTop) {
            wtoken.sendingToTop = false;
            moveAppWindowsLocked(wtoken, NN, false);
        }
    }
    mToTopApps.clear();
}
```

这段代码对那些被请求移动至窗口堆栈顶端的`Activity`组件进行处理。这些被请求移动至窗口堆栈顶端的`Activity`组件会被保存在`WindowManagerService`类的成员变量`mToTopApps`所描述的一个`ArrayList`中。同时，用来描述这些被请求移动至窗口堆栈顶端的`Activity`组件的`AppWindowToken`对象的成员变量`sendingToTop`的值也会等于`true`。对于这些`Activity`组件，是通过调用`WindowManagerService`类的成员函数`moveAppWindowsLocked`来将它们移动到窗口堆栈顶端的。注意，在移动之前，还会将对应的`AppWindowToken`对象的成员变量`sendingToTop`的值设置为`false`。除了可以请求`WindowManagerService`服务将指定的`Activity`组件移动到窗口堆栈顶端之外，还可以请求`WindowManagerService`服务将指定的`Activity`组件移动到窗口堆栈底端。这些被请求移动至窗口堆栈底端的`Activity`组件是保存在`WindowManagerService`类的成员变量`mToBottomApps`所描述的一个`ArrayList`中。不过，这些`Activity`组件要等到切换动画显示完成之后，才会被移动至窗口堆栈底端。

将那些被请求移动至窗口堆栈顶端的`Activity`组件都移动到窗口堆栈顶端之后，就可以将`WindowManagerService`类的成员变量`mToTopApps`所描述的一个`ArrayList`清空了。

我们继续往下阅读代码：

```
WindowState oldWallpaper = mWallpaperTarget;

adjustWallpaperWindowsLocked();
.....

// The top-most window will supply the layout params,
// and we will determine it below.
LayoutParams animLp = null;
AppWindowToken animToken = null;
int bestAnimLayer = -1;

.....
int foundWallpapers = 0;
// Do a first pass through the tokens for two
// things:
// (1) Determine if both the closing and opening
// app token sets are wallpaper targets, in which
// case special animations are needed
// (since the wallpaper needs to stay static
// behind them).
// (2) Find the layout params of the top-most
// application window in the tokens, which is
// what will control the animation theme.
final int NC = mClosingApps.size();
NN = NC + mOpeningApps.size();
for (i=0; i<NN; i++) {
    AppWindowToken wtoken;
```

```

int mode;
if (i < NC) {
    wtoken = mClosingApps.get(i);
    mode = 1;
} else {
    wtoken = mOpeningApps.get(i-NC);
    mode = 2;
}
if (mLowerWallpaperTarget != null) {
    if (mLowerWallpaperTarget.mAppToken == wtoken
        || mUpperWallpaperTarget.mAppToken == wtoken) {
        foundWallpapers |= mode;
    }
}
if (wtoken.appFullscreen) {
    WindowState ws = wtoken.findMainWindow();
    if (ws != null) {
        // If this is a compatibility mode
        // window, we will always use its anim.
        if ((ws.mAttrs.flags & FLAG_COMPATIBLE_WINDOW) != 0) {
            animLp = ws.mAttrs;
            animToken = ws.mAppToken;
            bestAnimLayer = Integer.MAX_VALUE;
        } else if (ws.mLayer > bestAnimLayer) {
            animLp = ws.mAttrs;
            animToken = ws.mAppToken;
            bestAnimLayer = ws.mLayer;
        }
    }
}
}
}

```

这段代码用来检查那些参与切换操作的Activity组件的窗口是否与壁纸窗口有关。如果有关的话，接下来要执行的切换动画就需要修改为与壁纸相关的类型。要检查那些参与切换操作的Activity组件的窗口是否与壁纸窗口有关，首先需要确保壁纸窗口目前是位于那些需要显示壁纸的窗口的下面的，这是通过调用WindowManagerService类的成员函数adjustWallpaperWindowsLocked来实现的。在调整壁纸窗口在窗口堆栈的位置之前，会首先将壁纸窗口的当前目标窗口保存在变量oldWallpaper中，这是因为接下来要执行的切换动画的类型与壁纸窗口当前是否具有目标窗口有关。这一点我们后面再分析。

参与切换操作的Activity组件可以划分为两类，一类是需要打开的，一类是需要关闭的，它们分别保存在WindowManagerService类的成员变量mOpeningApps和mClosingApps所描述的ArrayList中。因此，通过检查保存在这两个ArrayList中的Activity组件的窗口是否是是需要显示壁纸的，就可以知道那些参与切换操作的Activity组件的窗口是否与壁纸窗口有关。

从前面Android窗口管理服务WindowManagerService对壁纸窗口（Wallpaper Window）的管理分析一文可以知道，在调整壁纸窗口在窗口堆栈的位置的时候，如果刚好碰到系统在执行两个Activity组件的切换操作，并且这两个Activity组件都需要显示壁纸，那么Z轴位置较低的窗口就会保存在WindowManagerService类的成员变量mLowerWallpaperTarget中，而Z轴位置较高的窗口就会保存在WindowManagerService类的成员变量mUpperWallpaperTarget中。因此，这段代码就可以通过一个for循环来检查保存在WindowManagerService类的成员变量mOpeningApps和mClosingApps中那些AppWindowToken对象，如果它们刚好被WindowManagerService类的成员变量mLowerWallpaperTarget或者mUpperWallpaperTarget所描述的WindowState对象的成员变量mAppToken所引用，那么就说明那些参与切换操作的Activity组件的窗口是与壁纸窗口有关的。

最终得到的变量foundWallpapers的值就反映了那些参与切换操作的Activity组件的窗口是否与壁纸窗口有关，其中：

1. 变量foundWallpapers的值等于0表示与壁纸窗口无关；
2. 变量foundWallpapers的值等于1表示只有那些需要关闭的Activity组件与壁纸窗口无关；
3. 变量foundWallpapers的值等于2表示只有那些需要打开的Activity组件与壁纸窗口无关；
4. 变量foundWallpapers的值等于2表示需要打开和关闭的Activity组件均与壁纸窗口无关。

这段代码的for循环除了用来检查那些参与切换操作的Activity组件的窗口是否与壁纸窗口有关之外，还有另外一个重要的任务，那就是找到用来创建Activity组件切换动画的参数。用来创建Activity组件切换动画的参数是保存在一个类型为WindowManager.LayoutParams的对象中的，而这个类型为WindowManager.LayoutParams的对象是需要来自那些参与切换操作的Activity组件的窗口的。

我们知道，每一个窗口都具有一个WindowManager.LayoutParams对象，用来描述它的属性，因此，这段代码的for循环就是要从参与切换操作的Activity组件的窗口的WindowManager.LayoutParams对象中挑选出一个来创建切换动画，然后再将这个切换动画应用到每一个参与切换操作的Activity组件的窗口中去。这个被挑选中的窗口应用具有以下属

性:

1. 它所属的Activity组件是全屏显示的，即用来描述该Activity组件的AppWindowToken对象的成员变量appFullscreen的值等于true
2. 它是它所属的Activity组件的主窗口。一个Activity组件的主窗口可以通过调用对应的AppWindowToken对象的成员函数findMainWindow来获得。
3. 它是所有候选窗口中Z轴位置最高的，即用来描述它的一个WindowState对象的成员变量mLayer的值是最大的，或者它是一个兼容窗口，即用来描述它的一个WindowState对象的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量flags的值的FLAG_COMPATIBLEWINDOW位等于1。

一旦找到了这样的窗口，这段代码中的for循环就会将用来描述它的属性的一个WindowManager.LayoutParams对象保存在变量animLp中，并且会将用来描述它所属的Activity组件的一个AppWindowToken对象保存在变量animToken中。

我们继续往下阅读代码:

```
if (foundWallpapers == 3) {
    .....
    switch (transit) {
        case WindowManagerPolicy.TRANSIT_ACTIVITY_OPEN:
        case WindowManagerPolicy.TRANSIT_TASK_OPEN:
        case WindowManagerPolicy.TRANSIT_TASK_TO_FRONT:
            transit = WindowManagerPolicy.TRANSIT_WALLPAPER_INTRA_OPEN;
            break;
        case WindowManagerPolicy.TRANSIT_ACTIVITY_CLOSE:
        case WindowManagerPolicy.TRANSIT_TASK_CLOSE:
        case WindowManagerPolicy.TRANSIT_TASK_TO_BACK:
            transit = WindowManagerPolicy.TRANSIT_WALLPAPER_INTRA_CLOSE;
            break;
    }
    .....
} else if (oldWallpaper != null) {
    // We are transitioning from an activity with
    // a wallpaper to one without.
    transit = WindowManagerPolicy.TRANSIT_WALLPAPER_CLOSE;
    .....
} else if (mWallpaperTarget != null) {
    // We are transitioning from an activity without
    // a wallpaper to now showing the wallpaper
    transit = WindowManagerPolicy.TRANSIT_WALLPAPER_OPEN;
    .....
}

if ((transit & WindowManagerPolicy.TRANSIT_ENTER_MASK) != 0) {
    mLastEnterAnimToken = animToken;
    mLastEnterAnimParams = animLp;
} else if (mLastEnterAnimParams != null) {
    animLp = mLastEnterAnimParams;
    mLastEnterAnimToken = null;
    mLastEnterAnimParams = null;
}

// If all closing windows are obscured, then there is
// no need to do an animation. This is the case, for
// example, when this transition is being done behind
// the lock screen.
if (!mPolicy.allowAppAnimationsLw()) {
    animLp = null;
}
```

这段代码用来最终确定接下来要设置的切换动画的类型。如果变量foundWallpapers的值等于3，那么就说明正在打开和正在关闭的Activity组件的窗口均与壁纸窗口有关，这时候就会按照以下规则来修改之前所设置的切换动画的类型:

1. 如果之前所设置的切换动画是与打开操作相关的，即变量transit的值等于WindowManagerPolicy.TRANSITACTIVITYOPEN、WindowManagerPolicy.TRANSITTASKOPEN或者WindowManagerPolicy.TRANSITTASKTOFRONT，那么就会将实际的切换动画的类型设置为WindowManagerPolicy.TRANSITWALLPAPERINTRAOPEN。这种类型的切换动画是与打开壁纸窗口相关的。
2. 如果之前所设置的切换动画是与关闭操作相关的，即变量transit的值等于

WindowManagerPolicy.TRANSITACTIVITYCLOSE、WindowManagerPolicy.TRANSITTASKCLOSE或者WindowManagerPolicy.TRANSITTASKTOBACK，那么就会将实际的切换动画的类型设置为WindowManagerPolicy.TRANSITWALLPAPERINTRACLOSE。这种类型的切换动画是与关闭壁纸窗口相关的。

如果变量foundWallpapers的值不等于3，但是变量oldWallpaper的值不等于null，那么就说明当前正在执行的Activity组件切换操作是从一个需要显示壁纸的Activity组件切换到一个不需要显示壁纸的Activity组件上去，这时候就会将实际的切换动画的类型设置为WindowManagerPolicy.TRANSITWALLPAPERCLOSE。这种类型的切换动画是与关闭壁纸窗口相关的。

如果变量foundWallpapers的值不等于3，并且变量oldWallpaper的值等于null，但是WindowManagerService类的成员变量mWallpaperTarget的值不等于null，那么就说明当前正在执行的Activity组件切换操作是从一个不需要显示壁纸的Activity组件切换到一个需要显示壁纸的Activity组件上去，这时候就会将实际的切换动画的类型设置为WindowManagerPolicy.TRANSITWALLPAPEROPEN。这种类型的切换动画是与打开壁纸窗口相关的。

最终得到的切换动画的类型就保存在变量transit中。从上面的逻辑可以知道，最终得到的切换动画的类型可能是与关闭操作相关的，但是我们需要的是一个与打开操作相关的切换动画。因此，如果最终得到的切换动画的类型是与关闭操作相关的，那么我们就使用上一次使用的Activity组件切换动画。用来创建上一次使用的Activity组件切换动画的WindowManager.LayoutParams对象和AppWindowToken分别保存在WindowManagerService类的成员变量mLastEnterAnimParams和mLastEnterAnimToken中。

通过检查变量transit的值的TRANSITENTERMASK位是否不等于0，就可以知道最终得到的切换动画的类型是否是打开操作相关的。如果是与打开操作相关的，那么就会将前面获得的变量animToken和animLp的值保存在WindowManagerService类的成员变量mLastEnterAnimToken和mLastEnterAnimParams中，以便以后可以重复使用。如果是与关闭操作相关的，那么就会将WindowManagerService类的成员变量mLastEnterAnimParams的值保存在变量animLp中，以及将WindowManagerService类的成员变量mLastEnterAnimToken和mLastEnterAnimParams重置为null，以便表示上一次使用的Activity组件切换动画不是与打开操作相关的。

此时，如果我们所获得的变量animLp的值不等于null，那么它所指向的一个WindowManager.LayoutParams对象就可以用来创建一个Activity组件切换动画，但是如果窗口管理策略类表明此时不需要显示Activity组件切换动画，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数allowAppAnimationsLw的返回值等于false，那么前面得的变量animLp的值就会被重置为null。当正在执行的Activity组件切换操作是发生在锁屏窗口的后面时，就会出现这种情况。

我们继续往下阅读代码：

```

NN = mOpeningApps.size();
for (i=0; i<NN; i++) {
    AppWindowToken wtoken = mOpeningApps.get(i);
    .....
    wtoken.reportedVisible = false;
    wtoken.inPendingTransaction = false;
    wtoken.animation = null;
    setTokenVisibilityLocked(wtoken, animLp, true, transit, false);
    wtoken.updateReportedVisibilityLocked();
    wtoken.waitingToShow = false;
    wtoken.showAllWindowsLocked();
}
NN = mClosingApps.size();
for (i=0; i<NN; i++) {
    AppWindowToken wtoken = mClosingApps.get(i);
    .....
    wtoken.inPendingTransaction = false;
    wtoken.animation = null;
    setTokenVisibilityLocked(wtoken, animLp, false, transit, false);
    wtoken.updateReportedVisibilityLocked();
    wtoken.waitingToHide = false;
    // Force the allDrawn flag, because we want to start
    // this guy's animations regardless of whether it's
    // gotten drawn.
    wtoken.allDrawn = true;
}
.....

mOpeningApps.clear();
mClosingApps.clear();
```

这段代码将给所有参与了切换操作的Activity组件设置一个切换动画，而这个动画就来自前面所获得的变量animLp所指向的一个WindowManager.LayoutParams对象。由于参与了切换操作的Activity组件可以划分为两类，即一类是正在打开的，一类是正在关闭的，因此，我们就分别讨论这两种类型的Activity组件的切换动画的设置过程。对于正在打开的Activity组件，它们的切换动画的设置过程如下所示：

1. 找到对应的AppWindowToken对象；
2. 将对应的AppWindowToken对象的成员变量reportedVisible的值设置为false，表示还没有向ActivityManagerService服务报告过正在打开的Activity组件的可见性；
3. 将对应的AppWindowToken对象的成员变量inPendingTransaction的值设置为false，表示正在打开的Activity组件不是处于等待执行切换操作的状态了；
4. 将对应的AppWindowToken对象的成员变量animation的值设置为null，因为接下来要重新这个成员变量的值来描述正在打开的Activity组件的切换动画；
5. 调用WindowManagerService类的成员函数setTokenVisibilityLocked将正在打开的Activity组件的可见性设置为true，并且给正在打开的Activity组件设置一个切换动画，这个切换动画会保存在对应的AppWindowToken对象的成员变量animation中；
6. 调用对应的AppWindowToken对象的成员函数updateReportedVisibilityLocked向ActivityManagerService服务报告正在打开的Activity组件的可见性；
7. 将对应的AppWindowToken对象的成员变量waitingToShow的值设置为false，表示正在打开的Activity组件的窗口不是处于等待显示的状态了；
8. 调用对应的AppWindowToken对象的成员函数showAllWindowsLocked通知SurfaceFlinger服务将正在打开的Activity组件的窗口设置为可见的。

对于正在关闭的Activity组件，它们的切换动画的设置过程如下所示：

1. 找到对应的AppWindowToken对象；
2. 将对应的AppWindowToken对象的成员变量inPendingTransaction的值设置为false，表示正在关闭的Activity组件不是处于等待执行切换操作的状态了；
3. 将对应的AppWindowToken对象的成员变量animation的值设置为null，因为接下来要重新这个成员变量的值来描述正在关闭的Activity组件的切换动画；
4. 调用WindowManagerService类的成员函数setTokenVisibilityLocked将正在关闭的Activity组件的可见性设置为true，并且给正在关闭的Activity组件设置一个切换动画，这个切换动画会保存在对应的AppWindowToken对象的成员变量animation中；
5. 调用对应的AppWindowToken对象的成员函数updateReportedVisibilityLocked向ActivityManagerService服务报告正在关闭的Activity组件的可见性；
6. 将对应的AppWindowToken对象的成员变量waitingToHide的值设置为false，表示正在关闭的Activity组件的窗口不是处于等待隐藏的状态了；
7. 将对应的AppWindowToken对象的成员变量allDrawn的值设置为true，这样就可以使得前面所设置的切换动画得以执行。

给所有参与了切换操作的Activity组件都设置了一个切换动画之后，接下来就可以将WindowManagerService类的成员变量mOpeningApps和mClosingApps所描述的两个ArrayList清空了。

我们继续向下阅读最后一段代码：

```
// This has changed the visibility of windows, so perform
// a new layout to get them all up-to-date.
changes |= PhoneWindowManager.FINISH_LAYOUT_REDO_LAYOUT;
.....
}
}
```

由于前面的操作已经导致那些正在打开的Activity组件的窗口由不可见变为可见，即相当于是导致窗口堆栈发生了变化，这时候就需要重新计算各个窗口的大小，以便让各个窗口对自己的UI元素进行重新布局，这是通过将变量changes的值的PhoneWindowManager.FINISH_LAYOUT_REDO_LAYOUT位设置为1来实现的。从前面的分析可以知道，一旦变量changes的值不等于0，WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner开始的那个do...while循环就会重复执行，也就是会重复执行以下三个操作：

1. 计算各个窗口的大小，以便让各个窗口可以对自己的UI元素进行布局。
2. 计算各个窗口接下来要执行的动画。
3. 执行各个窗口的动画。

至此，我们就分析完成了Activity组件切换操作了。Activity组件的切换操作执行完成之后，参与了切换操作的Activity组件的窗口就会获得一个切换动画。一个窗口的切换动画，与其本身所设置的进入动画，以及其父窗口所设置的动画，一起形成了窗口的显示过程动画。在接下来的一篇文章中，我们就继续分析窗口的显示过程动画是如何执行的，敬请关注！

第九节 Android窗口管理服务WindowManagerService显示窗口动画的原理分析

在前一文中，我们分析了Activity组件的切换过程。从这个过程可以知道，所有参与切换操作的窗口都会被设置切换动画。事实上，一个窗口在打开（关闭）的过程中，除了可能会设置切换动画之外，它本身也可能会设置有进入（退出）动画。再进一步地，如果一个窗口是附加在另外一个窗口之上的，那么被附加窗口所设置的动画也会同时传递给该窗口。本文就详细分析WindowManagerService服务显示窗口动画的原理。

《Android系统源代码情景分析》一书正在进击的程序员网（<http://0xcc0xcd.com>）中连载，点击进入！

在Android系统中，窗口动画的本质就是对原始窗口施加一个变换（Transformation）。在线性数学中，对物体的形状进行变换是通过乘以一个矩阵（Matrix）来实现，目的就是物体进行偏移、旋转、缩放、切变、反射和投影等。因此，给窗口设置动画实际上就给窗口设置一个变换矩阵（Transformation Matrix）。

如前所述，一个窗口在打开（关闭）的过程，可能会被设置三个动画，它们分别是窗口本身所设置的进入（退出）动画（Self Transformation）、从被附加窗口传递过来的动画（Attached Transformation），以及宿主Activity组件传递过来的切换动画（App Transformation）。这三个Transformation组合在一起形成一个变换矩阵，以60fps的速度应用在窗口的原始形状之上，完成窗口的动画过程，如图1所示。

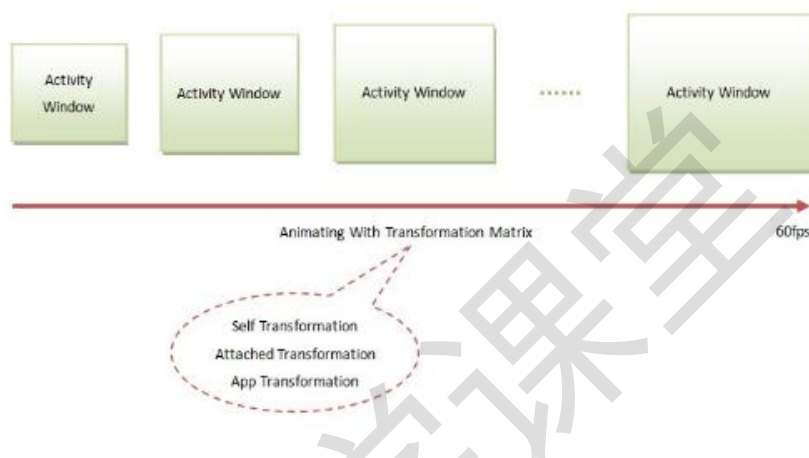


图1 窗口的动画显示过程

从上面的分析可以知道，窗口的变换矩阵是应用在窗口的原始位置和大小之上的，因此，在显示窗口的动画之前，除了要窗口设置变换矩阵之外，还要计算好窗口的原始位置和大小，以及布局和绘制好窗口的UI。在前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析和Android应用程序窗口（Activity）的测量（Measure）、布局（Layout）和绘制（Draw）过程分析这两篇文章中，我们已经分析过窗口的位置和大小计算过程以及窗口UI的布局和绘制过程了，本文主要关注窗口动画的设置、合成和显示过程。这三个过程通过以下四个部分的内容来描述：

1. 窗口动画的设置过程
2. 窗口动画的显示框架
3. 窗口动画的推进过程
4. 窗口动画的合成过程

其中，窗口动画的设置过程包括上述三个动画的设置过程，窗口动画的推进过程是指定动画的一步地迁移的过程，窗口动画的合成过程是指上述三个动画组合成一个变换矩阵的过程，后两个过程包含在了窗口动画的显示框架中。

一. 窗口动画的设置过程

窗口被设置的动画虽然可以达到三个，但是这三个动画可以归结为两类，一类是普通动画，例如，窗口在打开过程中被设置的进入动画和在关闭过程中被设置的退出动画，另一类是切换动画。其中，Self Transformation和Attached Transformation都是属于普通动画，而App Transformation属于切换动画。接下来我们就分别分析这两种类型的动画的设置过程。

1. 普通动画的设置过程

从前面Android窗口管理服务WindowManagerService显示Activity组件的启动窗口（Starting Window）的过程分析一文可以知道，窗口在打开的过程中，是通过调用WindowState类的成员函数performShowLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {
        .....

        boolean performShowLocked() {
```

```

.....

    if (mReadyToShow && isReadyForDisplay()) {
        .....

        if (!showSurfaceRobustlyLocked(this)) {
            return false;
        }

        .....

        applyEnterAnimationLocked(this);

        .....
    }
    return true;
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowState类的成员函数performShowLocked首先是调用WindowManagerService类的成员函数showSurfaceRobustlyLocked来通知SurfaceFlinger服务将当前正在处理的窗口设置为可见，接着再调用WindowManagerService类的成员函数applyEnterAnimationLocked来给当前正在处理的窗口设置一个进入动画，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private void applyEnterAnimationLocked(WindowState win) {
        int transit = WindowManagerPolicy.TRANSIT_SHOW;
        if (win.mEnterAnimationPending) {
            win.mEnterAnimationPending = false;
            transit = WindowManagerPolicy.TRANSIT_ENTER;
        }

        applyAnimationLocked(win, transit, true);
    }

    .....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。如果参数win所指向的一个WindowState对象的成员变量mEnterAnimationPending的值等于true，那么就说明它所描述的窗口正在等待显示，也就是正处于不可见到可见状态的过程中，那么WindowManagerService类的成员函数applyEnterAnimationLocked就会对该窗口设置一个类型为WindowManagerPolicy.TRANSIT_ENTER的动画，否则的话，就会对该窗口设置一个类型为WindowManagerPolicy.TRANSIT_SHOW的动画。

确定好窗口的动画类型之后，WindowManagerService类的成员函数applyEnterAnimationLocked就调用另外一个成员函数applyAnimationLocked来为窗口创建一个动画了。接下来我们先分析窗口在关闭的过程中所设置的动画类型，然后再来分析WindowManagerService类的成员函数applyAnimationLocked的实现。

从前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文可以知道，当应用程序进程请求WindowManagerService服务刷新一个窗口的时候，会调用到WindowManagerService类的成员函数relayoutWindow。WindowManagerService类的成员函数relayoutWindow在执行的过程中，如果发现需要将一个窗口从可见状态设置为不可见状态时，也就是发现需要关闭一个窗口时，就会对该窗口设置一个退出动画，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    public int relayoutWindow(Session session, IWindow client,
        WindowManager.LayoutParams attrs, int requestedWidth,
        int requestedHeight, int viewVisibility, boolean insetsPending,
        Rect outFrame, Rect outContentInsets, Rect outVisibleInsets,
        Configuration outConfig, Surface outSurface) {
        .....
    }
}

```

```

synchronized(mWindowMap) {
    WindowState win = windowForClientLocked(session, client, false);
    .....

    if (viewVisibility == View.VISIBLE &&
        (win.mAppToken == null || !win.mAppToken.clientHidden)) {
        .....
    } else {
        .....

        if (win.mSurface != null) {
            .....

            // If we are not currently running the exit animation, we
            // need to see about starting one.
            if (!win.mExiting || win.mSurfacePendingDestroy) {
                // Try starting an animation; if there isn't one, we
                // can destroy the surface right away.
                int transit = WindowManagerPolicy.TRANSIT_EXIT;
                .....

                if (!win.mSurfacePendingDestroy && win.isWinVisibleLw() &&
                    applyAnimationLocked(win, transit, false)) {
                    .....
                    win.mExiting = true;
                }

                .....
            }

            .....
        }

        .....
    }

    .....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。WindowState对象win描述的便是要刷新的窗口。当参数viewVisibility的值不等于View.VISIBLE时，就说明要将WindowState对象win所描述的窗口设置为不可见。另一方面，如果WindowState对象win的成员变量mAppToken的值不等于null，并且这个成员变量所指向的一个AppWindowToken对象的成员变量clientHidden的值等于true，那么就说明WindowState对象win所描述的窗口是一个与Activity组件相关的窗口，并且该Activity组件是处于不可见状态的。在这种情况下，也需要将WindowState对象win所描述的窗口设置为不可见。

一旦WindowState对象win所描述的窗口要设置为不可见，就需要考虑给它设置一个退出动画，不过有四个前提条件：

1. 该窗口有一个绘图表面，即WindowState对象win的成员变量mSurface的值不等于null；
2. 该窗口的绘图表面不是处于等待销毁的状态，即WindowState对象win的成员变量mSurfacePendingDestroy的值不等于true；
3. 该窗口不是处于正在关闭的状态，即WindowState对象win的成员变量mExiting的值不等于true；
4. 该窗口当前正在处于可见的状态，即WindowState对象win的成员isWinVisibleLw的返回值等于true。

在满足上述四个条件的情况下，就说明WindowState对象win所描述的窗口的状态要由可见变为不可见，因此，就需要给它设置一个退出动画，即一个类型为WindowManagerPolicy.TRANSIT_EXIT的动画，这同样是通过调用WindowManagerService类的成员函数applyAnimationLocked来实现的。

从上面的分析就可以知道，无论是窗口在打开时所需要的进入动画，还是窗口在关闭时所需要的退出动画，都是通过调用WindowManagerService类的成员函数applyAnimationLocked来设置的，它的实现如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private boolean applyAnimationLocked(WindowState win,
        int transit, boolean isEntrance) {

```

```

    if (win.mLocalAnimating && win.mAnimationIsEntrance == isEntrance) {
        // If we are trying to apply an animation, but already running
        // an animation of the same type, then just leave that one alone.
        return true;
    }

    // Only apply an animation if the display isn't frozen. If it is
    // frozen, there is no reason to animate and it can cause strange
    // artifacts when we unfreeze the display if some different animation
    // is running.
    if (!mDisplayFrozen && mPolicy.isScreenOn()) {
        int anim = mPolicy.selectAnimationLw(win, transit);
        int attr = -1;
        Animation a = null;
        if (anim != 0) {
            a = AnimationUtils.loadAnimation(mContext, anim);
        } else {
            switch (transit) {
                case WindowManagerPolicy.TRANSIT_ENTER:
                    attr = com.android.internal.R.styleable.WindowAnimation_windowEnterAnimation;
                    break;
                case WindowManagerPolicy.TRANSIT_EXIT:
                    attr = com.android.internal.R.styleable.WindowAnimation_windowExitAnimation;
                    break;
                case WindowManagerPolicy.TRANSIT_SHOW:
                    attr = com.android.internal.R.styleable.WindowAnimation_windowShowAnimation;
                    break;
                case WindowManagerPolicy.TRANSIT_HIDE:
                    attr = com.android.internal.R.styleable.WindowAnimation_windowHideAnimation;
                    break;
            }
            if (attr >= 0) {
                a = loadAnimation(win.mAttrs, attr);
            }
        }
        .....

        if (a != null) {
            .....

            win.setAnimation(a);
            win.mAnimationIsEntrance = isEntrance;
        }
    }

    .....

    return win.mAnimation != null;
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

参数win描述的是要设置动画的窗口，参数transit描述的是要设置的动画的类型，而参数isEntrance描述的是要设置的动画是进入类型还是退出类型的。

如果参数win所指向的一个WindowState对象的成员变量mLocalAnimating的值等于true，那么就说明它所描述的窗口已经被设置过动画了，并且这个动画正在显示的过程中。在这种情况下，如果这个WindowState对象的成员变量mAnimationIsEntrance的值等于参数isEntrance的值，那么就说明该窗口正在显示的动画就是所要求设置的动画，这时候就不需要给窗口重新设置一个动画了，因此，WindowManagerService类的成员函数applyAnimationLocked就直接返回了。

我们假设需要给参数win所描述的窗口设置一个新的动画，这时候还需要继续判断屏幕当前是否是处于非冻结和点亮的状态的。只有在屏幕不是被冻结并且是点亮的情况下，WindowManagerService类的成员函数applyAnimationLocked才真正需要给参数win所描述的窗口设置一个动画，否则的话，设置了也是无法显示的。当WindowManagerService类的成员变量mDisplayFrozen的时候，就说明屏幕不是被冻结的，而当WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值等于true的时候，就说明屏幕是点亮的。在满足上述两个条件的情况下，WindowManagerService类的成员函数applyAnimationLocked就开始给参数win所描述的窗口创建动画了。

WindowManagerService类的成员函数applyAnimationLocked首先是检查WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象是否可以作为参数win所描述的窗口提供一个类型为transit的动画。如果可以的话，那么调用这个PhoneWindowManager对象的成员函数selectAnimationLw的返回值anim就不等于0，这时候WindowManagerService类的成员函数applyAnimationLocked就可以调用AnimationUtils类的静态成员函数loadAnimation来根据该返回值anim来创建一个动画，并且保存在变量a中。

如果WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象不可以作为参数win所描述的窗口提供一个类型为transit的动画的话，那么WindowManagerService类的成员函数applyAnimationLocked就需要根据该窗口的布局参数来创建这个动画了。这个创建过程分为两步执行：

1. 将参数transit的值转化为一个对应的动画样式名称；
2. 调用WindowManagerService类的成员函数loadAnimation来在指定的窗口布局参数中创建前面第1步所指定样式名称的动画，并且保存在变量a中，其中，指定的窗口布局参数是由WindowState对象win的成员变量mAttrs所指向的一个WindowManager.LayoutParams对象来描述的。

最后，如果变量a的值不等于null，即前面成功地为参数win所描述的窗口创建了一个动画，那么接下来就会将该动画设置给参数win所描述的窗口。这是通过参数win所指向的一个WindowState对象的成员函数setAnimation来实现的，实际上就是将变量所指向的一个Animation对象保存在参数win所指向的一个WindowState对象的成员变量mAnimation中。同时，WindowManagerService类的成员函数applyAnimationLocked还会将参数isEntrance的值保存在参数win所指向的一个WindowState对象的成员变量mAnimationIsEntrance，以表明前面给它所设置的动画是属于进入类型还是退出类型的。

2. 切换动画的设置过程

从前面Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析一文可以知道，如果一个窗口属于一个Activity组件窗口，那么当该Activity组件被切换的时候，就会被设置一个切换动画，这是通过调用WindowManagerService类的成员函数setTokenVisibilityLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    boolean setTokenVisibilityLocked(AppWindowToken wtoken, WindowManager.LayoutParams lp,
        boolean visible, int transit, boolean performLayout) {
        boolean delayed = false;
        .....

        if (wtoken.hidden == visible) {
            final int N = wtoken.allAppWindows.size();
            .....

            boolean runningAppAnimation = false;

            if (transit != WindowManagerPolicy.TRANSIT_UNSET) {
                if (wtoken.animation == sDummyAnimation) {
                    wtoken.animation = null;
                }
                applyAnimationLocked(wtoken, lp, transit, visible);
                .....

                if (wtoken.animation != null) {
                    delayed = runningAppAnimation = true;
                }
            }

            for (int i=0; i<N; i++) {
                WindowState win = wtoken.allAppWindows.get(i);
                .....

                if (visible) {
                    if (!win.isVisibleNow()) {
                        if (!runningAppAnimation) {
                            applyAnimationLocked(win,
                                WindowManagerPolicy.TRANSIT_ENTER, true);
                        }
                        .....
                    }
                } else if (win.isVisibleNow()) {
                    if (!runningAppAnimation) {
                        applyAnimationLocked(win,
                            WindowManagerPolicy.TRANSIT_EXIT, false);
                    }
                    .....
                }
            }
        }
    }
}
```

```

    }

    .....

    }

    if (wtoken.animation != null) {
        delayed = true;
    }

    return delayed;
}

.....

}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。参数wtoken描述的是要切换的Activity组件，参数lp描述的是要用来创建切换动画的布局参数，参数transit描述的是要创建的切换动画的类型，而参数visible描述的是要切换的Activity组件接下来是否是可见的。

WindowManagerService类的成员函数setTokenVisibilityLocked首先判断要切换的Activity组件当前的可见性是否已经就是要设置的可见性，即参数wtoken所指向的一个AppWindowToken对象的成员变量hidden的值是否不等于参数visible的值。如果不等于的话，就说明要切换的Activity组件当前的可见性已经就是要设置的可见性了，这时候WindowManagerService类的成员函数setTokenVisibilityLocked就不用再为它设置切换动画了。

我们假设要切换的Activity组件当前的可见性不是要求设置的可见性，即参数wtoken所指向的一个AppWindowToken对象的成员变量hidden的值等于参数visible的值，那么WindowManagerService类的成员函数setTokenVisibilityLocked还会继续检查参数transit描述的是不是一个有效的动画类型，即它的值是否不等于WindowManagerPolicy.TRANSIT_UNSET。如果参数transit描述的是一个有效的动画类型的话，那么WindowManagerService类的成员函数setTokenVisibilityLocked接下来就会执行以下三个操作：

1. 判断要切换的Activity组件当前是否被设置了一个哑动画，即参数wtoken所指向的一个AppWindowToken对象的成员变量animation是否与WindowManagerService类的成员变量sDummyAnimation指向了同一个Animation对象。如果是的话，那么就会将wtoken所指向的一个AppWindowToken对象的成员变量animation的值设置为null，因为接下来要重新为它设置一个新的Animation对象。从前面Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析一文可以知道，一个需要参与切换的Activity组件会设置可见性的时候，是会被设置一个哑动画的。
2. 调用WindowManagerService类的四个参数版本的成员函数applyAnimationLocked根据参数lp、transit和visible的值来为要切换的Activity组件创建一个动画。
3. 如果第2步可以成功地要为要切换的Activity组件创建一个动画的话，那么这个动画就会保存在wtoken所指向的一个AppWindowToken对象的成员变量animation中，这时候就会将变量delayed和runningAppAnimation的值均设置为true。

变量runningAppAnimation的值等于true意味着与参数wtoken所描述的Activity组件所对应的窗口接下来要执行一个切换动画。在这种情况下，WindowManagerService类的成员函数setTokenVisibilityLocked就不需要为这些窗口单独设置一个进入或者退出类型的动画，否则的话，WindowManagerService类的成员函数setTokenVisibilityLocked就会根据这些窗口的当前可见性状态以及参数wtoken所描述的Activity组件被要求设置的可见性来单独设置一个进入或者退出类型的动画：

1. 如果一个窗口当前是不可见的，即用来描述它的一个WindowState对象的成员函数isVisibleNow的返回值等于false，但是参数wtoken所描述的Activity组件被要求设置成可见的，即参数visible的值等于true，那么就需要给该窗口设置一个类型为WindowManagerPolicy.TRANSIT_ENTER的动画；
2. 如果一个窗口当前是可见的，即用来描述它的一个WindowState对象的成员函数isVisibleNow的返回值等于true，但是参数wtoken所描述的Activity组件被要求设置成不可见的，即参数visible的值等于false，那么就需要给该窗口设置一个类型为WindowManagerPolicy.TRANSIT_EXIT的动画。

给与参数wtoken所描述的Activity组件所对应的窗口设置动画是通过调用WindowManagerService类的三个参数版本的成员函数applyAnimationLocked来实现的，这个成员函数在前面已经分析过了。另外，与参数wtoken所描述的Activity组件所对应的窗口是保存在参数wtoken所指向的一个AppWindowToken对象的成员变量allAppWindows所描述的一个ArrayList中的，因此，通过遍历这个ArrayList，就可以为与参数wtoken所描述的Activity组件所对应的每一个窗口设置一个动画。

最后，如果前面成功地为参数wtoken所描述的Activity组件创建了一个切换动画，即该参数所描述的一个AppWindowToken对象的成员变量animation的值不等于null，那么WindowManagerService类的成员函数setTokenVisibilityLocked的返回值delayed就会等于true，表示参数wtoken所描述的Activity组件要执行一个动画切换，同时也表明该Activity组件的窗口要延迟到切换动画显示结束后，才真正显示出来。

接下来，我们继续分析WindowManagerService类的四个参数版本的成员函数applyAnimationLocked的实现，以便可以了解Activity组件的切换动画的创建过程，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private boolean applyAnimationLocked(AppWindowToken wtoken,
        WindowManager.LayoutParams lp, int transit, boolean enter) {
        // Only apply an animation if the display isn't frozen. If it is

```

```

// frozen, there is no reason to animate and it can cause strange
// artifacts when we unfreeze the display if some different animation
// is running.
if (!mDisplayFrozen && mPolicy.isScreenOn()) {
    Animation a;
    if (lp != null && (lp.flags & FLAG_COMPATIBLE_WINDOW) != 0) {
        a = new FadeInOutAnimation(enter);
        .....
    } else if (mNextAppTransitionPackage != null) {
        a = loadAnimation(mNextAppTransitionPackage, enter ?
            mNextAppTransitionEnter : mNextAppTransitionExit);
    } else {
        int animAttr = 0;
        switch (transit) {
            case WindowManagerPolicy.TRANSIT_ACTIVITY_OPEN:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_activityOpenEnterAnimati
on
                    : com.android.internal.R.styleable.WindowAnimation_activityOpenExitAnimatio
n;
                break;
            case WindowManagerPolicy.TRANSIT_ACTIVITY_CLOSE:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_activityCloseEnterAnimat
ion
                    : com.android.internal.R.styleable.WindowAnimation_activityCloseExitAnimati
on;
                break;
            case WindowManagerPolicy.TRANSIT_TASK_OPEN:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_taskOpenEnterAnimation
                    : com.android.internal.R.styleable.WindowAnimation_taskOpenExitAnimation;
                break;
            case WindowManagerPolicy.TRANSIT_TASK_CLOSE:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_taskCloseEnterAnimation
                    : com.android.internal.R.styleable.WindowAnimation_taskCloseExitAnimation;
                break;
            case WindowManagerPolicy.TRANSIT_TASK_TO_FRONT:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_taskToFrontEnterAnimatio
n
                    : com.android.internal.R.styleable.WindowAnimation_taskToFrontExitAnimation;
                break;
            case WindowManagerPolicy.TRANSIT_TASK_TO_BACK:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_taskToBackEnterAnimation
                    : com.android.internal.R.styleable.WindowAnimation_taskToBackExitAnimation;
                break;
            case WindowManagerPolicy.TRANSIT_WALLPAPER_OPEN:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_wallpaperOpenEnterAnimat
ion
                    : com.android.internal.R.styleable.WindowAnimation_wallpaperOpenExitAnimati
on;
                break;
            case WindowManagerPolicy.TRANSIT_WALLPAPER_CLOSE:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_wallpaperCloseEnterAnima
tion
                    : com.android.internal.R.styleable.WindowAnimation_wallpaperCloseExitAnimat
ion;
                break;
            case WindowManagerPolicy.TRANSIT_WALLPAPER_INTRA_OPEN:
                animAttr = enter
                    ? com.android.internal.R.styleable.WindowAnimation_wallpaperIntraOpenEnterA
nimation
                    : com.android.internal.R.styleable.WindowAnimation_wallpaperIntraOpenExitAn
imation;
                break;

```

```

        case WindowManagerPolicy.TRANSIT_WALLPAPER_INTRA_CLOSE:
            animAttr = enter
                ? com.android.internal.R.styleable.WindowAnimation_wallpaperIntraCloseEnter
                : com.android.internal.R.styleable.WindowAnimation_wallpaperIntraCloseExitA
            Animation
            nimation;

            break;
        }
        a = animAttr != 0 ? loadAnimation(lp, animAttr) : null;
        .....
    }
    if (a != null) {
        .....
        wtoken.setAnimation(a);
    }
}

.....

return wtoken.animation != null;
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowManagerService类的四个参数版本的成员函数applyAnimationLocked和三个参数版本的成员函数applyAnimationLocked的实现是类似的，不过它是为Activity组件创建动画，并且：

1. 如果参数lp所描述的布局参数表明它是用来描述一个兼容窗口的，即它所指向的一个WindowManager.LayoutParams对象的成员变量flags的FLAG_COMPATIBLE_WINDOW位不等于0，那么创建的切换动画就固定为FadeInOutAnimation。
2. 如果WindowManagerService类的成员变量mNextAppTransitionPackage的值不等于null，那么就说明Package名称为mNextAppTransitionPackage的Activity组件指定了一个自定义的切换动画，其中，指定的进入动画的类型由WindowManagerService类的成员变量mNextAppTransitionEnter来描述，指定的退出动画的类型由WindowManagerService类的成员变量mNextAppTransitionExit来描述。在这种情况下，WindowManagerService服务就会使用这个指定的切换动画，而不是使用默认的切换动画。一般来说，一个个Activity组件在调用成员函数startActivity来通知ActivityManagerService服务启动另外一个Activity之外，可以马上调用另外一个成员函数overridePendingTransition来指定自定义的切换动画。
3. 切换动画的类型主要分三类：第一类是和Activity相关的；第二类是和Task相关的；第三类是和Wallpaper相关的。这一点可以参考前面Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析一文。

切换动画创建成功之后，就会调用参数wtoken所指向的一个AppWindowToken对象的成员函数setAnimation来保存在其成员变量animation中，这样就表示它所描述的Activity组件被设置了一个切换动画。

二. 窗口动画的显示框架

窗口动画是在WindowManagerService服务刷新系统UI的时候显示的。从前面前面Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析一文可以知道，WindowManagerService服务刷新系统UI是通过调用WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner来实现的，接下来我们就主要分析与窗口动画的显示框架相关的逻辑，如下所示：

```

public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final void performLayoutAndPlaceSurfacesLockedInner(
        boolean recoveringMemory) {
        .....

        Surface.openTransaction();
        .....

        try {
            .....
            int repeats = 0;
            int changes = 0;

            do {
                repeats++;
                if (repeats > 6) {
                    .....
                    break;
                }
            }
        }
    }
}

```



```

    }

    // 1. 计算各个窗口的大小
    // FIRST LOOP: Perform a layout, if needed.
    if (repeats < 4) {
        changes = performLayoutLockedInner();
        if (changes != 0) {
            continue;
        }
    } else {
        Slog.w(TAG, "Layout repeat skipped after too many iterations");
        changes = 0;
    }

    // 2. 推进各个Activity组件的切换动画
    // Update animations of all applications, including those
    // associated with exiting/removed apps
    boolean tokensAnimating = false;
    final int NAT = mAppTokens.size();
    for (i=0; i<NAT; i++) {
        if (mAppTokens.get(i).stepAnimationLocked(currentTime, dw, dh)) {
            tokensAnimating = true;
        }
    }
    final int NEAT = mExitingAppTokens.size();
    for (i=0; i<NEAT; i++) {
        if (mExitingAppTokens.get(i).stepAnimationLocked(currentTime, dw, dh)) {
            tokensAnimating = true;
        }
    }

    // 3. 推进各个窗口的动画
    // SECOND LOOP: Execute animations and update visibility of windows.
    .....

    animating = tokensAnimating;
    .....

    mPolicy.beginAnimationLw(dw, dh);

    final int N = mWindows.size();

    for (i=N-1; i>=0; i--) {
        WindowState w = mWindows.get(i);
        .....

        if (w.mSurface != null) {
            // Execute animation.
            .....
            if (w.stepAnimationLocked(currentTime, dw, dh)) {
                animating = true;
                .....
            }

            .....
        }

        .....

        mPolicy.animatingWindowLw(w, attrs);
    }

    .....

    changes |= mPolicy.finishAnimationLw();

    .....

} while (changes != 0);

```

```

// 4. 更新各个窗口的绘图表面
// THIRD LOOP: Update the surfaces of all windows.
.....

final int N = mWindows.size();

for (i=N-1; i>=0; i--) {
    WindowState w = mWindows.get(i);
    .....

    if (w.mSurface != null) {
        .....

        //计算实际要显示的大小和位置
        w.computeShownFrameLocked();
        .....

        //设置大小和位置等
        .....

        if (w.mAttachedHidden || !w.isReadyForDisplay()) {
            .....
        } else if (w.mLastLayer != w.mAnimLayer
            || w.mLastAlpha != w.mShownAlpha
            || w.mLastDsDx != w.mDsDx
            || w.mLastDtDx != w.mDtDx
            || w.mLastDsDy != w.mDsDy
            || w.mLastDtDy != w.mDtDy
            || w.mLastHScale != w.mHScale
            || w.mLastVScale != w.mVScale
            || w.mLastHidden) {
            .....

            //设置Z轴位置、Alpha通道和变换矩阵
            .....

            if (w.mLastHidden && !w.mDrawPending
                && !w.mCommitDrawPending
                && !w.mReadyToShow) {
                .....

                if (showSurfaceRobustlyLocked(w)) {
                    w.mHasDrawn = true;
                    w.mLastHidden = false;
                }

                .....
            }

            .....
        }

        .....
    }

    .....
}

.....
} catch (RuntimeException e) {
    .....
}

.....

Surface.closeTransaction();

.....

// 5. 检查是否需要再一次刷新系统UI

```

```

        if (needRelayout) {
            requestAnimationLocked(0);
        } else if (animating) {
            requestAnimationLocked(currentTime+(1000/60)-SystemClock.uptimeMillis());
        }
    }
    .....
}
.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。
WindowManagerService类的成员函数performLayoutAndPlaceSurfacesLockedInner按照以下步骤来显示窗口动画：

第一步是调用WindowManagerService类的成员函数performLayoutLockedInner来计算各个窗口的大小以及位置。只有知道了窗口的大小以及位置之后，我们才能应用一个变换矩阵，然后得到窗口下一步要显示的大小以及位置。这一步可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文。

第二步是推进各个Activity组件的切换动画，也就是计算每一个Activity组件下一步所要执行的动画。注意，只有那些正在参与切换操作的Activity组件才设置有动画，而只有设置有切换动画的Activity组件才需要计算它们下一步所要执行的动画。从前面第一部分的内容可以知道，如果一个Activity组件参与了切换操作，那么用来描述它的一个AppWindowToken对象的成员变量animation就会指向一个Animation对象，用来描述一个切换动画。

用来描述系统当前的Activity组件的AppWindowToken对象有一部分保存在WindowManagerService类的成员变量mAppTokens所描述的一个ArrayList中，另外一部分保存在WindowManagerService类的成员变量mExitingAppTokens所描述的一个ArrayList中。其中，保存在成员变量mAppTokens中的AppWindowToken对象有可能描述的就是正在打开的Activity组件，而保存在成员变量mExitingAppTokens中的AppWindowToken对象描述的就是正在关闭的Activity组件，这两类Activity组件都是参与了切换操作的，因此，我们需要计算它们下一步所要执行的动画，这是通过调用用来描述它们的AppWindowToken对象的成员函数stepAnimationLocked来实现的。注意，对于那些不是正在打开或者正在关闭的Activity组件，调用用来描述它们的AppWindowToken对象的成员函数stepAnimationLocked不会有任何效果，因为这些AppWindowToken对象的成员变量animation的值等于null。

在这一步中，如果有任何一个正在打开或者正在关闭的Activity组件的动画还没有执行完成，那么调用用来描述它们的AppWindowToken对象的成员函数stepAnimationLocked的返回值就会等于true，这时候变量tokensAnimating的值也会等于true，表示Activity组件的切换动画还在进行中。后面我们再详细分析AppWindowToken类的成员函数stepAnimationLocked的实现。

第三步是推进各个Activity组件的动画，也就是计算每一个窗口下一步所要执行的动画。注意，只有那些被设置有动画的窗口才需要计算它们下一步所要执行的动画。从前面第一部分的内容可以知道，如果一个窗口设置有动画，那么用来描述它的一个WindowState对象的成员变量mAnimation就会指向一个Animation对象，用来描述一个窗口动画。

用来描述系统当前的窗口的WindowState对象保存在WindowManagerService类的成员变量mWindows所描述的一个ArrayList中，只要调用这些WindowState对象的成员函数stepAnimationLocked，就可以计算它们所描述的窗口下一步所要执行的动画。同样，对于那些没有设置动画的窗口来说，调用用来描述它们的WindowState对象的成员函数stepAnimationLocked是不会有影响的，因为这些WindowState对象的成员变量mAnimation的值等于null。

注意，对于那些还没有创建绘图表面的窗口来说，即使它们设置有动画，在这一步里面，也是不需要计算它们下一步所要执行的动画的，这是因为一个没有绘图表面的窗口是无法对它应用动画的。

此外，在计算所有窗口下一步要执行的动画之前以及之后，会通知系统的窗口管理策略类窗口下一次要执行的动画就要开始计算了以及已经计算结束了，同时，每计算完成一个窗口下一次要执行的动画之后，也会通知系统的窗口管理策略类该窗口下一次要执行的动画已经计算完成了，这分别是通过调用WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数beginAnimationLw、finishAnimationLw和animatingWindowLw来实现的。

我们知道，在Android系统中，有两个特殊的系统窗口，即状态栏窗口和锁屏窗口。如果当前激活的窗口是一个全屏窗口的时候，那么系统的状态栏窗口就会被隐藏。同时，在显示锁屏窗口的时候，一般系统中的所有其它窗口都会被它挡在后面。但是有些被设置了FLAG_SHOW_WHEN_LOCKED标志位的窗口，能够显示在锁屏窗口的上面。还有些被设置了FLAG_DISMISS_KEYGUARD位的窗口，它们能够解散系统当前正在显示的锁屏窗口。这些全屏窗口、能够显示在锁屏窗口上面的窗口以及能够解散锁屏窗口的窗口，会影响到系统的状态栏窗口和锁屏窗口是否需要显示的问题，而这是需要由窗口管理策略类来决定的。因此，在计算所有窗口下一步要执行的动画的前后，以及计算每一个窗口下一次要执行的动画的过程中，都要通知一下窗口管理策略类，以便它可以决定是否要显示系统的状态栏窗口和锁屏窗口。

窗口管理策略类按照以下方法来决定是否需要显示系统的状态栏窗口和锁屏窗口：

1. 在计算所有窗口下一步要执行的动画之前，假设状态栏窗口是不可见的，而锁屏窗口是可见的、不可解散的；
2. 在计算一个窗口下一步要执行的动画的过程中，检查该窗口是否是可见的以及全屏显示的。如果该窗口是可见的，但是不是全屏的，那么就意味着状态栏窗口是可见的。同时，如果该窗口是可见的，并且也是全屏的，那么就会继续检查该窗口是否被设置了FLAG_SHOW_WHEN_LOCKED和FLAG_DISMISS_KEYGUARD标志位。如果这两个标志位分别被设置了，那么就意味着锁屏窗口是不可见的和需要解散的。
3. 在计算完成所有窗口下一步要执行的动画之后，根据第2步收集到的信息来决定是否要显示状态栏窗口以及锁屏窗口。如果系统当前存在状态栏窗口，并且第2步收集到的信息表明它是需要显示的，那么就会将它状态设置为可见的；另一方面，如果系统当前存在状态栏窗口，并且第2步收集到的信息表明系统当前有一个全屏的窗口，那么状态栏窗口无论如何都是需要隐藏的。如果系统当前存在锁屏窗口，并且第2步收集到的信息表明它是不需要显示的或者需要解散的，那么就会将它状态设置为不可见，否则的话，就会将它状态设置为可见。

在计算完成所有窗口下一步要执行的动画之后，如果状态栏窗口或者锁屏窗口的状态由可见变为不可见，或者由不可见变为可见，那么PhoneWindowManager类的成员函数finishAnimationLw的返回值就会不等于0，这意味着WindowManagerService服务需要重新布局系统中各个窗口，即重新计算各个窗口的大小、位置以及下一步要执行的动画等操作，因为状态栏窗口或者锁屏窗口的可见性变化引发窗口堆栈发生变化。这就是上述的第一步、第二步和第三步要放在一个while循环来执行的原因。但是，这个while循环不能无限地执行下去，否则的话，系统的UI就永远刷新不出来。这个while循环最多允许执行7次，并且各个窗口的大小和位置的重复计算次数最多为4次。

第四步是更新各个窗口的绘图表面。既然是更新各个窗口的绘图表面，那么就意味着只有具有绘图表面的窗口才需要更新。一个窗口如果具有绘图表面，那么用来描述它的一个WindowState对象的成员变量mSurface的值就不等于null。对于具有绘图表面的窗口，这一步主要是执行以下操作：

1. 调用用来描述它的WindowState对象的成员函数computeShownFrameLocked来计算它实际要显示的大小和位置，这是需要考虑它的原始大小和位置，以及它所被设置的变换矩阵。这个变换矩阵是通过组合它的动画来得到的，其中包括窗口本身所设置的及其所附加在的窗口所设置的动画，还有窗口所属的Activity组件所设置的切换动画。后面我们再分析WindowState类的成员函数computeShownFrameLocked的实现。

2. 将第1步计算得到的窗口实际要显示的大小以及位置设置到SurfaceFlinger服务中去。这一点可以参考前面Android窗口管理服务WindowManagerService计算窗口Z轴位置的过程分析一文

3. 如果窗口当前已经就准备就绪显示，即用来描述它的WindowState对象的成员函数isReadyForDisplay的返回值等于true，并且它所附加在的窗口是可见的，即用来描述它的WindowState对象的成员变量mAttachedHidden的值等于false，那么只要满足以下条件之一，就需要考虑通知SurfaceFlinger服务将该窗口的状态设置为可见的：

- A. 该窗口的Z轴位置发生了变化，即用来描述该窗口的WindowState对象的成员变量mLastLayer和mAnimLayer的值不相等；
- B. 该窗口的Alpha通道值发生了变化，即用来描述该窗口的WindowState对象的成员变量mLastAlpha和mShownAlpha的值不相等；
- C. 该窗口的变换矩阵发生了变化，即用来描述该窗口的WindowState对象的成员变量mLastDsDx、mLastDtDx、mLastDsDy和mLastDtDy的值不等于mDsDx、mDtDx、mDsDy和mDtDy的值；
- D. 该窗口在宽度和高度上所设置的缩放因子发生了变化，即用来描述该窗口的WindowState对象的成员变量mLastHScale和mLastVScale的值不等于mHScale和mVScale的值；
- E. 该窗口在上一次系统UI刷新时是不可见的，即用来描述该窗口的WindowState对象的成员变量mLastHidden的值等于true。

如果一个具有绘图表面的窗口满足上述条件，那么这一步会将该窗口的最新Z轴位置、Alpha通道值和变换矩阵设置到SurfaceFlinger服务中去，这一点可以参考前面Android窗口管理服务WindowManagerService计算窗口Z轴位置的过程分析一文。

此外，如果一个具有绘图表面的窗口满足的是上述的条件E，并且还满足以下条件：

- F. 它的UI已经绘制完成，即用来描述它的WindowState对象的成员变量mDrawPending和mCommitDrawPending值等于false；
- G. 它不是处于等待同一个窗口令牌的其它窗口的完成UI绘制的状态，即用来描述它的WindowState对象的成员变量mReadyToShow的值等于false。

那么这一步还会调用WindowManagerService类的成员函数showSurfaceRobustlyLocked来通知SurfaceFlinger服务将该窗口的状态设置为可见的。如果能够成功地通知SurfaceFlinger服务将该窗口的状态设置为可见，那么还会分别将用来描述该窗口的WindowState对象的成员变量mHasDrawn和mLastHidden的值设置为true和false，表示该窗口的UI已经绘制完成，并且状态已经设置为可见。

注意，上述四步对窗口的操作，例如设置窗口的大小、位置、Alpha通道值和变换矩阵等，都是在一个事务中执行的。这个事务从调用Surface类的静态成员函数openTransaction时开始，一直到调用Surface类的静态成员函数closeTransaction时结束。当事务结束的时候，前面所设置的窗口的状态才会批量地同步到SurfaceFlinger服务中去，这样就可以避免每修改一个窗口的一个状态，就触发SurfaceFlinger服务刷新一次系统的UI，造成屏幕闪烁。

第五步是检查是否需要再一次刷新系统UI。在三种情况下，是需要再一次刷新系统UI的。第一种情况是发现此时Activity组件的切换动画已经显示完成了；第二种情况是发现前面的操作会导致壁纸窗口的目标窗口被销毁了；第三种情况是发现此时还有窗口的动画未结束。

由于在Activity组件的切换过程中，很多操作都会被延迟执行，例如，窗口的Z轴位置的计算，因此，当出现第一种情况下，就需要重新执行这些延迟操作，主要就是重建窗口堆栈，以及计算每一个窗口的Z轴位置。在第二种情况下，由于当壁纸窗口的目标窗口被销毁之后，因此，就需要重新调整壁纸窗口在窗口堆栈中的位置。这两种情况都会导致变量needRelayout的值就等于true，表示需要马上重新刷新系统UI，这是通过以0为参数来调用WindowManagerService类的成员函数requestAnimationLocked来实现的。

在第三种情况中，变量animating的值会等于true，表示有窗口的动画还未结束，有可能是窗口本身的动画尚未结束，也有可能是Activity组件的切换动画尚未结束。在WindowManagerService服务中，窗口动画是以60帧每秒（fps）的速度来显示的，因此，这一步就会以约等于1000/60的参数来调用WindowManagerService类的成员函数requestAnimationLocked，表示要在1/60秒后再次刷新系统的UI，这样就可以把动画效果展现出来。

三. 窗口动画的推进过程

从前面第一部分的内容可以知道，窗口动画有可能是来自窗口本身所设置的动画，也有可能是来自于其宿主Activity组件所设置的切换动画，接下来我们就分别分析这两种类型的动画的推进过程。

1. 窗口动画的推进过程

从前面第二部分的内容可以知道，窗口动画的推进过程是由WindowState类的成员函数stepAnimationLocked的实现

的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {
        .....

        // Currently running animation.
        boolean mAnimating;
        boolean mLocalAnimating;
        Animation mAnimation;
        .....
        boolean mHasLocalTransformation;
        final Transformation mTransformation = new Transformation();
        .....

        // This must be called while inside a transaction. Returns true if
        // there is more animation to run.
        boolean stepAnimationLocked(long currentTime, int dw, int dh) {
            if (!mDisplayFrozen && mPolicy.isScreenOn()) {
                // We will run animations as long as the display isn't frozen.

                if (!mDrawPending && !mCommitDrawPending && mAnimation != null) {
                    .....
                    mHasLocalTransformation = true;
                    if (!mLocalAnimating) {
                        .....
                        mAnimation.initialize(mFrame.width(), mFrame.height(), dw, dh);
                        mAnimation.setStartTime(currentTime);
                        mLocalAnimating = true;
                        mAnimating = true;
                    }
                    mTransformation.clear();
                    final boolean more = mAnimation.getTransformation(
                        currentTime, mTransformation);
                    .....

                    if (more) {
                        // we're not done!
                        return true;
                    }
                    .....
                }
                .....
            }

            .....

            mAnimating = false;
            mLocalAnimating = false;
            mAnimation = null;
            .....

            mHasLocalTransformation = false;
            .....

            mTransformation.clear();
            .....

            return false;
        }

        .....
    }

    .....
}
```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。

WindowState类有几个成员变量是用来描述窗口的动画状态的，其中：

--mAnimating，表示窗口是否处于正在显示动画的过程中。

--mLocalAnimating，表示窗口的动画是否已经初始化过了。一个动画只有经过初始化之后，才能开始执行。

--mAnimation，表示窗口的动画对象。

--mHasLocalTransformation，表示窗口的动画是否是一个本地动画，即这个动画是否是来自窗口本身的。有时候一个窗口虽然正在显示动画，但是这个动画有可能是其宿主Activity组件的切换动画。在这种情况下，mHasLocalTransformation的值就会等于false。

--mTransformation，表示一个变换矩阵，是根据窗口动画的当前推进状态来计算得到的，用来改变窗口的大小以及位置。

窗口动画的推进只有以下四个条件均满足的情况下才会执行：

- (1). 屏幕没有被冻结，即WindowManagerService类的成员变量mDisplayFrozen的值等于false；
- (2). 屏幕是点亮的，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值等于true；
- (3). 窗口的UI已经绘制完成并且已经提交，即WindowState类的成员变量mDrawPending和mCommitDrawPending的值均等于false；
- (4). 窗口已经被设置过动画，即WindowState类的成员变量mAnimation的值不等于null。

一旦满足上述四个条件，那么WindowState类的成员函数stepAnimationLocked就会执行以下操作：

- (1). 将成员变量mHasLocalTransformation的值设置为true，表明窗口当前具有一个本地动画。
- (2). 检查成员变量mLocalAnimating的值是否等于false。如果等于false的话，那么就说明窗口的动画还没有经过初始化，这时候就会对该动画进行初始化，这是通过调用成员变量mAnimation所指向的一个Animation对象的成员函数initialize和setStartTime来实现的。窗口动画初始化完成之后，还需要将成员变量mLocalAnimating和mAnimating的值均设置为true，表明窗口动画已经初始化过了，并且窗口当前正在执行动画的过程中。
- (3). 将成员变量mTransformation所描述的变换矩阵的数据清空。
- (4). 调用成员变量mAnimation所指向的一个Animation对象的成员函数getTransformation来计算窗口动画下一步所对应的变换矩阵，并且将这个变换矩阵的数据保存在成员变量mTransformation。
- (5). 如果窗口的动画尚未结束显示，那么调用成员变量mAnimation所指向的一个Animation对象的成员函数getTransformation得到的返回值more就会等于true，这时候窗口动画的向前推进操作就完成了。
- (6). 如果窗口的动画已经结束显示，那么调用成员变量mAnimation所指向的一个Animation对象的成员函数getTransformation得到的返回值more就会等于false，这时候就需要执行一些清理工作。这些清理工作包括将成员变量mAnimating、mLocalAnimating和mHasLocalTransformation的值设置为false，以及将成员变量mAnimation的值设置为null，还有将成员变量mTransformation所描述的变换矩阵的数据清空。

最后，如果窗口的动画尚未结束显示，那么WindowState类的成员函数stepAnimationLocked会返回一个true值给调用者，否则的话，就会返回一个false值给调用者。

2. Activity组件切换动画的推进过程

从前面第二部分的内容可以知道，Activity组件切换动画的推进过程是由AppWindowToken类的成员函数stepAnimationLocked来实现的，如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    class AppWindowToken extends WindowToken {
        .....

        boolean animating;
        Animation animation;
        boolean hasTransformation;
        final Transformation transformation = new Transformation();
        .....

        // This must be called while inside a transaction.
        boolean stepAnimationLocked(long currentTime, int dw, int dh) {
            if (!mDisplayFrozen && mPolicy.isScreenOn()) {
                // We will run animations as long as the display isn't frozen.

                if (animation == sDummyAnimation) {
                    // This guy is going to animate, but not yet. For now count
                    // it as not animating for purposes of scheduling transactions;
                    // when it is really time to animate, this will be set to
                    // a real animation and the next call will execute normally.
                    return false;
                }
            }
        }
    }
}
```

```

    }

    if ((allDrawn || animating || startingDisplayed) && animation != null) {
        if (!animating) {
            .....
            animation.initialize(dw, dh, dw, dh);
            animation.setStartTime(currentTime);
            animating = true;
        }
        transformation.clear();
        final boolean more = animation.getTransformation(
            currentTime, transformation);
        .....
        if (more) {
            // we're done!
            hasTransformation = true;
            return true;
        }
        .....
        animation = null;
    }
}

.....

hasTransformation = false;
.....

animating = false;
.....

transformation.clear();
.....

return false;
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。AppWindowToken类有几个成员变量是用来描述Activity组件的切换动画状态的，其中：

- animating，表示Activity组件的切换动画是否已经初始化过了。
- animation，表示Activity组件的切换动画对象。
- mHasTransformation，表示Activity组件是否具有切换动画。
- transformation，表示一个变换矩阵，是根据Activity组件切换动画的当前推进状态来计算得到的，用来改变Activity组件窗口的大小以及位置。

Activity组件切换动画的推进只有以下五个条件均满足的情况下才会执行：

- (1). 屏幕没有被冻结，即WindowManagerService类的成员变量mDisplayFrozen的值等于false。
 - (2). 屏幕是点亮的，即WindowManagerService类的成员变量mPolicy所指向的一个PhoneWindowManager对象的成员函数isScreenOn的返回值等于true。
 - (3). Activity组件所设置的切换动画不是一个哑动画，即AppWindowToken类的成员变量animation与WindowManagerService类的静态成员变量sDummyAnimation不是指向同一个Animation对象。从前面Android窗口管理服务WindowManagerService切换Activity窗口（App Transition）的过程分析一文可以知道，一个Activity组件在进行切换之前，用来描述它的一个AppWindowToken对象的成员变量animation的值会被设置为sDummyAnimation，用来表示它准备要执行一个切换动画，但是这个切换动画还没有设置。
 - (4). Activity组件的所有窗口的UI都已经绘制完成，或者Activity组件的切换动画已经初始化过了，或者Activity组件的启动窗口已经结束显示，即AppWindowToken类的成员变量allDrawn、animating和startingDisplayed中的其中一个值等于true。
 - (5). Activity组件已经被设置过切换动画，即AppWindowToken类的成员变量animation的值不等于null。
- 一旦满足上述五个条件，那么AppWindowToken类的成员函数stepAnimationLocked就会执行以下操作：

(1). 检查成员变量animating的值是否等于false。如果等于false的话，那么就说明Activity组件的切换动画还没有经过初始化，这时候就会对该动画进行初始化，这是通过调用成员变量animation所指向的一个Animation对象的成员函数initialize和setStartTime来实现的。窗口动画初始化完成之后，还需要将成员变量animating的值设置为true，表明窗口动画已经初始化过了。

(2). 将成员变量transformation所描述的变换矩阵的数据清空。

(3). 调用成员变量animation所指向的一个Animation对象的成员函数getTransformation来计算Activity组件切换动画下一步所对应的变换矩阵，并且将这个变换矩阵的数据保存在成员变量transformation。

(4). 如果Activity组件切换动画尚未结束显示，那么调用成员变量animation所指向的一个Animation对象的成员函数getTransformation得到的返回值more就会等于true，这时候Activity组件切换动画的向前推进操作就完成了。

(6). 如果Activity组件切换动画已经结束显示，那么调用成员变量animation所指向的一个Animation对象的成员函数getTransformation得到的返回值more就会等于false，这时候就需要执行一些清理工作。这些清理工作包括将成员变量animating和hasTransformation的值设置为false，以及将成员变量animation的值设置为null，还有将成员变量transformation所描述的变换矩阵的数据清空。

最后，如果窗口的动画尚未结束显示，那么AppWindowToken类的成员函数stepAnimationLocked就会将成员变量hasTransformation的值设置为true，并且返回一个true值给调用者，否则的话，就会返回一个false值给调用者。

四. 窗口动画的合成过程

从前面第二部分的内容可以知道，WindowState类的成员函数computeShownFrameLocked负责合成窗口的动画，包括窗口本身所设置的进入（退出）动画、从被附加窗口传递过来的动画，以及宿主Activity组件传递过来的切换动画。窗口的这三个动画合成之后，就可以得到一个变换矩阵。将这个变换矩阵应用到窗口的原始大小和位置上去，就可以得到窗口经过动画变换后所得到的位置和大小。

WindowState类的成员函数computeShownFrameLocked的实现如下所示：

```
public class WindowManagerService extends IWindowManager.Stub
    implements Watchdog.Monitor {
    .....

    private final class WindowState implements WindowManagerPolicy.WindowState {
        .....

        // Actual frame shown on-screen (may be modified by animation)
        final Rect mShownFrame = new Rect();
        .....

        // Current transformation being applied.
        float mDsDx=1, mDtDx=0, mDsDy=0, mDtDy=1;
        .....

        // "Real" frame that the application sees.
        final Rect mFrame = new Rect();
        .....

        void computeShownFrameLocked() {
            final boolean selfTransformation = mHasLocalTransformation;
            Transformation attachedTransformation =
                (mAttachedWindow != null && mAttachedWindow.mHasLocalTransformation)
                ? mAttachedWindow.mTransformation : null;
            Transformation appTransformation =
                (mAppToken != null && mAppToken.hasTransformation)
                ? mAppToken.transformation : null;

            // Wallpapers are animated based on the "real" window they
            // are currently targeting.
            if (mAttrs.type == TYPE_WALLPAPER && mLowerWallpaperTarget == null
                && mWallpaperTarget != null) {
                if (mWallpaperTarget.mHasLocalTransformation &&
                    mWallpaperTarget.mAnimation != null &&
                    !mWallpaperTarget.mAnimation.getDetachWallpaper()) {
                    attachedTransformation = mWallpaperTarget.mTransformation;
                    .....
                }
                if (mWallpaperTarget.mAppToken != null &&
                    mWallpaperTarget.mAppToken.hasTransformation &&
                    mWallpaperTarget.mAppToken.animation != null &&
                    !mWallpaperTarget.mAppToken.animation.getDetachWallpaper()) {
                    appTransformation = mWallpaperTarget.mAppToken.transformation;
                    .....
                }
            }
        }
    }
}
```



```

    }

    if (selfTransformation != null || attachedTransformation != null) {
        // cache often used attributes locally
        final Rect frame = mFrame;
        final float tmpFloats[] = mTmpFloats;
        final Matrix tmpMatrix = mTmpMatrix;

        // Compute the desired transformation.
        tmpMatrix.setTranslate(0, 0);
        if (selfTransformation != null) {
            tmpMatrix.postConcat(selfTransformation.getMatrix());
        }
        tmpMatrix.postTranslate(frame.left, frame.top);
        if (attachedTransformation != null) {
            tmpMatrix.postConcat(attachedTransformation.getMatrix());
        }
        if (appTransformation != null) {
            tmpMatrix.postConcat(appTransformation.getMatrix());
        }

        // "convert" it into SurfaceFlinger's format
        // (a 2x2 matrix + an offset)
        // Here we must not transform the position of the surface
        // since it is already included in the transformation.
        //Slog.i(TAG, "Transform: " + matrix);

        tmpMatrix.getValues(tmpFloats);
        mDsDx = tmpFloats[Matrix.MSCALE_X];
        mDtDx = tmpFloats[Matrix.MSKEW_X];
        mDsDy = tmpFloats[Matrix.MSKEW_Y];
        mDtDy = tmpFloats[Matrix.MSCALE_Y];
        int x = (int)tmpFloats[Matrix.MTRANS_X] + mXOffset;
        int y = (int)tmpFloats[Matrix.MTRANS_Y] + mYOffset;
        int w = frame.width();
        int h = frame.height();
        mShownFrame.set(x, y, x+w, y+h);
        .....

        return;
    }

    mShownFrame.set(mFrame);
    if (mXOffset != 0 || mYOffset != 0) {
        mShownFrame.offset(mXOffset, mYOffset);
    }
    .....
    mDsDx = 1;
    mDtDx = 0;
    mDsDy = 0;
    mDtDy = 1;
}

.....
}

.....
}

```

这个函数定义在文件frameworks/base/services/java/com/android/server/WindowManagerService.java中。在分析WindowState类的成员函数computeShownFrameLocked的实现之前，我们先来看几个相关的成员变量：

--mFrame，用来描述窗口的实际位置以及大小。它们的计算过程可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文。

--mShownFrame，用来描述窗口当前所要显示的位置以及大小。

--mDsDx、mDtDx、mDsDy、mDtDy，用来描述窗口的变换矩阵（二维）。

WindowState类的成员函数computeShownFrameLocked的目标就根据窗口的实际位置、大小，以及窗口的动画，来计算

得到窗口当前所要显示的位置以及大小。

注意，在调用WindowState类的成员函数computeShownFrameLocked之前，窗口的实际位置和大小是已经计算好了的，并且窗口的动画也是已经向前推进好了的。窗口的实际位置和大小计算过程可以参考前面Android窗口管理服务WindowManagerService计算Activity窗口大小的过程分析一文，而窗口动画向前推进的过程可以参考前面第三部分的内容。

WindowState类的成员函数computeShownFrameLocked首先检查当前正在处理的窗口有多少个动画是需要合成的，即：

1. 检查成员变量mHasLocalTransformation的值是否等于true。如果等于true的话，那么就会将变量selfTransformation的值也设置为true，表示窗口本身设置有一个动画。这个动画要么是一个打开窗口类型的动画，要么是一个关闭窗口类型的动画。
2. 检查成员变量mAttachedWindow的值是否等于null。如果不等于null的话，那么就说明当前正在处理的窗口是附加在另外一个窗口之上。如果这个被附加的窗口也设置有动画，那么成员变量mAttachedWindow所指向的一个WindowState对象的成员变量mHasLocalTransformation的值就会等于true，这时候用来描述被附加窗口当前所要执行的动画的一个变换矩阵就由该WindowState对象的成员变量mTransformation来描述。这个变换矩阵最终会被保存在变量attachedTransformation中。
3. 检查成员变量mAppToken的值是否等于null。如果不等于null的话，那么就说明当前正在处理的窗口是一个Activity组件的窗口。如果这个Activity组件设置有切换动画，那么成员变量mAppToken所指向的一个AppWindowToken对象的成员变量hasTransformation的值就会等于true，这时候用来描述该Activity组件当前所要执行的切换动画的一个变换矩阵就由该AppWindowToken对象的成员变量transformation来描述。这个变换矩阵最终会被保存在变量appTransformation中。

WindowState类的成员函数computeShownFrameLocked接着检查当前正在处理的窗口是否是一个壁纸窗口，即检查成员变量mAttrs所指向的一个WindowManager.LayoutParams对象的成员变量type的值的TYPE_WALLPAPER位是否等于1。如果当前正在处理的窗口是一个壁纸窗口，并且它当前有且仅有一个目标窗口，即WindowManagerService类的成员变量mWallpaperTarget和mLowerWallpaperTarget的值分别不等于null和等于null。在这种情况下，需要对壁纸窗口的动画进行特殊处理，即：

1. 要把壁纸窗口所附加在的窗口的动画设置为壁纸窗口的目标窗口所附加在的窗口的动画，即将变量attachedTransformation指向用来描述壁纸窗口的目标窗口所附加在的窗口当前所要执行的动画的一个变换矩阵，前提是壁纸窗口的目标窗口设置有动画，并且这个目标窗口在结束动画过程后不会与壁纸窗口分离。
2. 要把壁纸窗口的切换动画设置为壁纸窗口的目标窗口的切换动画，即将变量appTransformation指向用来描述壁纸窗口的目标窗口当前所要执行的切换动画的一个变换矩阵，前提是壁纸窗口的目标窗口设置有切换动画，并且这个目标窗口结束动画过程后不会与壁纸窗口分离。

通过上面的处理之后，如果变量selfTransformation的值等于true，或者变量attachedTransformation和appTransformation的值不等于null，那么就说明当前正在处理的窗口有动画需要显示，因此，接下来就要将这些动画组合成一个总的变换矩阵。这个总的变换矩阵就是通过WindowState类的成员变量mTmpMatrix来描述的，它是通过下面的步骤来获得的：

1. 将该矩阵的偏移位置初始化为(0, 0)，这是通过调用变量tmpMatrix所描述的一个Matrix对象的成员函数setTranslate来实现的。
2. 如果变量selfTransformation的值等于true，那么就说明当前正在处理的窗口本身设置有动画。这个动画是通过WindowState类的成员变量mTransformation来描述的，调用这个成员变量所指向的一个Transformation对象的成员函数getMatrix就可以获得一个对应的变换矩阵。将这个变换矩阵与变量tmpMatrix所描述的变换矩阵相乘，就可以得到一个中间变换矩阵，这是通过调用变量tmpMatrix所指向的一个Matrix对象的成员函数postConcat来实现的。
3. 将上面第2步得到的中间变换矩阵的偏移位置设置为当前正在处理的窗口的实际位置，这是通过调用变量tmpMatrix所描述的一个Matrix对象的成员函数postTranslate来实现的，而当前正在处理的窗口的实际位置由变量frame所指向的一个Frame对象的成员变量left和top来描述。注意，变量frame和WindowState类的成员变量mFrame指向的是同一个Frame对象，因此它的成员变量left和top描述的是正在处理的窗口的实际位置。
4. 如果变量attachedTransformation的值不等于null，那么就说明当前正在处理的窗口所附加在的窗口设置有动画。这个动画就是通过变量attachedTransformation所指向的一个Transformation对象来描述的，调用这个Transformation对象的成员函数getMatrix就可以获得一个对应的变换矩阵。将这个变换矩阵与变量tmpMatrix所描述的变换矩阵相乘，就可以得到一个中间变换矩阵，这是通过调用变量tmpMatrix所指向的一个Matrix对象的成员函数postConcat来实现的。
5. 如果变量appTransformation的值不等于null，那么就说明当前正在处理的窗口的宿主Activity组件设置有切换动画。这个切换动画就是通过变量appTransformation所指向的一个Transformation对象来描述的，调用这个Transformation对象的成员函数getMatrix就可以获得一个对应的变换矩阵。将这个变换矩阵与变量tmpMatrix所描述的变换矩阵相乘，就可以得到一个中间变换矩阵，这是通过调用变量tmpMatrix所指向的一个Matrix对象的成员函数postConcat来实现的。

经过上面的5个步骤之后，最终得到的变换矩阵就保存在变量tmpMatrix中。由于这个变换矩阵是要设置到SurfaceFlinger服务中去的，因此就需要将这个变换矩阵转换为SurfaceFlinger服务所要求的格式。SurfaceFlinger服务所要求的变换矩阵的格式是由窗口在X轴和Y轴上的切变值以及缩放值来表示的，它们可以按照以下的步骤来获得：

1. 调用变量tmpMatrix所描述的一个Matrix对象的成员函数getValues来获得一个数组，并且保存在变量tmpFloats中。
2. 数组tmpFloats的第Matrix.MSKEW_X和Matrix.MSKEW_Y个位置的值就分别表示窗口在X轴和Y轴上的切变值，分别保存在WindowState类的成员变量mDxDx和mDxDy中。
3. 数组tmpFloats的第Matrix.MSCALE_X和Matrix.MSCALE_Y个位置的值就分别表示窗口在X轴和Y轴上的缩放值，分别保存在WindowState类的成员变量mDsDx和mDsDy中。

获得了当前正在处理的窗口的变换矩阵之后，接下来还要计算当前正在处理的窗口接下来要显示的位置以及大小。

前面得到的数组tmpFloats中的第Matrix.MTRANS_X和Matrix.MTRANS_Y个位置的值分别表示窗口在X轴和Y轴上的偏移值，它们实际就是窗口接下来要显示的位置。从前面Android窗口管理服务WindowManagerService对壁纸窗口（Wallpaper Window）的管理分析一文可以知道，如果当前正在处理的是一个壁纸窗口，那么WindowState类的成员变

量mXOffset和mYOffset描述的就是壁纸窗口相对其目标窗口的偏移值。对于普通的窗口来说，WindowState类的成员变量mXOffset和mYOffset的值等于0。因此，需要将保存在数组tmpFloats中的第Matrix.MTRANSX和Matrix.MTRANSY个位置的值与WindowState类的成员变量mXOffset和mYOffset的值分别相加，才能得到当前正在处理接下来要显示的位置。

由于前面获得的变换矩阵已经包含了当前正在处理的窗口的大小缩放因子，因此，我们就将当前正在处理的窗口的大小设置为它的实际大小即可。通过调用变量frame所指向的一个Frame对象的成员函数width和height可以获得当前正在处理的窗口的实际大小。

经过上面两步之后，当前正在处理的窗口接下来要显示的位置以及大小就计算完成了，其中，位置值保存在变量x和y中，而大小值保存在变量w和h，最终就可以将它们保存在WindowState类的成员变量mShownFrame所指向的一个Frame对象中。

如果当前正在处理的窗口没有动画可以显示，即变量selfTransformation的值等于false，并且变量attachedTransformation和appTransformation的值均等于null，那么WindowState类的成员函数computeShownFrameLocked的实现就简单了，它只要简单地将成员变量mFrame的内容设置到成员变量mShownFrame中，并且将成员变量mDxDx、mDxDy、mDxDy和mDxDy分别设置为1、0、0和1即可，表示当前正在处理的窗口既没有切变变换，也没有缩放变换。另外，如果WindowState类的成员变量mXOffset或者mYOffset的值不等于0，那么就需要将它们作来偏移值设置到成员变量mShownFrame所描述的一个Frame对象去，以便可以正确地计算出当前正在处理的窗口的位置。

至此，我们就分析完成窗口动画的显示过程了，整个WindowManagerService服务的分析也到此结束了。WindowManagerService服务可以说是整个Android应用程序框架层最为复杂的模块了，它与SurfaceFlinger服务一起为整个Android系统提供了UI服务，理解它对理解Android系统有着重要的意义。不过，要理解WindowManagerService服务的实现，是必须下些功夫的，同时也希望这个系列的文章能够帮助到大家。重新学习WindowManagerService服务请参考Android窗口管理服务WindowManagerService的简要介绍和学习计划一文。

————— 版权声明：本文为CSDN博主「罗升阳」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。 原文链接：
<https://blog.csdn.net/Luoshengyang/article/details/8611754>

第六章PKMS Android10.0 源码解读

PackageManagerService 简称 PKMS

第一节 前言 PKMS 是什么东西？



答：PackageManagerService（简称 PKMS），是 Android 系统中核心服务之一，负责应用程序的**安装，卸载，信息查询**，等工作。

第二节 PKMS 概述信息

Android系统启动时，会启动（应用程序管理服务PKMS），此服务负责扫描系统中特定的目录，寻找里面的APK格式的文件，并对这些文件进行解析，然后得到应用程序相关信息，最后完成应用程序的**安装**

PKMS在安装应用过程中，会全面解析应用程序的AndroidManifest.xml文件，来得到Activity, Service, BroadcastReceiver, ContextProvider 等信息，在结合PKMS服务就可以在OS中正常的使用应用程序了

在Android系统中，系统启动时由SystemService启动PKMS服务，启动该服务后会执行应用程序的安装过程，

接下来就会重点的介绍（SystemService启动PKMS服务的过程，讲解在Android系统中安装应用程序的过程）

简单来需知：PKMS 与 AMS 一样，也是Android系统核心服务之一，非常非常的重要，主要完成以下核心功能：

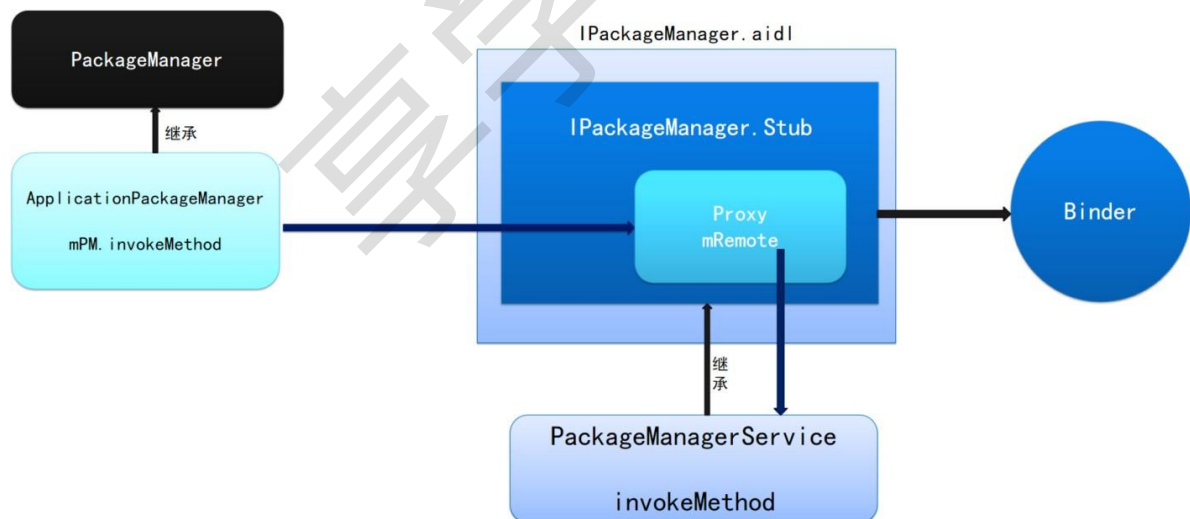
1. 解析AndroidManifest.xml清单文件，解析清单文件中的所有节点信息
2. 扫描.apk文件，安装系统应用，安装本地应用等
3. 管理本地应用，主要有，安装，卸载，应用信息查询 等

高亮课堂

同学们 我们分析的核心源码路径地址如下：

```
/frameworks/base/core/java/android/app/ApplicationPackageManager.java
/frameworks/base/services/java/com/android/server/SystemServer.java
/frameworks/base/services/core/java/com/android/server/pm/PackageManagerService.
java
/frameworks/base/services/core/java/com/android/server/pm/PackageDexOptimizer.ja
va
/frameworks/base/services/core/java/com/android/server/pm/Installer.java
/frameworks/base/services/core/java/com/android/server/pm/Settings.java
/frameworks/base/services/core/java/com/android/server/pm/permission/BasePermiss
ion.java
/frameworks/base/services/core/java/com/android/server/pm/PackageManagerService.
java
/frameworks/base/services/core/java/com/android/server/pm/permission/DefaultPerm
issionGrantPolicy.java
/frameworks/base/services/core/java/com/android/server/pm/permission/PermissionM
anagerService.java
/frameworks/base/core/java/android/content/pm/IPackageManager.aidl
/frameworks/base/core/java/android/content/pm/PackageManager.java
/frameworks/base/core/java/com/android/server/SystemConfig.java
```

第三节 PKMS角色位置



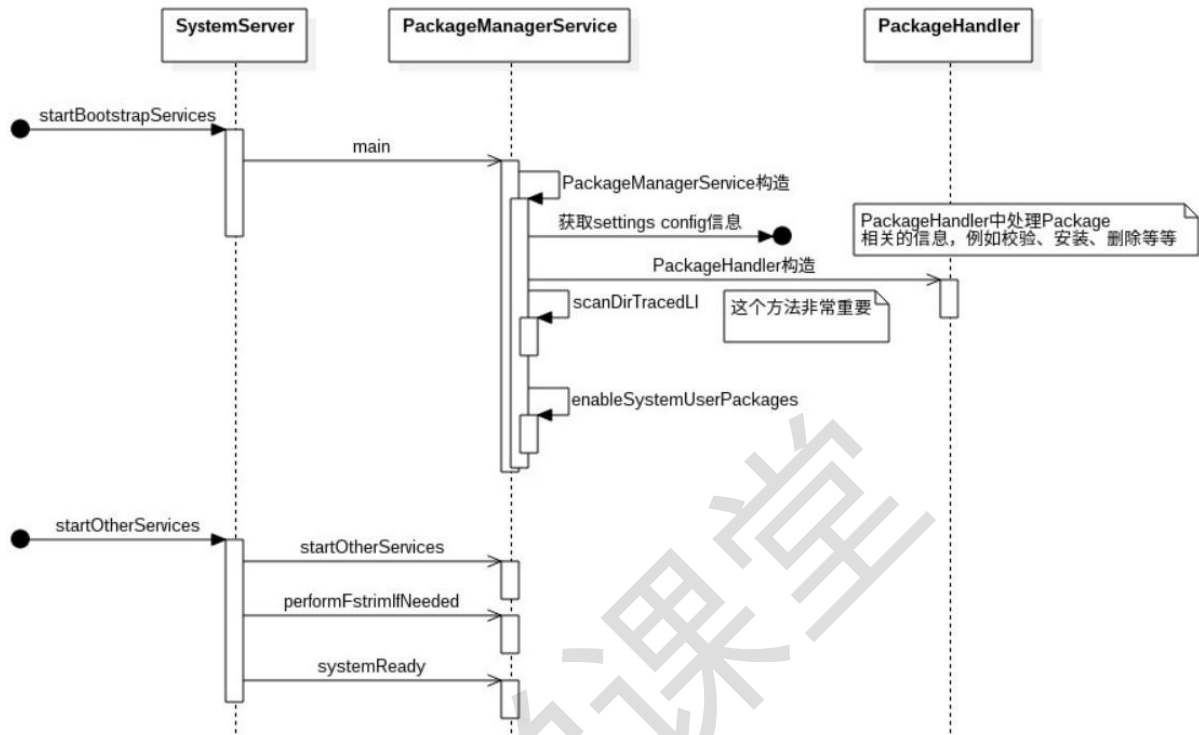
同学们注意：客户端可通过Context.getPackageManager()获得ApplicationPackageManager对象，而mPM指向的是Proxy代理，当调用到mPM.方法后，将会调用到IPackageManager的Proxy代理方法，然后通过Binder机制中的mRemote与服务端PackageManagerService通信并调用到PackageManagerService的方法；

自我总结：PKMS是属于Binder机制的服务端角色

接下来，我们就自己来手写一个简单的PKMS，同学们好不好

第四节 PKMS 启动过程分析

PKMS的过程图如下：



PKMS启动过程描述：

SystemServer启动PKMS：先是在SystemServer.startBootstrapServices()函数中启动PKMS服务，再调用startOtherServices()函数中对dex优化，磁盘管理功能，让PKMS进入systemReady状态。

七步走，文字不如画图：



第一步 到 第四步:

`startBootstrapServices()` 首先启动 `Installer` 服务, 也就是安装器, 随后判断当前的设备是否处于加密状态, 如果是则只是解析核心应用, 接着调用 `PackageManagerService` 的静态方法 `main` 来创建 `pms` 对象

第一步: 启动 `Installer` 服务

第二步: 获取设备是否加密(手机设置密码), 如果设备加密了, 则只解析“core”应用

第三步: 调用 `PKMS main` 方法初始化 `PackageManagerService`, 其中调用 `PackageManagerService()` 构造函数创建了 `PKMS` 对象

第四步: 如果设备没有加密, 操作它。管理 A/B OTA dexopting。

```
private void startBootstrapServices() {
    ...
    // 第一步: 启动Installer
    // 阻塞等待installd完成启动, 以便有机会创建具有适当权限的关键目录, 如/data/user。
    // 我们需要在初始化其他服务之前完成此任务。
    Installer installer = mSystemServiceManager.startService(Installer.class);
    mActivityManagerService.setInstaller(installer);

    ...
    // 第二步: 获取设备是否加密(手机设置密码), 如果设备加密了, 则只解析“core”应用, mOnlyCore
    // = true, 后面会频繁使用该变量进行条件判断
    String cryptState = VoldProperties.decrypt().orElse(""); if
    (ENCRYPTING_STATE.equals(cryptState)) {

        Slog.w(TAG, "Detected encryption in progress - only parsing core apps"); mOnlyCore =
        true;
    } else if (ENCRYPTED_STATE.equals(cryptState)) {
        Slog.w(TAG, "Device encrypted - only parsing core apps"); mOnlyCore
        = true;
    }

    // 第三步: 调用 main 方法 初始化 PackageManagerService
    mPackageManagerService = PackageManagerService.main(mSystemContext,
    installer,
        mFactoryTestMode != FactoryTest.FACTORY_TEST_OFF, mOnlyCore);

    // PKMS是否是第一次启动
    mFirstBoot = mPackageManagerService.isFirstBoot();

    // 第四步: 如果设备没有加密, 操作它。管理A/B OTA dexopting. if
    (!mOnlyCore) {
        boolean disableOtaDexopt = SystemProperties.getBoolean("config.disable_otadexopt",
            false);
        OtaDexoptService.main(mSystemContext, mPackageManagerService);
    }
    ...
}
```

第五步，第六步，第七步：

startOtherServices

博学课道

第五步：执行 `updatePackagesIfNeeded` ，完成dex优化；

第六步：执行 `performFstrimIfNeeded` ，完成磁盘维护；

第七步：调用`systemReady`，准备就绪。

```
private void startOtherServices() {
    ...
    if (!mOnlyCore) {
        ...
        // 第五步：如果设备没有加密，执行performDexOptUpgrade，完成dex优化；
        mPackageManagerService.updatePackagesIfNeeded();
    }
    ...
    // 第六步：最终执行performFstrim，完成磁盘维护
    mPackageManagerService.performFstrimIfNeeded();
    ...
    // 第七步：PKMS准备就绪
    mPackageManagerService.systemReady();
    ...
}
```

为什么分析第三步，需要给同学们说清楚：

第三步细节：PKMS.main() main

函数主要工作：

6. 检查Package编译相关系统属性
7. 调用PackageManagerService构造方法
8. 启用部分应用服务于多用户场景
9. 往ServiceManager中注册"package"和"package_native"。

```
public static PackageManagerService main(Context context, Installer installer,
    boolean factoryTest, boolean onlyCore) {
    // (1)检查Package编译相关系统属性
    PackageManagerServiceCompilerMapping.checkProperties();

    // (2)调用PackageManagerService构造方法，同学们可以参考【PKMS构造方法】
    PackageManagerService m = new PackageManagerService(context, installer,
        factoryTest, onlyCore);

    // (3)启用部分应用服务于多用户场景
    m.enableSystemUserPackages();

    // (4)往ServiceManager中注册"package"和"package_native"。
    ServiceManager.addService("package", m);
    final PackageManagerNative pmn = m.new PackageManagerNative();
    ServiceManager.addService("package_native", pmn);
    return m;
}
```

【PKMS构造方法】：

PKMS构造方法，整体描述图：



PKMS初始化时的核心部分为PKMS() 构造函数的内容，我们下面就来分析该流程：

PKMS的构造函数中由

两个重要的锁(mInstallLock、mPackages)：

mInstallLock：用来保护所有安装apk的访问权限，此操作通常涉及繁重的磁盘数据读写等操作，并且是单线程操作，故有时候会处理很慢，

此锁不会在已经持有mPackages锁的情况下火的，反之，在已经持有mInstallLock锁的情况下，立即获取mPackages是安全的

mPackages：用来解析内存中所有apk的package信息及相关状态。

和

5个阶段构成，下面会详细的来分析这些内容。

阶段1: BOOT_PROGRESS_PMS_START

阶段2: BOOT_PROGRESS_PMS_SYSTEM_SCAN_START

阶段3: BOOT_PROGRESS_PMS_DATA_SCAN_START

阶段4: BOOT_PROGRESS_PMS_SCAN_END

阶段5: BOOT_PROGRESS_PMS_READY

阶段1，阶段2，阶段3，阶段4，阶段5 的 EventLog：

```
public PackageManagerService(Context context, Installer installer,
    boolean factoryTest, boolean onlyCore) {
    ...
    // 阶段1: BOOT_PROGRESS_PMS_START
    EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_START,
        SystemClock.uptimeMillis());

    // 阶段2: BOOT_PROGRESS_PMS_SYSTEM_SCAN_START
    EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SYSTEM_SCAN_START,
        startTime);
    ...
}
```

```

        // 阶段3: BOOT_PROGRESS_PMS_DATA_SCAN_START
        if (!mOnlyCore) {

            EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_DATA_SCAN_START,
                                SystemClock.uptimeMillis());

        }
        ...
        // 阶段4: BOOT_PROGRESS_PMS_SCAN_END
        EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SCAN_END,
                            SystemClock.uptimeMillis());

        ...
        // 阶段5: BOOT_PROGRESS_PMS_READY
        EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_READY,
                            SystemClock.uptimeMillis());

    }

```

阶段1细节:

4. 构造 DisplayMetrics ，保存分辨率等相关信息；
5. 创建Installer对象，与installd交互；
6. 创建mPermissionManager对象，进行权限管理；
7. 构造Settings类，保存安装包信息，清除路径不存在的孤立应用，主要涉及/data/system/目录的 packages.xml, packages-backup.xml, packages.list, packages-stopped.xml, packages-stopped-backup.xml等文件。
8. 构造PackageDexOptimizer及DexManager类，处理dex优化；
9. 创建SystemConfig实例，获取系统配置信息，配置共享lib库；
10. 创建PackageManager的handler线程，循环处理外部安装相关消息。

```

public PackageManagerService(...)
{
    LockGuard.installLock(mPackages, LockGuard.INDEX_PACKAGES);
    EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_START,
                        SystemClock.uptimeMillis());
    mContext = context;

    mFactoryTest = factoryTest; // 一般为false, 即非工厂生产模式
    mOnlyCore = onlyCore; // 标记是否只加载核心服务

    // 【同学们注意】(1) 构造 DisplayMetrics ，保存分辨率等相关信息；
    mMetrics = new DisplayMetrics(); // 分辨率配置

    // 【同学们注意】(2) 创建Installer对象，与installd交互；
    mInstaller = installer; // 保存installer对象

    // 创建提供服务/数据的子组件。这里的顺序很重要，使用到了两个重要的同步锁：mInstallLock、
    mPackages
    synchronized (mInstallLock)
    {
        synchronized (mPackages)
        {
            // 公开系统组件使用的私有服务
            // 本地服务
            LocalServices.addService(

```

```

        PackageManagerInternal.class, new
PackageManagerInternalImpl());

        // 多用户管理服务
        sUserManager = new UserManagerService(context, this,
            new UserDataPreparer(mInstaller, mInstallLock, mContext, mOnlyCore),
mPackages);

        mComponentResolver = new ComponentResolver(sUserManager,
            LocalServices.getService(PackageManagerInternal.class), mPackages);

        // 【同学们注意】 (3) 创建mPermissionManager对象，进行权限管理；
        // 权限管理服务
        mPermissionManager = PermissionManagerService.create(context, mPackages
            /*externalLock*/);

        mDefaultPermissionPolicy = mPermissionManager.getDefaultPermissionGrantPolicy();

        // 创建Settings对象
        mSettings = new Settings(Environment.getDataDirectory(),
            mPermissionManager.getPermissionSettings(), mPackages);
    }
}

// 【同学们注意】 (4) 构造Settings类，保存安装包信息，清除路径不存在的孤立应用，主要涉及
// /data/system/目录的packages.xml, packages-backup.xml, packages.list,
// packages-stopped.xml, packages-stopped-backup.xml等文件。

// 添加system, phone, log, nfc, bluetooth, shell, se, networkstack 这8种
// shareUserId 到 mSettings;
mSettings.addSharedUserLPw("android.uid.system",
    Process.SYSTEM_UID,
        ApplicationInfo.FLAG_SYSTEM, ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);

mSettings.addSharedUserLPw("android.uid.phone", RADIO_UID, ApplicationInfo.FLAG_SYSTEM,
    ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
mSettings.addSharedUserLPw("android.uid.log",
    LOG_UID,
        ApplicationInfo.FLAG_SYSTEM, ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);

mSettings.addSharedUserLPw("android.uid.nfc", NFC_UID, ApplicationInfo.FLAG_SYSTEM,
    ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
mSettings.addSharedUserLPw("android.uid.bluetooth",
    BLUETOOTH_UID,
        ApplicationInfo.FLAG_SYSTEM, ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);

mSettings.addSharedUserLPw("android.uid.shell", SHELL_UID, ApplicationInfo.FLAG_SYSTEM,
    ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);
mSettings.addSharedUserLPw("android.uid.se", SE_UID,
        ApplicationInfo.FLAG_SYSTEM, ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);

mSettings.addSharedUserLPw("android.uid.networkstack", NETWORKSTACK_UID,
        ApplicationInfo.FLAG_SYSTEM,
    ApplicationInfo.PRIVATE_FLAG_PRIVILEGED);

...

```

```
// 【同学们注意】(5)构造PackageDexOptimizer及DexManager类，处理dex优化；  
// DexOpt优化  
mPackageDexOptimizer = new PackageDexOptimizer(installer, mInstallLock, context,
```

博学课道

```

        "*dexopt*");

mDexManager = new DexManager(mContext, this, mPackageDexOptimizer, installer,
mInstallLock);

// ART虚拟机管理服务
mArtManagerService = new ArtManagerService(mContext, this, installer, mInstallLock);

mMoveCallbacks = new MoveCallbacks(FgThread.get().getLooper());


mViewCompiler = new ViewCompiler(mInstallLock, mInstaller);
// 权限变化监听器
mOnPermissionChangeListeners = new
    OnPermissionChangeListeners(FgThread.get().getLooper());

mProtectedPackages = new ProtectedPackages(mContext);
mApexManager = new ApexManager(context);


// 获取默认分辨率
getDefaultDisplayMetrics(context, mMetrics);
// 【同学们注意】(6) 创建SystemConfig实例，获取系统配置信息，配置共享lib库；
// 拿到SystemConfig()的对象，其中会调用SystemConfig的readPermissions()完成权限的读取
SystemConfig systemConfig = SystemConfig.getInstance();
synchronized (mInstallLock) {
    // writer
    synchronized (mPackages) {
        // 【同学们注意】(7) 创建PackageManager的handler线程，循环处理外部安装相
        关消息。

        // 启动"PackageManager"线程，负责apk的安装、卸载
        mHandlerThread = new ServiceThread(TAG,
            Process.THREAD_PRIORITY_BACKGROUND, true/*allowIo*/);
        mHandlerThread.start();
        // 应用handler
        mHandler = new PackageHandler(mHandlerThread.getLooper());
        // 进程记录handler
        mProcessLoggingHandler = new ProcessLoggingHandler();
        // Watchdog监听ServiceThread是否超时：10分钟
        Watchdog.getInstance().addThread(mHandler, WATCHDOG_TIMEOUT);

        // Instant应用注册
        mInstantAppRegistry = new InstantAppRegistry(this);
        // 共享lib库配置
        ArrayMap<String, SystemConfig.SharedLibraryEntry> libConfig
            = systemConfig.getSharedLibraries(); final
            int builtInLibCount = libConfig.size(); for (int i = 0;
            i < builtInLibCount; i++) {

                String name = libConfig.keyAt(i); SystemConfig.SharedLibraryEntry entry=
libConfig.valueAt(i);

                addBuiltInSharedLibraryLocked(entry.filename, name);
            }
            ...
        // 读取安装相关SELinux策略

```

```
SELinuxMMAC.readInstallPolicy();
```

```
// 返回栈加载
```

```
FallbackCategoryProvider.loadFallbacks();
```

```
// 【【同学们注意：这段代码 等下下面会分析】】
```

```
//读取并解析/data/system下的XML文件
```

```
mFirstBoot = !mSettings.readLPw(sUserManager.getUsers(false));
```

高学课道

```

// 清理代码路径不存在的孤立软件包
final int packageSettingCount = mSettings.mPackages.size();
for (int i = packageSettingCount - 1; i >= 0; i--) {
    PackageSetting ps = mSettings.mPackages.valueAt(i);
    if (!isExternal(ps) && (ps.codePath == null ||
!ps.codePath.exists())
        && mSettings.getDisabledSystemPkgLPw(ps.name) !=
null) {
        mSettings.mPackages.removeAt(i);
        mSettings.enableSystemPackageLPw(ps.name);
    }
}

// 如果不是首次启动，也不是CORE应用，则拷贝预编译的DEX文件
if (!mOnlyCore && mFirstBoot)
{ requestCopyPreoptedFiles()
;
}
...
} // synchronized (mPackages)
}

```

同学们注意，此readLPw 是上面调下来的哦：

mSettings.readLPw

readLPw() 会扫描下面5个文件

18 <code>"/data/system/packages.xml"</code>	所有安装app信息
19 <code>"/data/system/packages-backup.xml"</code>	所有安装app信息之备份的信息记录
20 <code>"/data/system/packages.list"</code>	所有安装app信息
21 <code>"/data/system/packages-stopped.xml"</code>	所有强制停止app信息
22 <code>"/data/system/packages-stopped-backup.xml"</code>	所有强制停止app信息之备份的信息记录

个文件共分为三组，简单的作用描述如下：

packages.xml：PKMS 扫描完目标文件夹后会创建该文件。当系统进行程序安装、卸载和更新等操作时，均会更新该文件。该文件保存了系统中与 **package** 相关的一些信息。

packages.list：描述系统中存在的所有非系统自带的 APK 的信息。当这些程序有变动时，PKMS 就会更新该文件。

packages-stopped.xml：从系统自带的设置程序中进入应用程序页面，然后在选择强制停止（ForceStop）某个应用时，系统会将该应用的相关信息记录到此文件中。也就是该文件保存系统中被用户强制停止的 Package 的信息。

这些目录的指向，都在Settings中的构造函数完成，如下所示，得到目录后调用readLPw() 进行扫描

// 同学们：先看Settings构造函数

Settings(File dataDir, PermissionSettingspermission, Object

lock) {

mLock = lock;

mPermissions = permission;

mRuntimePermissionsPersistence = new RuntimePermissionPersistence(mLock);

mSystemDir = new File(dataDir, "system");

//mSystemDir指向目录"/data/system"


```
mSystemDir.mkdirs();           //创建 "/data/system"  
//设置权限FileUtils.setPermissions(mSystemDir.toString(),  
FileUtils.S_IRWXU|FileUtils.S_IRWXG
```

博学课道

```

        |FileUtils.S_IROTH|FileUtils.S_IXOTH,
        -1, -1);

// (1) 指向目录"/data/system/packages.xml"
mSettingsFilename = new File(mSystemDir, "packages.xml");
// (2) 指向目录"/data/system/packages-backup.xml"
mBackupSettingsFilename = new File(mSystemDir, "packages-backup.xml");
// (3) 指向目录"/data/system/packages.list"
mPackageListFilename = new File(mSystemDir, "packages.list");
FileUtils.setPermissions(mPackageListFilename, 0640, SYSTEM_UID,
PACKAGE_INFO_GID);
// (4) 指向目录"/data/system/packages-stopped.xml"
mStoppedPackagesFilename = new File(mSystemDir, "packages-stopped.xml");
// (5) 指向目录"/data/system/packages-stopped-backup.xml"
mBackupStoppedPackagesFilename = new File(mSystemDir, "packages-stopped-
backup.xml");
}

// 同学们：在看readLPw函数
[Settings.java]
boolean readLPw(@NonNull List<UserInfo> users)
{
    FileInputStream str = null;
    ...
    if (str == null) {
        str = new FileInputStream(mSettingsFilename);
    }
    // 解析 "/data/system/packages.xml" XmlPullParser
    parser = Xml.newPullParser();
    parser.setInput(str, StandardCharsets.UTF_8.name());

    int type;
    while ((type = parser.next()) != XmlPullParser.START_TAG && type !=
        XmlPullParser.END_DOCUMENT) {

        ;
    }
    int outerDepth = parser.getDepth();
    while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
        && (type != XmlPullParser.END_TAG || parser.getDepth() >
outerDepth)) {

        if (type == XmlPullParser.END_TAG || type == XmlPullParser.TEXT) { continue;
        }
        //根据XML的各个节点进行各种操作，例如读取权限、shared-user等
        String tagName = parser.getName();
        if (tagName.equals("package"))
            { readPackageLPw(parser);
        } else if (tagName.equals("permissions"))
            { mPermissions.readPermissions(parser);

```

```
    } else if (tagName.equals("permission-trees"))
        { mPermissions.readPermissionTrees(parser);

    } else if (tagName.equals("shared-user"))
        { readSharedUserLPw(parser);

    }...
}
str.close();
...
return true;
}
```

高学课道

阶段2细节:

- 3 从init.rc中获取环境变量BOOTCLASSPATH和SYSTEMSERVERCLASSPATH;
- 4 对于旧版本升级的情况, 将安装时获取权限变更为运行时申请权限;
- 5 扫描system/vendor/product/odm/oem等目录的priv-app、app、overlay包;
- 6 清除安装时临时文件以及其他不必要的信息。

```
public PackageManagerService(Context context, Installer installer, boolean
    factoryTest, boolean onlyCore) {

    synchronized (mInstallLock)
    { synchronized (mPackages) {

        // 记录扫描开始时间
        long startTime = SystemClock.uptimeMillis();

        EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SYSTEM_SCAN_START, startTime);

        // 【同学们注意】 (1)从init.rc中获取环境变量BOOTCLASSPATH和
        SYSTEMSERVERCLASSPATH;

        //获取环境变量, init.rc
        final String bootClassPath = System.getenv("BOOTCLASSPATH"); final
        String systemServerClassPath =
        System.getenv("SYSTEMSERVERCLASSPATH");
        ...
        // 获取system/framework目录
        File frameworkDir = new File(Environment.getRootDirectory(),
        "framework");

        // 获取内部版本
        final VersionInfo ver = mSettings.getInternalVersion();
        // 判断fingerprint是否有更新
        mIsUpgrade = !Build.FINGERPRINT.equals(ver.fingerprint);
        ...
        // 【同学们注意】 (2)对于旧版本升级的情况, 将安装时获取权限变更为运行时申请权限;

        // 对于Android M之前版本升级上来的情况, 需将系统应用程序权限从安装升级到运行时
        mPromoteSystemApps =
            mIsUpgrade && ver.sdkVersion <=
        Build.VERSION_CODES.LOLLIPOP_MR1;

        // 对于Android N之前版本升级上来的情况, 需像首次启动一样处理package
        mIsPreUpgrade = mIsUpgrade && ver.sdkVersion <
        Build.VERSION_CODES.N;

        mIsPreNMR1Upgrade = mIsUpgrade && ver.sdkVersion <
        Build.VERSION_CODES.N_MR1;

        mIsPreQUpgrade = mIsUpgrade && ver.sdkVersion <
```

```
Build.VERSION_CODES.Q;
```

// 在扫描之前保存预先存在的系统package的名称，不希望自动为新系统应用授予运行时权限

```
if (mPromoteSystemApps) { Iterator<PackageSetting>
    pkgSettingIter=
mSettings.mPackages.values().iterator();
    while (pkgSettingIter.hasNext()) { PackageSetting ps =
        pkgSettingIter.next(); if (isSystemApp(ps)) {
```

高学课道

```

        mExistingSystemPackages.add(ps.name);
    }
}

// 准备解析package的缓存
mCacheDir = preparePackageParserCache();
// 设置flag，而不在扫描安装时更改文件路径
int scanFlags = SCAN_BOOTING | SCAN_INITIAL;
...

// 【同学们注意：】(3)扫描system/vendor/product/odm/oem等目录的priv-app、
app、overlay包；
//扫描以下路径：

/vendor/overlay、/product/overlay、/product_services/overlay、/odm/overlay、/oem/ overlay、
/system/framework

/system/priv-app、/system/app、/vendor/priv-
app、/vendor/app、/odm/priv-app、/odm/app、/oem/app、/oem/priv-app、
/product/priv-app、/product/app、/product_services/priv- app、
/product_services/app、/product_services/priv-app
// [ PMSapk的安装]
scanDirTracedLI(new File(VENDOR_OVERLAY_DIR),...); scanDirTracedLI(new
File(PRODUCT_OVERLAY_DIR),...); scanDirTracedLI(new
File(PRODUCT_SERVICES_OVERLAY_DIR),...); scanDirTracedLI(new
File(ODM_OVERLAY_DIR),...); scanDirTracedLI(new File(OEM_OVERLAY_DIR),...);
...
final List<String> possiblyDeletedUpdatedSystemApps = new
ArrayList<>();

final List<String> stubSystemApps = new ArrayList<>();
// 删掉不存在的package
if (!mOnlyCore) {
    final Iterator<PackageParser.Package> pkgIterator =
mPackages.values().iterator();

    while (pkgIterator.hasNext()) {
        final PackageParser.Package pkg = pkgIterator.next(); if
(pkg.isStub) {
            stubSystemApps.add(pkg.packageName);
        }
    }

    final Iterator<PackageSetting> psit =
mSettings.mPackages.values().iterator();

    while (psit.hasNext()) { PackageSetting ps
        = psit.next();

        // 如果不是系统应用，则不被允许disable
        if ((ps.pkgFlags & ApplicationInfo.FLAG_SYSTEM) == 0) { continue;
        }

        // 如果应用被扫描，则不允许被擦除

```

```
final PackageParser.Package scannedPkg=  
mPackages.get(ps.name);  
  
if (scannedPkg != null) {  
    // 如果系统应用被扫描且存在disable应用列表中，则只能通过OTA升级添  
    if (mSettings.isDisabledSystemPackageLPr(ps.name)) {  
        ...  
        removePackageLI(scannedPkg, true);  
    }  
}
```

加

高学课道

```

        mExpectingBetter.put(ps.name, ps.codePath);
    }
    continue;
}
...
}
}
// 【同学们注意】(4)清除安装时临时文件以及其他不必要的信息。
// 删除临时文件
deleteTempPackageFiles();
// 删除没有关联应用的共享UID标识
mSettings.pruneSharedUsersLPw();
...
}
...
}
...
}

```

阶段3细节:

对于不仅仅解析核心应用的情况下，还处理data目录的应用信息，及时更新，祛除不必要的数据库。

```

public PackageManagerService(Context context, Installer installer, boolean
    factoryTest, boolean onlyCore) {
    synchronized (mInstallLock)
    { synchronized (mPackages) {
        ...
        if (!mOnlyCore) {
            EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_DATA_SCAN_START,
                SystemClock.uptimeMillis());
            scanDirTracedLI(sAppInstallDir, 0, scanFlags |
SCAN_REQUIRE_KNOWN, 0);
            ...
            // 移除通过OTA删除的更新系统应用程序的禁用package设置
            // 如果更新不再存在，则完全删除该应用。否则，撤销其系统权限
            for (int i = possiblyDeletedUpdatedSystemApps.size() - 1; i >=
0; --i) {
                final String packageName =
possiblyDeletedUpdatedSystemApps.get(i);
                final PackageParser.Package pkg =
mPackages.get(packageName);
                final String msg;

                mSettings.removeDisabledSystemPackageLPw(packageName);
                ...
            }
            // 确保期望在userdata分区上显示的所有系统应用程序实际显示
            // 如果从未出现过，需要回滚以恢复系统版本

```



```
for (int i = 0; i < mExpectingBetter.size(); i++) {  
    final String packageName = mExpectingBetter.keyAt(i); if  
    (!mPackages.containsKey(packageName)) {  
  
        final File scanFile = mExpectingBetter.valueAt(i);  
        ...  
        mSettings.enableSystemPackageLPw(packageName);  
    }  
}
```

博学课道

```

        try {
            //扫描APK
            scanPackageTracedLI(scanFile, reparseFlags,
rescanFlags, 0, null);
        } catch (PackageManagerException e) {
            Slog.e(TAG, "Failed to parse original system
package: "
                + e.getMessage());
        }
    }
}
// 解压缩并安装任何存根系统应用程序。必须最后执行此操作以确保替换或禁用所有存
根
installSystemStubPackages(stubSystemApps, scanFlags);
...
// 获取storage manager包名
mStorageManagerPackage = getStorageManagerPackageName();
// 解决受保护的action过滤器。只允许setup wizard（开机向导）为这些action
设置高优先级过滤器
mSetupWizardPackage = getSetupWizardPackageName();
...
// 更新客户端以确保持有正确的共享库路径
updateAllSharedLibrariesLocked(null,
Collections.unmodifiableMap(mPackages));
...
// 读取并更新要保留的package的上次使用时间
mPackageUsage.read(mPackages);
mCompilerStats.read();
    }
}
}
}
}

```

阶段4细节:

- 8 sdk版本变更，更新权限；
- 9 OTA升级后首次启动，清除不必要的缓存数据；
- 10 权限等默认项更新完后，清理相关数据；
- 11 更新package.xml

```

public PackageManagerService(Context context, Installer installer,
    boolean factoryTest, boolean onlyCore) {
    synchronized (mInstallLock)
    { synchronized (mPackages)
    {
        ...
        EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_SCAN_END,
            SystemClock.uptimeMillis());

        // 【同学们注意】(1) sdk版本变更，更新权限；
        // 如果自上次启动以来，平台SDK已改变，则需要重新授予应用程序权限以捕获出现的任何
        新权限

        final boolean sdkUpdated = (ver.sdkVersion != mSdkVersion);
    }
    }
}

```

```

        StorageManager.UUID_PRIVATE_INTERNAL, sdkUpdated,
mPackages.values(),

        mPermissionCallback);
...
// 如果这是第一次启动或来自Android M之前的版本的升级，并且它是正常启动，那需要在所有已定义的用户中初始化默认的首选应用程序
if (!onlyCore && (mPromoteSystemApps || mFirstBoot)) { for
    (UserInfo user : sUserManager.getUsers(true)) {

        mSettings.applyDefaultPreferredAppsLPw(user.id);
        primeDomainVerificationsLPw(user.id);

    }
}
// 在启动期间确实为系统用户准备存储，因为像SettingsProvider和SystemUI这样的核心系统应用程序无法等待用户启动
final int storageFlags;
if (StorageManager.isFileEncryptedNativeOrEmulated())
    { storageFlags = StorageManager.FLAG_STORAGE_DE;
} else {
    storageFlags = StorageManager.FLAG_STORAGE_DE |
StorageManager.FLAG_STORAGE_CE;
}
...
// 【同学们注意】(2) OTA升级后首次启动，清除不必要的缓存数据；
// 如果是在OTA之后首次启动，并且正常启动，那需要清除代码缓存目录，但不清除应用程序配置文件
if (mIsUpgrade && !onlyCore) {
    Slog.i(TAG, "Build fingerprint changed; clearing code caches"); for (int i =
0; i < mSettings.mPackages.size(); i++) {

        final PackageSetting ps = mSettings.mPackages.valueAt(i); if
        (Objects.equals(StorageManager.UUID_PRIVATE_INTERNAL,
ps.volumeUuid)) {

            // No apps are running this early, so no need to freeze
            clearAppDataLIF(ps.pkg, UserHandle.USER_ALL,

                FLAG_STORAGE_DE | FLAG_STORAGE_CE |

                FLAG_STORAGE_EXTERNAL

                    | Installer.FLAG_CLEAR_CODE_CACHE_ONLY);

        }
    }
    ver.fingerprint = Build.FINGERPRINT;
}

icons");

```

```

/
/
安      d-Q前的非系统应用程序在Launcher中隐藏他们的图标if (!onlyCore &&
装      mIsPreQUpgrade) {
    Slog.i(TAG, "Whitelisting all existing apps to hide their

A
n
d      int size =mSettings.mPackages.size(); for
r      (int i = 0; i < size; i++) {
o      final PackageSetting ps = mSettings.mPackages.valueAt(i); if
i      ((ps.pkgFlags & ApplicationInfo.FLAG_SYSTEM) != 0) {

        continue;
    }

ps.disableComponentLPw(PackageManager.APP_DETAILS_ACTIVITY_CLASS_NAME,
                        UserHandle.USER_SYSTEM);
    }
}

// 【同学们注意】 (3) 权限等默认项更新完后，清理相关数据;
```

```

        // 仅在权限或其它默认配置更新后清除
        mExistingSystemPackages.clear();
        mPromoteSystemApps = false;
        ...
        // 所有变更均在扫描过程中完成
        ver.databaseVersion = Settings.CURRENT_DATABASE_VERSION;

        // 【同学们注意】(4) 更新package.xml
        //降级去读取
        mSettings.writeLPr();
    }
}
}
}

```

阶段5细节:

GC回收内存 和一些细节而已

```

public PackageManagerService(Context context, Installer installer, boolean
    factoryTest, boolean onlyCore) {

    synchronized (mInstallLock)
    { synchronized (mPackages) {

        ...
        EventLog.writeEvent(EventLogTags.BOOT_PROGRESS_PMS_READY,
            SystemClock.uptimeMillis());
        ...
        //PermissionController 主持 缺陷许可证的授予和角色管理，所以这是核心系统的一个
        关键部分。
        mRequiredPermissionControllerPackage =
        getRequiredPermissionControllerLPr();
        ...
        updateInstantAppInstallerLocked(null);
        // 阅读并更新dex文件的用法
        // 在PM init结束时执行此操作，以便所有程序包都已协调其数据目录
        // 此时知道了包的代码路径，因此可以验证磁盘文件并构建内部缓存
        // 使用文件预计很小，因此与其他活动（例如包扫描）相比，加载和验证它应该花费相当小
        的时间

        final Map<Integer, List<PackageInfo>> userPackages = newHashMap<>
        ();

        for (int userId : userIds) {
            userPackages.put(userId, getInstalledPackages(/*flags*/ 0,
            userId).getList());
        }
        mDexManager.load(userPackages); if
        (mIsUpgrade) {

            MetricsLogger.histogram(null, "ota_package_manager_init_time", (int)
            (SystemClock.uptimeMillis() - startTime));
        }
    }
}

```

```
    }  
}  
  
...  
// 【同学们注意】GC回收内存  
// 打开应用之后，及时回收处理  
Runtime.getRuntime().gc();  
// 上面的初始扫描在持有mPackage锁的同时对installId进行了多次调用  
mInstaller.setWarnIfHeld(mPackages);
```

高学课道

```
} ...
```

Derry 2020年9月10日00:59:54 保存

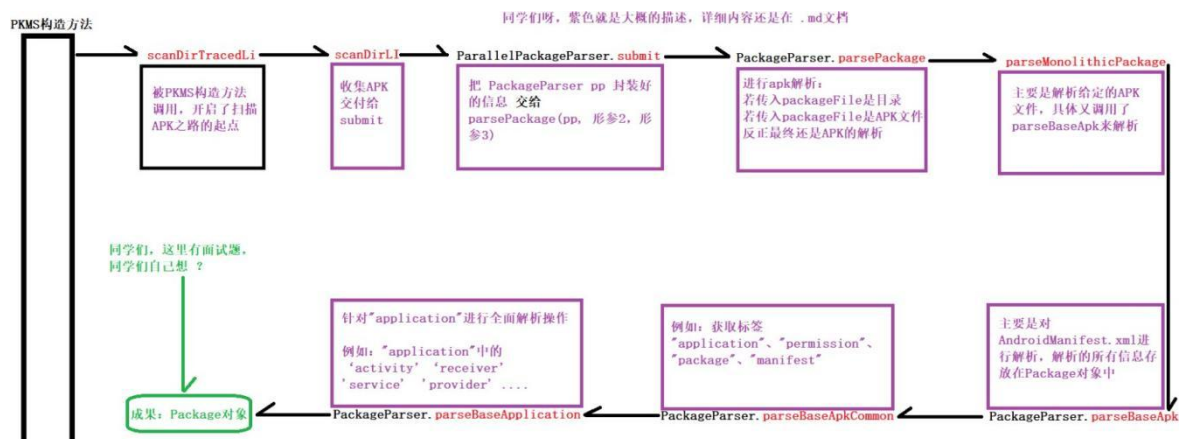
第五节 APK的扫描

同学们注意：PKMS的构造函数中调用了 `scanDirTracedLI` 方法来扫描某个目录的apk文件。

同学们注意：Android10.0 和其他低版本扫描的路径是不一样的：Android 10.0中，PKMS主要扫描以下路径的APK信息：

<code>/vendor/overlay</code>	系统的APP类别
<code>/product/overlay</code>	系统的APP类别
<code>/product_services/overlay</code>	系统的APP类别
<code>/odm/overlay</code>	系统的APP类别
<code>/oem/overlay</code>	系统的APP类别
<code>/system/framework</code>	系统的APP类别
<code>/system/priv-app</code>	系统的APP类别
<code>/system/app</code>	系统的APP类别
<code>/vendor/priv-app</code>	系统的APP类别
<code>/vendor/app</code>	系统的APP类别
<code>/odm/priv-app</code>	系统的APP类别
<code>/odm/app</code>	系统的APP类别
<code>/oem/app</code>	系统的APP类别
<code>/oem/priv-app</code>	系统的APP类别
<code>/product/priv-app</code>	系统的APP类别
<code>/product/app</code>	系统的APP类别
<code>/product_services/priv-app</code>	系统的APP类别
<code>/product_services/app</code>	系统的APP类别
<code>/product_services/priv-app</code>	系统的APP类别

APK的扫描，整体描述图：



PKMS. `scanDirTracedLi`: 首先加入了一些systtrace的日志追踪，然后调用`scanDirLI()`进行分析

```

private void scanDirTracedLI(File scanDir, final int parseFlags, int scanFlags,
long currentTime) {
    Trace.traceBegin(TRACE_TAG_PACKAGE_MANAGER, "scanDir [" +
scanDir.getAbsolutePath() + "]");
    try {
        // 【同学们注意】会调用此 scanDirLI函数
        scanDirLI(scanDir, parseFlags, scanFlags, currentTime);
    } finally {
        Trace.traceEnd(TRACE_TAG_PACKAGE_MANAGER);
    }
}

```

PKMS. **scanDirLI**: 使用了ParallelPackageParser的对象，ParallelPackageParser是一个队列，我们这里手机所有系统的apk，然后从这些队列里面取出apk，再调用**PackageParser** 解析进行解析：

```

private void scanDirLI(File scanDir, int parseFlags, int scanFlags, long currentTime) {

    final File[] files = scanDir.listFiles(); if
(ArrayUtils.isEmpty(files)) {

        Log.d(TAG, "No files in app dir " + scanDir); return;
    }

    if (DEBUG_PACKAGE_SCANNING) {
        Log.d(TAG, "Scanning app dir " + scanDir + " scanFlags=" + scanFlags
+ " flags=0x" + Integer.toHexString(parseFlags));
    }
    // parallelPackageParser是一个队列，收集系统 apk 文件，
    // 然后从这个队列里面一个个取出 apk ，调用 PackageParser 解析
    try (ParallelPackageParser parallelPackageParser = new
ParallelPackageParser(

        mSeparateProcesses, mOnlyCore, mMetrics, mCacheDir,
        mParallelPackageParserCallback)) {

        // Submit files for parsing in parallel int
        fileCount = 0;
        for (File file : files) {
            // 是Apk文件，或者是目录
            final boolean isPackage = (isApkFile(file) || file.isDirectory())
&& !PackageInstallerService.isStageName(file.getName());

            过滤掉非 apk 文件，如果不是则跳过继续扫描
            if (!isPackage) {
                // Ignore entries which are not packages
                continue;
            }
            // 把APK信息存入parallelPackageParser中的对象mQueue，PackageParser()函数赋给了队列
            中的pkg成员

            // 【同学们注意】 这里的 submit 函数 很重要，下面就会分析此函数
            parallelPackageParser.submit(file, parseFlags); fileCount++;

```



```
}
```

```
// Process results one by one  
for (; fileCount > 0; fileCount--) {  
    // 从parallelPackageParser中取出队列apk的信息
```

博学课道

```

ParallelPackageParser.ParseResult parseResult=
parallelPackageParser.take();

Throwable throwable = parseResult.throwable;
int errorCode = PackageManager.INSTALL_SUCCEEDED;

if (throwable == null) {
    // TODO(toddke): move lower in the scan chain
    // Static shared libraries have synthetic package names
    if (parseResult.pkg.applicationInfo.isStaticSharedLibrary())
        { renameStaticSharedLibraryPackage (parseResult.pkg);
    }
    try {
        //调用 scanPackageChildLI 方法扫描一个特定的 apk 文件
        // 该类的实例代表一个 APK 文件，所以它就是和 apk 文件对应的数据结构。
        scanPackageChildLI (parseResult.pkg, parseFlags, scanFlags, currentTime,
            null);
    } catch (PackageManagerException e)
        { errorCode = e.error;
        Slog.w(TAG, "Failed to scan " + parseResult.scanFile + ": " +
+ e.getMessage());
    }
    } else if (throwable instanceof
PackageParser.PackageParserException) {
        PackageParser.PackageParserException e=
(PackageParser.PackageParserException)
            throwable;
        errorCode = e.error;
        Slog.w(TAG, "Failed to parse " + parseResult.scanFile + ": " +
e.getMessage());
    } else {
        throw new IllegalStateException("Unexpected exception occurred
while parsing "

            + parseResult.scanFile, throwable);
    }

// Delete invalid userdata apps
//如果是非系统 apk 并且解析失败
if ((scanFlags & SCAN_AS_SYSTEM) == 0 &&
    errorCode != PackageManager.INSTALL_SUCCEEDED)
    { logCriticalInfo(Log.WARN,

        "Deleting invalid package at " +parseResult.scanFile);
        // 非系统 Package 扫描失败，删除文件
        removeCodePathLI (parseResult.scanFile);
    }
}
}

```

```
}  
}
```

ParallelPackageParser.**submit** :

把扫描路径中的APK等内容，放入队列mQueue，

并把parsePackage() pp 赋给ParseResult，用于后面的调用

```
public void submit(File scanFile, int parseFlags)  
{ mService.submit(() -> {
```

博学课道

```

        ParseResult pr = new ParseResult(); Trace.traceBegin(TRACE_TAG_PACKAGE_MANAGER,
        "parallel parsePackage [" +
scanFile + "]"); // 日志打印
        try {
            PackageParser pp = new PackageParser();
            pp.setSeparateProcesses(mSeparateProcesses);
            pp.setOnlyCoreApps(mOnlyCore);
            pp.setDisplayMetrics(mMetrics);
            pp.setCacheDir(mCacheDir);
            pp.setCallback(mPackageParserCallback); pr.scanFile
            = scanFile;

            // 并把parsePackage()与pp 赋值ParseResult, 用于后面的调用
            pr.pkg = parsePackage(pp, scanFile, parseFlags); // 【同学们注意】
        } catch (Throwable e) { pr.throwable =
            e;
        } finally {
            Trace.traceEnd(TRACE_TAG_PACKAGE_MANAGER);
        }
        try {
            // 把扫描路径中的APK等内容, 放入队列mQueue
            mQueue.put(pr);
        } catch (InterruptedException e)
        { Thread.currentThread().interrupt();

            // Propagate result to callers of take().
            // This is helpful to prevent main thread from getting stuck waiting
on

            // ParallelPackageParser to finish in case of interruption
            mInterruptedInThread = Thread.currentThread().getName();
        }
    });
}

```

通过 PackageParser.**parsePackage** 进行apk解析:

如果传入的packageFile是目录, 调用parseClusterPackage() 解析

如果传入的packageFile是APK文件, 调用parseMonolithicPackage() 解析

```
public Package parsePackage(File packageFile, int flags, boolean useCaches)
    throws PackageParserException {
    ...
    if (packageFile.isDirectory()) {
        //如果传入的packageFile是目录，调用parseClusterPackage()解析
        parsed = parseClusterPackage(packageFile, flags);
    } else {
        //如果是APK文件，就调用parseMonolithicPackage()解析
        parsed = parseMonolithicPackage(packageFile, flags); // 【同学们注意】下面
我们分析此函数
    }
    ...
    return parsed;
}
```

PackageParser.parseMonolithicPackage(), 它的作用是解析给定的APK文件, 将其作为单个单块包处理, 最终调用parseBaseApk() 进行解析

```
public Package parseMonolithicPackage(File apkFile, int flags) throws
PackageParserException {
    final PackageLite lite = parseMonolithicPackageLite(apkFile, flags);
    if (mOnlyCoreApps) {
        if (!lite.coreApp)
            { throw new
PackageParserException(INSTALL_PARSE_FAILED_MANIFEST_MALFORMED,
                        "Not a coreApp: " + apkFile);
        }
    }

    final SplitAssetLoader assetLoader = new DefaultSplitAssetLoader(lite,
flags);
    try {
        // 对核心应用解析 【同学们注意】 最终调用parseBaseApk()进行解析, 我们下面来分析
        final Package pkg = parseBaseApk(apkFile,
assetLoader.getBaseAssetManager(), flags);
        pkg.setCodePath(apkFile.getCanonicalPath());
        pkg.setUse32bitAbi(lite.use32bitAbi);
        return pkg;
    } catch (IOException e)
        { throw new
PackageParserException(INSTALL_PARSE_FAILED_UNEXPECTED_EXCEPTION,
                        "Failed to get path: " + apkFile, e);
    } finally {
        Ioutils.closeQuietly(assetLoader);
    }
}
```

PackageParser.parseBaseApk() 主要是对AndroidManifest.xml进行解析, 解析后所有的信息放在Package对象中

```
private Package parseBaseApk(File apkFile, AssetManager assets, int flags)
throws PackageParserException {
    final String apkPath = apkFile.getAbsolutePath();
    ...
    XmlResourceParser parser = null;
    ...
    final int cookie = assets.findCookieForPath(apkPath);
    if (cookie == 0) {
        throw new PackageParserException(INSTALL_PARSE_FAILED_BAD_MANIFEST,
                        "Failed adding asset path: " + apkPath);
    }
    // 获得一个 XML 资源解析对象, 该对象解析的是 APK 中的 AndroidManifest.xml 文件。
    parser = assets.openXmlResourceParser(cookie,
ANDROID_MANIFEST_FILENAME);
    final Resources res = new Resources(assets, mMetrics, null);

    final String[] outError = new String[1];

    // 再调用重载函数parseBaseApk()最终到parseBaseApkCommon(), 解析
    AndroidManifest.xml 后得到一个Package对象
```

```

// 【同学们注意】解析后所有的信息放在Package对象中
final Package pkg = parseBaseApk(apkPath, res, parser, flags, outError);
...
pkg.setVolumeUuid(volumeUuid);
pkg.setApplicationVolumeUuid(volumeUuid);
pkg.setBaseCodePath(apkPath);
pkg.setSigningDetails(SigningDetails.UNKNOWN);

return pkg;
...
}

```

parseBaseApk-----> **parseBaseApkCommon** , parseBaseApk省略了

PackageParser.**parseBaseApkCommon** 从AndroidManifest.xml中获取标签名, 解析标签中的各个item的内容, 存入Package对象中

例如: 获取标签 "application"、"permission"、"package"、"manifest" 同学们, 太多了, 省略了哈

```

private Package parseBaseApkCommon(Package pkg, Set<String> acceptedTags, Resources res,
    XmlResourceParser parser, int flags, String[] outError) throws
XmlPullParserException,
    IOException {
    TypedArray sa = res.obtainAttributes(parser,
        com.android.internal.R.styleable.AndroidManifest);

    //拿到AndroidManifest.xml 中的sharedUserId, 一般情况下有“android.uid.system”等信息
    String str =
        sa.getNonConfigurationString( com.android.internal.R.styleable.AndroidManifest_s
            haredUserId, 0);

    while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
        && (type != XmlPullParser.END_TAG || parser.getDepth() >
outerDepth)) {

        //从AndroidManifest.xml中获取标签名
        String tagName = parser.getName();
        //如果读到AndroidManifest.xml中的tag是“application”, 执行parseBaseApplication() 进 行解析
        if (tagName.equals(TAG_APPLICATION)) { if
            (foundApp) {

                ...

            }

            foundApp = true;

            // 解析“application”的信息, 赋值给pkg
            // 【同学们注意】这里解析到的是“application” <application 包含了 四大组件,

            if (!parseBaseApplication(pkg, res, parser, flags, outError)) { return null;
下面分析此操作
        }
    }
}

```

```
//如果标签是"permission"  
else if (tagName.equals(TAG_PERMISSION)) {  
    //进行"permission"的解析  
    if (!parsePermission(pkg, res, parser, outError)) { return  
        null;  
    }  
}
```

博学课道


```

        ....
    }
}
}

```

上面解析AndroidManifest.xml，会得到“**application**”、“**overlay**”、“**permission**”、“**uses-permission**”等信息

我们下面就针对“**application**”进行展开分析一下，进入 PackageParser.parseBaseApplication()函数

```

private boolean parseBaseApplication(Package owner, Resources res,
    XmlResourceParser parser, int flags, String[] outError)

while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
    && (type != XmlPullParser.END_TAG || parser.getDepth() >
innerDepth)) {
    // 获取“application”子标签的标签内容
    String tagName = parser.getName();
    // 如果标签是“activity”
    if (tagName.equals("activity")) { // 解析Activity的信息，把activity加入
Package对象
        Activity a = parseActivity(owner, res, parser, flags, outError, cachedArgs,
false,
            owner.baseHardwareAccelerated); if (a
== null) {
            mParseError = PackageManager.INSTALL_PARSE_FAILED_MANIFEST_MALFORMED;
            return false;
        }

        hasActivityOrder |= (a.order != 0);
        owner.activities.add(a);

    } else if (tagName.equals("receiver")) { // 如果标签是“receiver”，获取
receiver信息，加入Package对象
        Activity a = parseActivity(owner, res, parser, flags, outError, cachedArgs,
true, false);

        if (a == null)
            { mParseError =
PackageManager.INSTALL_PARSE_FAILED_MANIFEST_MALFORMED;
            return false;
        }

        hasReceiverOrder |= (a.order != 0);
        owner.receivers.add(a);

    } else if (tagName.equals("service")) { // 如果标签是“service”，获取service信息，加入

```

Package对象

```
Service s = parseService(owner, res, parser, flags, outError,
cachedArgs);

    if (s == null) { mParseError =
PackageManager.INSTALL_PARSE_FAILED_MANIFEST_MALFORMED;
        return false;
    }
```

博学课道

```

        hasServiceOrder != (s.order != 0);
        owner.services.add(s);

    }else if (tagName.equals("provider")) { // 如果标签是"provider", 获取
provider信息, 加入Package对象
        Provider p = parseProvider(owner, res, parser, flags, outError,
cachedArgs);
        if (p == null)
            { mParseError =
PackageManager.INSTALL_PARSE_FAILED_MANIFEST_MALFORMED;
            return false;
            }

        owner.providers.add(p);
    }
    ...
}
}

```

在 PackageParser 扫描完一个 APK 后, 此时系统已经根据该 APK 中 AndroidManifest.xml, 创建了一个完整的 Package 对象

APK的扫描, 自我总结:

第一步: 扫描APK, 解析AndroidManifest.xml文件, 得到清单文件各个标签内容

第二步: 解析清单文件到的信息由 Package 保存。从该类的成员变量可看出, 和 Android 四大组件相关的信息分别由 activities、receivers、providers、services 保存, 由于一个 APK 可声明多个组件, 因此activities 和 receivers等均声明为 ArrayList

第六节- APK的安装:

安装步骤一: 把Apk的信息通过IO流的形式写入到PackageInstaller.Session中

安装步骤二: 调用PackageInstaller.Session的commit方法, 把Apk的信息交给PKMS处理

安装步骤三: 进行Apk的Copy操作, 进行安装

安装的三步走, 整体描述图:

第一步

安装步骤一：把Apk的信息通过IO流的形式写入到PackageInstaller.Session中

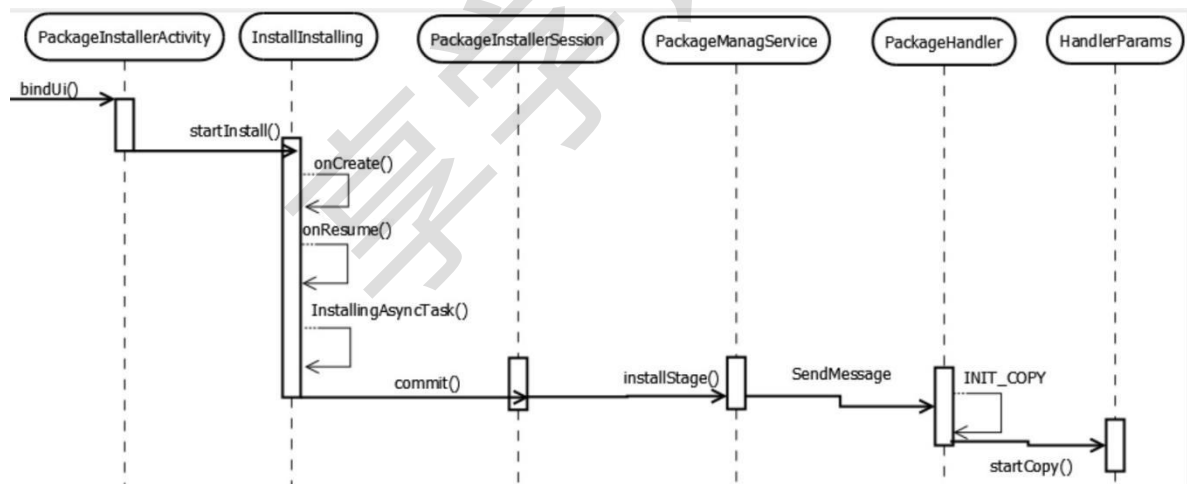
第二步

安装步骤二：调用PackageInstaller.Session的commit方法，
把Apk的信息交给PKMS处理

第三步

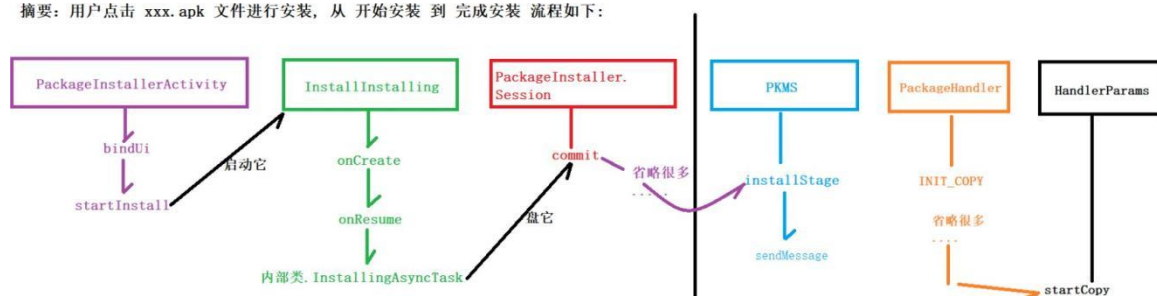
安装步骤三：进行Apk的Copy操作

用户点击 xxx.apk 文件进行安装，从 开始安装 到 完成安装 流程如下：



APK的安装，整体描述图：

摘要：用户点击 xxx.apk 文件进行安装，从 开始安装 到 完成安装 流程如下：



点击一个apk后，会弹出安装界面，点击确定按钮后，会进入PackageInstallerActivity 的 bindUi() 中的mAlert点击事件，弹出的安装界面底部显示的是一个dialog，主要由bindUi构成，上面有 ” 取消 “ 和 ” 安装 “ 两个按钮，点击安装后 调用startInstall() 进行安装：

```
private void bindUi()
{
    mAlert.setIcon(mAppSnippet.icon);
    mAlert.setTitle(mAppSnippet.label);
    mAlert.setView(R.layout.install_content_view);
    mAlert.setButton(DialogInterface.BUTTON_POSITIVE,
        getString(R.string.install),
        (ignored, ignored2) ->
        {
            if (mOk.isEnabled())
            {
                if (mSessionId != -1)
                {
                    mInstaller.setPermissionsResult(mSessionId, true);
                    finish();
                } else {
                    startInstall(); // 进行APK安装 【同学们注意】 下面开始分析
startInstall 做的事情
                }
            }
        }, null);
    mAlert.setButton(DialogInterface.BUTTON_NEGATIVE,
        getString(R.string.cancel),
        (ignored, ignored2) -> {
            // Cancel and finish
            setResult(RESULT_CANCELED);
            if (mSessionId != -1) {
                //如果mSessionId存在，执行setPermissionsResult()完成取消安装
                mInstaller.setPermissionsResult(mSessionId, false);
            }
            finish();
        }, null);
    setupAlert();

    mOk = mAlert.getButton(DialogInterface.BUTTON_POSITIVE);
    mOk.setEnabled(false);
}
```

startInstall方法组装了一个Intent，并跳转到 InstallInstalling 这个Activity，并关闭掉当前的PackageInstallerActivity。InstallInstalling主要用于向包管理器发送包的信息并处理包管理的回调：

```
private void startInstall() {
    // Start subactivity to actually install the application
    Intent newIntent = new Intent();
    newIntent.putExtra(PackageUtil.INTENT_ATTR_APPLICATION_INFO,
        mPkgInfo.applicationInfo);
    newIntent.setData(mPackageURI);
    // 设置Intent中的class为 InstallInstalling，用来进行Activity跳转
    // class InstallInstalling extends AlertActivity 【同学们注意】 下面会分析
InstallInstalling Activity
    newIntent.setClass(this, InstallInstalling.class);
    String installerPackageName =
        getIntent().getStringExtra( Intent.EXTRA_INSTALL
        // ...
}
```

```

    if (mOriginatingURI != null)
    { newIntent.putExtra(Intent.EXTRA_ORIGINATING_URI,
        mOriginatingURI);
    }
    if (mReferrerURI != null)
    { newIntent.putExtra(Intent.EXTRA_REFERRER,
        mReferrerURI);
    }
    if (mOriginatingUid != PackageManager.SessionParams.UID_UNKNOWN)
    { newIntent.putExtra(Intent.EXTRA_ORIGINATING_UID,
        mOriginatingUid);
    }
    if (installerPackageName != null)
    { newIntent.putExtra(Intent.EXTRA_INSTALLER_PACKAGE_NAME,
        installerPackageName);
    }
    if (getIntent().getBooleanExtra(Intent.EXTRA_RETURN_RESULT, false))
    { newIntent.putExtra(Intent.EXTRA_RETURN_RESULT, true);
    }
    newIntent.addFlags(Intent.FLAG_ACTIVITY_FORWARD_RESULT);
}

```

启动 `InstallInstalling`, 进入 `onCreate`, 重点是看 `onCreate` 函数中的六步:

```

protected void onCreate(@Nullable Bundle savedInstanceState)
{ super.onCreate(savedInstanceState);

    ApplicationInfo appInfo = getIntent()
        .getParcelableExtra(PackageUtil.INTENT_ATTR_APPLICATION_INFO); mPackageURI =
    getIntent().getData();

    if ("package".equals(mPackageURI.getScheme())) { try {

        getPackageManager().installExistingPackage(appInfo.packageName); launchSuccess();

    } catch (PackageManager.NameNotFoundException e)
        { launchFailure(PackageManager.INSTALL_FAILED_INTERNAL_ERROR, null);
        }
    } else {
        //根据mPackageURI创建一个对应的File
        final File sourceFile = new File(mPackageURI.getPath()); PackageUtil.AppSnippet
        as = PackageUtil.getAppSnippet(this, appInfo,
sourceFile);

        mAlert.setIcon(as.icon); mAlert.setTitle(as.label);
        mAlert.setView(R.layout.install_content_view);
        mAlert.setButton(DialogInterface.BUTTON_NEGATIVE,
getString(R.string.cancel),
            (ignored, ignored2) -> {
                if (mInstallingTask != null) { mInstallingTask.cancel(true);

```

```
}
```

```
if (mSessionId > 0) {
```

博学课道

```

getPackageManager().getPackageInstaller().abandonSession(mSessionId);
        mSessionId = 0;
    }

    setResult(RESULT_CANCELED);
    finish();

    }, null); setupAlert();

    requireViewById(R.id.installing).setVisibility(View.VISIBLE);

    // 第一步. 如果savedInstanceState不为null, 获取此前保存的mSessionId和
    mInstallId, 其中mSessionId是安装包的会话id, mInstallId是等待的安装事件id if
    (savedInstanceState != null) {
        mSessionId = savedInstanceState.getInt(SESSION_ID);
        mInstallId = savedInstanceState.getInt(INSTALL_ID);

        // Reregister for result; might instantly call back if result was delivered
while
        // activity was destroyed try
        {

            // 第二步. 根据mInstallId向InstallEventReceiver注册一个观察者,
            launchFinishBasedOnResult会接收到安装事件的回调,
            //无论安装成功或者失败都会关闭当前的Activity(InstallInstalling)。如果
            savedInstanceState为null, 代码的逻辑也是类似的
            InstallEventReceiver.addObserver(this, mInstallId,
                this::launchFinishBasedOnResult);

        } catch (EventResultPersister.OutOfIdsException e) {
            // Does not happen
        }
    } else {
        // 第三步. 创建SessionParams, 它用来代表安装会话的参数, 组装params
        PackageInstaller.SessionParams params = new
PackageInstaller.SessionParams(
            PackageInstaller.SessionParams.MODE_FULL_INSTALL);
        params.setInstallAsInstantApp(false);

        params.setReferrerUri(getIntent().getParcelableExtra(Intent.EXTRA_REFERRER));
        params.setOriginatingUri(getIntent()

            .getParcelableExtra(Intent.EXTRA_ORIGINATING_URI));

        params.setOriginatingUid(getIntent().getIntExtra(Intent.EXTRA_ORIGINATING_UID, UID_UNKNOWN));

        params.setInstallerPackageName(getIntent().getStringExtra( Intent.EXTRA_INSTALLER
            _PACKAGE_NAME));

        params.setInstallReason(PackageManager.INSTALL_REASON_USER);

```



```
// 第四步. 根据mPackageUri对包（APK）进行轻量级的解析，并将解析的参数赋值给
SessionParams

File file = new File(mPackageUri.getPath()); try {

    PackageParser.PackageLite pkg =
PackageParser.parsePackageLite(file, 0);

    params.setAppPackageName(pkg.packageName);
    params.setInstallLocation(pkg.installLocation);
    params.setSize(

        PackageHelper.calculateInstalledSize(pkg, false,
params.abiOverride));
```

```

    } catch (PackageParser.PackageParserException e) { Log.e(LOG_TAG, "Cannot
        parse package " + file + ". Assuming
defaults.");
        Log.e(LOG_TAG,
            "Cannot calculate installed size " + file + ". Tryonly
apk size.");
        params.setSize(file.length());
    } catch (IOException e)
    { Log.e(LOG_TAG,
        "Cannot calculate installed size " + file + ". Tryonly
apk size.");
        params.setSize(file.length());
    }

    try {
        // 第五步. 向InstallEventReceiver注册一个观察者返回一个新的mInstallId,
        //其中InstallEventReceiver继承自BroadcastReceiver, 用于接收安装事件并 回调给
        EventResultPersister。
        mInstallId = InstallEventReceiver
            .addObserver(this, EventResultPersister.GENERATE_NEW_ID,
                this::launchFinishBasedOnResult);
    } catch (EventResultPersister.OutOfIdsException e)
    { launchFailure(PackageManager.INSTALL_FAILED_INTERNAL_ERROR,
null);
    }

    try {
        // 第六步. PackageInstaller的createSession方法内部会通过
        IPackageInstaller与PackageInstallerService进行进程间通信,
        //最终调用的是PackageInstallerService的createSession方法来创建并返回
        mSessionId
        mSessionId =
getPackageManager().getPackageInstaller().createSession(params);
    } catch (IOException e)
    { launchFailure(PackageManager.INSTALL_FAILED_INTERNAL_ERROR,
null);
    }

    mCancelButton = mAlert.getButton(DialogInterface.BUTTON_NEGATIVE);

    mSessionCallback = new InstallSessionCallback();
}
}

```

同学们注意：以上**第六步**是重点 PackageInstaller 的 createSession() 内部会通过 IPackageInstaller 与 PackageInstallerService 进行进程间通信，最终调用的是 PackageInstallerService 的 createSession 方法来创建并返回 mSessionId

InstallInstalling.onResume 方法中，调用 onPostExecute() 方法，将 APK 的信息通过 IO 流的形式写入到 PackageInstaller.Session 中

```
protected void onResume()  
{ super.onResume();  
  // This is the first onResume in a single life of the activity
```

高学课道

```

        if (mInstallingTask == null) {
            PackageManager installer = getPackageManager().getPackageInstaller();
            // 获取 sessionInfo
            PackageManager.SessionInfo sessionInfo =
            installer.getSessionInfo(mSessionId);

            if (sessionInfo != null && !sessionInfo.isActive()) {
                // 【同学们注意】 最终执行onPostExecute() 下面来分析
                // 创建内部类InstallingAsyncTask的对象，调用execute()，最终进入
                onPostExecute()
                mInstallingTask = new InstallingAsyncTask();
                mInstallingTask.execute();
            } else {
                // we will receive a broadcast when the install is finished
                mCancelButton.setEnabled(false);
                setFinishOnTouchOutside(false);
            }
        }
    }
}

```

Installinstalling. InstallingAsyncTask: 关注 第一步 和 第二步

```

private final class InstallingAsyncTask extends AsyncTask<Void, Void,
PackageManager.Session> {
    volatile boolean isDone;

    // 第一步: doInBackground()会根据包(APK)的Uri, 将APK的信息通过IO流的形式写入到
    PackageManager.Session中
    @Override
    protected PackageManager.Session doInBackground(Void... params)
    { PackageManager.Session session;
        try {
            session =
            getPackageManager().getPackageInstaller().openSession(mSessionId);
        } catch (IOException e)
        { return null;
        }

        session.setStagingProgress(0);

        try {
            File file = new File(mPackageURI.getPath());

            try (InputStream in = new FileInputStream(file))
            { long sizeBytes = file.length();
                try (OutputStream out = session
                    .openWrite("PackageInstaller", 0, sizeBytes))
                { byte[] buffer = new byte[1024 * 1024];
                    while (true) {
                        int numRead = in.read(buffer);

                        if (numRead == -1)
                        { session.fsync(out
                            ); break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        if (isCancelled())
        {
            session.close();

            break;
        }
        //将APK的信息通过IO流的形式写入到PackageInstaller.Session中
        out.write(buffer, 0, numRead);
        if (sizeBytes > 0) {
            float fraction = ((float) numRead / (float)

                                session.addProgress(fraction);
sizeBytes);
        }
    }
}

return session;
} catch (IOException | SecurityException e) { Log.e(LOG_TAG,
    "Could not write package", e);

    session.close();

    return null;
} finally {
    synchronized (this)
    {
        isDone = true;
        notifyAll();
    }
}
}
}

```

// 第二步：最后在onPostExecute()中 调用PackageInstaller.Session的commit方法，进行安装

```

@Override
protected void onPostExecute(PackageInstaller.Session session) { if
    (session != null) {

        Intent broadcastIntent = new Intent(BROADCAST_ACTION);
        broadcastIntent.setFlags(Intent.FLAG_RECEIVER_FOREGROUND);
        broadcastIntent.setPackage(getPackageName());
        broadcastIntent.putExtra(EventResultPersister.EXTRA_ID, mInstallId);

        PendingIntent pendingIntent =
            PendingIntent.getBroadcast( InstallInstalling.this,
                mInstallId,

```

```
        broadcastIntent,  
        PendingIntent.FLAG_UPDATE_CURRENT);  
  
        // 【同学们注意】commit 下面会分析  
        // 调用PackageInstaller.Session的commit方法，进行安装  
        session.commit(pendingIntent.getIntentSender());  
        mCancelButton.setEnabled(false);  
        setFinishOnTouchOutside(false);  
    } else {  
  
        getPackageManager().getPackageInstaller().abandonSession(mSessionId);  
  
        if (!isCancelled()) { launchFailure(PackageManager.INSTALL_FAILED_INVALID_APK,  
            null);  
        }  
    }  
}
```

```

    }
}
}
}

```

PackageInstaller的commit()

```

[PackageInstaller.java] commit
public void commit(@NonNull IntentSender statusReceiver)
{ try {
    // mSession的类型为IPackageInstallerSession，这说明要通过
    IPackageInstallerSession来进行进程间的通信，最终会调用PackageInstallerSession的commit
    方法，这样代码逻辑就到了Java框架层的。
    // 调用IPackageInstallerSession的commit方法，跨进程调用到
    PackageInstallerSession.commit()
    mSession.commit(statusReceiver, false);
} catch (RemoteException e) {
    throw e.rethrowFromSystemServer();
}
}

```

PackageInstallerSession.commit()中

```

[PackageInstallerSession.java] commit()
public void commit(@NonNull IntentSender statusReceiver, boolean forTransfer)
{ if (mIsPerfLockAcquired && mPerfBoostInstall != null) {
    mPerfBoostInstall.perfLockRelease();
    mIsPerfLockAcquired = false;
}
...
// 调用markAsCommitted()
if (!markAsCommitted(statusReceiver, forTransfer))
{ return;
}
...
// 【同学们注意】向Handler发送一个类型为MSG_COMMIT的消息，下面会分析
mHandler.obtainMessage(MSG_COMMIT).sendToTarget();
}

```

MSG_COMMIT在handler中进行处理，进入handleCommit()

```

public boolean handleMessage(Message msg)
{ switch (msg.what) {
    case MSG_COMMIT:
        handleCommit();
        break;
}
}

private void handleCommit() {
    ...
}

```

```

List<PackageInstallerSession> childSessions = getChildSessions();

try {
    synchronized (mLock) {
        //最终调用installStage(), 进入PKMS
        commitNonStagedLocked(childSessions);
    }
} catch (PackageManagerException e) {
    final String completeMsg = ExceptionUtils.getCompleteMessage(e);
    Slog.e(TAG, "Commit of session " + sessionId + " failed: " +
completeMsg);
    destroyInternal();
    dispatchSessionFinished(e.error, completeMsg, null);
}
}

```

最终调用 `mPm.installStage()`，进入PKMS 【经过千辛万苦，终于要进入PKMS了】

```

private void commitNonStagedLocked(...) throws PackageManagerException
{ if (isMultiPackage()) {
    ...
    mPm.installStage(activeChildSessions); // 【同学们注意】跨越进程 进入
PKMS.installStage了
} else {
    mPm.installStage(committingSession);
}
}

```

PKMS.installStage

```

[PackageManagerService.java]
void installStage(ActiveInstallSession activeInstallSession)
{ if (DEBUG_INSTANT) {
    if ((activeInstallSession.getSessionParams().installFlags
        & PackageManager.INSTALL_INSTANT_APP) != 0) {
        Slog.d(TAG, "Ephemeral install of " +
activeInstallSession.getPackageName());
    }
}
// 第一步.创建了类型为INIT_COPY的消息
final Message msg = mHandler.obtainMessage(INIT_COPY);

// 第二步.创建InstallParams, 它对应于包的安装数据
final InstallParams params = new InstallParams(activeInstallSession);

params.setTraceMethod("installStage").setTraceCookie(System.identityHashCode(pa
rams));
msg.obj = params;

Trace.asyncTraceBegin	TRACE_TAG_PACKAGE_MANAGER, "installStage",
    System.identityHashCode(msg.obj));
Trace.asyncTraceBegin	TRACE_TAG_PACKAGE_MANAGER, "queueInstall",
    System.identityHashCode(msg.obj));

```



```

// 第三步. 将InstallParams通过消息发送出去。
mHandler.sendMessage(msg);
}

对INIT_COPY的消息的处理
[PackageManagerService.java]
void doHandleMessage(Message msg) { switch
    (msg.what) {

        case INIT_COPY: {
            HandlerParams params = (HandlerParams) msg.obj; if
            (params != null) {

                if (DEBUG_INSTALL) Slog.i(TAG, "init_copy: " + params);
                Trace.asyncTraceEnd(TRACE_TAG_PACKAGE_MANAGER, "queueInstall",
                    System.identityHashCode(params));
                Trace.traceBegin(TRACE_TAG_PACKAGE_MANAGER, "startCopy");

                // 【同学们注意】执行APK拷贝动作，这里会执行到 final void startCopy()
                params.startCopy();
                Trace.traceEnd(TRACE_TAG_PACKAGE_MANAGER);
            }
            break;
        }
    }
}

[PKMS.HandlerParams] final
void startCopy() {

    if (DEBUG_INSTALL) Slog.i(TAG, "startCopy " + mUser + ": " + this);
    handleStartCopy();

    handleReturnCode(); // 调用到下面 handleReturnCode
}

[PKMS.MultiPackageInstallParams] void
handleReturnCode() {

    if (mVerificationCompleted && mEnableRollbackCompleted) {
        .....
        if (mRet == PackageManager.INSTALL_SUCCEEDED) {
            mRet = mArgs.copyApk(); // 【同学们注意】下面会说到 copyApk
        }
        .....
    }
}
}

```

APK 拷贝 方法调用步骤如下：

PKMS

`copyApk()`

`doCopyApk()`

PackageManagerServiceUtils

`copyPacakge()`

`copyFile()`

// TODO 通过文件流的操作，把APK拷贝到/data/app等目录

博学课道

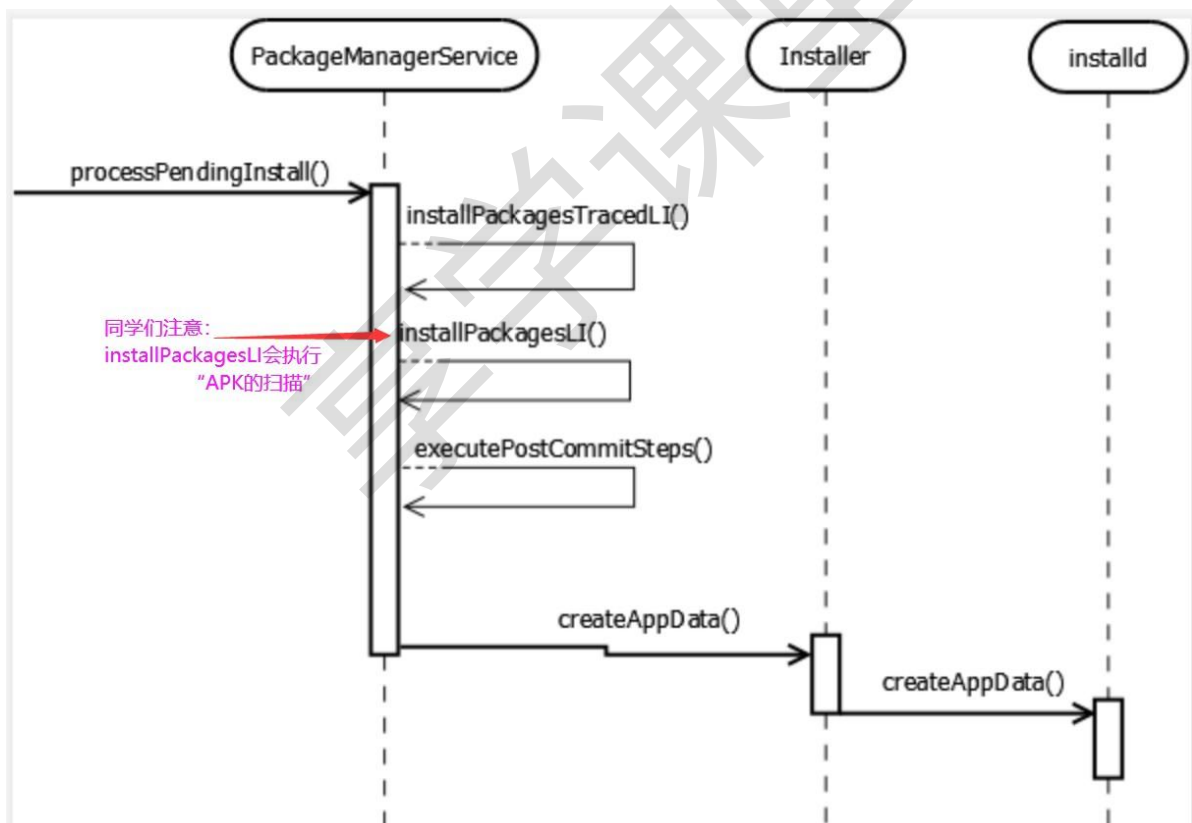
```

private static void copyFile(String sourcePath, File targetDir, String
targetName)
    throws ErrnoException, IOException {
    if (!FileUtils.isValidExtFilename(targetName)) {
        throw new IllegalArgumentException("Invalid filename: " + targetName);
    }
    slog.d(TAG, "Copying " + sourcePath + " to " + targetName);

    final File targetFile = new File(targetDir, targetName);
    final FileDescriptor targetFd = Os.open(targetFile.getAbsolutePath(),
        O_RDWR | O_CREAT, 0644);
    Os.chmod(targetFile.getAbsolutePath(), 0644);
    FileInputStream source = null;
    try {
        source = new FileInputStream(sourcePath);
        FileUtils.copy(source.getFD(), targetFd);
    } finally {
        IoUtils.closeQuietly(source);
    }
}

```

进入 Android 10.0 核心安装环节：



processPendingInstall:

```

private void processPendingInstall(final InstallArgs args, final int
currentStatus) {
    if (args.mMultiPackageInstallParams != null)
        { args.mMultiPackageInstallParams.tryProcessInstallRequest(args,
currentStatus);
    } else {
        //1. 设置安装参数
    }
}

```

```

        PackageInstalledInfo res = createPackageInstalledInfo(currentStatus);
        //2. 创建一个新线程，处理安装参数，进行安装
        processInstallRequestsAsync(
            res.returnCode == PackageManager.INSTALL_SUCCEEDED,
            Collections.singletonList(new InstallRequest(args, res)));
    }
}

private void processInstallRequestsAsync(boolean success,
    List<InstallRequest> installRequests) {

    mHandler.post(() -> { if
        (success) {

            for (InstallRequest request : installRequests) {
                //1. 如果之前安装失败，清除无用信息
                request.args.doPreInstall(request.installResult.returnCode);
            }

            synchronized (mInstallLock) {
                //2. installPackagesTracedLI 是安装过程的核心方法，然后调用
                installPackagesLI 进行安装。
                // 【同学们注意】下面会分析此函数 installPackagesTracedLI
                installPackagesTracedLI(installRequests);
            }

            for (InstallRequest request : installRequests) {
                //3. 如果之前安装失败，清除无用信息
                request.args.doPostInstall(
                    request.installResult.returnCode, request.installResult.uid);
            }
        }

        for (InstallRequest request : installRequests)
            { restoreAndPostInstall(request.args.user.getIdentifer(),
                request.installResult,
                    new PostInstallData(request.args, request.installResult,
                        null));
            }
        });
    }
}

```

installPackagesTracedLI

```
private void installPackagesLI(List<InstallRequest> requests) {  
    ...  
    // 环节一.Prepare 准备: 分析任何当前安装状态, 分析包并对其进行初始验证。  
    prepareResult = preparePackageLI(request.args, request.installResult);  
    ...  
    // 环节二.Scan 扫描: 考虑到prepare中收集的上下文, 询问已分析的包。  
    final List<ScanResult> scanResults = scanPackageTracedLI(  
        prepareResult.packageToScan,  
        prepareResult.parseFlags,  
        prepareResult.scanFlags, System.currentTimeMillis(),  
        request.args.user);  
    ...  
    // 环节三.Reconcile 调和: 在彼此的上下文和当前系统状态中验证扫描的包, 以确保安装成功。  
}
```

高亮课程

```

    ReconcileRequest reconcileRequest = new ReconcileRequest(preparedScans,
installArgs,
        installResults,
        prepareResults,
        mSharedLibraries,
        Collections.unmodifiableMap(mPackages), versionInfos,
        lastStaticSharedLibSettings);

    ...
    // 环节四.Commit 提交：提交所有扫描的包并更新系统状态。这是安装流中唯一可以修改系统状态的
    地方，必须在此阶段之前确定所有可预测的错误。
    commitPackagesLocked(commitRequest);

    ...
    // 环节五.完成APK的安装【同学们注意：下面会分析这个操作】
    executePostCommitSteps(commitRequest);
}

```

executePostCommitSteps 安装APK, 并为新的代码路径准备应用程序配置文件, 并再次检查是否需要dex优化

如果是直接安装新包，会为新的代码路径准备应用程序配置文件

如果是替换安装：其主要过程为更新设置，清除原有的某些APP数据，重新生成相关的app数据目录等步骤，同时要区分系统应用替换和非系统应用替换。而安装新包：则直接更新设置，生成APP数据即可。

```

[PackageManagerService.java] executePostCommitSteps()
private void executePostCommitSteps(CommitRequest commitRequest) { for
    (ReconciledPackage reconciledPkg :
commitRequest.reconciledPackages.values()) {
    ...
    //1) 进行安装
    prepareAppDataAfterInstallLIF(pkg);

    //2) 如果需要替换安装，则需要清楚原有的APP数据
    if (reconciledPkg.prepareResult.clearCodeCache) { clearAppDataLIF(pkg,
        UserHandle.USER_ALL, FLAG_STORAGE_DE
FLAG_STORAGE_CE
        | FLAG_STORAGE_EXTERNAL |
Installer.FLAG_CLEAR_CODE_CACHE_ONLY);
    }

    //3) 为新的代码路径准备应用程序配置文件。这需要在调用dexopt之前完成，以便任何安装时配 置
    文件都可以用于优化。
    mArtManagerService.prepareAppProfiles( pkg,

        resolveUserIds(reconciledPkg.installArgs.user.getIdentifier()),
        /* updateReferenceProfileContent= */ true);

    final boolean performDexopt =
        (!instantApp || Global.getInt(mContext.getContentResolver(),

```

```
Global.INSTANT_APP_DEXOPT_ENABLED, 0) != 0)

    && ((pkg.applicationInfo.flags &
ApplicationInfo.FLAG_DEBUGGABLE) == 0);

    if (performDexopt) {
```

博学课道

```

...
//4)执行dex优化
mPackageDexOptimizer.performDexOpt(pkg,
    null /* instructionSets */,
    getOrCreateCompilerPackageStats(pkg),
    mDexManager.getPackageUseInfoOrDefault(packageName),
    dexoptOptions);
}

BackgroundDexOptService.notifyPackageChanged(packageName);
}
}

```

prepareAppDataAfterInstallLIF:

通过一系列的调用，最终会调用到[Installer.java] createAppData()进行安装，交给installed进程进行APK的安装
调用栈如下：

```

prepareAppDataAfterInstallLIF()
|
prepareAppDataLIF()
|
prepareAppDataLeafLIF()
|
[Installer.java]
    createAppData()

```

```

private void prepareAppDataAfterInstallLIF(PackageParser.Package pkg) {
    ...
    for (UserInfo user : um.getUsers()) {
        ...
        if (ps.getInstalled(user.id)) {
            // TODO: when user data is locked, mark that we're still dirty
            prepareAppDataLIF(pkg, user.id, flags);
        }
    }
}

private void prepareAppDataLIF(PackageParser.Package pkg, int userId, int flags)
{
    if (pkg == null) {
        Slog.wtf(TAG, "Package was null!", new Throwable());
        return;
    }
    prepareAppDataLeafLIF(pkg, userId, flags);
    final int childCount = (pkg.childPackages != null) ?
pkg.childPackages.size() : 0;
    for (int i = 0; i < childCount; i++)
    { prepareAppDataLeafLIF(pkg.childPackages.get(i), userId,
        flags);
    }
}

private void prepareAppDataLeafLIF(PackageParser.Package pkg, int userId, int

```



```

...
try {
    // 调用installd守护进程的入口
    ceDataInode = mInstaller.createAppData(volumeUuid, packageName, userId,
flags,
        appId, seInfo, app.targetSdkVersion);
} catch (InstallerException e)
{ if (app.isSystemApp()) {
    destroyAppDataLeafLIF(pkg, userId, flags);
    try {
        ceDataInode = mInstaller.createAppData(volumeUuid, packageName,
userId, flags,
            appId, seInfo, app.targetSdkVersion);
    } catch (InstallerException e2) {
        ...
    }
}
}
}
}
}

```

Installer.createAppData 收尾工作，安装完成后，更新设置，更新安装锁等：

```

[Installer.java]
public long createAppData(String uuid, String packageName, int userId, int
flags, int appId,
    String seInfo, int targetSdkVersion) throws InstallerException
{ if (!checkBeforeRemote()) return -1;
try {
    // mInstalld 为IInstalld的对象，即通过Binder调用到 进程installd，最终调用
installd的createAppData()
    // 【同学们注意】mInstalld是一个aidl文件，通过此aidl文件调用到 Binder机制的服务
端，服务端哪里要操控底层....
    return mInstalld.createAppData(uuid, packageName, userId, flags, appId,
seInfo,
        targetSdkVersion);
} catch (Exception e) {
    throw InstallerException.from(e);
}
}
}

```

总结：安装的原理：

安装其实就是把apk文件copy到了对应的目录：

1. data/app/包名 —— 安装时把 apk文件复制到此目录，—— 可
以将文件取出并安装，和我们本身的apk 是一样的。



/data/data/packageName/(test.apk)

2. data/data/包名 —— 开辟存放应用程序的文件数据的文件夹
包括我们应用的 so库，缓存文件 等等。



/data/data/packageName/(db, cache)

3. 将apk中的dex文件安装到data/dalvik-cache目录下(dex文件是
dealvik虚拟机的可执行文件，其大小约为原始apk文件大小的四分之一)



/data/dalvik-cache/(profiles, x86)

第七节 PMS之权限扫描

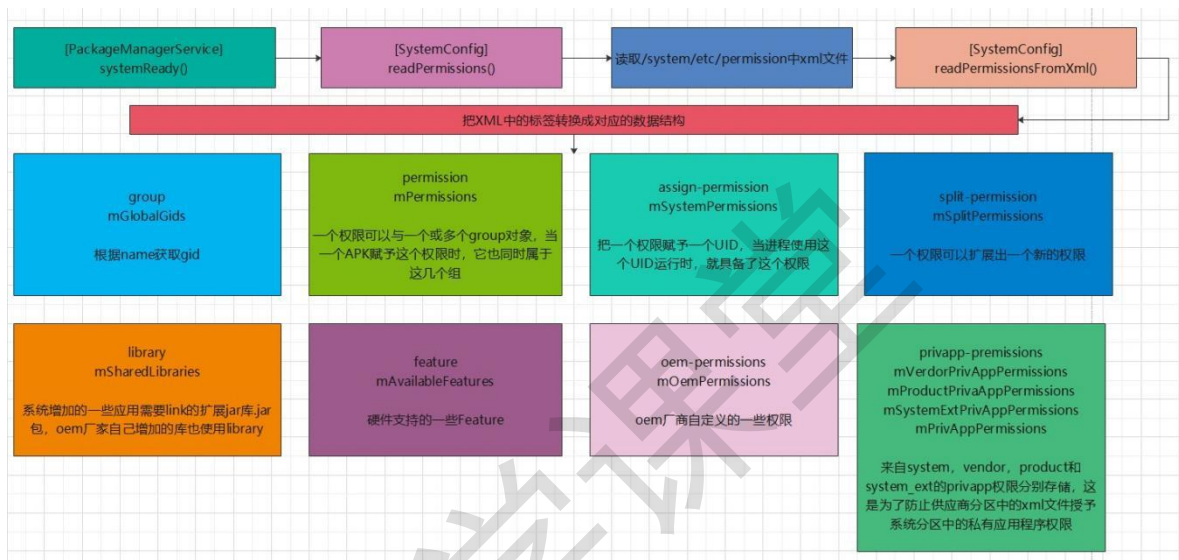
此“PMS之权限扫描”学习的目标是：PackageManagerService中执行systemReady()后，需求对/system/etc/permissions中的各种xml进行扫描，进行相应的权限存储，让以后可以使用，这就是本次“PMS只权限扫描”学习的目的

权限扫描：

PackageManagerService执行systemReady()时，通过SystemConfig的readPermissionsFromXml()来扫描读取/system/etc/permissions中的xml文件，包括platform.xml和系统支持的各种硬件模块的

feature主要工作：

整体图：



SystemConfig 的 readPermissions函数：

此函数目的：（扫描/system/etc/permissions中文件，调用readPermissionsFromXml()进行解析，存入SsystemConfig相应的成员数组变量中）

```
void readPermissions(File libraryDir, int permissionFlag) {
    ...
    // Iterate over the files in the directory and scan .xml files
    File platformFile = null;
    for (File f : libraryDir.listFiles())
    { if (!f.isFile()) {
        continue;
    }

    // 最后读取platform.xml
    if (f.getPath().endsWith("etc/permissions/platform.xml"))
    { platformFile = f;
        continue;
    }
    ...
    readPermissionsFromXml(f, permissionFlag);
}
```

```

        // Read platform permissions last so it will take precedence
        if (platformFile != null) {
            readPermissionsFromXml(platformFile, permissionFlag);
        }
    }
}

```

解析xml的标签节点，存入mGlobalGids、mPermissions、mSystemPermissions等成员变量中，供其他进行调用

```

private void readPermissionsFromXml(File permFile, int permissionFlag) {
    FileReader
    permReader = null;

    permReader = new FileReader(permFile);
    ...
    XmlPullParser parser = Xml.newPullParser();
    parser.setInput(permReader);

    while (true) {
        ...
        String name = parser.getName();
        switch (name) {
            //解析 group 标签，前面介绍的 XML 文件中没有单独使用该标签的地方
            case "group": {
                String gidStr = parser.getAttributeValue(null, "gid");
                if (gidStr != null) {
                    int gid = android.os.Process.getGidForName(gidStr);
                    //转换 XML 中的 gid字符串为整型，并保存到 mGlobalGids 中
                    mGlobalGids = appendInt(mGlobalGids, gid);
                } else {
                    Slog.w(TAG, "<" + name + "> without gid in " + permFile + " at "
                        + parser.getPositionDescription());
                }
                ...
            }
            break;
            case "permission": { //解析 permission 标签
                if (allowPermissions) {
                    String perm = parser.getAttributeValue(null, "name");
                    if (perm == null) {
                        Slog.w(TAG, "<" + name + "> without name in " + permFile
                            + " at " + parser.getPositionDescription());
                        XmlUtils.skipCurrentTag(parser);
                        break;
                    }
                    perm = perm.intern();
                    readPermission(parser, perm); //调用 readPermission 处理, 存入
                } else {
                    mPermissions

```

```
l         gNotAllowedInPartition(name, permFile, parser);  
o         XmlUtils.skipCurrentTag(parser);  
        }  
    } break;  
}  
}
```

博学广闻

```
}
```

查看 XML文件:

adb devices

adb shell

```
root@generic_x86:/ # cd /system/etc/permissions/
root@generic_x86:/system/etc/permissions # ls -all
-rw-r--r-- root    root      931 2020-07-21 02:15 android.hardware.camera.autofocus.xml
-rw-r--r-- root    root     1144 2020-07-21 02:15 android.hardware.touchscreen.multitouch.jazzhand.xml
-rw-r--r-- root    root      975 2020-07-21 02:15 android.hardware.usb.accessory.xml
-rw-r--r-- root    root     1050 2020-07-21 02:17 android.software.live_wallpaper.xml
-rw-r--r-- root    root      748 2020-07-21 02:15 android.software.webview.xml
-rw-r--r-- root    root      828 2020-07-21 02:16 com.android.location.provider.xml
-rw-r--r-- root    root      828 2020-07-21 02:16 com.android.media.remotedisplay.xml
-rw-r--r-- root    root      820 2020-07-21 02:16 com.android.mediarlm.signer.xml
-rw-r--r-- root    root      816 2020-07-21 02:15 com.google.android.maps.xml
-rw-r--r-- root    root      835 2020-07-21 02:16 com.google.android.media.effects.xml
-rw-r--r-- root    root     3915 2020-07-21 02:15 handheld_core_hardware.xml
-rw-r--r-- root    root     6281 2020-07-21 02:16 platform.xml
root@generic_x86:/system/etc/permissions #
```

然后在导出:

```
adb pull /system/etc/permissions
```

/system/etc/permissions中会存在很多的xml文件,例如我们看下 android.software.webview.xml的文件,内容如下:

里面只只有一个feature "android.software.webview",大部分的xml都是类似的定义方式

```
<?xml version="1.0" encoding="utf-8"?>
<permissions>
  <feature name="android.software.webview" />
</permissions>
```

让我们来简单的看下/system/etc/permissions/platform.xml的内容

```
<?xml version="1.0" encoding="utf-8"?>
<permissions>
  <permission name="android.permission.BLUETOOTH_ADMIN" >
    <group gid="net_bt_admin" />
  </permission>
  <permission name="android.permission.INTERNET" >
    <group gid="inet" />
  </permission>
  <permission name="android.permission.READ_LOGS" >
    <group gid="log" />
  </permission>
  ...
  <assign-permission name="android.permission.MODIFY_AUDIO_SETTINGS"
uid="media" />
  <assign-permission name="android.permission.ACCESS_SURFACE_FLINGER"
uid="media" />
  <assign-permission name="android.permission.WAKE_LOCK" uid="media" />
  ...
</permissions>
```

```

<split-permission name="android.permission.ACCESS_FINE_LOCATION">
    <new-permission name="android.permission.ACCESS_COARSE_LOCATION" />
</split-permission>
<split-permission name="android.permission.WRITE_EXTERNAL_STORAGE">
    <new-permission name="android.permission.READ_EXTERNAL_STORAGE" />
</split-permission>
<split-permission name="android.permission.READ_CONTACTS"
    targetSdk="16">
    <new-permission name="android.permission.READ_CALL_LOG" />
</split-permission>
...
<library name="android.test.base" file="/system/framework/android.test.base.jar" />
<library name="android.test.mock" file="/system/framework/android.test.mock.jar"
    dependency="android.test.base" />
<library name="android.test.runner" file="/system/framework/android.test.runner.jar"
    dependency="android.test.base:android.test.mock" />

<!-- In BOOT_JARS historically, and now added to legacy applications. -->
<library name="android.hidl.base-V1.0-java"
    file="/system/framework/android.hidl.base-V1.0-java.jar" />
<library name="android.hidl.manager-V1.0-java"
    file="/system/framework/android.hidl.manager-V1.0-java.jar"
    dependency="android.hidl.base-V1.0-java" />
...
</permissions>

```

以上platform.xml中出现的标签种类则较为多样，它们的含义分别是：

platform.xml中出现的标签种类则较为多样，它们的含义分别是：

<group>：根据name获取gid

<permission >标签：把属性name所描述的权限赋予给**<group>**标签中属性gid所表示的用户组，一个权限可以有多个group对象，当一个APK授权于这个这个权限时，它同时属于这几个组

<assign-permission>标签：把属性name所描述的权限赋予给uid属性所表示的用户

<split-permission>标签：一个权限可以扩展出一个新的权限

<library>标签：除framework中动态库以外的，所有系统会自动加载的动态库

<feature>标签：硬件支持的一些feature

<oem-permission>标签：oem厂商自己定义的一些权限

<privapp-permission>标签：来自system、vendor、product、system_ext的privapp权限分别存储，这是防止供应商分区中的xml授权于系统分区中的私有应用权限

最后将上面xml解析出来的数据做如下存储：

<group>标签gid属性的值会存放在mGlobalGids数组中；

<permission>标签,解析得到的值会存放在mPermissions集合中;

<assign-permission>标签解析得到的值会存放在mSystemPermissions中;

<split-permission>存储在mSplitPermissions

<library>标签解析得到的值会存放在mSharedLibraries中;

<feature>存储在mAvaliableFeatures

<oem-permission>存储在mOemPermissions

<privapp-permission>会根据不同的存储路径,分别存储在mVendorPrivAppPermissions、mProductPrivAppPermissions、mSystemExtPrivAppPermissions、mPrivAppPermissions

总结: 权限扫描, 扫描/system/etc/permissions中的xml, 存入相应的结构体中, 供之后权限管理使用

高亮课堂

=====

=====

=====

同学们注意：此次 ” PackageManagerService大综合笔记-6.0 “，只是给同学们写出来，不是授课内容哦

第八节 PackageManagerService大综合笔记

同学们我们开始进入前戏步骤：

首先 PackageManagerService 简称 PKMS

PKMS 是什么东西？



答：PackageManagerService（简称 PKMS），是 Android 系统中核心服务之一，负责应用程序的**安装，卸载，信息查询**，等工作。

PKMS 概述信息：

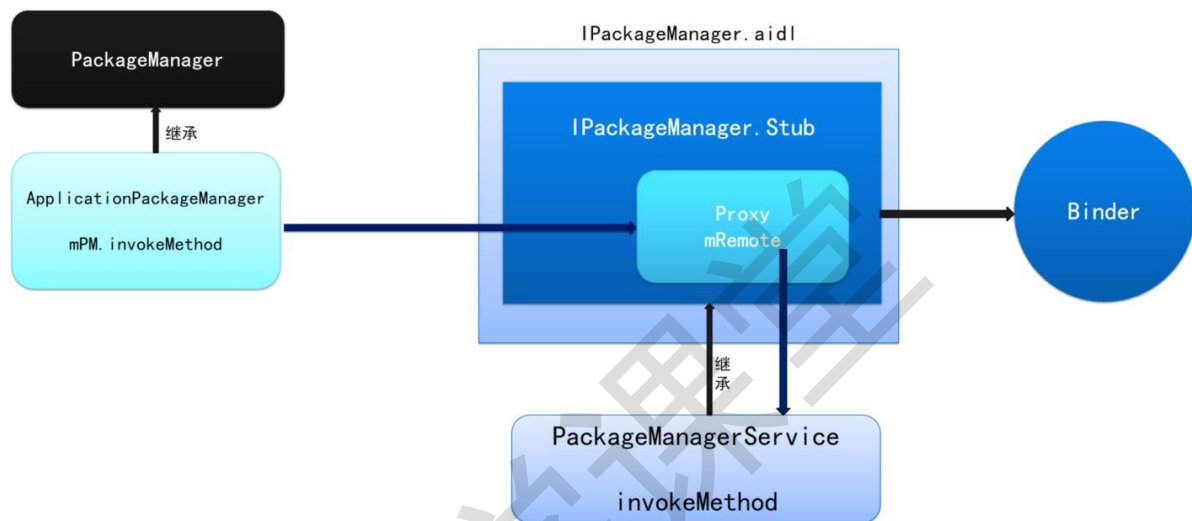
Android系统启动时，会启动（应用程序管理服务PKMS），此服务负责扫描系统中特定的目录，寻找里面的APK格式的文件，并对这些文件进行解析，然后得到应用程序相关信息，最后完成应用程序的安装

PKMS在安装应用过程中，会全面解析应用程序的AndroidManifest.xml文件，来得到Activity, Service, BroadcastReceiver, ContextProvider 等信息，在结合PKMS服务就可以在OS中正常的使用应用程序了

在Android系统中，系统启动时由SystemServer启动PKMS服务，启动该服务后会执行应用程序的安装过程，

接下来就会重点的介绍（SystemServer启动PKMS服务的过程，讲解在Android系统中安装应用程序的过程）

一部曲 - PKMS角色位置：



同学们注意：客户端可通过Context.getPackageManager()获得ApplicationPackageManager对象，而mPM指向的是Proxy代理，当调用到mPM.方法后，将会调用到IPackageManager的Proxy代理方法，然后通过Binder机制中的mRemote与服务端PackageManagerService通信并调用到PackageManagerService的方法；

自我总结：PKMS是属于Binder机制的服务端角色

接下来，我们就自己来手写一个简单的PKMS，同学们好不好

二部曲 - PKMS之前启动流程源码分析：

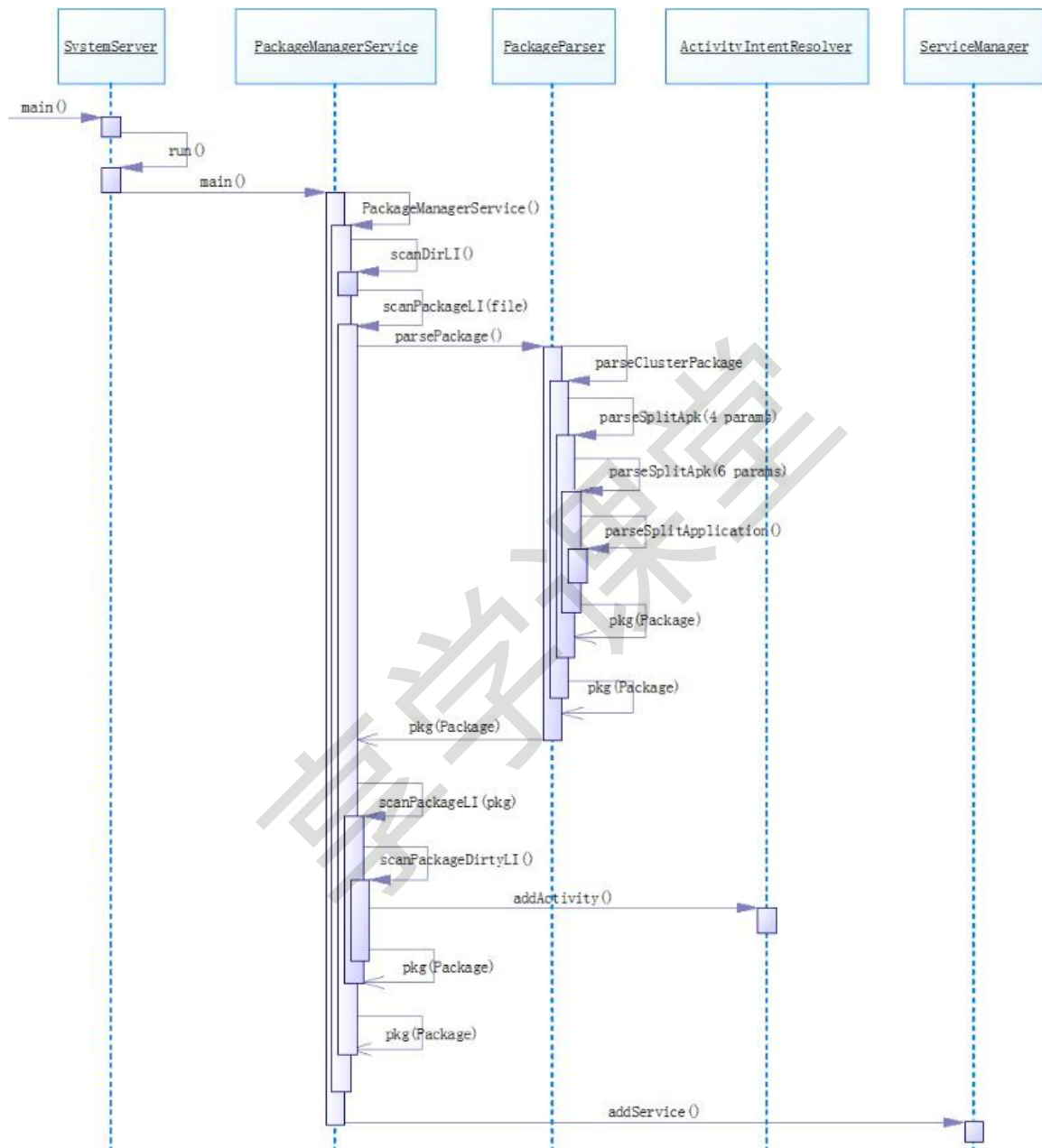
一、概述

目标：本文主要分析PKMS的初始化过程；

版本：Android 23

PKMS启动流程：Zygote --> SystemServer --> PackageManagerService (PMS)

二、PKMS初始化时序图



三、PKMS源码

在Android系统中，通过Zygote进程启动 SystemServer组件时会调用主函数main

此类目录：`frameworks/base/services/java/com/android/server/SystemServer.java`

```
public static void main(String[] args) { new
    SystemServer().run();
}

private void run() {
    // ...省略大段代码...
    // TODO 同学们注意：除了启动PKMS服务之外, 还启动了其他很多的服务，例如：
    ActivityManagerService等
    startBootstrapServices(); // 同学们 这是 引导服务
    startCoreServices(); // 同学们 这是 核心服务
    startOtherServices(); // 同学们 这是 其他服务
}

private void startBootstrapServices() {
    /*
    13创建ActivityManagerService实例
    14同学们注意：mActivityManagerService是被添加进SystemServiceManager维护起来的
    */
    mActivityManagerService =
        mSystemServiceManager.startService( ActivityManagerService.Lifecycle.class).getService();

    /*
    15创建PackageManagerService实例；
    16同学们注意：PackageManagerService没有被SystemServiceManager维护起来；
    17思考：那它到底被谁维护起来了呢？
    */
    mPackageManagerService = PackageManagerService.main(mSystemContext, installer,
        mFactoryTestMode != FactoryTest.FACTORY_TEST_OFF, mOnlyCore);
}
```

创建一个PKMS服务实例，然后把这个服务实例添加到 ServiceManager中去，ServiceManager是Android系统Binder进程间通信机制的进程，负责管理系统中的Binder对象

此类目

录：`/frameworks/base/services/core/java/com/android/server/pm/PackageManagerService.java`

```

public static PackageManagerService main(Context context, Installer installer,
    boolean factoryTest, boolean onlyCore) {
    .....

    PackageManagerService m = new PackageManagerService(context, installer,
        factoryTest, onlyCore);
    m.enableSystemUserPackages();

    // 在这里将PackageManagerService实例添加到ServiceManager中
    ServiceManager.addService("package", m);
    return m;
}

```

自我总结：在SystemService中

- | | | |
|---------------------------|---------------------------|-------|
| 22 PackageManagerService | 实例是被 ServiceManager | 存储起来； |
| 23 ActivityManagerService | 实例是被 SystemServiceManager | 存储起来； |

PKMS 构造方法八件核心任务：

- 30 创建Settings 对象，其内部创建了packages.xml、packages-backup.xml、packages.list 等文件，用于存储应用信息；
- 31 在 SystemConfig 对象中，读取系统配置来初始化 mGlobalGids、mSystemPermissions、mAvailableFeatures；
- 32 指定 /data 目录下的一系列的文件目录，如 /data/data、/data/app 等；
- 33 如果 packages.xml 文件存在，则读取并解析该文件信息，然后保存到Settings相应的字段中；
- 34 开始扫描指定目录下的apk文件，解析其Manifest文件，并将值赋值给Package对应的属性字段(这个步骤是重点)；
- 35 更新所有的共享库；
- 36 更新应用权限；
- 37 将数据写入packages.xml中；

```

/*
36 该PKMS构造方法主要的作用：
*
37 扫描指定文件路径下的文件：
381.vendorOverlayDir = "/vendor/overlay"
392.frameworkDir = "/system/framework"
403.privilegedAppDir = "/system/priv-app"
414.systemAppDir = "/system/app"
425.vendorAppDir = "/vendor/app"
436.mAppInstallDir = "/data/app"
447.mDrmAppPrivateInstallDir = "/data/app-private"
*/
public PackageManagerService(Context context, Installer installer, boolean
factoryTest, boolean onlyCore) {

    // ...代码省略...

    // 第一件事情.添加SharedUserSetting对象到mSettings中。
    mSettings = new Settings(mPackages);
    mSettings.addSharedUserLPw("android.uid.system", Process.SYSTEM_UID, ...);
}

```

```

mSettings.addSharedUserLPw("android.uid.phone", RADIO_UID, ...);
mSettings.addSharedUserLPw("android.uid.log", LOG_UID, ...);
mSettings.addSharedUserLPw("android.uid.nfc", NFC_UID, ...);
mSettings.addSharedUserLPw("android.uid.bluetooth", BLUETOOTH_UID, ...);
mSettings.addSharedUserLPw("android.uid.shell", SHELL_UID, ...);

// ...代码省略...

// 第二件事情. 读取系统配置来初始化mGlobalGids、mSystemPermissions、
mAvailableFeatures

SystemConfig systemConfig = SystemConfig.getInstance(); mGlobalGids
= systemConfig.getGlobalGids(); mSystemPermissions =
systemConfig.getSystemPermissions(); mAvailableFeatures =
systemConfig.getAvailableFeatures();

synchronized (mInstallLock) {
    // writer
    synchronized (mPackages) {
        mHandlerThread = new ServiceThread(TAG,
            Process.THREAD_PRIORITY_BACKGROUND, true /*allowIo*/);

        mHandlerThread.start();
        mHandler = new PackageHandler(mHandlerThread.getLooper());

        /*
        41 第三件事情. 指定 "/data" 目录下的文件夹, 便于使用;
        42 dataDir = "/data"
        43 mAppDataDir = "/data/data" 应用数据存储目录
        44 mAppInstallDir = "/data/app" 应用安装目录
        45 mAppLib32InstallDir = "/data/app-lib"
        46 mAsecInternalPath = "/data/app-asec"
        47 mUserAppDataDir = "/data/user"
        48 mDrmAppPrivateInstallDir = "/data/app-private"
        */

        File dataDir = Environment.getDataDirectory(); mAppDataDir =
        new File(dataDir, "data"); mAppInstallDir = new File(dataDir,
        "app"); mAppLib32InstallDir = new File(dataDir, "app-lib");

        mAsecInternalPath = new File(dataDir, "app-asec").getPath();
        mUserAppDataDir = new File(dataDir, "user"); mDrmAppPrivateInstallDir =
        new File(dataDir, "app-private");

        // 第四件事情. 从packages.xml文件中解析出信息(如果该文件存在), 并保存到
        Settings相应的字段中;

        mRestoredSettings = mSettings.readLPw(this,
        sUserManager.getUsers(false),

            mSdkVersion, mOnlyCore);
    }
}

```

```
// ...省略大段代码...

// 第五件事情. 使用scanDirLI()扫描指定目录下的apk文件, 解析其Manifest文件, 并将值赋
// 值给Package对应的属性字段:
// vendorOverlayDir = "/vendor/overlay"
File vendorOverlayDir = new File(VENDOR_OVERLAY_DIR);
// 同学们注意啦: scanDirLI()是重点, 留一下。
scanDirLI(vendorOverlayDir, PackageParser.PARSE_IS_SYSTEM
          | PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags |
SCAN_TRUSTED_OVERLAY, 0);
```

```

/*
49Find base frameworks (resource packages without code).
50frameworkDir = "/system/framework"
51framework包里都是jar包和apk
*/
File frameworkDir = new File(Environment.getRootDirectory(),
"framework");

scanDirLI(frameworkDir, PackageParser.PARSE_IS_SYSTEM
    | PackageParser.PARSE_IS_SYSTEM_DIR
    | PackageParser.PARSE_IS_PRIVILEGED,
    scanFlags | SCAN_NO_DEX, 0);

// Collected privileged system packages.
// privilegedAppDir = "/system/priv-app" final
File privilegedAppDir = new
File(Environment.getRootDirectory(), "priv-app"); scanDirLI(privilegedAppDir,
PackageParser.PARSE_IS_SYSTEM
    | PackageParser.PARSE_IS_SYSTEM_DIR
    | PackageParser.PARSE_IS_PRIVILEGED, scanFlags, 0);

// Collect ordinary system packages.
// systemAppDir = "/system/app"
final File systemAppDir = new File(Environment.getRootDirectory(),
"app");
scanDirLI(systemAppDir, PackageParser.PARSE_IS_SYSTEM
    | PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags, 0);

// Collect all vendor packages.
// vendorAppDir = "/vendor/app"
File vendorAppDir = new File("/vendor/app"); try {
    vendorAppDir = vendorAppDir.getCanonicalFile();
} catch (IOException e) {
    // failed to look up canonical path, continue with original one
}
scanDirLI(vendorAppDir, PackageParser.PARSE_IS_SYSTEM
    | PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags, 0);

// Collect all OEM packages.
// oemAppDir = "/oem/app"
final File oemAppDir = new File(Environment.getOemDirectory(),
"app");
scanDirLI(oemAppDir, PackageParser.PARSE_IS_SYSTEM
    | PackageParser.PARSE_IS_SYSTEM_DIR, scanFlags, 0);

```



```
// mOnlyCore = true表示系统package if
(!mOnlyCore) {

    // 这里扫描用户应用：mAppInstallDir = "/data/app"
    scanDirLI(mAppInstallDir, 0, scanFlags | SCAN_REQUIRE_KNOWN, 0);

    // 这里扫描用户应用：mDrmAppPrivateInstallDir = "/data/app-private"
    scanDirLI(mDrmAppPrivateInstallDir,

PackageParser.PARSE_FORWARD_LOCK,

        scanFlags | SCAN_REQUIRE_KNOWN, 0);

    // ...省略大段代码...
}
```

```

        // ...省略大段代码...

        // Now that we know all of the shared libraries, update all clients
        to have the correct library paths.
        // 第六件事情.更新所有的共享库;
        updateAllSharedLibrariesLPw();

        // ...省略代码...
        // 第七件事情.更新应用权限;
        updatePermissionsLPw(null, null, updateFlags);
        // ...省略代码...

        // 第八件事情.将数据写入packages.xml中;
        mSettings.writeLPw();
    } // synchronized (mPackages)
} // synchronized (mInstallLock)

// ...省略代码...
}

```

第一件事情细节：创建Settings 对象，其内部创建了packages.xml、packages-backup.xml、packages.list 等文件，用于存储应用信息；

此类目录：/frameworks/base/services/core/java/com/android/server/pm/Settings.java

```

Settings(Context context) {
    this(context, Environment.getDataDirectory());
}

/*
 * 在 "/data/system/" 目录下创建一系列的文件;
 */
Settings(File dataDir, Object lock) {
    // 创建一个"data/system"目录
    mSystemDir = new File(dataDir, "system");
    mSystemDir.mkdirs();

    // 同学们注意：会新建几个文件
    mSettingsFilename = new File(mSystemDir, "packages.xml");
    mBackupSettingsFilename = new File(mSystemDir, "packages-backup.xml");
    mPackageListFilename = new File(mSystemDir, "packages.list");
    FileUtils.setPermissions(mPackageListFilename, 0640, SYSTEM_UID,
        PACKAGE_INFO_GID);

    // Deprecated: Needed for migration
    mStoppedPackagesFilename = new File(mSystemDir, "packages-stopped.xml");
    mBackupStoppedPackagesFilename = new File(mSystemDir, "packages-stopped-
        backup.xml");
}

```

自我总结：

所在目录： /data/system/

packages.xml	记录系统中所有安装的应用信息，包括
基本信息、签名和权限	
packages-backup.xml packages.xml	文件的备份
packages.list	保存普通应用的数据目录和uid等信息
packages-stopped.xml	记录系统中被强制停止运行的应用信息。
packages-stopped-backup.xml packages-stopped.xml	文件的备份

Emulator Nexus_4_API_22 Android 5.1.1, API 22			
Name	Permissions	Date	Size
▼ system	drwxrwxr-x	2020-09-07 06:57	
▶ dropbox	drwx-----	2020-09-07 06:46	
▶ ifw	drwx-----	2020-08-28 03:33	
▶ inputmethod	drwx-----	2020-08-28 03:33	
▶ install_sessions	drwx-----	2020-08-28 03:33	
▶ job	drwx-----	2020-08-28 03:33	
▶ netstats	drwx-----	2020-08-28 03:33	
▶ procstats	drwx-----	2020-09-07 06:57	
▶ recent_images	drwx-----	2020-09-02 09:41	
▶ recent_tasks	drwx-----	2020-09-02 09:41	
▶ registered_services	drwxrwx--x	2020-08-28 03:34	
▶ shared_prefs	drwxrwx--x	2020-08-28 03:33	
▶ sync	drwx-----	2020-08-28 03:33	
▶ usagstats	drwx-----	2020-08-28 03:33	
▶ users	drwxrwxr-x	2020-08-28 03:33	
appops.xml	-rw-----	2020-09-07 06:47	3.2 KB
batterystats.bin	-rw-----	2020-09-07 06:55	39.1 KB
called_pre_boots.dat	-rw-----	2020-08-28 03:33	632 B
device_policies.xml	-rw-----	2020-08-28 03:34	234 B
entropy.dat	-rw-----	2020-09-02 04:31	512 B
framework_atlas.config	-rw-----	2020-08-28 03:33	130 B
install_sessions.xml	-rw-----	2020-09-02 09:39	70 B
last-fstrim	-rw-----	2020-08-28 03:33	0 B
locksettings.db	-rw-rw----	2020-08-28 03:33	4 KB
locksettings.db-shm	-rw-----	2020-08-28 03:33	32 KB
locksettings.db-wal	-rw-----	2020-08-28 03:33	64.4 KB
ndebugsocket	srwx-----	2020-08-28 03:33	
package-usage.list	-rw-r-----	2020-09-07 06:57	1.5 KB
packages.list	-rw-r-----	2020-09-02 09:41	7 KB
packages.xml	-rw-rw----	2020-09-02 09:41	133.5 KB
seapp_hash	-rw-----	2020-08-28 03:33	20 B

同学们这里是UID，留意一下啊：Settings.addSharedUserLPw(String name, int uid, int pkgFlags, int pkgPrivateFlags)

```

ArrayMap<String, SharedUserSetting> mSharedUsers = new ArrayMap<String,
SharedUserSetting>();

SharedUserSetting addSharedUserLPw(String name, int uid, int pkgFlags, int
pkgPrivateFlags) {
    SharedUserSetting s = mSharedUsers.get(name);

```

```

// ...代码省略...
s = new SharedUserSetting(name, pkgFlags, pkgPrivateFlags);
s.userId = uid;
if (addUserIdLPw(uid, s, name)) {
    // 将SharedUserSetting添加到mSharedUsers中;
    mSharedUsers.put(name, s);
    return s;
}
return null;
}

```

同学们注意：第四件事情细节：如果 `packages.xml` 文件存在，则读取并解析该文件信息，然后保存到 Settings 相应的字段中；

`packages.xml` 等文件的读取及解析

本方法实际就是对 `packages.xml`、`packages-backup.xml` 等文件进行操作，从中获取到对应的值；

```

/*
 * 本方法实际就是对packages.xml、packages-backup.xml等文件进行操作，从中获取到对应的值；
 */
boolean readLPw(PackageManagerService service, List<UserInfo> users,
    int sdkVersion, boolean onlyCore) {
    FileInputStream str = null;
    // 如果packages-backup.xml存在，则解析备份文件;
    if (mBackupSettingsFilename.exists()) {
        str = new FileInputStream(mBackupSettingsFilename);
        // ...代码省略...
        if (mSettingsFilename.exists())
            { mSettingsFilename.delete()
            ;
        }
    }
    // ...代码省略...

    // 如果packages-backup.xml不存在，则解析packages.xml
    if (str == null) {
        if (!mSettingsFilename.exists())
            { return false;
        }
        str = new FileInputStream(mSettingsFilename);
    }
    XmlPullParser parser = Xml.newPullParser();
    parser.setInput(str, StandardCharsets.UTF_8.name());

    // ...代码省略...

    int type;
    int outerDepth = parser.getDepth();
    while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
        && (type != XmlPullParser.END_TAG || parser.getDepth() >
outerDepth)) {
        // 接下去就是xml文件解析
        String tagName = parser.getName();
        if (tagName.equals("package")) {
            readPackageLPw(parser);

```

```

        } else if (tagName.equals("permissions"))
        { readPermissionsLPW(mPermissions,
            parser);
        }
        // ...省略很多xml节点解析代码...
    }
    str.close();

    // ...代码省略...

    if (mBackupStoppedPackagesFilename.exists()
        || mStoppedPackagesFilename.exists()) {
        // 这个方法的逻辑和readLPW()方法类似，只是这里读取的是packages-stopped.xml和
        packages-stopped-backup.xml 文件；
        readStoppedLPW();
        mBackupStoppedPackagesFilename.delete();
        mStoppedPackagesFilename.delete();
        // Migrate to new file format
        writePackageRestrictionsLPr(0);
    } else {
        // ...代码省略...
    }
    // ...代码省略...
    return true;
}

```

同学们注意：第五件事情细节：开始扫描指定目录下的apk文件，解析其Manifest文件，并将值赋值给Package对应的属性字段(这个步骤是重点)；

Manifest 文件的解析

扫描指定目录下的apk文件，解析其Manifest文件，并将值赋值给Package对应的属性字段(这个步骤是重点)；

```

/*
 * 扫描指定的文件夹，如果是该文件夹内的文件是apk文件或是文件夹，则继续扫描；
 */
private void scanDirLI(File dir, int parseFlags, int scanFlags, long
currentTime) {
    final File[] files = dir.listFiles();
    if (ArrayUtils.isEmpty(files)) return;

    for (File file : files) {
        // isApkFile(file):文件后缀是否为apk
        final boolean isPackage = (isApkFile(file) || file.isDirectory())
            && !PackageInstallerService.isStageName(file.getName());
        if (!isPackage) { // 如果是常规文件(非文件夹、非Apk文件)，则跳过；
            continue;
        }
        // 扫描指定的文件目录或apk
        scanPackageLI(file, parseFlags | PackageParser.PARSE_MUST_BE_APK,
            scanFlags, currentTime, null);
    }
}

```

```

/*
 * 解析Apk包中的Manifest.xml文件
 *
 * 返回的PackageParser.Package对象，表示一个Apk文件对应的数据结构；
 */
private PackageParser.Package scanPackageLI(File scanFile, int parseFlags, int
scanFlags,
                                           long currentTime, UserHandle user)
throws PackageManagerException
{
    PackageParser pp = new
    PackageParser();

    // 解析Apk包中的清单文件，将对应节点设置到Package对应的属性字段上；
    final PackageParser.Package pkg = pp.parsePackage(scanFile, parseFlags);

    // ...省略大段代码...

    // Verify certificates against what was last scanned.
    // 对 安装 的程序 进行 签名 验证（这里不展开）
    collectCertificatesLI(pp, ps, pkg, scanFile, parseFlags);

    // ...省略大段代码...

    // Note that we invoke the following method only if we are about to unpack an
    application
    PackageParser.Package scannedPkg = scanPackageLI(pkg, parseFlags, scanFlags
    | SCAN_UPDATE_SIGNATURE, currentTime, user);

    // ...省略代码...
    return scannedPkg;
}

```

类 `PackageParser` 主要功能就是解析Apk文件中的 `Manifest.xml` 文件
`pp.parsePackage(scanFile, parseFlags);`

```

/*
42`parseClusterPackage()、parseMonolithicPackage()`
43 这两个方法内部都会执行parseBaseApk();
*/
public Package parsePackage(File packageFile, int flags) throws
PackageParserException {
    if (packageFile.isDirectory()) { // 同学们注意：packageFile是文件夹
        return parseClusterPackage(packageFile, flags);
    } else { // 同学们注意：packageFile是Apk文件
        return parseMonolithicPackage(packageFile, flags);
    }
}

/*
44解析Apk文件
*/
public Package parseMonolithicPackage(File apkFile, int flags) throws
PackageParserException {
    // ...代码省略...
    final Package pkg = parseBaseApk(apkFile, assets, flags);
    // ...代码省略...
}

```

[illegible]

```
        return pkg;
    } catch (Exception e) {
        // ... 代码省略...
    }
}
```

```
/*
 * 解析Manifest文件下的所有节点，将他们存储到Package对应的字段中，并返回Package对象；
 */
```

博学课道


```

private Package parseBaseApk(Resources res, XmlPullParser parser, int flags, String[]
    outError) throws XmlPullParserException, IOException {

    final boolean trustedOverlay = (flags & PARSE_TRUSTED_OVERLAY) != 0;
    // ...代码省略...

    // 创建一个Package对象，用于存储从Manifest文件中解析出来的节点信息；
    final Package pkg = new Package(pkgName);

    // ...代码省略...
    int outerDepth = parser.getDepth();
    while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
        && (type != XmlPullParser.END_TAG || parser.getDepth() >
outerDepth)) {

        if (type == XmlPullParser.END_TAG || type == XmlPullParser.TEXT) { continue;
        }

        if (tagName.equals("application")) {
            // 解析Manifest文件中application节点之间的所有节点信息；
            if (!parseBaseApplication(pkg, res, parser, attrs, flags, outError))
{
                return null;
            }
        } else if (tagName.equals("uses-permission")) {
            // 解析Manifest文件中uses-permission节点信息，并将它存储到
Package.requestedPermissions字段中；
            if (!parseUsesPermission(pkg, res, parser, attrs)) { return
                null;
            }
        }
        // ...省略大段代码...
    }
    // ...省略大段代码...
    return pkg;
}

/**
64解析manifest文件中Application节点下的节点，将对应的节点分别保存在Package对象的对应字段属 性中；
    *
65如四大组件标签存放的字段：
66List<Activity> activities;
67List<Activity> receivers;
68List<Provider> providers;
69List<Service> services;
    */
private boolean parseBaseApplication(Package owner, Resources res, XmlPullParser parser,
    AttributeSet attrs, int flags, String[] outError)

    throws XmlPullParserException, IOException {

```

```
// ...省略大段代码...

// 开始解析
final int innerDepth = parser.getDepth(); int
type;

while ((type = parser.next()) != XmlPullParser.END_DOCUMENT
      && (type != XmlPullParser.END_TAG || parser.getDepth() >
innerDepth)) {
```

高学课道

```

// ...省略代码...

String tagName = parser.getName();
if (tagName.equals("activity")) {
    // 解析Manifest文件中的Activity节点
    Activity a = parseActivity(owner, res, parser, attrs,
        flags, outError, false, owner.baseHardwareAccelerated);
    // ...省略代码...
    // 将该Activity添加到PackageParser.Package.activities中;
    owner.activities.add(a);

} else if (tagName.equals("receiver")) {
    // 将该Receiver添加到PackageParser.Package.receivers中;
    owner.receivers.add(a);

} else if (tagName.equals("service")) {
    // 将该Service添加到PackageParser.Package.services中;
    owner.services.add(s);

} else if (tagName.equals("provider")) {
    // 将该Provider添加到PackageParser.Package.providers中;
    owner.providers.add(p);
}
// ...省略大段代码...
}
// ...省略代码...
return true;
}

```

自我总结： 同学们 到这里为止，这个Apk文件的AndroidManifest.xml文件的解析就结束了；

同学们注意： 第八件事情 将数据写入packages.xml中；

```

void writeLPr() {
    /*
    66前提: packages.xml存在
    671.备份不存在，就将packages.xml重命名成备份名；
    682.备份存在，就删除packages.xml文件；
    69总结：只要有一份文件即可；
    */
    if (mSettingsFilename.exists()) {
        if (!mBackupSettingsFilename.exists()) {
            if (!mSettingsFilename.renameTo(mBackupSettingsFilename))
                { return;
            }
        } else {
            mSettingsFilename.delete();
        }
    }
    // 开始向packages.xml文件中写入应用相关的数据；
    try {
        FileOutputStream fstr = new FileOutputStream(mSettingsFilename);
        BufferedOutputStream str = new BufferedOutputStream(fstr);

        //XmlSerializer serializer = XmlUtils.serializerInstance();
        XmlSerializer serializer = new FastXmlSerializer();
    }
}

```

```

        serializer.setOutput(str, StandardCharsets.UTF_8.name());
        serializer.startDocument(null, true);
        serializer.setFeature("http://xmlpull.org/v1/doc/features.html#indent-
output", true);
        serializer.startTag(null, "packages");

        // ...省略很多节点插入的代码...

        // 写入权限
        serializer.startTag(null, "permissions");
        for (BasePermission bp : mPermissions.values())
            { writePermissionLPr(serializer, bp);
        }
        serializer.endTag(null, "permissions");

        // 将Package数据写入
        for (final PackageSetting pkg : mPackages.values())
            { writePackageLPr(serializer, pkg);
        }
        // ...省略很多节点插入的代码...

        serializer.endTag(null, "packages");
        serializer.endDocument();

        str.flush(); FileUtils.sync(fstr);
        str.close();

        // 到这里, packages.xml已经写入了最新的数据, 所以需要删除packages-backup.xml文件
        mBackupSettingsFilename.delete();
        // 设置packages.xml文件的权限;
        FileUtils.setPermissions(mSettingsFilename.toString(),
            FileUtils.S_IRUSR|FileUtils.S_IWUSR
            |FileUtils.S_IRGRP|FileUtils.S_IWGRP,
            -1, -1);

        // 向packages.list文件中写入数据
        writePackageListLPr();
        writeAllUsersPackageRestrictionsLPr();
        writeAllRuntimePermissionsLPr(); return;
    } catch (Exception e) {
        //...
    }

    // Clean up partially written files if
    (mSettingsFilename.exists()) {
        if (!mSettingsFilename.delete()) {

```

```
}  
    }  
}
```

//...

博学课业网

大总结:

72在 `"/data/system"` 目录中创建一系列的文件用于存储应用的信息;

73在 `SystemConfig` 对象中读取系统配置来初始化 `mGlobalGids`、`mSystemPermissions`、`mAvailableFeatures`;

74指定 `"/data"` 目录下一些文件夹, 一部分是用于第4步的扫描目录的指定;

75读取 `packages.xml` 文件的数据, 解析并存储到Settings中;

76扫描指定文件目录下的Apk包, 解析其内部的Manifest.xml文件, 将值赋值给PMS和Settings、Package中;

77将扫描出来的应用信息重新写入 第八件事情的 `packages.xml` 文件中;

三部曲 - 有界面安装 与 无界面安装

1、概述

apk的安装有几种方式, 整体可分为2类, 一类是有界面安装, 一类是无界面安装。无界面安装又可分为内置apk开机安装和命令安装, 而命令安装又可分为2类, 一类是通过电脑的adb安装, 另一类是通过手机安装的pm命令。

今天我们主要介绍两种, 一种是有界面安装, 另一种就是通过电脑端的adb无界面安装。

2、有界面安装

有界面的安装方式相信同学们平时接触的都比较多了吧, 比如从网络上下载一个apk之后就会弹出安装界面, 那就是有界面安装方式。

有界面安装



首先我们要知道有界面安装方式的那个界面是从哪里来的，还有就是如何启动那个安装界面的。那个界面其实就是PackageInstallerActivity类，这是packages/app/PackageInstaller应用中的主界面，我们可以看一下该应用的Manifest文件。

```
<activity
    android:name=".PackageInstallerActivity"
    android:configChanges="orientation|keyboardHidden|screenSize"
    android:theme="@android:style/Theme.DeviceDefault.Light.NoActionBar"
    android:excludeFromRecents="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <action android:name="android.intent.action.INSTALL_PACKAGE" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:scheme="file" />
        //同学们注意：这里配置了application/vnd.android.package-archive这个字符串
        //的mime类型
        <data android:mimeType="application/vnd.android.package-archive" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.INSTALL_PACKAGE" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:scheme="file" />
        <data android:scheme="package" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.content.pm.action.CONFIRM_PERMISSIONS"
    />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

接下来我们直接进入PackageInstallerActivity类中。

```
public class PackageInstallerActivity extends Activity implements
    OnCancelListener, OnClickListener {

    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
    }

    // 同学们注意：在其OnResume方法中主要是加载布局和对apk包进行相关解析，将权限信息显示在界
    // 面上，而apk相关信息将保存下来，以便后期安装时使用
    protected void onResume()
    {
        super.onResume();

        mPm = getPackageManager();
        mInstaller = mPm.getPackageInstaller();
        mUserManager = (UserManager) getSystemService(Context.USER_SERVICE);

        //加载界面布局
        setContentView(R.layout.install_start);

        //读取apk包相关信息
        PackageParser.Package parsed = PackageUtil.getPackageInfo(sourceFile);
```

```

        //读取权限相关信息
        //.....
    }
}

```

当用户点击安装的时候，调用startInstall方法跳转到InstallAppProgress，并把当前需要安装的应用信息给传递过去，其中有应用安装的完整包名：

```

private void startInstall() {
    // Start subactivity to actually install the application
    Intent newIntent = new Intent();
    newIntent.putExtra(PackageUtil.INTENT_ATTR_APPLICATION_INFO,
        mPkgInfo.applicationInfo);
    newIntent.setData(mPackageURI);
    newIntent.setClass(this, InstallAppProgress.class);
    newIntent.putExtra(InstallAppProgress.EXTRA_MANIFEST_DIGEST,
        mPkgDigest);
    newIntent.putExtra(
        InstallAppProgress.EXTRA_INSTALL_FLOW_ANALYTICS,
        mInstallFlowAnalytics);
    String installerPackageName =
        getIntent().getStringExtra( Intent.EXTRA_INSTALLER_PACKAGE_NAME);
    if (mOriginatingURI != null)
        { newIntent.putExtra(Intent.EXTRA_ORIGINATING_URI, mOriginatingURI);
        }
    if (mReferrerURI != null)
        { newIntent.putExtra(Intent.EXTRA_REFERRER, mReferrerURI);
        }
    if (mOriginatingUid != VerificationParams.NO_UID)
        { newIntent.putExtra(Intent.EXTRA_ORIGINATING_UID, mOriginatingUid);
        }
    if (installerPackageName != null)
        { newIntent.putExtra(Intent.EXTRA_INSTALLER_PACKAGE_NAME,
            installerPackageName);
        }
    if (getIntent().getBooleanExtra(Intent.EXTRA_RETURN_RESULT, false))
        { newIntent.putExtra(Intent.EXTRA_RETURN_RESULT, true);
          newIntent.addFlags(Intent.FLAG_ACTIVITY_FORWARD_RESULT);
        }
    if (localLOGV) Log.i(TAG, "downloaded app uri="+mPackageURI);
    startActivity(newIntent);
    finish();
}

```

同学们注意：此Activity就是在安装过程中....

```

public class InstallAppProgress extends Activity implements
    View.OnClickListener, OnCancelListener {

    public void initView() {

        setContentView(R.layout.op_progress);
        PackageManager pm =getPackageManager();
    }
}

```



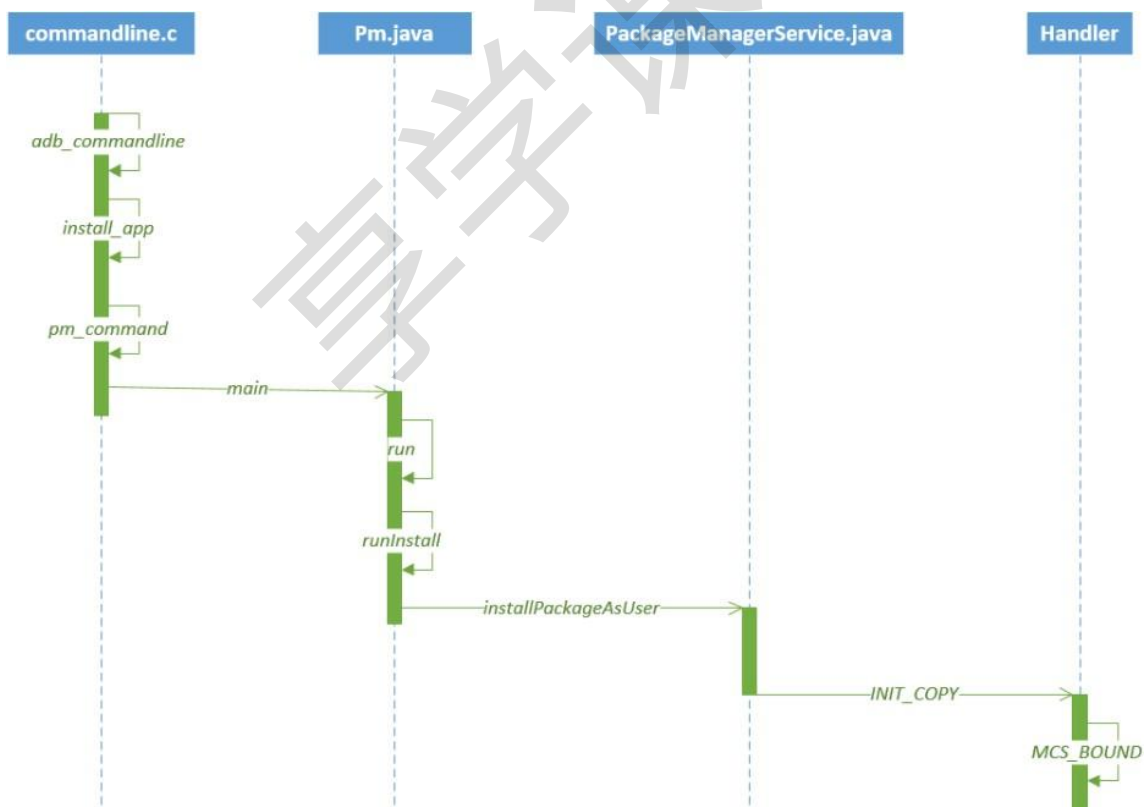
```

//.....
//创建安装完成后的监听接口
PackageInstallObserver observer = new PackageInstallObserver();

if ("package".equals(mPackageURI.getScheme()))
{ try {
    pm.installExistingPackage(mAppInfo.packageName);
    //监听安装结果
    observer.packageInstalled(mAppInfo.packageName,
        PackageManager.INSTALL_SUCCEEDED);
} catch (PackageManager.NameNotFoundException e)
{ observer.packageInstalled(mAppInfo.packageName,
    PackageManager.INSTALL_FAILED_INVALID_APK);
}
} else {
    pm.installPackageWithVerificationAndEncryption(mPackageURI,
observer, installFlags,
    installerPackageName, verificationParams, null);
}
}
}

```

3、无界面安装



都知道，adb install有很多的参数，但是我们就分析最简单的adb install qq.apk，adb是一个命令而install是其参数，这里我们直接进入处理install的代码逻辑

（system/core/adb/commandline.c）。

```

int adb_commandline(int argc, char **argv)
{
    //.....
    if (!strcmp(argv[0], "install"))
        { if (argc < 2) return
          usage();
          //调用install_app函数进行处理
          return install_app(ttype, serial, argc, argv);
        }
}

```

我们进入install_app函数，看其细节。

```

int install_app(transport_type transport, char* serial, int argc, char** argv)
{
    //手机内部存储路径
    static const char *const DATA_DEST = "/data/local/tmp/%s";
    //SD卡路径
    static const char *const SD_DEST = "/sdcard/tmp/%s";
    const char* where = DATA_DEST;

    for (i = 1; i < argc; i++) {
        //表示安装在SD卡上
        if (!strcmp(argv[i], "-s"))
            { where = SD_DEST;
            }
    }

    char* apk_file = argv[last_apk];
    //安装路径
    char apk_dest[PATH_MAX];
    snprintf(apk_dest, sizeof apk_dest, where, get_basename(apk_file));
    //调用do_sync_push将apk文件push到手机中
    int err = do_sync_push(apk_file, apk_dest, 0 /* no show progress */);
    if (err) {
        goto cleanup_apk;
    } else {
        argv[last_apk] = apk_dest; /* destination name, not source location */
    }
    //调用shell pm命令去安装
    pm_command(transport, serial, argc, argv);

cleanup_apk:
    //在手机中执行shell rm来删除刚刚推入的apk文件
    delete_file(transport, serial, apk_dest);
    return err;
}

```

这个方法主要的功能就是找到apk安装的路径，然后执行pm命令去安装，并在最终通过rm命令将apk进行删除，我们在来看一下pm_command函数的功能吧。

```

static int pm_command(transport_type transport, char* serial, int argc, char**
argv)

```

```

{
    char buf[4096];
    //通过pm命令去执行安装操作
    snprintf(buf, sizeof(buf), "shell:pm");

    while(argc-- > 0) {
        char *quoted = escape_arg(*argv++);
        strncat(buf, " ", sizeof(buf) - 1);
        strncat(buf, quoted, sizeof(buf) - 1);
        free(quoted);
    }

    send_shellcommand(transport, serial, buf);
    return 0;
}

```

那什么是pm呢？其实pm只是一个脚本，其源码所在路径（frameworks/base/cmds/pm/pm）。

```

# Script to start "pm" on the device, which has a very rudimentary
# shell.
#
base=/system
export CLASSPATH=$base/framework/pm.jar
exec app_process $base/bin com.android.commands.pm.Pm "$@"

```

可以发现其执行的是pm.jar包的main函数，我们进入Pm.java类。

```

public static void main(String[] args)
{
    int exitCode = 1;
    try {
        exitCode = new Pm().run(args);
    } catch (Exception e) {
        Log.e(TAG, "Error", e);
        System.err.println("Error: " + e);
        if (e instanceof RemoteException) {
            System.err.println(PM_NOT_RUNNING_ERR);
        }
    }
    System.exit(exitCode);
}

```

这里直接创建了Pm对象并调用其run方法，我们进入其run方法。

```

public int run(String[] args) throws IOException, RemoteException {
    mUm = IUserManager.Stub.asInterface(ServiceManager.getService("user"));
    //这里获得PKMS的代理对象
    mPm =
    IPackageManager.Stub.asInterface(ServiceManager.getService("package"));
    if (mPm == null) {
        System.err.println(PM_NOT_RUNNING_ERR);
        return 1;
    }
}

```

```

    }
    mInstaller = mPm.getPackageInstaller();

    //处理install参数，还有很多其他参数
    if ("install".equals(op)) {
        return runInstall();
    }
    //.....
}

```

可以发现这里又将安装的操作交给了runInstall这个方法，我们再次进入该方法。

```

private int runInstall() {
    while ((opt=nextOption()) != null) {
        //处理很多的参数命令
        if (opt.equals("-l")) {
            installFlags |= PackageManager.INSTALL_FORWARD_LOCK;
        } else if (opt.equals("-r")) {
            installFlags |= PackageManager.INSTALL_REPLACE_EXISTING;
        } else if (opt.equals("-i")) { installerPackageName
            =nextOptionData(); if (installerPackageName ==
            null) {
                System.err.println("Error: no value specified for -i"); return 1;
            }
        } else if (opt.equals("-t")) {
            installFlags |= PackageManager.INSTALL_ALLOW_TEST;
        } else if (opt.equals("-s")) {
            // Override if -s option is specified. installFlags |=
            PackageManager.INSTALL_EXTERNAL;
        } else if (opt.equals("-f")) {
            // Override if -s option is specified. installFlags |=
            PackageManager.INSTALL_INTERNAL;
        }
        //.....
    }

    //多用户手机时将所有用户都安装
    if (userId == UserHandle.USER_ALL) { userId
        = UserHandle.USER_OWNER;

        installFlags |= PackageManager.INSTALL_ALL_USERS;
    }

    //监听安装结果
    LocalPackageInstallObserver obs = new LocalPackageInstallObserver();

    //调用PKMS的installPackageAsUser方法进行安装操作
    mPm.installPackageAsUser(apkFilePath, obs.getBinder(),
        installFlags, installerPackageName, verificationParams, abi, userId);
}

```

```
synchronized (obs) {  
    while (!obs.finished) { obs.wait();  
    }  
    //当安装成功后打印Success
```

博学课道

```

        if (obs.result == PackageManager.INSTALL_SUCCEEDED)
        { System.out.println("Success");
          return 0;
        } else {
            System.err.println("Failure ["
                                + installFailureToString(obs)
                                + "]);
            return 1;
        }
    }
}

```

以上就是runInstall方法的主要内容，首先根据安装参数来设置installFlags属性值，然后创建LocalPackageInstallObserver对象来监听安装结果，最后调用PKMS对象的installPackageAsUser来执行安装操作，当然最后无论安装成功还是失败都需返回一个结果以供PC的命令行进行展示，接下来我们将进入PKMS的installPackageAsUser方法。

```

// originPath表示apk路径，observer是LocalPackageInstallObserver对象，用于监听apk安装结果，installFlags是安装参数
public void installPackageAsUser(String originPath, IPackageInstallObserver2
observer,int installFlags,...) {
    //.....
    //将操作通过发送Handler来处理
    final Message msg = mHandler.obtainMessage(INIT_COPY);
    //注意这里创建了InstallParams对象并将其传递给msg.obj对象，后面我们会详细分析这个对象
    msg.obj = new InstallParams(origin, observer,
installFlags,installerPackageName, verificationParams, user,
packageAbioverride);
    mHandler.sendMessage(msg);
}

```

可以看到installPackageAsUser方法还是蛮简单的，只是创建一个Message对象，然后通过Handler来发送INIT_COPY的消息，不过这里大家要注意参数的传递，我们进入Handler的处理消息的代码。

```

public void handleMessage(Message msg) {

    switch (msg.what)
    { case INIT_COPY:
      {
          // (1) 获得传递过来的params对象，其实际值是InstallParams
          HandlerParams params = (HandlerParams) msg.obj;
          //mPendingInstalls用于存储待安装应用，idx表示当前待安装个数
          int idx = mPendingInstalls.size();
          if (!mBound) {
              // (2) 通过bindService来启动另外一个服务
              if (!connectToService()) {
                  params.serviceError();
                  return;
              } else {
                  //如果另外一个服务已启动，将其添加到mPendingInstalls中
                  mPendingInstalls.add(idx, params);
              }
          } else {

```

```

        mPendingInstalls.add(idx, params);
        if (idx == 0) {
            // (3) 表示要启动安装
            mHandler.sendEmptyMessage(MCS_BOUND);
        }
    }
    break;
}
}
}
}

```

重点分析第三步来处理MCS_BOUND消息

```

public void handleMessage(Message msg) {

    switch (msg.what)
    { case MCS_BOUND:
      {

          if (msg.obj != null) {
              mContainerService = (IMediaContainerService) msg.obj;
          }
          //如果Service没有启动则不能安装程序
          if (mContainerService == null) {
              for (HandlerParams params : mPendingInstalls)
              { params.serviceError();
              }
              mPendingInstalls.clear();
          }else if (mPendingInstalls.size() > 0)
          { HandlerParams params =
            mPendingInstalls.get(0); if (params != null) {

                // 同学们注意：调用params对象的startCopy方法，该方法有基类
                //HandlerParams定义
                if (params.startCopy()) {
                    if (mPendingInstalls.size() > 0) {
                        //删除队列头
                        mPendingInstalls.remove(0);
                    }
                    if (mPendingInstalls.size() == 0)
                    { if (mBound) {

                        //如果安装请求完成了，在通过调用unbindService方法来解绑服
                        //务

                        removeMessages(MCS_UNBIND);
                        Message ubmsg = obtainMessage(MCS_UNBIND);
                        sendMessageDelayed(ubmsg, 10000);

                    }
                } else {
                    //如果还有待安装的事件，将继续发送MCS_BOUND消息来完成安装
                    mHandler.sendEmptyMessage(MCS_BOUND);
                }
            }
        }
    }
    break;
}
}
}
}

```

进入params.startCopy()方法进行分析，看其具体是如何进行安装的：

```
final boolean startCopy()
{
    boolean res;
    try {
        //MAX_RETRIES的值是4，表示默认安装4次，如果还不成功就表示安装失败
        if (++mRetries > MAX_RETRIES) {
            mHandler.sendEmptyMessage(MCS_GIVE_UP);
            handleServiceError();
            return false;
        } else {
            //调用HandlerParams子类的handleStartCopy方法 【同学们注意：这是第一步，
            //会先执行这个】
            handleStartCopy();
            res = true;
        }
    } catch (RemoteException e) {
        mHandler.sendEmptyMessage(MCS_RECONNECT);
        res = false;
    }
    //调用HandlerParams子类的handleReturnCode方法，将处理结果返回 【同学们注意：这是第
    //二步，后执行这个】
    handleReturnCode();
    return res;
}
```

同学们上面的startCopy方法，发现其主要就执行了两步，先执行HandlerParams子类的handleStartCopy方法，然后在执行其handleReturnCode方法将结果返回，我们进入该对象的handleStartCopy方法。

```
public void handleStartCopy() throws RemoteException {

    //根据adb install的参数来判断安装位置
    final boolean onSd = (installFlags & PackageManager.INSTALL_EXTERNAL) != 0;
    final boolean onInt = (installFlags & PackageManager.INSTALL_INTERNAL) != 0;

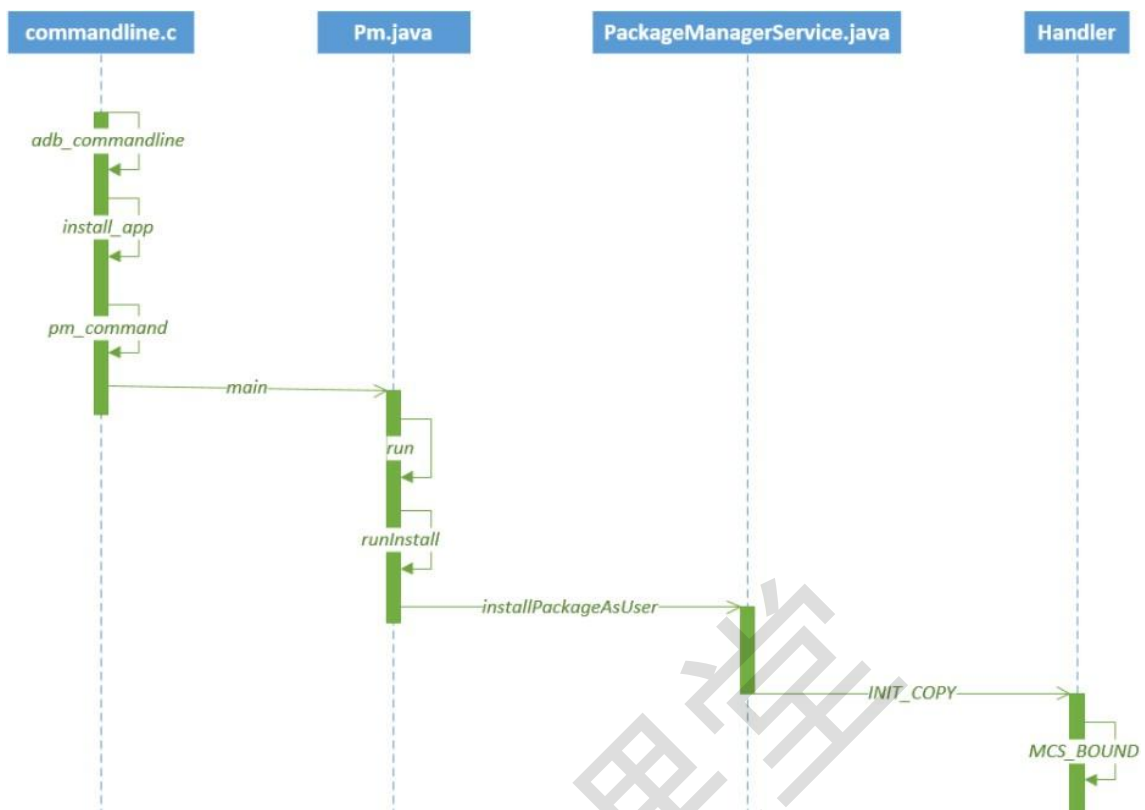
    //内部存储和SD卡不能同时安装
    if (onInt && onSd) {
        ret = PackageManager.INSTALL_FAILED_INVALID_INSTALL_LOCATION;
    } else {

        //通过StorageManager对象并查询内部存储空间最小余量
        final StorageManager storage = StorageManager.from(mContext);
        final long lowThreshold = storage.getStorageLowBytes(
            Environment.getDataDirectory());

        //.....
        //创建安装参数对象
        final InstallArgs args = createInstallArgs(this);
        //.....
        //调用InstallArgs对象的copyApk方法完成apk的拷贝
        ret = args.copyApk(mContainerService, true);
    }
}
```



```
}
```



handleReturnCode() 函数:

```
@Override
void handleReturnCode() {
    // If mArgs is null, then MCS couldn't be reached. When it
    // reconnects, it will try again to install. At that point, this
    // will succeed.
    if (mArgs != null)
    { processPendingInstall(mArgs,
        mRet);
    }
}
```

processPendingInstall() 函数:

```
private void processPendingInstall(final InstallArgs args, final int
currentStatus) {
    mHandler.post(new Runnable()
    { public void run() {
        mHandler.removeCallbacks(this);
        // Result object to be returned
        PackageInstalledInfo res = new PackageInstalledInfo();
        res.returnCode = currentStatus;
    }
    });
}
```

```

        res.uid = -1;
        res.pkg = null;
        res.removedInfo = new PackageRemovedInfo();
        if (res.returnCode == PackageManager.INSTALL_SUCCEEDED)
        { args.doPreInstall(res.returnCode);
          synchronized (mInstallLock) {
            // 同学们注意看这个函数
            installPackageLI(args, res);
          }
          args.doPostInstall(res.returnCode, res.uid);
        }
        .....
    }
});
}

```

installPackageLI() 函数:

```

private void installPackageLI(InstallArgs args, PackageInstalledInfo res) {
    .....
    final int parseFlags = mDefParseFlags | PackageParser.PARSE_CHATTY
        | (forwardLocked ? PackageParser.PARSE_FORWARD_LOCK : 0)
        | (onExternal ? PackageParser.PARSE_EXTERNAL_STORAGE : 0);
    PackageParser pp = new PackageParser();
    pp.setSeparateProcesses(mSeparateProcesses); pp.setDisplayMetrics(mMetrics);

    final PackageParser.Package pkg; try {
        pkg = pp.parsePackage(tmpPackageFile, parseFlags);
    } catch (PackageParserException e) {
        res.setError("Failed parse during installPackageLI", e); return;
    }

    ..... 又省略一万行代码，代码真多

    startIntentFilterVerifications(args.user.getIdentifier(), replace, pkg); if (replace)
    {
        replacePackageLI(pkg, parseFlags, scanFlags | SCAN_REPLACING,
args.user,

        installerPackageName, volumeUuid, res);
    } else {
        // 【同学们注意：此函数内部会调用到 scanPackageLI()】
        installNewPackageLI(pkg, parseFlags, scanFlags |
SCAN_DELETE_DATA_ON_FAILURES,

        args.user, installerPackageName, volumeUuid, res);
    }
}

```

```
true);    }  
synchronized (mPackages) {  
    final PackageSetting ps = mSettings.mPackages.get(pkgName); if (ps !=  
    null) {  
        res.newUsers = ps.queryInstalledUsers(sUserManager.getUserIds(),  
    }  
}
```

博学课道

```

    }
}

```

installNewPackageLI函数 调用到---> scanPackageLI() 函数:

```

private void installNewPackageLI(PackageParser.Package pkg, int parseFlags, int
scanFlags,
    UserHandle user, String installerPackageName, String volumeUuid,
    PackageInstalledInfo res) {
    // Remember this for later, in case we need to rollback this install
    String pkgName = pkg.packageName;

    .....

    try {
        // 【同学们注意：会执行 scanPackageLI 函数】
        PackageParser.Package newPackage = scanPackageLI(pkg, parseFlags,
scanFlags,
            System.currentTimeMillis(), user);

        updateSettingsLI(newPackage, installerPackageName, volumeUuid, null,
null, res, user);
        // delete the partially installed application. the data directory
will have to be
        // restored if it was already existing
        if (res.returnCode != PackageManager.INSTALL_SUCCEEDED) {
            // remove package from internal structures. Note that we want
deletePackageX to
            // delete the package data and cache directories that it created
in
            // scanPackageLocked, unless those directories existed before we
even tried to
            // install.
            deletePackageLI(pkgName, UserHandle.ALL, false, null, null,
                dataDirExists ? PackageManager.DELETE_KEEP_DATA : 0,
                res.removedInfo, true);
        }

    } catch (PackageManagerException e) {
        res.setError("Package couldn't be installed in " + pkg.codePath, e);
    }
}

```

scanPackageLI() 函数:

```

private PackageParser.Package scanPackageLI(File scanFile, int parseFlags, int
scanFlags,
    long currentTime, UserHandle user) throws PackageManagerException
{ if (DEBUG_INSTALL) Slog.d(TAG, "Parsing: " + scanFile);
    parseFlags |= mDefParseFlags;
    PackageParser pp = new PackageParser();
    pp.setSeparateProcesses(mSeparateProcesses);
    pp.setOnlyCoreApps(mOnlyCore);
}

```

```

pp.setDisplayMetrics(mMetrics);

if ((scanFlags & SCAN_TRUSTED_OVERLAY) != 0) { parseFlags |=
    PackageParser.PARSE_TRUSTED_OVERLAY;
}

final PackageParser.Package pkg; try {
    pkg = pp.parsePackage(scanFile, parseFlags);
} catch (PackageParserException e) {
    throw PackageManagerException.from(e);
}

PackageSetting ps = null;
PackageSetting updatedPkg;

// reader
synchronized (mPackages) {
    // Look to see if we already know about this package.
    String oldName = mSettings.mRenamedPackages.get(pkg.packageName); if
    (pkg.mOriginalPackages != null &&
pkg.mOriginalPackages.contains(oldName)) {
        // This package has been renamed to its original name.      Let's
        // use that.
        ps = mSettings.peekPackageLPr(oldName);
    }
    // If there was no original package, see one for the real package
name.

    if (ps == null) {
        ps = mSettings.peekPackageLPr(pkg.packageName);
    }

    // Check to see if this package could be hiding/updating a system
    // package.    Must look for it either under the original or real
    // package name depending on our state.
    updatedPkg = mSettings.getDisabledSystemPkgLPr(ps != null ? ps.name
: pkg.packageName);
    if (DEBUG_INSTALL && updatedPkg != null) Slog.d(TAG, "updatedPkg = "
+ updatedPkg);
}

boolean updatedPkgBetter = false;
// First check if this is a system package that may involve an update if
(updatedPkg != null && (parseFlags & PackageParser.PARSE_IS_SYSTEM) !=
0) {
    // If new package is not located in "/system/priv-app" (e.g. due to
an OTA),

```

```
/      it needs to drop FLAG_PRIVILEGED. if
/      (locationIsPrivileged(scanFile)) {

        updatedPkg.pkgPrivateFlags |=
ApplicationInfo.PRIVATE_FLAG_PRIVILEGED;
    } else {
        updatedPkg.pkgPrivateFlags &=
~ApplicationInfo.PRIVATE_FLAG_PRIVILEGED;
    }

    if (ps != null && !ps.codePath.equals(scanFile)) {
        // The path has changed from what was last scanned...      check the
        // version of the new path against what we have stored to

determine        // what to do.
```

```

        if (DEBUG_INSTALL) Slog.d(TAG, "Path changing from " +
ps.codePath);

        if (pkg.mVersionCode <= ps.versionCode) {
            // The system package has been updated and the codepath
does not match

            // Ignore entry. Skip it.
            if (DEBUG_INSTALL) Slog.i(TAG, "Package " + ps.name + " at "

+ scanFile

            + " ignored: updated version " + ps.versionCode
            + " better than this " + pkg.mVersionCode); if
(!updatedPkg.codePath.equals(scanFile)) {

                Slog.w(PackageManagerService.TAG, "Code path for hidden

system pkg : "
                                + ps.name + " changing from " +
updatedPkg.codePathString
                                + " to " + scanFile);
                updatedPkg.codePath = scanFile; updatedPkg.codePathString =
scanFile.toString(); updatedPkg.resourcePath = scanFile;
                updatedPkg.resourcePathString = scanFile.toString();
            }
            updatedPkg.pkg = pkg; throw
new
PackageManagerException(INSTALL_FAILED_DUPLICATE_PACKAGE,
                                "Package " + ps.name + " at " + scanFile
                                + " ignored: updated version " +
ps.versionCode

                                + " better than this " + pkg.mVersionCode);
        } else {
            // The current app on the system partition is better than
            // what we have updated to on the data partition; switch
            // back to the system partition version.
            // At this point, its safely assumed that package

installation for // apps in system partition will go through. If not there

won't be a working // version of the app
// writer
synchronized (mPackages) {
    // Just remove the loaded entries from package lists.
    mPackages.remove(ps.name);
}

```

```
logCriticalInfo(Log.WARN, "Package " + ps.name + " at " +  
  
scanFile  
  
    + " reverting from " + ps.codePathString  
    + ": new version " + pkg.mVersionCode  
    + " better than installed " + ps.versionCode);  
  
InstallArgs args =  
createInstallArgsForExisting(packageFlagsToInstallFlags(ps),  
  
    ps.codePathString, ps.resourcePathString,  
getAppDexInstructionSets(ps));  
  
    synchronized (mInstallLock)  
    { args.cleanupResourcesLI();  
    }  
    synchronized (mPackages) { mSettings.enableSystemPackageLPw(ps.name);
```



```

    }
    updatedPkgBetter = true;
}
}
}

if (updatedPkg != null) {
    // An updated system app will not have the PARSE_IS_SYSTEM flag set
    // initially
    parseFlags |= PackageParser.PARSE_IS_SYSTEM;

    // An updated privileged app will not have the PARSE_IS_PRIVILEGED
    // flag set initially
    if ((updatedPkg.pkgPrivateFlags &
ApplicationInfo.PRIVATE_FLAG_PRIVILEGED) != 0) {
        parseFlags |= PackageParser.PARSE_IS_PRIVILEGED;
    }
}

// 同学们 这里是：签名校验
// Verify certificates against what was last scanned
collectCertificatesLI(pp, ps, pkg, scanFile, parseFlags);

/*
76A new system app appeared, but we already had a non-system one of the
77same name installed earlier.
*/
boolean shouldHideSystemApp = false; if
(updatedPkg == null && ps !=null
    && (parseFlags & PackageParser.PARSE_IS_SYSTEM_DIR) != 0 &&
!isSystemApp(ps)) {
    /*
    • Check to make sure the signatures match first. If they don't,
    • wipe the installed application and its data.
    */
    if (compareSignatures(ps.signatures.mSignatures, pkg.mSignatures)
        != PackageManager.SIGNATURE_MATCH) { logCriticalInfo(Log.WARN,
        "Package " + ps.name + " appeared on
system, but"
        + " signatures don't match existing userdata copy;
removing");
        deletePackageLI(pkg.packageName, null, true, null, null, 0,
null, false);
        ps = null;
    } else {

```

```
/*
    • If the newly-added system app is an older version than the
    • already installed version, hide it. It will be scanned later
    • and re-added like an update.
*/
if (pkg.mVersionCode <= ps.versionCode)
    { shouldHideSystemApp = true;

    logCriticalInfo(Log.INFO, "Package " + ps.name + " appeared
at " + scanFile

    + " but new version " + pkg.mVersionCode + " better
than installed "

    + ps.versionCode + "; hiding system");
    } else {
```

```

/*
80The newly found system app is a newer version than the
81one previously installed. Simply remove the
82already-installed application and replace it with our own
83while keeping the application data.
*/
logCriticalInfo(Log.WARN, "Package " + ps.name + " at " +

    + " reverting from " + ps.codePathString + ": new

scanFile

    + pkg.mVersionCode + " better than installed " +

version "

    InstallArgs args =

ps.versionCode);
createInstallArgsForExisting(packageFlagsToInstallFlags(ps),
    ps.codePathString, ps.resourcePathString,
getAppDexInstructionSets(ps));

    synchronized (mInstallLock)
    { args.cleanUpResourcesLI();
    }
    }
}

// The apk is forward locked (not public) if its code and resources
// are kept in different files. (except for app in either system or
// vendor path).
// TODO grab this value from PackageSettings
if ((parseFlags & PackageParser.PARSE_IS_SYSTEM_DIR) == 0) { if (ps !=
    null && !ps.codePath.equals(ps.resourcePath)) {

        parseFlags |= PackageParser.PARSE_FORWARD_LOCK;

    }
}

// TODO: extend to support forward-locked splits String
resourcePath = null;

String baseResourcePath = null;
if ((parseFlags & PackageParser.PARSE_FORWARD_LOCK) != 0 &&
!updatedPkgBetter) {
    if (ps != null && ps.resourcePathString != null)
    { resourcePath = ps.resourcePathString;
      baseResourcePath = ps.resourcePathString;
    } else {

```

```
        // Should not happen at all. Just log an error.
        Slog.e(TAG, "Resource path not set for pkg : " +
pkg.packageName);
    }
    } else {
        resourcePath = pkg.codePath;
        baseResourcePath =pkg.baseCodePath;
    }

    // Set application objects path explicitly. pkg.applicationInfo.volumeUuid =
pkg.volumeUuid; pkg.applicationInfo.setCodePath(pkg.codePath);
pkg.applicationInfo.setBaseCodePath(pkg.baseCodePath);
pkg.applicationInfo.setSplitCodePaths(pkg.splitCodePaths);
pkg.applicationInfo.setResourcePath(resourcePath);
pkg.applicationInfo.setBaseResourcePath(baseResourcePath);
```

```

        pkg.applicationInfo.setSplitResourcePaths(pkg.splitCodePaths);

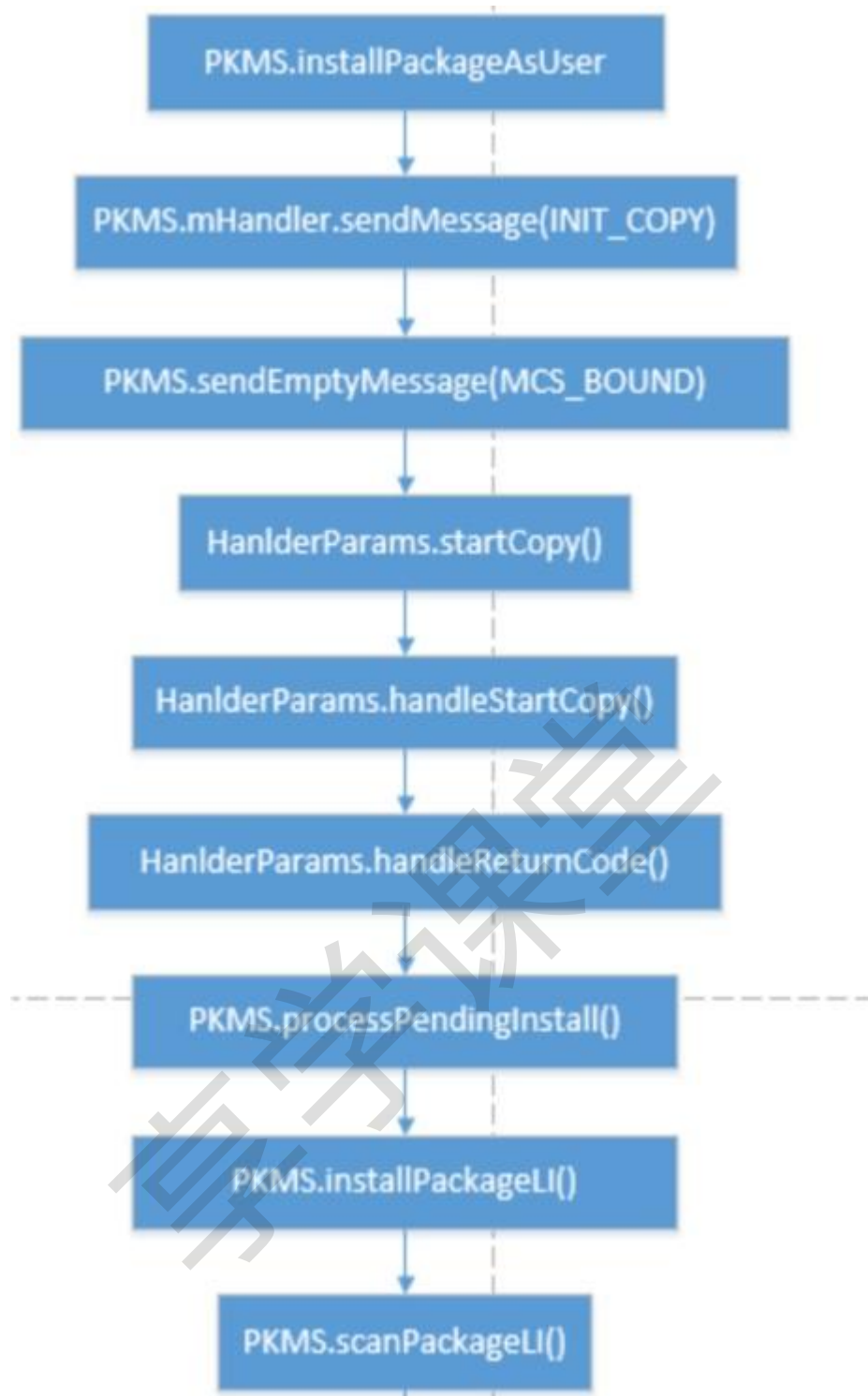
        // Note that we invoke the following method only if we are about to
        unpack an application
        PackageParser.Package scannedPkg = scanPackageLI(pkg, parseFlags,
        scanFlags
            | SCAN_UPDATE_SIGNATURE, currentTime, user);

        /*
        83If the system app should be overridden by a previously installed
        84data, hide the system app now and let the /data/app scan pick it up
        85again.
        */
        if (shouldHideSystemApp)
        { synchronized (mPackages)
        {
            /*
            • We have to grant systems permissions before we hide, because
            • grantPermissions will assume the package update is trying to
            • expand its permissions.
            */
            grantPermissionsLPw(pkg, true, pkg.packageName);
            mSettings.disableSystemPackageLPw(pkg.packageName);
        }
        }

        return scannedPkg;
    }
}

```

安装流程:



在PKMS构造函数中解析所需要的文件目录：

```
File dataDir = Environment.getDataDirectory();
mAppDataDir = new File(dataDir, "data");
mAppInstallDir = new File(dataDir, "app");
mAppLib32InstallDir = new File(dataDir, "app-lib");
mAsecInternalPath = new File(dataDir, "app-asec").getPath();
mUserAppDataDir = new File(dataDir, "user");
mDrmAppPrivateInstallDir = new File(dataDir, "app-private");
```

data	drwxrwx--x	2018-10-23 08:19
adb	drwx-----	2018-10-02 11:03
anr	drwxrwxr-x	2019-04-24 00:43
app	drwxrwx--x	2019-05-07 08:22
app-asec	drwx-----	2018-10-02 11:03
app-lib	drwxrwx--x	2018-10-02 11:03
app-private	drwxrwx--x	2018-10-02 11:03
backup	drwx-----	2019-05-07 08:22
dalvik-cache	drwxrwx--x	2018-10-02 11:03
data	drwxrwx--x	2019-04-29 10:05
dontpanic	drwxr-x---	2018-10-02 11:03
drm	drwxrwx---	2018-10-02 11:03
local	drwxr-x---	2018-10-02 11:03
lost+found	drwxrwx---	1969-12-31 19:00
media	drwxrwx---	2018-10-02 11:04
mediadr	drwxrwx---	2018-10-02 11:03
misc	drwxrwx--t	2018-10-02 11:03
nativetest	drwxrwx--x	2018-05-21 20:04
property	drwx-----	2019-05-07 06:12
resource-cache	drwxrwx--x	2018-10-02 11:03
security	drwx--x--x	2018-10-02 11:03
system	drwxrwxr-x	2019-05-07 08:47
tombstones	drwx-----	2018-12-07 21:14

总结：安装的原理：

安装其实就是把apk文件copy到了对应的目录：

1. data/app/包名 —— 安装时把 apk文件复制到此目录, —— 可以将文件取出并安装, 和我们本身的apk 是一样的。

→ /data/data/packageName/(test.apk)

2. data/data/包名 —— 开辟存放应用程序的文件数据的文件夹 包括我们应用的 so库, 缓存文件 等等。

→ /data/data/packageName/(db, cache)

3. 将apk中的dex文件安装到data/dalvik-cache目录下 (dex文件是 dealvik虚拟机的可执行文件, 其大小约为原始apk文件大小的四分之一)

→ /data/dalvik-cache/(profiles, x86)

附：本内容由享学课堂移动开发技术团队独家提供 <https://ke.qq.com/course/341933?tuin=9f978cfe&taid=9303882710988717>, 未经享学课堂授权不得转载、摘编或利用其它方式使用上述作品。已经享学课堂授权使用作品的, 应在授权范围内使用, 并注明“来源: 享学课堂”。违反上述声明者, 将追究其相关法律责任。

致谢：

感谢为本书提供素材的各位幕后英雄, 感谢罗升阳, 感谢享学课堂团队的辛勤整理, 谢谢你们! 并对所引用的书籍表示感谢!